

Comprehensive Documentation of QKD Simulation Framework

Reza Aramjou

۲۶ شهریور ۱۴۰۵

پیشگفتار

پیدایش یک فریم‌ورک امن کوانتومی

تحول از ایده‌های نظری در توزیع کلید کوانتومی (Quantum Key Distribution, QKD) به سامانه‌هایی که بتوان آن‌ها را در سطح پیاده‌سازی، اعتبارسنجی، بهینه‌سازی و نهایتاً استقرار شبکه‌ای بررسی کرد، مستلزم عبور از یک شکاف بنیادی میان دو جهان متفاوت است: از یک سو، جهان فیزیک کوانتومی و نظریه اطلاعات که با صورت‌بندی‌های انتزاعی، کران‌های امنیتی، فروض ایده‌آل و مدل‌های تحلیلی سروکار دارد؛ و از سوی دیگر، جهان مهندسی سامانه و نرم‌افزار که در آن، جزئیات اجرایی، قیود محاسباتی، ناپایداری‌های سخت‌افزاری، ساختار داده، همزمانی، تکرارپذیری و اعتبارسنجی، نقش تعیین‌کننده در قابل‌اعتماد بودن یک نتیجه ایفا می‌کنند. در عمل، بخش عمده‌ای از دشواری QKD نه در فهم اصول اولیه آن، بلکه در پر کردن دقیق، روشمند و قابل دفاع همین فاصله نهفته است.

مستندی که پیش رو قرار دارد، حاصل تلاشی برای پاسخ دادن به این نیاز است: طراحی، توسعه، سازمان‌دهی و اعتبارسنجی یک **فریم‌ورک شبیه‌سازی QKD** که بتواند به‌عنوان یک بستر پژوهشی جامع، دقیق، توسعه‌پذیر و مهندسی‌شده، رفتار سامانه‌های Discrete-Variable QKD را از سطح تولید پالس نوری تا سطح پردازش امنیتی و حتی مسیریابی شبکه‌ای مدل‌سازی کند. این فریم‌ورک صرفاً یک ابزار محاسبه‌گر نرخ کلید نیست، بلکه تلاشی است برای بازسازی یک خط لوله کامل از رخدادهای فیزیکی، آماری، الگوریتمی و معماری که در یک سامانه واقعی QKD حضور دارند. در این معنا، پروژه حاضر نه تنها به مدل‌سازی یک پروتکل، بلکه به ساخت یک زیرساخت تحقیقاتی برای مطالعه خانواده‌ای از سناریوهای QKD می‌پردازد.

انگیزه اصلی شکل‌گیری این فریم‌ورک از یک مشاهده نسبتاً روشن اما عمیق آغاز شد: بسیاری از ابزارها و مدل‌های موجود، اگرچه برای محاسبه تقریبی نرخ کلید، بازتولید چند نمودار مرجع، یا اجرای سناریوهای آموزشی مفیدند، اما برای پژوهش‌های جدی‌تر با محدودیت‌های بنیادی مواجه می‌شوند. بخشی از این محدودیت‌ها از ساده‌سازی بیش از حد لایه فیزیکی ناشی می‌شود؛ برای مثال، مدل‌سازی ایده‌آل یا ناقص آشکارسازها، نادیده گرفتن آبشارهای زمان مرده، صرف‌نظر کردن از afterpulsing، یا فروکاستن نویز به یک ثابت ساده. بخش دیگر از محدودیت‌ها به لایه امنیتی بازمی‌گردد؛ جایی که برخی ابزارها تنها یک اثبات امنیتی خاص را به صورت سخت‌کدگذاری شده پیاده‌سازی می‌کنند و در نتیجه، امکان مقایسه دقیق و منصفانه چند اثبات بر روی یک موتور فیزیکی واحد را از بین می‌برند. افزون بر این، در بسیاری از شبیه‌سازهای موجود، تحلیل‌ها در رژیم مجانبی انجام می‌شوند؛ در حالی که در سامانه‌های واقعی، کلید محدود، نوسانات آماری و بودجه امنیتی، بخش جدایی‌ناپذیر مسئله هستند.

فریم‌ورک حاضر برای رفع این مصالحه‌ها توسعه یافت. هدف آن ایجاد محیطی یکپارچه بود که در آن، پژوهشگر بتواند بدون قربانی کردن دقت فیزیکی، انعطاف‌پذیری رمزنگاری، شفافیت معماری و قابلیت توسعه، سناریوهای متنوع QKD را مطالعه کند. از همین رو، طراحی این سامانه از ابتدا بر اساس چند اصل هدایت‌کننده شکل گرفت: تفکیک روشن لایه‌ها، ماژولار بودن، قابلیت اعتبارسنجی تحلیلی، پشتیبانی از سناریوهای Finite-key، توانایی مقایسه چند اثبات امنیتی، و امکان توسعه به سطح شبکه‌های چندگره‌ای. این اصول نه صرفاً ترجیحات مهندسی، بلکه پاسخ‌هایی ضروری به ماهیت میان‌رشته‌ای و حساس QKD بودند.

این پروژه در عین آنکه از نظر مفهومی بر مبانی استاندارد QKD تکیه دارد، به صورت خاص برای پاسخ به نیازهای پژوهش تجربی و شبیه‌سازی دقیق طراحی شده است. به همین دلیل، تمرکز آن تنها بر «درست بودن» فرمول‌ها نیست، بلکه بر «درست بودن رفتار سامانه در تعامل اجزا» نیز هست. در عمل، یک اثبات امنیتی دقیق نیز اگر بر داده‌هایی سوار شود که به دلیل مدل‌سازی ناقص آشکارساز یا مدیریت نادرست رویدادهای همزمان مخدوش شده‌اند، خروجی معناداری نخواهد داشت. به همین ترتیب، یک موتور فیزیکی دقیق نیز اگر نتواند داده‌های خود را به درستی در اختیار لایه پس‌پردازش قرار دهد، ارزش پژوهشی محدودی خواهد داشت. از این رو، این فریم‌ورک بر پیوستگی دقیق و مهندسی‌شده کل زنجیره تأکید دارد.

به بیان دیگر، این فریم‌ورک تلاشی است برای آنکه شبیه‌سازی QKD از سطح یک اسکریپت محاسباتی مجزا

فراتر رود و به یک محیط پژوهشی قابل اتکا تبدیل شود؛ محیطی که بتواند برای طراحی آزمایش، مقایسه اثبات‌ها، تحلیل حساسیت، بهینه‌سازی پارامتر، بررسی سناریوهای چندگرمی و تولید نتایج بازتولیدپذیر به کار رود. این مستند نیز با همین نگاه تدوین شده است: نه فقط برای توضیح آنچه پیاده‌سازی شده، بلکه برای روشن‌سازی منطق علمی، معماری نرم‌افزاری و مسیر اعتبارسنجی‌ای که این فریم‌ورک بر آن بنا شده است.

فلسفه معماری

در طراحی این فریم‌ورک، معماری نرم‌افزار تنها یک مسئله سازمان‌دهی کد نبوده، بلکه بخشی از منطق علمی پروژه به‌شمار آمده است. هنگامی که یک سامانه قرار است به‌طور همزمان فیزیک منبع، تلفات کانال، دینامیک آشکارساز، آماداده‌های کلید محدود، اثبات‌های امنیتی متنوع، بهینه‌سازی عددی و حتی رفتار شبکه‌ای را در خود جای دهد، هرگونه آمیختگی نامنظم میان این اجزا به‌سرعت به پیچیدگی کنترل‌ناپذیر، دشواری در اعتبارسنجی و کاهش قابلیت اطمینان منجر خواهد شد. به همین دلیل، فلسفه معماری این پروژه بر پایه **تفکیک مسئولیت‌ها، ماژولار بودن، صراحت در قراردادهای داده، و استقلال نسبی لایه‌ها** شکل گرفته است.

در هسته این طراحی، تمایز روشنی میان دو ناحیه اصلی برقرار شده است: نخست، **لایه فیزیکی** که مسئول مدل‌سازی آن چیزی است که در سطح منبع، کانال و آشکارساز رخ می‌دهد؛ و دوم، **لایه پس‌پردازش و امنیت** که وظیفه تفسیر داده‌های مشاهده‌شده، غربال‌گری پایه، تخمین پارامتر، اعمال اثبات‌های امنیتی و محاسبه نرخ کلید امن را بر عهده دارد. این جداسازی دو مزیت مهم ایجاد می‌کند. از یک سو، امکان آن را فراهم می‌سازد که مدل‌های فیزیکی بدون دست‌کاری مستقیم در لایه اثبات‌ها توسعه یابند؛ و از سوی دیگر، به پژوهشگر اجازه می‌دهد چندین اثبات امنیتی را بر روی دقیقاً یک داده‌مولد فیزیکی مشترک اجرا کند. به این ترتیب، مقایسه‌ها نه تحت تأثیر تفاوت پیاده‌سازی‌های پراکنده، بلکه بر پایه یک بستر همگن و کنترل‌شده انجام می‌شوند.

یکی از عناصر شاخص این معماری، **موتور توزیع چندکارگری (Multi-Worker Dispatch Engine)** است. در شبیه‌سازی‌های رویدادمحور QKD، مقیاس مسئله به‌سرعت بزرگ می‌شود. زمانی که تحلیل در رژیم Finite-key، روی تعداد قابل توجهی پالس، با مدل‌های واقع‌گرایانه و مجموعه‌ای از پارامترها انجام می‌شود، هزینه محاسباتی اجرای ترتیبی به‌شدت افزایش می‌یابد. در چنین شرایطی، استفاده از پردازش موازی دیگر یک بهینه‌سازی لوکس نیست، بلکه یک الزام عملی است. با این حال، موازی‌سازی در این حوزه با چالش‌هایی همراه است که در بسیاری از سامانه‌های ساده نادیده گرفته می‌شوند. مهم‌ترین این چالش‌ها، **حفظ یکپارچگی هویت پالس‌ها و سازگاری رویدادها** در محیطی است که اقلام کاری روی کارگرهای مختلف، با زمان‌بندی‌های متفاوت و احتمالاً با تأخیرهای ناهمگون پردازش می‌شوند.

برای پاسخ به این مسئله، معماری فریم‌ورک از مکانیزمی مبتنی بر work itemهای مستقل و خودبسنده بهره می‌برد. هر قلم کاری، اطلاعات کافی درباره پالس، پایه، شدت، حالت و زمینه آماری خود را به همراه دارد و به‌گونه‌ای تعریف می‌شود که وابستگی پنهان یا شکننده به وضعیت اجرایی دیگر بخش‌ها نداشته باشد. این تصمیم معماری باعث می‌شود که نتایج به ترتیب زمان‌بندی کارگرها وابسته نشوند و پردازش موازی به تغییر خاموش در معنای داده‌ها منجر نگردد. در یک سامانه رمزنگاری یا شبه‌رمزنگاری، این موضوع اهمیتی فراتر از کارایی دارد؛ زیرا هرگونه ناسازگاری خاموش در هویت پالس‌ها یا تعلق رویدادها می‌تواند مستقیماً به تفسیر نادرست آمار امنیتی بینجامد.

از منظر طراحی داده، این فریم‌ورک تلاش می‌کند تا حد ممکن از قراردادهای صریح، ساختارهای داده مشخص و مسیرهای پردازش قابل رهگیری استفاده کند. این امر صرفاً برای زیبایی کد یا سهولت توسعه نیست؛ بلکه برای آن است که یک نتیجه عددی نهایی، در صورت نیاز، قابل ردیابی به مراحل پیشین خود باشد. پژوهشگر باید بتواند بفهمد یک نرخ کلید یا یک نرخ خطا چگونه از میان لایه‌های مختلف شکل گرفته است، چه عواملی بر آن اثر گذاشته‌اند، و در صورت بروز انحراف، منشأ آن در کدام بخش قرار دارد. در اینجا، معماری خوب عملاً نقش ابزار تبیین علمی را نیز ایفا می‌کند.

فلسفه معماری پروژه همچنین بر **توسعه‌پذیری کنترل‌شده** تأکید دارد. در سامانه‌های پژوهشی، نیازها ایستا نیستند. ممکن است در نسخه‌های بعدی، مدل جدیدی از منبع نوری، یک تقریب بهتر برای afterpulsing، یک اثبات امنیتی متفاوت، یا یک سیاست تازه برای مدیریت شبکه مورد نیاز باشد. اگر افزودن هر قابلیت جدید مستلزم بازنویسی چندین ماژول و شکستن فرضیات قبلی باشد، سامانه خیلی زود به نقطه فرسودگی می‌رسد. از این رو، طراحی کلاس‌های پایه، لایه‌های انتزاعی و رابط‌های داخلی به‌گونه‌ای انجام شده است که افزودن جزء جدید، عمدتاً مستلزم پیاده‌سازی همان جزء باشد، نه بازآرایی کل سیستم.

در نهایت، فلسفه این معماری را می‌توان چنین خلاصه کرد: دقت علمی تنها زمانی ارزشمند است که در یک ساختار مهندسی‌شده، شفاف، تکرارپذیر و قابل توسعه استقرار یابد. این فریم‌ورک تلاش کرده است دقیقاً چنین بستری را فراهم کند؛ بستری که در آن، شبیه‌سازی QKD از یک محاسبه موردی و محدود فراتر رود و به

یک دارایی پژوهشی و نرم‌افزاری پایدار تبدیل شود.

فرآیند جامع اعتبارسنجی

هیچ فریم‌ورک شبیه‌سازی، صرف‌نظر از پیچیدگی ظاهری، تنوع قابلیت‌ها یا ظرافت معماری، بدون یک فرایند اعتبارسنجی دقیق و چندلایه، شایسته اعتماد علمی نخواهد بود. این گزاره در مورد سامانه‌های QKD اهمیتی دوچندان دارد؛ زیرا خطا در چنین محیطی تنها یک اختلاف عددی ساده نیست، بلکه می‌تواند به تفسیر نادرست امنیت، خوش‌بینی کاذب نسبت به نرخ کلید، یا مقایسه‌های گمراه‌کننده میان مدل‌ها و اثبات‌ها منجر شود. از همین رو، اعتبارسنجی در این پروژه نه یک مرحلهٔ پسینی و تکمیلی، بلکه بخشی مرکزی از فرایند توسعه بوده است.

رویکرد اعتبارسنجی این فریم‌ورک بر چند اصل بنا شده است. نخست آنکه اعتبارسنجی نباید به سطح تست واحد توابع محدود شود، هرچند این تست‌ها لازم‌اند. دوم آنکه هر لایه باید تا حد امکان در برابر یک مرجع مستقل سنجیده شود: لایهٔ فیزیک در برابر روابط تحلیلی و پیش‌بینی‌های فرم‌بسته، لایهٔ امنیتی در برابر خواص مورد انتظار اثبات‌ها و رفتار مرزی آن‌ها، و لایهٔ شبکه در برابر منطق ساختاری سناریوهای مسیریابی و تأمین کلید. سوم آنکه اعتبارسنجی باید هم در سطح محلی و هم در سطح برهم‌کنش میان لایه‌ها انجام شود؛ زیرا بسیاری از خطاهای مهم نه در خود اجزا، بلکه در مرزهای میان آن‌ها پدیدار می‌شوند.

در این پروژه، اعتبارسنجی با اتکا به **بنچمارک‌های تحلیلی حقیقت پایه (Ground-Truth Analytical Benchmarks)**، آزمون‌های آماری، تست‌های واحد، تست‌های یکپارچه‌سازی، و سناریوهای کامل اجرایی صورت گرفته است. هدف از این ترکیب آن بوده است که از یک سو، اجزای منفرد به‌صورت ایزوله کنترل شوند، و از سوی دیگر، عملکرد کل خط لوله نیز در سناریوهای واقع‌گرایانه سنجیده شود.

۱. یکپارچگی لایه فیزیکی

نخستین و بنیادی‌ترین لایه‌ای که باید اعتبارسنجی شود، لایهٔ فیزیکی است. در این لایه، منبع نوری، کانال انتقال و آشکارسازها مجموعه‌ای از رخدادها را تولید می‌کنند که تمام مراحل بعدی بر آن‌ها استوار است. اگر این لایه دچار انحراف مفهومی یا عددی باشد، حتی دقیق‌ترین اثبات امنیتی نیز بر ورودی نادرست سوار خواهد شد. به همین دلیل، بخش مهمی از فرایند توسعه به اطمینان از صحت این لایه اختصاص یافت.

در سطح منبع، آمار نشر پالس‌ها برای مدل‌های مورد استفاده—از جمله توزیع پواسونی و در صورت نیاز مدل‌های حرارتی—در برابر روابط تحلیلی شناخته‌شده بررسی شد. در سطح کانال، مدل تضعیف فیبر نوری با تکیه بر قانون بیر—لمبرت و پارامترهای استاندارد انتقال بررسی گردید. در سطح آشکارسازی، رفتار نویز شمارش تاریک، احتمال کلیک، زمان مردهٔ غیرفلج‌شونده (Non-paralyzable Dead Time) و آثار وابسته به آن با مدل‌های مرجع تطبیق داده شد.

برای آنکه این تطبیق تنها به مقایسهٔ بصری یا تقریبی محدود نماند، از چارچوب‌های آماری صریح استفاده شد. به‌طور خاص، با به‌کارگیری یک آزمون آماری Z دوجمله‌ای و اتخاذ سطح معنی‌داری سخت‌گیرانه $\alpha = 10^{-4}$ ، بررسی شد که کمیت‌هایی مانند بهرهٔ کل یا Q_{μ} ، و نرخ خطای بیت کوانتومی (QBER)، در فواصل و سناریوهای مختلف، در تلورانس تعریف‌شده با پیش‌بینی‌های نظری هم‌خوان باشند. نتیجهٔ این فرایند نشان داد که موتور فیزیکی، در محدوده‌های عملی مورد مطالعه، رفتار مورد انتظار را با دقت قابل قبول بازتولید می‌کند. این موضوع اهمیت زیادی دارد، زیرا نشان می‌دهد انحرافات مشاهده‌شده در مراحل بعد، در صورت وقوع، احتمالاً ناشی از بخش‌های دیگر زنجیره‌اند، نه از لایهٔ پایهٔ فیزیکی.

۲. سازگارسازی چنداثباتی رمزنگاری

یکی از چالش‌های مهم در توسعهٔ فریم‌ورک‌های QKD آن است که بسیاری از پیاده‌سازی‌ها از همان ابتدا حول یک اثبات امنیتی خاص طراحی می‌شوند. در چنین وضعیتی، مسیر داده، ساختار آماری، توابع کمکی و حتی نام‌گذاری متغیرها به‌شکلی ناخواسته به منطق آن اثبات خاص وابسته می‌شود. نتیجهٔ آن است که افزودن اثباتی دیگر، به‌جای آنکه صرفاً توسعهٔ یک ماژول جدید باشد، به بازنویسی بخش‌هایی از کل خط لوله نیاز پیدا می‌کند. این وابستگی پنهان، یکی از موانع اصلی برای مقایسهٔ دقیق اثبات‌های امنیتی در محیط‌های فیزیکی مشترک است. در نسخه‌های اولیه‌تر این پروژه نیز خط لولهٔ اجرایی تا حدی به اثبات کلید محدود Lim2014 گره خورده بود. یکی از مهم‌ترین نقاط عطف در بلوغ معماری، معرفی یک **لایهٔ سازگارساز (Adapter Layer)** در اسکرپیت اجرایی اصلی بود. این لایه نقش مترجم ساختاری را ایفا می‌کند: داده‌ها و آمارهای خام یا نیمه‌پردازش‌شدهٔ حاصل از شبیه‌سازی را دریافت می‌کند و آن‌ها را به امضاها، ساختارها و قراردادهای مورد انتظار هر اثبات امنیتی

تبدیل می‌نماید. به این ترتیب، منطق فیزیکی و منطق اثباتی از یکدیگر جدا می‌شوند، در حالی که همچنان از یک زبان داده مشترک بهره می‌برند.

بر پایه این بازآرایی معماری، فریم‌ورک توانست چند خانواده اثباتی را با موفقیت پشتیبانی و اجرا کند:

- **Lim ۲۰۱۴**: به عنوان یکی از چارچوب‌های مهم Finite-key، این اثبات توانست در مقیاس‌هایی نظیر $N = 10^7$ ، نرخ‌های کلید محدود بهینه و معناداری تولید کند و به عنوان مرجع اصلی در بسیاری از مقایسه‌ها به کار رود.

- **Wang ۲۰۰۵**: این اثبات نرخ‌های کلیدی متمایزتر و در مواردی محافظه‌کارانه‌تر تولید کرد و نشان داد که تفاوت در روش کران‌بندی آماری و مفروضات تحلیلی چگونه مستقیماً بر خروجی نهایی اثر می‌گذارد.

- **Ma ۲۰۰۵**: اجرای این اثبات، به‌ویژه در نواحی‌ای که مفروضات گاوسی یا کفایت آماری به خوبی برقرار نیستند، به درستی رفتار محافظه‌کارانه نشان داد و در برخی رژیم‌ها منجر به نرخ کلید صفر شد. همین رفتار، خود نشانه‌ای از صحت سازگارسازی است؛ زیرا سامانه در این حالت تلاش نکرده است به صورت مصنوعی نرخ مثبت تولید کند.

- **Tight Proof**: این شاخه با بهره‌گیری از ساختار Lim ۲۰۱۴ و در عین حال اعمال تخمین‌های بهینه‌تر برای برخی کمیت‌ها، رفتار متفاوتی را در تحلیل امنیتی ارائه داد و نشان داد که فریم‌ورک آماده مقایسه نسخه‌های سخت‌گیرانه‌تر یا دقیق‌تر از اثبات‌های مرجع است.

اهمیت این دستاورد تنها در «اجرا شدن» چند اثبات خلاصه نمی‌شود. ارزش اصلی آن در این است که پژوهشگر اکنون می‌تواند دقیقاً یک سناریوی فیزیکی واحد را تحت چند لنز امنیتی مختلف مشاهده و تحلیل کند. این قابلیت، برای بررسی اثر انتخاب اثبات بر نرخ کلید، مقایسه محافظه‌کاری یا تهاجمی بودن کران‌ها، و مطالعه حساسیت خروجی به فروض آماری، اهمیت پژوهشی بالایی دارد.

۳. بهینه‌سازی پارامترها با برنامه‌ریزی خطی (LP)

سامانه‌های QKD ذاتاً به پارامترهای عملیاتی خود حساس‌اند. شدت سیگنال (μ)، شدت‌های فریب (ν)، احتمال انتخاب هر شدت، نسبت پایه‌ها، و گاه حتی تنظیمات اجرایی آشکارساز یا پنجره‌های گیتینگ، همگی می‌توانند به طور مستقیم و غیرمستقیم بر نرخ کلید امن، QBER، بازده و برد نهایی سامانه اثر بگذارند. در بسیاری از مطالعات ساده، این پارامترها یا به صورت دستی تنظیم می‌شوند یا از مقادیر مرجع در مقاله‌های پیشین اقتباس می‌گردند. با این حال، چنین رویکردی معمولاً برای دستیابی به بهترین عملکرد در یک سناریوی مشخص کافی نیست.

در این فریم‌ورک، یک لایه بهینه‌سازی برای جست‌وجوی نظام‌مند در فضای پارامترها توسعه یافته است. این لایه با استفاده از حلگرهای عددی مانند Nelder-Mead و همچنین ابزارهای مبتنی بر برنامه‌ریزی خطی (LP)، می‌تواند فضای شدت‌ها و احتمال‌ها را پیمایش کرده و پیکربندی‌هایی را بیابد که نرخ کلید امن را در شرایط معین بیشینه می‌کنند. اعتبارسنجی این بخش صرفاً به بررسی همگرایی عددی محدود نشد، بلکه خروجی‌های بهینه‌سازی با سناریوهای مرجع و مقادیر غیربهینه مقایسه شدند تا اثر واقعی بهینه‌سازی بر عملکرد نهایی سنجیده شود.

نتایج این بررسی‌ها نشان دادند که حلگر LP و لایه بهینه‌سازی نه تنها به درستی در فضای پارامتر حرکت می‌کنند، بلکه در بسیاری از فواصل عملی—به‌ویژه در بازه ۰ تا ۱۰۰ کیلومتر—توانسته‌اند پیکربندی‌هایی بیابند که به افزایش بسیار معنادار در نرخ کلید امن نسبت به مقادیر ثابت و سخت‌کدگذاری شده منجر می‌شوند. این نتیجه از منظر مهندسی اهمیت زیادی دارد، زیرا نشان می‌دهد بخشی از شکاف عملکردی میان یک سامانه «قابل اجرا» و یک سامانه «خوب تنظیم‌شده» را می‌توان به صورت محاسباتی و سیستماتیک کاهش داد. از منظر پژوهشی نیز، وجود این لایه باعث می‌شود که مقایسه اثبات‌ها یا مدل‌ها بر اساس پارامترهای منصفانه‌تر و نزدیک‌تر به بهینه انجام شود، نه صرفاً بر اساس انتخاب‌های دستی.

۴. توپولوژی شبکه و رله‌های قابل اعتماد

بخش بزرگی از ادبیات کلاسیک QKD بر لینک‌های نقطه‌به‌نقطه متمرکز است؛ حال آنکه در کاربردهای عملی، به‌ویژه زمانی که هدف فراتر رفتن از اتصال مستقیم دو نقطه باشد، مسئله شبکه‌سازی کوانتومی مطرح می‌شود. در چنین محیطی، دیگر تنها نرخ کلید یک لینک خاص مهم نیست، بلکه نحوه توزیع و مصرف کلیدها در سراسر توپولوژی، مدیریت لینک‌های میانی، تاب‌آوری در برابر خرابی، و منطق مسیریابی نیز به مسئله افزوده می‌شود.

به همین دلیل، یکی از ابعاد مهم اعتبارسنجی این پروژه، توسعه و بررسی یک لایه شبکه بر بستر موتور فیزیکی و امنیتی بود.

اوج این مرحله از اعتبارسنجی در شبیه‌سازی یک **توپولوژی ۷ گرهی** حاصل شد؛ توپولوژی‌ای که شامل فرستنده‌ها، گیرنده‌ها و رله‌های قابل اعتماد (Trusted Relays) بوده و امکان بررسی سناریوهای چندمرحله‌ای را فراهم می‌کرد. در این سناریو، موتور شبکه باید علاوه بر مدیریت تولید کلید روی لینک‌های منفرد، بتواند از این کلیدها در سطح بین‌گره‌ای نیز استفاده کند. این مسئله شامل نگهداری و به‌روزرسانی استخرهای کلید (Key Pools)، ارزیابی هزینه مسیرها، تصمیم‌گیری برای انتخاب مسیر مناسب، و در صورت لزوم، رله‌گذاری مرحله‌به‌مرحله کلید میان گره‌هاست.

اعتبارسنجی این لایه نشان داد که سامانه توانایی انجام موارد زیر را دارد:

- اجرای دقیق منطق **کوتاه‌ترین مسیر اول** (SPF+) بر اساس هزینه لینک و وضعیت استخر کلیدها؛
- انجام **رله‌گذاری مرحله‌به‌مرحله** (Hop-by-Hop Relay) و مدیریت ترکیب کلید در مسیرهای چندگامی؛
- نمایش **Failover پویا**، به این معنا که در صورت حذف یا قطع عمدی یک لینک اصلی، سامانه بتواند مسیر جایگزین مناسبی را شناسایی و سرویس توزیع کلید را از مسیر جدید ادامه دهد؛
- تغذیه و بازیابی خودکار استخرهای کلید برای جلوگیری از فروپاشی عملکرد شبکه در بارهای متغیر.

این سطح از اعتبارسنجی نشان می‌دهد که فریم‌ورک حاضر صرفاً ابزاری برای تولید چند شاخص فیزیکی نیست، بلکه می‌تواند در مطالعه معماری‌های گسترده‌تر QKD و رفتار شبکه‌ای نیز نقش ایفا کند. این ویژگی به‌ویژه برای پژوهش‌هایی که به استقرار شبکه‌های کوانتومی کاربردی، مدیریت کلید در مقیاس چندگره‌ای، یا ارزیابی تاب‌آوری مسیرها علاقه‌مندند، اهمیت ویژه دارد.

تاب‌آوری مهندسی

هر پروژه نرم‌افزاری که به‌طور جدی در معرض توسعه، بازطراحی، افزودن قابلیت و اعتبارسنجی مستمر قرار گیرد، دیر یا زود با موارد لبه، تضادهای معماری، یا ناهماهنگی‌های پنهانی روبه‌رو می‌شود. در سامانه‌ای مانند این فریم‌ورک، که در آن لایه‌های فیزیک، آمار، امنیت، همزمانی و شبکه در تعامل‌اند، چنین مواردی نه استثنای، بلکه بخشی طبیعی از مسیر بلوغ مهندسی هستند. تفاوت اصلی میان یک نمونه آزمایشگاهی و یک ابزار پژوهشی قابل اتکا، تا حد زیادی در نحوه مواجهه با همین جزئیات نهفته است.

در طول توسعه و اعتبارسنجی این فریم‌ورک، چندین مورد لبه شناسایی و اصلاح شدند که رفع آن‌ها نقشی مهم در افزایش تاب‌آوری سیستم ایفا کرد. بخشی از این اصلاحات به **تضادهای ارث‌بری** در امضاهای متدهای برخی اثبات‌ها، از جمله شاخه Tight، مربوط می‌شد. اگرچه این دسته از مسائل ممکن است در نگاه اول صرفاً به سازگاری کدنویسی مربوط به نظر برسند، اما در عمل می‌توانند منجر به اجرای نادرست متدها، تفسیر غلط داده‌های ورودی، یا ناسازگاری خاموش در مسیرهای اجرایی شوند. رفع این تعارض‌ها باعث شد قراردادهای میان کلاس‌های پایه و مشتق‌شده شفاف‌تر و پایدارتر شوند.

بخش دیگری از اصلاحات به **سریال‌سازی و بازیابی وضعیت**، به‌ویژه در سطح آشکارساز، مربوط بود. هنگامی که یک سامانه شبیه‌سازی قرار است اجزای خود را ذخیره و بازیابی کند، عدم کنترل دقیق نسخه طرح‌واره (Schema Version) می‌تواند به برش خاموش داده‌ها، جابه‌جایی فیلدها، یا بازیابی ناقص وضعیت منجر شود. به همین دلیل، بررسی‌های سخت‌گیرانه‌تری برای سازگاری نسخه طرح‌واره و درستی بازخوانی داده‌ها اعمال شد. این موضوع برای سناریوهایی که در آن‌ها تداوم وضعیت آشکارساز، carry-over رویدادها یا ادامه اجرای دسته‌ای اهمیت دارد، حیاتی است.

همچنین، برخی اصلاحات به **چینش فیلدهای تغییرناپذیر در ساختارهای داده** و نحوه پذیرش پارامترهای امنیتی چندگانه بازمی‌گشت. در ساختارهای مبتنی بر dataclass، کوچک‌ترین ناسازگاری در ترتیب فیلدها یا قرارداد سازنده می‌تواند در نسخه‌های مختلف یا هنگام گسترش مدل، به رفتارهای شکننده منتهی شود. بازتنظیم این بخش‌ها، انعطاف‌پذیری سیستم در افزودن پارامترهای جدید را افزایش داد و احتمال بروز خطاهای پنهان در ساخت اشیا را کاهش داد.

اهمیت این اصلاحات در آن است که نشان می‌دهند تاب‌آوری یک فریم‌ورک، تنها از طریق داشتن مدل‌های نظری خوب حاصل نمی‌شود. یک ابزار پژوهشی باید در برابر ورودی‌های مرزی، سناریوهای غیرمعمول، توسعه تدریجی و تنوع مسیرهای اجرا نیز پایدار باشد. از این منظر، تاب‌آوری مهندسی بخشی از اعتبار علمی پروژه است؛ زیرا نتایجی که بر بستر نرم‌افزار شکننده تولید شوند، حتی اگر از نظر فرمول‌بندی صحیح باشند، در سطح اعتمادپذیری پژوهشی با محدودیت مواجه خواهند بود.

راهنمای مطالعه این مستند

این مستند با این فرض تدوین شده است که خواننده ممکن است از یکی از چند مسیر متفاوت وارد آن شود: از مسیر نظریه QKD، از مسیر پیاده‌سازی نرم‌افزار، از مسیر اعتبارسنجی پژوهشی، یا از مسیر کاربردهای شبکه‌ای. به همین دلیل، ساختار سند به‌گونه‌ای طراحی شده که هم بتوان آن را به‌صورت ترتیبی از ابتدا تا انتها مطالعه کرد و هم بتوان از آن به‌عنوان یک مرجع موضوع‌محور استفاده نمود. به‌طور کلی، این سند در چند بخش اصلی سازمان‌دهی شده است:

• بخش اول: معماری فریم‌ورک و فیزیک پایه

در این بخش، مبانی مفهومی و محاسباتی لایه فیزیکی معرفی می‌شوند. مدل‌های منبع نوری، کانال، تلفات، نویز، و آشکارسازها در این قسمت تشریح می‌گردند و روشن می‌شود که هر یک چگونه در قالب ساختار نرم‌افزاری پروژه پیاده‌سازی شده‌اند.

• بخش دوم: اثبات‌های امنیتی رمزنگاری

این بخش به منطق امنیتی فریم‌ورک اختصاص دارد. در آن، چارچوب‌های پشتیبانی‌شده برای تخمین پارامتر، کران‌بندی آماری و محاسبه نرخ کلید امن ارائه می‌شوند. افزون بر فرمول‌بندی نظری، ارتباط این اثبات‌ها با رابط‌های نرم‌افزاری و داده‌های ورودی نیز توضیح داده می‌شود.

• بخش سوم: خط لوله اجرایی و لایه شبکه

در این بخش، مسیر کامل پردازش داده از مرحله اجرای شبیه‌سازی تا تولید خروجی و همچنین منطق موتور توزیع، کارگرها، هماهنگی موازی و قابلیت‌های شبکه‌ای تشریح می‌شود. این بخش برای فهم رفتار سامانه در مقیاس بزرگ و سناریوهای چندگره‌ای اهمیت ویژه دارد.

• بخش چهارم: اعتبارسنجی و تست

در این قسمت، بنچمارک‌های تحلیلی، روش‌های آزمون، نتایج اعتبارسنجی، و سناریوهای تستی که برای راستی‌آزمایی فریم‌ورک به‌کار رفته‌اند، ارائه می‌شود. این بخش برای خوانندگانی که به اعتمادپذیری نتایج و بازتولیدپذیری علاقه‌مندند، اهمیت محوری دارد.

در مجموع، هدف از این ساختاربندی آن است که مستند حاضر صرفاً توصیفی از یک کدبیس نباشد، بلکه پلی میان مبانی نظری QKD، مهندسی دقیق نرم‌افزار و اعتبارسنجی پژوهشی ایجاد کند. به بیان دیگر، این سند باید هم برای کسی که می‌خواهد منطق فیزیکی و امنیتی فریم‌ورک را درک کند مفید باشد، هم برای توسعه‌دهنده‌ای که می‌خواهد آن را گسترش دهد، و هم برای پژوهشگری که می‌خواهد به نتایج آن استناد کند یا آن‌ها را بازتولید نماید.

امید آن است که این فریم‌ورک بتواند به‌عنوان ابزاری قابل‌اعتماد، شفاف، توسعه‌پذیر و بازتولیدپذیر در اختیار پژوهشگران حوزه رمزنگاری کوانتومی قرار گیرد؛ ابزاری که نه تنها برای تحلیل یک سناریوی منفرد، بلکه برای ساختن و ارزیابی نسل بعدی مطالعات در حوزه استقرار سامانه‌ها و شبکه‌های کوانتومی کاربردی مفید باشد. اگر این پروژه بتواند حتی بخشی از مسیر میان صورت‌بندی‌های نظری QKD و سامانه‌های عملی و مهندسی‌شده آن را روشن‌تر کند، به هدف اصلی خود نزدیک شده است.

فهرست مطالب

آ	Preface
آ	پیشگفتار
۱	اول معماری چارچوب و فیزیک بنیادی
۲	۱ مقدمه و ساختار پروژه
۲	۱.۱ زمینه و انگیزه
۳	۲.۱ چالش گذار از QKD نظری به QKD عملی
۴	۳.۱ ضرورت شبیه‌سازی در سامانه‌های QKD
۵	۴.۱ بیان مسئله و هدف پروژه
۶	۵.۱ نمای کلی فریم‌ورک شبیه‌سازی
۷	۶.۱ مشارکت‌ها و قابلیت‌های اصلی فریم‌ورک
۸	۷.۱ ساختار نرم‌افزاری و اجرای پروژه
۸	۸.۱ ساختار گزارش
۱۰	۲ پیکربندی هسته و پارامترها
۱۰	۱.۲ نمای کلی و هدف ماژول params.py
۱۰	۲.۲ مبانی تئوری و فیزیکی در params.py
۱۱	۱.۲.۲ امنیت ترکیب‌پذیر و پارامترهای اِپسیلون
۱۱	۲.۲.۲ کارایی تصحیح خطا و ضریب f_{EC}
۱۱	۳.۲.۲ اتلاف کانال و مدل‌های چندجزئی
۱۱	۴.۲.۲ اختلال‌های غیرخطی WDM: پراکندگی رامان
۱۲	۵.۲.۲ فیزیک ماژولاسیون زیرحامل (SCM)
۱۲	۶.۲.۲ پراکندگی جبرانی فعال
۱۲	۳.۲ تحلیل معماری و ساختار داده‌ها در params.py
۱۲	۱.۳.۲ کلاس QKDParams: یک دیتاکلاس ایستا
۱۲	۲.۳.۲ دسته‌بندی فیلدها
۱۲	۳.۳.۲ پروتکل‌های تایپی و Protocol
۱۳	۴.۳.۲ شمارش‌ها (Enums)
۱۳	۵.۳.۲ سیستم نسخه‌سازی شما
۱۳	۶.۳.۲ محدودیت‌های سریال‌سازی
۱۳	۴.۲ پیاده‌سازی ریاضی و الگوریتمی در params.py
۱۳	۱.۴.۲ فرمول واحد اتلاف کل
۱۴	۲.۴.۲ اعتبارسنجی امنیت ترکیب‌پذیر
۱۴	۳.۴.۲ الگوریتم تبدیل ایمن شمارش‌ها
۱۵	۴.۴.۲ سریال‌سازی بازگشتی و حفاظت در برابر نشت seed
۱۵	۵.۴.۲ متد from_dict: بازسازی سازگار
۱۶	۶.۴.۲ متد with_channel_rebuilt و حفاظت یکپارچگی اتلاف
۱۶	۷.۴.۲ توابع کارخانه‌ی Lim 2014

۵.۲	ارسال و دریافت با سایر ماژول‌ها در <code>params.py</code>	۱۷
۱.۵.۲	وابستگی‌های ورودی	۱۷
۲.۵.۲	خروجی‌ها و نقاط مصرف	۱۷
۳.۵.۲	قرارداد سریال‌سازی	۱۷
۶.۲	نمای کلی و هدف ماژول <code>constants_helpers.py</code>	۱۸
۷.۲	مبانی تئوری و فیزیکی در <code>constants_helpers.py</code>	۱۸
۱.۷.۲	تحلیل عددی ممیز شناور	۱۸
۲.۷.۲	آنتروپی شانون و ناپایداری عددی	۱۸
۳.۷.۲	نرمال‌سازی L1 و توزیع احتمال	۱۹
۴.۷.۲	تبدیل دسی‌بل به خطی	۱۹
۵.۷.۲	بردار احتمال معتبر	۱۹
۸.۲	تحلیل معماری و ساختار داده‌ها در <code>constants_helpers.py</code>	۱۹
۱.۸.۲	فهرست توابع عمومی	۱۹
۲.۸.۲	توابع کمکی خصوصی	۲۰
۳.۸.۲	تابع <code>_to_real_1d_array()</code> : اعتبارسنجی آرایه‌ای	۲۰
۴.۸.۲	تایپ‌هینت‌های <code>NDArray</code> و <code>ArrayLike</code>	۲۰
۹.۲	پیاده‌سازی ریاضی و الگوریتمی در <code>constants_helpers.py</code>	۲۰
۱.۹.۲	تابع <code>is_close()</code> : مقایسه‌ی ایمن	۲۰
۲.۹.۲	تابع <code>is_valid_probability()</code> : اعتبارسنجی بازه‌ی احتمال	۲۱
۳.۹.۲	تابع <code>clamp_probability()</code> : برش احتمال	۲۱
۴.۹.۲	تابع <code>clamp_probabilities()</code> : نسخه‌ی برداری	۲۱
۵.۹.۲	تابع <code>renormalize_normalize_probabilities()</code> : نرمال‌سازی با <code>fallback</code>	۲۱
۶.۹.۲	تابع <code>db_to_linear()</code> : تبدیل ایمن	۲۲
۷.۹.۲	تابع <code>is_prob_vector()</code> : اعتبارسنجی بردار احتمال	۲۲
۱۰.۲	ارسال و دریافت با سایر ماژول‌ها در <code>constants_helpers.py</code>	۲۲
۱.۱۰.۲	وابستگی‌های ورودی	۲۲
۲.۱۰.۲	نقاط مصرف	۲۳
۳.۱۰.۲	قرارداد رابط	۲۳
۱۱.۲	نمای کلی و هدف ماژول <code>constants_definitions.py</code>	۲۳
۱۲.۲	مبانی تئوری و فیزیکی در <code>constants_definitions.py</code>	۲۴
۱.۱۲.۲	سلسله‌مراتب تلورانس	۲۴
۲.۱۲.۲	خطای ماشین و <code>Y1_SAFE_THRESHOLD</code>	۲۴
۳.۱۲.۲	ثابت‌های فیزیکی SI 2019	۲۵
۴.۱۲.۲	بذر تولیدنده‌ی عدد تصادفی PCG64	۲۵
۵.۱۲.۲	آستانه‌ی دم‌پواسون	۲۵
۶.۱۲.۲	روش‌های LP Solver	۲۵
۱۳.۲	تحلیل معماری و ساختار داده‌ها در <code>constants_definitions.py</code>	۲۶
۱.۱۳.۲	استفاده‌ی گسترده از <code>Final</code>	۲۶
۲.۱۳.۲	نوع <code>Literal</code> و <code>TypeAlias</code>	۲۶
۳.۱۳.۲	فهرست <code>PUBLIC_CONSTANT_NAMES</code>	۲۶
۴.۱۳.۲	ثابت <code>MIN_SUCCESSFUL_Z1_BASIS_EVENTS_FOR_PHASE_EST</code>	۲۶
۱۴.۲	پیاده‌سازی ریاضی و الگوریتمی در <code>constants_definitions.py</code>	۲۷
۱.۱۴.۲	محاسبه‌ی EPS از NumPy	۲۷
۲.۱۴.۲	محاسبه‌ی <code>Y1_SAFE_THRESHOLD</code> با ماکزیمم	۲۷
۳.۱۴.۲	تنظیم <code>ENTROPY_PROB_CLAMP</code> برابر با <code>NUMERIC_ABS_TOL</code>	۲۷
۴.۱۴.۲	محاسبه‌ی <code>LP_CONSTRAINT_VIOLATION_TOL</code>	۲۷
۵.۱۴.۲	ثابت‌های SI 2019	۲۸
۶.۱۴.۲	ماکروی بذر PCG64	۲۸
۱۵.۲	ارسال و دریافت با سایر ماژول‌ها در <code>constants_definitions.py</code>	۲۸
۱.۱۵.۲	عدم وابستگی ورودی	۲۸
۲.۱۵.۲	نقاط مصرف	۲۸

۲۹	۳.۱۵.۲	قرارداد <code>PUBLIC_CONSTANT_NAMES</code>
۲۹	۱۶.۲	نمای کلی و هدف ماژول <code>constants.py</code>
۲۹	۱۷.۲	مبانی تئوری و فیزیکی در <code>constants.py</code>
۲۹	۱.۱۷.۲	الگوی فساد در مهندسی نرم افزار
۲۹	۲.۱۷.۲	پایداری رابط و نسخه سازی
۳۰	۳.۱۷.۲	ثابت های فیزیکی تکمیلی
۳۰	۱۸.۲	تحلیل معماری و ساختار داده ها در <code>constants.py</code>
۳۰	۱.۱۸.۲	ساختار واردات
۳۰	۲.۱۸.۲	تعریف <code>__all__</code> پویا
۳۰	۳.۱۸.۲	تابع <code>as_dict()</code>
۳۱	۴.۱۸.۲	عدم استفاده از <code>Final</code>
۳۱	۱۹.۲	پیاده سازی ریاضی و الگوریتمی در <code>constants.py</code>
۳۱	۱.۱۹.۲	پیاده سازی <code>as_dict()</code>
۳۱	۲.۱۹.۲	مقدار SI 2019 سرعت نور
۳۱	۳.۱۹.۲	تعریف موازی <code>CONST_PLANCK</code>
۳۲	۴.۱۹.۲	ترکیب API از دو منبع
۳۲	۲۰.۲	ارسال و دریافت با سایر ماژول ها در <code>constants.py</code>
۳۲	۱.۲۰.۲	وابستگی های ورودی
۳۲	۲.۲۰.۲	نقاط مصرف
۳۲	۳.۲۰.۲	قرارداد نسخه گذاری
۳۳	۴.۲۰.۲	جمع بندی معماری چهار ماژول
۳۴	۳	فیزیک و شبیه سازی کانال
۳۴	۱.۳	نمای کلی و هدف ماژول
۳۵	۲.۳	مبانی تئوری و فیزیکی
۳۵	۱.۲.۳	قانون Beer-Lambert و تلفات فیبر نوری
۳۵	۲.۲.۳	تفکیک η_{channel} از بهره ی کلی آشکارساز
۳۵	۳.۲.۳	پدیده ی زیرنرمال بودن و سرریز عددی
۳۶	۴.۲.۳	نویز حرارتی Johnson-Nyquist
۳۶	۵.۲.۳	نویز شات Schottky
۳۶	۶.۲.۳	کسر تلفات تک کیلومتری و تقریب خطی
۳۷	۷.۲.۳	ترکیب آبخاری کانال ها
۳۷	۸.۲.۳	پاشش کروماتیک و محدودیت های مدل
۳۸	۳.۳	تحلیل معماری و ساختار داده ها
۳۸	۱.۳.۳	کلاس <code>FiberChannel</code> به عنوان <code>Frozen Dataclass</code>
۳۸	۲.۳.۳	فیلدهای کلاس و نقش آن ها
۳۹	۳.۳.۳	<code>__hash__ = None:</code>
۳۹	۴.۳.۳	زیرکلاس <code>AttenuationOnlyFiberChannel</code>
۳۹	۵.۳.۳	ثابت های آستانه ای
۴۰	۶.۳.۳	مدیریت <code>strict mode</code> با <code>ContextVar</code>
۴۰	۷.۳.۳	ثبت هشدارهای فیزیکی مشکوک
۴۰	۸.۳.۳	کلاس <code>ElectricalNoiseConfig</code>
۴۱	۹.۳.۳	کلاس <code>ChannelSimParams</code>
۴۱	۴.۳	پیاده سازی ریاضی و الگوریتمی
۴۱	۱.۴.۳	محاسبه ی ضریب انتقال در <code>_compute_transmittance</code>
۴۲	۲.۴.۳	بررسی سلامت ضریب انتقال
۴۳	۳.۴.۳	محاسبه ی کسر تلفات تک کیلومتری
۴۳	۴.۴.۳	تقریب خطی تلفات و هشدار منسوخ سازی
۴۴	۵.۴.۳	ترکیب آبخاری کانال ها
۴۴	۶.۴.۳	ترکیب آبخاری بدون اعتبارسنجی
۴۵	۷.۴.۳	محاسبه ی نویز حرارتی

۴۶	محاسبه‌ی نویز شات	۸.۴.۳
۴۶	ترکیب نویزهای مستقل	۹.۴.۳
۴۶	متد from_total_loss و ساخت دو مرحله‌ای	۱۰.۴.۳
۴۷	متد _fast_construct	۱۱.۴.۳
۴۸	متد validate	۱۲.۴.۳
۴۹	متد link_efficiency	۱۳.۴.۳
۴۹	توابع برداری transmittance_array و transmittance_scalar	۱۴.۴.۳
۵۰	ارسال و دریافت با سایر ماژول‌ها	۵.۳
۵۰	وابستگی‌های ورودی به channel.py	۱.۵.۳
۵۰	وابستگی‌های ورودی به noise_models.py	۲.۵.۳
۵۰	خروجی‌ها و مصرف‌کنندگان	۳.۵.۳
۵۱	جریان داده از ورودی تا خروجی	۴.۵.۳
۵۲	مدیریت نسخه‌گذاری و سازگارپسین	۵.۵.۳
۵۲	رفتار در برابر خطا و پیام‌های تشخیصی	۶.۵.۳
۵۲	جمع‌بندی معماری	۷.۵.۳
۵۳	۴ منابع نوری	
۵۳	نمای کلی و هدف ماژول	۱.۴
۵۴	مبانی تئوری و فیزیکی	۲.۴
۵۴	آمار پواسون و منابع لیزری تضعیف‌شده	۱.۲.۴
۵۴	آمار گرمایی (بوز-آیشتین) و منابع نور گرمایی	۲.۲.۴
۵۵	ماتریس چگالی در پایه‌ی فوک و مدل‌سازی حالت‌های غیرکلاسیک	۳.۲.۴
۵۵	فرآیندهای اتلاف روی ماتریس چگالی: تقسیم کانال و شکست کسپیل	۴.۲.۴
۵۶	مودولاتور ماخ-زندر و انتقال آرام‌شیب	۵.۲.۴
۵۶	مدولاسیون زیرجامل (SCM) و اتحاد ژاکوبی-آنگر	۶.۲.۴
۵۷	جیتر شدت: مدل گوسی با تصحیح بایاس	۷.۲.۴
۵۸	تحلیل معماری و ساختار داده‌ها	۳.۴
۵۸	سلسله‌مراتب وراثتی کلاس‌های منبع	۱.۳.۴
۵۸	کلاس SourceSimParams: بسته‌ی یخ‌زده‌ی پارامترهای شبیه‌سازی	۲.۳.۴
۵۹	کلاس MZMConfig: مودولاتور ماخ-زندر	۳.۳.۴
۵۹	کلاس BoundedLRUCache: کش LRU با اندازه‌ی محدود	۴.۳.۴
۶۰	ثابت‌های عددی ماژول-محلی	۵.۳.۴
۶۰	پیاده‌سازی ریاضی و الگوریتمی	۴.۴
۶۰	توابع تحلیلی توزیع تعداد فوتون	۱.۴.۴
۶۱	متد scm_scaling_factor: ضریب مقیاس SCM	۲.۴.۴
۶۲	متد calculate_effective_mus: اعمال ناکامل‌های کلاسیک	۳.۴.۴
۶۲	متد apply_intensity_jitter: مدل‌های جیتر	۴.۴.۴
۶۳	متد generate_photons: تولید نهایی فوتون	۵.۴.۴
۶۴	متد MZMConfig.apply: عملیات مودولاتور	۶.۴.۴
۶۵	متد build_rho_tensor: ساخت تانسور ماتریس چگالی	۷.۴.۴
۶۵	متد apply_dm_loss_processes: اعمال اتلاف روی ماتریس چگالی	۸.۴.۴
۶۶	ارسال و دریافت با سایر ماژول‌ها	۵.۴
۶۶	ورودی‌های ماژول	۱.۵.۴
۶۶	خروجی‌های ماژول	۲.۵.۴
۶۶	وابستگی‌های داخلی	۳.۵.۴
۶۷	جریان داده در شبیه‌سازی کامل	۴.۵.۴
۶۸	۵ مدل‌های آشکارساز و ساختارهای داده	
۶۸	نمای کلی و هدف ماژول	۱.۵
۶۸	مبانی تئوری و فیزیکی	۲.۵
۷۰	تحلیل معماری و ساختار داده‌ها	۳.۵
۷۲	پیاده‌سازی ریاضی و الگوریتمی	۴.۵

۷۴	ارسال و دریافت با سایر ماژول‌ها	۵.۵
۷۵	نمای کلی و هدف ماژول	۶.۵
۷۶	مبانی تئوری و فیزیکی	۷.۵
۷۷	تحلیل معماری و ساختار داده‌ها	۸.۵
۷۸	پیاده‌سازی ریاضی و الگوریتمی	۹.۵
۸۲	ارسال و دریافت با سایر ماژول‌ها	۱۰.۵
۸۳	نمای کلی و هدف ماژول	۱۱.۵
۸۴	مبانی تئوری و فیزیکی	۱۲.۵
۸۵	تحلیل معماری و ساختار داده‌ها	۱۳.۵
۸۶	پیاده‌سازی ریاضی و الگوریتمی	۱۴.۵
۸۸	ارسال و دریافت با سایر ماژول‌ها	۱۵.۵

۹۰ دوم اثبات‌های امنیتی رمزنگاری

۹۱	۶ ساختار بنیادین اثبات امنیت	
۹۱	۱.۶ نمای کلی و هدف ماژول	
۹۱	۲.۶ مبانی تئوری و فیزیکی	
۹۱	۱.۲.۶ امنیت قابل ترکیب و تقسیم بودجه ϵ	
۹۲	۲.۲.۶ تخمین پارامتر و کران‌های آماری	
۹۲	۳.۲.۶ آنتروپی باینری و تقویت حریم خصوصی	
۹۳	۴.۲.۶ برنامه‌ریزی خطی در تخمین Decoy	
۹۳	۳.۶ تحلیل معماری و ساختار داده‌ها	
۹۳	۱.۳.۶ واردسازی وابستگی‌ها	
۹۴	۲.۳.۶ ثابت‌های ماژول و TypeAlias	
۹۴	۳.۳.۶ شمارش‌ها و Enum‌های خطا	
۹۵	۴.۳.۶ Dataclass‌های داده	
۹۷	۵.۳.۶ کلاس پایه انتزاعی FiniteKeyProof	
۹۸	۴.۶ پیاده‌سازی ریاضی و الگوریتمی	
۹۸	۱.۴.۶ تبدیل نوع‌ها برای JSON	
۹۸	۲.۴.۶ کران‌های آماری در عمل	
۹۹	۳.۴.۶ محاسبه آنتروپی باینری	
۱۰۰	۴.۴.۶ متدهای ایمن عددی	
۱۰۱	۵.۴.۶ مدیریت زمان‌سنجی و گزارش‌گیری Audit	
۱۰۲	۶.۴.۶ توضیح تصمیم کلید	
۱۰۲	۷.۴.۶ مشتق کردن Seed فرزند با HMAC	
۱۰۲	۵.۶ ارسال و دریافت با سایر ماژول‌ها	
۱۰۲	۱.۵.۶ ورودی‌های ماژول	
۱۰۳	۲.۵.۶ خروجی‌های ماژول	
۱۰۳	۳.۵.۶ جریان داده در سراسر فریم‌ورک	

۱۰۵	۷ اثبات امنیت کلید محدود لیم ۲۰۱۴	
۱۰۵	۱.۷ نمای کلی و هدف ماژول	
۱۰۶	۲.۷ مبانی تئوری و فیزیکی	
۱۰۶	۱.۲.۷ پروتکل Decoy-State با سه شدت	
۱۰۶	۲.۲.۷ تخصیص ϵ و 21 رویداد شکست	
۱۰۷	۳.۲.۷ کران‌های Hoeffding و انگیزه استراتژی هیبرید	
۱۰۷	۴.۲.۷ استراتژی هیبرید Hoeffding/Chernoff	
۱۰۸	۵.۲.۷ فرمول تحلیلی کران پایین بازده خلأ	
۱۰۸	۶.۲.۷ فرمول تحلیلی کران پایین بازده تک‌فوتونی	
۱۰۹	۷.۲.۷ فرمول تحلیلی کران بالای خطای تک‌فوتونی	
۱۰۹	۸.۲.۷ کران بالای خطای فاز با نوسان نمونه‌گیری تصادفی	

۱۰۹	طول کلید امن و نشت تصحیح خطا	۹.۲.۷
۱۱۰	قید فیزیکی دفاعی روی طول کلید	۱۰.۲.۷
۱۱۰	تحلیل معماری و ساختار داده‌ها	۳.۷
۱۱۰	ثابت‌های ماژول و کنترل عددی	۱.۳.۷
۱۱۰	DecoyEstimatesLim2014	۲.۳.۷
۱۱۱	Lim2014Proof و سازنده آن	۳.۳.۷
۱۱۲	متدهای استاتیک ایمن عددی	۴.۳.۷
۱۱۳	TallyCounts اعتبارسنجی	۵.۳.۷
۱۱۳	پیاده‌سازی ریاضی و الگوریتمی	۴.۷
۱۱۳	اعتبارسنجی پیکربندی	۱.۴.۷
۱۱۴	محاسبه توزیع Poisson و τ_n	۲.۴.۷
۱۱۵	کران‌های هیبرید Hoeffding/Chernoff با دو پارامتر	۳.۴.۷
۱۱۶	مسیر N-decoy مبتنی بر LP	۴.۴.۷
۱۱۶	پیاده‌سازی معادله (۲): کران پایین بازده خلأ	۵.۴.۷
۱۱۷	پیاده‌سازی معادله (۳): کران پایین بازده تک‌فوتونی	۶.۴.۷
۱۱۷	پیاده‌سازی معادله (۴): کران بالای خطای تک‌فوتونی	۷.۴.۷
۱۱۸	پیاده‌سازی معادله (۵): کران بالای خطای فاز	۸.۴.۷
۱۱۹	محاسبه نشت تصحیح خطا	۹.۴.۷
۱۱۹	حل تکراری $\varepsilon_{\text{sec}} = \kappa \cdot \ell$	۱۰.۴.۷
۱۲۱	نرمال‌سازی stats_map و نتایج صفر	۱۱.۴.۷
۱۲۱	محاسبه اصلی طول کلید	۱۲.۴.۷
۱۲۳	calculate_key_length در قید دفاعی فیزیکی	۱۳.۴.۷
۱۲۴	ارسال و دریافت با سایر ماژول‌ها	۵.۷
۱۲۴	ورودی‌های ماژول	۱.۵.۷
۱۲۴	خروجی‌های ماژول	۲.۵.۷
۱۲۵	جریان داده در سراسر فریم‌ورک	۳.۵.۷
۱۲۶	برهان‌های (Wang, 2005) (Ma, 2005)	۸
۱۲۶	نمای کلی و هدف ماژول	۱.۸
۱۲۷	مبانی تئوری و فیزیکی	۲.۸
۱۲۷	حمله Photon-Number Splitting و انگیزه Decoy-State	۱.۲.۸
۱۲۷	توزیع آماری فوتون‌های ارسالی	۲.۲.۸
۱۲۸	بازده، خطا و Gain	۳.۲.۸
۱۲۸	کران پایین بازده تک‌فوتونی در پروتکل Vacuum + Weak	۴.۲.۸
۱۲۸	کران بالای خطای تک‌فوتونی	۵.۲.۸
۱۲۸	کسر برچسب‌گذاری‌شده (Tagged Fraction)	۶.۲.۸
۱۲۹	فرمول GLLP برای نرخ کلید	۷.۲.۸
۱۲۹	فرمول Weak GLLP (معادله ۴۳)	۸.۲.۸
۱۲۹	حد Asymptotic با Infinite Decoy	۹.۲.۸
۱۲۹	خطای منبع و چارچوب Wang 2008	۱۰.۲.۸
۱۳۰	نرخ کلید در چارچوب Wang 2008	۱۱.۲.۸
۱۳۰	تحلیل معماری و ساختار داده‌ها	۳.۸
۱۳۰	سلسله‌مراتب کلاس در ماژول ma2005.py	۱.۳.۸
۱۳۱	متد _get_prob_n و انتخاب توزیع آماری	۲.۳.۸
۱۳۱	نرمال‌سازی کلیدهای stats_map با _normalize_stats_map	۳.۳.۸
۱۳۲	تخصیص بودجه ε در چارچوب Ma 2005	۴.۳.۸
۱۳۳	ساختار dataclass در ماژول wang2005.py	۵.۳.۸
۱۳۳	اعتبارسنجی پیکربندی در Wang2005Proof	۶.۳.۸
۱۳۴	مدیریت خطای تطبیقی (Adaptive Error Correction)	۷.۳.۸
۱۳۴	نقشه‌ی نمادها (Notation Map)	۸.۳.۸
۱۳۵	پیاده‌سازی ریاضی و الگوریتمی	۴.۸
۱۳۵	محاسبه نرخ GLLP استاندارد	۱.۴.۸

۱۳۶	Vacuum + Weak	پیاده‌سازی پروتکل	۲.۴.۸
۱۳۹	General Two-Decoy	پیاده‌سازی پروتکل	۳.۴.۸
۱۳۹	One-Decoy	پیاده‌سازی پروتکل	۴.۴.۸
۱۴۰	LP	پیاده‌سازی پروتکل چند Decoy با	۵.۴.۸
۱۴۱	Asymptotic	پیاده‌سازی حد	۶.۴.۸
۱۴۲	Wang 2008	پیاده‌سازی چارچوب	۷.۴.۸
۱۴۳		ارسال و دریافت با سایر ماژول‌ها	۵.۸
۱۴۳	ma2005.py	ورودی‌های ماژول	۱.۵.۸
۱۴۴	wang2005.py	ورودی‌های ماژول	۲.۵.۸
۱۴۴		خروجی‌های ماژول	۳.۵.۸
۱۴۵		وابستگی‌های داخلی و خارجی	۴.۵.۸
۱۴۵		جریان داده در سراسر فریم‌ورک	۵.۵.۸
۱۴۶	Wang 2008 و Ma 2005	انتخاب میان	۶.۵.۸
۱۴۷		اثبات امنیت فشرده	۹
۱۴۷		نمای کلی و هدف ماژول	۱.۹
۱۴۸		مبانی تئوری و فیزیکی	۲.۹
۱۴۸		امنیت ترکیبی و بودجه‌ی ϵ	۱.۲.۹
۱۴۸		خطای فاز در مقابل خطای بیت	۲.۲.۹
۱۴۸		آنتروپی باینری و برش عددی	۳.۲.۹
۱۴۹		آنتروپی موجود و معادله‌ی کلید نهایی	۴.۲.۹
۱۴۹		تخمین شمارش تک‌فوتونی در دو پایه	۵.۲.۹
۱۵۰		جریمه‌های PA و تصحیح	۶.۲.۹
۱۵۰		تحلیل معماری و ساختار داده‌ها	۳.۹
۱۵۰		سلسله‌ی کلاس‌ها و وارث	۱.۳.۹
۱۵۰		شماره‌ی خطاهای ساختاریافته	۲.۳.۹
۱۵۱		ساختار dataclass برای ورودی و خروجی	۳.۳.۹
۱۵۱		استثنای سفارشی	۴.۳.۹
۱۵۲		ثابت‌های عددی و برش‌ها	۵.۳.۹
۱۵۲		پیاده‌سازی ریاضی و الگوریتمی	۴.۹
۱۵۲		تخصیص بودجه‌ی ϵ با حاشیه‌ی عددی	۱.۴.۹
۱۵۳		مند اصلی محاسبه‌ی طول کلید	۲.۴.۹
۱۵۵		محاسبه‌ی شمارش تک‌فوتونی در حالت Decoy-State	۳.۴.۹
۱۵۶		محاسبه‌ی نشت تصحیح خطا	۴.۴.۹
۱۵۶		محاسبه‌ی کران بالای خطای فاز	۵.۴.۹
۱۵۷		محاسبه‌ی جریمه‌های PA و تصحیح	۶.۴.۹
۱۵۷		توابع کمکی پایداری عددی	۷.۴.۹
۱۵۸		نرمال‌سازی تخمین‌های Decoy	۸.۴.۹
۱۵۸		نهایی‌سازی نتیجه و ثبت پروونانس	۹.۴.۹
۱۵۹		ارسال و دریافت با سایر ماژول‌ها	۵.۹
۱۵۹		ورودی‌های ماژول	۱.۵.۹
۱۶۰		خروجی‌های ماژول	۲.۵.۹
۱۶۰		وابستگی‌های داخلی و خارجی	۳.۵.۹
۱۶۰		جریان داده در سراسر فریم‌ورک	۴.۵.۹
۱۶۲		بهینه‌سازی پارامترها با استفاده از برنامه‌ریزی خطی (LP)	۱۰
۱۶۲		نمای کلی و هدف ماژول	۱.۱۰
۱۶۳		مبانی تئوری و فیزیکی	۲.۱۰
۱۶۳		نرخ کلید GLLP و وابستگی غیرخطی به μ	۱.۲.۱۰
۱۶۳		بهینه‌سازی ν برای کلید متناهی	۲.۲.۱۰
۱۶۳		بهینه‌سازی توامان ν و P_d, P_s	۳.۲.۱۰
۱۶۴		کانال Amplitude Damping و Chapman 2018	۴.۲.۱۰

۱۶۴	۵.۲.۱۰	بهینه‌سازی کلید متناهی Lim 2014
۱۶۴	۶.۲.۱۰	محدودیت‌های فیزیکی و منطق جریمه
۱۶۵	۳.۱۰	تحلیل معماری و ساختار داده‌ها
۱۶۵	۱.۳.۱۰	ساختار کلی ماژول
۱۶۵	۲.۳.۱۰	کلاس Ma2005GlobalOptimizer
۱۶۵	۳.۳.۱۰	متد objective و جریمه‌ها
۱۶۶	۴.۳.۱۰	متد gradient با تفاضل محدود
۱۶۶	۵.۳.۱۰	الگوی callback و تزریق وابستگی
۱۶۷	۶.۳.۱۰	Hint Type ها و ایمنی نوع
۱۶۷	۴.۱۰	پیاده‌سازی ریاضی و الگوریتمی
۱۶۷	۱.۴.۱۰	محاسبه μ بهینه بر اساس معادله ۱۲ مقاله Ma 2005
۱۶۸	۲.۴.۱۰	بهینه‌سازی ν یا روش Brent
۱۶۹	۳.۴.۱۰	بهینه‌سازی احتمالات توزیع پالس
۱۷۰	۴.۴.۱۰	بهینه‌سازی توامان سه پارامتر
۱۷۰	۵.۴.۱۰	بهینه‌سازی کلید متناهی Lim 2014
۱۷۱	۶.۴.۱۰	بهینه‌سازی زاویه چرخش در Chapman 2018
۱۷۲	۷.۴.۱۰	بهینه‌سازی اکتشافی شدت‌ها
۱۷۳	۵.۱۰	ارسال و دریافت با سایر ماژول‌ها
۱۷۳	۱.۵.۱۰	ورودی‌های ماژول
۱۷۳	۲.۵.۱۰	خروجی‌های ماژول
۱۷۴	۳.۵.۱۰	وابستگی‌های داخلی و خارجی
۱۷۵	۴.۵.۱۰	جریان داده در سراسر فریم‌ورک

۱۷۶ سوم خط لوله اجرا و لایه شبکه

۱۷۷	۱۱	اسکرپت اصلی اجرا
۱۷۷	۱.۱۱	نمای کلی و هدف ماژول
۱۷۸	۲.۱۱	مبانی تئوری و فیزیکی
۱۷۸	۱.۲.۱۱	ساختار پیکربندی سلسله‌مراتبی و اصل Truth of Source Single
۱۷۸	۲.۲.۱۱	تخصیص بودجه امنیتی ε و Security Composable
۱۷۸	۳.۲.۱۱	فیزیک دارک کانت و فرض B Appendix مقاله Lim ۲۰۱۴
۱۷۹	۴.۲.۱۱	مدل فیزیکی QBER سیگنال و دکوی
۱۷۹	۵.۲.۱۱	تئوری al. et Rogers (۲۰۰۷) برای نرخ بیت sifted
۱۷۹	۶.۲.۱۱	تئوری Time Dead آشکارساز: Non-Paralyzable
۱۷۹	۷.۲.۱۱	مدل Afterpulse آشکارساز
۱۸۰	۸.۲.۱۱	نویز Raman در DWDM و مدل فیزیکی
۱۸۰	۹.۲.۱۱	بهینه‌سازی پارامترهای پالس با GLLP
۱۸۰	۳.۱۱	تحلیل معماری و ساختار داده‌ها
۱۸۰	۱.۳.۱۱	DEFAULT_SWEEPS و DEFAULT_CONFIG
۱۸۰	۲.۳.۱۱	الگوی Adapter: توابع build_*
۱۸۱	۳.۳.۱۱	تابع parse_enum و یکپارچه‌سازی Enum
۱۸۱	۴.۳.۱۱	کلاس WorkerState_ و الگوی Encapsulation State
۱۸۱	۵.۳.۱۱	ساختار داده‌ی TallyCounts
۱۸۲	۶.۳.۱۱	SimulationResults به عنوان خروجی جامع
۱۸۲	۷.۳.۱۱	DETECTOR_METADATA_KEYS, DETECTOR_OVERRIDE_KEYS و PARAMS_METADATA_KEYS
۱۸۲	۸.۳.۱۱	الگوی DetectorSimParams SourceSimParams, ChannelSimParams, SimParams
۱۸۲	۴.۱۱	پیاده‌سازی ریاضی و الگوریتمی
۱۸۲	۱.۴.۱۱	تابع set_nested_value: اعمال overrides روی پیکربندی تودرتو
۱۸۳	۲.۴.۱۱	تابع build_epsilon_allocation: اعتبارسنجی بودجه امنیتی
۱۸۳	۳.۴.۱۱	تابع build_optical_source_config: استنتاج source_rate
۱۸۴	۴.۴.۱۱	تابع build_detector: الگوی from_config_dict

۱۸۴	tally_counts_from_sifting_results تابع	۵.۴.۱۱
۱۸۵	compute_raman_dark_rate_hz: تابع نویز Raman	۶.۴.۱۱
۱۸۵	_compute_rogers_analytic_sbr تابع	۷.۴.۱۱
۱۸۶	_balance_decoy_probabilities تابع	۸.۴.۱۱
۱۸۶	optimize_pulses_for_distance: تابع بهینه‌سازی پالس	۹.۴.۱۱
۱۸۶	initializer و init_worker تابع	۱۰.۴.۱۱
۱۸۷	run_single_simulation: تابع هسته اصلی	۱۱.۴.۱۱
۱۸۸	مدیریت API متفاوت کلاس‌های اثبات	۱۲.۴.۱۱
۱۸۹	مدیریت خطا و ساخت ردیف CSV خطا	۱۳.۴.۱۱
۱۸۹	run_and_save_csv: تابع هماهنگ‌کننده اصلی	۱۴.۴.۱۱
۱۹۰	محاسبه QBER اصلاح‌شده و فاصله اطمینان	۱۵.۴.۱۱
۱۹۰	استفاده از dispatch_work_items	۱۶.۴.۱۱
۱۹۰	build_pn_lookup_table تابع	۱۷.۴.۱۱
۱۹۱	log_detector_details و log_source_details توابع	۱۸.۴.۱۱
۱۹۱	pipeline توابع کمکی	۱۹.۴.۱۱
۱۹۱	الگوی __main__ و CLI	۲۰.۴.۱۱
۱۹۱	ارسال و دریافت با سایر ماژول‌ها	۵.۱۱
۱۹۱	ورودی‌های اصلی ماژول	۱.۵.۱۱
۱۹۲	خروجی‌های اصلی ماژول	۲.۵.۱۱
۱۹۲	تعامل با ماژول qkd.sources	۳.۵.۱۱
۱۹۲	تعامل با ماژول qkd.detectors	۴.۵.۱۱
۱۹۲	تعامل با ماژول qkd.channel	۵.۵.۱۱
۱۹۲	تعامل با ماژول qkd.protocols	۶.۵.۱۱
۱۹۲	تعامل با ماژول‌های اثبات امنیتی	۷.۵.۱۱
۱۹۲	تعامل با ماژول main_optimized_dispatch	۸.۵.۱۱
۱۹۳	تعامل با qkd.io، qkd.params، qkd.exceptions و qkd.datatypes	۹.۵.۱۱
۱۹۳	مدیریت خطا و تبدیل استثنا	۱۰.۵.۱۱
۱۹۳	مدیریت KeyboardInterrupt	۱۱.۵.۱۱
۱۹۳	نقش ماژول در اکوسیستم کلی	۱۲.۵.۱۱

۱۹۴	لایه شبکه QKD و مسیریابی	
۱۹۴	نمای کلی و هدف ماژول	۱.۱۲
۱۹۵	مبانی تئوری و فیزیکی	۲.۱۲
۱۹۵	مدل کانال فیبر نوری و انتقال	۱.۲.۱۲
۱۹۵	مدل خطای کوانتومی (QBER)	۲.۲.۱۲
۱۹۶	آنتروپی باینری و کسر امن کلید	۳.۲.۱۲
۱۹۶	مدل استخر کلید و فشار (Pool Pressure)	۴.۲.۱۲
۱۹۷	تابع هزینه SPF+ و الگوریتم Dijkstra	۵.۲.۱۲
۱۹۷	پروتکل Trusted Relay مبتنی بر XOR	۶.۲.۱۲
۱۹۸	سوییچ‌های نوری و پیکربندی پویا	۷.۲.۱۲
۱۹۸	تحلیل معماری و ساختار داده‌ها	۳.۱۲
۱۹۸	شمارش‌ها (Enums) و ثابت‌های سراسری	۱.۳.۱۲
۱۹۹	داده‌کلاس KeyBlock	۲.۳.۱۲
۱۹۹	کلاس KeyPool و مدیریت چرخه حیات	۳.۳.۱۲
۲۰۱	کلاس QKDLINK و تابع هزینه	۴.۳.۱۲
۲۰۲	کلاس OpticalSwitch	۵.۳.۱۲
۲۰۲	سلسله‌مراتب کلاس‌های گره	۶.۳.۱۲
۲۰۳	شبیه‌سازهای QKD	۷.۳.۱۲
۲۰۴	ارکستراتور KMS و کلاس QKDDNetwork7Nodes	۸.۳.۱۲
۲۰۴	پیاده‌سازی ریاضی و الگوریتمی	۴.۱۲
۲۰۴	بارگذاری پویای موتور فیزیکی	۱.۴.۱۲
۲۰۵	محاسبه نرخ کلید در شبیه‌ساز آماری	۲.۴.۱۲

۲۰۶	اجراهای موتور خط لوله	۳.۴.۱۲
۲۰۷	الگوریتم SPF+ و پیاده‌سازی Dijkstra	۴.۴.۱۲
۲۰۸	درخواست کلید خدمت و رله قابل اعتماد	۵.۴.۱۲
۲۱۱	مدیریت نشست QKD و پر کردن استخر	۶.۴.۱۲
۲۱۲	کالیبراسیون برای نوع شبیه‌ساز	۷.۴.۱۲
۲۱۳	ارسال و دریافت با سایر ماژول‌ها	۵.۱۲
۲۱۳	وابستگی به موتور فیزیکی main_optimized.py	۱.۵.۱۲
۲۱۳	ورودی‌ها به ماژول	۲.۵.۱۲
۲۱۴	خروجی‌ها و گزارش‌گیری	۳.۵.۱۲
۲۱۵	یکپارچه‌سازی با اصلاحات F-01 تا F-22	۴.۵.۱۲
۲۱۵	رفتار در حالت خرابی و بازیابی	۵.۵.۱۲
۲۱۶	رفتار حافظه و کارایی	۶.۵.۱۲

چهارم اعتبارسنجی و آزمون ۲۱۷

۱۳ اعتبارسنجی لایه فیزیکی ۲۱۸

۲۱۸	نمای کلی و هدف ماژول	۱.۱۳
۲۱۹	مبانی تئوری و فیزیکی	۲.۱۳
۲۱۹	کانال فیبر نوری و قانون Beer-Lambert	۱.۲.۱۳
۲۱۹	منبع Pulse Coherent Weak و توزیع Poisson	۲.۲.۱۳
۲۲۰	منبع Thermal و توزیع Bose-Einstein	۳.۲.۱۳
۲۲۰	مدل QBER برای منبع Poisson	۴.۲.۱۳
۲۲۱	مدل Time Dead آشکارساز SPAD	۵.۲.۱۳
۲۲۱	آزمون آماری Z-test دودویی	۶.۲.۱۳
۲۲۲	تحلیل معماری و ساختار داده‌ها	۳.۱۳
۲۲۲	ساختار کلی ماژول	۱.۳.۱۳
۲۲۲	توابع تحلیلی channel_transmittance	۲.۳.۱۳
۲۲۳	توابع تحلیلی Q_mu_thermal و Q_mu_poisson	۳.۳.۱۳
۲۲۳	تابع تحلیلی E_mu_poisson	۴.۳.۱۳
۲۲۳	تابع تحلیلی dead_time_correction	۵.۳.۱۳
۲۲۴	تابع آزمون آماری validate_statistic	۶.۳.۱۳
۲۲۴	تابع اجرای شبیه‌سازی run_isolated_simulation	۷.۳.۱۳
۲۲۶	پیاده‌سازی ریاضی و الگوریتمی	۴.۱۳
۲۲۶	الگوریتم TEST ۱: اعتبارسنجی منبع Poisson	۱.۴.۱۳
۲۲۶	الگوریتم TEST ۲: اعتبارسنجی منبع Thermal	۲.۴.۱۳
۲۲۷	الگوریتم TEST ۳: اعتبارسنجی Time Dead مستقل SPAD	۳.۴.۱۳
۲۲۸	محاسبه Q-score و آستانه بحرانی	۴.۴.۱۳
۲۲۸	تحلیل پیچیدگی و کارایی	۵.۴.۱۳
۲۲۸	ارسال و دریافت با سایر ماژول‌ها	۵.۱۳
۲۲۸	وابستگی به main_optimized.py	۱.۵.۱۳
۲۲۹	ورودی‌های ماژول	۲.۵.۱۳
۲۲۹	خروجی‌های ماژول	۳.۵.۱۳
۲۲۹	ارتباط با فایل CSV خروجی	۴.۵.۱۳
۲۳۰	یکپارچه‌سازی با سیستم CI/CD	۵.۵.۱۳
۲۳۰	محدودیت‌ها و توسعه‌های آینده	۶.۵.۱۳

۱۴ اعتبارسنجی رمزنگاری و تحلیلی ۲۳۲

۲۳۲	نمای کلی و هدف ماژول	۱.۱۴
۲۳۳	مبانی تئوری و فیزیکی	۲.۱۴
۲۳۳	آنتروپی باینری شانون و نقش آن در امنیت QKD	۱.۲.۱۴
۲۳۳	کاهیدگی فیبر و قانون Beer-Lambert	۲.۲.۱۴

۲۳۴	تئوری Decoy-State و معادلات $Lo2005$	۳.۲.۱۴
۲۳۶	تلورانس اعتبارسنجی و تفسیر کریپتوگرافیک	۴.۲.۱۴
۲۳۶	تحلیل معماری و ساختار داده‌ها	۳.۱۴
۲۳۶	ساختار کلی ماژول و الگوی Functions Module-Level	۱.۳.۱۴
۲۳۶	تحلیل تابع $H2$	۲.۳.۱۴
۲۳۷	تحلیل تابع channel_transmittance	۳.۳.۱۴
۲۳۷	تحلیل تابع get_analytics و ساختار خروجی	۴.۳.۱۴
۲۳۸	تحلیل تابع run_absolute_test و معماری پیکربندی	۵.۳.۱۴
۲۳۹	الگوی Comparison و Validation Tolerance-Based	۶.۳.۱۴
۲۴۰	ساختار پنج‌آزمونی و خروجی نهایی	۷.۳.۱۴
۲۴۰	تحلیل نقطه ورود __main__	۸.۳.۱۴
۲۴۱	پیاده‌سازی ریاضی و الگوریتمی	۴.۱۴
۲۴۱	پیاده‌سازی آنتروپی باینری با برش عددی	۱.۴.۱۴
۲۴۱	پیاده‌سازی فرمول $Lo2005$ برای $Y1$	۲.۴.۱۴
۲۴۱	پیاده‌سازی فرمول GLLP برای نرخ کلید	۳.۴.۱۴
۲۴۲	الگوریتم Import Lazy و مدیریت خطا	۴.۴.۱۴
۲۴۲	الگوریتم Copy Deep و جلوگیری از Mutation	۵.۴.۱۴
۲۴۲	الگوریتم Tolerance Absolute vs Relative	۶.۴.۱۴
۲۴۲	الگوریتم Pipeline Comparison و ساختار CSV	۷.۴.۱۴
۲۴۳	محاسبه ضریب انتقال برای مقایسه	۸.۴.۱۴
۲۴۳	ارسال و دریافت با سایر ماژول‌ها	۵.۱۴
۲۴۳	وابستگی به main_optimized	۱.۵.۱۴
۲۴۳	دیکشنری DEFAULT_CONFIG و Pattern Override	۲.۵.۱۴
۲۴۴	خروجی CSV و اسکیمای ستون‌ها	۳.۵.۱۴
۲۴۴	ارتباط با validate_physics.py	۴.۵.۱۴
۲۴۴	ارتباط با ماژول‌های اثبات $Lim-2014$ و $Ma-2005$	۵.۵.۱۴
۲۴۴	خروجی نهایی و استفاده در CI/CD	۶.۵.۱۴
۲۴۵	جمع‌بندی ارتباطات	۷.۵.۱۴

۱۵ اعتبارسنجی حل‌گر برنامه‌ریزی خطی

۲۴۶	نمای کلی و هدف ماژول	۱.۱۵
۲۴۷	مبانی تئوری و فیزیکی	۲.۱۵
۲۴۷	پروتکل Decoy-State و نقش پارامترهای شدت	۱.۲.۱۵
۲۴۷	آنتروپی باینری شانون و نقش آن در امنیت	۲.۲.۱۵
۲۴۷	کاهیدگی فیبر و قانون Beer-Lambert	۳.۲.۱۵
۲۴۸	Gain سیگنال و QBER سیگنال	۴.۲.۱۵
۲۴۸	Gain دکوی و استخراج $Y1$ به روش $Lo2005$	۵.۲.۱۵
۲۴۹	نرخ خطای تک‌فوتونی و بازده تک‌فوتونی	۶.۲.۱۵
۲۴۹	نرخ کلید امن مجانبی (GLLP)	۷.۲.۱۵
۲۴۹	قیدهای فیزیکی و ریاضی	۸.۲.۱۵
۲۵۰	تحلیل معماری و ساختار داده‌ها	۳.۱۵
۲۵۰	ساختار کلی ماژول و الگوی Functions Module-Level	۱.۳.۱۵
۲۵۰	تنظیم مسیر و Bootstrap Path	۲.۳.۱۵
۲۵۰	واردسازی توابع از فریم‌ورک	۳.۳.۱۵
۲۵۰	تحلیل تابع $H2$	۴.۳.۱۵
۲۵۱	تحلیل تابع analytical_skr و ساختار پارامترها	۵.۳.۱۵
۲۵۱	تحلیل تابع run_lp_test و الگوی Callback Cost	۶.۳.۱۵
۲۵۲	الگوی Comparison و Reference Hardcoded	۷.۳.۱۵
۲۵۲	الگوریتم Comparison و Verdict	۸.۳.۱۵
۲۵۳	نقطه ورود __main__ و سه‌آزمون	۹.۳.۱۵
۲۵۳	پیاده‌سازی ریاضی و الگوریتمی	۴.۱۵
۲۵۳	پیاده‌سازی آنتروپی باینری با برش عددی	۱.۴.۱۵

۲۵۳	پیاده‌سازی فرمول Lo2005 با اصلاحات فیزیکی	۲.۴.۱۵
۲۵۴	پیاده‌سازی قیدهای فیزیکی و Return Early	۳.۴.۱۵
۲۵۴	الگوریتم Callback Cost و Trick Negate	۴.۴.۱۵
۲۵۴	الگوریتم Capture Closure برای پارامترهای محیطی	۵.۴.۱۵
۲۵۵	الگوریتم Comparison Reference Hardcoded	۶.۴.۱۵
۲۵۵	الگوریتم Testing Three-Range	۷.۴.۱۵
۲۵۶	دریافت با سایر ماژول‌ها	۵.۱۵
۲۵۶	وابستگی به qkd.proofs.optimization	۱.۵.۱۵
۲۵۶	وابستگی به qkd.params	۲.۵.۱۵
۲۵۶	الگوی Oracle Independent و جلوگیری از Validation Circular	۳.۵.۱۵
۲۵۷	خروجی به stdout و گزارش متمرکز	۴.۵.۱۵
۲۵۷	ارتباط با absolute_crypto_validation.py	۵.۵.۱۵
۲۵۷	ارتباط با ماژول‌های اثبات Lim-2014 و Ma-2005	۶.۵.۱۵
۲۵۷	خروجی نهایی و استفاده در CI/CD	۷.۵.۱۵
۲۵۷	جمع‌بندی ارتباطات	۸.۵.۱۵
۲۵۸	۱۶ آزمون‌های هم‌روندی و حفظ هویت	
۲۵۸	نمای کلی و هدف ماژول	۱.۱۶
۲۵۹	مبانی تئوری و فیزیکی	۲.۱۶
۲۵۹	پروتکل BB84 و مفهوم هویت پالس	۱.۲.۱۶
۲۵۹	پیش‌بینی نتیجه اندازه‌گیری و قانون Born	۲.۲.۱۶
۲۵۹	تأخیر دوجمله‌ای و شبیه‌سازی کارگر واقعی	۳.۲.۱۶
۲۶۰	Kendall-Tau و معیار ناهم‌هنگی	۴.۲.۱۶
۲۶۰	مفهوم Identity-Bound-to-ID و Integrity Position-Independent	۵.۲.۱۶
۲۶۰	تحلیل معماری و ساختار داده‌ها	۳.۱۶
۲۶۰	ساختار کلی ماژول و الگوی Script-with-Helpers	۱.۳.۱۶
۲۶۰	ثابت IDENTITY_FIELDS و فیلدهای هویتی	۲.۳.۱۶
۲۶۱	تابع format_table و قالب‌بندی متنی	۳.۳.۱۶
۲۶۱	تابع short_identity و نمایش فشرده هویت	۴.۳.۱۶
۲۶۲	تابع identity_match و منطق مقایسه چهارمرحله‌ای	۵.۳.۱۶
۲۶۲	تابع measurement_match و اعتبارسنجی پیش‌بینی	۶.۳.۱۶
۲۶۲	تابع run_trace و الگوی Closure برای خروجی	۷.۳.۱۶
۲۶۳	تنظیم مسیر و الگوی Bootstrap Path	۸.۳.۱۶
۲۶۳	پیاده‌سازی ریاضی و الگوریتمی	۴.۱۶
۲۶۳	الگوریتم انتخاب تصادفی پالس‌های ردیابی‌شده	۱.۴.۱۶
۲۶۳	الگوریتم محاسبه Inversions Pairwise	۲.۴.۱۶
۲۶۴	الگوریتم یافتن Examples Swap Concrete	۳.۴.۱۶
۲۶۴	الگوریتم محاسبه Shifts Position	۴.۴.۱۶
۲۶۴	الگوریتم Counting Mismatch Stage-Level	۵.۴.۱۶
۲۶۵	الگوریتم Verdict و تفسیر نهایی	۶.۴.۱۶
۲۶۵	الگوریتم CLl با argparse	۷.۴.۱۶
۲۶۶	ارسال و دریافت با سایر ماژول‌ها	۵.۱۶
۲۶۶	وابستگی به tests/test_main_optimized_pulse_tracking	۱.۵.۱۶
۲۶۶	ورودی از main_optimized_dispatch	۲.۵.۱۶
۲۶۶	خروجی به stdout و فایل	۳.۵.۱۶
۲۶۶	ارتباط با فریم‌ورک آزمون	۴.۵.۱۶
۲۶۶	خروجی قابل تحلیل توسط ابزارهای خارجی	۵.۵.۱۶
۲۶۶	جمع‌بندی ارتباطات	۶.۵.۱۶

۲۶۸	پنجم نتایج شبیه‌سازی
۲۶۹	ششم نتایج شبیه‌سازی
۲۷۰	۱۷ تنظیمات آزمایش و کمپین‌های شبیه‌سازی
۲۷۰	۱.۱۷ نمای کلی کمپین‌ها
۲۷۰	۲.۱۷ خلاصه کمپین‌ها
۲۷۰	۳.۱۷ پیکربندی مشترک
۲۷۲	۱۸ نتایج خط پایه و مقیاس کلید محدود
۲۷۲	۱.۱۸ خط پایه BB۸۴ با اثبات Lim۲۰۱۴
۲۷۲	۲.۱۸ مقیاس کلید محدود (Finite-Key Scaling)
۲۷۳	۳.۱۸ اعتبارسنجی Rogers ۲۰۰۷
۲۷۴	۱۹ سناریوهای DWDM و بهینه‌سازی پالس
۲۷۴	۱.۱۹ تأثیر هم‌زیستی با کانال کلاسیک (DWDM)
۲۷۵	۲.۱۹ بهینه‌سازی خودکار پارامتر پالس
۲۷۶	۲۰ مقایسه‌های سیستماتیک
۲۷۶	۱.۲۰ مقایسه پروتکل‌ها
۲۷۶	۲.۲۰ مقایسه اثبات‌های امنیتی
۲۷۷	۳.۲۰ مقایسه آشکارسازها
۲۷۷	۴.۲۰ مقایسه روش‌های بازه اطمینان
۲۷۸	۲۱ تحلیل حساسیت
۲۷۸	۱.۲۱ حساسیت به QBER ذاتی و Misalignment

فهرست جداول

۲۷۰	۱.۱۷ خلاصه کمپین‌های شبیه‌سازی اجرا شده با <code>main_optimized.py</code>
-----	---

فهرست برنامه‌ها

بخش اول

معماری چارچوب و فیزیک بنیادی

فصل ۱

مقدمه و ساختار پروژه

۱.۱ زمینه و انگیزه

امنیت اطلاعات در دهه‌های اخیر به یکی از ارکان اصلی زیرساخت‌های دیجیتال، مخابراتی، مالی، صنعتی و حیاتی تبدیل شده است. بسیاری از سامانه‌های ارتباطی مدرن، از تبادلات بانکی و شبکه‌های داده گرفته تا زیرساخت‌های ابری، سامانه‌های کنترلی و ارتباطات حساس، بر این فرض تکیه دارند که داده‌ها می‌توانند با استفاده از کلیدهای رمزنگاری امن میان دو طرف محافظت و مبادله شوند. در رمزنگاری کلاسیک، به‌ویژه در شاخه رمزنگاری کلید عمومی، امنیت بسیاری از سازوکارهای رایج نه از قوانین بنیادین فیزیک، بلکه از دشواری محاسباتی برخی مسائل ریاضی حاصل می‌شود. برای مثال، امنیت الگوریتم‌هایی نظیر RSA به دشواری تجزیه اعداد صحیح بزرگ به عوامل اول و امنیت روش‌هایی مانند ECC به دشواری حل مسئله لگاریتم گسسته روی منحنی‌های بیضوی وابسته است.

با وجود موفقیت گسترده این خانواده از الگوریتم‌ها در کاربردهای عملی، این نوع امنیت ذاتاً امنیت محاسباتی است، نه امنیت اطلاعات نظری. به بیان دیگر، این سامانه‌ها تا زمانی امن تلقی می‌شوند که هیچ مهاجم عملی، در بازه زمانی قابل قبول، توان محاسباتی کافی برای حل مسائل زیربنایی آن‌ها را در اختیار نداشته باشد. این فرض برای دهه‌ها مبنای طراحی زیرساخت‌های امنیتی بوده است، اما ظهور محاسبات کوانتومی، به‌ویژه الگوریتم شور یا Shor's algorithm، نشان داد که یک رایانه کوانتومی مقیاس‌پذیر و دارای تصحیح خطای کافی، از نظر نظری قادر است مسائل فاکتورگیری و لگاریتم گسسته را در زمان چندجمله‌ای حل کند.

تهدید محاسبات کوانتومی تنها به زمان دسترسی عملی به چنین رایانه‌هایی محدود نیست. یکی از نگرانی‌های مهم در امنیت بلندمدت، سناریوی Harvest Now, Decrypt Later است؛ به این معنا که مهاجم می‌تواند امروز داده‌های رمزشده را ذخیره کند و در آینده، پس از دستیابی به توان پردازشی لازم، آن‌ها را رمزگشایی کند. بنابراین، حتی اگر رایانه‌های کوانتومی قدرتمند هنوز به‌صورت عملی و گسترده در دسترس نباشند، داده‌هایی که امروز با سازوکارهای سنتی منتقل می‌شوند، ممکن است در آینده محرمانگی بلندمدت خود را از دست بدهند. همین واقعیت، نیاز به روش‌هایی را برجسته می‌کند که امنیت آن‌ها تا حد امکان به توان محاسباتی مهاجم وابسته نباشد.

البته توزیع کلید کوانتومی تنها مسیر پاسخ به تهدید محاسبات کوانتومی نیست. رمزنگاری پساکوانتومی یا Post-Quantum Cryptography (PQC) نیز با هدف طراحی الگوریتم‌های کلاسیکی مقاوم در برابر حملات کوانتومی توسعه یافته است. تفاوت اصلی این رویکرد با توزیع کلید کوانتومی در آن است که PQC همچنان بر دشواری محاسباتی مسائل ریاضی جدید تکیه دارد، در حالی که توزیع کلید کوانتومی مسئله توزیع کلید را با استفاده از خواص فیزیکی سامانه‌های کوانتومی بررسی می‌کند. در عمل، این دو رویکرد می‌توانند مکمل یکدیگر باشند؛ به‌ویژه از آن جهت که سامانه‌های QKD نیز برای کانال کلاسیک خود به احراز هویت نیاز دارند.

در این بستر، توزیع کلید کوانتومی یا Quantum Key Distribution (QKD) به‌عنوان یکی از مهم‌ترین دستاوردهای نظری و عملی در تقاطع فیزیک کوانتومی، نظریه اطلاعات و امنیت ارتباطات مطرح می‌شود. ایده اصلی QKD آن است که دو طرف ارتباطی، که به‌طور قراردادی آلیس (Alice) و باب (Bob) نامیده می‌شوند، با ارسال حالت‌های کوانتومی و انجام پردازش کلاسیک احراز هویت‌شده، یک کلید محرمانه مشترک تولید کنند. امنیت QKD در چارچوب یک مدل امنیتی مشخص و تحت فروض پیاده‌سازی معین تحلیل می‌شود. در این چارچوب، تلاش مهاجم برای کسب اطلاعات از حالت‌های کوانتومی می‌تواند میان اطلاعات مهاجم و اختلال قابل‌مشاهده در داده‌های آلیس و باب مصالحه ایجاد کند. این اختلال از طریق آزمون آماری داده‌های نمونه‌برداری‌شده ارزیابی می‌شود.

این ویژگی، QKD را به‌ویژه از سازوکارهای رایج توزیع کلید مبتنی بر رمزنگاری کلید عمومی متمایز می‌کند؛

زیرا امنیت آن، در صورت برقراری فروض مدل امنیتی و وجود یک کانال کلاسیک احراز هویت شده، نه بر فرضیات پیچیدگی محاسباتی، بلکه بر اصولی مانند اثر اندازه‌گیری بر حالت کوانتومی، اصل عدم قطعیت و قضیه عدم کلونینگ استوار است. با این حال، این بیان نباید به معنای امنیت مطلق و مستقل از پیاده‌سازی تلقی شود. در سامانه‌های واقعی، اعتبار مدل منبع، آشکارساز، کانال، تصادفی‌سازی، نبود کانال‌های جانبی و نحوه تخصیص پارامترهای امنیتی همگی در نتیجه نهایی اثرگذارند.

در تحلیل‌های مدرن، امنیت کلید نهایی معمولاً با پارامتر شکست ϵ بیان می‌شود. به‌طور شهودی، کلید ϵ -امن کلیدی است که میزان تمایزپذیری آن از یک کلید ایده‌آل یکنواخت و مستقل از اطلاعات مهاجم، با پارامتر ϵ کران‌بندی شود. این پارامتر معمولاً احتمال شکست یا عدم تحقق کامل شرایط امنیتی مورد نظر را به‌صورت ترکیبی نمایش می‌دهد. این نگاه به‌ویژه در تحلیل‌های کلید محدود اهمیت دارد، زیرا در سامانه‌های عملی تعداد نمونه‌ها محدود است و نوسانات آماری نمی‌توانند نادیده گرفته شوند.

۲.۱ چالش گذار از QKD نظری به QKD عملی

پروتکل BB84 که در سال ۱۹۸۴ توسط Bennett و Brassard معرفی شد، نخستین و شناخته‌شده‌ترین پروتکل QKD است و همچنان یکی از چارچوب‌های مرجع در تحلیل‌های نظری و پیاده‌سازی‌های عملی به‌شمار می‌رود. در این پروتکل، آلیس بیت‌های اطلاعاتی را در یکی از دو پایه ناسازگار، که در صورت‌بندی ایده‌آل می‌توان آن‌ها را دو پایه متقابلاً ناریب دانست، کدگذاری کرده و برای ارسال می‌کند. باب نیز بدون اطلاع از انتخاب پایه آلیس، به‌صورت تصادفی یکی از پایه‌ها را برای اندازه‌گیری انتخاب می‌کند. پس از تبادل کوانتومی، دو طرف از طریق یک کانال کلاسیک احراز هویت شده، پایه‌های مورد استفاده را مقایسه کرده و تنها نمونه‌هایی را که در آن‌ها پایه‌ها منطبق بوده‌اند حفظ می‌کنند. این فرایند معمولاً غربال‌سازی پایه‌ها یا basis sifting نامیده می‌شود. سپس با برآورد نرخ خطا و اجرای مراحل پس‌پردازش کلاسیک، کلید نهایی استخراج می‌شود.

اهمیت BB84 از آنجا ناشی می‌شود که اندازه‌گیری حالت کوانتومی در پایه‌ای ناسازگار می‌تواند اختلالی آماری در داده‌ها ایجاد کند. بنابراین، حضور شنودگر نه صرفاً از طریق حدس یا شواهد غیرمستقیم، بلکه از طریق تحلیل آماری نرخ خطا و سایر کمیت‌های مشاهده‌شده ارزیابی می‌شود. در تحلیل امنیتی، نرخ خطای مشاهده‌شده تنها یک شاخص عملکردی نیست، بلکه برای کران‌بندی اطلاعات بالقوه مهاجم و، در بسیاری از صورت‌بندی‌ها، برای تخمین کمیت‌هایی مانند خطای فازی نیز به‌کار می‌رود.

با وجود این مبنای نظری، فاصله میان مدل ایده‌آل و پیاده‌سازی واقعی در عمل تعیین‌کننده است. بسیاری از تحلیل‌های ساده‌شده بر فرض‌هایی مانند وجود منبع تک‌فوتونی کامل، آشکارسازهای بی‌نقص، نبود نویز محیطی و تعداد نامحدود نمونه‌ها تکیه دارند؛ در حالی که هیچ‌یک از این فروض در سامانه‌های واقعی به‌طور کامل برقرار نیست. از همین رو، امنیت و عملکرد یک سامانه QKD عملی تنها با بررسی پروتکل ایده‌آل قابل ارزیابی نیست و باید اثر اجزای فیزیکی، آماری و نرم‌افزاری نیز در تحلیل وارد شود.

یکی از محدودیت‌های مهم در پیاده‌سازی عملی BB84 آن است که ساخت منابع تک‌فوتونی ایده‌آل از نظر فنی دشوار و پرهزینه است. در بسیاری از پیاده‌سازی‌ها، از پالس‌های همدوس تضعیف‌شده یا Weak Coherent Pulses (WCP) همراه با تصادفی‌سازی فاز استفاده می‌شود. در این حالت، پالس‌ها را می‌توان به‌صورت مخلوطی آماری از مؤلفه‌های n -فوتونی با توزیع پواسونی مدل کرد. بنابراین، احتمال غیرصفری برای ارسال پالس‌های چندفوتونی وجود دارد.

وجود پالس‌های چندفوتونی زمینه حملاتی را فراهم می‌کند که در مدل منبع تک‌فوتونی مطرح نیستند. مشهورترین نمونه این تهدید، حمله Photon-Number Splitting (PNS) است. در مدل ایده‌آل این حمله، مهاجم می‌تواند با تشخیص تعداد فوتون‌ها، پالس‌های چندفوتونی را شناسایی کند، بخشی از فوتون‌ها را نگه دارد و باقی پالس را به باب برساند. پس از اعلام پایه‌ها، مهاجم می‌تواند از فوتون ذخیره‌شده برای کسب اطلاعات درباره بخشی از کلید استفاده کند. اگر این رفتار در تحلیل امنیتی لحاظ نشود، ممکن است مهاجم اطلاعاتی درباره کلید به‌دست آورد، بی‌آنکه نرخ خطای مشاهده‌شده به‌تنهایی برای آشکارسازی حمله کافی باشد. بنابراین، کنترل اثر پالس‌های چندفوتونی بخش مهمی از تحلیل امنیتی عملی سامانه‌های مبتنی بر WCP است.

برای مقابله با این ضعف ساختاری، روش حالت‌های فریب یا Decoy-State Method معرفی شد که از مهم‌ترین پیشرفت‌ها در عملی‌سازی QKD محسوب می‌شود. در این رویکرد، آلیس به‌جای ارسال تمام پالس‌ها با یک شدت ثابت، به‌صورت تصادفی از چند شدت متفاوت استفاده می‌کند؛ معمولاً یک شدت سیگنال، یک یا چند شدت فریب ضعیف، و در بسیاری از موارد یک حالت خلأ یا vacuum decoy. هدف این روش آن است که رفتار کانال و آشکارسازی برای مؤلفه‌های فوتونی مختلف به‌صورت آماری تخمین زده شود.

در روش حالت‌های فریب، با فرض تصادفی‌سازی کامل فاز و نبود کانال جانبی وابسته به شدت، حالت‌های n -فوتونی حاصل از شدت‌های مختلف از دید مهاجم غیرقابل‌تمایز در نظر گرفته می‌شوند. در نتیجه، بازده و

نرخ خطای متناظر با هر مؤلفه n -فوتونی باید میان حالت‌های سیگنال و فریب مشترک باشد. آلیس و باب با مقایسهٔ آمار شدت‌های مختلف، می‌توانند بر کمیت‌هایی مانند بازده تک‌فوتونی Y_1 و نرخ خطای تک‌فوتونی e_1 کران بگذارند. این تخمین‌ها زیربنای استخراج نرخ کلید امن در حضور منابع غیرایده‌آل را تشکیل می‌دهند. به بیان دیگر، روش حالت‌های فریب امکان می‌دهد سهم پالس‌های تک‌فوتونی، که برای امنیت پروتکل نقش اصلی دارند، از سهم پالس‌های چندفوتونی، که بالقوه آسیب‌پذیرند، به‌صورت آماری تفکیک شود. در غیاب این روش، عملکرد بسیاری از سامانه‌های مبتنی بر WCP در فواصل متوسط و بلند یا از نظر امنیتی ناکافی می‌بود یا نرخ کلید آن‌ها به‌شدت افت می‌کرد. با بهره‌گیری از این تکنیک، می‌توان در شرایط واقع‌گرایانه و با در نظر گرفتن محدودیت‌های عملی، به نرخ کلید امن قابل‌استفاده دست یافت.

افزون بر مدل منبع، تحلیل سامانه‌های QKD باید میان دو رژیم مهم تمایز قائل شود: رژیم مجانبی یا asymptotic و رژیم کلید محدود یا finite-key. در مدل مجانبی فرض می‌شود که تعداد پالس‌های مبادله‌شده آن‌قدر زیاد است که نوسانات آماری ناچیز باشند و پارامترها تقریباً دقیق تخمین زده شوند. اما در عمل، هر آزمایش و هر سامانهٔ عملی با تعداد محدودی از پالس‌ها و زمان اجرایی محدود روبه‌رو است. در این وضعیت، نوسانات آماری و خطاهای نمونه‌برداری می‌توانند اثر مهمی بر برآورد کمیت‌های امنیتی بگذارند. از این رو، تحلیل‌های کلید محدود برای ارزیابی امنیت عملی سامانه‌هایی با اندازهٔ بلوک محدود ضروری‌اند. تحلیل مجانبی همچنان برای فهم رفتار پایه، مقایسه‌های اولیه و اعتبارسنجی تحلیلی ارزشمند است، اما اثر نوسانات آماری و بودجهٔ امنیتی را به‌طور کامل بازنمایی نمی‌کند. در بسیاری از صورت‌بندی‌های کلید محدود، طول کلید نهایی به کمیت‌هایی مانند کران پایین تعداد رویدادهای تک‌فوتونی، کران بالای خطای متناظر، نشت اطلاعات در تصحیح خطا و جمله‌های وابسته به پارامترهای امنیتی بستگی دارد. به‌صورت مفهومی، بیشینهٔ طول کلید قابل استخراج را می‌توان با رابطه‌ای از شکل زیر کران‌بندی کرد:

$$\ell \lesssim s_1^L [1 - h_2(e_1^U)] - \lambda_{EC} - \Delta(\epsilon),$$

در این رابطه، $h_2(\cdot)$ تابع آنتروپی دودویی، s_1^L کران پایین سهم تک‌فوتونی‌ها، e_1^U کران بالای نرخ خطای متناظر، λ_{EC} نشت اطلاعات در مرحلهٔ تصحیح خطا، و $\Delta(\epsilon)$ جمله‌ای وابسته به بودجهٔ امنیتی است. شکل دقیق این رابطه به اثبات امنیتی و فروض مدل وابسته است.

بنابراین، مطالعه و توسعهٔ سامانه‌های QKD تنها به شناخت اصول بنیادی فیزیک کوانتومی محدود نمی‌شود. چنین مطالعه‌ای مستلزم درک تعامل میان منبع نوری، کانال انتقال، نویزها، آشکارسازها، فرایندهای آماری، اثبات‌های امنیتی و مراحل پس‌پردازش کلاسیک است. پروژه‌ای که در این گزارش معرفی می‌شود، در همین نقطه قرار دارد: طراحی و پیاده‌سازی یک بستر شبیه‌سازی که بتواند فاصلهٔ میان مدل‌های نظری و رفتار عملی سامانه‌های QKD را به‌صورت کنترل‌شده و قابل‌اعتبارسنجی مطالعه کند و امکان تحلیل، اعتبارسنجی، مقایسه و توسعهٔ سناریوهای پیشرفته را فراهم کند.

۳.۱ ضرورت شبیه‌سازی در سامانه‌های QKD

هرچند اصول بنیادی QKD در چارچوب نظری به‌خوبی شناخته شده‌اند، گذار از اثبات امنیت روی کاغذ به یک سامانهٔ عملی و قابل‌اتکا، فرایندی پیچیده و چندلایه است. در یک پیاده‌سازی واقعی، هیچ‌یک از اجزای فیزیکی رفتار کاملاً ایده‌آل ندارند و همین انحراف از ایده‌آل، هم بر عملکرد و هم بر امنیت نهایی اثر می‌گذارد. در نتیجه، طراحی، تحلیل و بهینه‌سازی سامانه‌های QKD بدون اتکا به ابزارهای شبیه‌سازی دقیق، در بسیاری از موارد ناقص خواهد بود.

در لایهٔ منبع، پالس‌های نوری می‌توانند با نوساناتی در شدت، فاز، زمان‌بندی و حتی توزیع آماری تعداد فوتون همراه باشند. در عمل، منبع ممکن است دچار جیتر شدت یا intensity jitter شود، پالس‌ها از نظر شکل زمانی یکنواخت نباشند، یا در برخی شرایط از مدل ایده‌آل فاصله بگیرند. این انحرافات می‌توانند مستقیماً بر تخمین پارامترهای امنیتی و همچنین بر نرخ کلید مؤثر اثر بگذارند. از سوی دیگر، کانال انتقال، به‌ویژه در بستر فیبر نوری، با تضعیف، پراکندگی، وابستگی طول‌موجی، ناپایداری‌های محیطی و نویز پس‌زمینه همراه است. در فواصل بلند، کاهش بازده انتقال می‌تواند بر نرخ آشکارسازی و در نتیجه بر امکان‌پذیری تولید کلید امن اثر قابل‌ملاحظه‌ای بگذارد.

در سمت گیرنده نیز وضعیت به همین اندازه پیچیده است. آشکارسازهای تک‌فوتونی، به‌خصوص آشکارسازهای مبتنی بر SPAD، دارای رفتارهایی هستند که در تحلیل ایده‌آل معمولاً نادیده گرفته می‌شوند اما در عمل نقش تعیین‌کننده دارند. از جملهٔ این رفتارها می‌توان به شمارش تاریک یا dark count، زمان مرده یا dead time، پس‌تپش یا afterpulsing، محدودیت‌های گیتینگ زمانی، اشباع‌شدگی در نرخ‌های بالا، و سیاست‌های متفاوت در برخورد با رخداد‌های هم‌زمان آشکارسازها یا double clicks اشاره کرد.

هر یک از این عوامل می‌تواند هم بر نرخ خطای کوانتومی یا QBER و هم بر نرخ کلید نهایی اثر بگذارد. حتی در برخی موارد، عدم مدل‌سازی دقیق همین جزئیات موجب می‌شود نتیجه شبیه‌سازی به‌ظاهر امیدوارکننده باشد، در حالی که سامانه واقعی آن عملکرد را محقق نکند. بنابراین، شبیه‌سازی زمانی ارزشمند است که بتواند اثر این جزئیات را با دقت مناسب و در چارچوب فروض روشن وارد تحلیل کند.

اهمیت موضوع زمانی بیشتر روشن می‌شود که بدانیم بسیاری از تصمیمات طراحی در QKD به‌شدت پارامترمحور هستند. انتخاب شدت‌های سیگنال و فریب، احتمال استفاده از هر شدت، نرخ تکرار پالس، بازه زمانی گیت، مدل تلفات کانال، بازده آشکارسازی، و سیاست پردازش داده در لایه کلاسیک، همگی بر خروجی نهایی اثر می‌گذارند. از آنجا که این پارامترها معمولاً به‌صورت کوپل‌شده عمل می‌کنند، فهم اثر مستقل هر یک بدون وجود یک محیط شبیه‌سازی کنترل‌شده دشوار است. برای مثال، افزایش شدت سیگنال ممکن است نرخ آشکارسازی را بالا ببرد، اما هم‌زمان احتمال رخداد پالس‌های چندفوتونی را نیز افزایش دهد. از سوی دیگر، کاهش بیش از حد شدت، اگرچه آسیب‌پذیری در برابر حملات مرتبط با چندفوتونی‌ها را کاهش می‌دهد، ممکن است به افت شدید نرخ آشکارسازی و غلبه نویز تاریک منجر شود. چنین مصالحه‌هایی تنها از طریق تحلیل کمی و تکرارپذیر قابل ارزیابی‌اند.

از منظر پژوهشی و مهندسی، آزمایش‌های سخت‌افزاری QKD معمولاً پرهزینه، زمان‌بر، حساس به شرایط محیطی، و محدود از نظر قابلیت پیمایش فضای پارامتر هستند. ساخت یا پیکربندی مجدد یک آزمایش برای هر مجموعه جدید از پارامترها، نه‌تنها هزینه زمانی و مالی دارد، بلکه در بسیاری از موارد امکان اجرای گسترده سناریوهای مقایسه‌ای را محدود می‌کند. در مقابل، یک شبیه‌ساز می‌تواند تعداد زیادی سناریو را در یک محیط کنترل‌شده بررسی کند، نقاط بحرانی عملکرد را بیابد، فرضیه‌ها را پیش از پیاده‌سازی سخت‌افزاری آزمایش کند، و مسیر طراحی آزمایش واقعی را هدایت نماید. بنابراین، در بسیاری از مطالعات عملی QKD، شبیه‌سازی نه‌تنها ابزاری کمکی، بلکه بخشی از فرایند طراحی، اعتبارسنجی و بهینه‌سازی سامانه محسوب می‌شود.

یک شبیه‌ساز مفید برای QKD نباید تنها به محاسبه روابط بسته و فرمول‌های ساده مجانبی محدود شود. بسیاری از تحلیل‌های نظری در ساده‌ترین حالت، پارامترهایی مانند بهره آشکارسازی، نرخ خطا یا نرخ کلید را بر اساس روابط تحلیلی در شرایط ایده‌آل یا نیمه‌واقعی برآورد می‌کنند. این روابط برای فهم شهودی و اعتبارسنجی اولیه ارزشمندند، اما برای مدل‌سازی رفتار واقعی سامانه، به‌خصوص در مقیاس کلید محدود، کافی نیستند. در عمل، رویدادها ماهیتاً گسسته و آماری‌اند: فوتون یا به آشکارساز می‌رسد یا نمی‌رسد، کلیک رخ می‌دهد یا رخ نمی‌دهد، پالس ممکن است به‌دلیل زمان مرده از دست برود، و نوسانات تصادفی می‌توانند بر تخمین پارامترها اثر بگذارند.

بنابراین، شبیه‌سازی رویدادمحور و مبتنی بر مونت‌کارلو ابزاری مهم برای بازنمایی رخدادهای گسسته، وابستگی‌های زمانی و نوسانات آماری است. با این حال، نتایج چنین شبیه‌سازی‌ای باید در سناریوهای مناسب با روابط تحلیلی یا بنچمارک‌های مستقل مقایسه شوند تا هزینه محاسباتی بیشتر آن به افزایش قابل‌سنجش در دقت مدل منجر شود.

علاوه بر این، در یک سامانه پژوهشی، صرف شبیه‌سازی لایه کوانتومی کافی نیست. پس از مرحله آشکارسازی، مجموعه‌ای از پردازش‌های کلاسیک شامل غربال‌سازی پایه‌ها، نمونه‌برداری برای برآورد خطا، تخمین پارامترهای حالت فریب، مدل‌سازی هزینه تصحیح خطا، تخصیص بودجه‌های امنیتی، و محاسبه طول یا نرخ کلید امن انجام می‌شود. اگر این مراحل با دقت آماری کافی مدل نشوند، نتیجه شبیه‌سازی از منظر امنیتی قابل‌استناد نخواهد بود. به بیان دیگر، شبیه‌ساز مطلوب باید میان لایه فیزیکی سامانه و لایه نظریه اطلاعات ارتباط برقرار کند.

در نهایت، شبیه‌سازی دقیق امکان می‌دهد مدل‌های فیزیکی مختلف، اثبات‌های امنیتی متفاوت، یا معماری‌های اجرایی متنوع در یک بستر واحد با یکدیگر مقایسه شوند. چنین قابلیت‌هایی برای پژوهش‌هایی که هدف آن‌ها توسعه مدل جدید، اعتبارسنجی یک تقریب فیزیکی، سنجش اثر یک پارامتر، یا مقایسه چند روش تحلیل امنیت است، اهمیت زیادی دارد. از همین رو، توسعه یک بستر شبیه‌سازی توسعه‌پذیر و اعتبارسنجی‌پذیر، نه‌تنها برای پاسخ‌گویی به نیازهای این پروژه، بلکه به‌عنوان زیرساختی برای مطالعات بعدی نیز ارزشمند است.

۴.۱ بیان مسئله و هدف پروژه

مسئله اصلی این پروژه، طراحی و پیاده‌سازی یک فریم‌ورک شبیه‌سازی رویدادمحور برای تحلیل سامانه‌های DV-QKD است که بتواند مدل‌سازی فیزیکی، رفتار آماری، محدودیت‌های سخت‌افزاری، تحلیل‌های امنیتی و بخشی از نیازهای معماری شبکه‌ای را در یک بستر نرم‌افزاری واحد ترکیب کند. تمرکز نسخه فعلی فریم‌ورک بر خانواده سامانه‌های مبتنی بر BB84، منابع مبتنی بر پالس‌های هم‌دوس تضعیف‌شده، روش حالت‌های فریب و تحلیل‌های مجانبی و کلید محدود است. با این حال، معماری آن به‌گونه‌ای طراحی شده است که افزودن مدل‌ها و سناریوهای جدید در آینده با کمترین وابستگی غیرضروری به بخش‌های موجود انجام شود.

در این گزارش، منظور از فریم‌ورک، مجموعه‌ای از ماژول‌های نرم‌افزاری، رابط‌های برنامه‌نویسی، ابزارهای اجرا، و اسکریپت‌های اعتبارسنجی است که به صورت یکپارچه برای تعریف، اجرا و تحلیل سناریوهای QKD به کار می‌روند. هدف این فریم‌ورک تنها محاسبه چند خروجی نهایی نیست، بلکه بازنمایی مرحله به مرحله زنجیره تولید پالس، عبور از کانال، اثر نویزها، آشکارسازی، تجمیع آمار، تخمین پارامترها و محاسبه کمیت‌های امنیتی است. هدف‌های اصلی پروژه را می‌توان به صورت زیر خلاصه کرد:

۱. فراهم کردن محیطی برای مدل‌سازی پدیده‌های فیزیکی مؤثر بر عملکرد سامانه‌های QKD، از جمله منبع، کانال، نویز و آشکارساز؛
۲. ایجاد امکان مقایسه چند خانواده از تحلیل‌های امنیتی تحت شرایط فیزیکی و آماری مشترک؛
۳. پشتیبانی از تحلیل‌های کلید محدود، در کنار تحلیل‌های جانبی، برای بررسی اثر نوسانات آماری و بودجه‌های امنیتی؛
۴. فراهم کردن زیرساخت لازم برای اجرای شبیه‌سازی‌های رویدادمحور در مقیاس بزرگ و با قابلیت پردازش دسته‌ای یا موازی؛
۵. فراهم‌سازی معماری نرم‌افزاری ماژولار برای افزودن مدل‌های جدید منبع، کانال، آشکارساز، حمله یا اثبات امنیت؛
۶. ایجاد قابلیت اولیه برای مطالعه سناریوهای چندگره‌ای و شبکه‌ای مبتنی بر لینک‌های QKD.

در نسخه فعلی، تصحیح خطا و فشرده‌سازی محرمانگی در سطح اثر آن‌ها بر طول یا نرخ کلید نهایی مدل می‌شوند. به بیان دیگر، تمرکز اصلی فریم‌ورک بر محاسبه کمیت‌های امنیتی و هزینه‌های اطلاعاتی متناظر با مراحل پس‌پردازش است، نه الزاماً اجرای کامل یک پروتکل عملی تصحیح خطا یا اعمال فیزیکی یک تابع هش جهانی برای تولید بیت‌های نهایی کلید. جزئیات دقیق این مدل‌سازی در فصل‌های مربوط به پس‌پردازش و تحلیل امنیت ارائه می‌شود.

همچنین، اگرچه تمرکز اصلی فریم‌ورک بر تحلیل لینک‌های DV-QKD است، معماری آن به گونه‌ای توسعه یافته که بتوان چندین لینک را در قالب سناریوهای شبکه‌ای مبتنی بر رله مورد اعتماد ترکیب کرد. در چنین معماری‌ای، امنیت انتهایی به اعتماد به گره‌های میانی وابسته است؛ زیرا این گره‌ها در فرایند انتقال یا بازتوزیع کلید به اطلاعات کلیدی دسترسی دارند.

۵.۱ نمای کلی فریم‌ورک شبیه‌سازی

پروژه حاضر یک فریم‌ورک رویدادمحور برای شبیه‌سازی سامانه‌های Discrete-Variable Quantum Key Distribution (DV-QKD) ارائه می‌کند. این فریم‌ورک با هدف ترکیب مدل‌سازی فیزیکی، تحلیل آماری، محاسبه کمیت‌های امنیتی و معماری نرم‌افزاری توسعه‌پذیر طراحی شده است. رویکرد محاسباتی آن بر پایه شبیه‌سازی مونت کارلو و پردازش رویدادهاست؛ به این معنا که پالس‌ها، آشکارسازی‌ها و رخداد‌های مرتبط با آن‌ها به صورت نمونه‌های گسسته تولید و تحلیل می‌شوند.

در این معماری، هر پالس نوری به صورت یک رویداد مستقل با شناسه و ویژگی‌های مشخص مدل می‌شود. ویژگی‌های پالس می‌توانند در لایه‌های مختلف سامانه، از لحظه تولید در منبع نوری تا عبور از کانال، مواجهه با تلفات و نویز، رسیدن یا نرسیدن به آشکارساز، ثبت کلیک، و ورود به مراحل پس‌پردازش کلاسیک، تغییر کنند. این رویکرد امکان می‌دهد نه تنها خروجی نهایی، بلکه مسیر ایجاد آن و سهم هر بخش از سامانه نیز تحلیل شود. زنجیره شبیه‌سازی را می‌توان به صورت مفهومی در چند لایه توصیف کرد:

۱. **لایه منبع:** منبع نوری پالس‌ها را بر اساس مدل آماری انتخاب شده تولید می‌کند. در سناریوهای مبتنی بر منابع همدوس تضعیف شده، تعداد فوتون‌ها معمولاً با توزیع پواسونی و شدت‌های مختلف حالت فریب مدل می‌شود.
۲. **لایه کانال:** کانال انتقال اثر تلفات، طول مسیر، بازده انتقال و در صورت نیاز نویزهای پس‌زمینه را بر پالس‌ها اعمال می‌کند.
۳. **لایه مدل تهدید:** در صورت تعریف سناریوی حمله یا مدل تهدید، اثر آن با فروض مشخص به فرایند شبیه‌سازی افزوده می‌شود.

۴. **لایه آشکارسازی:** زیرسامانه آشکارسازی رفتارهایی مانند احتمال کلیک، شمارش تاریک، زمان مرده، گیتینگ زمانی، رخدادهای همزمان و سیاست رسیدگی به آن‌ها را مدل می‌کند.

۵. **لایه پسپردازش کلاسیک:** داده‌های خام وارد مراحل غربال‌سازی، تجمیع شمارش‌ها، تخمین پارامترهای امنیتی، اعمال مدل تصحیح خطا و محاسبه نرخ یا طول کلید امن می‌شوند.

۶. **لایه شبکه:** در صورت استفاده از سناریوهای چندگره‌ای، خروجی لینک‌ها در سطح شبکه ترکیب می‌شود و مفاهیمی مانند استخر کلید، مسیرهای جایگزین و رمزنگاری hop-by-hop قابل بررسی می‌شوند.

طراحی نرم‌افزاری فریم‌ورک بر چند اصل استوار است. نخست، منطق مربوط به پارامترها، منبع، کانال، آشکارساز، تحلیل امنیت، بهینه‌سازی و شبکه تا حد امکان به صورت ماژول‌های مستقل سازمان‌دهی شده است. این تفکیک باعث می‌شود تغییر یا جایگزینی یک مدل، کمترین اثر را بر سایر بخش‌ها داشته باشد. دوم، هویت و ویژگی‌های رویدادها در طول پردازش حفظ می‌شود تا تحلیل خطا، اشکال‌زدایی و اعتبارسنجی ممکن باشد. سوم، رابط‌های نرم‌افزاری به گونه‌ای طراحی شده‌اند که افزودن مدل‌های جدید بدون بازنویسی کل سیستم انجام شود. چهارم، خروجی‌ها باید بتوانند در سناریوهای ساده‌تر با روابط تحلیلی شناخته شده یا بنچمارک‌های مستقل مقایسه شوند.

با توجه به حجم بالای رویدادها در شبیه‌سازی‌های بزرگ، فریم‌ورک از پردازش دسته‌ای و سازوکارهای توزیع بار محاسباتی نیز پشتیبانی می‌کند. در چنین حالتی، حفظ یکپارچگی شناسه پالس‌ها، همگام‌سازی نتایج دسته‌ای و مدیریت رویدادهایی که اثرشان از یک دسته پردازشی به دسته بعد منتقل می‌شود، اهمیت زیادی دارد. ارزیابی کمی کارایی، شامل زمان اجرا، مصرف حافظه و رفتار مقیاس‌پذیری، در فصل مربوط به نتایج و اعتبارسنجی ارائه می‌شود.

در بخش تحلیل امنیت، فریم‌ورک امکان مقایسه چند خانواده از روش‌های محاسبه نرخ کلید را فراهم می‌کند؛ از جمله تحلیل‌های جانبی کلاسیک برای Decoy-State BB84، تحلیل‌های کلید محدود مبتنی بر کران‌های آماری، و نسخه‌هایی که با هدف ارائه کران‌های عددی تنگ‌تر توسعه یافته‌اند. هدف از قرار دادن این روش‌ها در یک بستر مشترک آن است که اثر تفاوت اثبات امنیتی از اثر تفاوت مدل فیزیکی یا فروض نرم‌افزاری تا حد امکان جدا شود.

بهینه‌سازی پارامترها نیز یکی از قابلیت‌های مهم فریم‌ورک است. انتخاب شدت سیگنال μ ، شدت فریب ν ، احتمال استفاده از هر شدت، و برخی تنظیمات کانال یا آشکارساز، اثر چشمگیری بر عملکرد سامانه دارد. از این رو، فریم‌ورک می‌تواند از روش‌های عددی و حلگرهای بهینه‌سازی برای پیمایش فضای پارامتر و شناسایی پیکربندی‌های مناسب‌تر استفاده کند. در مسائل خطی یا مراحل کران‌بندی مرتبط با تخمین پارامترها نیز استفاده از ابزارهای برنامه‌ریزی خطی قابل بررسی است.

در مجموع، این فریم‌ورک را می‌توان یک زیرساخت پژوهشی برای مطالعه سامانه‌های QKD دانست که با هدف توسعه تدریجی، مقایسه روش‌ها، تولید نتایج تکرارپذیر و پشتیبانی از مطالعات پیچیده‌تر در حوزه QKD و شبکه‌های کوانتومی طراحی شده است.

۶.۱ مشارکت‌ها و قابلیت‌های اصلی فریم‌ورک

مشارکت‌های اصلی این پروژه را می‌توان در چند محور خلاصه کرد:

۱. **شبیه‌سازی رویدادمحور لینک QKD:** توسعه یک زنجیره شبیه‌سازی که پالس‌ها، کانال، نویز، آشکارسازی و پسپردازش را به صورت مرحله‌به‌مرحله مدل می‌کند.

۲. **پشتیبانی از منابع همدوس تضعیف‌شده و حالت‌های فریب:** مدل‌سازی منابع مبتنی بر WCP و استفاده از شدت‌های مختلف برای تخمین آماری کمیت‌های تک‌فوتونی.

۳. **در نظر گرفتن محدودیت‌های سخت‌افزاری:** وارد کردن عواملی مانند تلفات کانال، شمارش تاریک، زمان مرده، گیتینگ و رخدادهای همزمان آشکارسازها در تحلیل عملکرد.

۴. **ترکیب تحلیل جانبی و کلید محدود:** فراهم کردن امکان بررسی سامانه در دو رژیم جانبی و کلید محدود، با توجه به اثر نوسانات آماری و بودجه امنیتی.

۵. **معماری ماژولار و توسعه‌پذیر:** تفکیک لایه‌های منبع، کانال، نویز، آشکارساز، اثبات امنیت، بهینه‌سازی و شبکه برای تسهیل نگهداری و افزودن مدل‌های جدید.

۶. **پشتیبانی از اعتبارسنجی داخلی:** استفاده از آزمون‌ها و سناریوهای کنترلی برای بررسی سازگاری روابط فیزیکی، پایداری خروجی‌ها و اتصال صحیح لایه‌های نرم‌افزاری.

۷. **زیرساخت اولیه برای سناریوهای شبکه‌ای:** فراهم کردن امکان مطالعه چند لینک و چند گره در قالب معماری‌های مبتنی بر رله مورد اعتماد.

این موارد دامنه اصلی پروژه را مشخص می‌کنند و مرز میان هدف پژوهشی، مدل‌سازی فیزیکی و پیاده‌سازی نرم‌افزاری را روشن می‌سازند. جزئیات هر یک از این محورها در فصل‌های بعدی گزارش ارائه خواهد شد.

۷.۱ ساختار نرم‌افزاری و اجرای پروژه

ساختار نرم‌افزاری پروژه بر پایه تفکیک مسئولیت‌ها طراحی شده است. در سطح مفهومی، ماژول‌های اصلی فریم‌ورک شامل مدیریت پارامترها، تعریف ساختارهای داده، مدل‌سازی منبع، کانال و نویز، مدل‌سازی آشکارساز، تحلیل امنیت، بهینه‌سازی، اجرای سناریوها و آزمون‌های اعتبارسنجی هستند. این تفکیک باعث می‌شود که هر بخش از سامانه بتواند مستقل‌تر توسعه یابد و اثر تغییرات آن بر سایر بخش‌ها کنترل شود.

در ریشه پروژه، فایل‌های اجرایی و اسکریپت‌های هماهنگ‌کننده قرار دارند. این فایل‌ها نقش نقطه ورود برای اجرای سناریوهای اصلی، اجرای پردازش‌های دسته‌ای یا موازی، اعتبارسنجی روابط فیزیکی و اجرای نمونه‌سناریوهای شبکه‌ای را بر عهده دارند. جزئیات نام فایل‌ها، وابستگی‌های دقیق آن‌ها و نحوه اجرای هر سناریو در فصل پیاده‌سازی یا پیوست راهنمای اجرا ارائه می‌شود.

هسته فریم‌ورک در قالب مجموعه‌ای از ماژول‌ها سازمان‌دهی شده است. ماژول مدیریت پارامترها، تنظیمات فیزیکی، امنیتی و اجرایی سناریو را نگهداری می‌کند. ماژول‌های داده‌ای، ساختارهای لازم برای انتقال شمارش‌ها، رویدادها و نتایج میانی میان لایه‌ها را تعریف می‌کنند. ماژول‌های فیزیکی، روابط مربوط به تلفات، تولید فوتون، نویز و آشکارسازی را پیاده‌سازی می‌کنند. لایه اثبات‌های امنیتی نیز از داده‌های خروجی لایه فیزیکی استفاده می‌کند تا کمیت‌هایی مانند نرخ کلید، طول کلید، کران‌های آماری و اثر هزینه‌های پس‌پردازش را محاسبه کند.

برای کنترل صحت توسعه، مجموعه‌ای از آزمون‌های واحد، آزمون‌های یکپارچه‌سازی و در صورت نیاز آزمون‌های رگرسیون در پروژه در نظر گرفته شده است. آزمون‌های واحد برای سنجش رفتار توابع یا کلاس‌های کوچک و مستقل به کار می‌روند؛ برای مثال، بررسی صحت محاسبه تلفات یا رفتار یک ساختار داده در شرایط مرزی. آزمون‌های یکپارچه‌سازی تعامل میان ماژول‌ها را بررسی می‌کنند؛ برای نمونه، سازگاری خروجی آشکارساز با ورودی تحلیل امنیت، یا حفظ هویت رویدادها در پردازش دسته‌ای. آزمون‌های رگرسیون نیز برای جلوگیری از بازگشت خطاهای قدیمی پس از اعمال تغییرات جدید مفیدند.

از نظر اجرایی، پیاده‌سازی اصلی پروژه با زبان پایتون انجام شده است. انتخاب پایتون به دلیل اکوسیستم علمی غنی آن، سرعت بالای توسعه، سهولت اعتبارسنجی پژوهشی و دسترسی به کتابخانه‌های عددی و آماری انجام شده است. کتابخانه‌هایی مانند NumPy، SciPy، pandas و pytest در بخش‌های مختلف پروژه برای محاسبات عددی، بهینه‌سازی، مدیریت داده‌های خروجی و اجرای آزمون‌ها مورد استفاده قرار می‌گیرند. بسته به تنظیمات پروژه و ویژگی‌های فعال‌شده، ممکن است وابستگی‌های کمکی دیگری نیز برای ثبت گزارش، ترسیم نمودار یا مدیریت داده‌ها استفاده شوند.

برای اجرای یک شبیه‌سازی پایه، کاربر معمولاً پارامترهای سناریو را در پیکربندی مربوط تنظیم کرده و سپس نقطه ورود مناسب را اجرا می‌کند. در طی اجرا، سیستم پارامترها را بارگذاری کرده، اجزای لازم را نمونه‌سازی می‌کند، رویدادها را طبق سناریوی مشخص تولید و پردازش می‌نماید، و در نهایت خروجی‌ها را در قالب فایل‌های ساخت‌یافته ذخیره می‌کند. در طراحی خروجی‌ها، پایداری داده‌ها و کاهش خطر خرابی فایل در اجراهای طولانی‌مدت یا دسته‌ای مورد توجه قرار گرفته است.

جزئیات کامل ساختار فایل‌ها، نام اسکریپت‌های اجرایی، وابستگی‌های نرم‌افزاری، نحوه نصب، روش اجرای سناریوهای پایه و پیشرفته، و شیوه بازتولید نتایج در فصل پیاده‌سازی و پیوست راهنمای اجرا ارائه خواهد شد. به این ترتیب، فصل حاضر از ورود به جزئیات پایین سطح نرم‌افزاری پرهیز می‌کند و تمرکز خود را بر معرفی مسئله، دامنه پروژه و معماری مفهومی فریم‌ورک نگه می‌دارد.

۸.۱ ساختار گزارش

ادامه این گزارش به گونه‌ای سازمان‌دهی شده است که خواننده ابتدا با مبانی نظری و سپس با مدل‌سازی، پیاده‌سازی، اعتبارسنجی و نتایج پروژه آشنا شود. به طور کلی، ساختار فصل‌های بعدی به شرح زیر است:

- در فصل مبانی نظری، اصول QKD، پروتکل BB84، منابع همدوس تضعیف‌شده، روش حالت‌های فریب، تحلیل مجانبی و تحلیل کلید محدود با جزئیات بیشتری بررسی می‌شوند.
- در فصل مدل فیزیکی و آماری، مدل‌های مورد استفاده برای منبع، کانال، نویز، آشکارساز و رخداد‌های رویدادمحور معرفی می‌شوند.
- در فصل تحلیل امنیت، روش‌های محاسبه نرخ یا طول کلید امن، کران‌بندی پارامترها، اثر نوسانات آماری و نقش بودجه امنیتی توضیح داده می‌شوند.
- در فصل معماری و پیاده‌سازی نرم‌افزار، ساختار ماژول‌ها، ارتباط میان لایه‌ها، نحوه مدیریت داده‌ها، پردازش دسته‌ای و جزئیات اجرایی فریم‌ورک ارائه می‌شود.
- در فصل اعتبارسنجی و نتایج، خروجی‌های شبیه‌سازی با روابط تحلیلی یا سناریوهای مرجع مقایسه شده و اثر پارامترهای مهم بر عملکرد سامانه بررسی می‌شود.
- در صورت ارائه سناریوهای شبکه‌ای، فصل مربوط به شبکه نحوه ترکیب لینک‌های QKD، مدیریت استخر کلید، مسیرهای جایگزین و معماری مبتنی بر رله مورد اعتماد را بررسی می‌کند.
- در پیوست‌ها، جزئیات نصب، اجرای کد، ساختار فایل‌ها، پیکربندی‌ها و اطلاعات تکمیلی لازم برای بازتولید نتایج آورده می‌شود.

در نتیجه، این فصل نقش نقشه راه گزارش را ایفا می‌کند. ابتدا زمینه نظری و انگیزه پژوهش را توضیح می‌دهد، سپس چالش‌های گذار از مدل ایده‌آل به سامانه عملی را روشن می‌سازد، مسئله و دامنه پروژه را مشخص می‌کند، نمایی کلی از فریم‌ورک ارائه می‌دهد، و در نهایت خواننده را برای ورود به فصل‌های تخصصی‌تر آماده می‌کند.

فصل ۲

پیکربندی هسته و پارامترها

۱.۲ نمای کلی و هدف ماژول `params.py`

ماژول `params.py` به عنوان لایه مرکزی مدیریت و اعتبارسنجی پیکربندی در فریم‌ورک شبیه‌سازی توزیع کلید کوانتومی (Quantum Key Distribution) عمل می‌کند. این ماژول یک لایه انتزاعی یکپارچه با محدودیت تغییرپذیری برای نگهداری پارامترهای ورودی یک اجرای شبیه‌سازی فراهم می‌سازد؛ از مشخصات منبع نوری و مدل کانال فیبر نوری تا ساختار آشکارساز، نوع پروتکل، پارامترهای امنیتی ترکیب‌پذیر (composable security) و مدل‌های فیزیکی اختلال‌های خطی و غیرخطی در سیستم‌های چندکاناله. هدف بنیادین این ماژول آن است که پارامترهای اصلی پیکربندی در یک شیء مشخص و اعتبارسنجی‌شده متمرکز شوند و سازگاری آن‌ها با فرض‌های مدل بررسی شود تا از پخش شدن خطاهای پیکربندی در سراسر پایگاه کد جلوگیری به عمل آید.

در معماری کلی فریم‌ورک، خروجی این ماژول یک نمونه از کلاس `QKDParams` است که به صورت یک قرارداد داده‌ای نهایی (data contract) برای موتور شبیه‌سازی، ماژول‌های محاسبه نرخ کلید (key rate) (computation) و زیرسیستم‌های گزارش‌گیری عمل می‌کند. هر تغییری در هر یک از اجزای فریم‌ورک – اعم از `Protocol`، `ThresholdDetector`، `FiberChannel`، `OpticalSource` – باید از طریق `QKDParams` یا متد `imn` (`with_channel_rebuild()`) عبور کند تا سازگاری درونی پارامترها حفظ شود. این الگو که گاهی «نقطه واحد حقیقت» (Single Source of Truth) نامیده می‌شود، به کاهش ناهماهنگی بین پارامترهای مرتبط کمک می‌کند و شناسایی زود هنگام ناسازگاری‌ها را ساده‌تر می‌سازد.

این ماژول افزون بر تعریف ساختار داده، چند مسئولیت تکمیلی را نیز بر عهده دارد. نخست، اعتبارسنجی جامع (validation) که در متد `_validate()` متمرکز شده و بازه مجاز بیش از پنجاه پارامتر را بررسی می‌کند. دوم، سریال‌سازی و بازسازی بازگشتی (serialization / deserialization) از طریق `to_serializable_dict()` و `from_dict()` که امکان ذخیره‌سازی، بارگذاری و بازتولید نتایج را فراهم می‌سازد. سوم، مدیریت نسخه‌سازی شما (schema versioning) با متغیر `CURRENT_SCHEMA_VERSION = "9.1"` که سازگاری نسلی پیکربندی‌ها را تضمین می‌کند. چهارم، ارائه توابع کارخان از پیش‌تنظیم‌شده (factory presets) نظیر `load_lim2014_dwdm_params()` و `load_lim2014_dedicated_params()` که پیکربندی مقالات مرجع را با دقت بالا بازتولید می‌کنند و به تکرارپذیری نتایج علمی کمک می‌کنند.

به عنوان یک ماژول کلیدی، `params.py` با بیش از ۱۷۰۰ خط کد و طراحی عمیقاً تایپ‌محور (type-driven design)، مرز واسط میان کاربر نهایی و موتور فیزیکی فریم‌ورک را تشکیل می‌دهد. هر خطای پیکربندی که از این سد عبور کند، می‌تواند در محاسبات پایین‌دستی بر نتایج عددی و تحلیل امنیتی اثر بگذارد؛ از این رو، طراحی این ماژول بر پایه اصل «سریع شکستن» (fail-fast) استوار است و هر پیکربندی ناسازگار پیش از ورود به حلقه شبیه‌سازی رد می‌شود.

۲.۲ مبانی تئوری و فیزیکی در `params.py`

تئوری پایه‌ای این ماژول بر سه ستون اصلی استوار است: تئوری امنیتی ترکیب‌پذیر کوانتومی، فیزیک اتلاف در کانال‌های نوری، و مدل‌های اختلال غیرخطی در سیستم‌های WDM. در ادامه‌ی این بخش هر یک از این ستون‌ها را با جزئیات بسط می‌دهیم.

۱.۲.۲ امنیت ترکیب‌پذیر و پارامترهای اِسیلون

در چارچوب امنیتی ترکیب‌پذیر که توسط Renner و König توسعه یافته، امنیت یک پروتکل QKD با یک پارامتر کلی ε مشخص می‌شود که احتمال انحراف خروجی پروتکل از توزیع ایده‌آل را محدود می‌کند. این پارامتر کل به چند مؤلفه‌ی مستقل تجزیه می‌شود:

$$\varepsilon_{\text{tot}} = \varepsilon_{\text{sec}} + \varepsilon_{\text{cor}} + \varepsilon_{\text{PE}} + \varepsilon_{\text{smooth}} + \varepsilon_{\text{PA}} + \varepsilon_{\text{SIF}} \quad (۱.۲)$$

که در آن ε_{sec} امنیت کلی، ε_{cor} صحت، ε_{PE} خطای تخمین پارامتر، $\varepsilon_{\text{smooth}}$ هموارسازی، ε_{PA} تقویت حریم خصوصی و ε_{SIF} فیلتر اطلاعات جانبی را نشان می‌دهد. ماژول `params.py` اعتبار هر یک از این شش پارامتر را به‌صورت جداگانه بررسی می‌کند و سپس شرط ترکیبی را به‌صورت:

$$\sum_i \varepsilon_i < 1.0 \quad (۲.۲)$$

اعمال می‌نماید. این شرط، شرط لازم برای آن است که مفهوم «امنیت محاسباتی کوانتومی» معنای فیزیکی داشته باشد؛ چرا که $\varepsilon \geq 1$ به معنای نبود هرگونه تضمین امنیتی است.

۲.۲.۲ کارایی تصحیح خطا و ضریب f_{EC}

در مرحله‌ی تصحیح خطا، یک پروتکل کلاسیک مانند Cascade یا LDPC برای هم‌سازسازی کلیدهای آلیس و باب به‌کار می‌رود. کارایی این مرحله با ضریب f_{EC} مشخص می‌شود که نسبت اطلاعات نشت‌کرده به حد نظری Shannon را نشان می‌دهد:

$$\text{leak}_{\text{EC}} = f_{\text{EC}} \cdot n \cdot h_2(Q_\mu) \quad (۳.۲)$$

که در آن $h_2(x) = -x \log_2 x - (1-x) \log_2 (1-x)$ آنترپی باینری و Q_μ نرخ خطای کوانتومی (QBER) است. در فریم‌ورک، f_{EC} باید در بازه‌ی $[1.0, 2.0]$ باشد و مقدار پیش‌فرض 1.16 به‌عنوان نمونه‌ای از کارایی متداول در پیاده‌سازی‌های عملی در نظر گرفته می‌شود.

۳.۲.۲ اتلاف کانال و مدل‌های چندجزئی

اتلاف کل کانال در فریم‌ورک از فرمول تجمعی زیر پیروی می‌کند:

$$\alpha_{\text{tot}} = \alpha_{\text{fiber}} \cdot L + \alpha_{\text{filter}} + \alpha_{\text{mod}} + \alpha_{\text{circ}} \quad (۴.۲)$$

که در آن α_{fiber} ضریب اتلاف فیبر بر حسب dB/km، L طول فیبر، α_{filter} اتلاف فیلتر، α_{mod} اتلاف ماژولاتور و α_{circ} اتلاف سیرکولاتور است. اتلاف فیلتر خود به سه مدل تقسیم می‌شود: ideal (بدون اتلاف)، awg (اتلاف ثابت شبکه‌ی آرایه‌ی موج‌بر، معمولاً ۳ dB) و `fbg_serial` (اتلاف تجمعی FBG که با تعداد کانال‌ها رشد می‌کند):

$$\alpha_{\text{filter}}^{\text{fbg}} = N_{\text{ch}} \cdot \alpha_{\text{per-ch FBG}} \quad (۵.۲)$$

۴.۲.۲ اختلال‌های غیرخطی WDM: پراکندگی رامان

در حالت هم‌زیستی کانال‌های کوانتومی و کلاسیک WDM، پدیده‌ی پراکندگی رامان خودبه‌خودی (Spontaneous Raman Scattering) منبع اصلی نویز پس‌زمینه در کانال کوانتومی است. مدل فریم‌ورک این نویز را به‌صورت زیر محاسبه می‌کند:

$$P_{\text{Raman}} = P_{\text{launch}} \cdot (N_{\text{WDM}} - 1) \cdot \gamma_{\text{Raman}} \cdot L_{\text{eff}} \quad (۶.۲)$$

که در آن طول مؤثر غیرخطی L_{eff} برابر است با:

$$L_{\text{eff}} = \frac{1 - e^{-\alpha L}}{\alpha}, \quad \alpha = 0.2 \times 0.23026 \text{ Np/km} \quad (۷.۲)$$

در تابع کارخانه‌ی `load_lim2014_dwdm_params()`، توان لانچ به‌گونه‌ای محاسبه می‌شود که توان دریافتی در آشکارساز برابر با -34 dBm (حداقل حساسیت گیرنده‌های تجاری DWDM SFP) باشد:

$$P_{\text{launch}}^{\text{dBm}} = -34 + \alpha_{\text{fiber}} \cdot L + \alpha_{\text{AWG}} \quad (۸.۲)$$

۵.۲.۲ فیزیکِ ماژولاسیونِ زیرحامل (SCM)

در پروتکل‌های SCM-QKD، سیگنالِ کوانتومی روی یک زیرحاملِ فرکانسیِ ماژوله می‌شود. پارامتر β (modulation_index) عمقِ ماژولاسیون را کنترل می‌کند و نویزِ محصولاتِ تقاطعیِ مرتبه‌ی دوم (CSO) و مرتبه‌ی سوم (CTB) را تحت تأثیر قرار می‌دهد. مدلِ cso_model بین سه حالت approximate, discrete_exact و combinatorial_uniform انتخاب می‌شود که هر یک مرتبه‌ی دقتِ محاسبات را تعیین می‌کند.

۶.۲.۲ پراکندگیِ جبرانیِ فعال

در کانال‌های طولانی، پراکندگیِ کروماتیکِ پالس‌ها را پهن می‌کند و نرخِ کلیکِ تک‌فوتونی را کاهش می‌دهد. پارامتر D (dispersion_parameter_ps_nm_km) این پراکندگی را با واحد $\text{ps}/(\text{nm} \cdot \text{km})$ مشخص می‌کند و در فیبرِ استاندارد SMF-28 حدود $17 \text{ ps}/(\text{nm} \cdot \text{km})$ در طول‌موجِ 1550 nm است. جبرانِ فعال با پرچم active_dispersion_compensation فعال می‌شود.

۳.۲ تحلیل معماری و ساختار داده‌ها در params.py

۱.۳.۲ کلاس QKDParams: یک دیتاکلاسِ ایستا

مرکزِ معماریِ این ماژول، کلاسِ QKDParams است که با دکوراتورهای زیر تعریف می‌شود:

```

۱ @dataclass(frozen=True, slots=True)
۲ class QKDParams:
۳     protocol: ProtocolFromModule
۴     source: OpticalSource
۵     channel: FiberChannel
۶     detector: ThresholdDetector
۷     num_bits: int
۸     photon_number_cap: int
۹     ...

```

پارامتر frozen=True از انتسابِ مستقیم به فیلدهای نمونه پس از ساخت جلوگیری می‌کند و هر اصلاح باید از طریق `dataclasses.replace()` یا `with_channel_rebuilt()` انجام شود. این محدودیت، ضمنِ تحمیل انضباطِ برنامه‌نویسی، اشتراک‌گذاریِ خواندنیِ نمونه‌ها را ساده‌تر می‌کند؛ اما به‌تنهایی تضمین‌کننده‌ی ایمنی کامل در محیط‌های همزمان نیست. دکوراتورِ slots=True که در پایتون ۳.۱۰ معرفی شده، می‌تواند با حذفِ `__dict__` سربارِ حافظه را کاهش دهد؛ میزانِ اثر آن بر سرعت و مصرفِ حافظه به نسخه‌ی Python و الگوی استفاده وابسته است.

۲.۳.۲ دسته‌بندیِ فیلدها

فیلدهای QKDParams به سه گروه اصلی تقسیم می‌شوند:

۱. **مؤلفه‌های فیزیکی (بدون مقدار پیش‌فرض):** `detector`, `channel`, `source`, `protocol`. این چهار مؤلفه، چهار رکن یک سیستمِ QKD را تشکیل می‌دهند و باید توسط کاربر به‌صراحت ارائه شوند.
۲. **پارامترهای امنیتی و محاسباتی:** `num_workers`, `batch_size`, `photon_number_cap`, `num_bits`. این فیلدها بسته به نقششان در مدل فیزیکی و تحلیل امنیتی، می‌توانند بر خروجی نهایی و نرخ کلید اثر بگذارند.
۳. **پارامترهای فیزیکی پیشرفته (با مقادیر پیش‌فرض):** این گروه شامل بیش از سی فیلد است و پارامترهای WDM, SCM, Lim 2014, Chapman 2018 و Ma 2005 را پوشش می‌دهد. استفاده از مقادیر پیش‌فرض به کاربر مبتدی اجازه می‌دهد بدون درگیری با جزئیات فیزیکی، یک شبیه‌سازی پایه را اجرا کند.

۳.۳.۲ پروتکل‌های تایپی و Protocol

فریم‌ورک از دو پروتکلِ تایپی برای تعریفِ رابط‌ها استفاده می‌کند:


```

1 class SerializableComponent(TypingProtocol):
2     def to_config_dict(self) -> Dict[str, Any]: ...
3
4 class QKDProtocol(SerializableComponent, TypingProtocol):
5     protocol_name: str

```

این پروتکل‌ها قراردادهایی هستند که هر مؤلفه‌ی فیزیکی باید آن‌ها را رعایت کند: متد `to_config_dict()` برای سریال‌سازی و صفت `protocol_name` برای شناسایی. این طراحی امکان بررسی ایستا (static checking) با `mypy` را فراهم می‌سازد و خطاهای نوعی را پیش از اجرا کشف می‌کند.

۴.۳.۲ شمارش‌ها (Enums)

ماژول از چندین شمارش برای محدود کردن مقادیر مجاز استفاده می‌کند: `SecurityProof` با مقادیر `CLOPPER_PEARSON`، `ConfidenceBoundMethod` با روش‌های `PAPER_2009`، `MDI_QKD`، `LIM_2014` و `NORMAL_APPROXIMATION`؛ `DecoderArchitecture` با حالت‌های `LOCAL`، `ENTANGLING` و این شمارش‌ها، انضباط دامنه‌ای را اعمال می‌کنند و از خطاهای عملیاتی جلوگیری می‌کنند.

۵.۳.۲ سیستم نسخه‌سازی شما

ثابت `CURRENT_SCHEMA_VERSION = "9.1"` و تابع کمکی `_parse_schema_version()` نسخه‌ی پیکربندی را به صورت یک چندتایی قابل مقایسه نمایش می‌دهند:

```

1 def _parse_schema_version(v: Any) -> tuple:
2     parts = [int(p) for p in v.split(".")]
3     while len(parts) > 1 and parts[-1] == 0:
4         parts.pop()
5     return tuple(parts) if parts else (0,)

```

این رویکرد تضمین می‌کند که نسخه‌های "1.9"، "1.9.0" و "1.9.0.0" هم‌ارز در نظر گرفته شوند و در عین حال "1.10" بزرگ‌تر از "1.9" باشد (ترتیب عددی، نه لغوی).

۶.۳.۲ محدودیت‌های سریال‌سازی

سه ثابت `MAX_SERIALIZABLE_LIST_LEN = 1000`، `MAX_SERIALIZABLE_DICT_LEN = 1000` و `MAX_SERIALIZABLE_RECURSION_DEPTH = 20` مرزهای امنیتی سریال‌سازی را تعیین می‌کنند. این محدودیت‌ها از حملات انکار سرویس مبتنی بر ساختارهای بازگشتی عمیق یا لیست‌های بسیار بزرگ جلوگیری می‌کنند و سازگار با توصیه‌های OWASP برای رمزگشایی JSON هستند.

۴.۲ پیاده‌سازی ریاضی و الگوریتمی در `params.py`

۱.۴.۲ فرمول واحد اتلاف کل

نقطه‌ی حقیقت واحد برای محاسبه‌ی اتلاف، تابع مستقل زیر است:

```

1 def _compute_total_loss_db_from_scalars(
2     *, distance_km, fiber_loss_db_km, filter_model,
3     filter_awg_loss_db, N_channels,
4     filter_fbg_loss_per_channel_db,
5     modulator_insertion_loss_db,
6     circulator_insertion_loss_db,
7 ) -> float:
8     if filter_model == "awg":
9         filter_loss = filter_awg_loss_db
10    elif filter_model == "fbg_serial":
11        filter_loss = N_channels * filter_fbg_loss_per_channel_db
12    elif filter_model == "ideal":
13        filter_loss = 0.0
14    else:

```

```

۱۵ raise ParameterValidationError(...)
۱۶ return (distance_km * fiber_loss_db_km
۱۷ + filter_loss
۱۸ + modulator_insertion_loss_db
۱۹ + circulator_insertion_loss_db)

```

این تابع به صورت معناداری keyword-only است (نشانه‌ی * تا از اشتباه جایگاهی آرگومان‌ها جلوگیری شود). این تابع توسط سه مسیر مجزا فراخوانی می‌شود: `_compute_total_loss_db()` (متد کلاس)، `from_dict()` و توابع کارخانه‌ی `*load_lim2014`. این الگو از یکپارچگی فرمول اطمینان می‌دهد و خطر انحراف تدریجی (drift) بین مسیرهای مختلف را از بین می‌برد.

۲.۴.۲ اعتبارسنجی امنیت ترکیب‌پذیر

حلقه‌ی اعتبارسنجی پارامترهای اِپسیلون در متد `_validate()` به صورت زیر پیاده‌سازی شده است:

```

۱ for eps_name, eps_val in (("eps_sec", self.eps_sec),
۲ ("eps_cor", self.eps_cor),
۳ ("eps_pe", self.eps_pe),
۴ ("eps_smooth", self.eps_smooth),
۵ ("eps_pa", self.eps_pa),
۶ ("eps_sif", self.eps_sif)):
۷ if not isinstance(eps_val, (int, float)) or isinstance(eps_val, bool):
۸ raise ParameterValidationError(...)
۹ if not math.isfinite(eps_val):
۱۰ raise ParameterValidationError(...)
۱۱ if eps_val <= 0.0:
۱۲ raise ParameterValidationError(...)
۱۳ if eps_val >= 1.0:
۱۴ raise ParameterValidationError(...)
۱۵
۱۶ total_epsilon = (self.eps_sec + self.eps_cor + self.eps_pe
۱۷ + self.eps_smooth + self.eps_pa + self.eps_sif)
۱۸ if not math.isfinite(total_epsilon) or total_epsilon >= 1.0:
۱۹ raise ParameterValidationError(...)

```

نکته‌ی ظریف این پیاده‌سازی، بررسی جداگانه‌ی هر مؤلفه پیش از جمع زدن است. این رویکرد از یک خطای نامحسوس جلوگیری می‌کند: اگر یکی از مؤلفه‌ها NaN باشد، جمع کلی نیز NaN خواهد شد و مقایسه‌ی $\text{NaN} \geq 1.0$ همواره False برمی‌گرداند و در نتیجه یک پارامتر نامعتبر به صورت خاموش عبور می‌کند. بررسی `math.isfinite()` در سطح مؤلفه این حفره را می‌بندد.

۳.۴.۲ الگوریتم تبدیل ایمن شمارش‌ها

تابع `_coerce_enum()` یک الگوریتم چندمرحله‌ای برای تبدیل ورودی‌های متنوع به اعضای شمارش ارائه می‌دهد:

```

۱ def _coerce_enum(enum_cls, value):
۲ if isinstance(value, enum_cls):
۳ return value
۴ if isinstance(value, str):
۵ if enum_cls not in _ENUM_LOOKUP_CACHE:
۶ _ENUM_LOOKUP_CACHE[enum_cls] = _build_enum_lookup(enum_cls)
۷ table = _ENUM_LOOKUP_CACHE[enum_cls]
۸ s = value.strip()
۹ hit = table.get(s) or table.get(s.upper())
۱۰ if hit is not None:
۱۱ return hit
۱۲ if isinstance(value, bool):
۱۳ raise ParameterValidationError(...)
۱۴ try:
۱۵ return enum_cls(value)
۱۶ except (ValueError, TypeError) as e:

```

```

۱۷ candidates = [m.name for m in enum_cls]
۱۸ if isinstance(value, str):
۱۹ matches = difflib.get_close_matches(
۲۰ value.strip().upper(),
۲۱ [c.upper() for c in candidates], n=1)
۲۲ if matches:
۲۳ suggestion = f" Did you mean '{matches[0]}'? "
۲۴ raise ParameterValidationError(...) from e

```

این تابع از یک کش ایستا (`_ENUM_LOOKUP_CACHE`) استفاده می‌کند تا جدول جستجوی نادیده‌گیرِ بزرگی و کوچکیِ حروف تنها یک‌بار برای هر شمارش ساخته شود. علاوه بر این، با استفاده از `difflib.get_close_matches()` پیشنهاد هوشمندانه‌ای به کاربر ارائه می‌دهد؛ مثلاً اگر کاربر "lim2014" را تایپ کند، پیام "Did you mean 'LIM_2014'?" دریافت می‌کند. این تجربه‌ی کاربریِ غنی، در پیکربندی‌های پیچیده‌ی علمی زمان قابل توجهی صرفه‌جویی می‌کند.

۴.۴.۲ سریال‌سازی بازگشتی و حفاظت در برابر نشتِ seed

تابع `_to_serializable()` یک الگوریتم بازگشتی برای تبدیل ساختارهای داده‌ی پیچیده‌ی پایتون به فرمت JSON-compatible است:

```

۱ def _to_serializable(o, _depth=0):
۲     if _depth > MAX_SERIALIZABLE_RECURSION_DEPTH:
۳         raise ParameterValidationError(...)
۴     if (not isinstance(o, type)
۵         and hasattr(o, "to_config_dict")
۶         and callable(o.to_config_dict)):
۷         return _to_serializable(o.to_config_dict(), _depth + 1)
۸     if isinstance(o, np.generic):
۹         return o.item()
۱۰    if isinstance(o, np.ndarray):
۱۱        if o.size > MAX_SERIALIZABLE_LIST_LEN:
۱۲            raise ParameterValidationError(...)
۱۳        return o.tolist()
۱۴    if isinstance(o, Enum):
۱۵        return o.value
۱۶    if dataclasses.is_dataclass(o) and not isinstance(o, type):
۱۷        return {f.name: _to_serializable(getattr(o, f.name), _depth + 1)
۱۸                for f in dataclasses.fields(o)}
۱۹    if isinstance(o, (set, frozenset)):
۲۰        _ordered = sorted(o, key=repr)
۲۱        return [_to_serializable(i, _depth + 1) for i in _ordered]
۲۲    ...

```

این تابع چند نکته‌ی مهم دارد: نخست، محدودیت عمق بازگشت (20) از سرریز پشته جلوگیری می‌کند. دوم، آرایه‌های NumPy به صورت خودکار به لیست تبدیل می‌شوند با بررسی اندازه. سوم، مجموعه‌ها (set) به صورت مرتب سریال‌سازی می‌شوند تا هش خروجی تکرارپذیر باشد و قابل استفاده در سیستم‌های کنترل نسخه باشد. تابع `_redact_seed_recursive()` لایه‌ی دفاعی مضاعفی است که اطمینان می‌دهد هیچ فایل شبیه seed (مانند `seed`, `master_seed`, `random_seed`) نمی‌تواند به خروجی سریال‌سازی راه یابد. این محافظت از نشت تصادفی seed در گزارش‌های آزمایشی – که ممکن است در مخزن عمومی Git منتشر شوند – جلوگیری می‌کند و امنیت عملیاتی شبیه‌سازی‌های مبتنی بر اعداد تصادفی شبه‌تصادفی را حفظ می‌نماید.

۵.۴.۲ متد from_dict: بازسازی سازگار

متد کلاسی `from_dict()` یک پیاده‌سازی طولانی و دقیق از الگوی «مصرف‌کند-سپس‌شناسایی‌ناشناخته» (consume-then-detect-unknowns) است. این متد با استفاده از `_coerce_type()` که هر کلید را از دیکشنری pop می‌کند، در پایان می‌تواند هر کلید باقی‌مانده را به عنوان «ناشناخته» شناسایی کند و خطای دقیقی ارائه دهد:

```

1 d_copy = {k: v for k, v in d_copy.items() if not k.startswith("_")}
2 if d_copy:
3     raise ConfigurationError(
4         f"Unknown parameter keys provided: {sorted(d_copy.keys())}")

```

این رویکرد در تجربه‌ی کاربری پیکربندی‌های علمی بسیار ارزشمند است؛ چرا که اشتباهاتِ عملیاتی نظیر `eps_sec` به جای `eps_secc` بلافاصله شناسایی می‌شوند و از نتایج خاموش نادرست جلوگیری می‌کنند.

۶.۴.۲ متد `with_channel_rebuilt` و حفاظت یکپارچگی اتلاف

هنگامی که کاربر می‌خواهد طول فیبر یا ضرایبِ اتلاف را تغییر دهد، نمی‌تواند به‌سادگی از `dataclasses.replace()` استفاده کند؛ چرا که فیلد `channel.total_loss_db` باید با فیلدهای مبدأ همگام بماند. متد `with_channel_rebuilt()` این مشکل را با محاسبه‌ی پیشینِ اتلافِ جدید و بازسازیِ کانال حل می‌کند:

```

1 def with_channel_rebuilt(self, **changes):
2     if "distance_km" in changes:
3         new_distance_km = float(changes.pop("distance_km"))
4     else:
5         new_distance_km = self.channel.distance_km
6         _new = {f.name: getattr(self, f.name) for f in dataclasses.fields(self)}
7         _new.update(changes)
8         new_total_loss = _compute_total_loss_db_from_scalars(
9             distance_km=new_distance_km,
10            fiber_loss_db_km=_new["fiber_loss_db_km"],
11            ...
12        )
13        new_channel = FiberChannel.from_total_loss(new_distance_km, new_total_loss)
14        return dataclasses.replace(self, channel=new_channel, **changes)

```

علاوه بر این، متد `_validate()` دارای گارد سازگاریِ اتلاف است:

```

1 _expected_total_loss = self._compute_total_loss_db()
2 if abs(self.channel.total_loss_db - _expected_total_loss) > 1e-9 * max(1.0, abs(
3     _expected_total_loss)):
4     raise ParameterValidationError(
5         f"channel.total_loss_db ({self.channel.total_loss_db}) does not match "
6         f"recomputed loss ({_expected_total_loss}). Use "
7         f"QKDParams.with_channel_rebuilt() or from_dict() when changing "
8         f"loss-affecting fields.")

```

این گارد از ناسازگاریِ پنهان بین فیلدها جلوگیری می‌کند: اگر کاربر از طریق `replace()` فیلد `filter_model` را بدون بازسازیِ کانال تغییر دهد، اتلافِ ذخیره‌شده در کانال دیگر با فیلدهای مبدأ تطابق نخواهد داشت و از نادرست شدن تدریجیِ نتایج جلوگیری می‌کند.

۷.۴.۲ توابع کارخانه‌ی Lim 2014

دو تابع `load_lim2014_dwdm_params()` و `load_lim2014_dedicated_params()` پیکربندی‌های مقاله‌ی مرجع (Lim et al. (2014) را بازتولید می‌کنند. در تابع دوم، محاسبه‌ی توانِ لانچ بر اساس فرمول زیر است:

```

1 if wdm_channel_power_dbm is None:
2     _fiber_loss_db = distance_km * params.fiber_loss_db_km
3     wdm_channel_power_dbm = -34.0 + _fiber_loss_db + _avg_filter_loss_db
4     if wdm_channel_power_dbm > 30.0:
5         raise ParameterValidationError(...)

```

این فرمول طوری تنظیم شده است که توان دریافتی در گیرنده‌ی کلاسیک نزدیک به -34 dBm (حساسیتِ گیرنده‌های تجاری) باقی بماند و در نتیجه‌ی مقایسه‌ی عادلانه‌ای بین سناریوهای فاصله‌ی مختلف میسر شود.

۵.۲ ارسال و دریافت با سایر ماژول‌ها در params.py

۱.۵.۲ وابستگی‌های ورودی

ماژول params.py به صورت گسترده‌ای از سایر بخش‌های فریم‌ورک واردات (import) انجام می‌دهد. این وابستگی‌ها را می‌توان در پنج دسته‌ی منطقی قرار داد:

۱. **مؤلفه‌های فیزیکی:** OpticalSource از sources، FiberChannel از channel، ThresholdDetector از detectors. این سه کلاس ساختمان‌های داده‌ای اجزای فیزیکی هستند که QKDParams در خود نگه می‌دارد.

۲. **پروتکل‌ها:** Protocol از protocols. به همراه کلاس‌های BB84DecoyProtocol، MDIQKDProtocol، B92Protocol و RedundantTransmissionProtocol که در متد from_dict() به صورت داخلی وارد می‌شوند.

۳. **نوع‌های داده و شمارش‌ها:** datatypes شامل SecurityProof، DoubleClickPolicy، DecoderArchitecture، SourceStatisticsType، PulseTypeConfig، ConfidenceBoundMethod، PulseEnsembleConfig و SourceErrorModel.

۴. **استثناها:** ConfigurationError و ParameterValidationError از exceptions. که در تمام مسیرهای خطا استفاده می‌شوند.

۵. **ثابت‌ها و ابزارها:** LP_SOLVER_METHODS و DEFAULT_POISSON_TAIL_THRESHOLD از constants، parse_bool از utils.validation، AfterpulseModel از detectors.

۲.۵.۲ خروجی‌ها و نقاط مصرف

خروجی اصلی ماژول، نمونه‌های QKDParams است که توسط چندین مصرف‌کننده‌ی پایین‌دستی استفاده می‌شود:

- **موتور شبیه‌سازی:** از فیلدهای protocol، source، channel، detector و num_bits برای اجرای حلقه‌ی اصلی تولید و اندازه‌گیری استفاده می‌کند.
- **محاسبه‌ی نرخ کلید:** از security_proof، ci_method، f_error_correction، تمامی پارامترهای eps_* و photon_number_cap برای محاسبه‌ی نرخ کلید نهایی استفاده می‌کند.
- **حلقه‌ی بهینه‌سازی:** از auto_optimize_lim2014 و auto_optimize_ma2005 برای تصمیم‌گیری در مورد اجرای بهینه‌سازی پارامترهای منبع استفاده می‌کند.
- **سیستم گزارش‌گیری:** از to_serializable_dict() برای تولید JSON قابل بازتولید استفاده می‌کند.

۳.۵.۲ قرارداد سریال‌سازی

متد to_serializable_dict() یک دیکشنری تودرتو با ساختار زیر تولید می‌کند:

```

1 {
2     "schema_version": "1.9",
3     "protocol_name": "bb84-decoy",
4     "protocol_config": {...},
5     "source_config": {...},
6     "channel_config": {...},
7     "detector_config": {...},
8     "num_bits": ..., "eps_sec": ..., ...
9 }
```

این ساختار با ورودی from_dict() همخوانی دارد و هدف آن حفظ هم‌ارزی رفت و برگشت (round-trip identity) در شرایط متداول است؛ به طوری که بازسازی نمونه از روی دیکشنری سریال‌سازی‌شده، شیئی با ویژگی‌های عملکردی یکسان ارائه می‌دهد. اگر خروجی to_serializable_dict() یک نمونه را به from_dict() بدهیم، نمونه‌ای هم‌ارز اصلی به دست می‌آید. این ویژگی در تست‌های خودکار (test_round_trip_identity) به صورت سیستماتیک بررسی می‌شود.

۶.۲ نمای کلی و هدف ماژول constants_helpers.py

ماژول constants_helpers.py مجموعه‌ای از توابع خالص (pure functions) برای اعتبارسنجی عددی، نرمال‌سازی احتمالات و تبدیل‌های یکانی (unit conversions) را در فریم‌ورک QKD ارائه می‌کند. اگرچه این ماژول از نظر حجمی کوچک به نظر می‌رسد (تنها حدود ۲۱۵ خط)، و به عنوان ابزار کمکی برای بهبود یکنواختی محاسبات عددی و کاهش خطاهای ناشی از مقایسه‌های اعشاری در فریم‌ورک به کار می‌رود. تمامی مقایسه‌های ممیز شناور، تمامی اعتبارسنجی‌های بازه‌ای احتمال و تمامی تبدیل‌های dB-to-linear در این فریم‌ورک از طریق این ماژول عبور می‌کنند و این تمرکز، اطمینان می‌دهد که خطاهای عددی به صورت یکسان در سراسر پایگاه کد شناسایی می‌شوند.

هدف بنیادین این ماژول رفع یک مشکل کلاسیک در محاسبات علمی پایتون است: مقایسه‌های نامعتبر اعداد اعشاری. در ریاضیات ممیز شناور، تساوی دقیق $b == a$ تقریباً همواره نادرست است؛ چرا که خطاهای گردسازی در طول محاسبات تجمع می‌کنند. به عنوان مثال، $2.0 + 1.0 == 3.0$ در پایتون False برمی‌گرداند. این ماژول با ارائه تابع `is_close()` که از تOLERانس‌های مطلق و نسبی استفاده می‌کند، این مشکل را به صورت سیستماتیک حل می‌کند.

علاوه بر این، ماژول مسئولیت حفاظت از یکپارچگی محاسبات آنتروپی را بر عهده دارد. آنتروپی شانون، $H(p) = -\sum p_i \log p_i$ ، در مرزهای $p = 0$ و $p = 1$ دارای رفتار نامطلوب است: در عبارت $p \log 0$ از نظر ریاضی به ۰ میل می‌کند، اما در محاسبات عددی به NaN یا $-\infty$ منجر می‌شود. تابع `clamp_probability()` با محدود کردن احتمالات به بازه $[\varepsilon, 1 - \varepsilon]$ این مشکل را برطرف می‌سازد و امکان ارزیابی پایدار آنتروپی را در طول شبیه‌سازی فراهم می‌کند.

۷.۲ مبانی تئوری و فیزیکی در constants_helpers.py

۱.۷.۲ تحلیل عددی ممیز شناور

استاندارد IEEE 754 که در پایتون از طریق نوع float پیاده‌سازی شده، اعداد اعشاری را با ۵۳ بیت مانتیس (mantissa) ارائه می‌دهد که تقریباً معادل ۱۵ رقم اعشار معنادار است. با این حال، خطای ماشین (machine epsilon) که به صورت $\varepsilon_{\text{mach}} = 2^{-52} \approx 2.22 \times 10^{-16}$ تعریف می‌شود، حد پایین خطاهای نسبی را مشخص می‌کند. در فریم‌ورک QKD، جایی که احتمالات بسیار کوچک (مانند 10^{-10}) با احتمالات بزرگ (نزدیک به ۱) در یک محاسبه ترکیب می‌شوند، درک این محدودیت حیاتی است. برای مقایسه‌ی دو عدد a و b در حضور خطای گردسازی، فرمول استاندارد زیر به کار می‌رود:

$$|a - b| \leq \max(\text{rel_tol} \cdot \max(|a|, |b|), \text{abs_tol}) \quad (9.2)$$

که در آن `rel_tol` تolerانس نسبی و `abs_tol` تolerانس مطلق است. تابع `math.isclose()` پایتون این فرمول را پیاده‌سازی می‌کند و تابع `is_close()` فریم‌ورک با استفاده از تolerانس‌های متمرکز در `constants_definitions.py` این رفتار را بومی‌سازی می‌کند.

۲.۷.۲ آنتروپی شانون و ناپایداری عددی

در محاسبات نرخ کلید QKD، آنتروپی شانون نرخ خطای کوانتومی به کرات محاسبه می‌شود:

$$H_2(Q) = -Q \log_2 Q - (1 - Q) \log_2 (1 - Q) \quad (10.2)$$

این تابع در $Q = 0$ و $Q = 1$ به صورت ریاضی به صورت $\lim_{x \rightarrow 0^+} x \log x = 0$ تعریف می‌شود، اما در محاسبات عددی $\log(0) = -\infty$ و در نتیجه $0 \cdot (-\infty) = \text{NaN}$ تولید می‌شود. راه حل استاندارد، محدود کردن (clamping) احتمالات به بازه $[\delta, 1 - \delta]$ با δ کوچک (مانند 10^{-12}) است:

$$p_{\text{clamped}} = \min(\max(p, \delta), 1 - \delta) \quad (11.2)$$

این رویکرد، خطای ناشی از تغییر δ را در حد $O(\delta \log \delta)$ نگه می‌دارد که برای $\delta = 10^{-12}$ خطی در حد 10^{-11} بیت است — بسیار کوچک‌تر از دقت امنیتی مورد نیاز.

۳.۷.۲ نرمال‌سازی L1 و توزیع احتمال

یک توزیع احتمال گسسته باید شرط $\sum_i p_i = 1$ را برآورده کند. در محاسبات عددی، خطاهای گردسازی می‌توانند باعث شوند که مجموع از ۱ منحرف شود. نرمال‌سازی L1 این مشکل را با تقسیم هر درایه بر مجموع حل می‌کند:

$$p_i^{\text{norm}} = \frac{p_i}{\sum_j p_j} \quad (۱۲.۲)$$

با این حال، اگر $\sum_j p_j = 0$ ، این فرمول به تقسیم بر صفر منجر می‌شود. در این حالت، یک fallback قطعی به یک بردار یک‌داغ (one-hot) که تمام جرم را در اندیس صفر قرار می‌دهد، استفاده می‌شود. این انتخاب، گزینه‌ای قراردادی برای جلوگیری از توقف شبیه‌سازی در مواجهه با بردارهای صفر است؛ هرچند از نظر تئوری اطلاعات، قرار دادن کل جرم روی یک اندیس با مفهوم توزیع حداکثر آنتروپی (یکنواخت) متفاوت است.

۴.۷.۲ تبدیل دسی‌بل به خطی

در مخابرات نوری، اتلاف و تقویت به صورت سنتی با واحد دسی‌بل (dB) بیان می‌شوند:

$$x_{\text{dB}} = 10 \log_{10}(x_{\text{linear}}) \quad (۱۳.۲)$$

تبدیل معکوس، که در فریم‌ورک پیاده‌سازی شده، به صورت زیر است:

$$x_{\text{linear}} = 10^{x_{\text{dB}}/10} \quad (۱۴.۲)$$

این تبدیل به دلیل ماهیت نمایی آن، دارای خطر سرریز (overflow) است: برای $x_{\text{dB}} = 3080$ ، خروجی برابر با 10^{308} است که در نزدیکی حد بالای float64 ($\approx 1.8 \times 10^{308}$) قرار دارد. برای $x_{\text{dB}} = 3100$ ، سرریز به $+\infty$ رخ می‌دهد. تابع `db_to_linear()` با بررسی صریح این حد، از تولید inf جلوگیری می‌کند.

۵.۷.۲ بردار احتمال معتبر

یک بردار احتمال معتبر باید چهار شرط را برآورده کند: یک‌بعدی بودن، مقدار حقیقی متناهی، نامنفی بودن هر درایه (با تلورانس)، و مجموع برابر با ۱ (با تلورانس). این شروط چهارگانه پایه‌ی ریاضی تمام توزیع‌های احتمالی است که در فریم‌ورک استفاده می‌شود – از توزیع پواسون فوتون‌ها تا توزیع پسین (posterior) خطا.

۸.۲ تحلیل معماری و ساختار داده‌ها در `constants_helpers.py`

۱.۸.۲ فهرست توابع عمومی

ماژول با تعریف یک چندتایی Final از نام توابع عمومی آغاز می‌شود:

```
1 PUBLIC_FUNCTION_NAMES: Final[tuple[str, ...]] = (
2     "clamp_probabilities",
3     "clamp_probability",
4     "db_to_linear",
5     "is_close",
6     "is_finite_non_negative",
7     "is_prob_vector",
8     "is_valid_probability",
9     "renormalize_probabilities",
10 )
11 __all__ = list(PUBLIC_FUNCTION_NAMES)
```

این الگو دو مزیت دارد: نخست، فهرست API عمومی به صورت یک منبع واحد نگهداری می‌شود و امکان بازبینی آسان را فراهم می‌سازد. دوم، از آنجا که این چندتایی با Final علامت‌گذاری شده، بررسی‌گرهای نوع ایستا هشدار می‌دهند اگر به اشتباه مقدار آن تغییر کند. ماژول `constants.py` از همین فهرست برای بازصادرات (re-export) توابع استفاده می‌کند.

۲.۸.۲ توابع کمکی خصوصی

دو تابع خصوصی `_is_real_scalar()` و `_to_real_scalar()` لایه‌ی اولیه‌ی فیلترینگ نوع را تشکیل می‌دهند:

```

۱ def _is_real_scalar(value: Any) -> bool:
۲     return isinstance(value, (Real, np.integer, np.floating)) and \
۳     not isinstance(value, (bool, np.bool_))

```

نکته‌ی ظریف این پیاده‌سازی، حذف صریح `bool` از مجموعه‌ی اعداد حقیقی است. در پایتون، `bool` یک زیرکلاس `int` است و در نتیجه `isinstance(True, int)` مقدار `True` برمی‌گرداند. این رفتار می‌تواند منجر به خطاهای خاموش شود: مثلاً `True + True` برابر با ۲ است. در فریم‌ورک QKD، جایی که احتمالات باید اعداد حقیقی کوچک باشند، یک `True` یا `False` به اشتباه به عنوان احتمال می‌تواند منجر به نرخ کلید نادرست شود. حذف صریح `bool` این حفره را می‌بندد.

۳.۸.۲ تابع `_to_real_1d_array()`: اعتبارسنجی آرایه‌ای

این تابع کمکی، ورودی‌های متنوع را به آرایه‌ی یک‌بعدی `float64` تبدیل می‌کند و چندین اعتبارسنجی حیاتی انجام می‌دهد:

```

۱ def _to_real_1d_array(values, *, name, allow_empty):
۲     arr = np.asarray(values)
۳     if arr.dtype == np.bool_:
۴         raise TypeError(f"{name} must contain only real numeric values, not booleans.")
۵     if arr.ndim == 0:
۶         raise ValueError(f"{name} must be a 1D array, but a scalar was provided.")
۷     if arr.ndim != 1:
۸         raise ValueError(f"{name} must be a 1D array, but got {arr.ndim}D input.")
۹     if not allow_empty and arr.size == 0:
۱۰        raise ValueError(f"{name} must not be empty.")
۱۱     if np.iscomplexobj(arr):
۱۲        raise TypeError(f"{name} must contain only real values.")
۱۳     arr = arr.astype(np.float64, copy=False)
۱۴     if not np.all(np.isfinite(arr)):
۱۵        raise ValueError(f"{name} must contain only finite values.")
۱۶     return arr

```

این تابع چندین فیلتر دفاعی دارد: رد کردن `bool`، رد کردن آرایه‌های چندبعدی، رد کردن مقادیر مختلط، و رد کردن `NaN` و `Inf`. این رویکرد چندلایه، تطبیق‌پذیری نوع (`type strictness`) را با پیام‌های خطای دقیق ترکیب می‌کند.

۴.۸.۲ تایپ‌هینت‌های `NDArray` و `ArrayLike`

ماژول از تایپ‌هینت‌های `NumPy` برای بیان دقیق قرارداد توابع استفاده می‌کند:

```

۱ from numpy.typing import ArrayLike, NDArray
۲
۳ def clamp_probabilities(probs: ArrayLike) -> NDArray[np.float64]: ...

```

`ArrayLike` یک نوع «پذیرنده» است که شامل لیست، چندتایی، و آرایه‌های `NumPy` می‌شود — هر چیزی که `np.asarray()` بتواند آن را تبدیل کند. `NDArray[np.float64]` نوع خروجی دقیق را مشخص می‌کند. این ترکیب، هم انعطاف‌پذیری پایتون اردوگانی (`duck-typing`) را حفظ می‌کند و هم دقت تایپ‌هینت را ارائه می‌دهد.

۹.۲ پیاده‌سازی ریاضی و الگوریتمی در `constants_helpers.py`

۱.۹.۲ تابع `is_close()`: مقایسه‌ی ایمن

پیاده‌سازی این تابع ساده اما حیاتی است:


```

۱ def is_close(a, b, *, rel_tol=NUMERIC_REL_TOL, abs_tol=NUMERIC_ABS_TOL):
۲     a_val = _to_real_scalar(a, name="a")
۳     b_val = _to_real_scalar(b, name="b")
۴     return math.isclose(a_val, b_val, rel_tol=rel_tol, abs_tol=abs_tol)

```

با استفاده از `math.isclose()` پایتون (نسخه ۵.۳ به بعد)، این تابع از فرمول استاندارد استفاده می‌کند. تلورانس‌های پیش‌فرض از `constants_definitions.py` وارد می‌شوند ($1e-9 = \text{NUMERIC_REL_TOL}$ و $1e-12 = \text{NUMERIC_ABS_TOL}$) که به‌دقت تنظیم شده‌اند تا هم حساسیت کافی برای محاسبات امنیتی داشته باشند و هم در برابر خطاهای گردسازی مقاوم باشند.

۲.۹.۲ تابع `is_valid_probability()`: اعتبارسنجی بازه‌ی احتمال

```

۱ def is_valid_probability(p: Any) -> bool:
۲     if not _is_real_scalar(p):
۳         return False
۴     p_val = float(p)
۵     return math.isfinite(p_val) and -NUMERIC_ABS_TOL <= p_val <= 1.0 + NUMERIC_ABS_TOL

```

این تابع یک تلورانس کوچک $\varepsilon = 10^{-12}$ در هر دو مرز اضافه می‌کند. این تلورانس به مقادیر نظیر 10^{-15} (که از خطای گردسازی ناشی می‌شوند) اجازه می‌دهد به‌عنوان ۰ معتبر شناخته شوند. این انعطاف در محاسبات بازگشتی، جایی که خطاهای کوچک تجمع می‌کنند، حیاتی است.

۳.۹.۲ تابع `clamp_probability()`: برش احتمال

```

۱ def clamp_probability(p: Real) -> float:
۲     p_val = _to_real_scalar(p, name="p")
۳     return min(max(p_val, ENTROPY_PROB_CLAMP), 1.0 - ENTROPY_PROB_CLAMP)

```

این تابع با فرمول $\min(\max(p, \delta), 1 - \delta)$ عمل می‌کند که در آن $\delta = \text{ENTROPY_PROB_CLAMP}$. این تابع `p` را در بازه $[\delta, 1 - \delta]$ نگه می‌دارد. همان‌طور که پیش‌تر ذکر شد، این برش برای پایداری محاسبات آنتروپی حیاتی است.

۴.۹.۲ تابع `clamp_probabilities()`: نسخه‌ی برداری

نسخه‌ی برداری از تابع قبلی، با استفاده از `np.clip()`:

```

۱ def clamp_probabilities(probs: ArrayLike) -> NDArray[np.float64]:
۲     arr = _to_real_1d_array(probs, name="probs", allow_empty=True)
۳     return np.clip(arr, ENTROPY_PROB_CLAMP, 1.0 - ENTROPY_PROB_CLAMP)

```

پارامتر `allow_empty=True` به این معناست که یک بردار خالی نیز مجاز است و خروجی آن نیز خالی خواهد بود. این رفتار برای توابع پایین‌دستی که ممکن است با لیست‌های خالی سروکار داشته باشند، مفید است.

۵.۹.۲ تابع `renormalize_normalize_probabilities()`: نرمال‌سازی با fallback

این تابع، پیچیده‌ترین تابع ماژول است:

```

۱ def renormalize_probabilities(probs: ArrayLike) -> NDArray[np.float64]:
۲     arr = _to_real_1d_array(probs, name="probs", allow_empty=False)
۳     p_clamped = clamp_probabilities(arr)
۴     total_p = float(np.sum(p_clamped))
۵
۶     if total_p < NUMERIC_ABS_TOL:
۷         warnings.warn(
۸             "renormalize_probabilities: total mass 0 after clamping; "
۹             "falling back to deterministic one-hot at index 0.",
۱۰         RuntimeWarning, stacklevel=2)
۱۱     p_new = np.zeros_like(p_clamped)
۱۲     p_new[0] = 1.0

```

```

۱۳     return p_new
۱۴
۱۵     return p_clamped / total_p

```

این پیاده‌سازی چند نکته‌ی مهم دارد. نخست، ورودی‌های خالی رد می‌شوند (allow_empty=False). دوم، پیش از نرمال‌سازی، احتمالات بریده می‌شوند تا از تقسیم بر صفر ناشی از $0 \log 0$ جلوگیری شود. سوم، اگر کل جرم پس از برش بسیار کوچک باشد، یک هشدار RuntimeWarning صادر می‌شود و یک one-hot برگردانده می‌شود. این fallback قطعی، رفتار تابع را در حضور ورودی‌های نامعتبر قابل‌پیش‌بینی می‌کند — یک ویژگی حیاتی برای سیستم‌های تولیدی.

۶.۹.۲ تابع db_to_linear(): تبدیل ایمن

```

۱     def db_to_linear(val_db: Real) -> float:
۲         val_db_f = _to_real_scalar(val_db, name="val_db")
۳         exponent = val_db_f / 10.0
۴         if exponent > 308:
۵             raise OverflowError(f"db_to_linear({val_db_f}) would overflow to infinity.")
۶         result = math.pow(10.0, exponent)
۷         if result == 0.0 and val_db_f != 0.0:
۸             raise OverflowError(f"db_to_linear({val_db_f}) underflows to zero.")
۹         return result

```

این تابع دو محافظت حیاتی دارد. نخست، اگر $x_{dB}/10 > 308$ ، سرریز 10^{308} به $+\infty$ پیش‌بینی شده و یک OverflowError صادر می‌شود. دوم، در صورتی که ورودی غیرصفر باشد اما خروجی به دلیل محدودیت‌های نمایش ممیز شناور به صفر تقریب داده شود (underflow)، سیستم با صدور خطا یا هشدار مانع از جریان یافتن مقادیر صفر ناخواسته در محاسبات حساس بعدی می‌شود؛ مگر آنکه این رفتار به‌وضوح در مدل فیزیکی تعریف شده باشد. این محافظت‌ها از تولید خاموش ∞ یا 0 در محاسبات پایین‌دستی جلوگیری می‌کنند.

۷.۹.۲ تابع is_prob_vector(): اعتبارسنجی بردار احتمال

```

۱     def is_prob_vector(p: ArrayLike) -> bool:
۲         try:
۳             arr = _to_real_1d_array(p, name="p", allow_empty=False)
۴             except (TypeError, ValueError) as exc:
۵                 msg = str(exc)
۶                 if ("must be a 1D array" in msg
۷                     or "must contain only" in msg
۸                     or "must not be empty" in msg):
۹                     return False
۱0                raise
۱1
۱2         if np.any(arr < -NUMERIC_ABS_TOL):
۱3             return False
۱4         return bool(np.isclose(np.sum(arr), 1.0, atol=PROB_SUM_TOL, rtol=0.0))

```

این تابع به‌جای صدور خطا، False برمی‌گرداند — یک الگوی متفاوت از سایر توابع. این رفتار برای استفاده در شرط‌های if طراحی شده است: کاربر می‌تواند بپرسد «آیا این یک بردار احتمال معتبر است؟» بدون آنکه استثنا رخ دهد. با این حال، خطاهای ساختاری (مانند مختلط بودن) همچنان به‌صورت استثنا ظاهر می‌شوند، چرا که آن‌ها خطاهای برنامه‌نویسی هستند نه خطاهای داده.

۱۰.۲ ارسال و دریافت با سایر ماژول‌ها در constants_helpers.py

۱.۱۰.۲ وابستگی‌های ورودی

این ماژول تنها از دو منبع واردات انجام می‌دهد:

```

1 import math
2 import warnings
3 from numbers import Real
4 from typing import Any, Final
5
6 import numpy as np
7 from numpy.typing import ArrayLike, NDArray
8
9 from .constants_definitions import (
10     ENTROPY_PROB_CLAMP,
11     NUMERIC_ABS_TOL,
12     NUMERIC_REL_TOL,
13     PROB_SUM_TOL,
14 )

```

همچنین یک `assert` در زمان واردات اجرا می‌شود:

```

1 assert 0.0 < ENTROPY_PROB_CLAMP < 0.5, (
2     f"ENTROPY_PROB_CLAMP must be in (0, 0.5), got {ENTROPY_PROB_CLAMP}"
3 )

```

این `assert` یک `contract check` است که در زمان بارگذاری ماژول اجرا می‌شود و اطمینان می‌دهد که مقدار `ENTROPY_PROB_CLAMP` در بازه‌ی منطقی $(0, 0.5)$ قرار دارد. اگر به اشتباه مقدار `0.6` تنظیم شود، کل فریم‌ورک از بارگذاری باز می‌ایستد و خطای صریح صادر می‌شود.

۲.۱۰.۲ نقاط مصرف

توابع این ماژول توسط چندین ماژول دیگر فریم‌ورک استفاده می‌شوند:

- **ماژول منابع نوری** (`sources.py`): برای اعتبارسنجی توزیع احتمال تعداد فوتون و محاسبه‌ی احتمالات نهایی.
- **ماژول آشکارسازها** (`detectors.py`): برای محاسبه‌ی احتمالات کلیک و نرمال‌سازی پس از تطبیق با `dark count`.
- **ماژول کانال** (`channel.py`): برای تبدیل اتلاف `dB` به خطی و محاسبه‌ی نرخ انتقال.
- **ماژول محاسبه‌ی نرخ کلید** (`keyrate.py`): برای محاسبه‌ی آنتروپی و مقایسه‌های عددی.
- **ماژول ثابت‌ها** (`constants.py`): برای بازصادرات عمومی از طریق فساد (`facade`).

۳.۱۰.۲ قرارداد رابط

این ماژول چند قرارداد رابط مهم را رعایت می‌کند:

۱. **خالص بودن** (`purity`): هیچ‌یک از توابع دارای اثر جانبی نیستند. ورودی‌ها تغییر نمی‌کنند و وضعیت سراسری خوانده نمی‌شود (به جز ثابت‌های `Final` که تنها خوانده می‌شوند).
۲. **قطعیت** (`determinism`): برای ورودی یکسان، خروجی همواره یکسان است. تنها استثنا صدور هشدار (`warnings.warn`) در حالت `fallback` است که یک اثر جانبی قابل قبول است.
۳. **اعتبارسنجی پیش از اجرا**: تمام توابع ورودی را پیش از پردازش اصلی اعتبارسنجی می‌کنند. این الگو، `fail-fast` نامیده می‌شود و از اجرای محاسبات نادرست جلوگیری می‌کند.
۴. **پیام‌های خطای دقیق**: تمام خطاها نام پارامتر متناظر را در بر دارند، که اشکال‌زدایی را ساده می‌کند.

۱۱.۲ نمای کلی و هدف ماژول `constants_definitions.py`

ماژول `constants_definitions.py` منبع مرکزی ثابت‌ها و نوع‌های تایپی فریم‌ورک QKD است. این ماژول کوچک (تنها ۶۳ خط) نقش یک `axiomatic foundation` را ایفا می‌کند: تمامی مقادیر عددی مورد استفاده در فریم‌ورک – از تلورانس‌های عددی تا ثابت‌های فیزیکی بنیادین – در اینجا تعریف می‌شوند و در هیچ جای دیگری

از پایگاه کد بازتعریف نمی‌گردند. این تمرکز، امکان تنظیم دقیق و یکنواخت رفتار عددی کل فریم‌ورک را از یک نقطه‌ی واحد فراهم می‌سازد.

هدف بنیادین این ماژول، رفع یک مشکل رایج در پروژه‌های علمی بزرگ است: پخش شدن مقادیر جادویی (magic numbers) در سراسر کد. در غیاب این ماژول، ممکن است مقدار 10^{-9} به عنوان تلورانس نسبی در یک فایل، مقدار 10^{-8} در فایلی دیگر و $1e-10$ در فایلی سوم به کار رود. این ناهماهنگی منجر به رفتار ناسازگار و اشکال‌زدایی دشوار می‌شود. با متمرکز کردن تمام ثابت‌ها در این ماژول، فریم‌ورک رفتاری یکنواخت و قابل توضیح ارائه می‌دهد.

علاوه بر ثابت‌های عددی، این ماژول مسئولیت تعریف نوع‌های تایپی مشترک را نیز بر عهده دارد. LPSolverMethod به عنوان یک Literal Type، گزینه‌های مجاز حلقه‌ی LP را محدود می‌کند و امکان بررسی ایستا را فراهم می‌سازد. فهرست PUBLIC_CONSTANT_NAMES به عنوان یک مرجع مرتب، ترتیب سریال‌سازی و خروجی گزارش‌گیری را پایدار نگه می‌دارد و در نتیجه گزارش‌های تولیدشده توسط نسخه‌های مختلف فریم‌ورک قابل مقایسه باقی می‌مانند.

۱۲.۲ مبانی تئوری و فیزیکی در constants_definitions.py

۱.۱۲.۲ سلسله‌مراتب تلورانس

یکی از مفاهیم کلیدی این ماژول، سلسله‌مراتب تلورانس است که در داک‌استرینگ ماژول به صورت زیر توضیح داده شده است:

$$\text{ENTROPY_PROB_CLAMP} \leq \text{NUMERIC_ABS_TOL} \ll \text{PROB_SUM_TOL} \quad (۱۵.۲)$$

این سلسله‌مراتب بر پایه‌ی تحلیل دقیق انتشار خطا (error propagation) طراحی شده است. فرض کنید یک بردار احتمال به طول N داریم که هر درایه با خطای مطلق حداکثر ε_{abs} ذخیره شده است. خطای تجمعی در مجموع بردار برابر است با:

$$\left| \sum_i p_i - 1 \right| \leq N \cdot \varepsilon_{\text{abs}} \quad (۱۶.۲)$$

برای آنکه این خطا از تلورانس مجموع ε_{sum} فراتر نرود، باید:

$$\varepsilon_{\text{abs}} \leq \frac{\varepsilon_{\text{sum}}}{N} \quad (۱۷.۲)$$

با مقادیر فریم‌ورک ($\varepsilon_{\text{abs}} = 10^{-12}$ و $\varepsilon_{\text{sum}} = 10^{-8}$)، این شرط برای $N < 10^4$ برقرار است. برای بردارهای بسیار بلند، ممکن است نیاز به بازنگری در PROB_SUM_TOL باشد.

۲.۱۲.۲ خطای ماشین و Y1_SAFE_THRESHOLD

ثابت Y1_SAFE_THRESHOLD برای محافظت از محاسبات تک‌فوتونی (single-photon yield) طراحی شده است. در پروتکل‌های Decoy-State، تخمین Y_1 (بازدهی تک‌فوتونی) یک مرحله‌ی بحرانی است که در آن، نسبت‌های عددی بسیار کوچک محاسبه می‌شوند. اگر مخرج این نسبت‌ها به خطای ماشین نزدیک شود، نتیجه ممکن است ناپایدار یا غیرقابل اعتماد شود. فرمول محاسبه‌ی این آستانه به صورت زیر است:

$$\text{Y1_SAFE_THRESHOLD} = \max(10^{-12}, 10^4 \cdot \varepsilon_{\text{mach}}) \quad (۱۸.۲)$$

که در آن $\varepsilon_{\text{mach}} \approx 2.22 \times 10^{-16}$ خطای ماشین IEEE 754 double است. ضریب 10^4 یک حاشیه‌ی امنیتی است که با تجربه‌ی عددی به دست آمده و کمک می‌کند که آستانه در برابر خطاهای گردکردن و شرایط مرزی محافظت شود.

۳.۱۲.۲ ثابت‌های فیزیکی SI 2019

سه ثابت فیزیکی این مازول بر اساس بازتعریف SI 2019 مقادیری شده‌اند که در آن این مقادیر با تعریف‌های مرجع SI 2019 سازگار هستند:

$$h = 6.62607015 \times 10^{-34} \text{ J} \cdot \text{s} \quad (۱۹.۲)$$

$$k_B = 1.380649 \times 10^{-23} \text{ J/K} \quad (۲۰.۲)$$

$$e = 1.602176634 \times 10^{-19} \text{ C} \quad (۲۱.۲)$$

ثابت پلانک h در محاسبه انرژی فوتون $E = h\nu$ ، ثابت بولتزمن k_B در محاسبه نویز حرارتی، و بار الکترون e در محاسبه نویز نوار عبور shot noise استفاده می‌شوند. این دقت با تعریف‌های مرجع و محاسبات مبتنی بر SI سازگار است.

۴.۱۲.۲ بذر تولیددهی عدد تصادفی PCG64

ثابت MAX_SEED_INT_PCG64 حد بالای بذر مجاز برای تولیددهی عدد تصادفی PCG64 را مشخص می‌کند:

$$\text{MAX_SEED_INT_PCG64} = 2^{63} - 1 = 9223372036854775807 \quad (۲۲.۲)$$

این مقدار برابر با $2^{63} - 1$ است که حد بالای int64 با علامت است. تولیدکننده PCG64 بذر ۱۲۸ بیتی می‌پذیرد، اما در فریم‌ورک تنها ۶۳ بیت برای سازگاری با JSON و سیستم‌های سریال‌سازی استفاده می‌شود. این محدودیت، سازگاری با JavaScript (که اعداد صحیح بزرگ‌تر از 2^{53} را به دقت نمایش نمی‌دهد) را قربانی می‌کند، اما در عوض تکرارپذیری ساده‌تر را فراهم می‌سازد.

۵.۱۲.۲ آستانه‌ی دم پواسون

ثابت $1e-9 = \text{DEFAULT_POISSON_TAIL_THRESHOLD}$ برای کنترل دقت تقریب دم توزیع پواسون استفاده می‌شود. در محاسبات Decoy-State، توزیع پواسون تعداد فوتون دارای دم‌های طولانی است که در محاسبات عددی باید بریده شوند. این آستانه تعیین می‌کند که در کجا دم بریده شود:

$$P(n > n_{\max}) < \epsilon_{\text{tail}} \quad (۲۳.۲)$$

با $\epsilon_{\text{tail}} = 10^{-9}$ می‌توان $n_{\max}$ را با دقت بالا محاسبه کرد و در عین حال زمان محاسبه را معقول نگه داشت.

۶.۱۲.۲ روش‌های LP Solver

نوع LPSolverMethod سه روش حل LP را مشخص می‌کند:

• "highs": روش پیش‌فرض که HiGHS را با تشخیص خودکار روش (interior point یا dual simplex) فراخوانی می‌کند.

• "highs-ds": Dual Simplex – مناسب برای مسائل کوچک و متوسط.

• "highs-ipm": Interior Point Method – مناسب برای مسائل بزرگ تنک.

این سه گزینه از کتابخانه SciPy (نسخه ۹.۱ به بعد) استفاده می‌کنند که HiGHS را به عنوان حلقه اصلی به کار می‌گیرد. HiGHS یکی از سریع‌ترین و پایدارترین حلقه‌های LP متن‌باز است.

۱۳.۲ تحلیل معماری و ساختار داده‌ها در constants_definitions.py

۱.۱۳.۲ استفاده‌ی گسترده از Final

یکی از ویژگی‌های برجسته‌ی این ماژول، استفاده‌ی سیستماتیک از نوع Final است:

```

۱ from typing import Final, Literal, TypeAlias
۲
۳ MAX_SEED_INT_PCG64: Final[int] = (1 << 63) - 1
۴ NUMERIC_ABS_TOL: Final[float] = 1e-12
۵ NUMERIC_REL_TOL: Final[float] = 1e-9
۶ EPS: Final[float] = float(np.finfo(float).eps)

```

نوع Final که در PEP 591 معرفی شده، به بررسی‌گرهای نوع ایستا می‌گوید که این متغیرها نباید دوباره مقداردهی شوند. این محدودیت، امنیت ثابت‌ها را در سطح تحلیل ایستا تضمین می‌کند و از تغییر تصادفی آنها – که می‌تواند منجر به رفتار نادرست فریم‌ورک شود – جلوگیری می‌کند.

۲.۱۳.۲ نوع Literal و TypeAlias

نوع LPSolverMethod یک نمونه‌ی نمادین از کاربرد Literal است:

```

۱ LPSolverMethod: TypeAlias = Literal["highs", "highs-ds", "highs-ipm"]
۲ LP_SOLVER_METHODS: Final[tuple[LPSolverMethod, ...]] = ("highs", "highs-ds", "highs-ipm")

```

در اینجا Literal یک نوع تایپی ایجاد می‌کند که تنها سه رشته‌ی مشخص را می‌پذیرد. این نوع به‌عنوان TypeAlias نام‌گذاری شده و سپس در یک چندتایی Final به‌عنوان مرجع اجرایی استفاده می‌شود. این الگوی دوگانه – نوع برای بررسی ایستا و چندتایی برای بررسی پویا – امکان اعتبارسنجی هر دو در نوع و زمان اجرا را فراهم می‌سازد.

۳.۱۳.۲ فهرست PUBLIC_CONSTANT_NAMES

این فهرست مرتب، ترتیب سریال‌سازی و گزارش‌گیری را پایدار نگه می‌دارد:

```

۱ PUBLIC_CONSTANT_NAMES: Final[tuple[str, ...]] = (
۲     "MAX_SEED_INT_PCG64",
۳     "MAX_SEED_INT_UINT32",
۴     "LP_SOLVER_METHODS",
۵     "MIN_SUCCESSFUL_Z1_BASIS_EVENTS_FOR_PHASE_EST",
۶     "EPS",
۷     "NUMERIC_ABS_TOL",
۸     "NUMERIC_REL_TOL",
۹     "Y1_SAFE_THRESHOLD",
۱۰    "ENTROPY_PROB_CLAMP",
۱۱    "PROB_SUM_TOL",
۱۲    "DEFAULT_POISSON_TAIL_THRESHOLD",
۱۳    "LP_CONSTRAINT_VIOLATION_TOL",
۱۴    "CONST_PLANCK",
۱۵    "CONST_BOLTZMANN",
۱۶    "CONST_ELECTRON_CHARGE",
۱۷ )

```

این ترتیب خاص تصادفی نیست؛ ثابت‌های عددی صحیح ابتدا، سپس تلورانس‌ها، و در پایان ثابت‌های فیزیکی. این ترتیب در خروجی as_dict() ماژول constants.py منعکس می‌شود و گزارش‌های تولیدشده در نسخه‌های مختلف فریم‌ورک قابل مقایسه باقی می‌مانند.

۴.۱۳.۲ ثابت MIN_SUCCESSFUL_Z1_BASIS_EVENTS_FOR_PHASE_EST

این ثابت خاص، حداقل تعداد رویداد موفق Z1 (تک فوتون در پایه ی Z) برای تخمین نرخ خطای فاز را مشخص می‌کند:

$$n_{Z1}^{\min} = 10 \quad (۲۴.۲)$$

اگر تعداد رویدادها کمتر از این مقدار باشد، تخمین خطای فاز به شدت ناپایدار می‌شود و می‌تواند به نتایج غیرفیزیکی منجر گردد. این حداقل، یک تعهد عملی است که با تجربه‌ی عددی به دست آمده و در پروتکل‌های MDI-QKD و Decoy-State BB84 کاربرد دارد.

۱۴.۲ پیاده‌سازی ریاضی و الگوریتمی در constants_definitions.py

۱.۱۴.۲ محاسبه‌ی EPS از NumPy

ثابت EPS به صورت پویا از NumPy محاسبه می‌شود:

```
1 EPS: Final[float] = float(np.finfo(float).eps)
```

تابع `np.finfo(float).eps` مقدار خطای ماشین را برای نوع `float` برمی‌گرداند، که برابر با 2^{-52} تا $2.22044605 \times 10^{-16}$ است. این مقدار به `float` ساده تبدیل می‌شود تا با توابع مازول `math` سازگار باشد. این رویکرد، خودسازگاری با پلتفرم را تضمین می‌کند: اگر فریم‌ورک در آینده به `float128` مهاجرت کند، EPS به صورت خودکار به روز می‌شود.

۲.۱۴.۲ محاسبه‌ی Y1_SAFE_THRESHOLD با ماکزیمم

این ثابت با فرمولی دو-بخشی محاسبه می‌شود:

```
1 _Y1_EPS_MULTIPLIER: Final[float] = 1e4
2 Y1_SAFE_THRESHOLD: Final[float] = max(1e-12, _Y1_EPS_MULTIPLIER * EPS)
```

فرمول ریاضی:

$$Y1_SAFE_THRESHOLD = \max(10^{-12}, 10^4 \cdot \epsilon_{\text{mach}}) \quad (۲۵.۲)$$

استفاده از `max` تضمین می‌کند که حتی اگر EPS به طور غیرعادی کوچک شود (مثلاً در یک پلتفرم با دقت چهارگانه)، آستانه هرگز از 10^{-12} پایین‌تر نیاید. این حد پایین، پایداری عددی را در شرایط غیرعادی تضمین می‌کند.

۳.۱۴.۲ تنظیم ENTROPY_PROB_CLAMP برابر با NUMERIC_ABS_TOL

```
1 ENTROPY_PROB_CLAMP: Final[float] = NUMERIC_ABS_TOL
```

این انتخاب آگاهانه به این معناست که برش احتمال برای محاسبات آنتروپی در همان سطح تلورانس مطلق کلی انجام می‌شود. این یکپارچگی، رفتار یکنواخت را تضمین می‌کند: اگر یک احتمال به 10^{-15} برسد (که با تلورانس مطلق به عنوان 0 پذیرفته می‌شود)، در محاسبات آنتروپی نیز به 10^{-12} بریده می‌شود و در نتیجه محاسبات متوالی سازگار باقی می‌مانند.

۴.۱۴.۲ محاسبه‌ی LP_CONSTRAINT_VIOLATION_TOL

```
1 LP_CONSTRAINT_VIOLATION_TOL: Final[float] = NUMERIC_REL_TOL
```

این تلورانس، حد مجاز نقض محدودیت‌ها در حلقه‌ی LP را مشخص می‌کند. استفاده از `NUMERIC_REL_TOL` = $1e-9$ تضمین می‌کند که حلقه‌ی LP محدودیت‌ها را با دقتی در حد تلورانس نسبی کلی فریم‌ورک رعایت کند. این تنظیم، تعادل مناسبی بین سخت‌گیری بیش‌ازحد (که باعث شکست حلقه می‌شود) و سهل‌گیری بیش‌ازحد (که منجر به نتایج نادرست می‌شود) برقرار می‌سازد.

۵.۱۴.۲ ثابت‌های SI 2019

ثابت‌های فیزیکی به صورت مستقیم و با دقت کامل تعریف می‌شوند:

```

1 CONST_PLANCK: Final[float] = 6.62607015e-34 # J·s (exact, 2019 SI)
2 CONST_BOLTZMANN: Final[float] = 1.380649e-23 # J/K (exact, 2019 SI)
3 CONST_ELECTRON_CHARGE: Final[float] = 1.602176634e-19 # C (exact, 2019 SI)

```

این مقادیر از بازتعریف SI 2019 گرفته شده‌اند که در آن این سه ثابت به صورت دقیق (exact) تعریف می‌شوند – نه اندازه‌گیری شده. این بدان معناست که در نظام SI، این اعداد بدون خطا هستند و در نتیجه فریم‌ورک از بالاترین دقت ممکن برخوردار است.

۶.۱۴.۲ ماکروی بذر PCG64

```

1 MAX_SEED_INT_PCG64: Final[int] = (1 << 63) - 1
2 MAX_SEED_INT_UINT32: Final[int] = (1 << 32) - 1

```

استفاده از عملگر << (شیفت چپ) به جای ** (توان)، یک بهینه‌سازی جزئی است که در سطح بایت‌کد پایتون، سریع‌تر اجرا می‌شود. فرمول ریاضی:

$$\text{MAX_SEED_INT_PCG64} = 2^{63} - 1 \quad (۲۶.۲)$$

$$\text{MAX_SEED_INT_UINT32} = 2^{32} - 1 \quad (۲۷.۲)$$

این دو مقدار، حد بالای بذر مجاز برای دو تولیدکننده‌ی متفاوت را مشخص می‌کنند: PCG64 (۶۳ بیت) و تولیدکننده‌های مبتنی بر uint32 (مانند MT19937).

۱۵.۲ ارسال و دریافت با سایر ماژول‌ها در constants_definitions.py

۱.۱۵.۲ عدم وابستگی ورودی

این ماژول تقریباً هیچ وابستگی ورودی ندارد:

```

1 from __future__ import annotations
2 from typing import Final, Literal, TypeAlias
3 import numpy as np

```

تنها وابستگی خارجی، NumPy است که برای محاسبه‌ی EPS استفاده می‌شود. این استقلال، ماژول را به یک پایه‌ی صلب فریم‌ورک تبدیل می‌کند: هیچ ماژول دیگری نمی‌تواند بر آن تأثیر بگذارد و در نتیجه رفتار آن کاملاً قابل پیش‌بینی است.

۲.۱۵.۲ نقاط مصرف

این ماژول توسط دو مصرف‌کننده اصلی استفاده می‌شود:

- **ماژول constants_helpers.py:** تمامی تلورانس‌های عددی و آستانه‌های احتمال از این ماژول وارد می‌شوند.
- **ماژول constants.py:** به عنوان فساد (facade) عمومی، تمامی نام‌های ثابت عمومی و توابع کمکی را بازصادرات می‌کند.

علاوه بر این، چندین ماژول دیگر نیز به صورت مستقیم از این ماژول واردات انجام می‌دهند – به‌ویژه params.py که LP_SOLVER_METHODS و DEFAULT_POISSON_TAIL_THRESHOLD را استفاده می‌کند.

۳.۱۵.۲ قرارداد PUBLIC_CONSTANT_NAMES

این فهرست، قراردادی برای ماژول‌های پایین‌دستی است:

```
__all__ = list(PUBLIC_CONSTANT_NAMES)
```

این بدان معناست که `from constants_definitions import *` تنها نام‌های فهرست‌شده را وارد می‌کند. این قرارداد، مرز API عمومی را به روشنی مشخص می‌کند و از وابستگی تصادفی به ثابت‌های خصوصی جلوگیری می‌کند.

۱۶.۲ نمای کلی و هدف ماژول constants.py

ماژول `constants.py` فساد (facade) عمومی لایه‌ی ثابت‌ها و توابع کمکی فریم‌ورک QKD است. این ماژول کوچک (تنها ۵۶ خط) نقش یک public API surface را ایفا می‌کند: تمامی ماژول‌های بیرونی که نیاز به دسترسی به ثابت‌ها یا توابع کمکی دارند، تنها از این ماژول واردات انجام می‌دهند و هرگز به صورت مستقیم به `constants_definitions.py` یا `constants_helpers.py` ارجاع نمی‌دهند. این لایه‌ی واسط، انعطاف‌پذیری معماری را به شدت افزایش می‌دهد: اگر در آینده تصمیم به بازسازی داخلی این دو ماژول گرفته شود، تنها `constants.py` نیاز به به‌روزرسانی خواهد داشت و سایر ماژول‌ها بی‌تأثیر باقی می‌مانند. هدف بنیادین این ماژول، پیاده‌سازی الگوی Facade Pattern است که توسط Gamma et al. در کتاب کلاسیک الگوهای طراحی معرفی شده. در این الگو، یک واسط ساده، زیرسیستم‌های پیچیده‌تر را پنهان می‌کند و یک نقطه‌ی دسترسی یکپارچه فراهم می‌سازد. در فریم‌ورک QKD، این الگو به‌ویژه ارزشمند است؛ چرا که تعداد ثابت‌ها و توابع کمکی در طول زمان توسعه افزایش می‌یابد و در نبود یک فساد، وابستگی‌های مستقیم پیچیده‌ای شکل می‌گیرد که بازسازی را دشوار می‌سازد.

علاوه بر نقش فساد، این ماژول مسئولیت ارائه‌ی متاداده‌ی نسخه و یک تابع کمکی `as_dict()` را نیز بر عهده دارد. متغیر `__version__ = "0.3.4"` نسخه‌ی فعلی API را مشخص می‌کند و امکان بررسی سازگاری در زمان اجرا را فراهم می‌سازد. تابع `as_dict()` یک دیکشنری مرتب از تمامی ثابت‌های عمومی تولید می‌کند که برای گزارش‌گیری، لاگ‌گذاری و ردیابی آزمایش‌ها استفاده می‌شود.

۱۷.۲ مبانی تئوری و فیزیکی در constants.py

۱.۱۷.۲ الگوی فساد در مهندسی نرم‌افزار

الگوی فساد بر پایه‌ی اصل Law of Demeter (قانون دمتر) استوار است: یک ماژول نباید دربارهِ ساختار داخلی ماژول‌های دیگر بدانند. در نبود فساد، اگر ماژول A نیاز به ثابت X داشته باشد، باید بداند که X در کدام زیرماژول قرار دارد. این دانش، وابستگی غیرضروری ایجاد می‌کند و بازسازی را دشوار می‌سازد. فساد این مشکل را با ارائه‌ی یک نقطه‌ی واحد دسترسی حل می‌کند. ماژول A تنها از `constants` واردات انجام می‌دهد و در نتیجه از ساختار داخلی فریم‌ورک بی‌خبر باقی می‌ماند. این رزایی، در پروژه‌های بزرگ که تعداد ماژول‌ها به صورت نمایی رشد می‌کند، حیاتی است.

۲.۱۷.۲ پایداری رابط و نسخه‌سازی

در مهندسی نرم‌افزار حرفه‌ای، API stability یک ارزش بنیادین است. کاربران یک کتابخانه انتظار دارند که نسخه‌های جدید، رفتار نسخه‌های قبلی را حفظ کنند یا با هشدار روشن تغییر دهند. ماژول `constants.py` با ارائه‌ی یک فهرست صریح `__all__`، API عمومی خود را به روشنی تعریف می‌کند:

```
__all__ = PUBLIC_CONSTANT_NAMES ∪ PUBLIC_FUNCTION_NAMES ∪ {as_dict} (۲۸.۲)
```

این تعریف صریح، ابزاری برای semantic versioning فراهم می‌سازد: هر تغییری در `__all__` به عنوان یک تغییر ناسازگار (breaking change) تلقی می‌شود و نیازمند افزایش نسخه‌ی اصلی است.

۳.۱۷.۲ ثابت‌های فیزیکی تکمیلی

علاوه بر ثابت‌های تعریف‌شده در `constants_definitions.py`، این ماژول دو ثابت فیزیکی تکمیلی را نیز تعریف می‌کند:

```
1  CONST_PLANCK = 6.62607015e-34      # J*s
2  CONST_SPEED_OF_LIGHT = 2.99792458e8 # m/s
```

سرعت نور در خلأ، $c = 2.99792458 \times 10^8$ m/s، یکی از ثابت‌های دقیق SI 2019 است که در محاسبه‌ی طول‌موج به‌فرکانس ($\lambda = c/\nu$) و محاسبه‌ی زمان پرواز فوتون در فیبر استفاده می‌شود. جالب توجه است که `CONST_PLANCK` در این ماژول نیز تعریف شده — یک تعریف موازی با `constants_definitions.py`. این تعریف موازی، اگرچه از نظر اصل DRY (Don't Repeat Yourself) ناقص به‌نظر می‌رسد، اما سازگاری با `przeszle` را تضمین می‌کند: کدهای قدیمی که از `constants.CONST_PLANCK` استفاده می‌کردند، همچنان کار خواهند کرد.

۱۸.۲ تحلیل معماری و ساختار داده‌ها در `constants.py`

۱.۱۸.۲ ساختار واردات

ساختار واردات این ماژول، الگوی فساد را به‌روشنی نشان می‌دهد:

```
1  from .constants_definitions import (
2  CONST_BOLTZMANN, CONST_PLANCK, CONST_ELECTRON_CHARGE,
3  DEFAULT_POISSON_TAIL_THRESHOLD, ENTROPY_PROB_CLAMP, EPS,
4  LP_CONSTRAINT_VIOLATION_TOL, LP_SOLVER_METHODS, LPSolverMethod,
5  MAX_SEED_INT_PCG64, MAX_SEED_INT_UINT32,
6  MIN_SUCCESSFUL_Z1_BASIS_EVENTS_FOR_PHASE_EST,
7  NUMERIC_ABS_TOL, NUMERIC_REL_TOL, PROB_SUM_TOL,
8  PUBLIC_CONSTANT_NAMES, Y1_SAFE_THRESHOLD,
9  )
10 from .constants_helpers import (
11 PUBLIC_FUNCTION_NAMES, clamp_probabilities, clamp_probability,
12 db_to_linear, is_close, is_finite_non_negative, is_prob_vector,
13 is_valid_probability, renormalize_probabilities,
14 )
```

دو بلوک واردات، دو منبع مجزا را نشان می‌دهند: ثابت‌ها از `constants_definitions` و توابع از `constants_helpers`. هر دو بلوک تمامی نام‌های عمومی را به‌صورت صریح فهرست می‌کنند — نه از * استفاده می‌کنند. این صراحت، اگرچه کد را طولانی‌تر می‌کند، اما وابستگی‌ها را به‌روشنی مستند می‌سازد و از واردات تصادفی نام‌های خصوصی جلوگیری می‌کند.

۲.۱۸.۲ تعریف `__all__` پویا

```
1  __version__ = "4.3.0"
2
3  __all__ = [*PUBLIC_CONSTANT_NAMES, *PUBLIC_FUNCTION_NAMES, "as_dict"]
```

این تعریف پویا، `__all__` را از دو منبع مرجع می‌سازد. مزیت این رویکرد آن است که اگر در آینده یک ثابت جدید به `PUBLIC_CONSTANT_NAMES` اضافه شود، به‌صورت خودکار به API عمومی نیز اضافه می‌شود — بدون آنکه نیاز به ویرایش دستی `constants.py` باشد. این Single Source of Truth، بار نگهداری را به‌شدت کاهش می‌دهد.

۳.۱۸.۲ تابع `as_dict()`

تنها تابع تعریف‌شده در این ماژول، `as_dict()` است:

```
1  def as_dict() -> dict[str, Any]:
2  """
3  Return public constants from this module as an ordered dictionary copy.
```

```

۴ """
۵ module_globals = globals()
۶ return {name: module_globals[name] for name in PUBLIC_CONSTANT_NAMES}

```

این تابع با استفاده از `globals()` تمامی ثابت‌های عمومی را به صورت یک دیکشنری مرتب برمی‌گرداند. ترتیب کلیدها به ترتیب `PUBLIC_CONSTANT_NAMES` است، که این تضمین را فراهم می‌سازد که خروجی در نسخه‌های مختلف فریم‌ورک پایدار بماند. این پایداری برای گزارش‌گیری و مقایسه‌ی آزمایش‌ها حیاتی است.

۴.۱۸.۲ عدم استفاده از Final

جالب توجه است که ثابت‌های تعریف‌شده در این ماژول (`CONST_SPEED_OF_LIGHT` و `CONST_PLANCK`) با `Final` علامت‌گذاری نشده‌اند. این یک ناسازگاری سبکی با `constants_definitions.py` است که در آینده باید اصلاح شود. در نسخه‌ی فعلی، این دو ثابت از نظر فنی قابل تغییر هستند، اما در عمل هرگز تغییر نمی‌کنند.

۱۹.۲ پیاده‌سازی ریاضی و الگوریتمی در `constants.py`

۱.۱۹.۲ پیاده‌سازی `as_dict()`

پیاده‌سازی این تابع، اگرچه کوتاه است، اما چند نکته‌ی ظریف دارد:

```

۱ def as_dict() -> dict[str, Any]:
۲     module_globals = globals()
۳     return {name: module_globals[name] for name in PUBLIC_CONSTANT_NAMES}

```

نخست، استفاده از `globals()` به جای `getattr(module, name)` یک بهینه‌سازی کوچک است: `globals()` مستقیماً به دیکشنری ماژول دسترسی دارد، در حالی که `getattr` نیاز به جستجوی صفت دارد. دوم، استفاده از `dictionary comprehension` به جای حلقه‌ی `for`، هم خوانایی را افزایش می‌دهد و هم از نظر بایت‌کد پایتون، کارآمدتر است.

سوم، ترتیب کلیدها در خروجی دقیقاً برابر با ترتیب `PUBLIC_CONSTANT_NAMES` است، چرا که دیکشنری پایتون (از نسخه‌ی ۷.۳ به بعد) ترتیب درج را حفظ می‌کند. این تضمین ترتیب، برای تولید `hash` پایدار و مقایسه‌ی JSON حیاتی است.

۲.۱۹.۲ مقدار SI 2019 سرعت نور

```

۱ CONST_SPEED_OF_LIGHT = 2.99792458e8 # m/s

```

این مقدار، طبق SI 2019، دقیق و غیرقابل تغییر است. فرمول ریاضی:

$$c = 299\,792\,458 \text{ m/s} \quad (۲۹.۲)$$

این ثابت در محاسبات زیر استفاده می‌شود:

- تبدیل طول موج به فرکانس: $\nu = c/\lambda$
- محاسبه‌ی زمان پرواز در فیبر: $t = L \cdot n/c$ که n ضریب شکست فیبر است.
- محاسبه‌ی انرژی فوتون: $E = h\nu = hc/\lambda$

۳.۱۹.۲ تعریف موازی `CONST_PLANCK`

همان‌طور که پیش‌تر ذکر شد، `CONST_PLANCK` هم در این ماژول و هم در `constants_definitions.py` تعریف شده است. مقدار هر دو یکسان است:

$$h = 6.62607015 \times 10^{-34} \text{ J} \cdot \text{s} \quad (۳۰.۲)$$

این تعریف موازی، در نگاه اول نقض اصل DRY به نظر می‌رسد، اما در عمل یک `compatibility shim` است. در گذشته، `CONST_PLANCK` تنها در `constants.py` تعریف می‌شد. با ظهور `constants_definitions.py`، تعریف اصلی به آن ماژول منتقل شد، اما برای حفظ سازگاری با کدهای قدیمی، تعریف موازی در اینجا نگه داشته شد. در آینده، با حذف تدریجی کدهای قدیمی، این تعریف موازی نیز باید حذف شود.

۴.۱۹.۲ ترکیب API از دو منبع

ساختار `__all__` نشان می‌دهد که چگونه دو منبع در یک API یکپارچه ترکیب می‌شوند:

```
__all__ = [*PUBLIC_CONSTANT_NAMES, *PUBLIC_FUNCTION_NAMES, "as_dict"]
```

این ترکیب، سه دسته از نام‌ها را در بر می‌گیرد:

۱. ثابت‌های عمومی (از `constants_definitions`)

۲. توابع کمکی عمومی (از `constants_helpers`)

۳. تابع `as_dict` (تعریف‌شده در این ماژول)

این الگو، یک API یکپارچه را ارائه می‌دهد که در آن تمامی منابع مورد نیاز کاربر در یک نقطه در دسترس است.

۲۰.۲ ارسال و دریافت با سایر ماژول‌ها در `constants.py`

۱.۲۰.۲ وابستگی‌های ورودی

این ماژول تنها به دو ماژول داخلی وابسته است:

- `constants_definitions.py`: برای تمامی ثابت‌ها و نوع‌های تایپی.

- `constants_helpers.py`: برای تمامی توابع کمکی.

وابستگی خارجی تنها، کتابخانه‌ی استاندارد `typing` (برای نوع `Any`) است. این استقلال کم، این ماژول را به یک نقطه‌ی سبک و سریع تبدیل می‌کند.

۲.۲۰.۲ نقاط مصرف

این ماژول توسط اکثر ماژول‌های فریم‌ورک استفاده می‌شود:

- **ماژول `params.py`**: از `LP_SOLVER_METHODS` و `DEFAULT_POISSON_TAIL_THRESHOLD` استفاده می‌کند.

- **ماژول `sources.py`**: از `clamp_probabilities` و `is_valid_probability` و ثابت‌های تلورانس استفاده می‌کند.

- **ماژول `detectors.py`**: از `db_to_linear` و `is_close` استفاده می‌کند.

- **ماژول `channel.py`**: از `db_to_linear` و ثابت‌های فیزیکی استفاده می‌کند.

- **اسکرپت‌های آزمایشی**: از `as_dict()` برای گزارش‌گیری و ثبت پیکربندی استفاده می‌کنند.

۳.۲۰.۲ قرارداد نسخه‌گذاری

متغیر `__version__ = "0.3.4"` یک قرارداد مهم را ارائه می‌دهد. این نسخه‌بندی از Semantic Versioning پیروی می‌کند:

- (4) Major: تغییرات ناسازگار API.

- (3) Minor: افزودن قابلیت‌های جدید به صورت سازگار.

- (0) Patch: اصلاح خطاها و بهبود عملکرد.

ماژول‌های پایین‌دستی می‌توانند با بررسی این نسخه، سازگاری خود را تضمین کنند:

$$\text{compatible} \iff \text{major}_{\text{required}} = \text{major}_{\text{actual}} \quad (۳۱.۲)$$

این قرارداد، در آینده‌ی فریم‌ورک که ممکن است API تغییرات اساسی تجربه کند، حیاتی خواهد بود.

۴.۲۰.۲ جمع‌بندی معماری چهار ماژول

چهار ماژول بررسی‌شده در این فصل، یک معماری لایه‌ای تمیز را تشکیل می‌دهند. در پایین لایه‌بندی، `constants_definitions.py` به‌عنوان منبع نهایی ثابت‌ها قرار دارد. در لایه بعدی، `constants_helpers.py` توابع کمکی عددی را بر پایه‌ی این ثابت‌ها ارائه می‌دهد. در لایه‌ی فساد، `constants.py` یک API یکپارچه از این دو ماژول فراهم می‌سازد. در بالاترین لایه، `params.py` از این API برای ساخت شیء پیکربندی مرکزی `QKDParams` استفاده می‌کند.

این جداسازی لایه‌ها، چند مزیت کلیدی دارد. نخست، `Single Source of Truth` برای هر مفهوم: ثابت‌ها تنها در یک نقطه تعریف می‌شوند، توابع کمکی تنها در یک نقطه پیاده‌سازی می‌شوند، و پیکربندی مرکزی تنها از یک کلاس نشأت می‌گیرد. دوم، قابلیت آزمون‌پذیری بالا: هر لایه می‌تواند به‌صورت مستقل آزمون شود. سوم، انعطاف‌پذیری بازسازی: تغییرات داخلی در یک لایه، بر لایه‌های بالاتر تأثیر نمی‌گذارد. این معماری، نمونه‌ای از بهترین شیوه‌های مهندسی نرم‌افزار در پروژه‌های علمی بزرگ است.

فصل ۳

فیزیک و شبیه‌سازی کانال

۱.۳ نمای کلی و هدف ماژول

این فصل به بررسی سه لایه‌ی به‌هم‌پیوسته در مدل‌سازی کانال کوانتومی در فریم‌ورک شبیه‌سازی QKD می‌پردازد: (الف) فایل `channel.py` حاوی کلاس اصلی `FiberChannel` و کلاس‌های کمکی آن است؛ (ب) فایل `_physics.py` که توابع محاسبات فیزیکی مشتق‌شده (مانند محاسبه‌ی `Transmittance`، کسر تلفات، و ترکیب آبخاری کانال‌ها) را از بدنه‌ی کلاس جدا می‌سازد؛ و (ج) فایل `noise_models.py` که مدل‌های نویز الکتریکی سمت منبع (مانند نویز حرارتی Johnson-Nyquist و نویز شات Schottky) را به‌صورت یک `dataclass` مستقل پیاده‌سازی می‌کند. این سه ماژول در کنار یکدیگر، چرخه‌ی کامل تبدیل پارامترهای فیزیکی فیبر (طول L و ضریب تلفات α) به کمیت‌های قابل‌استفاده در محاسبات نرخ کلید QKD را فراهم می‌سازند.

هدف بنیادین این مجموعه، ارائه‌ی یک مدل تغییر یافته‌ناپذیر (`immutable`)، قابل سریال‌سازی (`serializable`) و از نظر عددی پایدار از کانال فیبر نوری است که بر پایه‌ی قانون Beer-Lambert بنا شده است. در این قانون، ضریب انتقال کل کانال به‌صورت تابعی نمایی از حاصل‌ضرب طول فیبر در ضریب تلفات تعریف می‌شود. این رابطه که در ظاهر ساده به‌نظر می‌رسد، در عمل با چالش‌های متعددی روبه‌رو است: سرریز عددی (`underflow`) برای فواصل طولانی، ظهور مقادیر زیرنرمال (`subnormal`) که دقت محاسبات آنتروپی را تخریب می‌کنند، عدم تقارن round-trip میان $\alpha \cdot L$ و L_{total} به‌دلیل خطای گردکردن IEEE-754، و نیاز به سازوکارهایی برای تشخیص پیکربندی‌های فیزیکی مشکوک (`physical suspicion`) بدون توقف کامل شبیه‌سازی. این فصل به‌تفصیل نشان می‌دهد که چگونه هر یک از این چالش‌ها در سطح کد پایتون آدرس‌دهی شده‌اند.

طراحی این ماژول بر چهار اصل بنیادین استوار است. نخست، “fail-fast”: هرگونه ناسازگاری در زمان بارگذاری پیکربندی باید بلافاصله با یک استثنای صریح افشا گردد تا از انتشار خطا به لایه‌های پایین‌تر جلوگیری شود. دوم، “strict mode”: کاربر می‌تواند با فعال‌سازی این حالت، رفتار ماژول را از هشدار دادن (`warning`) به پرتاب استثنا (`raising`) تغییر دهد تا در خطوط لوله‌ی تولید (`production pipelines`) هیچ پیکربندی مشکوکی پذیرفته نشود. سوم، “reproducibility”: فراداده‌ی زمان ساخت (`construction-time metadata`)، حالت `strict` در زمان ساخت، و مسیر ساخت (`construction path`) در هر نمونه ثبت می‌شوند تا بازتولید آزمایش‌ها در محیط‌های مختلف تضمین گردد. چهارم، “type-safety”: استفاده‌ی گسترده از `Generic`، `TypeVar`، `Protocol` و `Final` اطمینان می‌دهد که تحلیل‌گرهای ایستا مانند `mypy` و `pyright` می‌توانند بسیاری از خطاهای نوعی را پیش از اجرای کد شناسایی کنند.

در سطح کلان، فایل `channel.py` با بیش از ۴۵۰۰ خط کد، سنگین‌ترین و پیچیده‌ترین جزء این مجموعه است. این فایل نه‌تنها کلاس `FiberChannel` را تعریف می‌کند، بلکه زیرکلاس `AttenuationOnlyFiberChannel` را به‌عنوان نشانگر نوعی برای کانال‌های صرفاً تلفاتی، توابع برداری `transmittance_array` و `transmittance_scalar` را برای حلقه‌های داغ Monte Carlo، کلاس `ChannelSimParams` را به‌عنوان بسته‌ی یخ‌زده‌ی کمیت‌های مشتق‌شده، و سازوکار `strict mode` را به‌صورت یک `ContextVar` پیاده‌سازی می‌کند. فایل `_physics.py` با حدود ۶۲۰ خط، توابعی را که در نسخه‌های پیشین به‌عنوان متدهای کلاس تعریف می‌شدند، به توابعی مستقل تبدیل کرده است؛ این تفکیک نگرانی‌ها (`separation of concerns`) امکان تست واحد آسان‌تر و استفاده‌ی مجدد منطق را فراهم می‌سازد. فایل `noise_models.py` با تنها ۸۰ خط، یک ماژول فشرده اما مهم برای مدل‌سازی نویز الکتریکی فوتودیود نظارتی در سمت فرستنده است که در سیستم‌های SCM-QKD و WDM برای تخمین نویز اضافی به‌کار می‌رود.

۲.۳ مبانی تئوری و فیزیکی

۱.۲.۳ قانون Beer-Lambert و تلفات فیبر نوری

در فیبر نوری سیلیکا، تضعیف سیگنال نوری ناشی از سه مکانیزم فیزیکی اصلی است: پراکندگی Rayleigh ناشی از نوسانات چگالی در مقیاس کوچک‌تر از طول موج، جذب ناشی از ناخالصی‌های یونی (به‌ویژه یون‌های OH^- در طول موج 1383 nm)، و پراکندگی Mie ناشی از ناخالصی‌های با مقیاس بزرگ‌تر. در طول موج‌های پنجره‌ی مخابراتی (1310 nm و 1550 nm)، پراکندگی Rayleigh مکانیزم غالب است و منجر به ضریب تلفات تقریباً ثابت به ازای واحد طول می‌گردد. این رفتار خطی با قانون Beer-Lambert مدل می‌شود:

$$T(L) = \frac{P(L)}{P(0)} = 10^{-\alpha L/10}, \quad (1.3)$$

که در آن T ضریب انتقال (transmittance)، $P(L)$ توان نوری خروجی در طول L ، $P(0)$ توان ورودی، α ضریب تلفات به واحد dB/km و L طول فیبر به کیلومتر است. تلفات کل به دسی‌بل از رابطه‌ی خطی زیر به دست می‌آید:

$$L_{\text{total}} [\text{dB}] = \alpha [\text{dB/km}] \times L [\text{km}], \quad (2.3)$$

و در نتیجه رابطه‌ی نمایی معادل برای ضریب انتقال به صورت $T = 10^{-L_{\text{total}}/10}$ نوشته می‌شود. در طول موج 1550 nm مقدار معمول $\alpha \approx 0.2$ dB/km است که به فاصله‌ی تقریبی 50 km نرخ کلید غیرصفری در BB84-Decoy با آشکارسازهای SNSPD اجازه می‌دهد.

۲.۲.۳ تفکیک η_{channel} از بهره‌ی کلی آشکارساز

در یک حلقه‌ی بودجه‌ی پیوند QKD، احتمال کلی آشکارسازی یک فوتون منتشرشده از سمت آلیس به صورت زیر تجزیه می‌شود:

$$\eta = \eta_{\text{channel}} \cdot \eta_{\text{detector}} \cdot \eta_{\text{coupling}}, \quad (3.3)$$

که در آن $\eta_{\text{channel}} = T$ همان ضریب انتقال فیبر است، η_{detector} بازدهی تک‌آشکارساز (معمولاً 0.1 تا 0.3 برای SNSPD و 0.05 تا 0.15 برای SPAD)، و η_{coupling} تلفات اتصالات، اتصال‌دهنده‌ها و جوش‌های فیبر (معمولاً 0.5 تا 0.9) است. یکی از خطاهای متداول در ادبیات شبیه‌سازی، استفاده‌ی مستقیم T به جای η در فرمول بهره‌ی Decoy-State:

$$Q_{\mu} = \sum_{n=0}^{\infty} Y_n \cdot \frac{\mu^n e^{-\mu}}{n!}, \quad Y_n = 1 - (1 - \eta)^n + Y_0, \quad (4.3)$$

است که می‌تواند منجر به جابه‌جایی منحنی نرخ کلید تا 50 تا 100 km گردد. به همین دلیل، کلاس FiberChannel صراحتاً تنها η_{channel} را ارائه می‌دهد و تابع کمکی link_efficiency ترکیب سه‌گانه‌ی رابطه‌ی (۳.۳) را در یک نقطه‌ی متمرکز انجام می‌دهد تا اطمینان حاصل شود که تمام محاسبات (تحلیل مقاطع Rogers et al. (2007) و فرمول بهره) از یک ترکیب‌بندی واحد استفاده می‌کنند.

۳.۲.۳ پدیده‌ی زیرنرمال بودن و سرریز عددی

در فواصل بسیار طولانی (مثلاً $L = 5000$ km با $\alpha = 0.2$)، مقدار $L_{\text{total}} = 1000$ dB و $T = 10^{-100}$ است که به راحتی در محدوده‌ی نرمال double نمایش داده می‌شود. اما برای $L = 50000$ km با همان α ، مقدار $T = 10^{-1000}$ است که بسیار کوچک‌تر از کوچک‌ترین عدد نرمال مثبت در IEEE-754 ($\approx 2.225 \times 10^{-308}$) می‌باشد. در این حالت دو سناریو رخ می‌دهد: (الف) اگر مقدار به صورت تدریجی کاهش یافته (gradual underflow) محاسبه شود، یک عدد زیرنرمال با تنها 1 تا 2 بیت معتبر مانتیس تولید می‌شود؛ (ب) اگر زیرنرمال‌ها در محیط اجرا غیرفعال باشند (FTZ/DAZ modes)، مقدار به صفر دقیق تبدیل می‌شود. هر دو حالت برای محاسبات QKD فاجعه‌بارند: یک عدد زیرنرمال در فرمول بهره $Y_n = 1 - (1 - \eta)^n$ به دلیل از دست رفتن بیت‌های معتبر، منجر به نتایج کاملاً اشتباه می‌شود؛ و یک مقدار صفر دقیق، در

لگاریتم آنتروپی شانون $H_2(p) = -p \log_2 p - (1-p) \log_2 (1-p)$ منجر به $\log(0) = -\infty$ و در نتیجه NaN می‌گردد. به همین دلیل، ماژول channel.py سه سطح تشخیص را پیاده‌سازی می‌کند: آستانه‌ی زیرنرمال TRANSMITTANCE_SUBNORMAL_THRESHOLD (برابر sys.float_info.min)، آستانه‌ی سرریز مطلق TRANSMITTANCE_UNDERFLOW_ABS_TOL (همان مقدار)، و آستانه‌ی قرارداد L, T به مقدار 10^{-12} . در حالت strict mode، هر یک از این شرایط به پرتاب استثنای ParameterValidationError منجر می‌شوند.

۴.۲.۳ نویز حرارتی Johnson-Nyquist

مقاومت اهمی در دمای غیرصفر، به دلیل حرکت حرارتی تصادفی الکترون‌ها، یک ولتاژ نویز تصادفی تولید می‌کند که به افتخار کاشفان آن، نویز Johnson-Nyquist نامیده می‌شود. توان طیفی این نویز در محدوده‌ی فرکانسی پایین (نسبت به دمای پلاسمایی) مسطح است و از رابطه‌ی زیر پیروی می‌کند:

$$\langle v_n^2 \rangle = 4 k_B T R \Delta f, \quad (۵.۳)$$

که در آن $k_B = 1.380649 \times 10^{-23}$ J/K ثابت بولتزمن، T دمای مطلق، R مقاومت بار (load resistance) و Δf پهنای باند الکتریکی است. انحراف معیار ولتاژ نویز حرارتی از ریشه‌گیری رابطه‌ی فوق به دست می‌آید:

$$\sigma_v^{(\text{thermal})} = \sqrt{4 k_B T R \Delta f}. \quad (۶.۳)$$

در دمای اتاق $T = 298.15$ K، با $R = 50 \Omega$ و $\Delta f = 1$ GHz، این مقدار تقریباً $28.9 \mu V$ است.

۵.۲.۳ نویز شات Schottky

عبور جریان الکتریکی از یک فراسو، به دلیل گسسته‌بودن بار الکترون، با نویز شات همراه است. توان طیفی جریان نویز شات از رابطه‌ی Schottky به دست می‌آید:

$$\langle i_n^2 \rangle = 2 e I_{DC} \Delta f, \quad (۷.۳)$$

که در آن $e = 1.60217663 \times 10^{-19}$ C بار الکترون، I_{DC} جریان مستمر فوتودیود نظارتی، و Δf پهنای باند است. با عبور از مقاومت بار، این نویز به ولتاژ تبدیل می‌شود:

$$\sigma_v^{(\text{shot})} = R \cdot \sqrt{2 e I_{DC} \Delta f}. \quad (۸.۳)$$

چون نویز حرارتی و نویز شات از فرایندهای فیزیکی مستقل هستند، ترکیب آن‌ها به صورت جمع توان‌ها انجام می‌شود:

$$\sigma_v^{(\text{total})} = \sqrt{(\sigma_v^{(\text{thermal})})^2 + (\sigma_v^{(\text{shot})})^2}. \quad (۹.۳)$$

این رابطه در متد total_voltage_std کلاس ElectricalNoiseConfig پیاده‌سازی شده است.

۶.۲.۳ کسر تلفات تک‌کیلومتری و تقریب خطی

کسر توان نوری از دست‌رفته در دقیقاً یک کیلومتر فیبر از رابطه‌ی زیر به دست می‌آید:

$$\ell = 1 - T(1 \text{ km}) = 1 - 10^{-\alpha/10}. \quad (۱۰.۳)$$

برای $\alpha = 0.2$ dB/km، این مقدار $\ell \approx 0.0451$ است. یک اشتباه متداول در ادبیات مهندسی، ضرب این کسر در طول فیبر برای تخمین تلفات کل است؛ اما این کار نادرست است زیرا کسر تلفات به صورت نمایی با فاصله ترکیب می‌شود:

$$\ell_{\text{total}}(L) = 1 - (1 - \ell)^L \neq \ell \cdot L. \quad (۱۱.۳)$$

برای $\alpha = 0.2$ و $L = 100$ km، تقریب خطی $\ell \cdot L \approx 4.51$ است که کاملاً غیرفیزیکی (بزرگ‌تر از ۱) می‌باشد، در حالی که مقدار دقیق $\ell_{\text{total}} \approx 0.990$ است. به همین دلیل، ماژول channel.py متد قدیمی

loss_fraction_per_km را منسوخ کرده و نام جدید single_km_power_loss_fraction را برای آن انتخاب کرده است تا ماهیت غیرخطی این کمیت به‌طور صریح در نام بازتاب یابد.

برای α های بسیار کوچک (نظیر $\alpha = 10^{-6}$ dB/km در شبیه‌سازی‌های فرضی)، فرمول مستقیم $1 - 10^{-\alpha/10}$ دچار لغو فاجعه‌بار (catastrophic cancellation) می‌شود: مقدار $0.99999976 \approx 10^{-10^{-7}}$ تنها 6 رقم معتبر دارد و تفریق از 1.0 منجر به از دست رفتن بیشتر بیت‌ها می‌شود. برای جلوگیری از این ناپایداری، از تابع math.expml که به‌صورت تخصصی برای محاسبه‌ی دقیق $e^x - 1$ برای x های کوچک طراحی شده، استفاده می‌گردد:

$$\ell = -\text{expml}\left(-\frac{\alpha \cdot \ln 10}{10}\right). \quad (12.3)$$

این فرمول برای هر مقدار α ، تمامی بیت‌های معتبر را حفظ می‌کند. تقریب خطی مرتبه‌ی اول تیلور از رابطه‌ی (۱۰.۳) به‌صورت زیر است:

$$\ell_{\text{linear}} \approx \frac{\alpha \cdot \ln 10}{10} \approx 0.2303 \cdot \alpha, \quad (13.3)$$

که برای $\alpha = 0.2$ مقدار $\ell_{\text{linear}} \approx 0.0461$ را می‌دهد. این تقریب واقعاً خطی است (به L وابسته نیست) و با فاصله به‌صورت خطی ترکیب می‌شود، اما تنها برای $\alpha \cdot L$ های کوچک معتبر است. برای $\alpha > 4.34$ dB/km (که در آن $10/\ln 10 \approx 4.34$) این تقریب از 1 فراتر می‌رود و فیزیکی نمی‌باشد.

۷.۲.۳ ترکیب آبخاری کانال‌ها

در یک زنجیره‌ی چندبُعدی از فیبرها با پارامترهای متفاوت، ضریب انتقال کل از حاصل‌ضرب ضرایب انتقال هر بخش به دست می‌آید:

$$T_{\text{total}} = \prod_{i=1}^N T_i = \prod_{i=1}^N 10^{-\alpha_i L_i / 10} = 10^{-\sum_i \alpha_i L_i / 10}, \quad (14.3)$$

که معادل با جمع تلفات در واحد دسی‌بل است: $L_{\text{total}} = \sum_i \alpha_i L_i$. اگر بخواهیم یک کانال معادل تک‌بُعدی با همان تلفات کل و طول کل تعریف کنیم، ضریب تلفات میانگین آن به‌صورت زیر محاسبه می‌شود:

$$\alpha_{\text{avg}} = \frac{L_{\text{total}}}{L_{\text{total_distance}}} = \frac{\sum_i \alpha_i L_i}{\sum_i L_i}. \quad (15.3)$$

این یک میانگین حسابی وزن‌دار است که به هیچ نوع فیبر استاندارد در هیچ طول موجی اشاره ندارد. به‌عنوان مثال، ترکیب یک فیبر 100 km با $\alpha = 0.2$ و یک فیبر 10 km با $\alpha = 3.0$ به $\alpha_{\text{avg}} \approx 0.47$ dB/km می‌انجامد که در هیچ طول موجی فیزیکی نیست. به همین دلیل، متد cascade در کد به‌صراحت هشدار می‌دهد که کانال معادل، یک “composite loss-equivalent” است و نباید در مقایسه‌های ادبی به‌عنوان فیبر واقعی به‌کار رود.

۸.۲.۳ پاشش کروماتیک و محدودیت‌های مدل

پاشش کروماتیک (Chromatic Dispersion) باعث پهن‌شدن پالس نوری در طول فیبر می‌شود و در صورتی که پهنای زمانی پالس از پنجره‌ی زمانی آشکارساز فراتر رود، به از بین رفتن اطلاعات می‌انجامد. ضریب پاشش $D(\lambda)$ به واحد ps/(nm · km) بیان می‌شود و پهن‌شدن زمانی از رابطه‌ی زیر تقریب زده می‌شود:

$$\Delta\tau \approx D(\lambda) \cdot L \cdot \Delta\lambda, \quad (16.3)$$

که در آن $\Delta\lambda$ پهنای طیفی منبع است. در فیبر استاندارد تک‌حالت در 1550 nm، $D \approx 17$ ps/(nm · km) است که برای منبع لیزری با $\Delta\lambda = 0.1$ nm در 100 km، پهن‌شدن 170 ps تولید می‌کند. این پدیده در مدل FiberChannel پیاده‌سازی نشده است (زیرا بر ضریب انتقال اثر مستقیم ندارد) اما در ChannelSimParams به‌عنوان فراداده‌ی عددی ثبت می‌گردد تا توسط موتور شبیه‌سازی آشکارساز برای مدل‌سازی پنجره‌ی زمانی قابل استفاده باشد.

۳.۳ تحلیل معماری و ساختار داده‌ها

۱.۳.۳ کلاس FiberChannel به‌عنوان Frozen Dataclass

کلاس FiberChannel با دو decorator اصلی تعریف می‌شود:

```

1 @dataclass(frozen=True, slots=True)
2 class FiberChannel:
3     """Models a simple optical fiber channel defined by attenuation."""
4     attenuation: AttenuationConfig
5     _construction_path: str = field(default="direct", repr=False)
6     _total_loss_db_override: Optional[float] = field(
7         init=False, default=None, repr=False
8     )
9     _transmittance: Any = field(init=False, repr=False, default=_UNSET)
10    _strict_mode_at_construction: bool = field(init=False, repr=False,
11        default=False)
12    wavelength_nm: Optional[float] = field(default=None, repr=False)
13    _metadata_extra: Mapping[str, Any] = field(
14        init=False, repr=False, default_factory=lambda: _EMPTY_METADATA_EXTRA
15    )
16    __hash__ = None # type: ignore[assignment,method-assign]
```

پرچم frozen=True کلاس را تغییرناپذیر می‌سازد؛ پس از ساخت نمونه، هیچ فیلدی نمی‌تواند با تخصیص معمولی تغییر کند. این ویژگی برای تضمین reproducibility حیاتی است. هرگاه تغییر ضروری باشد (مانند تنظیم `_total_loss_db_override` در `from_total_loss`)، باید از `object.__setattr__(...)` استفاده شود که تنها راه قانونی دور زدن محدودیت frozen است. پرچم slots=True از تخصیص `__dict__` جلوگیری کرده و به جای آن فضا را از طریق `__slots__` تخصیص می‌دهد؛ این بهینه‌سازی مصرف حافظه را تا 40% کاهش داده و سرعت دسترسی به فیلدها را افزایش می‌دهد.

۲.۳.۳ فیلدهای کلاس و نقش آن‌ها

فیلد attenuation (نوع: AttenuationConfig): تنها فیلد اجباری کاربر. این فیلد، یک dataclass دیگر از ماژول `qkd.datatypes` است که دو فیلد `fiber_length` (به km) و `attenuation_coefficient` (به dB/km) را در بر می‌گیرد. قرارداد AttenuationConfig آن است که `total_loss_db = fiber_length * attenuation_coefficient` باید برقرار باشد.

فیلد _construction_path (نوع: str, پیش‌فرض: "direct"): این فیلد مسیر ساخت نمونه را ثبت می‌کند تا در پیام‌های هشدار و فراداده‌ی بازتولید، ریشه‌ی خطا قابل ردیابی باشد. مقادیر ممکن عبارت‌اند از: "direct" (ساخت مستقیم)، "from_total_loss"، "from_tuple"، "from_config_dict"، "with_distance"، "with_loss"، "fast_construct" و "cascade".

فیلد _total_loss_db_override (نوع: Optional[float], init=False): این فیلد برای حفظ دقت round-trip در کانال‌های ساخته‌شده با `from_total_loss` طراحی شده است. هنگامی که کاربر `Ltotal` را به‌طور صریح ارائه می‌دهد، $\alpha = L_{total}/L$ محاسبه می‌شود که به‌دلیل گردکردن IEEE-754 ممکن است $\alpha \cdot L$ با `Ltotal` اصلی در سطح ULP متفاوت باشد. برای جلوگیری از این ناسازگاری، مقدار اصلی `Ltotal` در این فیلد ذخیره شده و در محاسبات `total_loss_db` و `transmittance` به جای $\alpha \cdot L$ به‌کار می‌رود.

فیلد _transmittance (نوع: Any, init=False, پیش‌فرض: _UNSET): این فیلد مقدار از پیش محاسبه‌شده‌ی ضریب انتقال را نگه می‌دارد. استفاده از نوع Any و پیش‌فرض `_UNSET` (یک singleton از کلاس `_Unset`) به ما اجازه می‌دهد بین حالت "هنوز محاسبه نشده" و "مقدار عددی معتبر" تمایز قائل شویم؛ این امر در `__repr__` و `__setstate__` حیاتی است.

فیلد _strict_mode_at_construction (نوع: bool, init=False): این فیلد مقدار `strict mode` را در زمان ساخت نمونه ثبت می‌کند (نه در زمان سریال‌سازی). این تمایز مهم است زیرا اگر کاربر بین ساخت و سریال‌سازی، حالت `strict` را تغییر دهد، فراداده‌ی بازتولید نباید جعلی شود.

فیلد `wavelength_nm` (نوع: `Optional[float]`): طول موج عملیاتی به نانومتر. هنگامی که تنظیم شود، یک بررسی سازگاری فیزیکی بر اساس جدول `TYPICAL_WAVELENGTH_ALPHA_RANGES` انجام می‌شود تا مقادیر α نامتعرف شناسایی شوند.

فیلد `_metadata_extra` (نوع: `Mapping[str, Any]`): این فیلد کلیدهای ناشناخته‌ی `_metadata` را که از `from_config_dict` عبور کرده‌اند، برای round-trip حفظ می‌کند. استفاده از `MappingProxyType` به جای `dict`، تغییرناپذیری این فیلد را در یک `dataclass` یخ‌زده تضمین می‌کند.

۳.۳.۳ `__hash__ = None:`

یکی از تصمیمات طراحی ظریف، تنظیم `__hash__ = None` پس از تعریف کلاس است:

```
۱ FiberChannel.__hash__ = None # type: ignore[assignment,method-assign]
۲
```

این کار به این دلیل ضروری است که تساوی مبتنی بر `is_close` غیرانتقالی است: اگر $a \approx b$ و $b \approx c$ ، نمی‌توان نتیجه گرفت که $a \approx c$. این ویژگی با قرارداد هش $\text{hash}(a) = \text{hash}(b) \Rightarrow a == b$ در تضاد است. به همین دلیل، کلاس به‌صورت صریح غیرقابل‌هش (`unhashable`) می‌شود و تلاش برای استفاده از نمونه‌ها به‌عنوان کلید `dict` یا عضو `set` با `TypeError` مواجه می‌گردد.

۴.۳.۳ زیرکلاس `AttenuationOnlyFiberChannel`

این زیرکلاس به‌عنوان یک نشانگر نوعی (type marker) عمل می‌کند:

```
۱ class AttenuationOnlyFiberChannel(FiberChannel):
۲     __slots__ = ()
۳     def __eq__(self, other: Any) -> bool:
۴         if not isinstance(other, AttenuationOnlyFiberChannel):
۵             return NotImplemented
۶         return super().__eq__(other)
۷
```

بدنه‌ی کلاس خالی است (`__slots__ = ()`) برای جلوگیری از تداخل slot در زیرکلاس‌سازی آینده) زیرا رفتار آن با کلاس پایه یکسان است. تنها تفاوت در `__eq__` است که `type(self) == type(other)` را الزامی می‌کند؛ به این ترتیب، یک `FiberChannel` و یک `AttenuationOnlyFiberChannel` با همان اسکالرها دیگر برابر در نظر گرفته نمی‌شوند. این امر به کدهای پایین‌دستی اجازه می‌دهد با `isinstance` بررسی کنند که آیا کانال ارائه‌شده واقعاً صرفاً تلفاتی است یا خیر.

۵.۳.۳ ثابت‌های آستانه‌ای

ماژول چندین ثابت آستانه‌ای در سطح ماژول تعریف می‌کند:

```
۱ TRANSMITTANCE_SUBNORMAL_THRESHOLD: float = sys.float_info.min
۲ TRANSMITTANCE_UNDERFLOW_THRESHOLD: float =
TRANSMITTANCE_SUBNORMAL_THRESHOLD
۳ MAX_DISTANCE_KM: float = 10_0000.
۴ WARNING_DISTANCE_KM: float = 1e-6
۵ MAX_FIBER_LOSS_DB_KM: float = 0.100
۶ _CONTRACT_CHECK_REL_TOL: float = 1e-12
۷ _CONTRACT_CHECK_ABS_TOL: float = 1e-12
۸ _ZERO_LOSS_ABS_TOL: float = 1e-12
۹ _TRANSMITTANCE_UNDERFLOW_ABS_TOL: float = sys.float_info.min
۱۰
```

این ثابت‌ها هر یک نقش مشخصی دارند. TRANSMITTANCE_SUBNORMAL_THRESHOLD کوچک‌ترین عدد نرمال مثبت در IEEE-754 را نشان می‌دهد و آستانه‌ی تشخیص مقادیر زیرنرمال است. MAX_DISTANCE_KM حداکثر طول فیبر قابل پذیرش برای یک قطعه‌ی منفرد را تعیین می‌کند (پیش از این 1000 km بود که در نسخه‌های جدید به 10000 km افزایش یافت تا لینک‌های اولترالانگ با تقویت‌کننده پوشش داده شوند). MAX_FIBER_LOSS_DB_KM حداکثر ضریب تلفات مجاز را تعیین می‌کند (افزایش از 10 به 100 dB/km برای پوشش فیبرهای پلاستیکی و فیبرهای تخصصی). CONTRACT_CHECK_* تلورانس‌های بررسی قرارداد $L = \alpha \cdot L$ هستند. ZERO_LOSS_ABS_TOL آستانه‌ی یکپارچه‌ی "تلفات صفر" است.

۶.۳.۳ مدیریت ContextVar با strict mode

یکی از نقاط قوت معماری، استفاده از ContextVar.contextvars برای مدیریت strict mode است:

```

1  _STRICT_MODE: "contextvars.ContextVar[bool]" = contextvars.ContextVar(
2  "QKD_STRICT",
3  default=os.environ.get("QKD_STRICT", "").lower()
4  in ("1", "true", "yes", "on"),
5  )
6
7  def get_strict_mode() -> bool:
8  return _STRICT_MODE.get()
9
10 def set_strict_mode(enabled: bool) -> contextvars.Token:
11 return _STRICT_MODE.set(bool(enabled))
12
13 def reset_strict_mode(token: contextvars.Token) -> None:
14 _STRICT_MODE.reset(token)
15

```

این رویکرد سه مزیت دارد: (الف) thread-safe و async-safe است (هر thread یا task مقدار مستقل خود را دارد)؛ (ب) امکان scoped strictness را فراهم می‌کند (یک token بازگردانده می‌شود که می‌توان با آن به مقدار قبلی بازگشت)؛ (ج) مقدار پیش‌فرض از متغیر محیطی QKD_STRICT خوانده می‌شود تا کاربر بتواند بدون تغییر کد، حالت را فعال کند.

۷.۳.۳ ثبت هشدارهای فیزیکی مشکوک

کلاس PhysicalSuspicionWarning به‌عنوان زیرکلاس UserWarning تعریف می‌شود:

```

1  class PhysicalSuspicionWarning(UserWarning):
2      """Warning emitted when a channel configuration is physically
3      suspicious."""
4
5  def _warn_physical_suspicion(msg: str, *, stacklevel: int = 2) -> None:
6      warnings.warn(msg, PhysicalSuspicionWarning, stacklevel=stacklevel)

```

استفاده از warnings.warn به جای logger.warning تضمین می‌کند که هشدارها به صورت پیش‌فرض به کاربر نمایش داده می‌شوند (برخلاف logger که نیازمند پیکربندی است). این هشدارها در مواردی مانند فاصله‌های زیرمیلی‌متری، $\alpha = 0$ با فاصله‌ی غیرصفر، و ناسازگاری طول موج با α صادر می‌شوند.

۸.۳.۳ کلاس ElectricalNoiseConfig

این کلاس در noise_models.py به صورت یک dataclass با slots=True (اما نه frozen=True، زیرا مقادیر آن ممکن است در طول شبیه‌سازی به روز شوند) تعریف می‌شود:

```

1  @dataclass(slots=True)

```

```

۲     class ElectricalNoiseConfig:
۳         temperature_k: float = 15.298
۴         bandwidth_hz: float = 1e9
۵         load_resistance_ohm: float = 0.50
۶         monitor_photocurrent_a: Optional[float] = None
۷
۸         def validate(self) -> None:
۹             if self.temperature_k <= 0:
۱0                raise ParameterValidationError("temperature_k must be positive.")
۱1             if self.bandwidth_hz < 0:
۱2                raise ParameterValidationError("bandwidth_hz must be non-negative.")
۱3             if self.load_resistance_ohm <= 0:
۱4                raise ParameterValidationError("load_resistance_ohm must be positive.")
۱5             if self.monitor_photocurrent_a is not None and
self.monitor_photocurrent_a < 0:
۱6                raise ParameterValidationError("monitor_photocurrent_a must be
non-negative.")
۱۷

```

چهار فیلد آن پارامترهای فیزیکی نویز الکتریکی را در بر می‌گیرند: دما، پهنای باند، مقاومت بار، و جریان فوتودیود نظارتی. متد `validate` به صورت صریح محدوده‌های فیزیکی را بررسی می‌کند؛ به عنوان مثال، دمای صفر یا منفی به دلیل نقض ترمودینامیک رد می‌شود.

۹.۳.۳ کلاس `ChannelSimParams`

این کلاس به عنوان یک بسته‌ی یخ‌زده‌ی کمیت‌های مشتق‌شده عمل می‌کند:

```

۱     @dataclasses.dataclass(frozen=True)
۲     class ChannelSimParams:
۳         transmittance: float
۴         total_loss_db: float
۵         fiber_loss_db_km: float
۶         distance_km: float
۷         dispersion_parameter_ps_nm_km: float = 0.0
۸         wavelength_nm: Optional[float] = None
۹

```

این کلاس با ثابت کردن مقادیر در زمان ساخت، اطمینان حاصل می‌کند که کد شبیه‌سازی پایین‌دستی همیشه از یک تصویر لحظه‌ای (snapshot) یکپارچه می‌خواند و خطر خواندن مقادیر قدیمی از دیکشنری پیکربندی را از بین می‌برد. تابع کارخانه‌ی `build_channel_sim_params` تنها مسیر تأییدشده‌ی ساخت این کلاس است.

۴.۳ پیاده‌سازی ریاضی و الگوریتمی

۱.۴.۳ محاسبه‌ی ضریب انتقال در `_compute_transmittance`

این متد که در `channel.py` و نسخه‌ی مستقل آن در `_physics.py` پیاده‌سازی شده، قلب محاسبات فیزیکی مازول است:

```

۱     def _compute_transmittance(self) -> float:
۲         total_db = self._effective_total_loss_db()
۳         return self._transmittance_from_loss_db(total_db)
۴
۵     @staticmethod
۶     def _transmittance_from_loss_db(total_db: float) -> float:
۷         if total_db == :0.0
۸         if math.copysign(0.1), total_db) < 0:

```

```

۹     _warn_physical_suspicion(
۱۰     f"_transmittance_from_loss_db: total_db is negative zero ((0.0-...",
۱۱     stacklevel=2,
۱۲     )
۱۳     return 0.1
۱۴     return math.pow(0.10), -total_db / (0.10)
۱۵

```

این کد سه نکته‌ی ظریف را آدرس‌دهی می‌کند. نخست، استفاده از `self.attenuation.total_loss_db` به جای `self._effective_total_loss_db()` از `_total_loss_db_override` را فراهم می‌کند. دوم، شاخه‌ی `total_db == 0.0` برای جلوگیری از محاسبه‌ی `math.pow(10.0, 0.0)` است که همیشه 1.0 را برمی‌گرداند؛ اما این شاخه‌ی صریح به ما اجازه می‌دهد نویز صفر منفی (`-0.0`) را که می‌تواند نشانه‌ی خطای بالادستی باشد، تشخیص دهیم. سوم، استفاده از `math.pow` به جای عملگر `**` به این دلیل است که `math.pow` در برابر `OverflowError` در پلتفرم‌های مختلف، رفتار یکدست‌تری دارد.

۲.۴.۳ بررسی سلامت ضریب انتقال

متد `_check_transmittance_health` یک بررسی چهارمرحله‌ای انجام می‌دهد:

```

۱     def _check_transmittance_health(self, transmittance: float) -> None:
۲         # NaN: reject unconditionally.
۳         if math.isnan(transmittance):
۴             raise ParameterValidationError(f"Transmittance computed as NaN...")
۵
۶         # Negative zero: suspicious, may mask upstream sign error.
۷         if transmittance == 0.0 and math.copysign(0.1), transmittance) < 0:
۸             neg_zero_msg = (...)
۹             if get_strict_mode():
۱۰                raise ParameterValidationError(neg_zero_msg, ...)
۱۱                _warn_physical_suspicion(neg_zero_msg, stacklevel=4)
۱۲
۱۳                total_db = self._effective_total_loss_db()
۱۴
۱۵                # Underflow: exact-zero transmittance with non-zero total_db.
۱۶                if transmittance == 0.0 and total_db > :0.0
۱۷                underflow_msg = (...)
۱۸                if get_strict_mode():
۱۹                raise ParameterValidationError(underflow_msg, ...)
۲۰                _warn_physical_suspicion(underflow_msg, stacklevel=4)
۲۱                # Subnormal: between 0 and smallest normal float.
۲۲                elif 0.0 < transmittance < TRANSMITTANCE_SUBNORMAL_THRESHOLD:
۲۳                subnormal_msg = (...)
۲۴                if get_strict_mode():
۲۵                raise ParameterValidationError(...)
۲۶                _warn_physical_suspicion(subnormal_msg, stacklevel=4)
۲۷
۲۸                # Range validation.
۲۹                if not 0.0 <= transmittance <= (0.1:
۳۰                raise ParameterValidationError(f"Transmittance out of range [0, 1]...")
۳۱

```

هر یک از این مراحل به یک ناپایداری عددی متفاوت اشاره دارد. NaN معمولاً ناشی از ورودی‌های NaN است (که نباید از `AttenuationConfig` معتبر عبور کنند، اما ممکن است از طریق `pickle` یا `monkey-patching` وارد شوند). زیرنرمال بودن نشانه‌ی فواصل بسیار طولانی است که دقت محاسبات را به شدت کاهش می‌دهد. سرریز به صفر نیز مشکل مشابهی دارد اما شدیدتر است زیرا در لگاریتم‌ها به $-\infty$ می‌انجامد.

۳.۴.۳ محاسبه‌ی کسر تلفات تک‌کیلومتری

این متد در `_physics.py` به صورت یک تابع مستقل پیاده‌سازی شده است:

```

1  def compute_single_km_power_loss_fraction(channel: Any) -> float:
2      alpha = get_fiber_loss_db_km(channel)
3      if not is_finite_non_negative(alpha):
4          raise ParameterValidationError(
5              f"single_km_power_loss_fraction requires finite non-negative "
6              f"alpha, got {alpha!r}",
7              param_name="fiber_loss_db_km",
8              param_value=alpha,
9          )
10     if alpha == 0.0:
11         return 0.0
12     return -math.expm1(-alpha * math.log(0.10) / (0.10
13

```

محاسبه‌ی ریاضی آن بر پایه‌ی رابطه‌ی (۱۲.۳) است. شاخه‌ی `alpha == 0.0` برای جلوگیری از محاسبه‌ی `expm1(0)` است که همیشه ۰ را برمی‌گرداند اما صرفه‌جویی در زمان محاسبه می‌کند. مهم‌تر از آن، این شاخه‌ی صریح، رفتار را در حالت $\alpha = 0$ (کانال بدون تلفات) تضمین می‌کند.

۴.۴.۳ تقریب خطی تلفات و هشدارِ منسوخ‌سازی

متد `compute_linear_loss_per_km` در `_physics.py` به صورت زیر پیاده‌سازی شده:

```

1  def compute_linear_loss_per_km(channel: Any) -> float:
2      global _LINEAR_LOSS_WARNED
3      with _LINEAR_LOSS_LOCK:
4          if not _LINEAR_LOSS_WARNED:
5              warnings.warn(
6                  "FiberChannel.linear_loss_per_km() is deprecated; use "
7                  "loss_fraction_per_km() for the exact per-km power-loss "
8                  "fraction...",
9                  DeprecationWarning,
10                 stacklevel=2,
11             )
12             _LINEAR_LOSS_WARNED = True
13
14     alpha = get_fiber_loss_db_km(channel)
15     val = alpha * math.log(0.10) / 0.10
16     if val > 0.1:
17         msg = (f"linear_loss_per_km() returned {val6.:g} > 0.1 for "
18             alpha={alpha6.:g}...)
19         if get_strict_mode():
20             raise ParameterValidationError(msg, ...)
21         _warn_physical_suspicion(msg, stacklevel=2)
22         val = 0.1 # clamp to physical range [0, 1]
23     return val
24

```

این متد سه ویژگی دارد. نخست، هشدارِ منسوخ‌سازی تنها یک‌بار در هر پردازش (`process`) با استفاده از `threading.Lock` برای `emit thread-safety` می‌شود. دوم، مقدار بازگردانده‌شده بر اساس رابطه‌ی (۱۳.۳) محاسبه می‌شود که واقعاً خطی است. سوم، اگر مقدار از ۱ فراتر رود (که برای $\alpha > 4.34$ dB/km رخ می‌دهد)، در حالت `strict` استثنا پرتاب می‌شود و در غیر این صورت، مقدار به ۱ بریده (`clamp`) می‌شود.

۵.۴.۳ ترکیب آبخاری کانال‌ها

این متد در `physics.py` به صورت زیر پیاده‌سازی شده:

```

1 def compute_cascade(channel: Any, other: Any) -> Any:
2     if not isinstance(other, type(channel)):
3         raise TypeError(f"cascade requires a {type(channel).__name__} argument,
4         ...")
5     total_loss_db = get_total_loss_db(channel) + get_total_loss_db(other)
6     total_distance = get_distance_km(channel) + get_distance_km(other)
7     if total_distance == :0.0
8         instance = type(channel)(
9             AttenuationConfig(fiber_length0.0=, attenuation_coefficient(0.0=,
10             _construction_path="cascade",
11         )
12     else:
13         instance = type(channel).from_total_loss(
14             total_distance, total_loss_db, _construction_path="cascade",
15         )
16     merged_metadata: Dict[str, Any] = dict(channel._metadata_extra)
17     merged_metadata.update(other._metadata_extra)
18     if merged_metadata:
19         object.__setattr__(instance, "_metadata_extra",
20         types.MappingProxyType(merged_metadata))
21     if total_distance > :0.0
22         avg_alpha = total_loss_db / total_distance
23         if avg_alpha < 05.0 or avg_alpha > :0.5
24             msg = (f"cascade: The cascaded average alpha={avg_alpha6.:g} dB/km "
25             f"is outside typical fiber ranges 05.0)--5 dB/km)...")
26         if get_strict_mode():
27             raise ParameterValidationError(msg, ...)
28         _warn_physical_suspicion(msg, stacklevel=2)
29     return instance

```

این کد چندین نکته‌ی طراحی را بازتاب می‌دهد. استفاده از `type(channel)` به جای `FiberChannel` به زیرکلاس‌ها اجازه می‌دهد نمونه‌ای از همان نوع زیرکلاس تولید کنند. شاخه‌ی `total_distance == 0.0` برای حالتی است که هر دو کانال طول صفر دارند؛ در این حالت، کانال معادل هویت (`identity`) با $\alpha = 0$ ساخته می‌شود. ترکیب فراداده‌ها با تقدم کانال دوم (`other`) در صورت تداخل کلید انجام می‌شود. هشدار α_{avg} خارج از محدوده، کاربر را از اشتباه گرفتن کانال معادل با فیبر واقعی بازمی‌دارد.

۶.۴.۳ ترکیب آبخاری بدون اعتبارسنجی

متد `compute_cascade_unchecked` برای حلقه‌های Monte Carlo با کارایی بالا طراحی شده:

```

1 def compute_cascade_unchecked(cls: Type[Any], channels: Sequence[Any]) ->
2     Any:
3     if not channels:
4         raise ParameterValidationError("_cascade_unchecked requires at least one
5         channel", ...)
6     for ch in channels:
7         if not isinstance(ch, cls):
8             raise TypeError(f"Expected {cls.__name__}, got {type(ch).__name__}")
9         total_loss = math.fsum(get_total_loss_db(ch) for ch in channels)
10        total_distance = math.fsum(get_distance_km(ch) for ch in channels)
11        if not is_finite_non_negative(total_loss):
12            raise ParameterValidationError(...)

```



```

۱۱     if not is_finite_non_negative(total_distance):
۱۲         raise ParameterValidationError(...)
۱۳     metadata_extra: Dict[str, Any] = {}
۱۴     for ch in channels:
۱۵         metadata_extra.update(ch._metadata_extra)
۱۶     if total_distance > MAX_DISTANCE_KM:
۱۷         raise ParameterValidationError(...)
۱۸     if total_distance == :0.0
۱۹     instance = fast_construct(cls, AttenuationConfig(0.0), (0.0,
۲۰         _construction_path="cascade")
۲۱     else:
۲۲         alpha = total_loss / total_distance
۲۳         if alpha > MAX_FIBER_LOSS_DB_KM:
۲۴             raise ParameterValidationError(...)
۲۵         if alpha < 05.0 or alpha > :0.5
۲۶             _warn_physical_suspicion(...)
۲۷         instance = fast_construct(cls, AttenuationConfig(total_distance, alpha),
۲۸             _construction_path="cascade",
۲۹             _total_loss_db_override=total_loss)
۳۰         _eff_loss = get_total_loss_db(instance)
۳۱         _expected_T = transmittance_from_loss_db(_eff_loss)
۳۲         assert math.isclose(instance._transmittance, _expected_T,
۳۳             rel_tol=0.0, abs_tol=CONTRACT_CHECK_ABS_TOL), \
۳۴             f"(L, T) contract violation after cascade: ..."
۳۵         if metadata_extra:
۳۶             object.__setattr__(instance, "_metadata_extra",
۳۷                 types.MappingProxyType(metadata_extra))
۳۸         return instance
۳۹

```

نکات کلیدی این پیاده‌سازی عبارت‌اند از: (الف) بررسی نوع پیش از دسترسی به ویژگی‌ها برای جلوگیری از `AttributeError`؛ (ب) استفاده از `math.fsum` برای جمع پایدار عددی در هزاران کانال؛ (ج) بررسی `NaN` و `Inf` در جمع تجمعی؛ (د) استفاده از `fast_construct` برای دور زدن اعتبارسنجی هر کانال؛ (ه) `assert` برای بررسی قرارداد (L, T) پس از ساخت (که در حالت `O` حذف می‌شود).

۷.۴.۳ محاسبه‌ی نویز حرارتی

این متد در `noise_models.py` پیاده‌سازی شده:

```

۱     def thermal_voltage_std(self) -> float:
۲         self.validate()
۳         return float(
۴             np.sqrt(
۵                 0.4
۶                 * CONST_BOLTZMANN
۷                 * self.temperature_k
۸                 * self.bandwidth_hz
۹                 * self.load_resistance_ohm
۱۰             )
۱۱         )
۱۲

```

این کد مستقیماً رابطه‌ی (۶.۳) را پیاده‌سازی می‌کند. فراخوانی `self.validate()` پیش از محاسبه، اطمینان حاصل می‌کند که پارامترها در محدوده‌ی فیزیکی هستند. استفاده از `numpy.sqrt` به جای `math.sqrt` به این دلیل است که در صورت عبور یک آرایه (که ممکن است در حالت‌های پردازش برداری رخ دهد)، کد همچنان کار کند.

۸.۴.۳ محاسبه‌ی نویز شات

```

1 def shot_voltage_std(self) -> float:
2     self.validate()
3     if self.monitor_photocurrent_a is None or self.monitor_photocurrent_a ==
:0.0
4         return 0.0
5     i_std = np.sqrt(
6         0.2
7         * CONST_ELECTRON_CHARGE
8         * self.monitor_photocurrent_a
9         * self.bandwidth_hz
10    )
11    return float(i_std * self.load_resistance_ohm)
12

```

این کد رابطه‌ی (۸.۳) را پیاده‌سازی می‌کند. شاخه‌ی بازگشت 0 در صورت نبود جریان، یک بهینه‌سازی کوچک است اما مهم‌تر از آن، از محاسبه‌ی $\sqrt{0}$ و تولید 0.0 جلوگیری می‌کند که در برخی محیط‌ها ممکن است منجر به هشدارهای runtime شود.

۹.۴.۳ ترکیب نویزهای مستقل

```

1 def total_voltage_std(self) -> float:
2     vt = self.thermal_voltage_std()
3     vs = self.shot_voltage_std()
4     return float(np.sqrt(vt * vt + vs * vs))
5

```

این متد رابطه‌ی (۹.۳) را پیاده‌سازی می‌کند. استفاده از $v_t^2 + v_s^2$ به جای $v_t \cdot v_t + v_s \cdot v_s$ (که ریاضیاً معادل است) ممکن است به دلیل بهینه‌سازی پردازنده، اندکی سریع‌تر باشد. این الگوی جمع توان‌ها در همه‌ی مدل‌های نویز مستقل فریم‌ورک استفاده می‌شود.

۱۰.۴.۳ متد from_total_loss و ساخت دو مرحله‌ای

این متد که برای حفظ دقت round-trip طراحی شده، الگوی ساخت دو مرحله‌ای را به کار می‌برد:

```

1 @classmethod
2 def from_total_loss(cls, distance_km, total_loss_db, *,
3 _construction_path="from_total_loss",
4 wavelength_nm=None):
5     coerced_distance = _coerce_numeric(distance_km, param_name="distance_km")
6     coerced_total_loss = _coerce_numeric(total_loss_db,
7 param_name="total_loss_db")
8
9     if not is_finite_non_negative(coerced_distance):
10        raise ParameterValidationError(...)
11    if not is_finite_non_negative(coerced_total_loss):
12        raise ParameterValidationError(...)
13
14    if coerced_distance == :0.0
15    if not is_close(coerced_total_loss, 0.0, abs_tol=_ZERO_LOSS_ABS_TOL):
16        raise ParameterValidationError(f"distance_km=0 requires
total_loss_db=0...")
17    attenuation = AttenuationConfig(fiber_length0.0=,
18 attenuation_coefficient(0.0=

```

```

۱۷     return cls(attenuation=attenuation, _construction_path=_construction_path,
۱۸                wavelength_nm=wavelength_nm)
۱۹
۲۰     if coerced_distance > MAX_DISTANCE_KM:
۲۱         raise ParameterValidationError(...)
۲۲
۲۳     if is_close(coerced_total_loss, 0.0, abs_tol=ZERO_LOSS_ABS_TOL):
۲۴         msg = (f"from_total_loss: distance_km={coerced_distance6.:g} > 0 "
۲۵               f"but total_loss_db={coerced_total_loss6.:g} is zero...")
۲۶         if get_strict_mode():
۲۷             raise ParameterValidationError(msg, ...)
۲۸         _warn_physical_suspicion(msg, stacklevel=2)
۲۹
۳۰     fiber_loss_db_km = coerced_total_loss / coerced_distance
۳۱     if not math.isfinite(fiber_loss_db_km):
۳۲         raise ParameterValidationError(...)
۳۳     if fiber_loss_db_km > MAX_FIBER_LOSS_DB_KM:
۳۴         raise ParameterValidationError(...)
۳۵
۳۶     attenuation = AttenuationConfig(fiber_length=coerced_distance,
۳۷                                     attenuation_coefficient=fiber_loss_db_km)
۳۸
۳۹     # Two-phase construction: skip health check during __post_init__
۴۰     _skip_health_token = _SKIP_TRANSMITTANCE_HEALTH.set(True)
۴۱     try:
۴۲         instance = cls(attenuation=attenuation,
۴۳                        _construction_path=_construction_path,
۴۴                        wavelength_nm=wavelength_nm)
۴۵     finally:
۴۶         _SKIP_TRANSMITTANCE_HEALTH.reset(_skip_health_token)
۴۷     object.__setattr__(instance, "_total_loss_db_override",
coerced_total_loss)
۴۸     new_t = cls._transmittance_from_loss_db(coerced_total_loss)
۴۹     object.__setattr__(instance, "_transmittance", new_t)
۵۰     instance._check_transmittance_health(new_t)
۵۱     return instance
۵۲

```

این متد سه ویژگی مهم دارد. نخست، الگوی ContextVar به نام `_SKIP_TRANSMITTANCE_HEALTH` برای دور زدن بررسی سلامت در `__post_init__` استفاده می‌شود؛ این کار از هشدارهای متناقض جلوگیری می‌کند (مقدار موقتاً محاسبه شده از $\alpha \cdot L$ ممکن است متفاوت از مقدار نهایی محاسبه شده از `override` باشد). دوم، `override` پس از ساخت نمونه با `object.__setattr__` تنظیم می‌شود. سوم، بررسی نهایی سلامت روی مقدار نهایی انجام می‌شود.

۱۱.۴.۳ متد `_fast_construct`

این متد برای حلقه‌های Monte Carlo طراحی شده و تمام اعتبارسنجی‌ها را دور می‌زند:

```

۱     @classmethod
۲     def _fast_construct(cls, attenuation, *,
۳                        _construction_path="fast_construct",
۴                        _total_loss_db_override=None,
۵                        _wavelength_nm=None):
۶         if not isinstance(attenuation, AttenuationConfig):
۷             raise TypeError(f"_fast_construct requires an AttenuationConfig, got ...")
۸         instance = object.__new__(cls)
۹         object.__setattr__(instance, "attenuation", attenuation)

```

```

10     object.__setattr__(instance, "_construction_path", _construction_path)
11     object.__setattr__(instance, "_total_loss_db_override",
    _total_loss_db_override)
12     object.__setattr__(instance, "_strict_mode_at_construction",
    get_strict_mode())
13     object.__setattr__(instance, "_metadata_extra", _EMPTY_METADATA_EXTRA)
14     object.__setattr__(instance, "wavelength_nm", _wavelength_nm)
15
16     total_db = (_total_loss_db_override if _total_loss_db_override is not None
17 else attenuation.total_loss_db)
18     t = cls._transmittance_from_loss_db(total_db)
19     if not math.isfinite(t):
20         raise ParameterValidationError(...)
21     if not 0.0 <= t <= (0.1:
22         raise ParameterValidationError(...)
23     if 0.0 < t < TRANSMITTANCE_SUBNORMAL_THRESHOLD:
24         logger.debug("_fast_construct: computed transmittance %r is subnormal...")
25     object.__setattr__(instance, "_transmittance", t)
26     return instance
27

```

این متد از `object.__new__(cls)` به جای فراخوانی سازنده‌ی کلاس استفاده می‌کند تا `__post_init__` اجرا نشود. سپس تمام فیلدها به صورت دستی با `object.__setattr__` تنظیم می‌شوند. با وجود دور زدن اعتبارسنجی، یک بررسی حداقلی `isfinite` و بازه‌ی $[0, 1]$ روی ضریب انتقال انجام می‌شود تا از تولید نمونه‌های کاملاً نادرست جلوگیری شود.

۱۲.۴.۳ متد `validate`

این متد برای بررسی سازگاری حالت مشتق شده پس از عملیاتی که ممکن است آن را فاسد کند، طراحی شده:

```

1     def validate(self) -> None:
2         self.validate_parameter("distance_km", self.distance_km, MAX_DISTANCE_KM)
3         self.validate_parameter("fiber_loss_db_km", self.fiber_loss_db_km,
4 MAX_FIBER_LOSS_DB_KM)
5
6         if self._total_loss_db_override is None:
7             expected_total_db = (self.attenuation.fiber_length
8 * self.attenuation.attenuation_coefficient)
9             if not is_close(self.total_loss_db, expected_total_db,
10 rel_tol=_CONTRACT_CHECK_REL_TOL,
11 abs_tol=_CONTRACT_CHECK_ABS_TOL):
12                 raise ParameterValidationError(f"Derived state inconsistency: ...")
13
14             expected_transmittance =
15 self._transmittance_from_loss_db(self.total_loss_db)
16
17             if expected_transmittance == 0.0 and self.transmittance == :0.0
18                 logger.warning("validate(): both expected and stored transmittance are
19 ...0.0")
20             elif not is_close(self.transmittance, expected_transmittance,
21 rel_tol=1e-9,
22 abs_tol=_TRANSMITTANCE_UNDERFLOW_ABS_TOL):
23                 raise ParameterValidationError(f"Derived state inconsistency:
24 transmittance...")
25
26             self._check_transmittance_health(self._transmittance)
27

```

```

۲۵     if 0.0 < self.transmittance < TRANSMITTANCE_SUBNORMAL_THRESHOLD:
۲۶         subnormal_warning = (f"validate(): Transmittance {self.transmittance3.:e}
۲۷         "
۲۸         f"is subnormal...")
۲۹         if get_strict_mode():
۳۰             raise ParameterValidationError(subnormal_warning, ...)
۳۱         _warn_physical_suspicion(subnormal_warning, stacklevel=2)

```

این متد سه سطح بررسی دارد: بازهی پارامترها، قراردادی $L = \alpha \cdot L$ (تنها در صورت نبود override)، و سازگاری ضریب انتقال. تلورانس `_TRANSMITTANCE_UNDERFLOW_ABS_TOL` به‌عنوان تلورانس مطلق برای همه‌ی مقایسه‌ها (حتی برای مقادیر نرمال) استفاده می‌شود تا مقادیر زیرنرمال که تنها 1 یا 2 بیت معتبر دارند، به‌اشتباه ناسازگار تشخیص داده نشوند.

۱۳.۴.۳ متد `link_efficiency`

این تابع کمکی ترکیب سه‌گانه‌ی رابطه‌ی (۳.۳) را متمرکز می‌کند:

```

۱     def link_efficiency(channel_transmittance, detector_efficiency,
۲     coupling_efficiency(0.1=:
۳     result = float(channel_transmittance) * float(detector_efficiency) \
۴     * float(coupling_efficiency)
۵     if result < :0.0
۶     return 0.0
۷     if result > :0.1
۸     return 0.1
۹     return result
۱۰

```

برش به بازهی $[0, 1]$ در اینجا عمدی است: اگر کاربر به‌اشتباه یک بازدهی آشکارساز بزرگ‌تر از 1 ارسال کند، نتیجه به‌جای پرتاب استثنا به 1 بریده می‌شود. این رفتار برای حلقه‌های Monte Carlo که در آنجا سرعت مهم‌تر از تشخیص خطاست، مناسب است.

۱۴.۴.۳ توابع برداری `transmittance_array` و `transmittance_scalar`

این دو تابع برای حلقه‌های Monte Carlo با کارایی بالا طراحی شده‌اند:

```

۱     def transmittance_array(distances_km, fiber_loss_db_km):
۲     distances_km = np.asarray(distances_km, dtype=np.float64)
۳     if fiber_loss_db_km == :0.0
۴     return np.ones_like(distances_km)
۵     total_loss_db = fiber_loss_db_km * distances_km
۶     return np.float_power(0.10), -total_loss_db / (0.10)
۷
۸     def transmittance_scalar(distance_km, fiber_loss_db_km):
۹     if fiber_loss_db_km == 0.0 or distance_km == :0.0
۱۰    return 0.1
۱۱    total_db = fiber_loss_db_km * distance_km
۱۲    return math.pow(0.10), -total_db / (0.10)
۱۳

```

تابع `transmittance_array` از قابلیت `NumPy vectorization` استفاده می‌کند و برای محاسبه‌ی همزمان ضرایب انتقال برای هزاران فاصله طراحی شده. تابع `transmittance_scalar` تنها از `math.pow` و حساب پایه استفاده می‌کند تا با `@njit` Numba سازگار باشد. هیچ‌کدام از این توابع هشدار اعتبارسنجی `emit` نمی‌کنند زیرا در حلقه‌های داغ، هزینه‌ی هشدارهای نقطه‌به‌نقطه بیش از حد است.

۵.۳ ارسال و دریافت با سایر ماژول‌ها

این سه ماژول با طیف گسترده‌ای از سایر اجزای فریم‌ورک در ارتباط هستند. در این بخش، این ارتباطات را به تفصیل بررسی می‌کنیم.

۱.۵.۳ وابستگی‌های ورودی به channel.py

ماژول `qkd.exceptions`: دو استثنا `ConfigurationError` و `ParameterValidationError` از این ماژول وارد می‌شوند. این دو استثنا نقش‌های متفاوتی دارند: `ParameterValidationError` برای خطاهای مربوط به مقدار یک پارامتر خاص (مانند `distance_km = -1`) استفاده می‌شود، در حالی که `ConfigurationError` برای خطاهای ساختاری پیکربندی (مانند بخش گم‌شده‌ی `channel_config`) به کار می‌رود. علاوه بر این، کلاسی `ChannelError` که در `channel.py` به صورت محلی تعریف شده، به عنوان کلاسی پایه برای تمام استثناهای مرتبط با کانال عمل می‌کند (هرچند که در نسخه‌ی فعلی، `ConfigurationError` و `ParameterValidationError` هنوز از آن ارث نمی‌برند). تا این ارث‌بری پیاده‌سازی شود، تاپل `CHANNEL_ERRORS = (ConfigurationError, ParameterValidationError)` به عنوان سازوکار ایمن `catch-all` توصیه می‌شود.

ماژول `qkd.constants`: دو تابع `is_finite_non_negative` و `is_close` از این ماژول وارد می‌شوند. `is_close` برای مقایسه‌های مبتنی بر تلورانس در تساوی و بررسی قراردادهای استفاده می‌شود. `is_finite_non_negative` برای اعتبارسنجی پارامترها به کار می‌رود و اطمینان حاصل می‌کند که مقادیر `NaN`، `Inf` و منفی رد می‌شوند.

ماژول `qkd.datatypes`: کلاسی `AttenuationConfig` از این ماژول وارد می‌شود. این کلاس یک `dataclass` ساده با دو فیلد `fiber_length` و `attenuation_coefficient` است. قرارداد آن این است که `total_loss_db = fiber_length * attenuation_coefficient` باید برقرار باشد.

ماژول `_schema (داخلی)`: این ماژول ثابت‌ها، تاپ‌ها و توابع پایه‌ای را که در نسخه‌ی تجزیه‌شده‌ی `channel.py` به اشتراک گذاشته می‌شوند، فراهم می‌سازد. از جمله می‌توان به `TRANSMITTANCE_SUBNORMAL_THRESHOLD`، `MAX_DISTANCE_KM`، `MAX_FIBER_LOSS_DB_KM`، `_CONTRACT_CHECK_ABS_TOL`، تاپ‌های `Transmittance`، `Distance`، `Loss` و `Numeric`، و توابع `get_strict_mode` و `warn_physical_suspicion` اشاره کرد.

ماژول `_gating (داخلی)`: توابع `transmittance_from_loss_db`، `validate_parameter`، `check_transmittance_health` و `effective_total_loss_db` از این ماژول وارد می‌شوند. این توابع لایه‌ی "گیتینگ" (`gating`) را تشکیل می‌دهند که اعتبارسنجی و محاسبات پایه‌ای را فراهم می‌سازد.

ماژول `_casting (داخلی)`: تابع `fast_construct` از این ماژول وارد می‌شود. این تابع همانند متد `_fast_construct` در کلاس، اما در سطح ماژول عمل می‌کند.

بسته‌های استاندارد: `contextlib` برای مدیریت زمینه، `ContextVar` برای `contextvars`، `dataclasses` برای `dataclasses`، `logging` برای لاگ‌گذاری، `math` برای توابع ریاضی، `types` برای `MappingProxyType`، `warnings` برای هشدارها، و `numpy` برای محاسبات برداری.

۲.۵.۳ وابستگی‌های ورودی به noise_models.py

ماژول `qkd.exceptions`: استثنا `ParameterValidationError` وارد می‌شود.

ماژول `qkd.constants`: دو ثابت فیزیکی `CONST_BOLTZMANN` و `CONST_ELECTRON_CHARGE` وارد می‌شوند. این ثابت‌ها در `constants_definitions.py` با مقادیر دقیق SI تعریف شده‌اند.

بسته‌های استاندارد: `dataclasses` برای `dataclasses`، `typing` برای `Optional`، و `numpy` برای `sqrt`.

۳.۵.۳ خروجی‌ها و مصرف‌کنندگان

نمونه‌ی `FiberChannel` پس از ساخت، در سراسر فریم‌ورک به عنوان مرجع مرکزی مدل کانال استفاده می‌شود. مصرف‌کنندگان اصلی عبارت‌اند از:

موتور شبیه‌سازی اصلی (`main_optimized.py`): این موتور از `FiberChannel` برای استخراج `distance_km`، `fiber_loss_db_km`، `total_loss_db`، `transmittance` و `build_channel_sim_params` تمام این کمیت‌ها را در یک `ChannelSimParams` یخ‌زده بسته‌بندی می‌کند.

تا موتور شبیه‌سازی همیشه از یک تصویر لحظه‌ای یکپارچه بخواند. این طراحی از خطر خواندن مقادیر قدیمی از دیکشنری پیکربندی در طول شبیه‌سازی جلوگیری می‌کند.

ماژول‌های اثبات امنیتی (proofs/): هر یک از ماژول‌های اثبات (مانند Ma 2005, Lim 2014 و MDI-QKD) از transmittance برای محاسبه بهره‌های Y_n و نرخ‌های خطای e_n استفاده می‌کنند. به‌عنوان مثال، در اثبات Ma 2005، نرخ کلید مجانبی از رابطه‌ی (۴.۳) محاسبه می‌شود که در آن η به‌طور مستقیم از $\text{link_efficiency}(\text{channel.transmittance}, \text{detector_efficiency}, \text{coupling_efficiency})$ به دست می‌آید. استفاده از تابع متمرکز link_efficiency تضمین می‌کند که تمام اثبات‌ها از یک ترکیب‌بندی واحد استفاده می‌کنند و از ناسازگاری پارامترها جلوگیری می‌شود.

ماژول آشکارساز (qkd.detectors): این ماژول از ChannelSimParams برای استخراج پارامترهای مرتبط با کانال (مانند $\text{dispersion_parameter_ps_nm_km}$) که در مدل‌سازی پنجره‌ی زمانی آشکارساز استفاده می‌شوند، بهره می‌برد. مقادیر transmittance و total_loss_db نیز در محاسبه‌ی نرخ شمارش تاریک مؤثر (effective dark count rate) که به دلیل نشت فوتون از کانال کلاسیک مجاور در سیستم‌های DWDM افزایش می‌یابد، استفاده می‌شوند.

ماژول Privacy Amplification: این ماژول از total_loss_db برای تخمین ϵ_{PE} (خطای تخمین پارامتر) استفاده می‌کند؛ هرچه تلفات کل بیشتر باشد، تعداد رویدادهای قابل مشاهده کمتر شده و ϵ_{PE} باید بزرگ‌تر در نظر گرفته شود.

ماژول‌های لاگ‌گذاری و بازتولید: متد to_config_dict برای ذخیره‌ی پیکربندی در فایل‌های JSON به کار می‌رود. این فایل‌ها امکان بازتولید دقیق آزمایش‌ها را فراهم می‌سازند. متد __repr__ و __str__ نیز برای لاگ‌های خوانا استفاده می‌شوند؛ این متدها برای مقادیر بسیار کوچک T به‌طور خودکار به نمای علمی تغییر می‌کنند.

ماژول ElectricalNoiseConfig: این کلاس به‌طور مستقل توسط ماژول منبع نوری (qkd.sources) برای مدل‌سازی نویز الکتریکی فوتودیود نظارتی استفاده می‌شود. خروجی $\text{total_voltage_std}()$ به موتور شبیه‌سازی ارسال می‌شود تا به‌عنوان نویز افزودنی در مدل آشکارساز اعمال گردد.

۴.۵.۳ جریان داده از ورودی تا خروجی

برای روشن‌تر شدن ارتباطات، جریان داده را می‌توان به‌صورت زیر خلاصه کرد:

۱. **ورودی:** کاربر یک دیکشنری پیکربندی (یا یک فایل JSON) ارائه می‌دهد که شامل کلیدهای fiber_loss_db_km ، distance_km (یا total_loss_db) و به‌اختیار wavelength_nm است.

۲. **بارگذاری:** متد from_config_dict دیکشنری را دریافت کرده و با استفاده از coerce_numeric مقادیر را به float تبدیل می‌کند. در صورت وجود total_loss_db ، از from_total_loss استفاده می‌کند تا دقت round-trip حفظ شود.

۳. **اعتبارسنجی:** سازنده‌ی FiberChannel متد __post_init__ را فراخوانی می‌کند که به‌نوبه‌ی خود هفت مرحله‌ی اعتبارسنجی را اجرا می‌کند: بررسی نوع، بازه‌ی پارامترها، معناداری α ، معناداری فاصله، محدودبودن تلفات کل، قرارداد $L = \alpha \cdot L$ ، و سازگاری طول موج.

۴. **محاسبه‌ی ضریب انتقال:** متد $\text{compute_transmittance}$ مقدار T را محاسبه کرده و در transmittance ذخیره می‌کند. سپس متد $\text{check_transmittance_health}$ بررسی چهارمرحله‌ای NaN، زیرنرمال، سرریز و بازه را انجام می‌دهد.

۵. **استفاده:** نمونه‌ی تأییدشده به موتور شبیه‌سازی، ماژول‌های اثبات امنیتی و ماژول آشکارساز ارسال می‌شود. تابع $\text{build_channel_sim_params}$ تمام کمیت‌های لازم را در یک ChannelSimParams بچ‌زده بسته‌بندی می‌کند.

۶. **ذخیره‌سازی:** متد to_config_dict پیکربندی نهایی را برای بازتولید آینده ذخیره می‌کند. این خروجی شامل schema_version ، فراداده‌ی زمان ساخت و تمام پارامترهای فیزیکی است.

۷. **لاگ‌گذاری:** متد __repr__ یک نمایش خوانا از کانال تولید می‌کند که در لاگ‌های شبیه‌سازی استفاده می‌شود.

۵.۵.۳ مدیریت نسخه‌گذاری و سازگاری پسین

فیلد `_metadata.version` در خروجی `to_config_dict` مدیریت می‌شود. در هنگام بارگذاری، اگر نسخه‌ی ذخیره‌شده با نسخه‌ی فعلی ("`__version__`" = "4.2.1") متفاوت باشد، یک هشدار ثبت می‌شود. علاوه بر این، چندین سازوکار سازگاری پسین پیاده‌سازی شده است: نام قدیمی `STRICT_MODE` (که اکنون یک `ContextVar` است) از طریق `__getattr__` در سطح ماژول (PEP 562) پشتیبانی می‌شود و هشدار منسوخ‌سازی `emit` می‌کند. نام قدیمی `loss_fraction_per_km` همچنان به‌عنوان نام مستعار منسوخ‌شده‌ی `single_km_power_loss_fraction` کار می‌کند. متد `from_config` نیز با هشدار منسوخ‌سازی پشتیبانی می‌شود و کاربران به استفاده‌ی مستقیم از سازنده تشویق می‌شوند.

۶.۵.۳ رفتار در برابر خطا و پیام‌های تشخیصی

یکی از ویژگی‌های برجسته‌ی این ماژول، کیفیت پیام‌های خطا است. تمامی استثناها با `context` دقیق تولید می‌شوند: نام پارامتر نامعتبر، مقدار دریافتی و نوع آن، نوع مورد انتظار، مقادیر مجاز (در صورت وجود)، و پیشنهاد‌های هوشمند در صورت وجود خطای تایپی نزدیک. این رویکرد، تجربه‌ی کاربر را به‌طور چشمگیری بهبود می‌بخشد. علاوه بر این، استفاده‌ی متمایز از `logger.warning` (برای موارد فنی قابل‌توجه اما غیربحرانی)، `warn_physical_suspicion` (برای موارد فیزیکی مشکوک) و `raise` (برای موارد غیرقابل‌قبول) به کاربر اجازه می‌دهد تا به‌سرعت تشخیص دهد که آیا شبیه‌سازی می‌تواند ادامه یابد یا خیر. در حالت `strict mode`، تمام هشدارهای `warn_physical_suspicion` به استثنا تبدیل می‌شوند تا در خطوط لوله‌ی تولید، هیچ پیکربندی مشکوکی پذیرفته نشود.

۷.۵.۳ جمع‌بندی معماری

به‌عنوان جمع‌بندی، این سه ماژول با طراحی چندلایه، تغییرناپذیر و نوع-امن خود، زیربنای پایدار فریم‌ورک شبیه‌سازی QKD را در لایه‌ی کانال فراهم می‌سازند. تفکیک سه‌گانه‌ی `channel.py` (کلاس اصلی و توابع کمکی)، `_physics.py` (توابع فیزیکی مستقل) و `noise_models.py` (مدل‌های نویز الکتریکی) الگوی تمیزی برای سازماندهی منطق پیچیده در پروژه‌های بزرگ فراهم می‌کند. کلاس `FiberChannel` با بیش از پنجاه متد و فیلد دقیقاً نوع‌دار، یک ظرف پیکربندی کامل و قابل اتکا فراهم می‌سازد که در عین سادگی استفاده، قدرت بیان کامل پیچیدگی‌های فیزیکی و عددی مدل‌سازی کانال کوانتومی را دارد. کلاس `ElectricalNoiseConfig` به‌عنوان یک ماژول مستقل اما مرتبط، امکان مدل‌سازی نویز الکتریکی سمت منبع را بدون افزوده‌کردن پیچیدگی به کلاس اصلی کانال فراهم می‌سازد. در مجموع، این معماری تعادلی ظریف میان دقت فیزیکی، پایداری عددی، کارایی محاسباتی و قابلیت استفاده فراهم می‌سازد که برای شبیه‌سازی‌های QKD در سطح تولید ضروری است.

فصل ۴

منابع نوری

۱.۴ نمای کلی و هدف ماژول

ماژول `sources.py` همراه ماژول کمکی `modulators.py`، لایه‌ی پایه‌ای مدل‌سازی منابع نوری را در فریم‌ورک شبیه‌سازی توزیع کلید کوانتومی (QKD) تشکیل می‌دهند. این دو فایل با هم، چرخه‌ی کامل تولید پالس نوری، اعمال ناکامل‌های کلاسیک شدت، تبدیل به تعداد فوتون و نمونه‌برداری آماری را پوشش می‌دهند. خروجی این ماژول، ورودی اصلی ماژول کانال `channel.py` است که در آن ضریب انتقال فیبر بر تعداد میانگین فوتون اعمال می‌شود و سپس به آشکارسازها ارسال می‌گردد.

هدف بنیادین این ماژول، ارائه‌ی یک مدل واحد، تغییرناپذیر (`immutable`) و قابل سریال‌سازی از منبع نوری است که هم برای منابع لیزری تضعیف‌شده با آمار پواسون و هم برای منابع نور گرمایی با آمار بوز-آیشتین و هم برای منابع حالت-کوانتومی-کلی با ماتریس چگالی در پایه‌ی فوک کار کند. این عموم‌بخشی از طریق یک سلسله‌مراتب وراثتی با کلاس پایه‌ی `OpticalSource` و دو زیرکلاس `DensityMatrixSource` و `PoissonSource` محقق می‌شود. ماژول `modulators.py` نیز کلاس `MZMConfig` را برای مدل‌سازی مودولاتور ماخ-زندر (Mach-Zehnder Modulator) فراهم می‌سازد که یکی از اجزای کلیدی لایه‌ی ناکامل‌های کلاسیک است.

حوزه‌ی `responsibility` این ماژول به‌دقت محدود شده است: این ماژول صراحتاً مدل‌سازی تصادفی‌سازی فاز، کانال‌های جانبی طیفی/زمانی/قطبش، همدوسی فاز میان پالس‌ها و پدیده‌ی `afterpulsing` (که یک اثر آشکارسازی است) را پوشش نمی‌دهد. این تصمیم طراحی بر اساس اصل تک‌مسئولیتی (`Single Responsibility Principle`) اتخاذ شده است تا هر ماژول تنها یک لایه‌ی فیزیکی را پوشش دهد و وابستگی‌های متقابل نامطلوب ایجاد نکند. به‌خصوص، تصادفی‌سازی فاز که در اثبات‌های امنیتی `BB84` `Decoy-State` فرض بنیادین است، در این ماژول صرفاً به‌عنوان یک پرچم فراداده‌ای (`phase_randomization_assumed`) ثبت می‌شود؛ این پرچم یک `claim` از سوی کاربر است که تجهیزات خارجی وی تصادفی‌سازی فاز هر پالس را تضمین می‌کند، نه یک ویژگی پیاده‌سازی‌شده در کد.

معماری داخلی ماژول بر دو لایه‌ی مجزا استوار است که تا حد امکان درون یک انتزاع واحد نگه داشته می‌شوند. لایه‌ی نخست، `State Preparation` یا انتخاب نوع پالس است که مشخص می‌کند کدام پالس (سیگنال یا یکی از دکوی‌ها) با چه احتمال و چه میانگین فوتون پایه تولید شود. لایه‌ی دوم، `Photon-Number Sampling` است که بر اساس آمار پواسون، گرمایی/هندسی یا قطر ماتریس چگالی در پایه‌ی فوک، تعداد فوتون را نمونه‌برداری می‌کند و سپس به‌صورت اختیاری، ناکامل‌های کلاسیک شدت (تضعیف `MZM`، تقسیم کانال، ضریب باند جانبی `SCM`، جیتر شدت و شکست گسیل) را اعمال می‌کند. این جداسازی به کاربر اجازه می‌دهد بدون تغییر در لایه‌ی فیزیکی، پیکربندی پالس‌ها را به‌صورت مستقل تنظیم کند.

یک نکته‌ی بحرانی درباره‌ی `phase_randomization_assumed`: این پرچم در اصلحیه‌ی `F-04` از `phase_randomized` تغییر نام یافت تا ماهیت صرفاً فراداده‌ای آن به‌روشنی در نام بازتاب یابد. وقتی این پرچم `True` باشد، کاربر اطمینان می‌دهد که نمونه‌برداری قطری پواسون/گرمایی با اثبات امنیتی سازگار است زیرا تصادفی‌سازی فاز، هر پالس را به یک مخلوط قطری از حالت‌های فوک تبدیل می‌کند. وقتی `False` باشد، نمونه‌برداری قطری برای تحلیل امنیتی `BB84` `Decoy-State` مناسب نیست و کاربر باید از `DensityMatrixSource` با ماتریس‌های چگالی کامل استفاده کند. این تمایز در تحلیل‌های امنیتی حیاتی است و نادیده‌گرفتن آن می‌تواند به تخمین نادرست نرخ کلید منجر شود.

۲.۴ مبانی تئوری و فیزیکی

۱.۲.۴ آمار پواسون و منابع لیزری تضعیف شده

منبع استاندارد در پروتکل های QKD، لیزری است که به شدت تضعیف می شود تا میانگین تعداد فوتون به ازای پالس به حدود $1 < \mu$ برسد. چنین منبعی به دلیل ماهیت آماری فرآیند گسیل خودبه خودی spontaneous emission در محیط بهره ی لیزر، از آمار پواسون پیروی می کند. تابع توزیع احتمال تعداد فوتون در یک پالس با میانگین μ به صورت زیر بیان می شود:

$$P(n; \mu) = \frac{\mu^n e^{-\mu}}{n!}, \quad n = 0, 1, 2, \dots \quad (۱.۴)$$

که در آن μ میانگین تعداد فوتون، n عدد فوتون و e پایه ی لگاریتم طبیعی است. این توزیع دارای ویژگی های مهمی است: میانگین و واریانس هر دو برابر μ هستند ($\mathbb{E}[n] = \text{Var}(n) = \mu$)، که به معنای پراکندگی نسبتاً پایین است. برای $\mu = 0.5$ (یک مقدار متداول در BB84-Decoy)، احتمال پالس خالی $P(0) = e^{-0.5} \approx 0.607$ است، احتمال تک فوتونی $P(1) = 0.5 \cdot e^{-0.5} \approx 0.303$ و احتمال چندفوتونی $P(n \geq 2) \approx 0.090$. این کسر غیرقابل چشم پوشی پالس های چندفوتونی، پایه ی حمله ی Photon-Number Splitting (PNS) است که Decoy-State برای مقابله با آن معرفی شده است.

محاسبه ی عددی پایدار توزیع پواسون برای μ های بزرگ یا n های بالا نیاز به احتیاط دارد. محاسبه ی مستقیم $\mu^n/n!$ به سرریز (overflow) می انجامد زیرا برای $\mu = 10$ و $n = 20$ مقدار $10^{20}/20! \approx 4.1 \times 10^{-4}$ است اما فاکتوریل 20 به خودی خود $\approx 2.4 \times 10^{18}$ است. پیاده سازی این فریم ورک از روش بازگشتی پایدار عددی استفاده می کند:

$$P(0; \mu) = e^{-\mu}, \quad (۲.۴)$$

$$P(n+1; \mu) = P(n; \mu) \cdot \frac{\mu}{n+1}, \quad (۳.۴)$$

که در آن ابتدا $P(0)$ از طریق $\text{math.exp}(-\mu)$ محاسبه می شود (که برای μ های معمول در QKD، یعنی $\mu < 10$ ، هیچ خطر سرریز ندارد) و سپس هر جمله از جمله ی قبلی با ضرب در $\mu/(n+1)$ به دست می آید. این روش علاوه بر پایداری عددی، از مزیت سرعت نیز برخوردار است زیرا نیازی به محاسبه ی فاکتوریل یا توان نیست و هر مرحله تنها یک ضرب و یک تقسیم دارد.

۲.۲.۴ آمار گرمایی (بوز-آیشتین) و منابع نور گرمایی

برخلاف لیزر که گسیل تحریکی همدوس دارد، منابع نور گرمایی (مانند blackbody یا chaotic light) از آمار بوز-آیشتین پیروی می کنند. توزیع تعداد فوتون در یک مد گرمایی تک فرکانس با میانگین μ به صورت زیر است:

$$P(n; \mu) = \frac{\mu^n}{(1+\mu)^{n+1}}, \quad n = 0, 1, 2, \dots \quad (۴.۴)$$

که این توزیع با توزیع هندسی با پارامتر $p = 1/(1+\mu)$ معادل است. ویژگی بارز این توزیع، واریانس بزرگتر از میانگین است: $\text{Var}(n) = \mu + \mu^2 = \mu(1+\mu)$ که برای $\mu = 0.5$ برابر با 0.75 است (در مقایسه با 0.5 در پواسون). این پراکندگی بیشتر، منابع گرمایی را برای QKD نامطلوب می سازد زیرا کسر پالس های چندفوتونی به طور قابل توجهی بالاتر است: برای $\mu = 0.5$ ، $P(n \geq 2) \approx 0.222$ در مقایسه با 0.090 در پواسون. با این حال، مدل سازی منابع گرمایی در فریم ورک برای دو هدف ضروری است: نخست، تحلیل امنیتی در برابر حمله ای که مهاجم منبع سیگنال را با منبع گرمایی جایگزین کند؛ و دوم، مدل سازی پس زمینه ی تابشی محیطی که در آشکارسازها به عنوان نویز ظاهر می شود. مانند پواسون، محاسبه ی پایدار عددی از طریق بازگشت انجام می شود:

$$P(0; \mu) = \frac{1}{1+\mu}, \quad (۵.۴)$$

$$P(n+1; \mu) = P(n; \mu) \cdot \frac{\mu}{1+\mu}, \quad (۶.۴)$$

که در آن نسبت $\mu/(1+\mu)$ یک بار پیش محاسبه می شود و سپس هر جمله از جمله ی قبلی با یک ضرب به دست می آید. این روش برای μ های بزرگ نیز پایدار است زیرا نسبت همواره کوچکتر از ۱ است.

۳.۲.۴ ماتریس چگالی در پایه فوک و مدل سازی حالت های غیرکلاسیک

برای منابعی که حالت های غیرکلاسیک مانند حالت های تک فوتونی واقعی (Fock states)، حالت های فشرده (squeezed states) یا حالت های گرهی شرودینگر (cat states) تولید می کنند، آمار پواسون یا گرمایی کافی نیست. این فریم ورک با کلاس DensityMatrixSource این حالت ها را پشتیبانی می کند که در آن هر پالس با یک ماتریس چگالی ρ در پایه فوک $\{|0\rangle, |1\rangle, |2\rangle, \dots\}$ مشخص می شود. قطر این ماتریس، توزیع احتمال تعداد فوتون را تشکیل می دهد:

$$p_n = \langle n | \rho | n \rangle = \rho_{nn}, \quad \sum_n p_n = \text{Tr}(\rho) = 1. \quad (۷.۴)$$

اعتبار فیزیکی ماتریس چگالی نیازمند سه شرط است: هرمیتی بودن ($\rho = \rho^\dagger$)، مثبت معین نیمه ($\rho \succeq 0$) و یک بودن رد ($\text{Tr}(\rho) = 1$). این سه شرط در تابع کمکی validate_density_matrix بررسی می شوند. هرمیتی بودن از طریق مقایسه ρ با $\rho^\dagger$ در تلورانس عددی تأیید می شود. مثبت معین بودن از طریق محاسبه مقدار ویژه کوچکتر و بررسی غیرمنفی بودن آن تأیید می شود. یک بودن رد نیز با مجموع قطر بررسی می گردد. عنصرهای خارج از قطر (ρ_{mn} برای $m \neq n$) که نشانگر همدوسی کوانتومی میان حالت های فوک هستند، در فرآیند نمونه برداری (photon-number sampling) نادیده گرفته می شوند زیرا اندازه گیری تعداد فوتون یک اندازه گیری قطری در پایه فوک است و همدوسی ها را از بین می برد. با این حال، این عنصرها برای تحلیل های امنیتی پیشرفته تر (مانند محاسبه آنتروپی فون نویمان) معتبر هستند و در ساختار داده ذخیره می شوند. در ادامه ی این بخش، مدل سازی فرآیندهای اتلاف روی ماتریس چگالی را بررسی می کنیم.

۴.۲.۴ فرآیندهای اتلاف روی ماتریس چگالی: تقسیم کانال و شکست گسیل

وقتی منبع ماتریس-چگالی با $N_{\text{channels}} > 1$ پیکربندی شود، یک شبکه ی تقسیم گر پرتو (beam-splitter network) N -کاناله فرض می شود که انرژی به طور یکنواخت میان کانال ها تقسیم می شود. ضریب گذرندگی هر کانال $\eta = 1/N$ است. تأثیر این تقسیم بر قطر ماتریس چگالی، یک تبدیل اتلاف برنولی-دوجمله ای (binomial loss) است:

$$p'_k = \sum_{n=k}^{\infty} p_n \binom{n}{k} \eta^k (1-\eta)^{n-k}, \quad (۸.۴)$$

که در آن p'_k احتمال مشاهده ی k فوتون در یک کانال خروجی است، به شرطی که n فوتون در ورودی تقسیم گر بوده است. این تبدیل، فیزیک یک تقسیم گر پرتو ایده آل را به دقت بازتولید می کند: هر فوتون با احتمال η به کانال مورد نظر می رود و با احتمال $1-\eta$ به یکی از کانال های دیگر. این فرمول در زمان ساخت DensityMatrixSource روی جدول احتمالات عدد-فوتون (_number_probs_table) اعمال می شود تا قطر ماتریس از ابتدا روی وضعیت پس از تقسیم تنظیم شود.

شکست گسیل (emission failure) نیز یک فرآیند اتلاف است که با احتمال p_{emit} 1- منبع اصلاً پالس گسیل نمی کند. مدل فیزیکی این پدیده، مخلوط کردن حالت گسیل شده با حالت خلاً است:

$$\rho' = p_{\text{emit}} \cdot \rho + (1 - p_{\text{emit}}) \cdot |0\rangle\langle 0|, \quad (۹.۴)$$

که بر قطر به صورت زیر تأثیر می گذارد:

$$p'_0 = p_{\text{emit}} \cdot p_0 + (1 - p_{\text{emit}}), \quad (۱۰.۴)$$

$$p'_k = p_{\text{emit}} \cdot p_k, \quad k > 0. \quad (۱۱.۴)$$

این تبدیل نیز در زمان ساخت اعمال می شود تا جدول احتمالات از ابتدا روی وضعیت پس از شکست گسیل تنظیم گردد. توجه شود که این دو فرآیند اتلاف، بر خلاف MZM، SCM و جیتر شدت، به صورت تبدیل های قطری روی ماتریس چگالی قابل بیان هستند و به همین دلیل در DensityMatrixSource به صورت خودکار اعمال می شوند. سایر ناکامل های کلاسیک که حالت را به صورت غیرقطری تغییر می دهند، در این کلاس نادیده گرفته می شوند و کاربر باید آن ها را پیش از ساخت به ماتریس چگالی اعمال کند.

۵.۲.۴ مودولاتور ماخ-زندر و انتقال آرام‌شیب

مودولاتور ماخ-زندر (Mach-Zehnder Modulator, MZM) یکی از اجزای کلیدی در سیستم‌های QKD با مدولاسیون شدت است. این مودولاتور از دو شاخه‌ی تداخلی تشکیل شده که با اعمال ولتاژ روی یکی از شاخه‌ها، اختلاف فاز ϕ میان دو مسیر ایجاد می‌شود. خروجی مودولاتور به صورت زیر مدل می‌شود:

$$\mu_{\text{out}} = \mu_{\text{min}} + (\mu_{\text{max}} - \mu_{\text{min}}) \cos^2(\phi), \quad (12.4)$$

که در آن μ_{min} میانگین فوتون نشتی (leakage) (مرتبط با نسبت خاموشی)، μ_{max} حداکثر شدت خروجی و ϕ فاز کل است که از جمع فاز سیگنال و فاز بایاس به دست می‌آید:

$$\phi = \frac{\pi V}{2V_{\pi}} + \phi_{\text{bias}}, \quad (13.4)$$

که در آن V ولتاژ اعمال شده، V_{π} ولتاژ نیم‌موج (half-wave voltage) (ولتاژی که برای تغییر فاز π نیاز است) و ϕ_{bias} فاز بایاس ثابت است. برای $\phi_{\text{bias}} = \pi/2$ ، نقطه‌ی کار روی شیب نیمه-حداکثر تنظیم می‌شود که بیشترین حساسیت خطی را فراهم می‌کند. نسبت خاموشی (extinction ratio) که با واحد دسی‌بل بیان می‌شود، نسبتی است میان حداکثر توان خروجی و حداقل آن (نشتی):

$$\text{ER}_{\text{dB}} = 10 \log_{10} \left(\frac{\mu_{\text{max}}}{\mu_{\text{min}}} \right). \quad (14.4)$$

از این رابطه، میانگین فوتون نشتی به صورت زیر محاسبه می‌شود:

$$\mu_{\text{min}} = \mu_{\text{max}} \cdot 10^{-\text{ER}_{\text{dB}}/10}. \quad (15.4)$$

این نشتی یک کانال جانبی امنیتی ایجاد می‌کند: حتی وقتی مودولاتور در حالت "خاموش" است، مقدار کمی نور گسیل می‌شود که می‌تواند اطلاعات بیت را به مهاجم نشت دهد. در اثبات‌های امنیتی QKD، این نشتی باید به صورت صریح در نظر گرفته شود و μ_{min} در محاسبه‌ی نرخ کلید لحاظ گردد. در این فریم‌ورک، به عنوان کف سختی در عملیات clip اعمال می‌شود تا خروجی مودولاتور هرگز کمتر از این مقدار نشود. نویز ولتاژ اعمالی که از راه درایور الکتریکی وارد می‌شود، باعث fluctuation تصادفی فاز می‌شود. اگر $V_{\text{noise}} \sim \mathcal{N}(0, \sigma_V^2)$ باشد، فاز نویزی برابر است با:

$$\phi_{\text{noisy}} = \frac{\pi(V_{\text{signal}} + V_{\text{noise}})}{2V_{\pi}} + \phi_{\text{bias}}, \quad (16.4)$$

و خروجی نهایی به صورت $\mu_{\text{min}} + (\mu_{\text{max}} - \mu_{\text{min}}) \cos^2(\phi_{\text{noisy}})$ محاسبه می‌شود. این نویز، یک fluctuations غیرخطی روی شدت ایجاد می‌کند که در ولتاژهای نویز بزرگ، می‌تواند به توزیع غیر-گاوسی روی μ_{out} منجر شود.

۶.۲.۴ مدولاسیون زیرحامل (SCM) و اتحاد ژاکوبی-آنگر

مدولاسیون زیرحامل (Subcarrier Modulation, SCM) تکنیکی است که در آن فاز حامل نوری با یک فرکانس رادیویی مدوله می‌شود تا چندین کانال فرعی روی یک حامل مشترک قرار گیرند. وقتی فاز با شاخص مدولاسیون m مدوله می‌شود، میدان خروجی از طریق اتحاد ژاکوبی-آنگر بسط می‌یابد:

$$e^{im \sin(\Omega t)} = \sum_{n=-\infty}^{\infty} J_n(m) e^{in\Omega t}, \quad (17.4)$$

که در آن $J_n(m)$ تابع بسل از نوع اول و مرتبه‌ی n است و Ω فرکانس زیرحامل. این بسط، باندهای جانبی متقارن در $n = \pm 1, \pm 2, \dots$ تولید می‌کند. توان هر باند جانبی از مرتبه‌ی n متناسب با $J_n(m)^2$ است. از آنجا که $J_{-n}(m) = (-1)^n J_n(m)$ ، توان باندهای $+n$ و $-n$ یکسان است.

در این فریم‌ورک، تنها باندهای جانبی مرتبه‌ی اول ($n = \pm 1$) در نظر گرفته می‌شوند. ضریب مقیاس SCM که به‌عنوان یک ضرب‌کننده‌ی سراسری روی میانگین فوتون اعمال می‌شود، به‌صورت زیر تعریف می‌شود:

$$S_{SCM}(m) = N_{sb} \cdot J_1(m)^2, \quad (18.4)$$

که در آن N_{sb} تعداد باندهای جانبی مورد استفاده است: $N_{sb} = 1$ برای فیلترینگ تک‌باند جانبی (SSB) و $N_{sb} = 2$ برای آشکارسازی دوباندی (DSB). برای مقادیر کوچک m ، از تقریب زاویه کوچک استفاده می‌شود:

$$J_1(m) \approx \frac{m}{2} - \frac{m^3}{16} + \mathcal{O}(m^5) \implies J_1(m)^2 \approx \frac{m^2}{4}, \quad (19.4)$$

که در نتیجه:

$$S_{SCM}(m) \approx N_{sb} \cdot \frac{m^2}{4}, \quad m \ll 1. \quad (20.4)$$

این تقریب در آستانه‌ی $m \leq 10^{-3}$ با خطای نسبی کمتر از 2.5×10^{-7} معتبر است (محاسبه‌شده از بسط $J_1(m) \approx m/2 - m^3/16$). این آستانه در ثابت `SCM_SMALL_ANGLE_LIMIT` تعریف شده است و در مقایسه با آستانه‌ی قدیمی 0.1، خطای مرزی را ۱۰۰۰ برابر کاهش می‌دهد که برای بهینه‌سازی مبتنی بر گرادیان حیاتی است. نکته‌ی ظریف درباره‌ی ناپیوستگی معنایی در $m = 0$ وقتی $m = 0$ باشد، SCM غیرفعال است و μ به‌عنوان میانگین فوتون پس از SCM تفسیر می‌شود، در حالی که برای هر $m > 0$ ، μ به‌عنوان انرژی کل گسیل‌شده تفسیر می‌شود و ضریب $S_{SCM}(m)$ کسر باند جانبی را می‌دهد. این تفسیر دوگانه در پرچم مستقل `scm_enabled` جدا شده است تا ناپیوستگی عددی به ناپیوستگی فیزیکی تبدیل شود. بهینه‌سازی مبتنی بر گرادیان در سراسر $m = 0$ نامعتبر است زیرا تفسیر فیزیکی به‌صورت ناپیوسته تغییر می‌کند.

۷.۲.۴ جیتر شدت: مدل گوسی با تصحیح بایاس

جیتر شدت (intensity jitter) `fluctuations` پالس-به-پالس میانگین فوتون را مدل می‌کند که از ناپایداری لیزر نیمه‌رسانا نشأت می‌گیرد. این فریم‌ورک دو مدل را پشتیبانی می‌کند: `RANDOM_GAUSSIAN` برای نویز گوسی افزودنی و `ADVERSARIAL_BLOCK` برای `fluctuations` همبسته‌ی بلوکی. در مدل گوسی، نویز به‌صورت زیر به میانگین فوتون اعمال می‌شود:

$$\mu_{\text{new}} = \mu \cdot |1 + \sigma z|, \quad z \sim \mathcal{N}(0, 1), \quad (21.4)$$

که در آن σ جیتر نسبی (`std/mean`) است. استفاده از قدرمطلق $|\cdot|$ به‌جای `clip` کردن (`max(0, \cdot)`) به این دلیل است که بایاس قدرمطلق از مرتبه‌ی دوم $P(X < 0)$ است، در حالی که بایاس `clip` از مرتبه‌ی اول است. با این حال، مدل قدرمطلق دارای بایاس مثبت سیستماتیک است:

$$\mathbb{E}[|1 + \sigma z|] = \sigma \sqrt{\frac{2}{\pi}} e^{-1/(2\sigma^2)} + \text{erf}\left(\frac{1}{\sigma\sqrt{2}}\right) > 1, \quad (22.4)$$

که در آن `erf` تابع خطا است. این بایاس در $\sigma = 0.5$ حدود 2% و در $\sigma = 1.0$ حدود 16% است. برای رفع این بایاس، فریم‌ورک از تصحیح تحلیلی استفاده می‌کند: مقدار μ_{new} بر `bias_factor` تقسیم می‌شود تا مدل میانگین-حفظ‌کننده شود ($\mathbb{E}[\mu_{\text{new}}] = \mu$):

```
1 if jitter > NUMERIC_ABS_TOL:
2     inv_sigma = 0.1 / jitter
3     bias_factor = (
4         jitter * np.sqrt(0.2) / np.pi * np.exp(5.0 - * inv_sigma * inv_sigma)
5         + scipy_erf(inv_sigma / np.sqrt(0.2))
6     )
7     mus = mus / bias_factor # mean-preserving correction
8
```

در مدل ADVERSARIAL_BLOCK، پالس‌ها به بلوک‌های متوالی اندازه‌ی B تقسیم می‌شوند و تمام پالس‌های یک بلوک، یک ضریب fluctuation مشترک دارند که از توزیع لگاریتم-نرمال نمونه‌برداری می‌شود:

$$\mu_{\text{new}} = \mu \cdot F_b, \quad F_b \sim \text{LogNormal}\left(-\frac{\sigma_{\ln}^2}{2}, \sigma_{\ln}^2\right), \quad (23.4)$$

که در آن b شناسه‌ی بلوک، F_b ضریب مشترک بلوک و $\sigma_{\ln}^2 = \ln(1 + \sigma^2)$ پارامتر لگاریتم-نرمال است. این پارامتری‌سازی به‌گونه‌ای انتخاب شده که $\mathbb{E}[F_b] = 1$ (میانگین-حفظ‌کننده) و $\text{Var}(F_b) = \sigma^2$ (واریانس برابر با جیتر نسبی). این مدل، سناریویی را شبیه‌سازی می‌کند که مهاجم بتواند شدت منبع را به‌صورت همبسته در بلوک‌هایی از پالس‌ها دستکاری کند، که در تحلیل‌های امنیتی پیشرفته حائز اهمیت است.

۳.۴ تحلیل معماری و ساختار داده‌ها

۱.۳.۴ سلسله‌مراتب وراثتی کلاس‌های منبع

معماری کلاس‌های منبع بر یک سلسله‌مراتب وراثتی با کلاس پایه‌ی OpticalSource و دو زیرکلاس DensityMatrixSource و PoissonSource استوار است. کلاس پایه با دو decorator تعریف می‌شود:

```

1 @dataclass(slots=True)
2 class OpticalSource:
3     """Generic optical source backed by OpticalSourceConfig."""
4     config: OpticalSourceConfig
5     security_metadata: Optional[SourceSecurityMetadata] = None
6     # ... per-instance fields like _base_mus_cache, _probabilities_cache, ...
7 
```

استفاده از @dataclass(slots=True) به‌جای frozen=True در این کلاس، یک تصمیم طراحی مهم است. برخلاف FiberChannel که کاملاً یخ‌زده است، OpticalSource نیاز به state داخلی قابل تغییر دارد: کش‌های _base_mus_cache و _probabilities_cache در زمان ساخت محاسبه می‌شوند، شمارنده‌های بلوک حریصانه (_adversarial_block_counter) در طول شبیه‌سازی به‌روز می‌شوند، و security_metadata در هر فراخوانی generate_photons به‌روز می‌گردد. برای جلوگیری از تغییر ناخواسته‌ی پیکربندی، __setattr__ override شده است تا فقط فیلدهای مجاز را قابل تغییر سازد. زیرکلاس DensityMatrixSource دو فیلد اضافی دارد:

• density_matrices: Optional[Dict[str, np.ndarray]] : نگاشت نام پالس به ماتریس چگالی.

• missing_dm_policy: str : سیاست برخورد با پالس‌های بدون ماتریس ("error" یا "vacuum").

• heterogeneous_dim_policy: str : سیاست برخورد با ابعاد ناهمگن ("error" یا "warn").

این زیرکلاس الگوی Template Method را با استفاده از _pre_init_hook و _post_init_hook پیاده می‌کند. این hook‌ها به زیرکلاس‌ها اجازه می‌دهند قبل و بعد از اعتبارسنجی پایه، کد اختصاصی خود را اجرا کنند. DensityMatrixSource._pre_init_hook ماتریس‌های چگالی را coerce می‌کند و سیاست‌ها را اعتبارسنجی می‌نماید، در حالی که _post_init_hook تانسور _rho_tensor و جدول _number_probs_table را می‌سازد.

زیرکلاس PoissonSource یک subclass convenience است که صرفاً تأیید می‌کند statistics_type == POISSON باشد. این کلاس برای خوانایی کد و تمایز صریح از DensityMatrixSource معرفی شده است.

۲.۳.۴ کلاس SourceSimParams: بسته‌ی یخ‌زده‌ی پارامترهای شبیه‌سازی

این کلاس با @dataclasses.dataclass(frozen=True) تعریف می‌شود و شامل ده فیلد است که در زمان اجرا توسط main_optimized.py استفاده می‌شوند:

```

1 @dataclasses.dataclass(frozen=True)
2 class SourceSimParams:
3     pulse_period_ns: float
4     source_rate: float
5     statistics_type: str # .value of SourceStatisticsType enum
6     error_model: str # .value of SourceErrorModel enum
7     modulation_index: float
8     intensity_jitter: float
9     N_channels: int
10    ideal_emission_probability: float
11    n_pulse_types: int
12    signal_pulse_index: int
13

```

این بسته از طریق تابع `build_source_params(source)` از یک `OpticalSource` معتبر ساخته می‌شود. یخ‌زده بودن این کلاس تضمین می‌کند که پارامترهای منبع در طول شبیه‌سازی ثابت بمانند و از رقابت `race condition` در شبیه‌سازی‌های چندرشته‌ای جلوگیری شود. این الگو، مشابه `ChannelSimParams` و `DetectorSimParams` است که در ماژول‌های دیگر فریم‌ورک پیاده‌سازی شده‌اند.

۳.۳.۴ کلاس `MZMConfig`: مودولاتور ماخ-زندر

این کلاس با `@dataclass(slots=True)` در ماژول جداگانه‌ی `modulators.py` تعریف می‌شود و چهار فیلد دارد:

- `v_pi: float = 3.5`: ولتاژ نیم‌موج به ولت.
- `phi_bias: float = np.pi / 2.0`: فاز بایاس.
- `max_intensity_mu: float = 1.0`: حداکثر میانگین فوتون.
- `extinction_ratio_db: float | None = None`: نسبت خاموشی به دسی‌بل.

متد `validate()` سه شرط را بررسی می‌کند: $V_{\pi} > 0$ ، $\mu_{\max} \geq 0$ و $ER_{dB} \geq 0$. متد `leakage_mu()` میانگین فوتون ناشی را از رابطه‌ی (۱۵.۴) محاسبه می‌کند. متد `apply(target_mus, voltage_noise_std, rng)` عملیات اصلی مودولاتور را انجام می‌دهد که در بخش بعد به تفصیل بررسی می‌شود.

۴.۳.۴ کلاس `_BoundedLRUCache`: کش LRU با اندازه‌ی محدود

این کلاس از `OrderedDict` ارث می‌برد و یک کش LRU (Least Recently Used) با اندازه‌ی حداکثر `ADVERSARIAL_CACHE_MAX_SIZE = 10000` برای نگهداری ضرایب `fluctuation` بلوک‌های حریصانه استفاده می‌شود تا همان بلوک فیزیکی همواره همان ضریب را دریافت کند، مستقل از نحوه‌ی `chunking` شبیه‌سازی:

```

1 class _BoundedLRUCache(OrderedDict):
2     def __init__(self, max_size: int = ADVERSARIAL_CACHE_MAX_SIZE):
3         super().__init__()
4         self._max_size = max_size
5         self._lock = threading.Lock() # thread-safety
6
7     def __setitem__(self, key, value):
8         with self._lock:
9             if key in self:
10                self.move_to_end(key)
11                super().__setitem__(key, value)
12            if len(self) > self._max_size:

```

```

۱۳ self.popitem(last=False) # evict oldest
۱۴

```

استفاده از `threading.Lock` برای `thread-safety` در شبیه‌سازی‌های چندرشته‌ای ضروری است. بدون این قفل، دسترسی همزمان به کش می‌تواند منجر به فساد داده شود. الگوریتم LRU: وقتی یک کلید جدید اضافه می‌شود و اندازه از حد تجاوز کند، قدیمی‌ترین ورودی (`last=False`) حذف می‌گردد.

۵.۳.۴ ثابت‌های عددی ماژول-محلی

این ماژول چندین ثابت عددی ماژول-محلی تعریف می‌کند که به دقت انتخاب شده‌اند و نباید از لایه‌های دیگر فریم‌ورک بازاستفاده شوند. جدول زیر مهم‌ترین این ثابت‌ها را فهرست می‌کند:

- `SCM_SMALL_ANGLE_LIMIT = 1e-3`: آستانه‌ی تقریب زاویه کوچک برای $J_1(m)$. خطای نسبی در این آستانه $\sim 2.5 \times 10^{-7}$ است.

- `SCM_POWER_DIVISOR = 4.0`: مقسوم‌علیه‌ی تقریب خطی $m^2/4$ برای توان باند جانبی.

- `SCM_NUM_SIDE BANDS = 1`: تعداد باندهای جانبی اول (۱ برای SSB، ۲ برای DSB).

- `MAX_GAMMA_SHAPE = 1e12`: سقف ایمن برای پارامتر شکل گاما.

- `GAUSSIAN_LARGE_JITTER_THRESHOLD = 0.1`: آستانه‌ی جیتر بزرگ.

- `SOURCE_NORMALIZATION_TOL = 1e-10`: تلورانس نرمال‌سازی جمع ردیف احتمالات.

- `SOURCE_DM_MEAN_RTOL = 1e-3`: تلورانس نسبی بررسی سازگاری μ .

- `PN_TRUNCATION_DIM = 20`: بعد پیش‌فرض برش تعداد فوتون.

- `MAX_MU = 1e6`: سقف ایمن برای میانگین فوتون.

- `ADVERSARIAL_CACHE_MAX_SIZE = 10000`: حداکثر اندازه‌ی کش بلوک حریصانه.

- `PHASE_RANDOMIZATION_ASSUMED_DEFAULT = True`: پیش‌فرض پرچم فرض تصادفی‌سازی فاز.

- `SCM_ENABLED_DEFAULT = False`: پیش‌فرض پرچم فعال‌بودن SCM.

این ثابت‌ها عمدتاً از اصلاحیه‌های بازبینی‌های چندباره (مثل F-04، F-08، F-25، F-27) نشأت می‌گیرند و هر یک پاسخی به یک خطای عددی یا فیزیکی مشخص هستند. مثلاً `MAX_MU` برای جلوگیری از `rng.poisson(np.inf)` معرفی شده که در غیر این صورت `ValueError` پرتاب می‌کند.

۴.۴ پیاده‌سازی ریاضی و الگوریتمی

۱.۴.۴ توابع تحلیلی توزیع تعداد فوتون

دو تابع `poisson_pn_array` و `thermal_pn_array` توابع خالص NumPy هستند که برای کار در حلقه‌های Numba طراحی شده‌اند و هیچ شیء پایتونی نمی‌سازند. تابع `photon_number_distribution` یک wrapper است که بر اساس نوع آمار، به یکی از این دو تابع `dispatch` می‌کند:

```

۱ def poisson_pn_array(mu: float, max_n: int = PN_TRUNCATION_DIM) ->
  np.ndarray:
۲     if mu < :0.0
۳         raise ParameterValidationError("mu must be >= 0 ...")
۴     if mu == :0.0
۵         result = np.zeros(max_n, dtype=np.float64)
۶         result[0] = 0.1

```



```

۷     return result
۸     result = np.empty(max_n, dtype=np.float64)
۹     # Numerically stable recurrence:  $P(0)=\exp(-\mu)$ ,  $P(n+1)=P(n)*\mu/(n+1)$ 
۱۰    result[0] = math.exp(-mu)
۱۱    for n in range(1, max_n):
۱۲        result[n] = result[n - 1] * mu / n
۱۳    return result
۱۴

```

خطبه خط:

- خطوط ۱-۲: اعتبارسنجی ورودی. $\mu < 0$ نامعتبر است زیرا میانگین فوتون نمی‌تواند منفی باشد.
- خطوط ۳-۶: حالت خاص $\mu = 0$. در این حالت $P(0) = 1$ و $P(n > 0) = 0$ که معادل حالت خلأ است.
- خط ۷: تخصیص آرایه‌ی float64 با طول max_n.
- خط ۹: محاسبه‌ی $P(0) = e^{-\mu}$ با math.exp که برای $\mu < 10$ پایدار است.
- خطوط ۱۰-۱۱: اعمال بازگشت (۳.۴). هر جمله از جمله‌ی قبلی با یک ضرب و یک تقسیم به‌دست می‌آید که از سرریز فاکتوریل جلوگیری می‌کند.

تابع thermal_pn_array الگوی مشابهی دارد اما با نسبت $\mu/(1+\mu)$ که همواره کوچک‌تر از ۱ است:

```

۱     def thermal_pn_array(mu: float, max_n: int = PN_TRUNCATION_DIM) ->
۲     np.ndarray:
۳         if mu == 0.0:
۴             result = np.zeros(max_n, dtype=np.float64)
۵             result[0] = 0.1
۶             return result
۷         result = np.empty(max_n, dtype=np.float64)
۸         ratio = mu / (0.1 + mu) #  $P(n+1) / P(n)$ , always < 1
۹         result[0] = 0.1 / (0.1 + mu)
۱۰        for n in range(1, max_n):
۱۱            result[n] = result[n - 1] * ratio
۱۲        return result
۱۳

```

استفاده از نسبت پیش‌محاسبه‌شده‌ی $\mu/(1+\mu)$ به‌جای محاسبه‌ی مستقیم $\mu^n/(1+\mu)^{n+1}$ هم از نظر سرعت (یک ضرب به‌جای توان و فاکتوریل) و هم از نظر پایداری عددی (نسبت همواره کوچک‌تر از ۱) برتری دارد. این الگوی بازگشتی در تمام توابع تحلیلی این فریم‌ورک برای توزیع‌های آماری استفاده می‌شود.

۲.۴.۴ متد scm_scaling_factor: ضریب مقیاس SCM

این متد، ضریب مقیاس باند جانبی SCM را محاسبه می‌کند. پیش‌اولویت اعمال به این ترتیب است: (۱) اگر scm_enabled خاموش باشد، ضریب ۱.۰؛ (۲) اگر use_linear_modulation_approximation روشن باشد، تقریب خطی اجباری؛ (۳) اگر use_small_angle_approximation روشن و $m \leq 10^{-3}$ ، تقریب زاویه کوچک؛ (۴) در غیر این صورت، فرمول دقیق بسل.

```

۱     def _scm_scaling_factor(self) -> float:
۲         if not getattr(self, 'scm_enabled', SCM_ENABLED_DEFAULT):
۳             return 0.1
۴         m = self.modulation_index
۵         nsb = self.scm_num_sidebands
۶         if self.use_linear_modulation_approximation:
۷             return float(nsb * (m ** 2) / SCM_POWER_DIVISOR)
۸         if self.use_small_angle_approximation and m <= SCM_SMALL_ANGLE_LIMIT:

```

```

9         return float(nsb * (m ** 2) / SCM_POWER_DIVISOR)
10        return float(nsb * (jv(1, m)) ** 2)
11

```

خطبه خط:

- خطوط ۱-۳: بررسی scm_enabled. استفاده از getattr با پیش فرض، سازگاری پسین را با نسخه های قدیمی که این پرچم را نداشتند تضمین می کند.
 - خط ۴: خواندن شاخص مدولاسیون m .
 - خط ۵: خواندن scm_num_sidebands که در هر نمونه به صورت مستقل ذخیره می شود (اصلاحیه ی I.4 از بازیابی ۷ام برای thread-safety).
 - خطوط ۶-۹: تقریب خطی اجباری. این شاخه برای بهینه سازی های عددی که نیاز به گرادیان تحلیلی دارند، معرفی شده است.
 - خطوط ۸-۹: تقریب زاویه کوچک. خطای نسبی در آستانه ی $m = 10^{-3}$ حدود 2.5×10^{-7} است که برای بهینه سازی مبتنی بر گرادیان کافی است.
 - خط ۱۰: فرمول دقیق با تابع بسل $J_1(m)$ از scipy.special.jv.
- نکته ی مهم درباره ی ناپیوستگی معنایی در $m = 0$: در نسخه های قدیمی، آستانه ی $m \leq 10^{-9}$ برای "غیرفعال سازی SCM" استفاده می شد که یک ناپیوستگی عددی ۳۰-مرتبهای ایجاد می کرد. اصلاحیه ی F-03 این ناپیوستگی را با جدا کردن پرچم فیزیکی scm_enabled از پارامتر پیوسته ی m برطرف کرد.

۳.۴.۴ متد calculate_effective_mus: اعمال ناکامل های کلاسیک

این متد، تمام ناکامل های کلاسیک شدت را به ترتیب فیزیکی درست روی میانگین فوتون پایه اعمال می کند. ترتیب عملیات به این صورت است:

```

1  def _calculate_effective_mus(self, base_mus, rng, pulse_positions=None):
2      # Step 1: Intensity jitter (laser fluctuation, before modulation).
3      effective_mus = self._apply_intensity_jitter(base_mus, rng,
4      pulse_positions)
5      # Step 2: MZM attenuation (with electrical-driver voltage noise).
6      effective_mus = self.mzm.apply(effective_mus,
7      self._driver_voltage_noise_std, rng)
8      # Step 3: SCM sideband factor.
9      effective_mus = effective_mus * self._scm_scaling_factor()
10     # Step 4: Channel split.
11     if self.N_channels > 1:
12         effective_mus = effective_mus / float(self.N_channels)
13     return effective_mus

```

ترتیب فیزیکی این مراحل بر اساس فیزیک منبع است: جیتر شدت از لیزر نیمه رسانا نشأت می گیرد و قبل از مدولاسیون خارجی اعمال می شود. سپس MZM با ولتاژ درایور، شدت را مدوله می کند. SCM در حوزه ی فرکانس بعد از مدولاسیون شدت اعمال می شود. در نهایت، تقسیم کانال در مرحله ی چندگانه سازی نوری انجام می گیرد. برای مدل های کاملاً ضربی (مثل MZM خطی و SCM ضربی)، این ترتیب از نظر ریاضی معادل هر ترتیب دیگری است، اما برای مدل های غیرخطی (مثل MZM با نشتی یا نویز افزودنی)، ترتیب مهم است.

۴.۴.۴ متد apply_intensity_jitter: مدل های جیتر

این متد، مدل های مختلف جیتر شدت را پیاده سازی می کند. شاخه ی RANDOM_GAUSSIAN با تصحیح بایاس:

```

1  if self.error_model == SourceErrorModel.RANDOM_GAUSSIAN:
2      jitter = float(self.intensity_jitter)
3      z = rng.standard_normal(size=mus.shape) # ALWAYS N draws
4      factor = 0.1 + jitter * z
5      mus = mus * np.abs(factor) # 0 * |factor| = 0 for vacuum
6      if jitter > NUMERIC_ABS_TOL:
7          inv_sigma = 0.1 / jitter
8          bias_factor = (
9              jitter * np.sqrt(0.2) / np.pi) * np.exp(5.0 - * inv_sigma * inv_sigma)
10             + scipy_erf(inv_sigma / np.sqrt(0.2))
11         )
12         mus = mus / bias_factor # mean-preserving correction
13     return mus
14

```

نکته‌ی کلیدی: طبق اصلاحیه‌ی F-10 از بازبینی ۸ام، همواره N نمونه از RNG مصرف می‌شود حتی برای پالس‌های خلاً ($\mu = 0$). این کار یکنواختی جریان RNG را در سراسر اسکن‌های پارامتری حفظ می‌کند: اگر تعداد نمونه‌های مصرفی به مقدار μ وابسته باشد، دو شبیه‌سازی با پیکربندی‌های مختلف دکوی، جریان‌های RNG متفاوتی خواهند داشت و قابلیت بازتولید از بین می‌رود.

شاخه‌ی ADVERSARIAL_BLOCK پیچیده‌تر است و از یک کش LRU پایدار برای حفظ همبستگی بلوک در سراسر فراخوانی‌ها استفاده می‌کند. پارامتر $\sigma_{\text{in}}^2 = \ln(1 + \sigma^2)$ به‌گونه‌ای انتخاب شده که توزیع لگاریتم-نرمال میانگین ۱ و واریانس σ^2 داشته باشد:

```

1  if self.error_model == SourceErrorModel.ADVERSARIAL_BLOCK:
2      block_size = int(self.adversarial_block_size)
3      n = len(mus)
4      sigma_sq = np.log(0.1) + self.intensity_jitter ** 2)
5      if pulse_positions is not None:
6          pulse_positions = np.asarray(pulse_positions, dtype=np.int64)
7          block_ids = pulse_positions // block_size
8          # ... lookup in _adversarial_block_factor_cache ...
9          fresh = rng.lognormal(mean=5.0 - * sigma_sq,
10             sigma=np.sqrt(sigma_sq),
11             size=len(uncached_block_ids))
12          # ... cache and apply ...
13     return mus * factors
14

```

وقتی `pulse_positions` ارائه شود (روش توصیه‌شده)، شناسه‌ی بلوک مستقیماً از `position//B` محاسبه می‌شود و کش پایدار، همان بلوک فیزیکی همواره همان ضریب را دریافت می‌کند. این اصلاحیه (بازبینی ۷ام، مسئله‌ی I.1) یک خطای همبستگی را برطرف کرد که در آن، فراخوانی‌های غیرمتوالی به بلوک‌های یکسان، ضرایب مستقل دریافت می‌کردند.

۵.۴.۴ متد `generate_photons`: تولید نهایی فوتون

این متد، نقطه‌ی ورودی اصلی شبیه‌سازی است که از منبع، تعداد فوتون تولید می‌کند. الگوریتم:

```

1  def generate_photons(self, alice_pulse_indices, rng, num_samples=1, *,
2      pulse_positions=None):
3      if alice_pulse_indices is None:
4          indices = self.sample_pulse_indices(rng, num_samples)
5          is_scalar = False
6      else:
7          indices, is_scalar = self._validate_pulse_indices(alice_pulse_indices)
8      if len(indices) == 0:

```

```

۹     return np.array([], dtype=np.int64)
۱۰    pp = None
۱۱    if pulse_positions is not None:
۱۲    pp = np.asarray(pulse_positions, dtype=np.int64)
۱۳    selected_mus = self._calculate_effective_mus(
۱۴    self._base_mus_cache[indices], rng, pulse_positions=pp
۱۵    )
۱۶    photons = self._sample_poisson_or_thermal(selected_mus, rng)
۱۷    # Emission-failure thinning
۱۸    p_emit = clamp_probability(self.ideal_emission_probability)
۱۹    n_attempted = len(indices)
۲۰    emit_mask = rng.random(size=n_attempted) < p_emit
۲۱    photons = np.where(emit_mask, photons, 0).astype(np.int64, copy=False)
۲۲    n_emitted = int(np.count_nonzero(emit_mask))
۲۳    self.update_internal_tallies(
۲۴    num_pulses_generated=n_emitted,
۲۵    num_pulses_attempted=n_attempted,
۲۶    )
۲۷    return int(photons[0]) if is_scalar else photons
۲۸

```

مراحل کلیدی:

- خطوط ۲-۷: اگر شاخص پالس None باشد، num_samples شاخص به صورت تصادفی نمونه‌برداری می‌شود. در غیر این صورت، شاخص‌های ارائه‌شده اعتبارسنجی می‌شوند.
- خطوط ۸-۹: مدیریت حالت خالی (num_samples = 0) که آرایه‌ی خالی برمی‌گرداند.
- خطوط ۱۰-۱۲: تبدیل pulse_positions به آرایه برای مدل بلوک حریصانه.
- خطوط ۱۳-۱۵: محاسبه‌ی μ_{eff} از μ_{base} با اعمال تمام ناکامل‌های کلاسیک.
- خط ۱۶: نمونه‌برداری تعداد فوتون از پواسون یا گرمایی.
- خطوط ۱۷-۲۰: اعمال شکست گسیل به صورت thinning. طبق اصلاحیه‌ی F-41 و II.3، فراخوانی rng.random همواره انجام می‌شود تا جریان RNG در سراسر تغییرات p_{emit} یکنواخت بماند.
- خطوط ۲۱-۲۵: به‌روزرسانی شمارنده‌های security_metadata برای تحلیل‌های پسین.

۶.۴.۴ MZMConfig.apply: عملیات مودولاتور

این متد در فایل modulators.py پیاده‌سازی شده و به صورت زیر کار می‌کند:

```

۱     def apply(self, target_mus, voltage_noise_std, rng):
۲     self.validate()
۳     i_min = self.leakage_mu()
۴     i_max = self.max_intensity_mu
۵     delta_i = i_max - i_min
۶     target = np.clip(np.asarray(target_mus, dtype=np.float64), i_min, i_max)
۷     if delta_i < NUMERIC_ABS_TOL:
۸     normalized = np.zeros_like(target)
۹     else:
۱۰    normalized = np.clip((target - i_min) / delta_i, 0.0, (0.1
۱۱    theta_target = np.arccos(np.sqrt(normalized))
۱۲    v_signal = 0.2 * self.v_pi / np.pi * (theta_target - self.phi_bias)
۱۳    if voltage_noise_std > :0.0
۱۴    v_noise = rng.normal(loc=0.0, scale=voltage_noise_std, size=target.shape)
۱۵    else:

```

```

۱۶ v_noise = 0.0
۱۷ v_total = v_signal + v_noise
۱۸ noisy_phase = (np.pi * v_total) / (0.2 * self.v_pi) + self.phi_bias
۱۹ return i_min + delta_i * (np.cos(noisy_phase) ** 2)
۲۰

```

این پیاده‌سازی، الگوریتم معکوس‌سازی تابع انتقال MZM را برای یافتن ولتاژ لازم، و سپس اعمال نویز روی آن ولتاژ، و در نهایت اعمال تابع انتقال به ولتاژ نویزی را انجام می‌دهد. این روش سه مرحله‌ای، فیزیک واقعی مودولاتور را بازتولید می‌کند: ابتدا ولتاژ سیگنال از روی شدت هدف محاسبه می‌شود (معکوس تابع انتقال از رابطه (۱۲.۴))، سپس نویز الکتریکی به آن افزوده می‌شود، و در نهایت خروجی واقعی از تابع انتقال $\cos^2(\phi)$ به دست می‌آید. این رویکرد، برخلاف افزودن مستقیم نویز به شدت، اثر غیرخطی MZM را به درستی مدل می‌کند. خط به خط:

- خط ۲: اعتبارسنجی پارامترهای مودولاتور.
- خطوط ۳-۵: محاسبه $\mu_{\min}$ (نشتی)، $\mu_{\max}$ و $\Delta = \mu_{\max} - \mu_{\min}$.
- خط ۶: clip کردن شدت هدف به بازه $[\mu_{\min}, \mu_{\max}]$. شدت‌های خارج از این بازه غیرفیزیکی هستند.
- خطوط ۷-۱۰: نرمال‌سازی شدت به بازه $[0, 1]$. اگر Δ بسیار کوچک باشد (مثلاً وقتی $\mu_{\max} = \mu_{\min}$)، همه‌ی شدت‌ها به صفر نرمال می‌شوند.
- خط ۱۱: معکوس تابع انتقال. از $\cos^2(\theta) = \text{normalized}$ نتیجه می‌گیریم $\theta = \arccos(\sqrt{\text{normalized}})$.
- خط ۱۲: محاسبه ولتاژ سیگنال از معکوس رابطه (۱۳.۴): $V = (2V_{\pi}/\pi)(\theta - \phi_{\text{bias}})$.
- خطوط ۱۳-۱۶: نمونه‌برداری نویز گوسی با انحراف معیار σ_V .
- خط ۱۷: جمع ولتاژ سیگنال و نویز.
- خط ۱۸: محاسبه فاز نویزی از رابطه (۱۳.۴).
- خط ۱۹: اعمال تابع انتقال $\cos^2(\phi)$ و افزودن کف نشتی.

۷.۴.۴ متد build_rho_tensor: ساخت تانسور ماتریس چگالی

این متد در DensityMatrixSource، ماتریس‌های چگالی با ابعاد ناهمگن را در یک تانسور سه‌بعدی با بعد یکسان embed می‌کند. هر ماتریس در گوشه‌ی بالا-چپ یک ماتریس صفر $\text{max_dim} \times \text{max_dim}$ قرار می‌گیرد. این zero-padding فیزیکی معنادار است: حالت‌های فوق با احتمال صفر را نشان می‌دهد. پس از padding، قطر نرمال می‌شود تا جمع برابر ۱ باشد.

۸.۴.۴ متد apply_dm_loss_processes: اعمال اتلاف روی ماتریس چگالی

این متد، دو فرآیند اتلاف قابل‌بیان به صورت قطری (تقسیم کانال و شکست گسیل) را در زمان ساخت اعمال می‌کند:

- تقسیم کانال: تبدیل برنولی-دوجمله‌ای از رابطه (۸.۴) با $\eta = 1/N$.
- شکست گسیل: مخلوط با خلأ از رابطه‌های (۱۰.۴) و (۱۱.۴).

این دو فرآیند در زمان ساخت روی number_probs_table اعمال می‌شوند تا جدول از ابتدا روی وضعیت پس از اتلاف تنظیم شود. سایر ناکامل‌های کلاسیک (مثل MZM، SCM و جیتر) که به صورت غیرقطری عمل می‌کنند، در DensityMatrixSource نادیده گرفته می‌شوند و در صورت پیکربندی غیرپیش‌فرض، هشدار ثبت می‌شود.

۵.۴ ارسال و دریافت با سایر ماژول‌ها

۱.۵.۴ ورودی‌های ماژول

ماژول `sources.py` از طریق کلاس `OpticalSourceConfig` (که در `datatypes.py` تعریف شده) پیکربندی می‌شود. این پیکربندی شامل موارد زیر است:

- `source_rate`: نرخ تکرار منبع به هرتز.
- `pulse_configs`: تاپل پیکربندی پالس‌ها (سیگنال و دکوی‌ها).
- `statistics_type`: نوع آمار (POISSON یا THERMAL).
- `error_model`: مدل جیتر شدت.
- `intensity_jitter`: جیتر نسبی.
- `modulation_index`: شاخص مدولاسیون SCM.
- `N_channels`: تعداد کانال‌ها.
- `ideal_emission_probability`: احتمال گسیل.
- `mzm`: شیء مودولاتور MZM از نوع `MZMConfig`.
- `electrical_noise`: شیء نویز الکتریکی از `noise_models.py`.

منابع متعددی برای ساخت این پیکربندی وجود دارد: `from_dict` (از دیکشنری سریال‌شده)، `from_protocol_parameters` (از `ProtocolParameters`)، `from_pulse_ensemble` (از `PulseEnsembleConfig`)، و `from_intensity_config` (از `IntensityConfig`). هر یک از این `factory method`ها، `coerce` و اعتبارسنجی مختصر به خود را انجام می‌دهند. برای `DensityMatrixSource`، ورودی اضافی `density_matrices` (نگاشت نام پالس به ماتریس چگالی) نیز لازم است. این ماتریس‌ها باید در پایه‌ی فوک باشند و شروطِ هرمیتی، مثبت‌معین و یک‌بودنِ رد را برآورده کنند.

۲.۵.۴ خروجی‌های ماژول

خروجی اصلی این ماژول، تعداد فوتون تولیدشده به‌ازای پالس است که از طریق متد `generate_photons` در دسترس قرار می‌گیرد. این خروجی به‌صورت یک آرایه‌ی `int64` (یا یک اسکالر در حالت تک‌پالس) برگردانده می‌شود و مستقیماً به ماژول کانال `channel.py` ارسال می‌شود تا ضریب انتقال فیبر روی آن اعمال شود. به‌علاوه، متد `calculate_effective_mus` میانگین فوتون مؤثر را (بدون نمونه‌برداری) برمی‌گرداند که برای تحلیل‌های تحلیلی نرخ کلید استفاده می‌شود.

خروجی تحلیلی دیگر، توزیع احتمال تعداد فوتون $P(n)$ است که از طریق متد `number_probabilities_for_pulse` برای هر پالس قابل دسترسی است. این خروجی در ماژول‌های تحلیل امنیتی (مانند تخمین e_1 و Y_1 در Decoy-State) استفاده می‌شود. توابع ماژول-سطحی `poisson_pn_array` و `thermal_pn_array` نیز برای محاسبات تحلیلی مستقل (بدون نیاز به شیء منبع) در دسترس هستند. بسته‌ی `SourceSimParams` نیز از طریق `build_source_sim_params` به ماژول `main_optimized.py` ارسال می‌شود تا پارامترهای اجرایی شبیه‌سازی (مثل دوره‌ی پالس، نرخ منبع، نوع آمار) در اختیار موتور شبیه‌سازی قرار گیرد. این بسته‌ی یخ‌زده تضمین می‌کند که پارامترها در طول شبیه‌سازی ثابت بمانند.

۳.۵.۴ وابستگی‌های داخلی

این ماژول به چندین ماژول دیگر وابسته است:

- `constants.py`: برای ثابت‌های عددی مثل `NUMERIC_ABS_TOL` و `MAX_SEED_INT_PCG64` و توابع کمکی مثل `clamp_probability` و `is_close`.

- `datatypes.py`: برای تعاریف `SourceErrorModel`, `PulseTypeConfig`, `OpticalSourceConfig` و `SourceStatisticsType`.
- `exceptions.py`: برای `ParameterValidationError`.
- `security_metadata.py`: برای `SourceSecurityMetadata` که شمارنده‌های امنیتی را نگه می‌دارد.
- `type_defs.py`: برای `RNGType` (نوع `NumPy Generator`).
- `utils/utils.py`: برای `sanitize_for_serialization`.
- `scipy.special`: برای توابع بسل `jv` و تابع خطا `erf`.

این وابستگی‌ها، یک گراف وابستگی خطی از لایه‌ی پایین (ثابت‌ها و تایپ‌ها) به لایه‌ی بالای فیزیکی (منابع) تشکیل می‌دهند. ماژول `modulators.py` به صورت مستقل عمل می‌کند و تنها به `constants.py` و `exceptions.py` وابسته است، که به `OpticalSource` اجازه می‌دهد آن را به عنوان یک کامپوننت قابل تعویض استفاده کند.

۴.۵.۴ جریان داده در شبیه‌سازی کامل

جریان داده از منبع تا آشکارساز به صورت زیر است:

۱. کاربر، `OpticalSource` را از طریق `from_dict` یا `from_protocol_parameters` می‌سازد.
۲. `build_source_sim_params` بسته‌ی اجرایی را برای موتور شبیه‌سازی می‌سازد.
۳. در هر تکرار شبیه‌سازی، `generate_photons` فراخوانی می‌شود و آرایه‌ای از تعداد فوتون تولید می‌کند.
۴. این آرایه به `FiberChannel.transmittance` ارسال می‌شود تا ضریب انتقال روی هر فوتون اعمال شود (به صورت ضرب در T و سپس گردکردن به عدد صحیح).
۵. خروجی کانال به ماژول آشکارساز `detectors.py` ارسال می‌شود تا کارایی آشکارساز، نرخ شمارش تاریک و پدیده‌ی `afterpulsing` اعمال شود.
۶. در نهایت، رویدادهای آشکارسازی به ماژول `post-processing` (تصحیح خطا و تقویت حریم خصوصی) ارسال می‌شوند.

این جریان خطی، مدولار بودن فریم‌ورک را تضمین می‌کند: هر ماژول مسئولیت یک لایه‌ی فیزیکی مشخص را دارد و خروجی آن، ورودی ماژول بعدی است. این معماری، امکان جایگزینی آسان هر ماژول با یک پیاده‌سازی متفاوت (مثلاً مدل آشکارساز پیشرفته‌تر) را بدون تغییر در سایر ماژول‌ها فراهم می‌سازد.

فصل ۵

مدل‌های آشکارساز و ساختارهای داده

۱.۵ نمای کلی و هدف ماژول

ماژول `qkd.detectors` یکی از بزرگ‌ترین، پیچیده‌ترین و از منظر فیزیکی حساس‌ترین اجزای فریم‌ورک شبیه‌سازی QKD است. این ماژول مدل کاملی از آشکارسازهای تک‌فوتونی را برای شبیه‌سازی پروتکل BB84- Decoy به صورت رویداد-محور (event-driven) پیاده‌سازی می‌کند و سه نوع آشکارساز را به صورت یکپارچه پشتیبانی می‌نماید: آشکارسازهای آوانش نوری (SPAD) در حالت آستانه‌ای، آشکارسازهای ابررسانا نانوسیمی (SNSPD) با مدل‌سازی اختصاصی از وابستگی دمایی نرخ شمارش تاریک، و آشکارسازهای تفکیک‌پذیر عدد فوتون (PNRD) با خروجی شمارشی. هدف اصلی این ماژول، فراهم‌آوردن یک موتور شبیه‌سازی واقع‌گرایانه است که اثرات پدیده‌های مخرب فیزیکی نظیر زمان مرده (dead time)، پالس‌های پسین (afterpulsing)، شمارش تاریک (dark counts)، لرزش زمانی (timing jitter) و پهن‌شدگی کروماتیک (chromatic dispersion) را با دقت ریاضی و با وفاداری به ادبیات علمی، به‌ویژه مرجع سنتی (Rogers et al. (2007)، بازتولید کند. این ماژول نقش گره اصلی در گراف وابستگی‌های فریم‌ورک را ایفا می‌کند؛ به این معنا که خروجی کانال فیبر نوری (آرایه‌های تعداد فوتون و پیامد ایده‌آل مسیریابی) به عنوان ورودی این ماژول عرضه می‌شود و خروجی آن، یعنی آرایه‌های کلیک بولی D0 و D1 به همراه اشیای تشخیصی `DetectionDiagnostics`، پایه‌ی محاسبات نرخ کلید امن در پروتکل‌های `decoy-state` و `MDI-QKD` را شکل می‌دهد. پیچیدگی این ماژول نه تنها از تنوع پدیده‌های فیزیکی مدل‌سازی‌شده برمی‌آید، بلکه از لزوم حفظ سازگاری نتایج میان مسیرهای مختلف شبیه‌سازی (آستانه‌ای در برابر PNRD) و لزوم تضمین بازتولیدپذیری آماری (reproducibility) برای کارهای وابسته به انتشار علمی ناشی می‌شود.

یک ویژگی برجسته این ماژول، سیستم کنترل نسخه‌بندی حالت داخلی است که با ثابت `STATE_VERSION = 19` و فهرست نسخه‌های قدیمی `LEGACY_STATE_VERSIONS` مدیریت می‌شود. این سیستم امکان مهاجرت رو به جلوی حالت‌های ذخیره‌شده از نسخه‌های قدیمی‌تر را فراهم می‌سازد و در عین حال با مکانیزم کنترل یکپارچگی هش پیکربندی (SHA-256 config hash) و کنترل نسخه کد فیزیک (`_PHYSICS_CODE_VERSION`) (10) از بارگذاری ناسازگار حالت‌های ذخیره‌شده با طرح متفاوت `const_params` یا معناشناسی متفاوت AP-`reschedule` جلوگیری می‌کند. این طراحی محافظه‌کارانه برای کاربردهای انتشار علمی که در آن‌ها لازم است نتایج یک شبیه‌سازی با همان بذر تصادفی دقیقاً بازتولید شوند، حیاتی است. ماژول همچنین از یک مکانیزم حالت سخت‌گیرانه (`strict_mode`) که به صورت پیش‌فرض فعال است، بهره می‌برد. این مکانیزم تقریباً هر ساده‌سازی فیزیکی مسئله‌ساز را یا به یک استثنای صریح تبدیل می‌کند یا هشدار می‌دهد، تا از انتشار نتایج نادرست فیزیکی جلوگیری شود. برای مثال، حالت سخت‌گیرانه استفاده از مدل زمان مرده پارالایز (paralyzable) در سطح یک SPAD منفرد را که بر اساس Rogers et al. نادرست است، رد می‌کند و در عوض پارالایزپذیری ظهور-سطح-پایه را تنها در صورت فعال‌سازی سیاست `ROGERS_2007` شبیه‌سازی می‌نماید. این رویکرد دفاعی، ماژول را برای استفاده در پروژه‌های پایان‌نامه و مقالات ژورنالی که در آن‌ها دقت فیزیکی فراتر از سرعت اجرا اهمیت دارد، مناسب می‌سازد.

۲.۵ مبانی تئوری و فیزیکی

مدل‌سازی آشکارسازهای تک‌فوتونی در این ماژول بر چند ستون فیزیکی استوار است که در ادامه به تفصیل بسط داده می‌شوند. نخست، فیزیک SPAD بر پدیده شکست بهره‌آوری (avalanche breakdown) در دیودهای

معکوس‌بایاس استوار است. وقتی یک فوتون در ناحیه تخلیه جذب می‌شود و یک حامل بار تولید می‌کند، میدان الکتریکی قوی حامل را شتاب می‌دهد و شکست بهره‌آوری آغاز می‌شود؛ در نتیجه یک پالس جریان قابل‌اندازه‌گیری ایجاد می‌گردد. در این حالت که به حالت گایگر (Geiger mode) معروف است، آشکارساز به هر فوتون با احتمال η (بازده آشکارساز) پاسخ می‌دهد، اما پس از هر کلیک نیازمند یک پنجره زمانی برای خاموش‌سازی (quenching) و بازبایی به حالت آماده‌به‌کار است که به آن زمان مرده τ_d می‌گوئیم. این زمان مرده در مدل حاضر از نوع غیر-پارالایز (non-paralyzable) است؛ یعنی فوتون‌هایی که در طول پنجره مرده می‌رسند هیچ اثری بر آشکارساز ندارند و پنجره مرده را تمدید نمی‌کنند. این انتخاب بر اساس §3 §6 Rogers et al. (2007) صورت گرفته که صراحتاً بیان می‌کنند SPADهای منفرد غیر-پارالایز هستند و پارالایزپذیری صرفاً یک ویژگی نوپدید (emergent) در سطح جفت آشکارسازهای پایه به‌همراه قانون سیفتنگ (sifting rule) است.

دومین ستون فیزیکی، پدیده پالس‌های پسین (afterpulsing) است. در SPADهای InGaAs/InP که در بازه مخابراتی 1550 nm استفاده می‌شوند، حامل‌های بار در طول شکست بهره‌آوری می‌توانند در تله‌های سطحی (surface trap levels) به دام بیفتند و سپس با یک تأخیر تصادفی آزاد شوند. اگر آزادسازی در زمانی رخ دهد که آشکارساز در حالت آماده‌به‌کار باشد، یک کلیک کاذب (afterpulse) تولید می‌شود. مدل زمان تأخیر در این ماژول از دو نوع توزیع پشتیبانی می‌کند: توزیع نمایی پیوسته با میانگین τ_{ap} (پارامتر afterpulse_lifetime_ns) و توزیع هندسی گسسته روی شبکه دوره پالس. در حالت هندسی، پارامتر p به‌صورت زیر از طول عمر به‌دست می‌آید:

$$p = 1 - \exp\left(-\frac{T}{\tau_{ap}}\right) \quad (1.5)$$

که در آن T دوره پالس (pulse_period_ns) است. این انتخاب تضمین می‌کند که τ_{ap} در هر دو حالت به‌عنوان زمان مشخصه توزین تأخیر تفسیر شود. احتمال تولید پالس پسین پس از هر کلیک با p_{ap} (پارامتر afterpulse_prob) کنترل می‌شود و عمق آشپاری (cascade depth) با کران سخت $\text{MAX_AFTERPULSE_CASCADE} = 32$ محدود می‌گردد تا از زنجیره‌های بازگشتی بی‌نهایت جلوگیری شود. وقتی یک پالس پسین در طول پنجره زمان مرده فعال می‌شود، هسته پردازشی آن را به زمانی پس از پایان پنجره مرده برنامه‌ریزی مجدد می‌کند؛ این برنامه‌ریزی مجدد می‌تواند با سیاست END (پایان پنجره مرده) یا سیاست UNIFORM (توزیع یکنواخت در بازه [last_fire + dt, last_fire + dt + ap_lifetime]) انجام شود. سومین ستون، شمارش تاریک (dark count) است. شمارش تاریک در SPADها به‌دو مکانیسم اصلی برمی‌گردد: نشر تونلی ناشی از تله (trap-assisted tunneling) و تولید حرارتی-نمایی (thermal generation). مدل حاضر از تقریب آرنیوس (Arrhenius) با انرژی فعال‌سازی $E_a = 0.15$ eV برای بخش دمایی و یک وابستگی درجه دوم به بایاس مازاد (excess bias) برای بخش ولتاژی استفاده می‌کند. بایاس مازاد به‌صورت $V_{eb} = V_{bias} - V_{bd}$ تعریف می‌شود، که در آن V_{bd} ولتاژ شکست است. اصلاح Review-v19 F-01 وابستگی دمایی ولتاژ شکست را با ضریب 50 mV/K برای InGaAs مدل می‌کند:

$$V_{bd}^{op} = V_{bd}^{ref} + \alpha_{bd} (T_{op} - T_{ref}) \quad (2.5)$$

که در آن α_{bd} پارامتر breakdown_temp_coeff_mv_per_k است. این اصلاح از یک باگ خاموش می‌کشد که پیش از آن همان V_{bd} برای هر دو نقطه عملیاتی و مرجع استفاده می‌شد و برای اختلاف دمایی 50 K مقادیر اشتباه $\text{voltage_factor} = 1.0$ به‌جای 0.25 تولید می‌کرد. مقیاس‌کننده نهایی نرخ تاریک SPAD به‌صورت زیر به‌دست می‌آید:

$$\mathcal{S}_{SPAD}(T, V) = \exp\left[\frac{E_a}{k_B T_{ref}} - \frac{E_a}{k_B T}\right] \cdot \left(\frac{V_{eb}^{op}}{V_{eb}^{ref}}\right)^2 \quad (3.5)$$

که در آن k_B ثابت بولتزمن است. باید تأکید کرد که این مدل صرفاً یک تقریب کیفی است و برای کارهای انتشار علمی توصیه می‌شود نرخ تاریک اندازه‌گیری‌شده در نقطه عملیاتی مستقیماً تأمین شود (با قرار دادن $T_{ref} = T_{op}$ تا مقیاس‌کننده به 1.0 فروکد کند).

برای SNSPDها، مکانیسم شمارش تاریک متفاوت است: آشکارساز در دماهای بسیار پایین (نظیر 2-4 K) کار می‌کند و منبع غالب نویز، تابش جسم سیاه (black-body radiation) از محیط اطراف است که با طیف پلانک

توصیف می‌شود. مدل حاضر شار فوتون جسم سیاه را در فرکانس‌های بالاتر از فرکانس برش $\nu_{cut} = c/\lambda_{cut}$ (با $\lambda_{cut} \approx 1550$ nm) به صورت زیر محاسبه می‌کند:

$$\Phi(T) = \frac{2(k_B T)^3}{c^2 h^3} \sum_{n=1}^{\infty} e^{-nx} \left(\frac{x^2}{n} + \frac{2x}{n^2} + \frac{2}{n^3} \right) \quad (۴.۵)$$

که در آن $x = h\nu_{cut}/(k_B T)$ است. این سری با حد به $n \rightarrow \infty$ همگراست و در عمل با کران $\text{SNSPD_PLANCK_MAX_TERMS} = 200$ و تلورانس نسبی 10^{-20} متوقف می‌شود. مقیاس‌کننده دمای SNSPD نسبت $\Phi(T)/\Phi(T_{ref})$ است؛ به علاوه یک فاکتور تجربی وابسته به جریان بایاس $f(x) = \exp(-\alpha/(1-x))$ (با $x = I_{bias}/I_{sw}$) اعمال می‌شود که به صورت کیفی رشد نرخ تاریک را هنگام نزدیک شدن جریان بایاس به جریان سوئیچینگ بازتولید می‌کند. این مقیاس‌کننده صریحاً به عنوان کیفی-تنها (QUALITATIVE ONLY) مستند شده و در حالت سخت‌گیرانه اگر انحراف آن از 1.0 بیش از 5% باشد، استثنا صادر می‌کند. چهارمین ستون، لرزش زمانی (timing jitter) و پهن‌شدگی کروماتیک (chromatic dispersion) است. لرزش زمانی به منظور بازتولید عدم قطعیت در زمان‌بندی الکترونیک آشکارساز و پهن‌شدگی پالس نوری در فیبر، در مرحله بن کردن (binning) به هر رویداد کلیک اعمال می‌شود. مدل از توزیع گوسی با عرض نیم-ماکزیمم $\text{FWHM} = \text{jitter_fwhm_ns}$ استفاده می‌کند که به انحراف معیار از رابطه آشنای زیر تبدیل می‌شود:

$$\sigma = \frac{\text{FWHM}}{2\sqrt{2 \ln 2}} \quad (۵.۵)$$

نکته فیزیکی مهم این است که پهن‌شدگی کروماتیک صرفاً به رویدادهای کلیک با منشأ سیگنال اعمال می‌شود و نه به شمارش‌های تاریک و پالس‌های پسین، زیرا این دو پدیده درون آشکارساز تولید می‌شوند و از فیبر عبور نمی‌کنند. این تفکیک با پرچم `is_sig` در هسته پردازشی پیگیری می‌شود (اصلاح Review-v12 F3). پنجمین ستون، سیاست سیفتنگ دو-کلیک (double-click sifting policy) است که با Enum به نام `DoubleClickPolicy` پیاده‌سازی شده و چهار حالت دارد: `DISCARD` (رد هر دو کلیک هم‌زمان)، `RANDOM` (انتخاب تصادفی یکی از دو کلیک)، `KEEP_BOTH` (نگه‌داشتن هر دو) و `ROGERS_2007`. حالت `ROGERS_2007` مطابق مقاله Rogers et al. (2007) یک دنباله تشخیص (detection sequence) را به صورت یک اجرای ماکزیمال از رویدادهای تشخیص روی $D_1 \cup D_0$ تعریف می‌کند که در آن رویدادهای متوالی با فاصله کمتر از τ_d جدا شده‌اند. هر دنباله نهایتاً یک بیت سیفتی (sifted bit) (اولین رویداد) تولید می‌کند و سایر رویدادها در همان دنباله دور ریخته می‌شوند و در شمارنده `rogers_discarded_events` ثبت می‌گردند.

۳.۵ تحلیل معماری و ساختار داده‌ها

معماری این ماژول چندلایه است و از یک مجموعه غنی از `dataclass`ها، `Enum`ها و `NamedTuple`ها تشکیل شده که هر کدام نقش مشخصی در جداسازی نگرانی‌ها (separation of concerns) ایفا می‌کنند. در رأس این معماری، کلاس اصلی `SinglePhotonDetector` قرار دارد که یک `dataclass` با `slots=True` (اما بدون `frozen=True`) است تا امکان تغییر حالت درون‌رونده (in-process state) را حفظ کند و در عین حال مصرف حافظه را بهینه نماید. این کلاس به صورت هدفمند غیر-ایستا (non-frozen) طراحی شده زیرا باید حالت درون‌روندهی آشکارساز (مانند زمان آخرین کلیک، زمان AP معلق، عمق آبشار، زمان DC بعدی و رویدادهای carry-over) را در طول شبیه‌سازی به‌روزرسانی کند. با این حال، پیکربندی آشکارساز (یعنی تمام فیلدهای فیزیکی مانند بازده، نرخ تاریک، زمان مرده و غیره) پس از ساخت از طریق اعتبارسنجی در `__post_init__` به صورت عملی تغییرناپذیر باقی می‌مانند؛ زیرا هیچ متد عمومی برای تغییر آن‌ها وجود ندارد.

فیلدهای پیکربندی به سه گروه تقسیم می‌شوند. گروه نخست، فیلدهای پایه عملکرد آشکارساز هستند: `det_eff_d0` و `det_eff_d1` (بازده‌های نامتقارن دو آشکارساز)، `dark_rate` و `dark_rate_d1` (نرخ‌های تاریک نامتقارن)، `qber_intrinsic` و `misalignment` که به ترتیب نرخ خطای ذاتی کوانتومی و ناهم‌راستایی پایه را کنترل می‌کنند، و `detector_type` که بین سه مسیر `SPD`، `SNSPD` و `PNRD` انتخاب می‌کند. گروه دوم، فیلدهای زمان‌بندی و نویز هستند: `dead_time_ns`، `jitter_fwhm_ns`، `afterpulse_prob`، `afterpulse_lifetime_ns` و حالت‌های نامتقارن لرزش `jitter_fwhm_ns_d0/d1`. این فیلدها با `meta-` (مانند `noiseless_copy()`) به صورت خودکار صفر شوند و امکان شبیه‌سازی حالت ایده‌آل (بدون نویز) فراهم شود. گروه سوم، فیلدهای سخت‌افزاری/محیطی هستند:

`ref_bias_voltage`, `ref_temperature_k`, `breakdown_voltage`, `bias_voltage`, `temperature_k`, `switching_current`, `bias_current` که برای محاسبه مقیاس‌کننده‌های دمایی/ولتاژی نرخ تاریک به کار می‌روند.

حالت درون‌رونده در یک `dataclass` جداگانه به نام `DetectorState` نگهداری می‌شود که به صورت یک فیلد `init=False` درون آشکارساز قرار گرفته و در فرآیند سریال‌سازی `to_config_dict()` از آن عبور نمی‌کند. این جداسازی صریح میان طراحی امنی برای جلوگیری از نشت حالت درون‌رونده به خروجی پیکربندی است و به کاربر اجازه می‌دهد یک آشکارساز را با همان پیکربندی اما با حالت بازنشانی‌شده بسازد. فیلدهای `DetectorState` شامل: `last_abs_fire_time_ns_d0/d1` (با سنتینل $-\infty$ به معنای «هرگز کلیک نکرده»)، `pending_ap_time_d0/d1` (با سنتینل < 0 به معنای «بدون AP معلق»)، `ap_depth_d0/d1` (عمق آبشار AP معلق)، `next_dc_time_d0/d1` (زمان برنامه‌ریزی‌شده شمارش تاریک بعدی)، `carry_over_events` (تاپل از `CarryOverEvent`‌ها که رویدادهای ورودی پس از پایان بلاک را برای بلاک بعدی نگه می‌دارند)، `total_time_processed_ns` (ساعت زمان مطلق پردازش‌شده)، `afterpulse_total_count` (شمارنده تجمعی AP) و `last_path` (یا `pnrd` که برای کشف تغییر مسیر و هشدار درباره از دست رفتن حالت پیوسته به کار می‌رود) است.

`NamedTuple` به نام `CarryOverEvent` برای ساختاردهی به رویدادهای `carry-over` معرفی شده است. این ساختار با نام‌گذاری صریح فیلدها (`is_signal_origin`, `src_pulse_idx`, `det_id`, `time_abs_ns`) از جابه‌جایی تصادفی مؤلفه‌ها هنگام تغییر شکل تاپل جلوگیری می‌کند و کد را خود-مستندکننده می‌سازد. خروجی شبیه‌سازی در دو `NamedTuple` جداگانه بازگردانده می‌شود: `DetectionResult` برای خروجی استاندارد (با آرایه‌های کلیک، تشخیصی، و `state_snapshot` به صورت رشته JSON) و `PhotonResolvedResult` برای تحلیل‌های `decoy-state yield` که در آن فیلدهای اضافی `arrivals`، `target_d0` و `ideal_target_d0` نیز در دسترس هستند.

اشیای تشخیصی در `dataclass` ایستای `DetectionDiagnostics` جمع‌آوری می‌شوند که بیش از 25 فیلد شمارشی دارد: از `resolved_to_d0/d1` و `double_clicks_total` (برای سیاست RANDOM) گرفته تا `combined_flips_with_arrivals`، `deadtime_dropped`، `rogers_discarded_events`، `qber_flips_only`، `isi_events` (تداخل بین-سمبلی)، `afterpulse_events`، `dark_count_events`، `multiple_fires_per_slot` و `imd_click_events`. این کلاس دارای `property`های مفیدی مانند `tossed_breakdown` (که تجزیه دسته‌بندی‌شده رویدادهای دورریخته را برمی‌گرداند) و `total_discarded` است. شاخص `kernel_tossed` که پیش از Review-v19 F-11 با تفریق محاسبه می‌شد و ممکن بود منفی شود، اکنون مستقیماً از `diagnose` هسته (`DIAG_TOSSED_CORE`) پر می‌شود تا از پنهان‌شدن باگ‌های حسابداری تشخیصی جلوگیری شود.

یک `dataclass` ایستای دیگر به نام `DetectorConstants` تمام ثابت‌های تنظیمی را متمرکز می‌کند تا تغییر آن‌ها بدون اعتبارسنجی مجدد در برابر داده‌های مشخصه‌سازی امکان‌پذیر نباشد. این کلاس شامل مقادیری مانند `SPAD_DARK_ACTIVATION_EV = 32`، `MAX_AFTERPULSE_CASCADE = 32`، `LOG_SAFE_EPS = 0.10`، `DEFAULT_RNG_BUFFER_MULT = 200`، `SNSPD_PLANCK_MAX_TERMS = 15.0` و چندین ثابت مرتبط با تحلیل Rogers است. Enum‌های پشتیبانی‌شده شامل `DetectorType` (`NON_PARALYZABLE/PARALYZABLE`)، `DeadTimeModel` (`SPD/SNSPD/PNRD`)، `DetectorTopology` (`EXPONENTIAL/GEOMETRIC`)، `AfterpulseModel` (`INDEPENDENT_SPADS/SHARED_SPAD`)، `DecoderArchitecture` (`DOUBLE_CLICK_POLICY`)، `APRescheduleMode` (`END/UNIFORM`) هستند.

کلاس کمکی `EntanglingDecoder` برای رمزگشای درهم‌تنیدگی در پروتکل‌های MDI-QKD و بازگشت متقارن (`redundant return`) طراحی شده است. این `dataclass` ایستا با `slots=True` از عملگرهای کوانتومی CPTP برای اعمال دیفیزینگ (`dephasing`) آگاه از دیدپذیری (`visibility-aware`) استفاده می‌کند: درایه‌های خارج‌قطری ماتریس چگالی مشترک ρ_{joint} با ضریب V (دیدپذیری) مقیاس می‌شوند در حالی که قطر (جمعیت‌ها) دست‌نخورده باقی می‌ماند. این کلاس از `redundancy_M` در $\{2, 3\}$ پشتیبانی می‌کند و برای $M = 2$ از پروژکتورهای بل-حالت $|\Psi^+\rangle$ و $|\Psi^-\rangle$ استفاده می‌کند:

$$|\Psi^\pm\rangle = \frac{1}{\sqrt{2}} (|01\rangle \pm |10\rangle) \quad (6.5)$$

و برای $M = 3$ از کلمات کد کیوبیتی-۳ برای کد فلیپ-بیت که حالت‌های $|000\rangle$ ، $|011\rangle$ ، $|101\rangle$ و $|110\rangle$ را به عنوان پایه‌های سندروم دو-بیتی در نظر می‌گیرد.

۴.۵ پیاده‌سازی ریاضی و الگوریتمی

مسیر آستانه‌ای (threshold path) که برای SPD/SNSPD فعال می‌شود، یک موتور رویداد-محور کامل را پیاده‌سازی می‌کند. در گام نخست، نازک‌سازی کانال (channel thinning) با توزیع دو-جمله‌ای انجام می‌شود: برای هر پالس، تعداد فوتون‌های رسیده به آشکارساز با $\text{Binomial}(n, \eta_{ch})$ arrivals نمونه‌برداری می‌شود که در آن n تعداد فوتون منبع و $\eta_{ch} = \eta_{eff} \cdot F_{int}$ شامل شدت بازده کانال و فاکتور ناهم‌خوانی شدت است. سپس احتمال کلیک در هر پالس برای m فوتون وارد شده با فرمول زیر محاسبه می‌شود:

$$P_{click}(m, \eta) = 1 - (1 - \eta)^m \quad (۷.۵)$$

این محاسبه با $\log1p$ و $\expm1$ برای پایداری عددی پیاده‌سازی می‌شود تا برای مقادیر کوچک η خطای گردکردن IEEE-754 به حداقل برسد. در ادامه، مسیریابی به D0/D1 با احتمال ترکیبی فلیپ $p_{combined}$ انجام می‌شود که پیش‌فرض آن از فرمول XOR زیر به‌دست می‌آید:

$$p_{combined} = m(1 - p_{qber}) + (1 - m)p_{qber} \quad (۸.۵)$$

که در آن m ناهم‌راستایی (misalignment) و $p_{qber} = q_{intrinsic} + (1 - V)/2$ است که V دیدپذیری (visibility) تداخل است. این فرمول XOR فرض استقلال آماری میان ناهم‌راستایی و خطای QBER را دارد که در BB84 واقعی از نظر فیزیکی نادرست است (هر دو از همان نقص‌های دیدپذیری ناشی می‌شوند)؛ به همین دلیل حالت سخت‌گیرانه استفاده از این فرمول را نیازمند ارائه صریح flip_prob_override یا فعال‌سازی allow_xor_flip_formula=True می‌کند.

محاسبه نرخ تاریک پویا در متد `_calculate_dynamic_rates()` انجام می‌شود که برای SPAD از مقیاس‌کننده آرنیوس-درجه دوم و برای SNSPD از مقیاس‌کننده طیف پلانک استفاده می‌کند. این متد برای جلوگیری از محاسبه مکرر انتگرال پلانک (که تا 200 حمله سری می‌تواند داشته باشد) در هر فراخوانی شبیه‌سازی، کش می‌شود. کد زیر پیاده‌سازی مقیاس‌کننده SPAD را نشان می‌دهد:

```

1 def _spad_dark_scaler(self) -> float:
2     # Review-v19 F-01: temperature-dependent breakdown voltage
3     coeff_V_per_K = self.breakdown_temp_coeff_mv_per_k * 1e-3
4     delta_T = self.temperature_k - self.ref_temperature_k
5     bdv_op = self.breakdown_voltage + coeff_V_per_K * delta_T
6     excess_bias = self.bias_voltage - bdv_op
7     ref_excess_bias = self.ref_bias_voltage - self.breakdown_voltage
8     if ref_excess_bias <= EPS:
9         ref_excess_bias = _REF_EXCESS_BIAS_FALLBACK_V
10    voltage_factor = (excess_bias / ref_excess_bias) ** 2
11
12    kT = CONST_BOLTZMANN * self.temperature_k
13    ref_kT = CONST_BOLTZMANN * self.ref_temperature_k
14    e_act = _SPAD_DARK_ACTIVATION_EV * _EV_TO_JOULE
15
16    exponent = (e_act / ref_kT) - (e_act / kT)
17    if exponent < 0.700- or exponent > :0.700
18    # guard against numerical overflow in math.exp
19    if self.strict_mode:
20        raise ParameterValidationError(...)
21    exponent = max(0.700-, min(0.700), exponent))
22    temp_factor = math.exp(exponent)
23    return temp_factor * voltage_factor
24

```

در خط نخست، ضریب mV/K به V/K تبدیل می‌شود تا در محاسبه جبری با V سازگار باشد. سپس اختلاف دما ΔT و ولتاژ شکست عملیاتی V_{bd}^{op} با اصلاح دمایی محاسبه می‌شوند. بایاس مازاد عملیاتی و مرجع به‌ترتیب $V_{eb}^{ref} = V_{bias}^{ref} - V_{bd}^{ref}$ و $V_{eb}^{op} = V_{bias}^{op} - V_{bd}^{op}$ هستند. اگر بایاس مازاد مرجع نزدیک به صفر باشد، از مقدار جایگزین `_REF_EXCESS_BIAS_FALLBACK_V` استفاده می‌شود تا از تقسیم بر صفر جلوگیری شود. فاکتور

ولتاژ به صورت درجه دوم و فاکتور دما به صورت نمایی Arrhenius محاسبه می‌شوند. محافظت عددی برای بازه $[-700, 700]$ اعمال می‌شود تا از سرریز exp جلوگیری شود؛ این محدوده برای اکثر پیکربندی‌های واقع‌گرایانه (دماهای 200–400 K) کافی است.

هسته پردازش رویداد که در ماژول جداگانه `detector_kernel.py` پیاده‌سازی شده (که فصل جداگانه‌ای به آن اختصاص دارد)، در این ماژول توسط متد `_simulate_event_driven()` فراخوانی می‌شود. این متد ابتدا آرایه‌های رویداد ساختار-یافته را می‌سازد، سپس `const_params` (یک آرایه float64 با 15 مؤلفه که شامل زمان مرده، احتمال AP، طول عمر AP، دوره پالس، زمان شروع بلاک، مدت بلاک، نرخ‌های تاریک D0/D1 و... است) و بافر RNG پیش‌کشیده را آماده می‌کند، و سپس هسته را فرامی‌خواند. خروجی هسته شامل زمان‌های کلیک، شناسه آشکارساز، پرچم منشأ سیگنال، آرایه تشخیصی، و تاپل حالت-خروج است که برای به‌روزرسانی `_DetectorState` استفاده می‌شود.

مسیر PNRD در متد `_simulate_pnrnd()` به صورت کاملاً متفاوتی پیاده‌سازی شده است. در این مسیر، اثر زمان مرده، پس‌پالس و فرآیند زمانی پیوسته شمارش‌های تاریک لحاظ نمی‌شود؛ در عوض، برای هر پالس از یک مدل پواسونی مستقل جهت شبیه‌سازی شمارش‌های تاریک استفاده می‌شود. این انتخاب از نظر سازگاری درونی مسیر قابل قبول است، زیرا آشکارسازهای واقعی PNRD معمولاً در رژیمی کار می‌کنند که در آن زمان مرده ناچیز است. کد زیر، نازک‌سازی کانال و مسیریابی را در مسیر PNRD نشان می‌دهد:

```

1  # Channel thinning (binomial)
2  photon_numbers_i = np.asarray(photon_numbers, dtype=np.int64)
3  arrivals = rng.binomial(photon_numbers_i, eff_trans)
4
5  if per_photon_routing_pnrnd and has_multiphoton:
6  # Per-photon routing (the default, physically correct)
7  n_total = int(np.sum(arrivals))
8  if n_total > 0:
9  photon_flips = rng.random(n_total) < p_combined_flip
10 ideal_per_pulse = np.asarray(ideal_outcomes_d0, dtype=np.bool_)
11 photon_ideal = np.repeat(ideal_per_pulse, arrivals)
12 photon_actual_to_d0 = photon_ideal ^ photon_flips
13 pulse_idx = np.repeat(np.arange(num_pulses), arrivals)
14 arrivals_d0 = np.zeros(num_pulses, dtype=np.int64)
15 np.add.at(arrivals_d0, pulse_idx,
16 photon_actual_to_d0.astype(np.int64))
17 arrivals_d1 = arrivals - arrivals_d0
18 # ...
19 det_counts_d0 = rng.binomial(arrivals_d0, self.det_eff_d0)
20 det_counts_d1 = rng.binomial(arrivals_d1, self.det_eff_d1)
21 mu_dc_d0 = max(0.0, dr0) * float(pulse_period_ns) * 1e-9
22 dark_counts_d0 = rng.poisson(mu_dc_d0, size=num_pulses)
23 final_d0 = det_counts_d0 + dark_counts_d0 + imd_counts_d0
24

```

این کد نشان می‌دهد که مسیر PNRD از `rng.binomial` برای نازک‌سازی کانال و بازده آشکارساز استفاده می‌کند و `rng.poisson` را برای شمارش تاریک هر-پالس به کار می‌گیرد. حالت پیش‌فرض `per_photon_routing=True` مسیریابی هر-فوتون را (که فیزیکی صحیح است) انتخاب می‌کند؛ اگر ورودی چند-فوتونی در حالت سخت‌گیرانه با `per_photon_routing=False` ارائه شود، استثنا صادر می‌شود زیرا مسیریابی هر-پالس بازده Y_n decoy-state برای $n \geq 2$ را به صورت خاموش تحریف می‌کند.

تحلیل‌های تحلیلی (Rogers et al. (2007) به صورت چند تابع سطح-ماژول پیاده‌سازی شده‌اند. تابع `rogers_P_00(p, k, eps)` حالت پایدار احتمال اینکه هر دو آشکارساز در یک پایه زنده باشند را با تکرار ارزش (value iteration) روی فضای حالت دوبعدی $\{(i, j) : 0 \leq i, j \leq k\}$ محاسبه می‌کند. این تابع برای k بزرگ با کران تکرار تطبیقی $\min(50000, \max(5000, k^2 \cdot 100))$ و تلورانس 10^{-12} همگرا می‌شود. تابع `rogers_T_N(N, p, k)` توزیع احتمال طول دنباله را با فرمول هندسی بازتولید می‌کند:

$$T_N = (1 - s)^{N-1} \cdot s, \quad s = (1 - 2p)^k \quad (9.5)$$

که در آن $p = L/8$ (پارامتر از-دست‌رفته پیوند) و $k = \rho_{TX} \cdot \tau$ (تعداد دوره‌های ارسال در هر زمان مرده) است. تابع `rogers_S(p, k)` احتمال سیفتنگ موفق یک بیت از یک دنباله را به صورت زیر محاسبه می‌کند:

$$S(p, k) = \sum_{N=1}^{N_{max}} T_N(p, k) \cdot \left(1 - \frac{1}{2^N}\right) \quad (10.5)$$

و نرخ بیت سیفتی (SBR) از معادله (15) مقاله به دست می‌آید:

$$SBR = \rho_{TX} \cdot 8p \cdot P_{00}(p, k) \cdot S(p, k) \quad (11.5)$$

همچنین تقریب‌های جانبی مقاله در توابع `rogers_sbr_max(\tau)` و `rogers_rho_tx_max(L, \tau)` پیاده‌سازی شده‌اند که به ترتیب نرخ بیت سیفتی پیشینه و نرخ ارسال پیشینه را در شرط بهینه‌سازی ارائه می‌کنند. پیاده‌سازی سیاست سیفتنگ `ROGERS_2007` در متد ایستا `apply_rogers_2007_sifting()` قرار دارد. این متد یک اجرای ماکزیمال از رویدادهای کلیک را با بررسی فاصله زمانی متوالی شناسایی می‌کند: یک رویداد به عنوان شروع دنباله در نظر گرفته می‌شود اگر اولین رویداد باشد یا شکاف زمانی $\tau_d \geq$ از رویداد قبلی داشته باشد. سپس تنها رویدادهای شروع دنباله (که در پنجره گیت نیز قرار دارند) در آرایه‌های کلیک نهایی نگه داشته می‌شوند و سایر رویدادها به عنوان `rogers_discarded_events` شمارش می‌گردند. این متد از زمان‌های واقعی (`true times`) برای مقایسه شکاف با زمان مرده استفاده می‌کند (نه زمان‌های لرزش‌دیده) که از نظر فیزیکی صحیح است: ساعت زمان مرده آشکارساز در زمان واقعی رسیدن فوتون شروع به کار می‌کند، نه در زمان لرزش‌دیده‌ی اندازه‌گیری.

۵.۵ ارسال و دریافت با سایر ماژول‌ها

این ماژول به عنوان گره میانی میان کانال فیبر نوری و لایه تحلیل کلید امن عمل می‌کند و رابط‌های ورودی و خروجی متعددی دارد. از منظر ورودی، متد عمومی `simulate_detection()` یک سیگنال چندوجهی دریافت می‌کند که از ماژول‌های بالادست می‌آید: پارامتر `channel_transmittance` (یک عدد `float` در بازه `[0, 1]`) از ماژول کانال فیبر نوری `qkd.channel` می‌آید و حاصل قانون بیر-لامبرت $T = 10^{-L_{total}/10}$ است؛ آرایه `photon_numbers` (از نوع `NDArray[np.int64]`) از ماژول منبع نوری `qkd.source` می‌آید و تعداد فوتون‌های ارسالی آلیس در هر پالس را با توزیع پواسون یا توزیع چندجمله‌ای `decoy` ارائه می‌کند؛ آرایه `ideal_outcomes_d0` (از نوع `NDArray[np.bool_]`) از ماژول رمزگذاری پایه `qkd.encoder` می‌آید و نشان می‌دهد کدام پالس‌ها در حالت ایده‌آل به `D0` مسیریابی شده‌اند؛ آرایه `basis_selector` اختیاری نیز از همان ماژول می‌آید و برای فعال‌سازی بازده پایه-وابسته (`basis-dependent efficiency`) با فیلدهای `det_eff_d0_z/x` و `det_eff_d1_z/x` به کار می‌رود.

شیء `rng` از نوع `numpy.random.Generator` یا از ماژول بالادستی `qkd.simulation_driver` می‌آید یا توسط کاربر به صورت مستقیم ساخته می‌شود و بذر آن برای بازتولیدپذیری شبیه‌سازی حیاتی است. ماژول از مکانیزم `RNG state snapshot/restore` برای اطمینان از اینکه تلاش‌های مجدد بافر (`buffer retries`) نتایج یکسانی با همان بذر تولید کنند، استفاده می‌کند. کلمات کلیدی `kwargs` شامل پارامترهای فیزیکی `SCM` مانند `modulation_index`، `visibility_bias_drift_psi1/psi2`، `visibility_mismatch_dm`، `mu_signal` و `N_channels` است که از ماژول تحلیل `SCM` (`qkd.utils.scm_analysis`) می‌آیند و برای محاسبه شدت `IMD/FWM` به کار می‌روند. پارامترهای `WDM` مانند `include_wdm_fwm_noise` نیز از ماژول تحلیل `WDM` عرضه می‌شوند.

از منظر خروجی، ماژول یک `DetectionResult` (یا `PhotonResolvedResult`) برمی‌گرداند که شامل سه بخش اصلی است. آرایه‌های `click0` و `click1` (از نوع `NDArray[np.bool_]`) برای آستانه‌ای و `PNRD` برای `NDArray[np.int64]` به طول `num_pulses` هستند و به ماژول سیفتنگ پایه (`qkd.sifting`) ارسال می‌شوند تا بیت‌های سیفتی استخراج شوند. شیء `DetectionDiagnostics` به ماژول گزارش‌گیری و تجزیه و تحلیل آماری (`qkd.analysis`) ارسال می‌شود و در آن معیارهایی مانند `QBER` مشاهده شده، بازده `decoy-state` Y_n ، نرخ `IMD` و نرخ `ISI` محاسبه می‌شوند. رشته `state_snapshot` (یک `JSON` سریال شده با `allow_nan=False`) به ماژول مدیریت حالت (`qkd.state_manager`) ارسال می‌شود تا در صورت نیاز به ادامه شبیه‌سازی در بلاک‌های بعدی، حالت آشکارساز به صورت دقیق قابل بازیابی باشد.

از منظر وابستگی‌های درون‌پروژه‌ای، این ماژول از چندین ماژول زیرین واردات دارد: `qkd.datatypes` برای `Enum`ها و `dataclass`های مشترک مانند `DoubleClickPolicy`، `DetectorType`، `DetectionConfig`، `DecoderArchitecture` و `TallyCounts`؛ `qkd.exceptions` برای

(`param_name`, `param_value`, `context`) هستند؛ `qkd.constants` برای توابع کمتری مانند `is_valid_probability`، `clamp_probability`، `is_finite_non_negative`، `CONST_BOLTZMANN`، `CONST_PLANCK`، `CONST_ELECTRON_CHARGE`، `CONST_SPEED_OF_LIGHT` و `sanitize_for_serialization` برای `qkd.utils` و `qkd.utils.scm_analysis`، `calculate_wdm_fwm_noise` و `calculate_intensity_mismatch_factor`، `calculate_visibility` این ماژول همچنین یک لایه سازگار-کاهش (`graceful fallback`) برای زمانی که بسته `qkd` کامل نصب نباشد (مانند زمان اجرای تست‌های مستقل) دارد که در آن نسخه‌های ساده‌شده‌ی تمام `Enum`ها، ثابت‌ها و توابع کمتری به‌صورت درون‌ماژولی تعریف می‌شوند. این لایه با پرچم `_HAS_QKD_PACKAGE` کنترل می‌شود و امکان تست‌گذاری جداگانه ماژول را بدون نیاز به کل زیرساخت پروژه فراهم می‌سازد. با این حال، در کاربردهای عملی، همیشه نسخه کامل بسته `qkd` باید نصب باشد تا تمام اعتبارسنجی‌ها و رفتارهای سخت‌گیرانه فعال شوند.

از منظر مصرف‌کنندگان اصلی، ماژول `qkd.main_optimized` (موتور اصلی شبیه‌سازی) از `build_detector_sim_params()` برای ساخت یک بسته ایستای `DetectorSimParams` از آشکارساز اعتبارسنجی‌شده استفاده می‌کند؛ این بسته شامل یازده پارامتر مشتق‌شده از آشکارساز است و برای جلوگیری از خواندن مکرر از دیکشنری پیکربندی در حلقه اصلی شبیه‌سازی به کار می‌رود. ماژول‌های تحلیل کلید امن مانند `qkd.key_rate` نیز از خروجی‌های `DetectionDiagnostics` به‌ویژه `combined_flips_with_arrivals`، `dark_count_events` و `afterpulse_events` برای تغذیه فرمول‌های نرخ کلید `GLLP` و `decoy-state` استفاده می‌کنند. در نهایت، تابع `is_threshold_detector()` به‌عنوان یک تابع کمکی برای سایر ماژول‌هایی که نیاز به بررسی نوع آشکارساز دارند (مانند انتخاب مسیر محاسبه بازده) فراهم شده است.

۶.۵ نمای کلی و هدف ماژول

ماژول `qkd.detector_kernel` هسته پردازش رویداد (`event-processing kernel`) و پوشش کامپایل `Numba` را برای موتور شبیه‌سازی آشکارساز فریم‌ورک `QKD` در خود جای داده است. این ماژول نقش حلقه داغ (`hot loop`) پردازش متوالی رویدادها را ایفا می‌کند و عملکرد کل فریم‌ورک شبیه‌سازی به‌طور مستقیم به بهینه‌سازی این ماژول وابسته است. هدف اصلی، پیاده‌سازی یک هسته پردازشی است که رویدادهای کلیک آشکارساز را با وفاداری کامل به فیزیک پدیده‌های موقت (`time-domain phenomena`) - شامل زمان مرده غیر-پارالایز، تولید پالس پسین با کران آبخاری، زمان‌بندی پیوسته شمارش تاریک و `carry-over` رویدادهای ورودی پس از پایان بلاک - پردازش کند و در عین حال با مکانیزم `Numba JIT` به سرعت اجرای نزدیک به `C` دست یابد. این ماژول از ماژول مادر `qkd.detectors` استخراج شده تا جداسازی صریح میان منطق پردازش رویداد (پُر از بحث‌های عددی و فیزیکی) و منطق سطح بالای آشکارساز (ساخت پیکربندی، اعتبارسنجی، سریال‌سازی) را فراهم سازد. پیچیدگی اصلی این ماژول از چند منبع نشئت می‌گیرد. نخست، هسته باید با ترتیب زمانی سخت‌گیرانه رویدادها را پردازش کند و در زمان‌های تساوی، اولویت صریح `input > afterpulse > dark count` را اعمال نماید. این اولویت‌بندی برای تضمین بازتولیدپذیری (`reproducibility`) نتایج در پلتفرم‌های مختلف و در حضور نویز تصادفی حیاتی است. دوم، هسته باید حالت آشکارساز را در طول چندین بلاک شبیه‌سازی به‌صورت پیوسته نگه دارد؛ به این معنا که رویدادهای `afterpulse` و شمارش تاریک که در پایان یک بلاک برنامه‌ریزی شده‌اند، در بلاک بعدی به‌صورت دقیق در زمان‌های مطلق صحیح خود پردازش شوند. این نیازمند یک `state-out tuple` غنی است که تمام حالت موقت را در خود نگه دارد. سوم، هسته باید با چندین سناریوی فرسایشی (`exhaustion scenarios`) مقابله کند: فرسایش بافر `RNG`، فرسایش بافر خروجی، فرسایش بافر `carry-over`، و فراتر رفتن از کران تکرار. در هر یک از این موارد، هسته باید به‌صورت تشخیصی (`diagnostic`) رفتار کند و خروجی معتبری تولید نماید که به کاربر امکان دیباگ و تلاش مجدد با بافر بزرگ‌تر را بدهد.

این ماژول به‌صورت آگاهانه بر روی مکانیزم `Numba JIT` تکیه دارد اما در صورت عدم دسترسی یا شکست کامپایل، به‌صورت شفاف به پیاده‌سازی `pure-Python` بازمی‌گردد. این طراحی `graceful fallback` با پرچم `NUMBA_AVAILABLE` کنترل می‌شود و در صورت وقوع، تنها یک هشدار لاگ صادر می‌کند. نکته مهم این است که کامپایل `Numba` با `fastmath=False` انجام می‌شود تا از بازترتیب عملیات `IEEE-754-violating` توسط کامپایلر جلوگیری شود و نتایج `bit-identical` میان مسیر `Numba-compiled` و `pure-Python` (در محدوده عملیاتی هسته) تضمین گردد. با این حال، مستندات ماژول به‌صراحت اشاره می‌کنند که این تضمین به‌صورت رسمی `bit-identical` نیست و نقاط واگرایی شناخته‌شده‌ای (مانند `math.log` روی ورودی‌های زیر-نرمال و عرض `int()` در پایتون در برابر `Numba`) وجود دارند که در محدوده عملیاتی هسته (زمان‌مهرهای کمتر از 10^{16} و `max_steps` کمتر از 10^7) غیرقابل‌دسترس هستند.

ماژول از ماژول خواهری `qkd.detector_datastructures` دو نهاد حیاتی را وارد می‌کند: ثابت `_SENTINEL_T` (یک مقدار شناور بسیار بزرگ که به‌عنوان سنتینل «هیچ رویداد» استفاده می‌شود) و `event_np_dtype` (یک `NumPy structured dtype` که شکل هر رویداد را در آرایه ورودی تعریف می‌کند). این وابستگی به‌صورت سختگیرانه اعمال می‌شود و در صورت عدم دسترسی، به‌جای `fallback` خاموش، استثنای `ImportError` صادر می‌کند؛ زیرا یک `fallback` با `dtype` متفاوت می‌تواند به واگرایی پنهان میان مسیرهای `pure-Python` و `Numba-compiled` منجر شود که دیباگ آن بسیار دشوار است. این تصمیم طراحی بر اساس ممیزی F9 گرفته شده و سیاست صریح ماژول را به «no silent fallback for dtype divergence» تبدیل می‌کند.

۷.۵ مبانی تئوری و فیزیکی

هسته پردازش رویداد بر پایه چند مفهوم بنیادین از فیزیک آشکارسازهای تک‌فوتونی و تئوری صف‌های رویداد استوار است. نخست، مدل زمان مرده غیر-پارالایز (non-paralyzable dead-time model) که طبق Rogers et al. (2007) رفتار صحیح یک SPAD منفرد را توصیف می‌کند. در این مدل، وقتی یک کلیک در زمان t_f رخ می‌دهد، آشکارساز تا زمان $t_f + \tau_d$ به هیچ ورودی حساس نیست و فوتون‌های رسیده در طول این پنجره هیچ تأثیری بر آشکارساز ندارند - نه کلیک تولید می‌کنند و نه پنجره مرده را تمدید می‌نمایند. این رفتار با بررسی ساده زیر در هسته پیاده‌سازی می‌شود:

$$\text{if}(t_{\text{current}} - t_{\text{last_fire}}) < \tau_d \Rightarrow \text{suppress} \quad (12.5)$$

نکته فیزیکی مهم این است که این بررسی با زمان واقعی (true time) رویداد انجام می‌شود، نه با زمان لرزش‌دیده (jittered time). این تمایز حیاتی است زیرا ساعت زمان مرده آشکارساز در زمان واقعی رسیدن فوتون شروع به کار می‌کند، در حالی که زمان ثبت‌شده در خروجی (که برای بن کردن به پالس‌ها استفاده می‌شود) تحت تأثیر لرزش الکترونیک است.

دومین مفهوم، مدل تولید پالس پسین (afterpulse generation) با توزیع تأخیر است. وقتی یک کلیک رخ می‌دهد، با احتمال p_{ap} یک پالس پسین تولید می‌شود که زمان آغاز آن از توزیع تأخیر نمایی یا هندسی نمونه‌برداری می‌شود. در توزیع پیوسته، تأخیر با $\text{decay} = -\ln(u) \cdot \tau_{ap}$ نمونه‌برداری می‌شود که در آن $u \sim \text{Uniform}(0, 1)$ و τ_{ap} طول عمر پالس پسین است:

$$P(\text{delay} > t) = \exp\left(-\frac{t}{\tau_{ap}}\right) \quad (13.5)$$

در توزیع هندسی گسسته روی شبکه دوره پالس T ، پارامتر هندسی $p = 1 - \exp(-T/\tau_{ap})$ به‌گونه‌ای انتخاب می‌شود که میانگین تأخیر در هر دو توزیع برابر τ_{ap} باشد. تعداد گام‌های تأخیر با $K = \lceil \ln(u)/\ln(1-p) \rceil$ نمونه‌برداری می‌شود و $\text{decay} = K \cdot T$ تنظیم می‌گردد. یک کران سخت `max_steps` (که تطبیقی با طول عمر و مدت بلاک است) از نمونه‌برداری تأخیرهای بسیار بزرگ جلوگیری می‌کند تا از سرریز بافر RNG جلوگیری شود. سومین مفهوم، عمق آبشار پالس پسین (afterpulse cascade depth) است. وقتی خود یک پالس پسین کلیک می‌کند، می‌تواند پالس پسین دیگری تولید کند که این چرخه می‌تواند به‌صورت بی‌نهایت ادامه یابد. در عمل، احتمال تولید در هر مرحله به‌صورت نمایی کاهش می‌یابد اما برای جلوگیری از زنجیره‌های طولانی، یک کران سخت `MAX_AFTERPULSE_CASCADE = 32` اعمال می‌شود. وقتی این کران رسید، عمق آبشار به مقدار کران قفل می‌شود تا شروع فوری یک آبشار جدید در کلیک بعدی همان آشکارساز جلوگیری شود. یک آبشار جدید تنها می‌تواند از یک کلیک غیر-AP (سیگنال، شمارش تاریک یا IMD) آغاز شود که عمق را به 1 بازنشانی می‌کند. چهارمین مفهوم، زمان‌بندی پیوسته شمارش تاریک (continuous-time dark-count scheduling) است. شمارش تاریک به‌صورت یک فرآیند پواسون همگن با نرخ λ مدل می‌شود که در آن زمان میان-رسیدها از توزیع نمایی با میانگین $1/\lambda$ پیروی می‌کند:

$$P(\text{inter-arrival} > t) = \exp(-\lambda t) \quad (14.5)$$

نمونه‌برداری با تبدیل معکوس (inverse transform sampling) انجام می‌شود: $\text{wait} = -\ln(u)/\lambda$ که در آن $u \sim \text{Uniform}(0, 1)$. برای جلوگیری از $\ln(0) = -\infty$ یک کران پایین `log_safe_eps = 1e-300` اعمال

می‌شود که به حداکثر تأخیر $\approx 690/\lambda$ منجر می‌شود (که برای هر مدت بلاک واقع‌گرایانه عملاً نامحدود است). نکته مهم این است که وقتی نرخ شاری تاریک میان بلاک‌ها به صفر تغییر می‌کند، یک زمان شاری تاریک معلق از بلاک قبلی باید به صورت صریح باطل شود؛ در غیر این صورت یک شاری تاریک در بلاکی که نرخ صفر است اجرا می‌شود که از نظر فیزیکی محال است (اصلاح F-H9).

پنجمین مفهوم، carry-over رویدادهای ورودی است. وقتی بلاک شبیه‌سازی پایان می‌یابد، ممکن است رویدادهای ورودی وجود داشته باشند که زمان مطلق آن‌ها فراتر از `time_end_abs` است. این رویدادها نباید در بلاک فعلی پردازش شوند؛ به جای آن، در بافر carry-over ذخیره می‌شوند تا در بلاک بعدی به عنوان رویدادهای ورودی اولیه فراخوانی شوند. نکته ظریف این است که رویدادهای ورودی که در طول پنجره زمان مرده رخ داده‌اند (و در نتیجه در بلاک فعلی سرکوب شده‌اند) نباید به بافر carry-over منتقل شوند؛ زیرا پنجره مرده در بلاک بعدی نیز بر اساس `last_fire` قبلی اعمال می‌شود و انتقال چنین رویدادی منجر به سرکوب مضاعف یا حتی رفتار نامشخص می‌شود. این نکته در اصلاح C4 به صورت صریح پیاده‌سازی شده است.

ششمین مفهوم، برنامه‌ریزی مجدد پالس پسین در زمان مرده (`AP reschedule in dead time`) است. وقتی یک پالس پسین در طول پنجره زمان مرده فعال می‌شود، نمی‌تواند کلیک تولید کند و باید به زمانی پس از پایان پنجره مرده برنامه‌ریزی مجدد شود. این ماژول دو سیاست برنامه‌ریزی مجدد را پشتیبانی می‌کند: سیاست END که پالس پسین را در انتهای پنجره مرده ($t_{last_fire} + \tau_d$) قرار می‌دهد، و سیاست UNIFORM که آن را به صورت یکنواخت در بازه $[t_{last_fire} + \tau_d, t_{last_fire} + \tau_d + \tau_{ap}]$ نمونه‌برداری می‌کند. سیاست UNIFORM فیزیکیاً صحیح‌تر است زیرا زمان آزادسازی تله‌ها به صورت یکنواخت در طول عمر تله توزیع شده است. اصلاح Review-v12 F2 یک باگ حیاتی را برطرف کرد که در آن بازه نمونه‌برداری به اشتباه $[t_{last_fire}, t_{last_fire} + \tau_d]$ (درون پنجره مرده) بود که منجر به حلقه‌های بی‌نهایت برنامه‌ریزی مجدد می‌شد.

۸.۵ تحلیل معماری و ساختار داده‌ها

معماری این ماژول حول دو نهاد اصلی سازمان‌دهی شده: تابع `_process_events_logic()` که پیاده‌سازی pure-Python هسته است و متغیر `_process_events_numba` که یا به همین تابع (در صورت عدم دسترسی Numba) یا به نسخه JIT-compiled آن اشاره می‌کند. این طراحی به مصرف‌کنندگان اجازه می‌دهد همیشه از `_process_events_numba` استفاده کنند و نگران دسترس‌پذیری Numba نباشند. در زمان `import` ماژول، در صورت دسترس بودن Numba، تلاش کامپایل با `njit(cache=True, fastmath=False)` انجام می‌شود و یک کامپایل اجباری (one-shot) با ورودی‌های کوچک برای اشکال‌زدایی زود هنگام خطاهای کامپایل و پر کردن کش دیسک اجرا می‌گردد. اگر کامپایل شکست بخورد، یک هشدار لاگ صادر شده و `_process_events_numba` به `_process_events_logic` بازمی‌گردد.

امضای تابع `_process_events_logic()` به صورت زیر است:

```

1  def _process_events_logic(
2      sorted_events: NDArray[Any],           # structured array
3      last_abs_fire_d0: float,               # state-in: last fire times
4      last_abs_fire_d1: float,
5      pending_ap_d0: float,                 # state-in: pending AP times
6      pending_ap_d1: float,
7      next_dc_d0_in: float,                 # state-in: next DC times
8      next_dc_d1_in: float,
9      const_params: NDArray[np.float64],   # 15-float bundle
10     rng_floats: NDArray[np.float64],      # pre-drawn RNG buffer
11     co_buffer: NDArray[Any],              # carry-over output buffer
12     out_times: NDArray[np.float64],       # fire-time output buffer
13     out_dets: NDArray[np.int64],          # detector-id output buffer
14     out_sigs: NDArray[np.bool_],          # signal-origin flag buffer
15     cascade_depths_in: NDArray[np.int64], # 2-element AP depth state
16     ap_overflow_times: NDArray[np.float64], # 4-element multi-trap overflow
17     ap_overflow_depths: NDArray[np.int64], # 4-element overflow depths
18 ):
19 
```

این امضای طولانی به دلیل نیاز هسته به دسترسی مستقیم به تمام حالت و بافرهای ورودی/خروجی بدون عبور از لایه‌های شیء-گرا است؛ این طراحی برای کارایی Numba حیاتی است زیرا Numba نمی‌تواند به صورت کارآمد با اشیاء پایتون تعامل داشته باشد و آرایه‌های NumPy بهترین نوع داده برای کامپایل JIT هستند. پارامتر

sorted_events یک آرایه ساختار-یافته با dtype event_np_dtype است که شامل چهار فیلد det, time, idx و sig برای هر رویداد است؛ این فیلدها به ترتیب زمان مطلق، شناسه آشکارساز، شاخص پالس منبع و پرچم منشأ سیگنال را نگه می‌دارند.

پارامتر const_params یک آرایه float64 با طول ثابت CP_LEN = 15 است که تمام پارامترهای فیزیکی و پیکربندی را در خود جای داده. این پارامترها با شاخص‌های نام‌دار CP_AP_PROB, CP_DEAD_TIME, CP_DR0, CP_BATCH_DURATION, CP_TIME_START, CP_PULSE_PERIOD, CP_AP_LIFETIME, CP_GATE_OFFSET, CP_GATE_WIDTH, CP_GATED, CP_MAX_CASCADE, CP_AP_GEOMETRIC, CP_DR1, CP_SHARED_SPAD و CP_AP_RESCHEDULE_UNIFORM دسترسی می‌شوند. این رویکرد با شاخص‌های نام‌دار (به جای دیکشنری یا کلاس) برای کارایی Numba ضروری است زیرا دسترسی به آرایه با شاخص عددی سریع‌ترین روش دسترسی در کد کامپایل‌شده است. مقادیر بولی به صورت float64 ذخیره می‌شوند و با مقایسه ≥ 0.5 به بولی تبدیل می‌شوند (نه > 0.5 که در نقطه میانی 0.5 رفتار شکننده‌ای دارد).

پارامترهای ap_overflow_depths و ap_overflow_times آرایه‌هایی با طول 4 هستند که multi-trap overflow را برای هر آشکارساز پیگیری می‌کنند. دو شکاف نخست (0, 1) برای D0 و دو شکاف دوم (2, 3) برای D1 اختصاص داده شده‌اند. این طراحی به هسته اجازه می‌دهد تا دو AP معلق اضافی به ازای هر آشکارساز را پیگیری کند، که تقریبی بهتر از مدل تک-AP معلق قبلی است. وقتی یک AP جدید باید برنامه‌ریزی شود و هر سه شکاف (یک اصلی + دو سرریز) پر هستند، جدیدترین AP در شکاف سرریز با جدیدترین زمان بازنویسی می‌شود (تصمیمی که کمترین تأثیر را دارد زیرا AP جدید احتمالاً زودتر از AP سرریز قدیمی فعال می‌شود). این بازنویسی در شمارنده تشخیصی DIAG_AP_OVERWRITE ثبت می‌گردد.

کدهای وضعیت بازگشتی توسط ثابت‌های مازولی STATUS_OK = 0, STATUS_RNG_EXHAUSTED, STATUS_CARRY_OVER_BUFFER_EXHAUSTED = 3 و STATUS_OUTPUT_BUFFER_EXHAUSTED = 2, STATUS_ITERATION_CAP_EXCEEDED = 4 تعریف شده‌اند. این کدها به مصرف‌کننده اجازه می‌دهند به صورت برنامه‌نویسی رفتار متناسب با وضعیت را اتخاذ کنند؛ مثلاً در صورت STATUS_RNG_EXHAUSTED، مصرف‌کننده می‌تواند بافر RNG بزرگ‌تری تخصیص دهد و تلاش مجدد کند. شاخص‌های تشخیصی DIAG_IMD_CLICKS, DIAG_TOSSED_CORE, DIAG_DARKCOUNT, DIAG_AFTERPULSE, DIAG_DEADTIME, DIAG_MULTIPLE_FIRES_PER_SLOT و DIAG_DEADTIME_DROPPED شاخص‌هایی در آرایه خروجی diags هستند که آمار تفکیکی از انواع رویدادها را در اختیار مصرف‌کننده قرار می‌دهند. خروجی تابع، یک تاپل هشت‌عضوی به فرمت (final_times, final_dets, final_sigs, diags, status_code, co_count, state_out) است. متغیر state_out خود یک ساختار داده‌ای جامع است که شامل 14 مؤلفه برای به‌روزرسانی وضعیت آشکارساز می‌باشد: زمان‌های آخرین کلیک برای D0/D1، زمان‌های AP معلق برای D0/D1، زمان‌های DC بعدی برای D0/D1، عمق‌های آبشار برای D0/D1، اشاره گر RNG، شمارنده‌های بازنویسی AP و کپی‌هایی از آرایه‌های سرریز AP. وجود این کپی‌ها ضروری است؛ زیرا آرایه‌های محلی ممکن است در بلاک زمانی بعدی تغییر کنند و وضعیت باید مستقل از آن‌ها حفظ شود.

۹.۵ پیاده‌سازی ریاضی و الگوریتمی

هسته پردازش از یک حلقه اصلی تشکیل شده که در هر تکرار، رویداد بعدی را با توجه به اولویت `input > dark count > afterpulse` و پردازش می‌کند. در ادامه، گزیده‌ای از کد انتخاب رویداد ارائه شده است:

```

1 while True:
2     iteration += 1
3     if iteration > max_iterations:
4         status_code = STATUS_ITERATION_CAP_EXCEEDED
5         break
6
7     t_input = _SENTINEL_T
8     if input_ptr < input_len:
9         t_input = float(sorted_events[input_ptr]["time"])
10
11     # Find the earliest in-batch AP across both detectors
12     t_next_ap = _SENTINEL_T
13     ap_det = -1
14     if t_ap_0 >= 0.0 and t_ap_0 < time_end_abs and t_ap_0 < t_next_ap:
15         t_next_ap = t_ap_0; ap_det = 0
16     if t_ap_1 >= 0.0 and t_ap_1 < time_end_abs and t_ap_1 < t_next_ap:

```

```

۱۷     t_next_ap = t_ap_1; ap_det = 1
۱۸
۱۹     # Filter past-batch-end DCs; track which detector wins
۲۰     _dc0_c = t_dc_0 if (t_dc_0 < _SENTINEL_T and t_dc_0 < time_end_abs) else _SENTINEL_T
۲۱     _dc1_c = t_dc_1 if (t_dc_1 < _SENTINEL_T and t_dc_1 < time_end_abs) else _SENTINEL_T
۲۲     if _dc0_c <= _dc1_c and _dc0_c < _SENTINEL_T:
۲۳         t_next_dc = _dc0_c; dc_det = 0
۲۴     elif _dc1_c < _SENTINEL_T:
۲۵         t_next_dc = _dc1_c; dc_det = 1
۲۶     else:
۲۷         t_next_dc = _SENTINEL_T; dc_det = -1
۲۸
۲۹     # Early exit: no more events to process
۳۰     if (input_ptr >= input_len
۳۱         and (t_ap_0 < 0.0 or t_ap_0 >= time_end_abs)
۳۲         and (t_ap_1 < 0.0 or t_ap_1 >= time_end_abs)
۳۳         and (t_dc_0 >= _SENTINEL_T or t_dc_0 >= time_end_abs)
۳۴         and (t_dc_1 >= _SENTINEL_T or t_dc_1 >= time_end_abs)):
۳۵         break
۳۶
۳۷     # Priority selection: input > afterpulse > dark count
۳۸     if t_input <= t_next_ap and t_input <= t_next_dc:
۳۹         # Input wins (priority 1)
۴۰         current_time = t_input
۴۱         current_det = int(sorted_events[input_ptr]["det"])
۴۲         src_idx = int(sorted_events[input_ptr]["idx"])
۴۳         is_sig = bool(sorted_events[input_ptr]["sig"])
۴۴         input_ptr += 1
۴۵     elif t_next_ap <= t_next_dc:
۴۶         # Afterpulse wins (priority 2 over dark count)
۴۷         current_time = t_next_ap
۴۸         current_det = ap_det
۴۹         is_ap = True
۵۰         # ...
۵۱     else:
۵۲         # Dark count wins
۵۳         current_time = t_next_dc
۵۴         current_det = dc_det
۵۵         is_dark = True
۵۶

```

این کد چندین ویژگی طراحی مهم را نشان می‌دهد. نخست، بررسی «خروج زودهنگام» (early exit) پیش از بررسی فرسایش بافر انجام می‌شود؛ این ترتیب بر اساس اصلاح Review-v12 F13 تغییر کرده تا از خروج اشتباه با کد فرسایش وقتی حلقه به صورت تمیز پایان می‌یافت جلوگیری شود. دوم، مقایسه‌ها با $\leq$ دقیق انجام می‌شوند (نه با تلورانس EPS) که این مقایسه‌ها را انتقالی (transitive) و قابل بازتولید در پلتفرم‌های مختلف می‌سازد. سوم، فیلتر کردن AP ها و DC های پس از پایان بلاک به صورت صریح انجام می‌شود تا فقط رویدادهای درون-بلاک به عنوان کاندیدای «رویداد بعدی» انتخاب شوند. پس از انتخاب رویداد، چندین بررسی متوالی انجام می‌شود. ابتدا بررسی زمان مرده:

```

۱     if shared_spad:
۲         # Shared dead-time clock: most recent fire on EITHER detector
۳         last_fire = t_fire_0 if t_fire_0 > t_fire_1 else t_fire_1
۴         else:
۵         last_fire = t_fire_0 if current_det == 0 else t_fire_1
۶
۷         # Non-paralyzable dead-time check
۸         if (current_time - last_fire) < dt_ns:
۹             # Distinguish batch-end input events (DIAG_DEADTIME_DROPPED)
۱۰            # from in-batch events (DIAG_DEADTIME)
۱۱            if current_time >= time_end_abs and not is_dark and not is_ap:
۱۲                diags[DIAG_DEADTIME_DROPPED] += 1
۱۳            continue

```

```

۱۴     diags[DIAG_DEADTIME] += 1
۱۵
۱۶     if is_ap:
۱۷         # Reschedule the AP after the dead-time window
۱۸         if ap_reschedule_uniform:
۱۹             u_release = rng_floats[rng_ptr]; rng_ptr += 1
۲۰             release_time = last_fire + dt_ns + u_release * ap_lifetime
۲۱         else:
۲۲             release_time = last_fire + dt_ns
۲۳             rescheduled = release_time + _KERNEL_AP_RESCHEDULE_SLACK_NS
۲۴             # ... restore to t_ap_0 or t_ap_1
۲۵             continue
۲۶
۲۷         # In-batch input/dark events in dead time: silently dropped
۲۸         continue
۲۹

```

این بخش نشان می‌دهد که چگونه مدل زمان مرده غیر-پارالایز پیاده‌سازی شده: اگر رویداد در پنجره مرده باشد، بدون تأثیر بر ساعت زمان مرده سرکوب می‌شود. تمایز مهمی میان رویدادهای ورودی در پایان بلاک (که در DIAG_DEADTIME_DROPPED شمارش می‌شوند اما در carry-over قرار نمی‌گیرند) و رویدادهای درون-بلاک (که در DIAG_DEADTIME شمارش می‌شوند) وجود دارد. برای پالس‌های پسین که در زمان مرده فعال می‌شوند، سیاست END یا UNIFORM برای برنامه‌ریزی مجدد اعمال می‌گردد؛ یک slack کوچک (--KER_) $\text{NEL_AP_RESCHEDULE_SLACK_NS} = 1\text{e-6}$ برای جلوگیری از شرط مرزی دقیق $t = t_{\text{last_fire}} + \tau_d$ اضافه می‌شود.

تولید پالس پسین پس از هر کلیک موفق با کد زیر انجام می‌شود:

```

۱     if ap_prob > :0.0
۲         u_fire = rng_floats[rng_ptr]; rng_ptr += 1
۳         if u_fire < ap_prob:
۴             # Determine the current cascade depth for this detector
۵             if is_ap:
۶                 # AP-originated fire: depth = parent_depth + 1
۷                 current_depth = _consumed_ap_depth + 1
۸             else:
۹                 # Signal/dark/IMD-originated fire: first-generation AP
۱0                current_depth = 1
۱۱
۱۲             if (current_depth <= per_detector_depth_limit
۱۳                 and global_ap_count < global_cascade_limit):
۱۴                 u_decay = rng_floats[rng_ptr]; rng_ptr += 1
۱۵                 u_decay = max(log_safe_eps, u_decay)
۱۶
۱۷             if ap_model_geometric:
۱۸                 if pulse_period_ns > EPS and p_geometric > :0.0
۱۹                     # Standard geometric sampling:
۲۰                     # K = ceil(log(U)/log(1-p))
۲۱                     raw_steps = int(math.ceil(math.log(u_decay) / log_1mp))
۲۲                     steps = max(1, min(raw_steps, max_steps))
۲۳                     decay = steps * pulse_period_ns
۲۴                 else:
۲۵                     decay = 0.0
۲۶                 else:
۲۷                     # Exponential sampling: decay = -ln(u) * tau
۲۸                     decay = -math.log(u_decay) * ap_lifetime
۲۹
۳۰             sched_time = current_time + max(0.0, decay)
۳۱
۳۲             # Schedule the AP, handling overflow slots
۳۳             if current_det == 0:
۳۴                 if t_ap_0 < :0.0
۳۵                 t_ap_0 = sched_time

```

```

۳۶ ap_depth_0 = current_depth
۳۷ else:
۳۸     # Try overflow slots 0, 1 for DO
۳۹     # ... (overflow management logic)
۴۰     # ...
۴۱     global_ap_count += 1
۴۲

```

این کد چندین نکته مهم را نشان می‌دهد. نخست، عمق آبشار به صورت صریح پیگیری می‌شود؛ اگر خود کلیک یک AP باشد، عمق پالس پسین جدید $\text{depth}_{\text{parent}} + 1$ است؛ در غیر این صورت (سیگنال، شمارش تاریک یا IMD)، عمق از 1 آغاز می‌شود. این تمایز برای جلوگیری از شروع فوری یک آبشار جدید پس از رسیدن به کران ضروری است. دوم، نمونه‌برداری هندسی از فرمول استاندارد $K = \lceil \ln(u) / \ln(1 - p) \rceil$ استفاده می‌کند که $P(K = k) = (1 - p)^{k-1} \cdot p$ را برای $k = 1, 2, \dots$ تولید می‌کند. سوم، کران max_steps (که تطبیقی با طول عمر و مدت بلاک است) از نمونه‌برداری تأخیرهای بسیار بزرگ جلوگیری می‌کند. چهارم، مدیریت overflow برای AP‌های معلق اضافی پیاده‌سازی شده که تقریبی بهتر از مدل تک-AP معلق قبلی ارائه می‌دهد. محاسبه کران تکرار تطبیقی (adaptive iteration cap) بر اساس تعداد تخمینی کل رویدادها انجام می‌شود:

$$\text{max_iterations} = \max(10^7, 100 \cdot \text{est_total_events}) \quad (15.5)$$

که در آن $\text{est_total_events} = \text{input_len} + (\text{dr}_0 + \text{dr}_1) \cdot 10^{-9} \cdot \text{batch_duration} + 100$ است. این کران تطبیقی (که بر اساس اصلاح F-17 Review-v15 معرفی شده) از رسیدن زودرس به کران ثابت قبلی 10^7 برای بلاک‌های بزرگ با 10^6 پالس و شمارش تاریک بالا جلوگیری می‌کند. یک کران سخت بالا 10^8 نیز اعمال می‌شود تا از مسیرهای پاتولوژیک جلوگیری شود. محاسبه پارامتر هندسی p با کد زیر انجام می‌شود:

```

1  if ap_model_geometric and ap_lifetime > EPS and pulse_period_ns > EPS:
2  p_geometric = 0.1 - math.exp(-pulse_period_ns / ap_lifetime)
3  # Clamp to [0, 1 - EPS] to avoid log1p(-1) = -inf
4  if p_geometric < :0.0
5  p_geometric = 0.0
6  elif p_geometric >= 0.1 - EPS:
7  p_geometric = 0.1 - EPS
8  if p_geometric <= :0.0
9  log_1mp = 0.0 # degenerate; effectively no afterpulse
10 else:
11 log_1mp = math.log1p(-p_geometric)
12 else:
13 p_geometric = 0.0
14 log_1mp = 0.0
15

```

محاسبه $\log_{1mp}$ با استفاده از $\text{math.log1p}(-p)$ به جای $\text{math.log}(1 - p)$ به منظور بهبود دقت عددی در نزدیکی $p = 0$ انجام می‌شود. اعمال کران $1 - \varepsilon$ از بروز خطای $\log(0) = -\infty$ جلوگیری می‌کند. این پارامترها یک‌بار در ابتدای هر فراخوانی هسته محاسبه شده و در طول حلقه اصلی استفاده می‌شوند. پوشش کامپایل Numba با کد زیر پیاده‌سازی شده:

```

1  # Default to the pure-Python implementation
2  _process_events_numba = _process_events_logic
3
4  if NUMBA_AVAILABLE:
5  try:
6  _process_events_numba = njit(cache=True, fastmath=False)(
7  _process_events_logic
8  )
9  # Force a one-shot compile to surface compilation errors eagerly
10 _seed_events = np.zeros(2, dtype=event_np_dtype)
11 _seed_events["time"] = 0.1, (0.2
12 _seed_events["det"] = (0, 1)

```

```

۱۳ _seed_events["idx"] = (0, 1)
۱۴ _seed_events["sig"] = (True, False)
۱۵ _seed_cp = np.zeros(CP_LEN, dtype=np.float64)
۱۶ _seed_cp[CP_PULSE_PERIOD] = 0.1
۱۷ _seed_cp[CP_DEAD_TIME] = 0.50
۱۸ _seed_cp[CP_BATCH_DURATION] = 0.1000
۱۹ _seed_cp[CP_AP_LIFETIME] = 0.50
۲۰ _seed_cp[CP_MAX_CASCADE] = 0.5
۲۱ _ = _process_events_numba(
۲۲     _seed_events,
۲۳     0.1-e9, 0.1-e9, 0.1-, 0.1-, 0.1-, 0.1-,
۲۴     _seed_cp,
۲۵     np.zeros(32, dtype=np.float64),
۲۶     np.zeros(8, dtype=event_np_dtype),
۲۷     np.zeros(16, dtype=np.float64),
۲۸     np.zeros(16, dtype=np.int64),
۲۹     np.zeros(16, dtype=np.bool_),
۳۰     np.zeros(2, dtype=np.int64),
۳۱     np.full(4, 0.1-, dtype=np.float64),
۳۲     np.zeros(4, dtype=np.int64),
۳۳ )
۳۴ except Exception as e:
۳۵     logger.warning(
۳۶         "Failed to compile Numba logic, falling back to pure Python: %s",
۳۷         e,
۳۸     )
۳۹     _process_events_numba = _process_events_logic
۴۰

```

این پیاده‌سازی چندین ویژگی دارد. نخست، njit(cache=True) کش دیسک را فعال می‌کند تا در اجراهای بعدی، زمان کامپایل اولیه حذف شود. دوم، fastmath=False از بازترتیب عملیات IEEE-754-violating توسط کامپایلر جلوگیری می‌کند و در نتیجه نتایج bit-identical (در محدوده عملیاتی) میان مسیر Numba و pure-Python را تضمین می‌نماید. سوم، کامپایل اجباری با ورودی‌های کوچک در زمان import خطاهای کامپایل را زود هنگام آشکار می‌کند و کش دیسک را پر می‌کند. چهارم، در صورت شکست کامپایل، با هشدار لاگ به pure-Python بازمی‌گردد تا ماژول همواره قابل استفاده باشد. این طراحی graceful برای محیط‌هایی که Numba در آن‌ها نصب نیست یا نسخه ناسازگار دارد، حیاتی است.

۱۰.۵ ارسال و دریافت با سایر ماژول‌ها

این ماژول به‌عنوان هسته پردازشی درونی، در نقش یک سرویس‌دهنده سطح پایین برای ماژول سطح بالای qkd.detectors عمل کرده و رابط‌های مشخصی با ماژول‌های هم‌راستا دارد. مصرف‌کننده اصلی این ماژول، متد simulate_event_driven() از کلاس SinglePhotonDetector است که هسته را به این صورت فراخوانی می‌کند: نخست، آرایه‌های رویداد ساختاریافته را با استفاده از متد کمکی build_event_array() می‌سازد؛ سپس آرایه const_params را با 15 مؤلفه از فیلدهای اعتبارسنجی‌شده آشکارساز پر می‌کند؛ سپس بافر RNG را با rng.random(est_events * rng_buffer_mult) پیش‌بارگذاری می‌کند؛ و در نهایت هسته را فراخوانی کرده و خروجی را به‌صورت ساختاریافته پردازش می‌نماید. این الگوی فراخوانی، جداسازی صریحی میان لایه بالا (ساخت پارامترها و پردازش پسین) و لایه پایین (پردازش رویداد) ایجاد می‌کند.

از منظر ورودی، هسته چندین نوع داده دریافت می‌کند. آرایه sorted_events که از qkd.detector_datastructures با dtype event_np_dtype وارد می‌شود، شامل رویدادهای کلیک سیگنال (که از خروجی کانال فیبر و آشکارساز منبع مشتق شده‌اند) و رویدادهای کلیک IMD/FWM (که از تحلیل SCM/WDM می‌آیند) است. این رویدادها پیش از فراخوانی هسته بر اساس زمان مطلق مرتب می‌شوند تا هسته بتواند به‌صورت کارآمد با یک عبور خطی آن‌ها را پردازش کند. آرایه cascade_depths_in که عمق آبشار فعلی AP معلق برای هر آشکارساز را در بر دارد، از DetectorState آشکارساز استخراج می‌شود.

از منظر خروجی، هسته چندین ساختار داده تولید می‌کند. آرایه‌های out_times، out_dets و out_sigs (که به‌عنوان بافر از سوی فراخواننده تخصیص داده می‌شوند و هسته تنها آن‌ها را پر می‌کند) حاوی زمان‌های کلیک، شناسه آشکارساز و پرچم منشأ سیگنال برای رویدادهای کلیک موفق هستند. این آرایه‌ها در متد

`_simulate_event_driven()` به مرحله بن کردن (binning) ارسال می‌شوند که در آن زمان‌های کلیک با لرزش گوسی به پالس‌های متناظر نگاشته می‌شوند و در نهایت آرایه‌های بولی `click0` و `click1` تولید می‌گردند. آرایه `diags` شامل 8 شمارنده تشخیصی است که در شیء `DetectionDiagnostics` کپی می‌شوند و به مصرف‌کنندگان نهایی (مانند تحلیل کلید امن) ارائه می‌گردند.

تاپل `state_out` که حاوی 14 مؤلفه است، برای به‌روزرسانی `_DetectorState` آشکارساز استفاده می‌شود. این به‌روزرسانی به‌صورت زیر انجام می‌شود: زمان‌های آخرین کلیک `D0/D1` در فیلدهای `last_abs_fire_time_ns_d0/d1` ذخیره می‌شوند؛ زمان‌های `AP` معلق `D0/D1` در `pending_ap_time_d0/d1`؛ عمق‌های آبخار در `ap_depth_d0/d1`؛ زمان‌های `DC` بعدی در `next_dc_time_d0/d1`؛ و آرایه‌های سرریز `AP` در فیلدهای مخصوص به خود. این به‌روزرسانی پیوسته تضمین می‌کند که بلاک بعدی شبیه‌سازی می‌تواند حالت دقیق را از بلاک قبلی به ارث ببرد و پیوستگی فیزیکی زمان مرده، پالس پسین و شمارش تاریک را حفظ کند.

از منظر وابستگی‌های درون‌پروژه‌ای، این ماژول به‌صورت سختگیرانه به ماژول `qkd.detector_datastructures` وابسته است و در صورت عدم دسترسی، به‌جای `fallback` خاموش، استثنای `ImportError` صادر می‌کند. این وابستگی شامل دو نهاد است: ثابت `_SENTINEL_T` (یک مقدار شناور بسیار بزرگ که به‌عنوان سنتینل «هیچ رویداد» استفاده می‌شود) و `event_np_dtype` (یک `NumPy` `structured dtype` که شکل هر رویداد را تعریف می‌کند). ثابت‌های شاخص `CP_*` و `DIAG_*` و `STATUS_*` نیز از ماژول `qkd.detector_datastructures` (یا ماژول خواهری آن) واردات می‌شوند تا یکپارچگی شاخص‌ها در سراسر فریم‌ورک تضمین شود. این طراحی یک-منبع (single source of truth) برای ثابت‌های شاخص از واگرایی احتمالی جلوگیری می‌کند.

تعامل با کتابخانه `Numba` نیز از طریق `try/except` انجام می‌شود. در صورت دسترس بودن `Numba`، دکوراتور `njit` به‌صورت پویا واردات می‌شود و در غیر این صورت، یک دکوراتور `no-op` جایگزین تعریف می‌گردد. این رویکرد امکان استفاده از ماژول را در محیط‌هایی که `Numba` نصب نیست (مانند برخی محیط‌های `CI` سبک) فراهم می‌سازد. تصمیم آگاهانه برای عدم اعلام صریح امضا روی تابع هسته (`no explicit signature on the hot-loop kernel`) برای جلوگیری از وابستگی‌های شکننده `ABI` به `numba.types.Tuple` و `numba.types.Array` که در نسخه‌های مختلف `Numba` تغییر کرده‌اند، اتخاذ شده است. این طراحی به ماژول اجازه می‌دهد با طیف وسیع‌تری از نسخه‌های `Numba` سازگار بماند.

لاگر ماژول (`logging.getLogger(__name__)`) در سطح `WARNING` برای گزارش شکست کامپایل `Numba` و در سطح `INFO` برای گزارش رویدادهای مهم (مانند بازگشت به `pure-Python`) به کار می‌رود. این سطح‌بندی به کاربران اجازه می‌دهد با تنظیم سطح لاگ، میزان جزئیات گزارش‌ها را کنترل کنند. لاگ سطح `DEBUG` در این ماژول استفاده نمی‌شود زیرا هسته در حلقه داغ اجرا می‌شود و هرگونه لاگ اضافی به‌طور قابل‌توجهی عملکرد را کاهش می‌دهد؛ به‌جای آن، آمار تفکیکی در آرایه `diags` جمع‌آوری می‌شود که پس از پایان اجرای هسته توسط مصرف‌کننده قابل بررسی است.

در نهایت، این ماژول به‌صورت مستقیم با مکانیزم `state snapshot/restore` `RNG` ماژول `qkd.detectors` تعامل دارد. مصرف‌کننده پیش از فراخوانی هسته، حالت `RNG` را با `copy_rng_state(rng.bit_generator.state)` عکس‌برداری می‌کند و در صورت نیاز به تلاش مجدد با بافر بزرگ‌تر، حالت را بازیابی می‌نماید. این مکانیزم تضمین می‌کند که تلاش‌های مجدد با همان بذر `RNG` نتایج یکسانی تولید کنند (تا جایی که بافر `RNG` بزرگ‌تر اجازه می‌دهد). نکته مهم این است که پس از اصلاح `Review-v12 F1`، حالت `RNG` دیگر پس از `pre-draw` بازگردانده نمی‌شود؛ زیرا این بازگردانی به‌صورت قطعی کش‌های `RNG` هسته و پس-هسته را همبسته می‌کرد و آمار `ISI`، `gate-loss` و `double-click` را تحریف می‌نمود. این تصمیم طراحی برای بازتولیدپذیری صحیح آماری حیاتی است.

۱۱.۵ نمای کلی و هدف ماژول

ماژول `qkd._gating` یک لایه اعتبارسنجی و گارد (`validation / gating layer`) است که به‌صورت هدفمند از کلاس اصلی `FiberChannel` استخراج شده تا نگرانی‌های مرتبط با کنترل سلامت فیزیکی و سازگاری پیکربندی کانال را در یک مجموعه منسجم از توابع مستقل متمرکز کند. این ماژول در راستای اصل مهندسی نرم‌افزار تفکیک نگرانی‌ها (`separation of concerns`) طراحی شده و هر تابع آن به‌جای ارجاع به `self` از پارامتر صریح `channel` به‌عنوان آرگومان اول استفاده می‌کند؛ این طراحی امکان آزمون‌پذیری واحد (`unit testability`)، استفاده مجدد در زمینه‌های متفاوت (مانند اعتبارسنجی پس از `deserialization`) و امکان جایگزینی آسان در `mock`‌های تست را فراهم می‌سازد. هدف نهایی، فراهم‌آوردن یک لایه دفاعی چندگانه (`defense in depth`) است که هرگونه پیکربندی غیرفیزیکی کانال را پیش از ورود به حلقه اصلی شبیه‌سازی شناسایی و رد کند.

اهمیت این ماژول از آنجا ناشی می‌شود که فریم‌ورک شبیه‌سازی QKD به‌عنوان یک ابزار تحقیقاتی-آکادمیک، همواره با پیکربندی‌های متنوع و گاهی غیرعادی از سوی کاربران مواجه است؛ مقادیر منفی برای طول فیبر، ضرایب تضعیف غیرمنطقی (مانند $\alpha = 0$ با طول غیرصفر که فیبر بی‌تلفاتی را بر روی فاصله متناهی ارائه می‌دهد و از نظر فیزیکی محال است)، طول‌موج‌های خارج از بازه مخابراتی، و تطبیق نادرست میان `total_loss_db` و حاصلضرب $\alpha \cdot L$ تنها چند نمونه از خطاهای متداول هستند. اگر این خطاها به‌صورت خاموش از لایه اعتبارسنجی عبور کنند، در محاسبات پایین‌دستی مانند فرمول‌های بازده `decoy-state`، نرخ کلید امن و QBER به نتایج ظاهراً معتبر اما فیزیکی غلط منجر می‌شوند که می‌توانند ماه‌ها تلاش تحقیقاتی را بی‌اثر کنند. این ماژول با سیاست دو-سطحی `strict/non-strict` (که توسط `ContextVar` جهانی `get_strict_mode()` کنترل می‌شود) به کاربر اجازه می‌دهد در حالت پیش‌فرض سخت‌گیرانه هرگونه تخلف را به‌صورت استثنا مشاهده کند و در حالت اکتشافی (`exploratory`) صرفاً هشدار دریافت نماید.

ماژول `gating` شامل چهارده تابع عمومی است که در `__all__` فهرست شده‌اند: `check_type`، `check_meaningful_distance`، `check_meaningful_alpha`، `validate_parameter_ranges`، `check_total_loss_finite`، `check_attenuation_config_contract`، `compute_transmittance`، `validate_wavelength_range`، `check_wavelength_consistency`، `transmittance_from_loss_db`، `effective_total_loss_db`، `check_transmittance_health`، `validate_channel` و `validate_parameter`. هر یک از این توابع به یک جنبه خاص از سلامت کانال می‌پردازند و در مجموع یک شبکه اعتبارسنجی کامل را تشکیل می‌دهند که از نوع شیء `attenuation` آغاز شده و به انسجام حالت مشتق‌شده (`derived state`) مانند `transmittance` ختم می‌شود.

این ماژول به‌عنوان زیرسیستمی فرعی از معماری بزرگ‌تر `qkd.channel` عمل می‌کند و در خط لوله ساخت کانال به‌صورت متوالی فراخوانی می‌شود. در معماری کلی، پس از ساخت اولیه شیء `FiberChannel` (که خود یک `dataclass` ایستا با فیلدهای `wavelength_nm`، `attenuation` و `total_loss_db_override` است)، متد `__post_init__` این کلاس توابع گیتینگ را به‌ترتیب زیر فرا می‌خواند: نخست `check_type`، سپس `validate_parameter_ranges`، سپس `check_meaningful_alpha` و `check_meaningful_distance`، سپس `check_total_loss_finite` و `check_attenuation_config_contract`، و در نهایت `check_wavelength_consistency` و `validate_wavelength_range` و `check_transmittance_health`. این ترتیب به‌گونه‌ای انتخاب شده که هر گیت بر اساس خروجی‌های گیت‌های قبلی عمل کند و در صورت بروز خطا در هر مرحله، مراحل بعدی اجرا نشوند.

۱۲.۵ مبانی تئوری و فیزیکی

بنای نظری این ماژول بر چند اصل بنیادین فیزیک فیبر نوری و نظریه اعتبارسنجی عددی استوار است. نخست، قانون بیر-لامبرت (`Beer-Lambert law`) که پایه محاسبه انتقال (`transmittance`) کانال است. این قانون بیان می‌کند که شدت نور پس از عبور از یک محیط با ضریب تضعیف α به طول L به‌صورت نمایی کاهش می‌یابد:

$$T = \frac{I(L)}{I(0)} = 10^{-\alpha L/10} \quad (16.5)$$

که در آن α بر حسب dB/km ، L بر حسب km و T یک عدد بی‌بعد در بازه $[0, 1]$ است. در این ماژول، تابع `transmittance_from_loss_db()` این محاسبه را با استفاده از `math.pow` (به‌جای عملگر `**`) پیاده‌سازی می‌کند تا در برابر `OverflowError` در پلتفرم‌های مختلف مقاوم باشد. یک نکته ظریف اما مهم این است که برای $L_{total} = 0$ (یعنی فیبر بی‌تلفات با طول صفر)، انتقال برابر 1 بازگردانده می‌شود، اما این شاخه با استفاده از برابری دقیق `L_total == 0.0` (به‌جای `is_close` با تolerance 10^{-12}) ارزیابی می‌شود. این انتخاب عمدی است: تolerance 10^{-12} قبلی برای $\alpha = 0.2$ و $L = 5 \times 10^{-12}$ km ، مقدار غیرصفر L_{total} را به‌صورت خاموش صفر تلقی می‌کرد و انتقال نادرست 1.0 بازمی‌گرداند.

دومین اصل، رفتار اعداد ممیز-شناور IEEE-754 و پدیده‌های پاتولوژیک آن، مانند صفر منفی 0.0 و اعداد زیرنرمال `subnormal` است. در استاندارد IEEE-754، رابطه `0.0 == -0.0` برقرار است، اما این دو مقدار از نظر فیزیکی متفاوت تلقی می‌شوند؛ زیرا 0.0 معمولاً نشان‌دهنده یک خطای علامت در بالادست `upstream` `sign error` یا حاصل سرریز یک مقدار منفی بسیار کوچک (مانند -1×10^{-324} که به 0.0 تبدیل می‌شود) است. این ماژول با استفاده از `math.copysign(1.0, value) < 0` به‌صورت صریح 0.0 را تشخیص داده و در حالت سخت‌گیرانه، استثنا صادر می‌کند. به‌طور مشابه، اعداد زیرنرمال `subnormal floats` که کوچک‌تر از

`sys.float_info.min` هستند، نهایتاً یک بیت دقت مانتیس دارند و در فرمول‌های QKD (مانند محاسبه Y_n یا e_1) به نتایج کاملاً نادرست منجر می‌شوند؛ لذا این مازول هرگونه عدد زیرنرمال را به صورت غیرقابل بازگشت رد می‌کند.

سومین اصل، مدل‌سازی فیزیکی ضرایب تضعیف بر اساس طول‌موج است. فیلرهای سیلیکایی در طول‌موج‌های استاندارد مخابراتی دارای ضرایب تضعیف مشخصی هستند: در 850 nm (پنجره اول) نزدیک به 2.5 dB/km، در 1310 nm (پنجره دوم) نزدیک به 0.35 dB/km، و در 1550 nm (پنجره سوم) نزدیک به 0.20 dB/km. این مازول با استفاده از دیکشنری `TYPICAL_WAVELENGTH_ALPHA_RANGES` بازه‌های مجاز را تعریف می‌کند و تابع `check_wavelength_consistency()` نزدیک‌ترین طول‌موج شناخته شده را با `WAVELENGTH_CHECK_TOLERANCE_NM` پیدا کرده و اعتبارسنجی می‌کند. این بررسی از ارسال پیکربندی‌هایی مانند $\alpha = 0.05$ dB/km در 1550 nm (که از نظر فیزیکی محال است زیرا کم‌تلفات‌ترین فیبر تجاری در این طول‌موج نزدیک به 0.15 dB/km است) جلوگیری می‌کند.

چهارمین اصل، مفهوم قرارداد ساختاری (structural contract) میان `dataclass`‌های مرتبط است. در این فریم‌ورک، `AttenuationConfig` یک `dataclass` با سه فیلد اصلی `fiber_length`، `attenuation_coefficient` و `total_loss_db` است و قرارداد آن این است که `total_loss_db` باید برابر با `attenuation_coefficient * fiber_length` باشد. این قرارداد به صورت ضمنی توسط تمامی کدهای پایین‌دستی فرض می‌شود؛ به‌ویژه، محاسبه انتقال $T = 10^{-L_{total}/10}$ به این قرارداد وابسته است. تابع `check_attenuation_config_contract()` این قرارداد را با تلورانس نسبی `_CONTRACT_CHECK_ABS_TOL` و تلورانس مطلق `_CONTRACT_CHECK_REL_TOL` (که هر دو در `_schema` تعریف شده‌اند) اعتبارسنجی می‌کند. یک جنبه ظریف این است که وقتی `total_loss_db_override` تنظیم شده باشد (یعنی کاربر به صورت دستی مقدار `total_loss_db` را تنظیم کرده باشد)، قرارداد همچنان بررسی می‌شود اما با تلورانس نسبی سخت‌گیرانه 10^{-9} ؛ زیرا فرض می‌شود `override` باید نهایتاً چند ULP از $\alpha \cdot L$ فاصله داشته باشد و انحراف بزرگ‌تر نشانگر یک پیکربندی ناسازگار است.

پنجمین اصل، مدیریت هشدارهای فیزیکی-مشکوک (physically suspicious) است. این مازول دو سطح پاسخ را تفکیک می‌کند: استثنای سخت (`ParameterValidationError`) که در حالت سخت‌گیرانه برای خطاهای قطعی (مانند $T = \text{NaN}$ یا $\alpha = 0$ با $L > 0$) صادر می‌شود، و هشدار نرم (`PhysicalSuspicionWarning`) که از طریق تابع کمکی `warn_physical_suspicion()` برای موارد مشکوک اما نه قطعاً غلط (مانند $\alpha < 0.05$) بدون تنظیم طول‌موج) منتشر می‌شود. این هشدارها به صورت پیش‌فرض توسط پایتون نمایش داده می‌شوند (زیرا از `warnings.warn` با کلاس هشدار غیر-`DeprecationWarning` استفاده می‌کنند) و در عین حال توسط کاربر قابل فیلتر هستند.

۱۳.۵ تحلیل معماری و ساختار داده‌ها

معماری این مازول تابع-محور (function-oriented) است و از الگوی تزریق وابستگی (dependency injection) برای جداسازی منطق اعتبارسنجی از شیء کانال استفاده می‌کند. هر تابع به جای دسترسی به فیلدهای `self`، یک شیء `channel` (از نوع Any) دریافت می‌کند و از طریق ویژگی‌های آن شیء (`channel.attenuation`)، `channel.wavelength_nm`، `channel._total_loss_db_override` (غیره) به داده‌ها دسترسی پیدا می‌کند. این طراحی مزایای متعددی دارد: نخست، توابع قابل استفاده مجدد برای هر شیء‌ای که پروتکل duck-typed مشابهی داشته باشد (نه فقط `FiberChannel` اصلی) هستند؛ دوم، توابع به راحتی با اشیاء `mock` قابل آزمون هستند؛ سوم، وابستگی‌های مازول حداقل است و تنها به مازول‌های `qkd.exceptions`، `qkd.constants`، `qkd.datatypes` و `qkd._schema` نیاز دارد.

ورودی‌های مازول از طریق `import` از مازول `_schema` فراهم می‌شوند و شامل چندین دسته از ثابت‌ها و توابع هستند. ثابت‌های فیزیکی و سخت‌گیرانه شامل: `TRANSMITTANCE_SUBNORMAL_THRESHOLD` (آستانه تشخیص اعداد زیر-نرمال، معمولاً 2.2×10^{-308})، `MAX_DISTANCE_KM` و `WARNING_DISTANCE_KM` (کران‌های فیزیکی برای طول فیبر)، `MAX_FIBER_LOSS_DB_KM` (کران فیزیکی برای ضریب تضعیف)، `TYPICAL_WAVELENGTH_ALPHA_RANGES` (دیکشنری بازه‌های مجاز α بر اساس طول‌موج)، و `WAVELENGTH_CHECK_TOLERANCE_NM` (تلورانس جست‌وجوی طول‌موج) هستند. ثابت‌های تلورانس عددی شامل: `_TRANSMITTANCE_UNDERFLOW_ABS_TOL` (که برابر با `sys.float_info.min` است و برای مقایسه انتقال‌های زیر-نرمال به کار می‌رود)، `_CONTRACT_CHECK_REL_TOL` و `_CONTRACT_CHECK_ABS_TOL` (برای اعتبارسنجی قرارداد `AttenuationConfig`)، و `_ZERO_LOSS_ABS_TOL` هستند.

مازول از مکانیزم `ContextVar` پایتون برای کنترل حالت سخت‌گیرانه استفاده می‌کند. تابع

`get_strict_mode()` مقدار بولی این حالت را برمی‌گرداند و در تمامی توابع گیتینگ برای تصمیم‌گیری میان صدور استثنا یا هشدار به کار می‌رود. این مکانیزم به جای `ClassVar` از `ContextVar` استفاده می‌کند تا امکان تنظیم محلی (`thread-local / context-local`) حالت سخت‌گیرانه را در تست‌ها و شبیه‌سازی‌های موازی فراهم کند؛ به این معنا که یک تست می‌تواند بدون تأثیر بر سایر تست‌ها، حالت سخت‌گیرانه را غیرفعال کند. علاوه بر این، `SKIP_TRANSMITTANCE_HEALTH` یک `ContextVar` بولی دیگر است که اجازه می‌دهد در مسیرهای سریع ساخت (`fast-construction paths`) از بررسی سلامت انتقال صرف‌نظر شود.

دیکشنری `VALIDATED_UNITS` نقشه‌ای از نام پارامتر به واحد آن (مانند: `"km"`, `{"distance_km": "km"}`, `"dB/km"`, `"fiber_loss_db_km": "dB/km"`) است که در پیام‌های خطا برای ارائه توضیحات کاربر-پسند استفاده می‌شود. این رویکرد به ماژول امکان می‌دهد بدون سخت‌کد نویسی واحد در هر پیام خطا، پیام‌های یکپارچه و قابل‌توسعه تولید کند. تابع `validate_parameter(name, value, upper_bound)` یکی از این دیکشنری برای استخراج واحد استفاده می‌کند و در عین حال با بررسی ویژگی‌های `shape` و `dtype` ورودی، آرایه‌های `NumPy/JAX/Torch` را به صورت زود هنگام رد می‌کند؛ زیرا `math.isfinite` بر روی آرایه‌ها با `TypeError` مبهم مواجه می‌شود. این لایه دفاعی جلوی عبور آرایه‌ها به لایه‌های پایین‌تر اعتبارسنجی که انتظار اسکالر دارند را می‌گیرد.

استثنای اصلی ماژول، `ParameterValidationError` است که از ماژول `qkd.exceptions` فراخوانی شده و دارای امضای جامعی به صورت `(msg, *, param_name=None, param_value=None, context=None)` می‌باشد. این امضای ساختاریافته به کاربر اجازه می‌دهد خطاها را به صورت برنامه‌نویسی پردازش کند (مانند فیلتر بر اساس `param_name`) و در عین حال، پیام‌های خطای خوانایی برای انسان تولید نماید. فیلد `context` یک دیکشنری است که اطلاعات تکمیلی نظیر مقادیر مرتبط، مسیر ساخت و تلورانس‌های استفاده‌شده را در خود نگه می‌دارد؛ این داده‌ها برای دیباگ و گزارش‌گیری خودکار بسیار ارزشمند هستند. لاگر ماژول از طریق `logging.getLogger(__name__) = logger` ساخته شده و نام آن به `"qkd.gating"` تنظیم می‌شود. این لاگر در سطح `DEBUG` برای گزارش وضعیت اعتبارسنجی موفق و در سطح `WARNING` برای هشدارهای غیرسخت‌گیرانه به کار می‌رود. این دوگانگی به کاربر اجازه می‌دهد با تنظیم سطح لاگ، میزان جزئیات گزارش‌ها را کنترل کند: سطح `WARNING` (پیش‌فرض) تنها هشدارهای مهم را نشان می‌دهد، در حالی که سطح `DEBUG` جریان کامل اعتبارسنجی را برای دیباگ نمایش می‌دهد.

۱۴.۵ پیاده‌سازی ریاضی و الگوریتمی

الگوریتم محاسبه انتقال در تابع `transmittance_from_loss_db()` با چندین محافظت عددی پیاده‌سازی شده است. کد زیر نسخه کامل این تابع را نشان می‌دهد:

```

1 def transmittance_from_loss_db(total_db: float) -> float:
2     if total_db == :0.0
3         # F-31 / P-06 / N-1: check for negative zero.
4         # ``0.0- == 0.0`` is True in IEEE 754, so this branch
5         # activates for .0.0- Negative zero can only arise from
6         # (a) an upstream sign error, or (b) a very small
7         # negative value underflowing (e.g. -1e-324 -> (0.0-,
8         # which would represent gain. In both cases returning
9         # transmittance0.1= is physically unsafe, so raise
10        # unconditionally.
11        if math.copysign(0.1), total_db) < 0:
12            raise ParameterValidationError(
13                f"total_loss_db is negative zero ((0.0- . ...",
14                param_name="total_loss_db",
15                param_value=total_db,
16            )
17        return 0.1
18    if total_db < :0.0
19        raise ParameterValidationError(
20            f"total_loss_db is negative ({total_db!r}), implying "
21            f"gain rather than attenuation. This is physically "
22            f"impossible for a passive fiber channel. ...",
23            param_name="total_loss_db",
24            param_value=total_db,
25        )

```

```

۲۶     return math.pow(0.10), -total_db / (0.10)
۲۷

```

این پیاده‌سازی سه شاخه دارد. شاخه نخست ($L_{total} = 0$) با برابری دقیق ارزیابی می‌شود و در آن ابتدا علامت صفر با `copysign` بررسی می‌شود؛ اگر منفی باشد، استثنا صادر می‌شود زیرا -0.0 معمولاً نشانه یک خطای بالا-دست است. شاخه دوم ($L_{total} < 0$) به صورت غیرقابل-بازگشت رد می‌شود زیرا $L_{total} < 0$ به معنای تقویت (gain) است که برای یک فیبر پسیو فیزیکی محال است. شاخه سوم از `math.pow` به جای عملگر `**` استفاده می‌کند زیرا `math.pow` در برابر `OverflowError` در پلتفرم‌های مختلف رفتار یکنواخت‌تری دارد. باید توجه داشت که برای مقادیر بسیار بزرگ L_{total} (مانند $L_{total} > 308 \times 10 / \ln 10 \approx 1338$)، `math.pow dB`، $L_{total} > 308 \times 10 / \ln 10 \approx 1338$ به `0.0` سرریز می‌شود که خود در تابع `check_transmittance_health()` به عنوان خطای `underflow` شناسایی می‌شود.

تابع `check_transmittance_health()` چهار سطح کنترل سلامت را به صورت متوالی اعمال می‌کند. این تابع پس از محاسبه انتقال در مسیر ساخت کانال فراخوانی می‌شود و آن را برای پاتولوژی‌های مختلف بررسی می‌کند:

```

1  def check_transmittance_health(channel, transmittance):
2      # NaN: reject unconditionally.
3      if math.isnan(transmittance):
4          raise ParameterValidationError(
5              f"Transmittance computed as NaN for distance_km="
6              f"{channel.attenuation.fiber_length!r}, ...",
7              param_name="transmittance",
8              param_value=transmittance,
9              context={...},
10             )
11
12     # P-06: Detect negative-zero transmittance.
13     if transmittance == 0.0 and math.copysign(0.1), transmittance) < 0:
14         # ... issue PhysicalSuspicionWarning or raise in strict mode
15
16     total_db = effective_total_loss_db(channel)
17
18     # Underflow: exact-zero transmittance with non-zero total_db.
19     if transmittance == 0.0 and total_db > :0.0
20         raise ParameterValidationError(
21             f"Transmittance underflowed to exactly 0.0 for ...",
22             param_name="transmittance", ...
23         )
24
25     # Subnormal transmittance.
26     elif 0.0 < transmittance < TRANSMITTANCE_SUBNORMAL_THRESHOLD:
27         raise ParameterValidationError(
28             f"Transmittance {transmittance!r} is subnormal ...",
29             param_name="transmittance", ...
30         )
31
32     # Range validation.
33     if not 0.0) <= transmittance <= (0.1:
34         raise ParameterValidationError(
35             f"Transmittance {transmittance!r} is out of the physical "
36             f"range 0.0], .[0.1 ...",
37             param_name="transmittance", ...
38         )
39

```

این تابع به ترتیب چهار بررسی را انجام می‌دهد: NaN (رد غیرقابل-بازگشت)، صفر منفی (هشدار یا رد بر اساس حالت سخت‌گیرانه)، `underflow` (یعنی $T = 0$ با $L_{total} > 0$ که رد غیرقابل-بازگشت است زیرا در فرمول‌های QKD منجر به تقسیم بر صفر می‌شود)، `subnormal` (رد غیرقابل-بازگشت زیرا اعداد زیر-نرمال نهایتاً یک بیت دقت دارند و در محاسبات به نتایج تصادفی منجر می‌شوند) و در نهایت بررسی بازه $[0, 1]$. توجه شود که

هیچ‌کدام از این بررسی‌ها به‌صورت خاموش مقدار را تصحیح نمی‌کنند (clamp نمی‌کنند)؛ زیرا چنین تصحیحی می‌تواند خطای بالا-دست را پنهان کند و به نتایج ظاهراً معتبر اما فیزیکی غلط منجر شود. این سیاست صریح در داک‌استریکینگ تابع به‌صورت `We silently clamp NOT do explicitly` مستند شده است.

تابع `check_attenuation_config_contract()` دارای دو مسیر است: مسیر `override` (وقتی `total_loss_db_override` تنظیم شده) و مسیر قرارداد استاندارد. در مسیر `override`، تابع بررسی می‌کند که آیا `override` به‌طور وحشتناکی با $\alpha \cdot L$ ناسازگار است یا خیر. این بررسی با تلورانس نسبی 10^{-9} انجام می‌شود که عمده‌اً سخت‌گیرانه است زیرا فرض می‌شود `override` نهایتاً چند ULP از $\alpha \cdot L$ فاصله داشته باشد:

$$|L_{\text{override}} - \alpha \cdot L| > 10^{-9} \cdot \max(|\alpha \cdot L|, 1) \Rightarrow \text{خطا} \quad (17.5)$$

در مسیر قرارداد استاندارد، یعنی در حالتی که `override` فعال نیست، تابع با استفاده از تلورانس‌های `_CONTRACT_CHECK_ABS_TOL` و `_CONTRACT_CHECK_REL_TOL` بررسی می‌کند که مقدار `total_loss_db_override` با مقدار αL سازگار باشد. نکته مهم در مسیر `override` آن است که اگر $\alpha = 0$ ، $L = 0$ و مقدار `override` نیز برابر با صفر باشد، تابع بدون ایجاد خطا بازمی‌گردد؛ زیرا این حالت متناظر با یک کانال بدیهی است که نیازی به بررسی ندارد. در مقابل، اگر $L > 0$ و $\alpha = 0$ باشد، یعنی با فیلتری بی‌تلفات و دارای طول ناصفر مواجه باشیم، بررسی قرارداد همچنان انجام می‌شود؛ زیرا چنین حالتی از نظر فیزیکی نامعتبر است.

تابع `check_wavelength_consistency()` با جست‌وجو در `TYPICAL_WAVELENGTH_ALPHA_RANGES`، نزدیک‌ترین طول‌موج شناخته‌شده را تعیین می‌کند. این جست‌وجو به‌صورت ترتیبی روی طول‌موج‌های مرتب‌شده انجام می‌شود تا در صورت تساوی فاصله، نزدیک‌ترین طول‌موج کوتاه‌تر به‌طور قطعی برگزیده شود. تلورانس جست‌وجو با استفاده از `math.nextafter(WAVELENGTH_CHECK_TOLERANCE_NM, float("inf"))` استفاده می‌گردد؛ استفاده از `nextafter` در اینجا آگاهانه است و از رد طول‌موج‌هایی که در عمل روی مرز تلورانس هستند اما به‌دلیل خطاهای گردکردن IEEE-754 اندکی فراتر می‌روند، جلوگیری می‌کند. در ادامه، بررسی می‌شود که آیا α در بازه مجاز $[\alpha_{\min}, \alpha_{\max}]$ برای آن طول‌موج قرار دارد یا خیر؛ در صورت عدم تطابق، در حالت سخت‌گیرانه استثنا صادر می‌شود و در غیر این صورت، هشدار `PhysicalSuspicionWarning` نمایش داده می‌شود.

تابع `validate_parameter(name, value, upper_bound)` شامل چندین سازوکار حفاظتی است. ابتدا، خود پارامتر `upper_bound` بررسی می‌شود تا نامنفی بودن آن تضمین شود؛ زیرا مقدار منفی برای این پارامتر می‌تواند همه مقادیر نامنفی را به‌صورت خاموش معتبر تلقی کرده و عملاً سازوکار اعتبارسنجی را غیرفعال کند. سپس، ورودی پیش از ادامه پردازش بررسی می‌شود تا از ایجاد خطاهای مبهم از نوع `TypeError` جلوگیری شود. اگر ورودی دارای ویژگی‌های `shape` و `dtype` باشد، به این معنا که یک آرایه از نوع `NumPy/JAX/Torch` است، تابع استثنا صادر می‌کند.

در مرحله بعد، تابع `is_finite_non_negative(value)` درون یک بلوک `try/except` فراخوانی می‌شود تا `TypeError` ناشی از ورودی‌های مختلط `complex` به `ParameterValidationError` تبدیل شود. در نهایت، اگر شرط `value > upper_bound` برقرار باشد، استثنای `ParameterValidationError` همراه با واحد متناظر استخراج‌شده از `VALIDATED_UNITS` صادر می‌شود.

تابع مرکزی `validate_channel()` تمامی بررسی‌های بالا را به‌صورت یکپارچه فراخوانی می‌کند و علاوه بر آن‌ها، انسجام حالت مشتق‌شده را نیز اعتبارسنجی می‌کند. این تابع برای کشف فساد حالت ناشی از `pickling`، `monkey-patching` یا ارتقای ماژول `qkd.datatypes` طراحی شده و در API عمومی به‌عنوان نقطه ورودی برای اعتبارسنجی مجدد در نظر گرفته شده است. تابع به‌صورت صریح در `__post_init__` فراخوانی نمی‌شود زیرا آن بررسی‌ها از قبل انجام شده‌اند؛ اما کاربر می‌تواند آن را به‌صورت دستی پس از عملیاتی که ممکن است حالت را فاسد کند فراخوانی نماید. مکانیزم اختیاری `set_validate_on_construction(True)` اجازه می‌دهد این تابع در انتهای هر `__post_init__` به‌صورت خودکار فراخوانی شود تا اطمینان اضافی فراهم شود.

۱۵.۵ ارسال و دریافت با سایر ماژول‌ها

این ماژول به‌عنوان یک لایه میانی در معماری فریم‌ورک عمل می‌کند و رابط‌های مشخصی با ماژول‌های بالا-دست و پایین-دست دارد. از منظر بالا-دست، مصرف‌کننده اصلی این ماژول کلاس `FiberChannel` در `qkd.channel` است که در متد `__post_init__` خود توابع گیتینگ را به‌ترتیب زیر فرا می‌خواند: نخست `check_type()` برای اطمینان از اینکه `channel.attenuation` یک `AttenuationConfig` واقعی است (نه مثلاً یک دیکشنری خام که بعداً منجر به `AttributeError` می‌شود)، سپس `validate_parameter_ranges()`

برای بررسی $L \leq \text{MAX_DISTANCE_KM}$ و $\alpha \leq \text{MAX_FIBER_LOSS_DB_KM}$ سپس چهار بررسی معناداری فیزیکی ($_meaningful_alpha/distance$)، سپس `check_total_loss_finite()` و `check_attenuation_config_contract()`، و در نهایت بررسی‌های طول موج و سلامت انتقال. این ترتیب به‌گونه‌ای است که هر گیت بر اساس خروجی گیت‌های قبلی عمل کند.

ماژول‌های دیگر که از این لایه استفاده می‌کنند شامل: `qkd.simulation_driver` که در زمان ساخت کانال‌های متعدد برای شبیه‌سازی Monte Carlo، توابع گیتینگ را به‌صورت ضمنی از طریق سازنده `FiberChannel` فراخوانی می‌کند؛ `qkd.state_manager` که پس از بارگذاری حالت کانال از دیسک، `validate_channel()` را فراخوانی می‌کند تا از انسجام حالت بارگذاری‌شده اطمینان حاصل کند؛ و `qkd.cli` که در زمان پردازش آرگومان‌های خط فرمان، `validate_parameter()` را برای اعتبارسنجی اولیه پارامترها پیش از ساخت کانال به کار می‌برد.

از منظر پایین-دست، خروجی اصلی این ماژول مقادیر اعتبارسنجی‌شده انتقال T و L_{total} هستند که توسط کانال فیبر در اختیار ماژول آشکارساز (`qkd.detectors`) قرار می‌گیرند. به‌طور خاص، تابع `compute_transmittance()` انتقال نهایی را به‌عنوان یک `float` در بازه $[0, 1]$ بازمی‌گرداند که سپس به‌عنوان آرگومان `channel_transmittance` به متد `simulate_detection()` آشکارساز ارسال می‌شود. اعتبارسنجی سخت‌گیرانه این ماژول تضمین می‌کند که این مقدار هرگز NaN، زیر-نرمال، یا خارج از بازه فیزیکی نباشد، که در نتیجه از خطاهای خاموش در محاسبات بازده `decoy-state` و نرخ کلید جلوگیری می‌کند.

از منظر وابستگی‌های درون‌پروژه‌ای، این ماژول به‌صورت مستقیم به چهار ماژول زیرین وابسته است: `qkd.exceptions` برای `ParameterValidationError`؛ `qkd.constants` برای توابع کمربندی `is_close` و `qkd.datatypes`؛ `is_finite_non_negative` برای `qkd._schema`؛ و `ContextVar`، تایپ‌ها و سنتینل‌ها. این وابستگی‌ها عمدتاً به‌صورت `static` (در زمان `import`) حل می‌شوند و این ماژول بر خلاف ماژول‌هایی مانند `qkd.detectors` لایه `fallback` برای حالت `standalone` ندارد؛ زیرا فرض بر این است که این ماژول همواره در بستر کامل بسته `qkd` اجرا می‌شود.

لاگر ماژول در سطح `WARNING` برای هشدارهای غیرسخت‌گیرانه و در سطح `DEBUG` برای گزارش جزئیات اعتبارسنجی موفق به کار می‌رود. این دو سطح به کاربران اجازه می‌دهد با تنظیم سطح لاگ، میزان جزئیات گزارش‌ها را کنترل کنند: تنظیم `logging.getLogger("qkd._gating").setLevel(logging.DEBUG)` جریان کامل اعتبارسنجی را برای دیباگ نمایش می‌دهد، در حالی که سطح پیش‌فرض `WARNING` تنها هشدارهای مهم را نشان می‌دهد. این طراحی به‌خصوص برای دیباگ پیکربندی‌های پیچیده که در آن‌ها چندین کانال به‌صورت موازی ساخته می‌شوند (مانند شبیه‌سازی `sweep` روی طول موج یا فاصله) مفید است.

در نهایت، باید به این نکته اشاره کرد که این ماژول به‌صورت مستقیم با مکانیزم `ContextVar` پایتون برای کنترل حالت سخت‌گیرانه تعامل دارد. تابع `get_strict_mode()` در هر فراخوانی مقدار فعلی `ContextVar` را می‌خواند، که به این معنا است که تغییر حالت سخت‌گیرانه در طول اجرا (مانند `with strict_mode_context(False):`) به‌صورت پویا توسط تمامی توابع گیتینگ مشاهده می‌شود. این مکانیزم برای تست‌هایی که نیاز به غیرفعال کردن موقت حالت سخت‌گیرانه دارند (مانند تست‌های `boundary case` که عمده پیکربندی غیرفیزیکی می‌سازند) حیاتی است و در عین حال تضمین می‌کند که حالت پیش‌فرض (سخت‌گیرانه) برای کاربردهای انتشار علمی همواره فعال باشد.

بخش دوم

اثبات‌های امنیتی رمزنگاری

فصل ۶

ساختار بنیادین اثبات امنیت

۱.۶ نمای کلی و هدف ماژول

فایل `qkd/proofs/base.py` ستون فقرات لایه اثبات‌های امنیتی در فریم‌ورک شبیه‌سازی توزیع کلید کوانتومی است. این ماژول یک `Abstract Base Class` به نام `FiniteKeyProof` معرفی می‌کند که قرارداد رابطی همه اثبات‌های امنیتی کلید متناهی را در سراسر سیستم استاندارد می‌سازد. اثبات‌های امنیتی کلید متناهی به دسته‌ای از الگوریتم‌ها گفته می‌شود که به جای اتکای صرف به حد `asymptotic` توزیع کلید، تأمین امنیت برای طول کلید متناهی را با پارامتر امنیتی ϵ_{sec} قابل اثبات تضمین می‌کنند. در دنیای واقعی، تعداد پالس‌های ارسالی همواره محدود است و الگوهای `asymptotic` مانند `Gottesman-Lo-Lütkenhaus-Preskill` سنتی به تنهایی پاسخگو نیستند؛ از این رو لایه‌بندی صریحی از تقسیم ϵ -بودجه، تخمین پارامتر، محاسبه بازده تک‌فوتونی و کسر فاز خطا ضروری می‌شود.

هدف اصلی این ماژول، فراهم آوردن یک رابط یکپارچه برای همه زیرکلاس‌های اثبات است؛ به گونه‌ای که هر زیرکلاس فقط به پیاده‌سازی چهار متد انتزاعی `allocate_epsilon`، `notation_map`، `estimate_yields_and_errors` و `get_epsilon_policy` و کران‌های آماری، آنتروپی باینری، تقسیم ایمن، تشخیص خطاهای عددی و گزارش‌گیری `audit` را از کلاس پایه به ارث ببرد. این طراحی از الگوی `Template Method` پیروی می‌کند و اطمینان می‌دهد که هر پیاده‌سازی اثبات، بدون توجه به اینکه مبتنی بر چارچوب `Ma 2005` باشد یا `Lim 2014` یا `GLLP`، خروجی‌ها و رفتار سازگار و قابل مقایسه تولید می‌کند. به علاوه، کلاس پایه لایه‌ای از دفاع عددی شامل `clamp` احتمالات، `safe log`، `safe divide` و تشخیص مقادیر نامتناهی را فراهم می‌سازد که در محاسبات آنتروپی و `log-likelihood` اهمیت حیاتی دارد. این فایل علاوه بر کلاس پایه، چندین `dataclass` مورد نیاز برای انتقال داده میان لایه‌ها را تعریف می‌کند. این `dataclass`ها شامل `TallyCounts` برای ثبت شمارش‌های آماری، `EpsilonAllocation` برای ذخیره تقسیم بودجه ϵ ، `SolverDiagnostics` برای گزارش وضعیت `solver` برنامه‌ریزی خطی، `DecoyEstimates` برای نگهداری کران‌های پایین بازده تک‌فوتونی و کران بالای خطای تک‌فوتونی و نهایتاً `KeyCalculationResult` برای ارائه نتیجه نهایی محاسبه طول کلید امن است. همه این ساختارها با `@dataclass(frozen=True)` تعریف شده‌اند تا از تغییر ناخواسته پس از ساخت جلوگیری شود و قابلیت `hashing` و استفاده به عنوان کلید در `cache` را داشته باشند. این تصمیم طراحی در اثبات‌های امنیتی بحرانی است، زیرا هرگونه جهش ناگهانی در مقادیر میانی می‌تواند به نتیجه نهایی رنگ امنیتی بزند.

وظیفه نهایی این ماژول، تضمین یکپارچگی و تکرارپذیری نتایج امنیتی است. به همین دلیل، متغیرهای محیطی مانند `lib_version`، `run_id`، نسخه `numpy` و `scipy` در گزارش‌های `audit` ثبت می‌شوند تا هر اجرا به طور کامل قابل بازتولید باشد. این ویژگی برای گزارش‌های امنیتی حساس که نیاز به بازبینی توسط شخص ثالث دارند، الزامی است.

۲.۶ مبانی تئوری و فیزیکی

۱.۲.۶ امنیت قابل ترکیب و تقسیم بودجه ϵ

تعریف مدرن امنیت در QKD بر پایه مفهوم `composable security` استوار است که نخستین بار توسط Ben-Or و همکاران و سپس توسط Canetti بسط داده شد. در این چارچوب، امنیت یک پروتکل با یک پارامتر $\epsilon_{\text{sec}} \in (0, 1)$ کمیت‌سازی می‌شود که نشان می‌دهد فاصله آماری میان خروجی پروتکل و یک حالت ایده‌آل

(توزیع کلید یکنواخت و مستقل از اطلاعات دشمن) چقدر است. اگر ρ_{real} حالت نهایی مشترک آلیس، باب و ایو و ρ_{ideal} حالت ایده‌آل باشد، شرط امنیت به صورت زیر نوشته می‌شود:

$$\frac{1}{2} \|\rho_{\text{real}} - \rho_{\text{ideal}}\|_1 \leq \varepsilon_{\text{sec}}, \quad (۱.۶)$$

که در آن $\|\cdot\|_1$ نرم رد ماتریس چگالی است. این تعریف به ما اجازه می‌دهد پروتکل QKD را مانند یک جعبه سیاه با سایر پروتکل‌های رمزنگاری ترکیب کنیم و جمع کل ε ‌ها از قانون زیر پیروی کند:

$$\varepsilon_{\text{total}} \leq \sum_i \varepsilon_i. \quad (۲.۶)$$

در فایل `base.py`، این بودجه به چهار مؤلفه شکسته می‌شود:

$$\varepsilon_{\text{sec}} = \varepsilon_{\text{PE}} + \varepsilon_{\text{EC}} + \varepsilon_{\text{PA}} + \varepsilon_{\text{sm}}, \quad (۳.۶)$$

که در آن ε_{PE} احتمال شکست تخمین پارامتر، ε_{EC} احتمال شکست تصحیح خطا، ε_{PA} احتمال شکست تقویت حریم خصوصی و ε_{sm} پارامتر هموارسازی در `smooth min-entropy` است. این تقسیم در `EpsilonAllocation` ذخیره شده و در متد `validate` با یک `policy function` بررسی می‌شود که مجموع ε ‌ها از کل مجاز تجاوز نکند.

۲.۲.۶ تخمین پارامتر و کران‌های آماری

در پروتکل‌های Decoy-State، آلیس به طور تصادفی شدت پالس را میان چندین حالت (سیگنال، decoy، کم‌شدت و خلا) تغییر می‌دهد و باب تعداد آشکارسازی‌ها و خطاها را در هر حالت می‌شمارد. هدف، تخمین کران پایین بازده تک‌فوتونی Y_1^L و کران بالای خطای تک‌فوتونی e_1^U است. به دلیل ماهیت تصادفی این شمارش‌ها، تخمین قطعی ممکن نیست و باید از کران‌های آماری استفاده کرد. این مازول سه روش را پشتیبانی می‌کند: Clopper-Pearson، Hoeffding، Gaussian.

Clopper-Pearson دقیق‌ترین روش و بر پایه توزیع Beta است. اگر k آشکارسازی از n پالس مشاهده شود، کران پایین و بالا برای احتمال واقعی p به این شکل به دست می‌آیند:

$$p_{\text{lower}} = B\left(\frac{\alpha}{2}; k, n - k + 1\right), \quad (۴.۶)$$

$$p_{\text{upper}} = B\left(1 - \frac{\alpha}{2}; k + 1, n - k\right), \quad (۵.۶)$$

که در آن $B(q; a, b)$ تابع quantile توزیع Beta و α احتمال شکست است. این کران‌ها محافظه‌کارانه‌ترین کران ممکن هستند و در عین حال نیازمند محاسبه `beta.ppf` از SciPy با نسخه حداقل ۸.۱ می‌باشند؛ به همین دلیل فایل در ابتدا بررسی نسخه انجام می‌دهد. Hoeffding تقریبی‌تر اما ساده‌تر است:

$$p \in \left[\hat{p} - \sqrt{\frac{\ln(1/\alpha)}{2n}}, \hat{p} + \sqrt{\frac{\ln(1/\alpha)}{2n}} \right], \quad (۶.۶)$$

که در آن $\hat{p} = k/n$ در نهایت Gaussian از تقریب نرمال استفاده می‌کند:

$$p \in \left[\hat{p} - z_{1-\alpha/2} \sqrt{\frac{\hat{p}(1-\hat{p})}{n}}, \hat{p} + z_{1-\alpha/2} \sqrt{\frac{\hat{p}(1-\hat{p})}{n}} \right]. \quad (۷.۶)$$

این سه روش در متد `get_bounds` با توجه به `ci_method` در `QKDParams` انتخاب می‌شوند.

۳.۲.۶ آنتروپی باینری و تقویت حریم خصوصی

آنتروپی باینری Shannon یک عنصر محوری در محاسبه طول کلید امن است:

$$h(p) = -p \log_2(p) - (1-p) \log_2(1-p). \quad (۸.۶)$$

در اثبات‌های مبتنی بر GLLP و Ma 2005، طول کلید امن به صورت زیر به دست می‌آید:

$$\ell = s_{z,1}^L \left[1 - h\left(e_{\text{ph}}^U\right) \right] - \lambda_{\text{EC}} - 6 \log_2\left(\frac{19}{\varepsilon_{\text{sec}}}\right) - \log_2\left(\frac{2}{\varepsilon_{\text{cor}}}\right), \quad (9.6)$$

که در آن $s_{z,1}^L$ کران پایین تعداد آشکارسازی‌های تک‌فوتونی در پایه Z ، e_{ph}^U کران بالای خطای فاز، λ_{EC} میزان نشت اطلاعات ناشی از تصحیح خطا و ε_{cor} پارامتر صحت است. محاسبه $h(p)$ در نزدیکی کران‌های $p=0$ و $p=1$ ، به دلیل حضور عبارت‌های $p \log p$ و $(1-p) \log(1-p)$ ، از نظر عددی ناپایدار می‌شود؛ هرچند طبق قرارداد ریاضی، $0 \log 0 = 0$ در نظر گرفته می‌شود. از این رو، متد `binary_entropy` با استفاده از `np.clip` مقدار p را به بازه $[\delta, 1-\delta]$ محدود می‌کند، که در آن

$$\delta = \text{ENTROPY_PROB_CLAMP}.$$

این محدودسازی، همراه با استفاده از `np.nan_to_num`، تضمین می‌کند که خروجی همواره مقداری متناهی و قابل استفاده باشد.

۴.۲.۶ برنامه‌ریزی خطی در تخمین Decoy

تخمین Y_1^L معمولاً به حل یک مسأله برنامه‌ریزی خطی (LP) نیاز دارد. در چارچوب Ma 2005، این مسأله با حداقل سازی Y_1 تحت قیود فیزیکی که از معادلات تعادل بازده به دست می‌آیند، صورت می‌گیرد. اگر solver نتواند جواب بهینه پیدا کند یا باقی‌مانده LP بزرگ باشد، `SolverDiagnostics` این وضعیت را ثبت می‌کند و متد `_conservative_decoy_estimate` یک تخمین محافظه‌کارانه با $Y_1^L = 0$ و $e_1^U = 0.5$ برمی‌گرداند. این مقدار به معنای آن است که در بدترین حالت، هیچ تک‌فوتونی امن قابل استخراج نیست و طول کلید صفر می‌شود.

۳.۶ تحلیل معماری و ساختار داده‌ها

۱.۳.۶ واردسازی وابستگی‌ها

فایل با یک مجموعه `try/except` ساختارمند برای واردسازی وابستگی‌های داخلی و خارجی آغاز می‌شود. این الگو دو هدف را دنبال می‌کند: نخست، در صورت نبود یک وابستگی، پیام خطای روشنی با `ConfigurationError` تولید شود و دوم، بررسی نسخه SciPy حداقل ۸.۱ تضمین گردد. این بررسی به دلیل رفتار متفاوت `beta.ppf` در نسخه‌های قدیمی ضروری است. کد مربوطه به این شکل است:

```

1  try:
2  import scipy
3  from scipy import special
4  from scipy.stats import beta
5  if tuple(map(int, scipy.__version__.split('.')[:2])) < (1, 8):
6  raise ConfigurationError(
7  "SciPy >= 8.1 is required for reliable beta.ppf behavior.",
8  code=QKDErrorCode.CONFIG,
9  context={"installed_version": scipy.__version__,
10         "required_version": "8.1=<"}
11  )
12  except ImportError as exc:
13  raise ConfigurationError(
14  "SciPy is required for Clopper-Pearson confidence intervals.",
15  code=QKDErrorCode.CONFIG,
16  cause=exc
17  ) from exc
18
```

در خط اول، `scipy` به عنوان ماژول اصلی وارد می‌شود. در خط دوم، زیرماژول `special` برای توابع خاص ریاضی مانند `betainc` و در خط سوم توزیع `beta` برای محاسبه `quantile` در کران‌های `Clopper-Pearson` وارد می‌شوند. خط چهارم نسخه را تجزیه و مقایسه می‌کند و در صورت عدم تطابق، استثنای سفارشی با `QKDErrorCode.CONFIG` پرتاب می‌کند. این کد طراحی شده تا در محیط‌های عملیاتی که نسخه‌های قدیمی SciPy ممکن است نصب باشند، پیش از وقوع خطای عددی غیرقابل ردیابی، متوقف شود. علاوه بر این، یک واردسازی امن برای `mpmath` وجود دارد که در صورت موجود بودن، امکان محاسبات با دقت دلخواه را فراهم می‌کند:

```

۱ try:
۲     import mpmath
۳ except ImportError:
۴     mpmath = None
۵

```

این الگو از نوع optional dependency است؛ یعنی اگر mpmath نصب نباشد، برنامه متوقف نمی‌شود بلکه متغیر mpmath روی None تنظیم می‌شود و کد می‌تواند با استفاده از شرط use_mpmath تصمیم بگیرد که از مسیر دقت بالا استفاده کند یا خیر.

۲.۳.۶ ثابت‌های مازول و TypeAlias

دو ثابت نمادین در سطح مازول برای کلیدهای آماری تعریف می‌شوند:

```

۱ TALLY_KEY_Z_SIGNAL = "Z_signal"
۲ TALLY_KEY_X_SIGNAL = "X_signal"
۳

```

این دو ثابت نشان‌دهنده پایه‌های اندازه‌گیری در پروتکل BB84 هستند. پایه Z پایه محاسباتی استاندارد و پایه X پایه Hadamard است. در Decoy-State، آلیس و باب پس از تبادل، آمار را بر اساس پایه تفکیک می‌کنند تا بتوانند خطای فاز e_{ph} را به صورت جداگانه از خطای بیت e_{bit} تخمین بزنند. استفاده از ثابت به جای رشته خام، خطاهای تایپی را حذف می‌کند و در بازبینی کد به وضوح مشخص می‌کند که چه کلیدی مورد نظر است. همچنین $T = \text{TypeVar}("T")$ برای استفاده در توابع عمومی تعریف شده و `__version__ = "4.2.0"` نسخه فعلی مازول را مشخص می‌کند. این نسخه در گزارش‌های audit درج می‌شود تا در صورت بروز مشکل، نسخه دقیق کد قابل ردیابی باشد.

۳.۳.۶ شمارش‌ها و Enumهای خطا

ProofMode یک Enum سه‌حالتی است که رفتار سیستم در مواجهه با خطاهای عددی و هشدارها را تعیین می‌کند:

```

۱ class ProofMode(Enum):
۲     PRODUCTION = auto()
۳     DEBUG = auto()
۴     AUDIT = auto()
۵

```

حالت PRODUCTION حالت پیش‌فرض است که در آن خطاهای غیربحرانی با هشدار log می‌شوند و اجرا ادامه می‌یابد. حالت DEBUG برای محیط توسعه است و اطلاعات بیشتری در log ثبت می‌کند. حالت AUDIT سخت‌گیرانه‌ترین حالت است که در آن هر گونه انحراف از شرط‌های فیزیکی یا عددی به استثنای کامل منجر می‌شود. این حالت برای کاربردهای حساس مانند certification یا بازبینی توسط شخص ثالث طراحی شده است. متدهای `is_debug_mode` و `is_audit_mode` در کلاس پایه به این Enum متکی‌اند تا تصمیم‌گیری در مورد رفتار خطا را ساده کنند. `ErrorCode` شش کد خطای متمایز را تعریف می‌کند که در گزارش نتیجه محاسبه کلید استفاده می‌شوند تا علت دقیق تولید نشدن کلید قابل ردیابی باشد:

```

۱ class ErrorCode(Enum):
۲     INSUFFICIENT_STATISTICS = auto()
۳     LP_INFEASIBLE = auto()
۴     NUMERICAL_CLAMP_APPLIED = auto()
۵     LEAKAGE_EXCEEDS_ENTROPY = auto()
۶     HIGH_SOLVER_RESIDUAL = auto()
۷     SAFE_DIVIDE_FALLBACK = auto()
۸

```

هر یک از این کدها به یک شرایط مشخص اشاره دارد. `INSUFFICIENT_STATISTICS` زمانی فعال می‌شود که Y_1^L از آستانه `min_y1_threshold` کمتر باشد. `LP_INFEASIBLE` نشان‌دهنده ناپذیرفتگی مسأله برنامه‌ریزی خطی است. `NUMERICAL_CLAMP_APPLIED` هشدار می‌دهد که یک مقدار منفی به صفر clamp شده است. `LEAKAGE_EXCEEDS_ENTROPY` بحرانی‌ترین

کد است و نشان می‌دهد که نشت تصحیح خطا و تقویت حریم خصوصی از آنتروپی قابل استخراج بیشتر شده است. HIGH_SOLVER_RESIDUAL باقی‌مانده بالای solver را علامت می‌زند و SAFE_DIVIDE_FALLBACK نشان می‌دهد که تقسیم ایمن به مقدار پیش‌فرض بازگشته است.

۴.۳.۶ Dataclass های داده

TallyCounts

این dataclass ساده اما حیاتی است و دو فیلد دارد: detections و errors. هر دو از نوع int و با مقدار پیش‌فرض صفر تعریف شده‌اند. با استفاده از @dataclass(frozen=True) غیرقابل تغییر شده و در متد __post_init__ دو شرط اعتبارسنجی دارد:

```

1  @dataclass(frozen=True)
2  class TallyCounts:
3      detections: int = 0
4      errors: int = 0
5
6      def __post_init__(self):
7          if self.detections < 0 or self.errors < 0:
8              raise QKDParamError(
9                  "Tally counts cannot be negative.",
10                 code=QKDErrorCode.PARAM_VALIDATION,
11                 context={"detections": self.detections, "errors": self.errors}
12             )
13             if self.errors > self.detections:
14                 raise QKDParamError(
15                     f"Error count ({self.errors}) cannot exceed detection count ({self.detections}).",
16                     code=QKDErrorCode.PARAM_VALIDATION,
17                     context={"detections": self.detections, "errors": self.errors}
18                 )
19

```

شرط نخست عدم منفی بودن شمارش‌ها را تضمین می‌کند که یک ضرورت فیزیکی است. شرط دوم منطقی‌تر است: تعداد خطاها نمی‌تواند از تعداد آشکارسازی‌ها بیشتر باشد، زیرا هر خطا نیازمند یک آشکارسازی است. استفاده از frozen=True اطمینان می‌دهد که پس از ساخت، شمارش‌ها قابل تغییر نیستند و در نتیجه در سراسر محاسبات ثابت می‌مانند. این ویژگی برای صحت‌سنجی نتایج و جلوگیری از تغییرات ناخواسته در طول محاسبات بسیار مهم است.

EpsilonAllocation

این ساختار تقسیم بودجه ϵ را در چهار مؤلفه نگه می‌دارد:

```

1  @dataclass
2  class EpsilonAllocation:
3      eps_sec: float
4      eps_pe: float
5      eps_smooth: float
6      eps_pa: float
7
8      def validate(self, policy: Callable[["EpsilonAllocation"], float]) -> None:
9          consumed = policy(self)
10         if consumed > self.eps_sec:
11             raise QKDParamError(
12                 f"Epsilon allocation exceeds budget: {consumed2.:e} > {self.eps_sec2.:e}",
13                 code=QKDErrorCode.PARAM_VALIDATION,
14                 context={"consumed": consumed, "budget": self.eps_sec,
15                         "policy": policy.__name__}
16             )
17

```

برخلاف TallyCounts، این کلاس frozen نیست زیرا ممکن است زیرکلاس‌ها در طول زمان تخصیص را تنظیم کنند. متد validate یک policy function می‌گیرد که جمع موزون ϵ ‌ها را محاسبه می‌کند و اگر از کل بودجه ϵ_{sec} تجاوز کند، استثنا پرتاب می‌کند. این الگو به زیرکلاس‌ها اجازه می‌دهد سیاست‌های متفاوتی برای

تخصیص ε پیاده‌سازی کنند (مثلاً تخصیص مساوی، تخصیص بر اساس اهمیت نسبی یا تخصیص بهینه) بدون آنکه کلاس پایه وابستگی به سیاست خاصی داشته باشد.

SolverDiagnostics

این dataclass برای ثبت اطلاعات مربوط به حل مسأله LP در تخمین Decoy به کار می‌رود:

```

۱ @dataclass(frozen=True)
۲ class SolverDiagnostics:
۳     solver_name: str
۴     is_success: bool
۵     status_message: str
۶     residual_norm: Optional[float] = None
۷     timings: Dict[str, float] = field(default_factory=dict)
۸     numeric_diagnostics: Dict[str, Any] = field(default_factory=dict)
۹ 
```

فیلد solver_name نام solver (مانند cvxopt یا scipy.optimize.linprog) را ذخیره می‌کند. is_success نشان می‌دهد که آیا solver به راه‌حل بهینه دست یافته است یا خیر. status_message پیام متنی solver را نگه می‌دارد. residual_norm نرم باقی‌مانده مسأله را نشان می‌دهد و در تشخیص کیفیت راه‌حل کاربرد دارد. دو فیلد timings و numeric_diagnostics از نوع Dict با default_factory=dict هستند تا در صورت عدم ارائه مقدار، دیکشنری خالی ایجاد شود. استفاده از default_factory به جای مقدار پیش‌فرض مستقیم، از مشکل معروف mutable default argument در پایتون جلوگیری می‌کند.

DecoyEstimates

این ساختار خروجی اصلی مرحله تخمین Decoy است:

```

۱ @dataclass(frozen=True)
۲ class DecoyEstimates:
۳     yield_1_lower_bound: float
۴     error_rate_1_upper_bound: float
۵     is_feasible: bool
۶     failure_prob_used: float
۷     diagnostics: SolverDiagnostics
۸
۹     def __repr__(self) -> str:
۱۰         return (f"DecoyEstimates(Y1_L={self.yield_1_lower_bound2.:e}, "
۱۱             f"e1_U={self.error_rate_1_upper_bound3.:f}, feasible={self.is_feasible})")
۱۲
۱۳     def as_serializable(self) -> Dict[str, Any]:
۱۴         data = asdict(self, dict_factory=_json_safe_dict)
۱۵         data['metadata'] = {
۱۶             'schema_version': '2.1', 'lib_version': __version__,
۱۷             'numpy_version': np.__version__, 'scipy_version': scipy.__version__,
۱۸         }
۱۹         return data
۲۰ 
```

دو فیلد اصلی yield_1_lower_bound (یعنی Y_1^L) و error_rate_1_upper_bound (یعنی e_1^U) هستند که مستقیماً در فرمول طول کلید وارد می‌شوند. is_feasible نشان می‌دهد که آیا LP یک راه‌حل قابل قبول پیدا کرده است یا خیر. failure_prob_used احتمال شکستی است که در محاسبه کران‌ها استفاده شده (معمولاً ε_E). متد as_serializable با استفاده از _json_safe_dict نوع‌های numpy و Enum را به نوع‌های بومی پایتون تبدیل می‌کند و متادیتای نسخه را اضافه می‌کند تا خروجی به راحتی به JSON سریال‌سازی شود.

KeyCalculationResult

این dataclass مهم‌ترین خروجی کل اثبات امنیتی است:

```

۱ @dataclass(frozen=True)
۲ class KeyCalculationResult:
۳     secure_key_length: int

```

```

۴ privacy_amplification_term: float
۵ error_correction_leakage: float
۶ phase_error_rate_upper_bound: float
۷ diagnostics: Any = field(default_factory=dict, repr=False)
۸ error_codes: list[ErrorCode] = field(default_factory=list)
۹ weak_gllp_rate: Optional[float] = None
۱۰ tagged_fraction_bound: Optional[float] = None
۱۱

```

چهار فیلد اصلی secure_key_length (ℓ)، $\text{privacy_amplification_term}$ (جریمه تقویت حریم خصوصی)، $\text{error_correction_leakage}$ (λ_{EC}) و $\text{phase_error_rate_upper_bound}$ (e_{ph}^U) هستند. diagnostics از نوع Any تعریف شده تا هم لیست رشته و هم دیکشنری ساختاریافته را بپذیرد. دو فیلد اختیاری weak_gllp_rate و $\text{tagged_fraction_bound}$ برای مقایسه با چارچوب GLLP سنتی طراحی شده‌اند. $\text{tagged_fraction_bound}$ کسر فوتون‌های برچسب‌خورده (تک‌فوتون‌هایی که اطلاعات فاز آن‌ها نشت کرده) را نشان می‌دهد و در فرمول GLLP نرخ کلید را به صورت زیر محدود می‌کند:

$$R_{\text{GLLP}} \geq q [Q_1 (1 - h(e_1^U)) - Q_\mu f h(E_\mu)] , \quad (10.6)$$

که در آن Q_1 بازده تک‌فوتونی، Q_μ بازده سیگنال، E_μ خطای بیت سیگنال و f کارایی تصحیح خطا است.

۵.۳.۶ کلاس پایه انتزاعی FiniteKeyProof

این کلاس با ABC به عنوان کلاس پایه انتزاعی تعریف شده و سه ویژگی کلاسی دارد:

```

۱ class FiniteKeyProof(ABC):
۲     __implementation_version__: str = "0.0.0"
۳     allowed_ci_methods: frozenset[ConfidenceBoundMethod] = frozenset(ConfidenceBoundMethod)
۴     min_y1_threshold: float = 1e-9
۵

```

`__implementation_version__` به زیرکلاس‌ها اجازه می‌دهد نسخه پیاده‌سازی خود را ثبت کنند. `allowed_ci_methods` یک `frozenset` از روش‌های مجاز کران آماری است که به صورت پیش‌فرض همه روش‌های `ConfidenceBoundMethod` را شامل می‌شود؛ زیرکلاس‌ها می‌توانند آن را محدودتر کنند. `min_y1_threshold` آستانه 10^{-9} برای Y_1^L است که اگر کمتر از آن باشد، تخمین به عنوان غیر قابل اتکا در نظر گرفته می‌شود. این آستانه بر اساس تجربه عملی تنظیم شده تا از تولید کلید نامعتبر در شرایط آمار ضعیف جلوگیری کند.

متد سازنده `__init__` مجموعه‌ای از تنظیمات اولیه را انجام می‌دهد:

```

۱ def __init__(self, params: QKDParams, mode: ProofMode = ProofMode.PRODUCTION):
۲     self.p = params
۳     self.mode = mode
۴     self.logger = logging.getLogger(self.__class__.__name__)
۵     self.run_id = getattr(params, 'run_id', 'no-run-id')
۶     self.min_prob_clamp = getattr(params, 'entropy_clamp', ENTROPY_PROB_CLAMP)
۷     self.use_mpmath = getattr(params, 'use_mpmath', False) and mpmath is not None
۸
۹     self.logger.info("event=init run_id=%s proof=%s version=%s numpy=%s scipy=%s",
۱۰ self.run_id, self.__class__.__name__, self.__implementation_version__,
۱۱ np.__version__, scipy.__version__)
۱۲
۱۳ self.validate_physical_params()
۱۴ self._check_notation_map()
۱۵
۱۶ if self.p.ci_method not in self.allowed_ci_methods:
۱۷     msg = f"CI method '{self.p.ci_method.name}' is not recommended for this proof."
۱۸     if self.is_audit_mode():
۱۹         raise QKDParamError(
۲۰ msg,
۲۱ code=QKDErrorCode.PARAM_VALIDATION,
۲۲ context={"ci_method": self.p.ci_method.name,
۲۳ "allowed": [m.name for m in self.allowed_ci_methods]}
۲۴ )
۲۵     self.logger.warning("[%s] %s", self.run_id, msg)
۲۶
۲۷ self.eps_alloc = self.allocate_epsilon()

```

```

۲۸ self.eps_alloc.validate()
۲۹

```

در خطوط نخست، پارامترهای QKD و حالت اجرا ذخیره می‌شوند. سپس یک logger اختصاصی با نام کلاس ایجاد می‌شود تا لاگ‌ها به راحتی فیلتر شوند. run_id شناسه یکتای اجرا است که در صورت عدم تنظیم، مقدار 'no-run-id' می‌گیرد. min_prob_clamp حد احتمالات در محاسبه آنتروپی است که از پارامترها خوانده می‌شود و در صورت نبود، از ثابت ENTROPY_PROB_CLAMP استفاده می‌کند. use_mpmath پرچمی است که در صورت تنظیم در پارامترها و موجود بودن mpmath، فعال می‌شود. سپس متدهای validate_physical_params و check_notation_map فراخوانی می‌شوند تا صحت پارامترها و پیاده‌سازی زیرکلاس بررسی شود. در ادامه، اگر روش CI مورد استفاده در فهرست مجاز نباشد، در حالت AUDIT استثنا پرتاب می‌شود و در غیر این صورت هشدار داده می‌شود. نهایتاً، allocate_epsilon فراخوانی شده و اعتبارسنجی می‌شود.

۴.۶ پیاده‌سازی ریاضی و الگوریتمی

۱.۴.۶ تبدیل نوع‌ها برای JSON

تابع کمکی _json_safe_dict یک dict_factory برای asdict است که نوع‌های numpy و Enum را به نوع‌های بومی پایتون تبدیل می‌کند:

```

۱ def _json_safe_dict(data: Iterable[Tuple[str, Any]]) -> Dict[str, Any]:
۲     """Recursively converts numpy/enum types to native Python types for JSON."""
۳     d: Dict[str, Any] = {}
۴     for key, value in data:
۵         if isinstance(value, (np.integer, np.int64)):
۶             d[key] = int(value)
۷         elif isinstance(value, (np.floating, np.float64)):
۸             d[key] = float(value)
۹         elif isinstance(value, np.ndarray):
۱0            if value.size > 1000:
۱1                d[key] = {'data': value[:1000].tolist(), 'truncated': True}
۱2            else:
۱3                d[key] = value.tolist()
۱4            elif isinstance(value, np.bool_):
۱5                d[key] = bool(value)
۱6            elif isinstance(value, Enum):
۱7                d[key] = value.name
۱8            elif isinstance(value, dict):
۱9                d[key] = _json_safe_dict(value.items())
۲0            else:
۲1                d[key] = value
۲2     return d
۲۳

```

این تابع برای هر جفت کلید-مقدار، نوع مقدار را بررسی می‌کند. اگر np.integer یا np.int64 باشد به int تبدیل می‌شود. اگر np.floating یا np.float64 باشد به float تبدیل می‌شود. برای آرایه‌های numpy، اگر اندازه از ۱۰۰۰ تجاوز کند، تنها ۱۰۰۰ عنصر اول به همراه پرچم truncated ذخیره می‌شوند تا حجم گزارش audit کنترل‌پذیر بماند. این تصمیم طراحی بر اساس تجربه عملی اتخاذ شده، زیرا در شبیه‌سازی‌های بزرگ آرایه‌های ده‌ها هزار عنغانی می‌توانند گزارش را غیرقابل مدیریت کنند. np.bool_ به bool و Enum به نام آن تبدیل می‌شود. اگر مقدار خود دیکشنری باشد، فراخوانی بازگشتی انجام می‌شود. این تابع در متدهای KeyCalculationResult و DecoyEstimates as_serializable استفاده می‌شود.

۲.۴.۶ کران‌های آماری در عمل

متد get_bounds مهم‌ترین تابع آماری کلاس پایه است. این متد کران پایین و بالا را برای یک احتمال ناشناخته p بر اساس k موفقیت در n آزمایش با احتمال شکست failure_prob محاسبه می‌کند:

```

۱ def get_bounds(self, k: int, n: int, failure_prob: float,
۲     sided: Literal["two", "one_upper", "one_lower"] = "two",
۳     diagnostics: Dict | None = None) -> Tuple[float, float]:
۴     k, n = int(k), int(n)

```

```

۵     if not (0 <= k <= n):
۶         raise QKDParamError(
۷             f"k must be in [0, n], got k={k}, n={n}.",
۸             code=QKDErrorCode.PARAM_VALIDATION,
۹             context={"k": k, "n": n}
۱0        )
۱۱     if not (1e-300 < failure_prob < 1):
۱۲         raise QKDParamError(
۱۳             f"Failure probability must be in (1e-300, 1), got {failure_prob}.",
۱۴             param_name="failure_prob",
۱۵             param_value=failure_prob,
۱۶             code=QKDErrorCode.PARAM_VALIDATION
۱۷        )
۱۸     if n == 0: return 0.0), (0.1
۱۹     alpha = float(failure_prob if sided != "two" else failure_prob / (0.2
۲۰
۲۱     if diagnostics is not None:
۲۲         diagnostics['alpha_used'] = alpha
۲۳
۲۴     sided_arg = "two-sided" if sided == "two" else ("upper" if sided == "one_upper" else "lower")
۲۵
۲۶     if self.p.ci_method == ConfidenceBoundMethod.CLOPPER_PEARSON:
۲۷         interval = qkd_math.clopper_pearson_bounds(k, n, failure_prob, side=sided_arg)
۲۸         lower, upper = interval.lower, interval.upper
۲۹     elif self.p.ci_method == ConfidenceBoundMethod.HOEFFDING:
۳۰         interval = qkd_math.hoeffding_bounds(k, n, failure_prob, side=sided_arg)
۳۱         lower, upper = interval.lower, interval.upper
۳۲     elif self.p.ci_method == ConfidenceBoundMethod.GAUSSIAN:
۳۳         interval = qkd_math.gaussian_bounds(k, n, failure_prob, side=sided_arg)
۳۴         lower, upper = interval.lower, interval.upper
۳۵     else:
۳۶         raise QKDSimulationError(
۳۷             f"CI method '{self.p.ci_method.name}' not implemented.",
۳۸             code=QKDErrorCode.CONFIG,
۳۹             context={"ci_method": self.p.ci_method.name}
۴۰        )
۴۱
۴۲     self._assert_finite("lower bound", lower)
۴۳     self._assert_finite("upper bound", upper)
۴۴     return (lower, upper)
۴۵

```

ابتدا k و n به `int` تبدیل می‌شوند تا از خطاهای نوع `numpy` جلوگیری شود. شرط $0 \leq k \leq n$ یک ضرورت منطقی است: تعداد موفقیت‌ها نمی‌تواند از تعداد آزمایش‌ها بیشتر باشد یا منفی باشد. شرط $10^{-300} < \text{failure_prob} < 1$ محدوده معتبر احتمال شکست را تضمین می‌کند. کران پایین 10^{-300} به جای صفر تنظیم شده تا از خطاهای عددی در محاسبات `log` جلوگیری شود. اگر $n = 0$ باشد، یعنی هیچ آزمایشی انجام نشده، کران پیش‌فرض $(0, 1)$ برگردانده می‌شود که نشان‌دهنده عدم اطلاع است. پارامتر `sided` سه حالت دارد: `two` برای کران دو طرفه، `one_upper` برای کران بالا فقط و `one_lower` برای کران پایین فقط. در حالت دو طرفه، α به دو نیم تقسیم می‌شود تا کران از هر دو طرف پوشش $1 - \alpha$ داشته باشد. این تنظیم به صورت زیر فرمول‌بندی می‌شود:

$$\alpha_{\text{eff}} = \begin{cases} \alpha/2 & \text{if sided = two,} \\ \alpha & \text{otherwise.} \end{cases} \quad (11.6)$$

سپس بر اساس `ci_method` در پارامترها، تابع مربوطه از ماژول `qkd_math` فراخوانی می‌شود. هر سه تابع یک شیء `interval` با ویژگی‌های `lower` و `upper` برمی‌گردانند. در پایان، متد `_assert_finite` مقادیر را بررسی می‌کند تا اطمینان حاصل شود که متناهی هستند. در حالت `AUDIT`، مقدار نامتناهی به استثنا منجر می‌شود.

۳.۴.۶ محاسبه آنتروپی باینری

متد `binary_entropy` آنتروپی باینری را به صورت برداری محاسبه می‌کند:

```

۱     def binary_entropy(self, p_err: float | np.ndarray | list) -> float | np.ndarray:
۲         p = np.asarray(p_err, dtype=np.float64)
۳         p = np.clip(p, self.min_prob_clamp, 0.1 - self.min_prob_clamp)
۴         with np.errstate(divide='ignore', invalid='ignore'),

```



```

۵     over='raise' if self.is_audit_mode() else 'warn'):
۶     h = -p * np.log2(p) - (1 - p) * np.log2(1 - p)
۷     h = np.nan_to_num(h, nan(0.0=
۸     return h.item() if isinstance(p_err, (float, int)) else h
۹

```

ورودی می‌تواند یک عدد، یک لیست یا یک آرایه numpy باشد. در خط دوم، ورودی به آرایه float64 تبدیل می‌شود. در خط سوم، مقدار p به بازه $[\delta, 1 - \delta]$ محدود می‌شود که در آن $\delta = \text{min_prob_clamp}$. این clip از محاسبه $\log(0)$ که به $-\infty$ منجر می‌شود، جلوگیری می‌کند. محاسبه آنتروپی به صورت زیر انجام می‌شود:

$$h(p) = -p \log_2(p) - (1 - p) \log_2(1 - p), \quad (۱۲.۶)$$

که با توجه به clip، همواره مقدار متناهی خواهد داشت. با این حال، به دلیل احتمال سرریز در محاسبات log برای مقادیر بسیار کوچک، از np.errstate برای کنترل رفتار خطا استفاده می‌شود. در حالت AUDIT، سرریز به استثنا تبدیل می‌شود و در غیر این صورت هشدار داده می‌شود. در نهایت، np.nan_to_num هر مقدار NaN باقی‌مانده را به صفر تبدیل می‌کند. اگر ورودی اصلی یک عدد بوده، خروجی هم به صورت اسکالر با item() برگردانده می‌شود.

۴.۴.۶ متدهای ایمن عددی

کلاس پایه چهار متد کمکی برای مدیریت شرایط عددی بحرانی دارد. متد _safe_log برای محاسبه لگاریتم طبیعی با حداقل مقدار ایمن استفاده می‌شود:

```

۱     def _safe_log(self, x: float) -> float:
۲     if x < 1e-300:
۳         if self.is_audit_mode():
۴             raise QKDSimulationError(
۵                 "Log argument too small in AUDIT mode.",
۶                 code=QKDErrorCode.SIMULATION,
۷                 context={"x": x}
۸             )
۹         self.logger.warning("Log argument %e is smaller than safe limit, clamping.", x)
۱۰        x = 1e-300
۱۱        return math.log(x)
۱۲

```

اگر x کمتر از 10^{-300} باشد، در حالت AUDIT استثنا پرتاب می‌شود و در غیر این صورت x به 10^{-300} clamp می‌شود تا از خطای math domain error جلوگیری شود. این متد در محاسبات log-likelihood و relative entropy که در برخی اثبات‌های امنیتی به کار می‌روند، حیاتی است. متد _safe_divide برای تقسیم ایمن با مخرج نزدیک به صفر است:

```

۱     def _safe_divide(self, num: float, den: float, default: float = 0.0,
۲                     name: str = '', error_codes: list | None = None) -> float:
۳     if abs(den) < NUMERIC_ABS_TOL:
۴         if error_codes is not None:
۵             error_codes.append(ErrorCode.SAFE_DIVIDE_FALLBACK)
۶         self.logger.warning("Denominator for '%s' is near zero (%e). Returning default value %f.",
۷                               name, den, default)
۸         return default
۹     return num / den
۱۰

```

اگر قدر مطلق مخرج کمتر از NUMERIC_ABS_TOL باشد، مقدار default (پیش‌فرض صفر) برگردانده می‌شود و در صورت ارائه لیست error_codes، کد SAFE_DIVIDE_FALLBACK به آن اضافه می‌شود. این متد در محاسباتی مانند نسبت خطا به آشکارسازی که ممکن است مخرج صفر داشته باشند، ضروری است. متد _clamp_nonneg مقادیر منفی کوچک را به صفر تبدیل می‌کند:

```

۱     def _clamp_nonneg(self, x: float, name: str, error_codes: list | None = None) -> float:
۲     if x < 0:
۳         if error_codes is not None:
۴             error_codes.append(ErrorCode.NUMERICAL_CLAMP_APPLIED)
۵         if x < -NUMERIC_ABS_TOL and self.is_audit_mode():
۶             raise QKDSimulationError(

```



```

۷     f"Negative value for {name} ({x}) exceeded tolerance in AUDIT mode.",
۸     code=QKDErrorCode.SIMULATION,
۹     context={"name": name, "value": x, "tolerance": -NUMERIC_ABS_TOL}
۱۰    )
۱۱    self.logger.debug("Clamped negative value for %s: %e -> 0.0", name, x)
۱۲    return 0.0
۱۳    return x
۱۴

```

مقادیر منفی کوچک (در حد تلورانس) به صفر clamp می‌شوند زیرا در محاسبات عددی با نوسانات ممیز شناور، ممکن است مقادیری که باید صفر باشند، منفی کوچک گزارش شوند. اما مقادیر منفی بزرگ در حالت AUDIT به استثنا منجر می‌شوند، زیرا نشان‌دهنده خطای منطقی جدی است. متد `_clamp_key_length` طول کلید نهایی را به عدد صحیح نامنفی تبدیل می‌کند:

```

۱     def _clamp_key_length(self, l_float: float) -> int:
۲         return int(max(0.0), math.floor(l_float)))
۳

```

ابتدا با `max(0.0, ...)` اطمینان حاصل می‌شود که مقدار منفی نیست، سپس با `math.floor` به عدد صحیح پایین گرد می‌شود و نهایتاً به `int` تبدیل می‌شود. این رفتار محافظه‌کارانه تضمین می‌کند که طول کلید هرگز بیش از مقدار مجاز ریاضی نباشد.

۵.۴.۶ مدیریت زمان‌سنجی و گزارش‌گیری Audit

متد `_timed` یک context manager برای ثبت زمان اجرای قطعات کد است:

```

۱     @contextmanager
۲     def _timed(self, timings: Dict, key: str) -> Generator:
۳         t0 = time.perf_counter()
۴         yield
۵         timings[key] = time.perf_counter() - t0
۶

```

با استفاده از `time.perf_counter` که بالاترین دقت زمان‌سنجی را ارائه می‌دهد، زمان شروع و پایان ثبت شده و اختلاف در دیکشنری `timings` با کلید مربوطه ذخیره می‌شود. این اطلاعات در `SolverDiagnostics` برای شناسایی گلوگاه‌های عملکرد به کار می‌رود. متد `audit_dump` یک گزارش کامل از اجرا تولید می‌کند:

```

۱     def audit_dump(self, result: KeyCalculationResult, decoy: DecoyEstimates,
۲     stats: Dict, start_time: float) -> Dict:
۳         dump = {
۴             "params": self.p.to_summary_dict(redact=True),
۵             "mode": self.mode.name,
۶             "proof_implementation": f"{self.__class__.__name__} v{self.__implementation_version__}",
۷             "stats_map": {k: asdict(v) for k, v in stats.items()},
۸             "decoy_estimates": decoy.as_serializable(),
۹             "key_calculation_result": result.as_serializable(),
۱۰            "timestamps": {'start': start_time, 'end': time.time()},
۱۱        }
۱۲        if self.is_audit_mode():
۱۳            y1_margin = decoy.yield_1_lower_bound - self.min_y1_threshold
۱۴            e1_margin = 5.0 - decoy.error_rate_1_upper_bound
۱۵            dump['sensitivity_summary'] = {'y1_margin': y1_margin, 'e1_margin': e1_margin}
۱۶        return dump
۱۷

```

گزارش شامل پارامترها (با `redact=True` برای حذف اطلاعات حساس مانند کلیدها)، حالت اجرا، نسخه پیاده‌سازی، آمار کامل، تخمین‌های Decoy، نتیجه محاسبه کلید و زمان‌بندی است. در حالت AUDIT، یک خلاصه حساسیت اضافه می‌شود که حاشیه Y_1^L نسبت به آستانه و حاشیه e_1^U نسبت به حد ۵.۰ (حد بی‌اطلاعی) را نشان می‌دهد. این حاشیه‌ها معیار مهمی برای سنجش کیفیت اجرا هستند؛ هرچه حاشیه بیشتر، اجرا امن‌تر است.

۶.۴.۶ توضیح تصمیم کلید

متد `explain_key_decision` یک پیام متنی قابل فهم برای انسان تولید می‌کند که علت تولید نشدن کلید را توضیح می‌دهد:

```

1 def explain_key_decision(self, result: KeyCalculationResult, decoy: DecoyEstimates) -> str:
2     if result.secure_key_length > 0:
3         return f"Secure key generated. Final length: {result.secure_key_length} bits."
4     reasons = []
5     if ErrorCode.INSUFFICIENT_STATISTICS in result.error_codes:
6         reasons.append(f"insufficient single-photon yield (Y1_L={decoy.yield_1_lower_bound2.:e} "
7             f"< threshold={self.min_y1_threshold2.:e})")
8     if ErrorCode.LP_INFEASIBLE in result.error_codes:
9         reasons.append("decoy-state analysis was infeasible")
10    if ErrorCode.LEAKAGE_EXCEEDS_ENTROPY in result.error_codes:
11        reasons.append(f"information leakage ({result.error_correction_leakage + result.
12            privacy_amplification_term2.:f}) "
13            "exceeded available entropy")
14    if not reasons:
15        reasons.append("an unspecified condition led to a non-positive key length")
16    return f"No secure key generated because {'', and '}.join(reasons)}."

```

اگر طول کلید مثبت باشد، پیام موفقیت برگردانده می‌شود. در غیر این صورت، بر اساس `error_codes` در نتیجه، دلایل مختلف جمع‌آوری می‌شوند. اگر هیچ کد خطای مشخصی وجود نداشته باشد، پیام عمومی نمایش داده می‌شود. این متد برای `logging` و گزارش به کاربر نهایی بسیار مفید است و تشخیص علت مشکلات را ساده می‌کند.

۷.۴.۶ مشتق کردن Seed فرزند با HMAC

تابع `derive_child_seed` برای تولید `seed`های قطعی و قابل بازتولید از یک `master seed` استفاده می‌شود:

```

1 @lru_cache(maxsize=128)
2 def derive_child_seed(master_seed: int, index: int) -> int:
3     """Deterministically derives a child seed. Not for cryptographic use."""
4     key = str(master_seed).encode()
5     msg = str(index).encode()
6     digest = hmac.new(key, msg, digestmod=hashlib.sha256).digest()
7     return int.from_bytes(digest[:8], 'big') % MAX_SEED_INT_PCG64 or 1
8

```

این تابع با استفاده از HMAC-SHA256 یک `seed` فرزند از `master seed` و یک اندیس تولید می‌کند. کلید HMAC از `master seed` و پیام از اندیس ساخته می‌شود. سپس ۸ بایت اول `digest` به عدد صحیح تبدیل شده و با `MAX_SEED_INT_PCG64` (حداکثر مقدار مجاز برای PCG64) `modulo` می‌شود. عبارت `or 1` اطمینان می‌کند که نتیجه هرگز صفر نباشد زیرا PCG64 به `seed` غیر صفر نیاز دارد. این تابع با `@lru_cache(maxsize=128)` کش می‌شود تا در فراخوانی‌های مکرر با همان ورودی، محاسبه مجدد انجام نشود. این مهم است زیرا در شبیه‌سازی‌های بزرگ با هزاران `seed` فرزند، صرفه‌جویی در زمان محاسبه قابل توجه است. از نظر ریاضی، این تابع به صورت زیر عمل می‌کند:

$$seed_i = (int_{64}(HMAC_{SHA256}(M, i)) \bmod N_{\max}) \vee 1, \quad (13.6)$$

که در آن M دانه اصلی (`master seed`)، i اندیس و $N_{\max} = MAX_SEED_INT_PCG64$ است. عملگر $\vee$ در اینجا به معنای OR بیتی است تا صفر بودن احتمالی خروجی حاصل را به یک تبدیل کند. توجه شود که در مستندات `docstring` به صراحت اشاره شده است که این تابع برای مصارف غیررمزنگاری طراحی شده و صرفاً برای تولید دانه‌های (`seed`های) تکرارپذیر در شبیه‌سازی به کار می‌رود.

۵.۶ ارسال و دریافت با سایر ماژول‌ها

۱.۵.۶ ورودی‌های ماژول

ماژول `base.py` مجموعه‌ای از وابستگی‌های داخلی و خارجی دارد. وابستگی‌های خارجی شامل `numpy` برای محاسبات برداری، `scipy` برای توابع آماری و توزیع `beta`، و `mpmath` (اختیاری) برای دقت بالا هستند.

وابستگی‌های داخلی به صورت زیر ساختارمند شده‌اند:

- `qkd.params.QKDParams`: کلاس پارامترهای QKD که تمام تنظیمات شبیه‌سازی را در خود نگه می‌دارد. این کلاس در سازنده `FiniteKeyProof` وارد می‌شود و در سراسر طول عمر نمونه در دسترس است.
 - `qkd.datatypes.ConfidenceBoundMethod`: Enum که روش کران آماری را مشخص می‌کند. این مقدار در `QKDParams.ci_method` ذخیره شده و در متد `get_bounds` استفاده می‌شود.
 - `qkd.constants`: شامل ثابت‌های عددی مانند `ENTROPY_PROB_CLAMP`, `NUMERIC_ABS_TOL` و `MAX_SEED_INT_PCG64`. این ثابت‌ها در سراسر کد برای تنظیم رفتار عددی استفاده می‌شوند.
 - `qkd.exceptions`: شامل استثنای سفارشی مانند `ConfigurationError`, `ParameterValidationError`, `QKDSimulationError` و `ErrorCode`. این استثنایها در سراسر کد برای گزارش خطاهای ساختاریافته به کار می‌روند.
 - `qkd.utils.math`: ماژول توابع ریاضی کمکی شامل `clopper_pearson_bounds`, `gaussian_bounds` و `hoeffding_bounds`. این توابع پیاده‌سازی واقعی کران‌های آماری را ارائه می‌دهند و ماژول `base.py` تنها آن‌ها را فراخوانی می‌کند.
- این الگوی واردسازی با `try/except` به ماژول اجازه می‌دهد در صورت بروز مشکل، پیام خطای روشنی تولید کند و به جای خطای مبهم `ImportError`، اطلاعات زمینه‌ای درباره ماژول مفقود ارائه دهد.

۲.۵.۶ خروجی‌های ماژول

خروجی اصلی این ماژول کلاس `FiniteKeyProof` و `dataclass`های مرتبط است که در `__all__` فهرست شده‌اند:

```

۱ __all__ = [
۲     'FiniteKeyProof', 'DecoyEstimates', 'KeyCalculationResult', 'TallyCounts',
۳     'EpsilonAllocation', 'SolverDiagnostics', 'ProofMode', 'ErrorCode',
۴     'ConfidenceBoundMethod', 'TALLY_KEY_Z_SIGNAL', 'TALLY_KEY_X_SIGNAL',
۵     'derive_child_seed'
۶ ]
۷
```

این فهرست به ماژول‌های دیگر اجازه می‌دهد با `from qkd.proofs.base import *` فقط نمادهای عمومی مورد نظر را وارد کنند. زیرکلاس‌های اثبات امنیتی مانند `Ma2005Proof`, `Lim2014Proof` و `GLLPProof` (که در فایل‌های جداگانه پیاده‌سازی می‌شوند) از این کلاس پایه ارث می‌برند و متدهای انتزاعی را پیاده‌سازی می‌کنند.

خروجی عملی ماژول در زمان اجرا به دو شکل است:

۱. شیء `DecoyEstimates` که خروجی متد `estimate_yields_and_errors` است و شامل e_1^U , Y_1^L , `failure_prob_used`, `is_feasible` و `diagnostics` می‌شود. این شیء به عنوان ورودی به متد `calculate_key_length` داده می‌شود.

۲. شیء `KeyCalculationResult` که خروجی متد `calculate_key_length` است و شامل طول کلید امن، نشت‌ها، کران خطای فاز، کدهای خطا و اطلاعات تشخیصی است. این شیء توسط لایه بالایی فریم‌ورک (مانند `qkd.simulator`) دریافت شده و در گزارش نهایی شبیه‌سازی استفاده می‌شود.

۳.۵.۶ جریان داده در سراسر فریم‌ورک

جریان داده کلی به این شکل است:

۱. لایه شبیه‌ساز پالس‌های نوری را تولید و از کانال کوانتومی عبور می‌دهد.
۲. آشکارسازها در سمت باب، پالس‌ها را ثبت کرده و شمارش‌ها را در `TallyCounts` ذخیره می‌کنند.
۳. این شمارش‌ها به صورت یک دیکشنری `stats_map` با کلیدهایی مانند `TALLY_KEY_Z_SIGNAL` و `TALLY_KEY_X_SIGNAL` به اثبات امنیتی داده می‌شوند.

۴. متد `estimate_yields_and_errors` با استفاده از کران‌های آماری `get_bounds` و حل `LP`، `DecoyEstimates` را تولید می‌کند.

۵. متد `calculate_key_length` با استفاده از `DecoyEstimates`، `binary_entropy` و متدهای ایمن عددی، `KeyCalculationResult` نهایی را محاسبه می‌کند.

۶. لایه بالایی می‌تواند با `explain_key_decision` علت نتیجه را به صورت متنی دریافت کند و با `audit_dump` یک گزارش کامل برای `audit` تولید کند.

این جداسازی مراحل به گونه‌ای طراحی شده که هر مرحله به صورت مستقل قابل آزمایش و اعتبارسنجی باشد. به عنوان مثال، می‌توان `DecoyEstimates` را به صورت دستی ساخت و به `calculate_key_length` داد تا رفتار آن در شرایط خاص بررسی شود. این ماژول‌بندی از اصل `Single Responsibility` پیروی می‌کند و قابلیت نگهداری و توسعه کد را به طور قابل توجهی افزایش می‌دهد.

فصل ۷

اثبات امنیت کلید محدود لیم ۲۰۱۴

۱.۷ نمای کلی و هدف ماژول

فایل `qkd/proofs/lim2014.py` پیاده‌سازی وفادارانه چارچوب اثبات امنیتی کلید متناهی مبتنی بر Decoy-State است که توسط Xu, Walenta, Curty, Lim و Zbinden در سال ۲۰۱۴ در مقاله مرجع “Concise security bounds for practical decoy-state quantum key distribution” (در مجله Physical Review A، جلد ۸۹، شماره ۲، صفحه ۰۲۲۳۰۷) ارائه شده است. این چارچوب به دلیل سادگی، فشردگی و کاربردیپذیری عملی، به یکی از پذیرفته‌شده‌ترین مراجع برای محاسبه طول کلید امن در سیستم‌های تجاری و آزمایشگاهی QKD تبدیل شده است. مزیت اصلی این چارچوب نسبت به روش‌های قبلی مانند Ma 2005 این است که فرمول‌های بسته و تحلیلی برای تخمین کران‌های پایین بازده خلأ $s_{X,0}$ ، کران پایین بازده تک‌فوتونی $s_{X,1}$ ، کران بالای خطای تک‌فوتونی $v_{Z,1}$ و کران بالای خطای فاز φ_X ارائه می‌دهد که نیاز به حل برنامه‌ریزی خطی (LP) را برای حالت استاندارد سه شدت (سیگنال، decoy ضعیف و خلأ) از بین می‌برد. این فایل، خروجی نهایی محاسبه امنیتی کلید را در قالب شیء `KeyCalculationResult` تولید می‌کند که شامل طول کلید امن، نشئت تصحیح خطا، کران بالای خطای فاز، کدهای خطا و اطلاعات تشخیصی (diagnostics) است.

هدف این ماژول دوگانه است: نخست، ارائه یک مرجع عددی قابل اعتماد و قابل بازتولید برای محاسبه طول کلید امن در چارچوب Lim 2014 که در آن تمام فرمول‌های معادلات (۱) تا (۵) مقاله دقیقاً به صورت کد پیاده‌سازی شده‌اند؛ دوم، فراهم آوردن یک لایه توسعه برای مواردی که کاربر بیش از سه شدت نور نیاز دارد. در حالت چهار یا چند شدت، چارچوب Lim 2014 فرمول بسته ارائه نمی‌کند و صرفاً در پیوست مقاله اشاره می‌کند که تحلیل به طور سراسر به هر تعداد سطح شدت قابل تعمیم است. این ماژول برای پوشش این حالت، یک مسیر جایگزین مبتنی بر LP با محدودیت‌های غیرمنفی‌بودن و کران بالای صریح و با یک `tail relaxation` ارائه می‌دهد که در گزارش diagnostics به روشنی به عنوان توسعه فراتر از نتایج بسته مقاله علامت‌گذاری شده است. این شفافیت روش‌شناسی برای کاربردهای حساس مانند `certification` و بازبینی امنیتی توسط شخص ثالث ضروری است.

یکی از نوآوری‌های کلیدی این نسخه از پیاده‌سازی، استفاده از یک استراتژی Hoeffding/Chernoff برای بازه‌های اطمینان است که در ضمیمه A مقاله صراحتاً مجاز شناخته شده است. مقاله اصلی برای سادگی فرمول بسته، از نامساوی Hoeffding با یک نوسان δ یکسان برای همه شدت‌ها استفاده می‌کند، اما این انتخاب در شرایطی که شمارش decoy یا خلأ بسیار کم است (مانند فواصل طولانی کانال که در آن‌ها n_{X,μ_3} ممکن است تنها چند رویداد باشد)، به طور پاتولوژیک شل می‌شود و کران‌های پایین منفی یا کران‌های بالا بیش از کل فیزیکی تولید می‌کند. پیاده‌سازی هیبرید در سه رژیم عمل می‌کند: برای شمارش‌های بزرگ ($n_k \geq 10 \delta_{\text{paper}}$) از Hoeffding وفادار به کاغذ استفاده می‌کند؛ برای شمارش‌های کوچک اما ناصفر ($0 < n_k < 10 \delta_{\text{paper}}$) از Chernoff به‌ازای هر شدت با $\delta_k = \sqrt{2n_k \ln(1/\varepsilon)}$ استفاده می‌کند که تنگ‌تر از Hoeffding است؛ و برای $n_k = 0$ از کران بالای Chernoff ضربی $\ln(1/\varepsilon_{\text{per-event}})$ (قانون سه) استفاده می‌کند که متناهی و بسیار کوچکتر از δ_{paper} است. این استراتژی به‌طور قابل اثبات کوچکتر یا مساوی Hoeffding برای همه n_k است، بنابراین امنیت هرگز تضعیف نمی‌شود.

این فایل از کلاس پایه `FiniteKeyProof` (که در `qkd/proofs/base.py` تعریف شده) ارث می‌برد و متدهای انتزاعی `estimate_yields_and_errors`، `get_epsilon_policy`، `allocate_epsilons` و `calculate_key_length` و همچنین یک `dataclass` جدید به نام `DecoyEstimatesLim2014` معرفی می‌کند که فیلدهای اضافی مانند $s_{X,0}^L$ ، $s_{X,1}^L$ ، $s_{Z,1}^L$ ، $v_{Z,1}^U$ و φ_X^U را نگه

می‌دارد و از DecoyEstimates پایه ارث می‌برد. این طراحی به کاربر اجازه می‌دهد هم به رابط عمومی FiniteKeyProof و هم به اطلاعات اختصاصی چارچوب Lim 2014 دسترسی داشته باشد. علاوه بر این، سازنده کلاس __init__ به صورت خودکار روش بازه اطمینان پارامتر ci_method را به ChernoFF بازنویسی می‌کند زیرا امنیت کلید متناهی در این چارچوب به طور صریح به کران‌های تمرکز Chernoff وابسته است. وظیفه نهایی این ماژول، تبدیل شمارش‌های آماری شبکه آشکارسازی به یک عدد صحیح مثبت (یا صفر) به عنوان طول کلید امن است. این تبدیل با رعایت سخت‌گیرانه تمام محافظه‌کاری‌های ریاضی و آماری انجام می‌شود تا هیچ طول کلیدی که از نظر تئوری قابل اثبات نیست، تولید نشود. یک لایه دفاعی فیزیکی جدید در متد calculate_key_length اضافه شده است که طول کلید نهایی را به مجموع بیت‌های sifted $(n_X + n_Z)$ محدود می‌کند؛ این قید در صورت بروز هرگونه باگ داخلی (مانند حلقه kappa-solver، fallback در LP، یا سرریز در تخمین decoy) فعال می‌شود و یک هشدار گزارش می‌کند. در صورت بروز هرگونه ناسازگاری پیکربندی، آمار ناکافی یا خطای عددی، خروجی به جای استثنا، یک KeyCalculationResult با طول کلید صفر و کد خطای مناسب تولید می‌کند تا خط لوله شبیه‌سازی متوقف نشود و علت شکست در گزارش تشخیصی ثبت گردد.

۲.۷ مبانی تئوری و فیزیکی

۱.۲.۷ پروتکل Decoy-State با سه شدت

در پروتکل استاندارد BB84 با یک منبع تک‌شدت، امنیت در برابر حمله Photon-Number Splitting (PNS) قابل تضمین نیست؛ زیرا یک مهاجم با تقسیم فوتون‌های چندفوتونی می‌تواند بدون ایجاد خطا اطلاعات کلید را استخراج کند. راه‌حل Decoy-State این است که آلیس به طور تصادفی شدت نور ارسالی را میان چندین سطح تغییر می‌دهد و باب پس از اندازه‌گیری، با مقایسه نرخ آشکارسازی در شدت‌های مختلف، بازده تک‌فوتونی را تخمین می‌زند. در چارچوب Lim 2014، سه شدت $\mu_1 > \mu_2 > \mu_3 \geq 0$ در نظر گرفته می‌شود که μ_1 شدت سیگنال، μ_2 شدت decoy ضعیف و μ_3 شدت خلأ است. شرط کلیدی $\mu_1 > \mu_2 + \mu_3$ در تابع validate_three_intensity_order بررسی می‌شود و تضمین می‌کند که مخرج فرمول‌های تحلیلی مثبت باشد. این شرط در کنار $\mu_2 > \mu_3 \geq 0$ یک مجموعه کامل از قیود ترتیبی را تشکیل می‌دهد که در آن مخرج توزیع آماری فوتون‌های ارسالی از یک منبع لیز با توزیع Poisson پیروی می‌کند. اگر میانگین فوتون μ باشد، احتمال ارسال n فوتون برابر است با:

$$P(n | \mu) = e^{-\mu} \frac{\mu^n}{n!}. \quad (۱.۷)$$

احتمال کلی تولید n فوتون بدون توجه به شدت انتخاب‌شده، τ_n نامیده می‌شود و به صورت زیر تعریف می‌شود:

$$\tau_n = \sum_k p_k P(n | \mu_k) = \sum_k p_k e^{-\mu_k} \frac{\mu_k^n}{n!}, \quad (۲.۷)$$

که در آن p_k احتمال انتخاب شدت μ_k توسط آلیس است. این کمیت در تابع get_tau_n پیاده‌سازی شده و نقش مرکزی در همه فرمول‌های تحلیلی چارچوب دارد. توجه شود که τ_1 و τ_0 به طور خاص در معادلات (۲) و (۳) ظاهر می‌شوند و مثبت بودن آن‌ها یک پیش‌نیاز برای ادامه محاسبه است؛ در صورت صفر شدن این مقادیر (که می‌تواند در پیکربندی‌های غیرفیزیکی رخ دهد)، ماژول به یک نتیجه صفر با کد خطای INSUFFICIENT_STATISTICS بازمی‌گردد.

۲.۲.۷ تخصیص ε و ۲۱ رویداد شکست

یکی از نوآوری‌های کلیدی Lim 2014، ارائه یک طرح صریح برای تقسیم بودجه امنیتی ε_{sec} میان ۲۱ رویداد شکست مستقل است. در ضمیمه B مقاله، نشان داده می‌شود که اگر هر رویداد شکست به یک مقدار مشترک ε تنظیم شود، کل ε_{sec} از رابطه زیر به دست می‌آید:

$$\varepsilon_{\text{sec}} = 21 \varepsilon \quad \Rightarrow \quad \varepsilon = \frac{\varepsilon_{\text{sec}}}{21}. \quad (۳.۷)$$

این ۲۱ رویداد شامل کران‌های Hoeffding برای شمارش آشکارسازی و خطا در چندین پایه و شدت، کران تخمین خطای فاز، کران تصحیح خطا و کران تقویت حریم خصوصی است. در کد،

ثابت $\text{LIM2014_FAILURE_EVENTS_3_INTENSITY} = 21.0$ این مقدار را نگه می‌دارد و در تابع allocate_epsilons برای محاسبه ε تک‌آزمون استفاده می‌شود. این تخصیص محافظه‌کارانه است زیرا فرض می‌کند همه ۲۱ رویداد همزمان در بدترین حالت ممکن شکست می‌خورند. در مسیر N-decoy (بیش از سه شدت)، همین تخصیص ۲۱ رویدادی به صورت محافظه‌کارانه بازاستفاده می‌شود زیرا مقاله فرمول بسته‌ای برای این حالت ارائه نمی‌کند.

علاوه بر این، یک پارامتر مرتبط به نام failure_prob_used در $\text{DecoyEstimatesLim2014}$ وجود دارد که در عمل برابر ε_{sec} استفاده شده در محاسبه است. این فیلد به کاربر اجازه می‌دهد بداند آیا در مسیر kappa-solver تکراری، یک ε_{sec} متفاوت از مقدار پیش فرض p.eps_sec استفاده شده است یا خیر. این شفافیت برای بازبینی امنیتی و audit حساس است زیرا تغییر ε_{sec} مستقیماً بر طول کلید نهایی تأثیر می‌گذارد.

۳.۲.۷ کران‌های Hoeffding و انگیزه استراتژی هیبرید

برای تخمین بازده و خطا از روی شمارش‌های مشاهده شده، چارچوب Lim 2014 از نامساوی Hoeffding استفاده می‌کند. اگر $n_{X,k}$ تعداد آشکارسازی‌های مشاهده شده در پایه X با شدت μ_k باشد و $n_X = \sum_k n_{X,k}$ کل آشکارسازی‌های پایه X باشد، کران‌های بالا و پایین با یک نوسان δ به صورت زیر تعریف می‌شوند:

$$n_{X,k}^{\pm} = \frac{e^{\mu_k}}{p_k} (n_{X,k} \pm \delta), \quad \delta = \sqrt{\frac{n_X}{2} \ln\left(\frac{21}{\varepsilon_{\text{sec}}}\right)}. \quad (۴.۷)$$

ضریب پیشانی e^{μ_k}/p_k برای تبدیل شمارش مشاهده شده در شدت k به تخمین کلان بازده آن شدت است. در پیاده‌سازی مقاله، δ برای همه شدت‌ها یکسان و تنها به n_X (کل پایه) وابسته است. این انتخاب ساده و بسته است، اما یک ضعف عملی جدی دارد: وقتی شمارش یک شدت خاص $n_{X,k}$ بسیار کمتر از n_X باشد (مانند شمارش خلأ در فواصل طولانی که در آن n_{X,μ_3} ممکن است تنها چند رویداد باشد در حالی که n_X صدها هزار است)، نوسان δ مبتنی بر n_X می‌تواند به طور قابل توجهی بزرگتر از خود $n_{X,k}$ شود. به عنوان مثال، در یک شبیه‌سازی بلندمدت با $n_X = 10^6$ و $\varepsilon_{\text{sec}} = 10^{-9}$ ، مقدار $\delta \approx \sqrt{5 \times 10^5 \times \ln(21 \times 10^9)} \approx 141$ می‌شود، در حالی که شمارش مشاهده شده خلأ ممکن است تنها $n_{X,\mu_3} = 5$ باشد. در این حالت، کران بالای Hoeffding $n_{X,\mu_3}^+ \approx (e^{\mu_3}/p_3) \times 146$ می‌شود که حدود ۱۱۴ برابر مشاهده است و کاملاً غیرفیزیکی است.

این مشکل در ضمیمه A مقاله به صورت صریح تأیید می‌شود: “the Hoeffding choice is for closed-form simplicity; tighter concentration inequalities may be used” هیبرید، این مجوز را به کار می‌گیرد تا در رژیم‌هایی که Hoeffding پاتولوژیک است، از کران‌های تمرکز تنگ‌تر استفاده کند بدون آنکه فرض امنیتی تغییر کند. این استراتژی در ادامه توضیح داده می‌شود.

۴.۲.۷ استراتژی هیبرید Hoeffding/Chernoff

استراتژی هیبرید بر اساس یک آستانه که نسبت خطای آماری نسبی را کنترل می‌کند، تعریف می‌شود. اگر $n_k \geq 10 \delta_{\text{paper}}$ نوسان Hoeffding مقاله باشد، آستانه هیبرید به صورت $\delta_{\text{paper}} = \sqrt{(n_X/2) \ln(21/\varepsilon_{\text{sec}})}$ تعریف می‌شود. این آستانه منطقی است زیرا در این نقطه، نسبت $\delta_{\text{paper}}/n_k \leq 0.1$ ، یعنی خطای آماری نسبی کمتر از ۱۰٪ است که در آن Hoeffding به طور معقولی تنگ عمل می‌کند. در سه رژیم زیر، بازه اطمینان به صورت متفاوت محاسبه می‌شود:

رژیم ۱ (شمارش بزرگ): وقتی $n_k \geq 10 \delta_{\text{paper}}$ و $n_k > 0$ از Hoeffding وفادار به کاغذ استفاده می‌شود:

$$n_{X,k}^{\pm} = \frac{e^{\mu_k}}{p_k} (n_{X,k} \pm \delta_{\text{paper}}). \quad (۵.۷)$$

این رژیم در شرایطی که شمارش کافی وجود دارد، رفتار پیاده‌سازی را دقیقاً با مقاله همخوان می‌کند.
رژیم ۲ (شمارش کوچک اما ناصفر): وقتی $0 < n_k < 10 \delta_{\text{paper}}$ از Chernoff به‌ازای هر شدت استفاده می‌شود:

$$n_{X,k}^{\pm} = \frac{e^{\mu_k}}{p_k} (n_{X,k} \pm \delta_k), \quad \delta_k = \sqrt{2 n_k \ln\left(\frac{1}{\varepsilon_{\text{per-event}}}\right)}, \quad (۶.۷)$$

که در آن $\varepsilon_{\text{per-event}} = \varepsilon_{\text{sec}}/21$ طبق ضمیمه B است. توجه شود که $\ln(21/\varepsilon_{\text{sec}}) = \ln(1/\varepsilon_{\text{per-event}})$ ، بنابراین همان جمله لگاریتمی در هر دو رژیم استفاده می‌شود و تنها تفاوت در مقیاس نوسان است: Hoeffding

$n_X/2$ مقیاس می‌گیرد در حالی که Chernoff با n_k مقیاس می‌گیرد. برای $n_k \ll n_X$ ، این تفاوت چشمگیر است و کران تنگ‌تر تولید می‌کند.
رژیم ۳ (شمارش صفر): وقتی $n_k = 0$ ، از کران بالای Chernoff ضریبی استفاده می‌شود که از قانون سه برای مشاهدات صفر در توزیع Binomial/Poisson استخراج می‌شود:

$$P(X = 0 | \mu) = e^{-\mu} \leq \varepsilon \Rightarrow \mu \leq \ln\left(\frac{1}{\varepsilon}\right), \quad (۷.۷)$$

بنابراین کران بالای کران فیزیکی متناهی $\ln(1/\varepsilon_{\text{per-event}})$ است. این کران در مقایسه با Hoeffding که برای $n_k = 0$ به δ_{paper} می‌رسد (که می‌تواند بزرگ باشد)، بسیار کوچکتر و فیزیکی‌تر است. برای $\varepsilon_{\text{sec}} = 10^{-9}$ ، این کران برابر $\ln(21 \times 10^9) \approx 23.6$ است که یک کران فوق‌العاده تنگ‌تر از $\delta_{\text{paper}} \approx 141$ در همان شرایط است. اثبات اینکه این استراتژی هیبرید هرگز امنیت را تضعیف نمی‌کند، ساده است: هر کران تمرکزی که پوشش $1 - \varepsilon$ ارائه می‌کند، در چارچوب امنیتی معتبر است. از آنجا که Chernoff برای شمارش‌های کوچک تنگ‌تر از Hoeffding است (یعنی کران پایین بزرگتر و کران بالا کوچکتر تولید می‌کند، که هر دو محافظه‌کارانه‌تر هستند)، نتیجه هیبرید همیشه حداقل به اندازه Hoeffding محافظه‌کارانه است. در نتیجه، طول کلید نهایی همیشه کمتر یا مساوی طول کلید Hoeffding خالص است، که این دقیقاً همان جهت مطلوب برای امنیت است.

۵.۲.۷ فرمول تحلیلی کران پایین بازده خلأ

معادله (۲) مقاله، کران پایین تعداد رویدادهای خلأ در پایه X را به صورت زیر ارائه می‌دهد:

$$s_{X,0} \geq \tau_0 \frac{\mu_2 n_{X,\mu_3}^- - \mu_3 n_{X,\mu_2}^+}{\mu_2 - \mu_3}. \quad (۸.۷)$$

در این فرمول، n_{X,μ_3}^- کران پایین آشکارسازی در شدت خلأ است که با کمترین مقدار ممکن، صورت کسر را کوچک می‌کند و در نتیجه کران پایین محافظه‌کارانه می‌شود. به طور متقابل، n_{X,μ_2}^+ کران بالای آشکارسازی در شدت decoy ضعیف است که با بزرگ‌ترین مقدار ممکن، جمله کم‌شونده را بزرگ می‌کند و باز هم کران را سخت‌گیرانه می‌سازد. این انتخاب‌های جهت‌دار تضمین می‌کنند که تخمین نهایی، در بدترین حالت نوسان آماری، همچنان یک کران پایین معتبر باشد. در تابع calc_s0_lower این فرمول پیاده‌سازی شده و در صورت منفی شدن صورت کسر، خروجی به صفر برده می‌شود. در نسخه فعلی، با معرفی استراتژی هیبرید، کران‌های n_{X,μ_2}^+ و n_{X,μ_3}^- به صورت خودکار در رژیم مناسب (بزرگ، کوچک یا صفر) محاسبه می‌شوند، که به طور خاص برای $n_{X,\mu_3} = 0$ (که در فواصل طولانی رایج است) یک کران پایین معنادار به جای یک مقدار منفی تولید می‌کند.

۶.۲.۷ فرمول تحلیلی کران پایین بازده تک‌فوتونی

معادله (۳) مقاله، کران پایین تعداد رویدادهای تک‌فوتونی در پایه X را به صورت زیر ارائه می‌دهد:

$$s_{X,1} \geq \tau_1 \mu_1 \frac{T}{D}, \quad (۹.۷)$$

که در آن:

$$T = n_{X,\mu_2}^- - n_{X,\mu_3}^+ - \frac{\mu_2^2 - \mu_3^2}{\mu_1^2} \left(n_{X,\mu_1}^+ - \frac{s_{X,0}}{\tau_0} \right), \quad (۱۰.۷)$$

$$D = \mu_1(\mu_2 - \mu_3) - \mu_2^2 + \mu_3^2. \quad (۱۱.۷)$$

مخرج D با شرط $\mu_1 > \mu_2 + \mu_3$ مثبت می‌ماند. این فرمول پیچیده‌تر از معادله (۲) است زیرا از تخمین بازده خلأ $s_{X,0}$ نیز استفاده می‌کند تا اثر فوتون‌های چندتایی حذف شود. در تابع calc_s1_lower این فرمول با دریافت $\mu_3, \mu_2, \mu_1, \tau_1$ ، کران‌های Chernoff/Hoeffding برای هر سه شدت و نسبت $s_{X,0}/\tau_0$ پیاده‌سازی شده است. همانند معادله (۲)، انتخاب جهت‌دار کران‌ها (کران پایین برای μ_2 و کران بالا برای μ_3 و μ_1) تضمین می‌کند که نتیجه یک کران پایین معتبر باشد. در صورت منفی شدن صورت T ، خروجی به صفر برده می‌شود زیرا تعداد رویدادهای تک‌فوتونی نمی‌تواند منفی باشد.

۷.۲.۷ فرمول تحلیلی کران بالای خطای تک فوتونی

معادله (۴) مقاله، کران بالای تعداد خطاهای تک فوتونی در پایه Z را به صورت زیر ارائه می‌دهد:

$$v_{Z,1} \leq \tau_1 \frac{m_{Z,\mu_2}^+ - m_{Z,\mu_3}^-}{\mu_2 - \mu_3}, \quad (۱۲.۷)$$

که در آن $m_{Z,k}^\pm$ کران‌های Chernoff/Hoeffding برای شمارش خطاها در پایه Z با شدت μ_k هستند. در اینجا از کران بالای خطاهای decoy ضعیف و کران پایین خطاهای خلأ استفاده می‌شود تا کران بالای محافظه‌کارانه به دست آید. تابع `_calc_v1_upper` این محاسبه را پیاده‌سازی می‌کند و خروجی را به یک کران فیزیکی (مثلاً n_Z) محدود می‌سازد. توجه شود که این کران بالا برای خطاها است، بنابراین در جهت مخالف کران‌های پایین عمل می‌کند: بزرگ‌ترین مقدار ممکن برای m_{Z,μ_2}^+ و کوچک‌ترین مقدار ممکن برای m_{Z,μ_3}^- انتخاب می‌شوند تا صورت کسر بزرگ و در نتیجه کران بالا محافظه‌کارانه شود.

۸.۲.۷ کران بالای خطای فاز با نوسان نمونه‌گیری تصادفی

معادله (۵) مقاله، کران بالای خطای فاز φ_X را با در نظر گرفتن نوسان نمونه‌گیری تصادفی (random-sampling fluctuation) ارائه می‌دهد:

$$\varphi_X = \frac{c_{X,1}}{s_{X,1}} \leq \frac{v_{Z,1}}{s_{Z,1}} + \gamma(a, b, c, d), \quad (۱۳.۷)$$

که در آن:

$$a = \frac{\varepsilon_{\text{sec}}}{21}, \quad (۱۴.۷)$$

$$b = \frac{v_{Z,1}}{s_{Z,1}}, \quad (۱۵.۷)$$

$$c = s_{Z,1}, \quad (۱۶.۷)$$

$$d = s_{X,1}, \quad (۱۷.۷)$$

$$\gamma(a, b, c, d) = \sqrt{\frac{(c+d)(1-b)b}{cd \ln 2} \log_2 \left(\frac{c+d}{cd(1-b)ba^2} \right)}. \quad (۱۸.۷)$$

این کران یکی از پیچیده‌ترین بخش‌های چارچوب Lim 2014 است و در تابع `_calc_phi_X_upper` پیاده‌سازی شده است. در حالت‌های تباهی ($b \in \{0, 1\}$ یا $c \leq 0$ یا $d \leq 0$)، تابع به صورت ایمن به مقدار بدترین حالت 0.5 بازمی‌گردد که با کاغذ همخوان است. ریشه دوم در فرمول اصلی به صراحت در مقاله آمده است (اگرچه چیدمان تایپی ممکن است آن را پنهان کند) و در کد با `math.sqrt` پیاده‌سازی شده است. مدیریت دقیق این حالت‌های تباهی در ادامه در بخش پیاده‌سازی توضیح داده می‌شود.

۹.۲.۷ طول کلید امن و نشت تصحیح خطا

معادله (۱) مقاله، طول کلید امن نهایی را به صورت زیر ارائه می‌دهد:

$$\ell = s_{X,0} + s_{X,1}(1 - h(\varphi_X)) - \lambda_{\text{EC}} - 6 \log_2 \left(\frac{21}{\varepsilon_{\text{sec}}} \right) - \log_2 \left(\frac{2}{\varepsilon_{\text{cor}}} \right), \quad (۱۹.۷)$$

که در آن $h(p) = -p \log_2 p - (1-p) \log_2 (1-p)$ آنتروپی باینری است. نشت تصحیح خطا λ_{EC} در حالت استاندارد به صورت زیر محاسبه می‌شود:

$$\lambda_{\text{EC}} = f_{\text{EC}} n_X h(e_{\text{obs}}), \quad (۲۰.۷)$$

که در آن $f_{\text{EC}} \geq 1$ کارایی تصحیح خطا و $e_{\text{obs}} = m_X/n_X$ خطای بیت مشاهده‌شده در پایه X است. در مدل tomamichel (اختیاری، بر اساس مرجع [۳۷] مقاله یعنی Tomamichel et al., arXiv:1401.5194)، جمله $\log_2(2/\varepsilon_{\text{cor}})$ به λ_{EC} اضافه می‌شود تا یک مدل نشت دقیق‌تر ارائه گردد و در نتیجه جمله جداگانه صحت صفر می‌شود تا از شمارش مضاعف جلوگیری شود. جمله $6 \log_2(21/\varepsilon_{\text{sec}})$ شش رویداد شکست مرتبط با تقویت حریم خصوصی (privacy amplification) را پوشش می‌دهد و در گزارش diagnostics به صورت `pa_term` ذخیره می‌شود.

۱۰.۲.۷ قید فیزیکی دفاعی روی طول کلید

یکی از additions مهم در این نسخه از پیاده‌سازی، یک لایه دفاعی فیزیکی در متد `calculate_key_length` است که پس از محاسبه نهایی طول کلید اعمال می‌شود. این قید بر اساس یک اصل بدیهی فیزیکی است: **بیت‌های کلید امن زیرمجموعه‌ای از بیت‌های خام sifted هستند**، پس طول کلید نمی‌تواند از مجموع $n_X + n_Z$ (یعنی کل بیت‌های sifted) بیشتر شود. این قید در شرایط عادی هرگز فعال نمی‌شود زیرا فرمول‌های Lim 2014 به‌طور ذاتی محافظه‌کارانه هستند، اما به‌عنوان یک تور ایمنی در برابر باگ‌های داخلی طراحی شده است. سه سناریویی که در آن‌ها این قید ممکن است فعال شود عبارتند از: (۱) حلقه `kappa-solver` تکراری که به‌دلیل عدم همگرایی، یک ε_{sec} نامعتبر تولید کرده و در نتیجه ℓ بزرگ می‌شود؛ (۲) fallback در مسیر LP که ممکن است مقادیر غیرصفر برای کران پایین بازده تولید کند حتی وقتی آمار کافی نیست؛ (۳) سرریز در تخمین decoy به‌دلیل خطاهای عددی در محاسبات Chernoff. در هر یک از این موارد، قید دفاعی طول کلید را به $n_X + n_Z$ محدود می‌کند، یک هشدار در `log` ثبت می‌کند و نتیجه بازنویسی‌شده را بازمی‌گرداند. این طراحی در فلسفه fail-safe عمل می‌کند: حتی در صورت بروز باگ داخلی، خروجی هرگز از نظر فیزیکی نامعتبر نخواهد بود.

۳.۷ تحلیل معماری و ساختار داده‌ها

۱.۳.۷ ثابت‌های مازول و کنترل عددی

فایل با تعریف چند ثابت کلیدی آغاز می‌شود که رفتار عددی و امنیتی پیاده‌سازی را کنترل می‌کنند:

```

۱ MAX_ERROR_RATE = 5.0
۲ HOEFFDING_FACTOR = 0.10
۳ PROB_SUM_TOL = 1e-9
۴ MIN_EPS = 1e-300
۵ LIM2014_FAILURE_EVENTS_3_INTENSITY = 0.21
۶ DEFAULT_KAPPA_MAX_ITERATIONS = 50
۷ DEFAULT_KAPPA_ABS_TOL_BITS = 0.100
۸ DEFAULT_KAPPA_REL_TOL = 1e-6
۹ MAX_SECURITY_EPS = 0.1 - 1e-15
۱۰
```

ثابت `MAX_ERROR_RATE = 0.5` حد بالای فیزیکی خطای بیت است که در حالت بدترین حالت (یعنی زمانی که هیچ اطلاعاتی از کانال استخراج نمی‌شود) به کار می‌رود. این مقدار برای clamp کردن خطای فاز و خطای مشاهده‌شده به بازه $[0, 0.5]$ استفاده می‌شود. `HOEFFDING_FACTOR = 10.0` ثابت جدیدی است که آستانه انتخاب رژیم در استراتژی هیبرید را تعیین می‌کند: وقتی $n_k \geq \text{HOEFFDING_FACTOR} \times \delta_{\text{paper}}$ ، از Hoeffding استفاده می‌شود. مقدار ۱۰ به این معناست که وقتی خطای آماری نسبی $\delta_{\text{paper}}/n_k \leq 10\%$ است، Hoeffding به‌طور معقولی تنگ عمل می‌کند و نیازی به Chernoff نیست. `PROB_SUM_TOL = 1e-9` تلورانس قابل قبول برای مجموع احتمال‌های شدت نور است. `MIN_EPS = 1e-300` کف بسیار کوچکی برای ε است که از خطاهای عددی در محاسبات `log` جلوگیری می‌کند.

`LIM2014_FAILURE_EVENTS_3_INTENSITY = 21.0` تعداد رویدادهای شکست در طرح تخصیص ε است که در ضمیمه B مقاله توضیح داده شده است. سه ثابت `DEFAULT_KAPPA_MAX_ITERATIONS`، `DEFAULT_KAPPA_ABS_TOL_BITS` و `DEFAULT_KAPPA_REL_TOL` برای حل تکراری معادله $\varepsilon_{\text{sec}} = \kappa \cdot \ell$ استفاده می‌شوند که در بخش SIV مقاله معرفی شده است. در این روش، به طول کلید متناسب می‌شود و باید با تکرار حل شود زیرا ℓ خود به ε_{sec} وابسته است. حداکثر ۵۰ تکرار با تلورانس مطلق ۱۰۰ بیت و تلورانس نسبی 10^{-6} به عنوان معیار همگرایی در نظر گرفته شده است. `MAX_SECURITY_EPS = 1.0 - 1e-15` کران بالای ε_{sec} است که از تنظیم آن به ۱ (یعنی عدم امنیت کامل) جلوگیری می‌کند.

۲.۳.۷ DecoyEstimatesLim2014

این `dataclass` یک ساختار داده تخصصی برای نگهداری تخمین‌های میانی چارچوب Lim 2014 است:

```

۱ @dataclass(frozen=True)
۲ class DecoyEstimatesLim2014(DecoyEstimates):
۳     """
۴     Intermediate decoy estimates for the Lim-2014 calculation.
۵     """
۶
۷     s_X_0_lower: float = 0.0
```

```

۸ s_X_1_lower: float = 0.0
۹ s_Z_1_lower: float = 0.0
۱۰ v_Z_1_upper: float = 0.0
۱۱ phi_X_upper: float = MAX_ERROR_RATE
۱۲ is_feasible: bool = False
۱۳ failure_prob_used: float = 0.1
۱۴ diagnostics: Dict[str, Any] = field(default_factory=dict)
۱۵
۱۶ def to_dict(self) -> Dict[str, Any]:
۱۷     return _json_safe_dict(self)
۱۸

```

این کلاس از DecoyEstimates (تعریف شده در base.py) ارث می‌برد و فیلدهای اضافی اختصاصی چارچوب Lim 2014 را اضافه می‌کند. پنج فیلد اصلی φ_X^U و $v_{Z,1}^U$ ، $s_{Z,1}^L$ ، $s_{X,1}^L$ ، $s_{X,0}^L$ هستند که مستقیماً از معادلات (۲) تا (۵) مقاله استخراج می‌شوند. is_feasible نشان می‌دهد که آیا تخمین معتبر بوده و طول کلید مثبت تولید شده است یا خیر. failure_prob_used احتمال شکستی است که در محاسبه کران‌ها استفاده شده (معمولاً ϵ_{sec} یا $\epsilon_{\text{sec}}/21$). diagnostics یک دیکشنری برای اطلاعات تشخیصی کامل است. استفاده از frozen=True اطمینان می‌دهد که پس از ساخت، مقادیر قابل تغییر نیستند که این ویژگی برای صحت‌سنجی نتایج امنیتی حیاتی است. متد to_dict برای سریال‌سازی به JSON با تبدیل نوع‌های numpy و Enum به نوع‌های بومی پایتون استفاده می‌شود.

۳.۳.۷ Lim2014Proof و سازنده آن

کلاس اصلی ماژول، Lim2014Proof است که به صورت dataclass و frozen تعریف شده و از FiniteKeyProof ارث می‌برد. سازنده این کلاس در نسخه فعلی دارای یک منطق جدید برای اعمال روش بازه اطمینان است:

```

۱ @dataclass(frozen=True)
۲ class Lim2014Proof(FiniteKeyProof):
۳     """
۴     Lim et al. 2014 finite-key proof implementation.
۵
۶     The three-intensity analytical path implements Eqs. (1)-(5) of the paper
۷     exactly. For more than three intensities, a conservative LP-based
۸     N-decoy estimate is used (see module docstring).
۹     """
۱۰
۱۱     p: "QKDParams" # type: ignore[assignment]
۱۲
۱۳     def __init__(self, p: "QKDParams"):
۱۴         import logging
۱۵         from qkd.proofs.base import ENTROPY_PROB_CLAMP, ProofMode
۱۶         try:
۱۷             import mpmath
۱۸             has_mpmath = True
۱۹         except ImportError:
۲۰             has_mpmath = False
۲۱
۲۲         object.__setattr__(self, 'p', p)
۲۳         object.__setattr__(self, 'mode', ProofMode.PRODUCTION)
۲۴         object.__setattr__(self, 'logger', logging.getLogger(self.__class__.__name__))
۲۵         object.__setattr__(self, 'run_id', getattr(p, 'run_id', 'no-run-id'))
۲۶         object.__setattr__(self, 'min_prob_clamp', getattr(p, 'entropy_clamp', ENTROPY_PROB_CLAMP))
۲۷         object.__setattr__(self, 'use_mpmath', getattr(p, 'use_mpmath', False) and has_mpmath)
۲۸         if getattr(p, 'ci_method', None) is not None:
۲۹             from ..datatypes import ConfidenceBoundMethod
۳۰             if p.ci_method != ConfidenceBoundMethod.CHERNOFF:
۳۱                 import logging
۳۲                 logging.getLogger(self.__class__.__name__).warning(
۳۳                     f"Lim2014 proof requires CHERNOFF bounds for finite-key security. "
۳۴                     f"Overriding '{p.ci_method.name}' with CHERNOFF."
۳۵                 )
۳۶         object.__setattr__(p, 'ci_method', ConfidenceBoundMethod.CHERNOFF)
۳۷

```

از آنجا که کلاس @dataclass(frozen=True) است، تخصیص مستقیم ویژگی‌ها در __init__ مجاز نیست. به جای آن از object.__setattr__ برای دور زدن این محدودیت استفاده می‌شود. این الگو به ما

اجازه می‌دهد کلاس را به صورت immutable نگه داریم در حالی که هنوز بتوانیم در زمان ساخت، وابستگی‌ها را تزریق کنیم. سپس mpmath به صورت اختیاری وارد می‌شود و اگر موجود نباشد، پرچم use_mpmath به False تنظیم می‌شود. این طراحی امکان استفاده در محیط‌هایی بدون mpmath را فراهم می‌کند.

بخش جدید و حیاتی سازنده، بررسی p.ci_method است. امنیت کلید متناهی در چارچوب Lim 2014 به طور صریح به کران‌های تمرکز Chernoff وابسته است (هم در رژیم Hoeffding و هم در رژیم Chernoff هیبرید، زیرا Hoeffding یک حالت خاص از Chernoff است). اگر کاربر روش دیگری مانند CONFIDENCE_INTERVAL یا BOOTSTRAP را تنظیم کرده باشد، سازنده یک هشدار ثبت می‌کند و روش را به CHERNOFF بازنویسی می‌کند. این override تهاجم‌ها با فلسفه fail-safe است: به جای ارائه یک نتیجه نامن، پیاده‌سازی به طور خودکار به تنظیمات امن تغییر می‌یابد و کاربر را مطلع می‌سازد. توجه شود که این بازنویسی روی خود شیء p انجام می‌شود (object.__setattr__(p, ...))، که یک side-effect است؛ این انتخاب آگاهانه است زیرا در غیر این صورت، تنظیمات پیکربندی کاربر در طول خط لوله شبیه‌سازی ناسازگار می‌ماند.

۴.۳.۷ متدهای استاتیک ایمن عددی

کلاس چهار متد استاتیک برای محاسبات ایمن عددی دارد. متد _validate_epsilon_value اعتبارسنجی ϵ را انجام می‌دهد:

```

1 @staticmethod
2 def _validate_epsilon_value(value: float, name: str) -> None:
3     if not math.isfinite(float(value)) or not 0.0 < float(value) < (0.1:
4         raise ParameterValidationError(f"{name} must be finite and in the open interval (0, 1).")
5

```

این متد بررسی می‌کند که مقدار متناهی باشد و در بازه باز (0, 1) قرار داشته باشد. بازه باز به این معناست که $\epsilon = 0$ (که منجر به تقسیم بر صفر در فرمول‌های log می‌شود) و $\epsilon = 1$ (که به معنای عدم امنیت است) رد می‌شوند. این اعتبارسنجی برای همه ϵ های ورودی به کار می‌رود: ϵ_{cor} ، ϵ_{sec} و ϵ تک‌آزمون. در مسیر kappa-solver تکراری نیز برای هر ϵ_{target} جدید اعمال می‌شود تا از مقادیر نامعتبر جلوگیری شود. متد _safe_log و _safe_log2 لگاریتم‌های ایمن هستند:

```

1 @staticmethod
2 def _safe_log(x: float) -> float:
3     if not math.isfinite(float(x)) or x <= :0.0
4         raise ParameterValidationError(f"log argument must be finite and positive; got {x!r}.")
5     return math.log(x)
6
7 @staticmethod
8 def _safe_log2(x: float) -> float:
9     if not math.isfinite(float(x)) or x <= :0.0
10        raise ParameterValidationError(f"log2 argument must be finite and positive; got {x!r}.")
11    return math.log2(x)
12

```

برخلاف متد _safe_log در کلاس پایه که مقدار بسیار کوچک را به 10^{-300} clamp می‌کند، این پیاده‌سازی در صورت دریافت ورودی نامعتبر، استثنا پرتاب می‌کند. این رفتار سخت‌گیرانه‌تر به این دلیل است که در چارچوب Lim 2014، هر ورودی غیرمثبت به لگاریتم نشانگر یک خطای منطقی جدی است که باید ریشه‌یابی شود، نه یک خطای عددی قابل چشم‌پوشی. متد _clamp_probability احتمالات را به بازه $[0, u]$ محدود می‌کند که در آن u به صورت پیش‌فرض ۱ است اما می‌تواند به 0.5 (برای خطاها) تنظیم شود:

```

1 @staticmethod
2 def _clamp_probability(x: float, *, upper: float = (0.1 -> float:
3     if not math.isfinite(float(x)):
4         return upper
5     return min(max(float(x), (0.0, upper)
6

```

اگر ورودی نامتناهی باشد، به کران بالای بازه بازمی‌گردد که در حالت خطا به معنای بدترین حالت (یعنی عدم امنیت) است.

۵.۳.۷ اعتبارسنجی TallyCounts

متد `_validate_tally_counts` یک لایه دفاعی برای صحت‌سنجی شمارش‌های آماری ورودی است:

```

1 def _validate_tally_counts(self, stats: TallyCounts, pulse_name: str) -> None:
2     required = ("sent", "sifted_x", "sifted_z", "errors_sifted_x", "errors_sifted_z")
3     missing = [name for name in required if not hasattr(stats, name)]
4     if missing:
5         raise ParameterValidationError(
6             f"TallyCounts for pulse {pulse_name!r} is missing required fields: {missing}."
7         )
8
9     sent = float(stats.sent)
10    sifted_x = float(stats.sifted_x)
11    sifted_z = float(stats.sifted_z)
12    err_x = float(stats.errors_sifted_x)
13    err_z = float(stats.errors_sifted_z)
14
15    values = {
16        "sent": sent,
17        "sifted_x": sifted_x,
18        "sifted_z": sifted_z,
19        "errors_sifted_x": err_x,
20        "errors_sifted_z": err_z,
21    }
22
23    for name, value in values.items():
24        if not math.isfinite(value) or value < :0.0
25        raise ParameterValidationError(
26            f"{name} for pulse {pulse_name!r} must be finite and non-negative."
27        )
28
29        if err_x > sifted_x:
30            raise ParameterValidationError(
31                f"errors_sifted_x exceeds sifted_x for pulse {pulse_name!r}: {err_x} > {sifted_x}."
32            )
33        if err_z > sifted_z:
34            raise ParameterValidationError(
35                f"errors_sifted_z exceeds sifted_z for pulse {pulse_name!r}: {err_z} > {sifted_z}."
36            )
37        if sifted_x + sifted_z > sent + 1e-9:
38            raise ParameterValidationError(
39                f"sifted_x + sifted_z exceeds sent for pulse {pulse_name!r}."
40            )
41

```

این متد چهار سطح اعتبارسنجی انجام می‌دهد: (۱) حضور تمام فیلدهای مورد نیاز `sent`، `sifted_x`، `sifted_z`، `errors_sifted_x` و `errors_sifted_z` در شیء `TallyCounts`؛ (۲) متناهی و نامنفی بودن تمام مقادیر؛ (۳) عدم تجاوز خطاها از شمارش متناظر ($m_X \leq n_X$ و $m_Z \leq n_Z$) که یک قید فیزیکی بدیهی است؛ (۴) عدم تجاوز مجموع `sifted` از `sent` با تلورانس 10^{-9} . این اعتبارسنجی‌ها قبل از هر محاسبه انجام می‌شوند تا از خطاهای دیرهنگام که ریشه‌یابی آن‌ها دشوار است، جلوگیری شود. در صورت شکست هر یک از این اعتبارسنجی‌ها، یک `ParameterValidationError` با پیام مشخص پرتاب می‌شود که نام پالس و فیلد مشکل‌دار را شامل می‌شود.

۴.۷ پیاده‌سازی ریاضی و الگوریتمی

۱.۴.۷ اعتبارسنجی پیکربندی

متد `_validate_configuration` تمام پیش‌نیازهای چارچوب Lim 2014 را بررسی می‌کند:

```

1 def _validate_configuration(self) -> None:
2     self._validate_epsilon_value(self.p.eps_sec, "eps_sec")
3     self._validate_epsilon_value(self.p.eps_cor, "eps_cor")
4
5     if not math.isfinite(float(self.p.f_error_correction)) or self.p.f_error_correction < :0.1
6     raise ParameterValidationError("f_error_correction must be finite and >= 1.")
7

```

```

۸ ec_model = getattr(self.p, "error_correction_model", "standard")
۹ if ec_model not in {"standard", "lim", "tomamichel"}:
۱۰     raise ParameterValidationError(
۱۱         "error_correction_model must be one of: 'standard', 'lim', 'tomamichel'."
۱۲     )
۱۳
۱۴ photon_cap = int(getattr(self.p, "photon_number_cap", 0))
۱۵ if photon_cap < 1:
۱۶     raise ParameterValidationError("photon_number_cap must be at least 1.")
۱۷
۱۸ pulse_configs = list(getattr(self.p.source, "pulse_configs", []))
۱۹ if len(pulse_configs) < 3:
۲۰     raise ConfigurationError("Lim2014Proof requires at least three pulse configurations.")
۲۱
۲۲ names = [pc.name for pc in pulse_configs]
۲۳ if len(set(names)) != len(names):
۲۴     raise ConfigurationError("Pulse configuration names must be unique.")
۲۵
۲۶ prob_sum = 0.0
۲۷ for pc in pulse_configs:
۲۸     mu = float(pc.mean_photon_number)
۲۹     prob = float(pc.probability)
۳۰
۳۱     if not math.isfinite(mu) or mu < :0.0
۳۲         raise ParameterValidationError(f"Invalid mean photon number for pulse {pc.name!r}: {mu!r}.")
۳۳     if not math.isfinite(prob) or prob <= :0.0
۳۴         raise ParameterValidationError(f"Pulse probability for {pc.name!r} must be finite and positive.")
۳۵     prob_sum += prob
۳۶
۳۷ if abs(prob_sum - (0.1 > PROB_SUM_TOL):
۳۸     raise ParameterValidationError(f"Pulse probabilities must sum to 1; got {prob_sum!r}.")
۳۹
۴۰ kappa = getattr(self.p, "security_constant_kappa", None)
۴۱ if kappa is not None:
۴۲     if not math.isfinite(float(kappa)) or float(kappa) <= :0.0
۴۳         raise ParameterValidationError("security_constant_kappa must be finite and positive when set.")
۴۴

```

این متد به ترتیب اعتبارسنجی می‌کند: (۱) مقادیر ε_{cor} و ε_{sec} در بازه $(0, 1)$ باشند؛ (۲) ضریب کارایی تصحیح خطا $f_{\text{EC}} \geq 1$ باشد (مقدار کمتر از ۱ از نظر تئوری رمزنگاری غیرممکن است زیرا تصحیح خطا نمی‌تواند بهتر از حد Shannon باشد)؛ (۳) مدل تصحیح خطا یکی از سه حالت مجاز `standard`، `lim` یا `tomamichel` باشد؛ (۴) کسر تعداد فوتون `photon_number_cap` حداقل ۱ باشد (برای مسیر LP ضروری است)؛ (۵) حداقل سه پیکربندی شدت نور وجود داشته باشد؛ (۶) نام پیکربندی‌ها یکتا باشند؛ (۷) مجموع احتمالات برابر ۱ باشد با تلوآنس `PROB_SUM_TOL`؛ (۸) در صورت تنظیم κ ، مقدار آن مثبت و متناهی باشد. این اعتبارسنجی‌ها قبل از هر محاسبه انجام می‌شوند.

۲.۴.۷ محاسبه توزیع Poisson و τ_n

تابع `_poisson_pmf_list` توزیع Poisson را تا حد n_{cap} به صورت بازگشتی و پایدار عددی محاسبه می‌کند:

```

۱ @staticmethod
۲ def _poisson_pmf_list(mu: float, n_cap: int) -> List[float]:
۳     if mu < 0.0 or not math.isfinite(mu):
۴         raise ParameterValidationError(f"Poisson mean must be finite and non-negative; got {mu!r}.")
۵     pmfs = [0.0] * (n_cap + 1)
۶     pmfs[0] = math.exp(-mu)
۷     for n in range(1, n_cap + 1):
۸         pmfs[n] = pmfs[n - 1] * mu / float(n)
۹     return pmfs
۱۰

```

این پیاده‌سازی از رابطه بازگشتی $P(n | \mu) = P(n - 1 | \mu) \cdot \mu / n$ استفاده می‌کند که از محاسبه مستقیم فاکتوریل $n!$ که برای n بزرگ سرریز می‌شود، اجتناب می‌کند. این روش پایداری عددی بسیار بالایی دارد و حتی برای $n = 100$ نیز نتایج دقیق تولید می‌کند. مقدار $P(0 | \mu) = e^{-\mu}$ به عنوان نقطه شروع بازگشت استفاده می‌شود.

تابع `_get_tau_n` مقدار τ_n را برای n مشخص محاسبه می‌کند:

```

1 def _get_tau_n(self, n: int, pulse_map: Mapping[str, float], p_k: Mapping[str, float]) -> float:
2     """
3     tau_n := sum_k p_k * e^{-mu_k} * mu_k^n / n!
4     (paper, definition below Eq. (2)).
5     """
6     if n < 0:
7         raise ParameterValidationError("n must be non-negative.")
8
9     tau_n = 0.0
10    for name, mu in pulse_map.items():
11        prob_k = p_k[name]
12        pmf = math.exp(-mu) if n == 0 else math.exp(-mu) * (mu**n) / math.factorial(n)
13        tau_n += prob_k * pmf
14    return max(0.0, tau_n)
15

```

این تابع دقیقاً تعریف $\tau_n = \sum_k p_k P(n | \mu_k)$ را پیاده‌سازی می‌کند. در حالت $n = 0$ برای جلوگیری از محاسبه 0^0 ، از شاخه خاص $\text{math.exp}(-\mu)$ استفاده می‌شود. تابع `_get_tau_list` همان محاسبه را به صورت برداری برای همه n از ۰ تا n_{cap} انجام می‌دهد که در مسیر LP استفاده می‌شود. تابع `_get_tail_probability` احتمال دم را محاسبه می‌کند، یعنی $1 - \sum_{n=0}^{n_{\text{cap}}} P(n | \mu)$ که در `tail relaxation` مسیر LP به کار می‌رود. این تابع برای مدیریت فوتون‌های چندتایی فراتر از کسر n_{cap} ضروری است.

۳.۴.۷ کران‌های هیبرید Hoeffding/Chernoff با دو پارامتر

متد `_get_bounds` قلب استراتژی هیبرید است و کران‌های بازه اطمینان را در سه رژیم پیاده‌سازی می‌کند:

```

1 def _get_bounds(self, n_k: float, n_total: float, eps_ln_term: float,
2 clip_total: Optional[float] = None) -> Tuple[float, float]:
3     """
4     Hybrid Hoeffding / per-intensity Chernoff confidence interval.
5
6     Paper Lim2014 Appendix A uses Hoeffding's inequality with a single
7     delta = sqrt(n_total/2 * ln(21/eps_sec)) that is the SAME for all
8     intensities k. This is a sufficient condition for security but is
9     pathologically loose when n_k << n_total (e.g. vacuum counts at long
10    distance: observed 5, Hoeffding upper 572, 114x the observation).
11
12    Hybrid strategy (security-preserving, tighter for small counts):
13    - When n_k is "large" (n_k >= 10 * delta_paper, i.e. relative
14    statistical error < 10%), use the paper's Hoeffding bound.
15    - When n_k is "small" but nonzero (0 < n_k < 10 * delta_paper),
16    use per-intensity multiplicative Chernoff:
17    delta_k = sqrt(2 * n_k * ln(1/eps_per_event))
18    - When n_k = 0, the upper bound is ln(1/eps_per_event) (rule of
19    three for Poisson/binomial zero observations).
20
21    # HYBRID-HOEFFDING-CHERNOFF v1
22    """
23    if clip_total is None:
24        clip_total = n_total
25
26    if n_total < :0.0:
27        raise ParameterValidationError("n_total must be non-negative.")
28    if clip_total < :0.0:
29        raise ParameterValidationError("clip_total must be non-negative.")
30    if eps_ln_term < 0.0 or not math.isfinite(eps_ln_term):
31        raise ParameterValidationError("eps_ln_term must be finite and non-negative.")
32    if n_k < :0.0:
33        raise ParameterValidationError("n_k must be non-negative.")
34
35    # --- Hoeffding delta (paper Eq. A1): same for all k, scales with n_total.
36    delta_paper = math.sqrt((n_total / (0.2 * eps_ln_term))
37
38    # --- Hybrid selection ---
39    hoeffding_threshold = HOEFFDING_FACTOR * delta_paper
40
41    if n_k >= hoeffding_threshold and n_k > :0.0:
42        # Paper-faithful Hoeffding regime (large counts).
43        lower = max(0.0, n_k - delta_paper)
44

```



```

۴۴ upper = min(n_k + delta_paper, clip_total)
۴۵ elif n_k > :0.0
۴۶ # Per-intensity multiplicative Chernoff regime (small but nonzero).
۴۷ delta_k = math.sqrt(0.2) * n_k * eps_ln_term
۴۸ lower = max(0.0), n_k - delta_k
۴۹ upper = min(n_k + delta_k, clip_total)
۵۰ else:
۵۱ # n_k = 0: multiplicative Chernoff upper bound (rule of three).
۵۲ lower = 0.0
۵۳ upper = min(eps_ln_term, clip_total)
۵۴
۵۵ return lower, upper
۵۶

```

پارامتر n_{total} مقیاس محاسبه δ_{paper} را تعیین می‌کند و clip_total کران بالای فیزیکی برای clip کردن است. در پیاده‌سازی استاندارد مقاله، هر دو برابر n_X (یا n_Z) هستند؛ اما در این پیاده‌سازی، n_{total} می‌تواند برابر $n_{X,k}$ (شمارش خود شدت) باشد تا δ کوچکتر و تنگ‌تر شود. این امر از یک مشکل عملی جلوگیری می‌کند: وقتی شمارش decoy یا خلأ بسیار کم است، ضریب پیشانی e^{μ_k}/p_k می‌تواند δ مبتنی بر n_X را به قدری بزرگ کند که کران‌های پایین منفی و کران‌های بالا بیش از کل فیزیکی شوند.

نکته کلیدی در این پیاده‌سازی، استفاده از eps_ln_term به‌عنوان آرگومان ورودی است. این متغیر برابر $\ln(21/\varepsilon_{\text{sec}}) = \ln(1/\varepsilon_{\text{per-event}})$ است (زیرا $\varepsilon_{\text{per-event}} = \varepsilon_{\text{sec}}/21$). این یکپارچه‌سازی هوشمندانه امکان استفاده از همان جمله لگاریتمی در هر دو رژیم Hoeffding و Chernoff را فراهم می‌کند بدون آنکه نیاز به محاسبه جداگانه باشد. در رژیم صفر، کران بالای $\ln(1/\varepsilon_{\text{per-event}})$ مستقیماً برابر eps_ln_term است که یک کران فیزیکی متناهی و معقول ارائه می‌دهد. در نتیجه، این متد هیچ‌گاه یک کران بالای نامتناهی یا غیرفیزیکی تولید نمی‌کند.

۴.۴.۷ مسیر N-decoy مبتنی بر LP

وقتی تعداد شدت‌ها بیش از ۳ باشد، پیاده‌سازی به مسیر LP تغییر مسیر می‌دهد. متد `_solve_n_decoy_lp` یک برنامه‌ریزی خطی محافظه‌کارانه با محدودیت‌های غیرمنفی‌بودن، کران بالای صریح و `tail relaxation` را حل می‌کند. این متد سه نوع هدف را پشتیبانی می‌کند: $s0_lower$ (کران پایین بازده خلأ)، $s1_lower$ (کران پایین بازده تک‌فوتونی) و $v1_upper$ (کران بالای خطای تک‌فوتونی). در حالت $v1_upper$ ، تابع هدف به $-v1$ تبدیل می‌شود زیرا LP استاندارد مسأله کمینه‌سازی را حل می‌کند و برای یافتن کران بالا باید منفی متغیر را کمینه کرد. این مسیر با فراخوانی `solve_lp` از ماژول `qkd.proofs.utils_lp` که به `scipy.optimize.linprog` یا `cvxopt` متصل است، حل می‌شود. در صورت شکست LP (مثلاً به دلیل عدم `feasibility`)، یک `fallback` محافظه‌کارانه اعمال می‌شود: کران‌های پایین به صفر و کران‌های بالا به حد فیزیکی بازمی‌گردند. این رفتار `fail-safe` تضمین می‌کند که حتی در صورت شکست `solver`، خروجی هرگز خوش‌بینانه نخواهد بود. توجه شود که این مسیر در گزارش `diagnostics` به‌روشنی به‌عنوان توسعه فراتر از نتایج بسته مقاله علامت‌گذاری شده است.

۵.۴.۷ پیاده‌سازی معادله (۲): کران پایین بازده خلأ

تابع `_calc_s0_lower` معادله (۲) مقاله را پیاده‌سازی می‌کند:

```

۱ def _calc_s0_lower(
۲     self,
۳     tau_0: float,
۴     mu2: float,
۵     mu3: float,
۶     n_mu2_plus: float,
۷     n_mu3_minus: float,
۸ ) -> float:
۹     """
۱۰     Eq. (2):
۱۱     s_{X,0} >= tau_0 * (mu2 * n_{X,mu3} - mu3 * n_{X,mu2}) / (mu2 - mu3)
۱۲     """
۱۳     denominator = mu2 - mu3
۱۴     if denominator <= :0.0
۱۵         raise ConfigurationError("Invalid s0 denominator; require mu2 > mu3.")
۱۶
۱۷     numerator = tau_0 * (mu2 * n_mu3_minus - mu3 * n_mu2_plus)

```



```

۱۸     return max(0.0), numerator / denominator)
۱۹

```

پارامترهای ورودی به ترتیب $\tau_0, \mu_2, \mu_3, n_{X,\mu_2}^+$ (کران بالای آشکارسازی decoy) و n_{X,μ_3}^- (کران پایین آشکارسازی خلأ) هستند. توجه شود که در صورت کسر، μ_2 با کران پایین خلأ و μ_3 با کران بالای decoy ضرب می‌شوند تا صورت کسر به کوچکترین مقدار ممکن برسد و در نتیجه کران پایین محافظه‌کارانه به دست آید. در صورت منفی شدن صورت (که ممکن است در شرایط نوسان شدید آماری رخ دهد)، خروجی با $\max(0.0, \dots)$ به صفر برده می‌شود زیرا تعداد رویدادهای خلأ نمی‌تواند منفی باشد. نکته مهم این است که در نسخه فعلی با استراتژی هیبرید، n_{X,μ_3}^- دیگر هرگز به مقدار منفی Hoeffding منجر نمی‌شود، زیرا در رژیم $n_k = 0$ ، کران پایین صفر و کران بالا $\ln(1/\varepsilon)$ است.

۶.۴.۷ پیاده‌سازی معادله (۳): کران پایین بازده تک‌فوتونی

تابع `_calc_s1_lower` معادله (۳) مقاله را پیاده‌سازی می‌کند:

```

۱  def _calc_s1_lower(
۲      self,
۳      tau_1: float,
۴      mu1: float,
۵      mu2: float,
۶      mu3: float,
۷      n_mu1_plus: float,
۸      n_mu2_minus: float,
۹      n_mu3_plus: float,
۱0     s_0_lower_over_tau_0: float,
۱۱ ) -> float:
۱۲     """
۱۳     Eq. (3):
۱۴     s_{X,1} >= tau_1 * mu1 * T / D
۱۵     where
۱۶     T = n^-_{X,mu2} - n^+_{X,mu3} - ((mu2^2 - mu3^2)/mu1^2) * (n^+_{X,mu1} - s_{X,0}/tau_0)
۱۷     D = mu1*(mu2 - mu3) - mu2^2 + mu3^2
۱۸     """
۱۹     mu1_sq = mu1 * mu1
۲۰     mu2_sq = mu2 * mu2
۲۱     mu3_sq = mu3 * mu3
۲۲
۲۳     denominator = mu1 * (mu2 - mu3) - mu2_sq + mu3_sq
۲۴     if denominator <= :0.0
۲۵         raise ConfigurationError("Invalid s1 denominator; source intensities do not satisfy formula
۲۶             conditions.")
۲۷
۲۸     term = n_mu2_minus - n_mu3_plus - ((mu2_sq - mu3_sq) / mu1_sq) * (
۲۹         n_mu1_plus - s_0_lower_over_tau_0
۳۰     )
۳۱     numerator = tau_1 * mu1 * term
۳۲     return max(0.0), numerator / denominator)
۳۳

```

این تابع پیچیده‌ترین فرمول تحلیلی چارچوب است. مخرج $D = \mu_1(\mu_2 - \mu_3) - \mu_2^2 + \mu_3^2$ با شرط $\mu_1 > \mu_2 + \mu_3$ مثبت می‌ماند که در تابع `_validate_three_intensity_order` بررسی می‌شود. در صورت کسر، سه جمله وجود دارد: n_{X,μ_2}^- با کران پایین decoy، n_{X,μ_3}^+ با کران بالای خلأ، و یک جمله تصحیحی که شامل n_{X,μ_1}^+ (کران بالای سیگنال) و نسبت $s_{X,0}/\tau_0$ (تخمین بازده خلأ) است. این جمله تصحیحی اثر فوتون‌های چندتایی را حذف می‌کند. در صورت منفی شدن صورت، خروجی به صفر برده می‌شود.

۷.۴.۷ پیاده‌سازی معادله (۴): کران بالای خطای تک‌فوتونی

تابع `_calc_v1_upper` معادله (۴) مقاله را پیاده‌سازی می‌کند:

```

۱  def _calc_v1_upper(
۲      self,
۳      tau_1: float,
۴      mu2: float,
۵      mu3: float,
۶      m_mu2_plus: float,

```

```

۷     m_mu3_minus: float,
۸     physical_upper: float,
۹     ) -> float:
۱۰     """
۱۱     Eq. (4):
۱۲      $v_{\{Z,1\}} \leq \tau_{u,1} * (m_{\{Z,\mu_2\}}^+ - m_{\{Z,\mu_3\}}^-) / (\mu_2 - \mu_3)$ 
۱۳     """
۱۴     denominator = mu2 - mu3
۱۵     if denominator <= :0.0
۱۶     raise ConfigurationError("Invalid v1 denominator; require mu2 > mu3.")
۱۷
۱۸     numerator = tau_1 * (m_mu2_plus - m_mu3_minus)
۱۹     return min(max0.0), numerator / denominator, max0.0), physical_upper))
۲۰

```

این تابع کران بالای تعداد خطاهای تکفوتونی در پایه Z را محاسبه می‌کند. در اینجا از کران بالای خطاهای decoy ضعیف (m_{Z,μ_2}^+) و کران پایین خطاهای خلأ (m_{Z,μ_3}^-) استفاده می‌شود تا کران بالا به دست آید. خروجی به یک کران فیزیکی (مثلاً n_Z یا m_Z) محدود می‌شود زیرا تعداد خطاها نمی‌تواند از تعداد آشکارسازی‌ها بیشتر باشد. این clip فیزیکی اطمینان می‌دهد که در شرایط نوسان شدید، کران بالای غیرمنطقی به دست نیاید.

۸.۴.۷ پیاده‌سازی معادله (۵): کران بالای خطای فاز

تابع `_calc_phi_X_upper` پیچیده‌ترین بخش چارچوب را پیاده‌سازی می‌کند:

```

۱     def _calc_phi_X_upper(
۲         self,
۳         v_Z_1_upper: float,
۴         s_Z_1_lower: float,
۵         s_X_1_lower: float,
۶         eps_a: float,
۷         ) -> float:
۸         """
۹         Eq. (5):
۱۰          $\phi_{i,X} = c_{\{X,1\}} / s_{\{X,1\}} \leq v_{\{Z,1\}} / s_{\{Z,1\}} + \gamma(a, b, c, d)$ 
۱۱         """
۱۲         self._validate_epsilon_value(eps_a, "eps_phase_est")
۱۳
۱۴         if s_Z_1_lower <= 0.0 or s_X_1_lower <= :0.0
۱۵         return MAX_ERROR_RATE
۱۶
۱۷         e_Z_1 = self._safe_divide(v_Z_1_upper, s_Z_1_lower, default=MAX_ERROR_RATE)
۱۸         e_Z_1 = self._clamp_probability(e_Z_1, upper=MAX_ERROR_RATE)
۱۹
۲۰         if e_Z_1 >= MAX_ERROR_RATE:
۲۱         return MAX_ERROR_RATE
۲۲
۲۳         c = float(s_Z_1_lower)
۲۴         d = float(s_X_1_lower)
۲۵
۲۶         if c <= 0.0 or d <= :0.0
۲۷         return MAX_ERROR_RATE
۲۸
۲۹         b = min(max(e_Z_1, MIN_EPS), MAX_ERROR_RATE - 1e-15)
۳۰         a = max(float(eps_a), MIN_EPS)
۳۱
۳۲         denominator = c * d * 0.1 - b) * b * (a * a)
۳۳         if denominator <= 0.0 or not math.isfinite(denominator):
۳۴         return MAX_ERROR_RATE
۳۵
۳۶         log_arg = (c + d) / denominator
۳۷         if log_arg <= 0.1 or not math.isfinite(log_arg):
۳۸         gamma = 0.0
۳۹         else:
۴۰         term1 = ((c + d) * 0.1 - b) * b / (c * d * math.log(0.2))
۴۱         term2 = self._safe_log2(log_arg)
۴۲         product = term1 * term2
۴۳         gamma = math.sqrt(product) if product > 0.0 and math.isfinite(product) else MAX_ERROR_RATE
۴۴
۴۵         return min(max0.0), e_Z_1 + gamma), MAX_ERROR_RATE)
۴۶

```

این تابع شامل چندین لایه دفاعی برای مدیریت حالت‌های تباهی است. در ابتدا، ε_a (یعنی $\varepsilon_{\text{sec}}/21$) اعتبارسنجی می‌شود. سپس اگر $s_{Z,1}$ یا $s_{X,1}$ صفر یا منفی باشند، تابع به صورت ایمن به 0.5 بازمی‌گردد. نسبت $e_{Z,1} = v_{Z,1}/s_{Z,1}$ با `_safe_divide` محاسبه می‌شود تا از تقسیم بر صفر جلوگیری شود و سپس به $[0, 0.5]$ clamp می‌شود. اگر $e_{Z,1}$ به 0.5 برسد، تابع بلافاصله 0.5 برمی‌گرداند زیرا در این حالت هیچ امنیتی قابل استخراج نیست.

نکته ظریف این تابع، مدیریت حالت‌های تباهی $b \in \{0, 1\}$ است که در آن‌ها فرمول γ تکین می‌شود. در این پیاده‌سازی، b با $\min(\max(e_{Z,1}, \text{MIN_EPS}), \text{MAX_ERROR_RATE} - 1e-15)$ به بازه $[10^{-300}, 0.5]$ محدود می‌شود تا از تکینگی جلوگیری شود. این clamp بسیار کوچک است و در عمل نتیجه نهایی را تغییر نمی‌دهد، اما از خطاهای عددی مانند `ZeroDivisionError` یا `ValueError: math domain error` جلوگیری می‌کند. اگر مخرج γ صفر یا نامتناهی شود، تابع به 0.5 بازمی‌گردد که با رویکرد بدترین حالت مقاله همخوان است.

۹.۴.۷ محاسبه نشت تصحیح خطا

تابع `_calc_error_correction_leakage` دو مدل نشت را پشتیبانی می‌کند:

```

1 def _calc_error_correction_leakage(self, n_x: float, qber_x: float, eps_cor: float) -> Tuple[float
2     , float]:
3     """
4     Returns (leak_EC, separate_correctness_term).
5
6     Models:
7     - 'standard' / 'lim': paper Eq. (1).
8     leak_EC      = f_EC * n_X * h(e_obs)
9     corr_term    = log2(2 / eps_cor)
10    - 'tomamichel': Ref. [37] of the paper. Includes the correctness
11    logarithmic overhead in the leakage term.
12    """
13    self._validate_epsilon_value(eps_cor, "eps_cor")
14
15    qber_x = self._clamp_probability(qber_x, upper=MAX_ERROR_RATE)
16    h_e = self.binary_entropy(qber_x)
17    asymptotic_leak = float(self.p.f_error_correction * n_x * h_e)
18    corr_term = self._safe_log20.2) / eps_cor)
19
20    model = getattr(self.p, "error_correction_model", "standard")
21    if model == "tomamichel":
22        return asymptotic_leak + corr_term, 0.0
23
24    return asymptotic_leak, corr_term

```

در مدل استاندارد (که با مدل `lim` نیز برابر است)، نشت تصحیح خطا $\lambda_{\text{EC}} = f_{\text{EC}} \cdot n_X \cdot h(e_{\text{obs}})$ است و جمله صحت $\log_2(2/\varepsilon_{\text{cor}})$ به صورت جداگانه در معادله (۱) کم می‌شود. در مدل `tomamichel`، جمله صحت به λ_{EC} اضافه می‌شود و جمله جداگانه صفر می‌گردد تا از شمارش مضاعف جلوگیری شود. این طراحی به کاربر اجازه می‌دهد مدل دقیق‌تر `Tomamichel` را انتخاب کند، در حالی که پیش‌فرض بر مدل استاندارد مقاله است.

۱۰.۴.۷ حل تکراری $\varepsilon_{\text{sec}} = \kappa \cdot \ell$

در بخش §IV مقاله، روشی پیشنهاد می‌شود که در آن ε_{sec} به طول کلید متناسب می‌شود: $\varepsilon_{\text{sec}} = \kappa \cdot \ell$. این یک معادله ضمنی است زیرا ℓ خود به ε_{sec} وابسته است. تابع `_iterative_kappa_solver` این معادله را با تکرار حل می‌کند:

```

1 def _iterative_kappa_solver(
2     self,
3     stats_map: Mapping[Union[str, int], TallyCounts],
4     ) -> KeyCalculationResult:
5     """
6     Implements the paper's eps_sec = kappa * l convention (paper §IV).
7     """
8     kappa = float(self.p.security_constant_kappa)

```

```

۹     if not math.isfinite(kappa) or kappa <= :0.0
۱۰         raise ParameterValidationError("security_constant_kappa must be finite and positive.")
۱۱
۱۲     n_total = sum(float(s.sent) for s in stats_map.values())
۱۳     l_est = max(0.1), min(n_total * 0.1, n_total)
۱۴
۱۵     last_result: KeyCalculationResult | None = None
۱۶     converged = False
۱۷
۱۸     for iteration in range(DEFAULT_KAPPA_MAX_ITERATIONS):
۱۹         eps_target = kappa * max(l_est, (0.1
۲۰             # floor at 1e-10: smaller eps explodes Chernoff bounds (ln(eps/21) > 26)
۲۱             eps_target = min(max(eps_target, 1e-10), MAX_SECURITY_EPS)
۲۲
۲۳         result = self._calculate_key_length_core(stats_map, eps_target)
۲۴         last_result = result
۲۵         l_new = float(result.secure_key_length)
۲۶
۲۷         abs_delta = abs(l_new - l_est)
۲۸         rel_delta = abs_delta / max(0.1), abs(l_est))
۲۹
۳۰         if abs_delta <= DEFAULT_KAPPA_ABS_TOL_BITS or rel_delta <= DEFAULT_KAPPA_REL_TOL:
۳۱             converged = True
۳۲             result.diagnostics.setdefault("dynamic_epsilon", {})
۳۳             result.diagnostics["dynamic_epsilon"].update(
۳۴                 {
۳۵                     "converged": True,
۳۶                     "iterations": iteration + 1,
۳۷                     "kappa": kappa,
۳۸                     "eps_sec_final": eps_target,
۳۹                 }
۴۰             )
۴۱             return result
۴۲
۴۳         if l_new <= :0.0
۴۴             l_est = max(0.1), 5.0 * l_est)
۴۵         else:
۴۶             l_est = 5.0 * (l_est + l_new)
۴۷
۴۸         assert last_result is not None
۴۹         last_result.diagnostics.setdefault("dynamic_epsilon", {})
۵۰         last_result.diagnostics["dynamic_epsilon"].update(
۵۱             {
۵۲                 "converged": converged,
۵۳                 "iterations": DEFAULT_KAPPA_MAX_ITERATIONS,
۵۴                 "kappa": kappa,
۵۵                 "warning": "dynamic epsilon iteration did not converge",
۵۶             }
۵۷         )
۵۸         last_result.diagnostics.setdefault("warnings", []).append("Dynamic epsilon iteration did not
            converge.")
۵۹     return last_result
۶۰

```

الگوریتم با یک تخمین اولیه $l_{\text{est}} = \max(1, \min(0.01 \cdot n_{\text{total}}, n_{\text{total}}))$ از کل پالس‌ها را به عنوان طول کلید تقریبی در نظر می‌گیرد. در هر تکرار، $\varepsilon_{\text{target}} = \kappa \cdot \max(l_{\text{est}}, 1)$ محاسبه می‌شود. یک نکته حیاتی در این نسخه این است که floor $\varepsilon_{\text{target}}$ از 10^{-15} به 10^{-10} افزایش یافته است. این تغییر ضروری است زیرا با معرفی استراتژی هیبرید Hoeffding/Chernoff، کران‌های Chernoff مستقیماً به $\ln(1/\varepsilon)$ وابسته هستند و برای $\varepsilon < 10^{-10}$ ، مقدار $\ln(21/\varepsilon) > 26$ می‌شود که باعث می‌شود کران‌های Chernoff به‌طور پاتولوژیک شل شوند (کران بالا بیش از حد بزرگ، کران پایین منفی). با کف 10^{-10} ، حداکثر مقدار $\ln(21/\varepsilon) \approx 25.8$ است که در محدوده قابل قبول عددی باقی می‌ماند.

معیار همگرایی ترکیبی از تلورانس مطلق (۱۰۰ بیت) و تلورانس نسبی (10^{-6}) است. در صورت عدم همگرایی، تخمین به صورت میانگین حسابی بین تخمین قبلی و مقدار جدید به‌روزرسانی می‌شود. اگر $l_{\text{new}} \leq 0$ (یعنی کلیدی تولید نشده)، تخمین به نصف کاهش می‌یابد تا ε سخت‌گیرانه‌تر شود. در صورت عدم همگرایی پس از ۵۰ تکرار، آخرین نتیجه با هشدار بازگردانده می‌شود. این طراحی fail-soft تضمین می‌کند که حتی در صورت عدم همگرایی، یک خروجی معتبر (با احتمالاً طول کلید صفر) تولید می‌شود و خط لوله شبیه‌سازی متوقف نمی‌شود.

۱۱.۴.۷ نرمال سازی stats_map و نتایج صفر

متد `_normalize_stats_map` یک لایه انعطاف پذیر برای تطبیق کلیدهای `stats_map` با نام پالس ها فراهم می کند:

```

1 def _normalize_stats_map(
2     self,
3     stats_map: Mapping[Union[str, int], TallyCounts],
4     pulse_names: List[str],
5 ) -> Dict[str, TallyCounts]:
6     if all(name in stats_map for name in pulse_names):
7         return {name: stats_map[name] for name in pulse_names}
8
9     if all(str(i) in stats_map for i in range(len(pulse_names))):
10        return {pulse_names[i]: stats_map[str(i)] for i in range(len(pulse_names))}
11
12    if all(i in stats_map for i in range(len(pulse_names))):
13        return {pulse_names[i]: stats_map[i] for i in range(len(pulse_names))}
14
15    raise ParameterValidationError(
16        f"stats_map keys do not match pulse names or positional indices. "
17        f"Expected names {pulse_names!r}; got {list(stats_map.keys())!r}."
18    )
19

```

این متد سه فرمت کلید را پشتیبانی می کند: (۱) کلیدهای نام پالس (مانند "signal"، "decoy"، "vacuum") که صریح ترین حالت است؛ (۲) کلیدهای اندیس رشته ای (مانند "0"، "1"، "2") که معمولاً در خروجی CSV/JSON استفاده می شود؛ (۳) کلیدهای اندیس صحیح (مانند 0، 1، 2) که در تعاملات درون حافظه ای رایج است. این انعطاف پذیری به ماژول اجازه می دهد با انواع فرمت های ورودی از لایه های مختلف شبیه سازی کار کند بدون آنکه نیاز به تبدیل صریح باشد. در صورت عدم تطابق با هیچ یک از این سه فرمت، یک `ParameterValidationError` با پیام مشخص پرتاب می شود.

متد `_zero_result` یک ابزار کمکی برای تولید نتایج صفر با اطلاعات تشخیصی غنی است:

```

1 def _zero_result(
2     self,
3     branch: str,
4     diagnostics: Dict[str, Any],
5     error_codes: List[ErrorCode] | None = None,
6 ) -> KeyCalculationResult:
7     diagnostics = dict(diagnostics)
8     diagnostics["return_branch"] = branch
9     return KeyCalculationResult(
10        secure_key_length=0,
11        privacy_amplification_term=0.0,
12        error_correction_leakage=0.0,
13        phase_error_rate_upper_bound=MAX_ERROR_RATE,
14        error_codes=error_codes or [ErrorCode.INSUFFICIENT_STATISTICS],
15        diagnostics=_json_safe_dict(diagnostics.items()),
16    )
17

```

این متد در تمام مسیرهای شکست استفاده می شود: آمار ناکافی `(invalid_or_mismatched_stats)`، `stats_map` نامطابق `(insufficient_statistics_nX_or_nZ)`، `(invalid_tau)` نامعتبر τ ، و پیکربندی سه شدت نامعتبر `(invalid_three_intensity_configuration)`. هر شاخه در `diagnostics` به صورت `return_branch` ثبت می شود تا ریشه یابی شکست ساده شود. پیش فرض کد خطا `INSUFFICIENT_STATISTICS` است، اما می توان کدهای دیگری نیز ارسال کرد. استفاده از `_json_safe_dict` اطمینان می دهد که `diagnostics` قابل سریال سازی به JSON است.

۱۲.۴.۷ محاسبه اصلی طول کلید

تابع `_calculate_key_length_core` هسته اصلی محاسبه است. پس از اعتبارسنجی و آماده سازی، شمارش های مشاهده شده را به کران های Hoeffding/Chernoff تبدیل می کند، تخمین های $s_{Z,1}^L$ ، $s_{X,1}^L$ ، $s_{X,0}^L$ و φ_X^U و $v_{Z,1}^U$ را محاسبه می کند و در نهایت معادله (۱) را ارزیابی می کند. این متد با آماده سازی `diagnostics` کامل آغاز می شود:

```

1 diagnostics: Dict[str, Any] = {
2     "epsilon_accounting": {
3         "scheme": "Lim2014 Appendix B (21-event split, eps_sec = 21*eps)",
4         "eps_sec_used": float(eps_sec_override),
5         "single_test_epsilon": float(epsilon),
6         "failure_events": LIM2014_FAILURE_EVENTS_3_INTENSITY,
7         "hoeffding_ln_term": float(eps_ln_term),
8         "bound_method": "hoeffding_chernoff_hybrid",
9         "note": (
10             "Uses a hybrid Hoeffding / per-intensity Chernoff bound. "
11             "For large counts (n_k >= 10 * delta_paper, relative "
12             "statistical error <= 10%), uses the paper-faithful "
13             "Hoeffding delta = sqrt(n_total/2 * ln(21/eps_sec)). "
14             "For small counts (0 < n_k < threshold), uses the tighter "
15             "per-intensity Chernoff delta_k = sqrt(2 * n_k * ln(1/eps)). "
16             "For n_k = 0, uses the multiplicative Chernoff upper bound "
17             "ln(1/eps_per_event) (rule of three). This is provably "
18             "<= Hoeffding for all n_k, so security is never weakened."
19         ),
20     },
21     "mu_map": {k: float(v) for k, v in mu_map.items()},
22     "p_k_map": {k: float(v) for k, v in p_k_map.items()},
23     "warnings": [],
24 }
25

```

این ساختار diagnostics غنی به کاربر اجازه می‌دهد تمام جزئیات محاسبه را بازبینی کند. فیلد bound_method با مقدار "hoeffding_chernoff_hybrid" به‌طور صریح نشان می‌دهد که از استراتژی هیبرید استفاده شده است. فیلد note یک توضیح کامل از سه رژیم ارائه می‌دهد که برای امنیت و بازبینی توسط شخص ثالث ضروری است. سپس شمارش‌های n_X , n_Z , m_X , m_Z و تفکیک آن‌ها به‌ازای هر شدت در diagnostics ذخیره می‌شوند.

در مسیر تحلیلی سه‌شدت، کران‌های Hoeffding/Chernoff برای هر شدت در هر دو پایه محاسبه و در diagnostics ذخیره می‌شوند:

```

1 n_X_k_bounds = {name: self._get_bounds(n_X_k[name], n_X, eps_ln_term, clip_total=n_X) for name in
2   mu_map}
3 n_Z_k_bounds = {name: self._get_bounds(n_Z_k[name], n_Z, eps_ln_term, clip_total=n_Z) for name in
4   mu_map}
5 m_Z_k_bounds = {name: self._get_bounds(m_Z_k[name], m_Z, eps_ln_term, clip_total=m_Z) for name in
6   mu_map}
7
8 diagnostics["n_X_k_bounds"] = {
9     k: [float(v[0]), float(v[1])] for k, v in n_X_k_bounds.items()
10 }
11

```

سپس ضریب پیشنهادی e^{μ_k}/p_k برای تبدیل کران‌های خام به فرم استفاده‌شده در معادلات (۲)-(۴) اعمال می‌شود. به‌عنوان مثال، برای کران بالای بازده خلأ:

```

1 s_X_0_L = self._calc_s0_lower(
2     tau_0,
3     mu2,
4     mu3,
5     n_X_k_bounds[k2][1] / p2 * math.exp(mu2), # decoy UPPER bound (n^+_{X,mu2})
6     n_X_k_bounds[k3][0] / p3 * math.exp(mu3), # vacuum LOWER bound (n^-_{X,mu3})
7 )
8

```

نکته مهم در این نسخه این است که کران بالای خلأ (n_{X,μ_3}^+) اکنون به‌درستی توسط `_get_bounds` به‌صورت $\ln(1/\varepsilon)$ محاسبه می‌شود وقتی مشاهده صفر است (کران تنگ Chernoff، نه δ_{paper} مبتنی بر n_X). این یک patch ضد محافظه‌کارانه نیست، بلکه یک اصلاح درست است: در نسخه‌های قبلی، وقتی $n_{X,\mu_3} = 0$ ، کران بالای Hoeffding به δ_{paper} می‌رسید که می‌تواند بسیار بزرگتر از حد معقول باشد. با استراتژی هیبرید، این کران به $\ln(1/\varepsilon)$ محدود می‌شود که هم متناهی و هم تنگ‌تر است. در نتیجه، تخمین $s_{X,0}^L$ در فواصل طولانی (جایی که شمارش خلأ اغلب صفر است) به‌طور قابل توجهی بهبود می‌یابد و طول کلید نهایی بالاتر می‌شود بدون آنکه امنیت تضعیف شود.

پس از محاسبه φ_X^U و λ_{EC} ، معادله (۱) به صورت مستقیم ارزیابی می‌شود:

```

۱ phi_X_U = self._calc_phi_X_upper(v_Z_1_U, s_Z_1_L, s_X_1_L, epsilon)
۲
۳ qber_x = self._safe_divide(m_X, n_X, default=MAX_ERROR_RATE)
۴ qber_x = self._clamp_probability(qber_x, upper=MAX_ERROR_RATE)
۵
۶ leak_EC, corr_term = self._calc_error_correction_leakage(n_X, qber_x, self.p.eps_cor)
۷
۸ # Eq. (1):
۹ # l = s_{X,0} + s_{X,1} * (1 - h(phi_X))
۱۰ # - lambda_EC - 6*log2(21/eps_sec) - log2(2/eps_cor)
۱۱ term_s0 = s_X_0_L
۱۲ term_s1 = s_X_1_L * 0.1 - self.binary_entropy(phi_X_U)
۱۳
۱۴ pa_term = 0.6 * self._safe_log2(LIM2014_FAILURE_EVENTS_3_INTENSITY / eps_sec_override)
۱۵
۱۶ key_len = term_s0 + term_s1 - leak_EC - pa_term - corr_term
۱۷ key_len_clamped = self._clamp_key_length(float(key_len))
۱۸

```

جمله $6 \log_2(21/\epsilon_{\text{sec}})$ که در مقاله به عنوان جمله privacy amplification شناخته می‌شود، شش رویداد شکست مرتبط با تقویت حریم خصوصی را پوشش می‌دهد. در نهایت طول کلید با `_clamp_key_length` به عدد صحیح نامنفی تبدیل می‌شود. اگر طول خام منفی باشد (یعنی نشت‌ها از آنتروپی بیشتر شوند)، هشدار در diagnostics ثبت می‌شود. کدهای خطای مناسب نیز به نتیجه اضافه می‌شوند تا علت شکست قابل ردیابی باشد.

۱۳.۴.۷ قید دفاعی فیزیکی در `calculate_key_length`

متد `calculate_key_length` یک لایه دفاعی فیزیکی برای قید طول کلید به کلید sifted کل اعمال می‌کند:

```

۱ def calculate_key_length(
۲     self,
۳     stats_map: Mapping[Union[str, int], TallyCounts],
۴ ) -> KeyCalculationResult:
۵     self._validate_configuration()
۶
۷     if getattr(self.p, "security_constant_kappa", None) is not None:
۸         result = self._iterative_kappa_solver(stats_map)
۹     else:
۱۰        result = self._calculate_key_length_core(stats_map, self.p.eps_sec)
۱۱
۱۲        # === Defensive physical clamp ===
۱۳        # The secure key length CANNOT exceed the total number of basis-matched
۱۴        # raw detections (n_X + n_Z = total sifted bits).
۱۵        try:
۱۶            _n_total = 0
۱۷            for _k, _v in stats_map.items():
۱۸                if hasattr(_v, 'sifted'):
۱۹                    _n_total += int(_v.sifted)
۲۰            if _n_total > 0 and result.secure_key_length > _n_total:
۲۱                import logging as _logging
۲۲                _logging.getLogger('Lim2014Proof').warning(
۲۳                    f'Defensive clamp triggered: secure_key_length {result.secure_key_length} '
۲۴                    f'> total sifted {_n_total}; clamping to {_n_total}. '
۲۵                    f'This indicates an internal bug in the kappa solver or decoy estimation.'
۲۶                )
۲۷                from qkd.datatypes import KeyCalculationResult as _KCR
۲۸                result = _KCR(
۲۹                    secure_key_length=_n_total,
۳۰                    privacy_amplification_term=result.privacy_amplification_term,
۳۱                    error_correction_leakage=result.error_correction_leakage,
۳۲                    phase_error_rate_upper_bound=result.phase_error_rate_upper_bound,
۳۳                    error_codes=result.error_codes,
۳۴                    diagnostics=result.diagnostics,
۳۵                )
۳۶        except Exception as _e:
۳۷            import logging as _logging
۳۸            _logging.getLogger('Lim2014Proof').debug(

```



```

۳۹         f'Defensive clamp check failed (non-fatal): {_e}'
۴۰     )
۴۱
۴۲     return result
۴۳

```

این قید بر اساس یک اصل بدیهی فیزیکی است: **بیت‌های کلید امن زیرمجموعه‌ای از بیت‌های خام sifted هستند.** در شرایط عادی، این قید هرگز فعال نمی‌شود زیرا فرمول‌های Lim 2014 به‌طور ذاتی محافظه‌کارانه هستند و طول کلید نهایی همیشه کمتر از n_X (که خود زیرمجموعه‌ای از کل sifted است) می‌شود. اما به‌عنوان یک تور ایمنی در برابر باگ‌های داخلی طراحی شده است. اگر در اثر یک باگ در حلقه kappa-solver, fallback, LP، یا سرریز در تخمین decoy، طول کلید نهایی از $n_X + n_Z$ بیشتر شود، این قید فعال می‌شود، طول کلید را به $n_X + n_Z$ محدود می‌کند، یک هشدار در log ثبت می‌کند و نتیجه بازنویسی‌شده را بازمی‌گرداند. نکته طراحی مهم این است که این قید در یک بلوک try/except قرار دارد. اگر خود بررسی قید به هر دلیلی (مانند فقدان فیلد sifted در یک TallyCounts قدیمی) شکست بخورد، استثنا به‌صورت غیرکشنده (non-fatal) در سطح debug ثبت می‌شود و نتیجه اصلی بدون تغییر بازگردانده می‌شود. این طراحی تضمین می‌کند که قید دفاعی هرگز به‌طور خود یک منبع شکست جدید معرفی نکند. این فلسفه fail-safe با اصل امنیتی defense in depth همخوان است: چندین لایه دفاعی مستقل، هر کدام با fallback امن.

۵.۷ ارسال و دریافت با سایر ماژول‌ها

۱.۵.۷ ورودی‌های ماژول

ماژول lim2014.py وابستگی‌های داخلی و خارجی متعددی دارد. وابستگی‌های داخلی شامل موارد زیر است:

- `qkd.proofs.base`: کلاس پایه `KeyCalculationResult`, `DecoyEstimates`, `FiniteKeyProof` و تابع کمکی `_json_safe_dict` از این ماژول وارد می‌شوند. این کلاس پایه رابط عمومی و خدمات مشترک مانند آنتروپی باینری، clamp عددی و گزارش‌گیری audit را فراهم می‌سازد.

- `qkd.proofs.utils_lp`: تابع `solve_lp` برای حل مسأله برنامه‌ریزی خطی در مسیر N-decoy وارد می‌شود. این تابع از `scipy.optimize.linprog` یا `cvxopt` به عنوان solver زیرین استفاده می‌کند.

- `qkd.datatypes`: `ConfidenceBoundMethod`, `TallyCounts`, `EpsilonAllocation` و `KeyCalculationResult` از این ماژول وارد می‌شوند. `TallyCounts` شامل فیلدهای `sifted_x`, `sifted_z`, `errors_sifted_x` و `errors_sifted_z` است که شمارش‌های آماری شبکه آشکارسازی را نگه می‌دارد. `ConfidenceBoundMethod` برای اعمال روش CHERNOFF در سازنده استفاده می‌شود.

- `qkd.exceptions`: استثناهای سفارشی `ConfigurationError`, `LPFailureError` و `ParameterValidationError` برای گزارش خطاهای ساختاریافته.

- `qkd.params.QKDParams`: کلاس پارامترهای QKD که شامل f_{EC} , ϵ_{cor} , ϵ_{sec} , `photon_number_cap`, `ci_method`, `lp_solver_method`, `error_correction_model`, `security_constant_kappa` و `source.pulse_configs` است. توجه شود که `ci_method` در سازنده به CHERNOFF بازنویسی می‌شود زیرا امنیت کلید متناهی در این چارچوب به‌طور صریح به کران‌های Chernoff وابسته است.

وابستگی‌های خارجی شامل `numpy` برای محاسبات برداری در مسیر LP، `math` برای توابع ریاضی پایه، `logging` برای ثبت رویدادها و `mpmath` (اختیاری) برای محاسبات با دقت بالا هستند.

۲.۵.۷ خروجی‌های ماژول

خروجی اصلی ماژول دو شیء است:

۱. `KeyCalculationResult`: این شیء از کلاس پایه به ارث رسیده و شامل فیلدهای `secure_key_length` (ℓ)، `privacy_amplification_term` (جریمه PA)، `error_correction_leakage` (λ_{EC})، `phase_error_rate_upper_bound` (φ_X^U)، `error_codes`

و diagnostics است. diagnostics شامل اطلاعات کامل مانند epsilon_accounting (با $\tau_1, \tau_0, v_{Z,1}^U, s_{Z,1}^L, s_{X,1}^L, s_{X,0}^L$), مقادیر میانی (note و hoeffding_ln_term, bound_method) کران‌های Hoeffding/Chernoff برای هر شدت، مدل تصحیح خطای انتخاب‌شده و در صورت استفاده، اطلاعات همگرایی kappa-solver است.

۲. DecoyEstimatesLim2014: این شیء خروجی متد estimate_yields_and_errors است و فیلدهای اختصاصی چارچوب Lim 2014 را نگه می‌دارد. این شیء برای کاربرانی که می‌خواهند تخمین‌های میانی را بدون محاسبه کامل طول کلید بررسی کنند، مفید است.

۳.۵.۷ جریان داده در سراسر فریم‌ورک

جریان داده کلی به این شکل است:

۱. لایه شبیه‌ساز پالس‌های نوری را با شدت‌های μ_1, μ_2, μ_3 تولید و از کانال کوانتومی عبور می‌دهد.
۲. آشکارسازها در سمت باب، آشکارسازی‌ها و خطاها را در دو پایه X و Z و برای هر شدت ثبت کرده و در TallyCounts ذخیره می‌کنند.
۳. این شمارش‌ها به صورت یک دیکشنری stats_map با کلیدهای نام شدت، اندیس رشته‌ای یا اندیس صحیح به اثبات امنیتی داده می‌شوند. متد normalize_stats_map به‌طور خودکار فرمت کلید را تشخیص می‌دهد.
۴. متد calculate_key_length ابتدا پیکربندی را اعتبارسنجی می‌کند. اگر security_constant_kappa تنظیم شده باشد، مسیر kappa-solver تکراری انتخاب می‌شود؛ در غیر این صورت مسیر مستقیم با ε_{sec} ثابت.
۵. در مسیر مستقیم، ε تک‌آزمون به صورت $\varepsilon_{\text{sec}}/21$ محاسبه می‌شود و جمله Chernoff/Hoeffding $\ln(21/\varepsilon_{\text{sec}})$ آماده می‌شود.
۶. سپس τ_1 و τ_0 محاسبه می‌شوند. اگر تعداد شدت‌ها بیش از ۳ باشد، مسیر LP انتخاب می‌شود؛ در غیر این صورت مسیر تحلیلی معادلات (۲) تا (۵).
۷. در مسیر تحلیلی، شدت‌ها به ترتیب نزولی مرتب می‌شوند تا نقش سیگنال، decoy و خلأ مشخص شود. سپس کران‌های هیبرید Hoeffding/Chernoff برای هر شدت در هر دو پایه محاسبه می‌شوند. این کران‌ها به‌طور خودکار در رژیم مناسب (بزرگ، کوچک یا صفر) عمل می‌کنند.
۸. با استفاده از این کران‌ها، $s_{Z,1}^L, s_{X,1}^L, s_{X,0}^L$ و $v_{Z,1}^U$ محاسبه می‌شوند.
۹. سپس φ_X^U با تابع calc_phi_X_upper محاسبه می‌شود.
۱۰. در نهایت، معادله (۱) ارزیابی می‌شود و طول کلید امن به دست می‌آید.
۱۱. قید دفاعی فیزیکی بررسی می‌کند که طول کلید از کل sifted $(n_X + n_Z)$ بیشتر نشود. در صورت فعال شدن، هشدار ثبت و مقدار بازنویسی می‌شود.
۱۲. لایه بالایی می‌تواند با explain_key_decision (از کلاس پایه) علت نتیجه را به صورت متنی دریافت کند و با audit_dump یک گزارش کامل برای بازبینی امنیتی تولید کند.

این جداسازی مراحل به گونه‌ای طراحی شده که هر مرحله قابل آزمایش و بازبینی مستقل باشد. به عنوان مثال، می‌توان DecoyEstimatesLim2014 را به صورت دستی ساخت و رفتار مرحله نهایی محاسبه کلید را در شرایط خاص بررسی کرد. این ماثول‌بندی از اصل Single Responsibility پیروی می‌کند و قابلیت نگهداری و توسعه کد را به طور قابل توجهی افزایش می‌دهد. علاوه بر این، شفافیت کامل در diagnostics به کاربر اجازه می‌دهد هر مرحله از محاسبه را ردیابی کرده و در صورت نیاز، فرضیات تئوریک را برای کاربرد خاص خود بازبینی کند. استراتژی هیبرید Hoeffding/Chernoff به‌طور خاص برای شبیه‌سازی‌های بلندمسافت که در آن‌ها شمارش decoy یا خلأ ممکن است صفر یا بسیار کم باشد، یک بهبود چشمگیر در کارایی عملی فراهم می‌کند بدون آنچه هیچ‌گونه تضعیف امنیتی به همراه داشته باشد.

فصل ۸

برهان‌های (Wang, 2005) (Ma, 2005)

۱.۸ نمای کلی و هدف ماژول

فایل‌های `qkd/proofs/wang2005.py` و `qkd/proofs/ma2005.py` دو ستون فقرات تئوریک چارچوب شبیه‌سازی QKD در زمینه پروتکل‌های Decoy-State را تشکیل می‌دهند. این دو ماژول به صورت موازی عمل می‌کنند و هر کدام پاسخگوی نیاز متفاوتی در تحلیل امنیتی هستند: ماژول `ma2005.py` پیاده‌سازی وفادارانه و چندگانه چارچوب مرجع Ma, Qi, Zhao, Lo (سال ۲۰۰۵) است که در مقاله “Practical decoy state for quantum key distribution” (جلد ۷۲، شماره ۱۲۳۲۶) منتشر شده، و ماژول `wang2005.py` پیاده‌سازی چارچوب تکمیلی Wang (سال ۲۰۰۸) است که در مقاله “General theory of decoy-state quantum cryptography with source errors” (جلد ۷۷، شماره ۰۴۲۳۱۱) ارائه شده است. در حالی که چارچوب Ma 2005 فرض می‌کند که منبع نوری آلایس یک منبع `coherent state` ایده‌آل با شدت دقیقاً مشخص است، چارچوب Wang 2008 این فرض را کنار می‌گذارد و اجازه می‌دهد شدت منبع دارای نوسان باشد یا حتی خود حالت کوانتومی از یک `coherent state` ایده‌آل منحرف شود. این دو چارچوب در کنار هم طیف کاملی از فرضیات منبع را پوشش می‌دهند و به کاربر اجازه می‌دهند با انتخاب ماژول مناسب، تحلیل امنیتی متناسب با شرایط واقعی منبع خود را انتخاب کند.

ماژول `ma2005.py` شامل پنج پیاده‌سازی مجزا از چارچوب Ma 2005 است که هر کدام برای سناریوی خاصی بهینه شده‌اند. این پنج کلاس عبارتند از: `Ma2005VacuumWeakProof` که پروتکل کلاسیک “Vacuum + Weak” را با دو `decoy` (یک `decoy` ضعیف با شدت ν و یک `decoy` خلاً) پیاده‌سازی می‌کند و در برابر نوسان شدت منبع به صورت `worst-case` مقاوم است؛ `Ma2005GeneralTwoDecoyProof` که حالت تعمیم‌یافته دو `decoy` را با استفاده از فرمول‌های تحلیلی معادلات (۱۸)، (۲۱) و (۲۵) مقاله پیاده می‌کند و بر خلاف `Vacuum + Weak`، فرض خلاً بودن یکی از `decoy`ها را ندارد؛ `Ma2005OneDecoyProof` که پروتکل تک `decoy` را با دو حالت تحلیلی (با یا بدون فرض `zero-background`) پیاده‌سازی می‌کند و برای شبکه‌هایی با محدودیت سخت‌افزاری مناسب است؛ `Ma2005GeneralMultiDecoyProof` که برای حالت چند `decoy` (بیش از دو) از برنامه‌ریزی خطی (LP) برای یافتن کران‌های بهینه e_1 و Y_1 استفاده می‌کند و تنها پیاده‌سازی در این ماژول است که از توزیع `Thermal` پشتیبانی می‌کند؛ و در نهایت `Ma2005AsymptoticProof` که حد نظری “Infinite Decoy State” را با معادلات (۲۷) و (۲۸) مقاله پیاده می‌کند و به عنوان مرجع بالای عملکرد (`upper bound`) عمل می‌کند. ماژول `wang2005.py` یک پیاده‌سازی تک‌کلاسی به نام `Wang2005Proof` دارد که از کلاس پایه `FiniteKeyProof` ارث می‌برد و یک `dataclass` تخصصی به نام `DecoyEstimatesWang2005` را معرفی می‌کند که فیلد اضافی `intensity_error_bound` را برای ثبت کران خطای منبع نگه می‌دارد. این ماژول از همان فرمول پایه `Vacuum + Weak` استفاده می‌کند اما تمام شدت‌ها (μ و ν) را با کران‌های بالا و پایین ($\mu^L, \mu^U, \nu^L, \nu^U$) جایگزین می‌کند تا در بدترین حالت نوسان منبع، امنیت حفظ شود. علاوه بر این، ماژول `wang2005.py` از یک مفهوم جدید به نام `source fidelity` پشتیبانی می‌کند که نشان می‌دهد حالت واقعی منبع چقدر به یک `coherent state` ایده‌آل نزدیک است. اگر `fidelity` کمتر از یک باشد، ماژول به صورت خودکار یک `mapping` محافظه‌کارانه از $(1 - F)$ به یک `delta` مؤثر انجام می‌دهد و آن را در فرمول‌های کران شدت جایگزین می‌کند.

وظیفه نهایی هر دو ماژول، تبدیل شمارش‌های آماری شبکه آشکارسازی (که در شیء `TallyCounts` ذخیره شده‌اند) به یک تخمین امن از طول کلید نهایی است. این تبدیل با رعایت سخت‌گیرانه تمام محافظه‌کاری‌های ریاضی، آماری و فیزیکی انجام می‌شود تا هیچ طول کلیدی که از نظر تئوری قابل اثبات نیست، تولید نشود. در صورت بروز ناسازگاری پیکربندی، آمار ناکافی، یا خطای عددی، خروجی به جای `raise` کردن استثنا، یک شیء

KeyCalculationResult با طول کلید صفر و کد خطای مناسب تولید می‌کند تا خط لوله شبیه‌سازی متوقف نشود و علت شکست در گزارش تشخیصی (diagnostics) ثبت گردد. این طراحی fail-safe برای کاربردهای تولیدی که در آن‌ها قطع شبیه‌سازی هزینه بر است، حیاتی است.

اهمیت تاریخی و علمی این دو ماژول در این است که چارچوب Ma 2005 نخستین پیاده‌سازی عملی و قابل استفاده از ایده Decoy-State بود که توسط Hwang, Lo (۲۰۰۳) ارائه شده بود. پیش از این، ایده Decoy-State تنها یک مفهوم نظری بود و پیاده‌سازی عملی آن به دلیل پیچیدگی محاسباتی و فقدان فرمول‌های بسته ممکن نبود. Ma 2005 با ارائه فرمول‌های تحلیلی بسته برای کران‌های Y_1 و e_1 ، این ایده را به یک فناوری قابل اجرا تبدیل کرد. چارچوب Wang 2008 سپس این پیاده‌سازی را با در نظر گرفتن نقص‌های واقعی منبع (مانند نوسان شدت لیزر و انحراف از حالت coherent) توسعه داد. این دو چارچوب در کنار هم پایه‌ای برای تمام پروتکل‌های Decoy-State تجاری مدرن (مانند شبکه‌های Toshiba, ID Quantique, Quantum CTEK) شده‌اند.

۲.۸ مبانی تئوری و فیزیکی

۱.۲.۸ حمله Photon-Number Splitting و انگیزه Decoy-State

در پروتکل استاندارد BB84 با یک منبع تک‌شدت، امنیت در برابر حمله Photon-Number Splitting (PNS) قابل تضمین نیست. در این حمله، یک مهاجم (او) با استفاده از یک beam splitter غیرتعاملی (ND-PS) یا تکنیک‌های QND، یک فوتون از هر پالس چندفوتونی استخراج می‌کند و بقیه را به سمت باب ارسال می‌کند. از آنجا که این کار بدون ایجاد خطا انجام می‌شود، باب متوجه حضور او نمی‌شود. پس از پایه‌گذاری عمومی، او فوتون‌های ذخیره‌شده خود را در پایه درست اندازه می‌گیرد و کلید را به طور کامل به دست می‌آورد. در سیستم‌های واقعی با منبع coherent state، بخش قابل توجهی از پالس‌ها چندفوتونی هستند (به ویژه وقتی $\mu \approx 0.5$ که برای دستیابی به نرخ کلید مطلوب انتخاب می‌شود)، و در نتیجه حمله PNS یک تهدید جدی است.

راه حل Decoy-State این است که آلیس به طور تصادفی شدت نور ارسالی را میان چندین سطح تغییر می‌دهد. باب پس از اندازه‌گیری، با مقایسه نرخ آشکارسازی در شدت‌های مختلف، می‌تواند تخمین بزند که مهاجم در حال انجام PNS هست یا نه. ایده کلیدی این است که بازده تک‌فوتونی Y_1 و خطای تک‌فوتونی e_1 مستقل از انتخاب شدت آلیس است (زیرا کانال و آشکارساز فقط تعداد فوتون‌ها را می‌بینند نه شدت انتخاب‌شده را). در نتیجه، با ترکیب اطلاعات از چندین شدت، می‌توان Y_1 و e_1 را به صورت کران‌های بسته تخمین زد. چارچوب Ma 2005 نخستین کسی بود که این تخمین را به صورت فرمول‌های تحلیلی بسته ارائه داد.

۲.۲.۸ توزیع آماری فوتون‌های ارسالی

توزیع آماری فوتون‌های ارسالی از یک منبع به نوع آماری آن بستگی دارد. دو نوع آماری متداول در شبیه‌سازی QKD عبارتند از Poisson (برای منبع coherent state که از لیزر معمولی به دست می‌آید) و Thermal (برای منبع thermal state که از منابع حرارتی یا chaotic light به دست می‌آید). اگر میانگین فوتون μ باشد، احتمال ارسال n فوتون در توزیع Poisson برابر است با:

$$P(n | \mu) = e^{-\mu} \frac{\mu^n}{n!}, \quad (1.8)$$

و در توزیع Thermal برابر است با:

$$P(n | \mu) = \frac{\mu^n}{(1 + \mu)^{n+1}}. \quad (2.8)$$

تفاوت کلیدی این دو توزیع در واریانس است: برای Poisson واریانس برابر میانگین ($\text{Var} = \mu$) است، در حالی که برای Thermal واریانس برابر $\mu(1 + \mu)$ است (بزرگ‌تر از میانگین). این بدان معناست که در Thermal، بخش بیشتری از پالس‌ها چندفوتونی هستند و در نتیجه حمله PNS تهدید جدی‌تری است. در کد، تابع `_get_prob_n` در کلاس `Ma2005BaseProof` این دو توزیع را بر اساس پرچم `is_thermal` (که از پیکربندی منبع استخراج می‌شود) انتخاب می‌کند. تنها کلاس `Ma2005GeneralMultiDecoyProof` از توزیع Thermal پشتیبانی می‌کند زیرا تنها این کلاس از LP استفاده می‌کند و فرمول‌های تحلیلی بسته برای Thermal پیچیده‌تر و در دسترس نیستند.

۳.۲.۸ بازده، خطا و Gain

برای هر شدت μ ، دو کمیت قابل مشاهده تعریف می‌شوند: Gain (نرخ آشکارسازی) Q_μ و $QBER$ E_μ . این دو کمیت به ترتیب برابرند با:

$$Q_\mu = \sum_{n=0}^{\infty} Y_n P(n | \mu), \quad (3.8)$$

$$E_\mu Q_\mu = \sum_{n=0}^{\infty} e_n Y_n P(n | \mu), \quad (4.8)$$

که در آن Y_n بازده (احتمال آشکارسازی مشروط به ارسال n فوتون) و e_n خطای تکفوتونی مشروط به ارسال n فوتون است. این کمیت‌ها مستقل از انتخاب شدت آلیس هستند و تنها به خصوصیات کانال و آشکارساز بستگی دارند. این مستقل بودن، پایه ریاضی کل ایده Decoy-State است. در کد، این کمیت‌ها به صورت $Q_\mu = \text{sifted/sent}$ و $E_\mu = \text{errors_sifted/sifted}$ از شیء TallyCounts استخراج می‌شوند. بازده Y_n را می‌توان به صورت $Y_n = Y_0 + \eta_n$ تجزیه کرد که در آن Y_0 نرخ آشکارسازی dark count (مستقل از فوتون ارسالی) و η_n احتمال آشکارسازی حداقل یک فوتون از n فوتون ارسالی است. برای یک کانال با عبور η و آشکارساز با راندمان η_{det} ، بازده تکفوتونی برابر $Y_1 = Y_0 + \eta_{\text{channel}} \eta_{\text{det}}$ است. این رابطه در کلاس Ma2005AsymptoticProof برای استخراج η از روی مشاهدات و سپس محاسبه Y_1 استفاده می‌شود.

۴.۲.۸ کران پایین بازده تکفوتونی در پروتکل Vacuum + Weak

معادله (۳۴) مقاله Ma 2005، کران پایین Y_1 را برای پروتکل Vacuum + Weak (با شدت‌های سیگنال μ ، decoy ضعیف ν و خلأ) ارائه می‌دهد:

$$Y_1 \geq \frac{\mu}{\mu\nu - \nu^2} \left[Q_\nu e^\nu - Q_\mu e^\mu \frac{\nu^2}{\mu^2} - Y_0 \frac{\mu^2 - \nu^2}{\mu^2} \right]. \quad (5.8)$$

در این فرمول، Q_μ و Q_ν به ترتیب Gain مشاهده‌شده در شدت decoy و سیگنال هستند. برای محافظه‌کاری، از کران‌های بالا و پایین آماری استفاده می‌شود: Q_ν با کران پایین Q_ν^L و Q_μ با کران بالا Q_μ^U جایگزین می‌شوند تا کران پایین Y_1 تا حد ممکن سخت‌گیرانه باشد. این انتخاب‌های جهت‌دار تضمین می‌کنند که تخمین نهایی، در بدترین حالت نوسان آماری، همچنان یک کران پایین معتبر باشد.

۵.۲.۸ کران بالای خطای تکفوتونی

معادله (۳۷) مقاله Ma 2005، کران بالای e_1 را ارائه می‌دهد:

$$e_1 \leq \frac{E_\nu Q_\nu e^\nu - e_0 Y_0}{Y_1 \nu}, \quad (6.8)$$

که در آن $e_0 = 0.5$ خطای آشکارسازی خلأ است (زیرا dark count کاملاً تصادفی است و احتمال خطا برابر ۵۰٪ دارد). در این فرمول، $E_\nu Q_\nu$ با کران بالا $E_\nu^U Q_\nu^U$ جایگزین می‌شود تا کران بالای e_1 تا حد ممکن بزرگ (سخت‌گیرانه) باشد. در کد، شرط $Y_1 \cdot \nu > 10^{-20}$ بررسی می‌شود تا از تقسیم بر صفر جلوگیری شود؛ اگر این شرط نقض شود، $e_1 = 0.5$ تنظیم می‌شود که معادل بدترین حالت ممکن (کامل تصادفی) است.

۶.۲.۸ کسر برچسب‌گذاری‌شده (Tagged Fraction)

یک مفهوم کلیدی دیگر در چارچوب Ma 2005، کسر برچسب‌گذاری‌شده Δ است که نشان می‌دهد چه کسری از پالس‌های سیگنال به جای تکفوتونی، چندفوتونی هستند. معادله (۳۶) مقاله، کران بالای Δ را به صورت زیر ارائه می‌دهد:

$$\Delta \leq \frac{\nu}{\mu - \nu} \left[\frac{\nu e^{-\nu} Q_\mu}{\mu e^{-\mu} Q_\nu} - 1 \right] + \frac{\nu e^{-\nu} Y_0}{\mu Q_\nu}. \quad (7.8)$$

این کران در متد `_calculate_weak_gllp_rate` برای محاسبه نرخ کلید در حالت Weak GLLP (معادله ۴۳) استفاده می‌شود. اگر $\Delta \geq 1$ باشد، به این معناست که تمام پالس‌های سیگنال چندفوتونی هستند و هیچ امنیتی وجود ندارد؛ در این حالت، نرخ کلید به صفر برده می‌شود.

۷.۲.۸ فرمول GLLP برای نرخ کلید

چارچوب (GLLP) Gottesman-Lo-Lütkenhaus-Preskill فرمول کلی نرخ کلید امن را به صورت زیر ارائه می‌دهد:

$$R = q \{ Q_1 [1 - H_2(e_1)] - Q_\mu f(E_\mu) H_2(E_\mu) \}, \quad (۸.۸)$$

که در آن q فاکتور پایه‌گذاری (برای BB84 برابر ۵۰٪ است زیرا نیمی از پالس‌ها در پایه اشتباه دور ریخته می‌شوند)، $Q_1 = Y_1 P(1 | \mu) = Y_1 \mu e^{-\mu}$ Gain تک‌فوتونی، $H_2(\cdot)$ آنترپی باینری، $f(E_\mu)$ بازدهی تصحیح خطا و جمله اول ترم تقویت حریم خصوصی (PA) و جمله دوم ترم نشت تصحیح خطا (EC leakage) است. این فرمول در متد `calculate_gllp_rate` پیاده‌سازی شده است. خروجی این متد یک شیء `KeyCalculationResult` است که شامل طول کلید امن (به صورت صحیح)، کران بالای خطای فاز (e_1)، نشت تصحیح خطا و یک دیکشنری `diagnostics` با تمام مقادیر میانی است.

۸.۲.۸ فرمول Weak GLLP (معادله ۴۳)

علاوه بر فرمول استاندارد GLLP، مقاله Ma 2005 یک فرمول جایگزین به نام Weak GLLP (معادله ۴۳) ارائه می‌دهد که از کسر برچسب‌گذاری شده Δ به جای Y_1 و e_1 استفاده می‌کند:

$$R = q Q_\mu \left[-H_2(E_\mu) + (1 - \Delta) \left(1 - H_2\left(\frac{E_\mu}{1 - \Delta}\right) \right) \right]. \quad (۹.۸)$$

این فرمول در حالتی که تخمین دقیق Y_1 دشوار است اما کران Δ قابل محاسبه است، مفید است. در کد، اگر پرچم `use_weak_gllp` در پیکربندی فعال باشد، این فرمول به جای فرمول استاندارد استفاده می‌شود. یک شرط فیزیکی در کد بررسی می‌شود: اگر $E_\mu / (1 - \Delta) \geq 0.5$ باشد، نرخ کلید به صفر برده می‌شود زیرا آنترپی باینری در ۵۰٪ بیشینه است و ترم PA منفی می‌شود.

۹.۲.۸ حد Infinite Decoy با Asymptotic

معادلات (۲۷) و (۲۸) مقاله Ma 2005، حد نظری Infinite Decoy State را ارائه می‌دهند که در آن تعداد decoy‌ها به بی‌نهایت میل می‌کند و تمام Y_n و e_n به طور کامل قابل تخمین هستند:

$$Y_1 = Y_0 + \eta, \quad (۱۰.۸)$$

$$e_1 = \frac{e_0 Y_0 + e_{\text{det}} \eta}{Y_1}, \quad (۱۱.۸)$$

که در آن $\eta = \eta_{\text{channel}} \cdot \eta_{\text{det}}$ عبور کلی و e_{det} خطای ذاتی آشکارساز است. در کلاس `Ma2005AsymptoticProof`، η به صورت غیرمستقیم از روی Q_μ استخراج می‌شود:

$$Q_\mu = Y_0 + 1 - e^{-\eta \mu} \Rightarrow \eta = -\frac{\ln(1 + Y_0 - Q_\mu)}{\mu}. \quad (۱۲.۸)$$

اگر $1 + Y_0 - Q_\mu \leq 0$ (که یک ناهنجاری فیزیکی است و نشان می‌دهد Q_μ بیش از حد بزرگ است)، کد از مقدار تئوریک η از پیکربندی به عنوان fallback استفاده می‌کند. این کلاس به عنوان مرجع بالای عملکرد عمل می‌کند و اجازه می‌دهد کاربر ببیند در شرایط ایده‌آل، طول کلید چقدر می‌تواند باشد.

۱۰.۲.۸ خطای منبع و چارچوب Wang 2008

در چارچوب Wang 2008، فرض می‌شود که شدت منبع به طور دقیق مشخص نیست بلکه در یک بازه $[\mu^L, \mu^U]$ قرار دارد. این مدل‌سازی برای منابع لیزر واقعی که دارای نوسان شدت هستند، ضروری است. کران‌های بالا و پایین شدت به صورت زیر تعریف می‌شوند:

$$\mu^U = \mu (1 + \delta), \quad (۱۳.۸)$$

$$\mu^L = \mu (1 - \delta), \quad (۱۴.۸)$$

$$\nu^U = \nu (1 + \delta), \quad (۱۵.۸)$$

$$\nu^L = \nu (1 - \delta), \quad (۱۶.۸)$$

که در آن δ کران خطای شدت است. در کد، این δ از دو منبع ممکن است به دست آید: یا مستقیماً از `intensity_jitter` پیکربندی منبع خوانده می‌شود، یا از طریق `source fidelity` به صورت غیرمستقیم محاسبه می‌شود. اگر `source fidelity` $F < 1$ باشد، یک `mapping` محافظه‌کارانه به صورت $\delta = \max(\delta_{\text{jitter}}, 1 - F)$ انجام می‌شود. این `mapping` فرض می‌کند که انحراف از حالت `coherent` ایده‌آل، حداقل به اندازه $(1 - F)$ در شدت مؤثر اثر دارد.

سپس فرمول کران پایین Y_1 در چارچوب Wang 2008 (معادله ۵۶ مقاله) به صورت زیر بازنویسی می‌شود:

$$Y_1 \geq \frac{\mu^L}{\mu^U \nu^L - (\nu^U)^2} \left[Q_\nu e^{\nu^L} - Q_\mu e^{\mu^U} \frac{(\nu^U)^2}{(\mu^L)^2} - Q_{\text{vac}} \frac{(\mu^U)^2 - (\nu^L)^2}{(\mu^L)^2} \right]. \quad (17.8)$$

تفاوت کلیدی با فرمول Ma 2005 این است که تمام شدت‌ها با کران‌های بدترین حالت جایگزین شده‌اند. به طور خاص، μ با μ^U در مخرج‌های جمله‌های کم‌شونده و با μ^L در ضریب پیشانی جایگزین شده است تا کران تا حد ممکن سخت‌گیرانه باشد. به طور متقابل، ν با ν^L در جملات مثبت و با ν^U در جملات منفی جایگزین شده است. این انتخاب‌های جهت‌دار تضمین می‌کنند که کران نهایی، در بدترین حالت نوسان منبع، همچنان معتبر باشد.

کران بالای e_1 نیز به صورت مشابه بازنویسی می‌شود:

$$e_1 \leq \frac{E_\nu Q_\nu e^{\nu^U} - E_{\text{vac}} Q_{\text{vac}}}{Y_1 \nu^L}. \quad (18.8)$$

در این فرمول، e^ν با e^{ν^U} (کران بالا) جایگزین شده تا صورت کسر بزرگ‌تر و در نتیجه کران بالای e_1 سخت‌گیرانه‌تر شود. همچنین Y_1 و ν در مخرج با کران پایین Y_1^L و ν^L جایگزین شده‌اند تا کران تا حد ممکن بزرگ باشد. در کد، اگر $Y_1 \cdot \nu^L \leq 0$ باشد، $e_1 = 0.5$ تنظیم می‌شود که معادل بدترین حالت است.

۱۱.۲.۸ نرخ کلید در چارچوب Wang 2008

محاسبه نرخ کلید در چارچوب Wang 2008 از همان فرمول استاندارد GLLP استفاده می‌کند اما با کران‌های Y_1 و e_1 که از فرمول‌های بالا به دست آمده‌اند:

$$R = \frac{1}{2} [Y_1^L \mu e^{-\mu} (1 - H_2(e_1^U)) - Q_\mu f_{\text{EC}} H_2(E_\mu)]. \quad (19.8)$$

در اینجا $q = 0.5$ برای BB84 ثابت فرض شده و f_{EC} از پیکربندی خوانده می‌شود. در کد، این محاسبه در متد `calculate_key_length` کلاس `Wang2005Proof` پیاده‌سازی شده است. خروجی نهایی یک شیء `KeyCalculationResult` است که شامل طول کلید امن، نشت تصحیح خطا و کران بالای خطای فاز است.

۳.۸ تحلیل معماری و ساختار داده‌ها

۱.۳.۸ سلسله‌مراتب کلاس در ماژول `ma2005.py`

ماژول `ma2005.py` از یک طراحی سلسله‌مراتبی استفاده می‌کند که در آن یک کلاس پایه به نام `Ma2005BaseProof` تمام منطق مشترک را نگه می‌دارد و پنج کلاس مشتق، هر کدام یک پروتکل خاص را پیاده‌سازی می‌کنند. کلاس پایه از کلاس انتزاعی `FiniteKeyProof` (که در `qkd/proofs/base.py`) تعریف شده (ارث می‌برد و متدهای انتزاعی `allocate_epsilons`، `get_epsilon_policy`، `notation_map` و `calculate_key_length`، `estimate_yields_and_errors` را پیاده‌سازی می‌کند. این طراحی از اصل DRY (Don't Repeat Yourself) پیروی می‌کند و از تکرار کد در پنج کلاس مشتق جلوگیری می‌کند.

در ادامه، ساختار کلاس پایه `Ma2005BaseProof` را به صورت خلاصه نمایش می‌دهیم:

```
1 class Ma2005BaseProof(FiniteKeyProof):
2     """Base class for Ma 2005 proofs sharing the GLLP rate formula."""
3
4     def __init__(self, params: QKDParams, *args, **kwargs):
5         if params.ci_method != ConfidenceBoundMethod.CHERNOFF:
6             logger.warning(
7                 f"Ma2005 proof requires CHERNOFF bounds for finite-key security. "
```

```

۸      f"Overriding '{params.ci_method.name}' with CHERNOFF."
۹      )
۱۰     object.__setattr__(params, 'ci_method', ConfidenceBoundMethod.CHERNOFF)
۱۱
۱۲     super().__init__(params, *args, **kwargs)
۱۳     self.is_thermal = (self.p.source.statistics_type == SourceStatisticsType.THERMAL)
۱۴

```

در این کد، سازنده کلاس ابتدا بررسی می‌کند که روش فاصله اطمینان (CI) در پیکربندی برابر CHERNOFF باشد. اگر کاربر روش دیگری (مانند Gaussian یا Clopper-Pearson) انتخاب کرده باشد، یک هشدار ثبت می‌شود و روش به صورت صریح به CHERNOFF تغییر می‌یابد. دلیل این الزام امنیتی آن است که فرمول کران پایین Y_1^L شامل تفریق جمله‌های تقویت‌شده با e^μ است؛ اگر کران‌های فاصله اطمینان بیش از حد پهن باشند (مانند کران‌های Gaussian که با وابستگی $\mathcal{O}(1/\sqrt{N})$ همگرا می‌شوند)، این تفریق می‌تواند Y_1^L را منفی کند که فیزیکی نیست و به معنای شکست تخمین است. در مقابل، کران‌های Chernoff که از نوع multiplicative هستند (یعنی $Y_1^L > 0$ را در رژیم کلید متناهی حفظ کنند).

یک نکته فنی مهم در این پیاده‌سازی، استفاده از `object.__setattr__(params, ...)` است. این الگو به این دلیل استفاده می‌شود که شیء `QKDParams` در این فریم‌ورک به صورت `frozen` (غیرقابل تغییر) تعریف شده است و تخصیص مستقیم `params.ci_method = ...` منجر به `FrozenInstanceError` می‌شود. دور زدن این محدودیت با `object.__setattr__` به ما اجازه می‌دهد در شرایط خاص (مانند الزام امنیتی به CHERNOFF) مقدار فیلد را بازنویسی کنیم، در حالی که در شرایط عادی همچنان از مزایای `immutability` (مانند جلوگیری از تغییرات ناخواسته و قابلیت `hashing` بهره می‌بریم). این یک الگوی شناخته‌شده در پایتون برای `frozen dataclass`‌هاست که در آن نیاز به بازنویسی موقت یک فیلد وجود دارد. سپس پرچم `is_thermal` از پیکربندی منبع استخراج می‌شود که تعیین می‌کند کدام توزیع آماری (Poisson یا Thermal) در محاسبات استفاده شود.

۲.۳.۸ متد `_get_prob_n` و انتخاب توزیع آماری

این متد کلیدی‌ترین تابع مشترک در کلاس پایه است که احتمال ارسال n فوتون با میانگین μ را بر اساس نوع آماری منبع محاسبه می‌کند:

```

۱      def _get_prob_n(self, mu: float, n: int) -> float:
۲          """Calculates P(n/mu) handling both Poisson and Thermal cases."""
۳          if self.is_thermal:
۴              return (mu**n / 0.1) + mu**(n + 1)
۵          else:
۶              return math.exp(-mu) * (mu**n) / math.factorial(n)
۷

```

این متد در دو مسیر اصلی استفاده می‌شود: نخست، در متد `_calculate_gllp_rate` برای محاسبه $P(1 | \mu) = \mu e^{-\mu}$ که برای محاسبه $Q_1 = Y_1 P(1 | \mu)$ ضروری است؛ دوم، در کلاس `Ma2005GeneralMultiDecoyProof` برای ساخت ماتریس LP که در آن هر سطر معادله $Q_\mu = \sum_n Y_n P(n | \mu)$ را نشان می‌دهد. این متد به صورت `polymorphic` عمل می‌کند: با تغییر پرچم `is_thermal`، کل رفتار سیستم بدون تغییر کد قابل تغییر است. این طراحی از اصل `Open-Closed Principle` پیروی می‌کند: کد برای توسعه باز اما برای تغییر بسته است.

۳.۳.۸ نرمال‌سازی کلیدهای `stats_map` با `_normalize_stats_map`

یکی از چالش‌های عملی در اتصال لایه شبیه‌سازی به لایه اثبات امنیتی، تنوع فرمت‌های ورودی است. لایه شبیه‌ساز ممکن است کلیدهای دیکشنری `stats_map` را به سه شکل مختلف ارائه دهد: نام‌های رشته‌ای پالس (مانند "vacuum"، "decoy"، "signal")، کلیدهای عددی رشته‌ای (مانند "0"، "1"، "2") یا کلیدهای عدد صحیح (مانند 0، 1، 2). متد `_normalize_stats_map` این سه فرمت را به یک فرمت استاندارد با کلیدهای نامی تبدیل می‌کند تا تمام کد داخلی ماژول `Ma 2005` بتواند با اطمینان از کلیدهای نامی استفاده کند:

```

۱      def _normalize_stats_map(
۲          self,
۳          stats_map: Mapping[Union[str, int], TallyCounts],
۴          pulse_names: List[str],

```

```

۵     ) -> Dict[str, TallyCounts]:
۶     """Normalize stats_map keys to use pulse name strings."""
۷     # Try #1: name keys already present
۸     if all(name in stats_map for name in pulse_names):
۹         return {name: stats_map[name] for name in pulse_names}
۱۰
۱۱     # Try #2: string-integer keys ("0", "1", "2")
۱۲     if all(str(i) in stats_map for i in range(len(pulse_names))):
۱۳         return {pulse_names[i]: stats_map[str(i)]
۱۴                 for i in range(len(pulse_names))}
۱۵
۱۶     # Try #3: integer keys (0, 1, 2)
۱۷     if all(i in stats_map for i in range(len(pulse_names))):
۱۸         return {pulse_names[i]: stats_map[i]
۱۹                 for i in range(len(pulse_names))}
۲۰
۲۱     raise ParameterValidationError(
۲۲         f"stats_map keys do not match pulse names or positional indices. "
۲۳         f"Expected names {pulse_names!r}; got {list(stats_map.keys())!r}."
۲۴     )
۲۵
۲۶     def _get_pulse_names(self) -> List[str]:
۲۷         """Return the ordered list of pulse names from the source config."""
۲۸         return [pc.name for pc in self.p.source.pulse_configs]
۲۹

```

این متد با سه تلاش متوالی کار می‌کند: نخست، فرض می‌کند که کلیدها از قبل نام‌های پالس هستند و اگر این فرض درست باشد، همان دیکشنری را با فیلتراسیون به نام‌های معتبر برمی‌گرداند. دوم، اگر کلیدها رشته‌های عددی باشند، آن‌ها را به نام‌های پالس بر اساس ترتیب موقعیتی نگاشت می‌کند. سوم، اگر کلیدها اعداد صحیح باشند، همان کار را انجام می‌دهد. اگر هیچ یک از این سه حالت منطبق نباشد، استثنای `ParameterValidationError` با پیام تشخیصی مشخص `raise` می‌شود. این طراحی از اصل `Pos-tels Law` (در سخاوتمندی باشید، در محافظه‌کاری سخت‌گیرانه) پیروی می‌کند: ورودی‌های متنوع پذیرفته می‌شوند، اما خروجی همیشه یک فرمت ثابت و معتبر است. تمام پنج کلاس مشتق از `Ma 2005` در ابتدای متد `estimate_yields_and_errors` خود، این نرمال‌سازی را فراخوانی می‌کنند تا از سازگاری ورودی اطمینان حاصل شود.

۴.۳.۸ تخصیص بودجه ϵ در چارچوب `Ma 2005`

متد `allocate_epsilon` در کلاس پایه، بودجه امنیتی کلی ϵ_{sec} را میان اجزای مختلف تقسیم می‌کند. چارچوب `Ma 2005` از معادله زیر برای تقسیم بودجه استفاده می‌کند:

$$\epsilon_{\text{sec}} \geq \epsilon_{\text{PE}} + 2\epsilon_{\text{smooth}} + \epsilon_{\text{PA}} + \epsilon_{\text{cor}} + \epsilon_{\text{phase_est}}. \quad (۲۰.۸)$$

در این معادله، ϵ_{PE} بودجه تخمین پارامتر، ϵ_{smooth} بودجه هموارسازی، ϵ_{PA} بودجه تقویت حریم خصوصی، ϵ_{cor} بودجه تصحیح خطا و $\epsilon_{\text{phase_est}}$ بودجه تخمین خطای فاز است. در کد، $\epsilon_{\text{phase_est}} = \epsilon_{\text{PE}}/10$ تنظیم می‌شود که یک تقسیم محافظه‌کارانه است. اگر مجموع بودجه‌های استفاده‌شده از ϵ_{sec} بیشتر یا مساوی شود (که ناسازگاری پیکربری را نشان می‌دهد)، به جای جایگزینی صریح با $\epsilon_{\text{sec}}/8$ (که قصد کاربر را به طور خام نادیده می‌گیرد)، کد از `fix v2 [A] Gap` استفاده می‌کند: مقادیر به صورت متناسب نرمال می‌شوند تا مجموعشان دقیقاً برابر ϵ_{sec} شود. این رویکرد وزن‌دهی نسبی که کاربر مشخص کرده را حفظ می‌کند و فقط مقیاس کلی را تعدیل می‌کند. فرمول نرمال‌سازی به صورت زیر است:

$$\text{scale} = \frac{\epsilon_{\text{sec}}}{\text{used_budget}}, \quad \epsilon_k^{\text{norm}} = \epsilon_k \cdot \text{scale}, \quad \forall k \in \{\text{PE}, \text{smooth}, \text{cor}\}. \quad (۲۱.۸)$$

سپس $\epsilon_{\text{phase_est}}^{\text{norm}} = \epsilon_{\text{PE}}^{\text{norm}}/10$ و $\epsilon_{\text{PA}}^{\text{norm}} = \epsilon_{\text{sec}} - (\epsilon_{\text{PE}}^{\text{norm}} + 2\epsilon_{\text{smooth}}^{\text{norm}} + \epsilon_{\text{cor}}^{\text{norm}} + \epsilon_{\text{phase_est}}^{\text{norm}})$ محاسبه می‌شوند. اگر $\epsilon_{\text{PA}}^{\text{norm}} < 0$ شود، به صفر تنظیم می‌شود. در شرایط عادی (زمانی که بودجه کافی است)، ϵ_{PA} به صورت باقی‌مانده $\epsilon_{\text{sec}} - \text{used_budget}$ محاسبه می‌شود تا کل بودجه به طور کامل استفاده شود. پیاده‌سازی کامل این متد در ادامه ارائه شده است:

```

۱     def allocate_epsilon(self) -> EpsilonAllocation:

```



```

۲ # Full constraint: eps_pe + 2*eps_smooth + eps_pa + eps_cor + eps_phase_est <= eps_sec
۳ eps_phase_est = self.p.eps_pe / 0.10
۴ used_budget = (self.p.eps_pe + 0.2 * self.p.eps_smooth
۵ + self.p.eps_cor + eps_phase_est)
۶
۷ if used_budget >= self.p.eps_sec:
۸ # Gap [A] fix (v2): normalize proportionally so they sum to eps_sec
۹ scale = self.p.eps_sec / used_budget if used_budget > 0 else 0.1
۱۰ eps_cor_n = self.p.eps_cor * scale
۱۱ eps_pe_n = self.p.eps_pe * scale
۱۲ eps_smooth_n = self.p.eps_smooth * scale
۱۳ eps_phase_est_n = eps_pe_n / 0.10
۱۴ eps_pa_n = (self.p.eps_sec
۱۵ - (eps_pe_n + 0.2 * eps_smooth_n
۱۶ + eps_cor_n + eps_phase_est_n))
۱۷ if eps_pa_n < 0.0
۱۸ eps_pa_n = 0.0
۱۹ return EpsilonAllocation(
۲۰ eps_sec=self.p.eps_sec,
۲۱ eps_cor=eps_cor_n,
۲۲ eps_pe=eps_pe_n,
۲۳ eps_smooth=eps_smooth_n,
۲۴ eps_pa=eps_pa_n,
۲۵ eps_phase_est=eps_phase_est_n
۲۶ )
۲۷
۲۸ eps_pa = self.p.eps_sec - used_budget
۲۹ return EpsilonAllocation(
۳۰ eps_sec=self.p.eps_sec,
۳۱ eps_cor=self.p.eps_cor,
۳۲ eps_pe=self.p.eps_pe,
۳۳ eps_smooth=self.p.eps_smooth,
۳۴ eps_pa=eps_pa,
۳۵ eps_phase_est=eps_phase_est
۳۶ )
۳۷

```

این پیاده‌سازی دو ویژگی مهم دارد: نخست، در مسیر نرمال‌سازی متناسب، تمام اجزا (به جز ϵ_{PA}) با همان ضریب scale کوچک می‌شوند تا نسبت‌های نسبی حفظ شوند؛ دوم، ϵ_{PA} به عنوان باقی‌مانده محاسبه می‌شود تا تضمین شود مجموع دقیقاً برابر ϵ_{sec} است. این رویکرد نسبت به روش قبلی (جایگزینی همه با $\epsilon_{sec}/8$) برتری دارد زیرا اگر کاربر مثلاً ϵ_{PE} را سه برابر تنظیم کرده باشد، این نسبت پس از نرمال‌سازی نیز حفظ می‌شود و معنای فیزیکی پیکربندی حفظ می‌گردد.

۵.۳.۸ ساختار dataclass در ماژول wang2005.py

ماژول wang2005.py یک dataclass جدید به نام DecoyEstimatesWang2005 معرفی می‌کند که از DecoyEstimates پایه ارث می‌برد و فیلد اضافی intensity_error_bound را اضافه می‌کند:

```

۱ @dataclass(frozen=True)
۲ class DecoyEstimatesWang2005(DecoyEstimates):
۳     """Decoy estimates including source error bounds."""
۴     intensity_error_bound: float = 0.0 # delta in Wang 2008
۵

```

استفاده از دکوراتور @dataclass(frozen=True) دو مزیت دارد: نخست، شیء پس از ساخت غیرقابل تغییر (immutable) می‌شود که از تغییرات ناخواسته در طول محاسبات جلوگیری می‌کند و امنیت نوع داده را تضمین می‌کند؛ دوم، شیء به طور خودکار hashable می‌شود که اجازه می‌دهد در مجموعه‌ها یا به عنوان کلید دیکشنری استفاده شود. این ویژگی برای caching و memoization در محاسبات تکراری مفید است. مقدار پیش‌فرض ۰.۰ برای intensity_error_bound به این معناست که در صورت عدم تنظیم صریح، فرض می‌شود منبع ایده‌آل است ($\delta = 0$).

۶.۳.۸ اعتبارسنجی پیکربری در Wang2005Proof

کلاس Wang2005Proof در سازنده خود، پیکربری منبع را اعتبارسنجی می‌کند تا اطمینان حاصل شود که سه نوع پالس signal, decoy و vacuum همگی موجود هستند:

```

1 def _verify_pulse_configs(self):
2     required = {"signal", "decoy", "vacuum"}
3     current = {p.name for p in self.p.source.pulse_configs}
4     if not required.issubset(current):
5         raise ConfigurationError(
6             f"Wang2005Proof requires pulse types: {required}. Found: {current}"
7         )
8

```

این اعتبارسنجی در زمان ساخت شیء انجام می‌شود (نه در زمان محاسبه) تا خطا در همان مراحل اولیه شناسایی شود. اگر پیکربندی ناقص باشد، استثنای `raise ConfigurationError` می‌شود که یک استثنای سفارشی در ماژول `qkd.exceptions` است. این طراحی fail-fast از اجرای طولانی شبیه‌سازی و شکست در مراحل بعدی جلوگیری می‌کند. استفاده از `set` برای مقایسه مورد نیاز و موجود، ترتیب پالس‌ها را نادیده می‌گیرد و فقط حضور آن‌ها را بررسی می‌کند.

۷.۳.۸ مدیریت خطای تطبیقی (Adaptive Error Correction)

یکی از ویژگی‌های پیشرفته کلاس پایه `Ma2005BaseProof`، پشتیبانی از مدل خطای تطبیقی است. متد `_calculate_f_ec` بر اساس نرخ خطای مشاهده‌شده، بازدهی تصحیح خطا را به صورت پویا تنظیم می‌کند:

```

1 def _calculate_f_ec(self, error_rate: float) -> float:
2     """Calculates adaptive error correction efficiency if configured."""
3     config = self.p.f_ec_dynamic_config
4     if config and config.get("model") == "linear":
5         base = config.get("base", (1.1
6         slope = config.get("slope", (0.0
7         return base + slope * error_rate
8     return self.p.f_error_correction
9

```

در این کد، اگر پیکربندی `f_ec_dynamic_config` تنظیم شده باشد و مدل آن `linear` باشد، بازدهی به صورت $f_{EC} = base + slope \cdot E_{\mu}$ محاسبه می‌شود. این مدل‌سازی برای شبیه‌سازی واقع‌بینانه‌تر از سیستم‌های واقعی ضروری است زیرا در عمل، کدهای تصحیح خطا در نرخ‌های خطای بالا کارایی کمتری دارند. مقادیر پیش‌فرض `base = 1.1` و `slope = 0.0` معادل یک مدل ثابت است که در صورت عدم تنظیم پیکربندی پویا، استفاده می‌شود. این طراحی انعطاف‌پذیر اجازه می‌دهد کاربر بدون تغییر کد، مدل‌های مختلف تصحیح خطا را آزمایش کند.

۸.۳.۸ نقشه‌ی نمادها (Notation Map)

هر دو ماژول متد `notation_map` را پیاده‌سازی می‌کنند که یک دیکشنری از نمادهای ریاضی به نام متغیرهای کد ارائه می‌دهد. این نقشه برای ترجمه بین فرمول‌های مقاله و کد، و برای تولید گزارش‌های قابل فهم توسط انسان استفاده می‌شود. به عنوان مثال، نقشه `Ma 2005` شامل موارد زیر است:

```

1 def notation_map(self) -> Dict[str, str]:
2     return {
3         "Y_1": "yield_1_lower_bound",
4         "e_1": "error_rate_1_upper_bound",
5         "Q_mu": "Gain of signal state",
6         "E_mu": "QBER of signal state",
7         "\\Delta": "tagged_fraction (Delta)",
8         "f(e)": "f_error_correction",
9         "Y_1^L": "yield_1_lower_bound",
10        "e_ph": "error_rate_1_upper_bound",
11        "s_z_1^L": "Calculated as Q_1 * N_sent (single-photon count)"
12    }
13

```

این نقشه به ابزارهای بصری‌سازی و گزارش‌گیری اجازه می‌دهد به طور خودکار نمادهای ریاضی را در خروجی‌ها نمایش دهند و در عین حال، در کد از نام‌های توصیفی استفاده کنند. این جداسازی لایه نمایش از لایه محاسبه، از اصول کلیدی مهندسی نرم‌افزار است و قابلیت نگهداری کد را به طور قابل توجهی افزایش می‌دهد.

۴.۸ پیاده‌سازی ریاضی و الگوریتمی

۱.۴.۸ محاسبه نرخ GLLP استاندارد

متد `_calculate_gllp_rate` در کلاس پایه، هسته اصلی محاسبه نرخ کلید در چارچوب Ma 2005 است. این متد دو حالت دارد: حالت استاندارد GLLP و حالت Weak GLLP. در ادامه، نسخه کامل این متد را با توضیح خط به خط ارائه می‌دهیم:

```

1  def _calculate_gllp_rate(
2      self,
3      decoy_estimates: DecoyEstimates,
4      stats_map: Dict[str, TallyCounts]
5  ) -> KeyCalculationResult:
6      signal_stats = stats_map.get("signal")
7      if not signal_stats or signal_stats.sent == 0:
8          return KeyCalculationResult(0, 0.0, 0.0, 5.0,
9              error_codes=[ErrorCode.INSUFFICIENT_STATISTICS])
10
11      mu = self.p.source.get_pulse_config_by_name("signal").mean_photon_number
12      q_factor = 5.0
13
14      Q_mu = signal_stats.sifted / signal_stats.sent
15      E_mu = signal_stats.errors_sifted / signal_stats.sifted \
16      if signal_stats.sifted > 0 else 5.0
17      f_ec = self._calculate_f_ec(E_mu)
18
19      Y1_L = decoy_estimates.yield_1_lower_bound
20      e1_U = decoy_estimates.error_rate_1_upper_bound
21
22      prob_1_mu = self._get_prob_n(mu, 1)
23      Q_1 = Y1_L * prob_1_mu
24

```

در خطوط ابتدایی، آمار سیگنال از `stats_map` استخراج می‌شود. اگر آمار موجود نباشد یا تعداد پالس‌های ارسالی صفر باشد، یک نتیجه با طول کلید صفر و کد خطای `INSUFFICIENT_STATISTICS` بازگردانده می‌شود. سپس شدت سیگنال μ از پیکربندی خوانده می‌شود و q -factor برابر 0.5 (برای BB84) تنظیم می‌شود. مقادیر قابل مشاهده Q_μ و E_μ از آمار محاسبه می‌شوند. در صورتی که `sifted = 0` (هیچ آشکارسازی در پایه درست نباشد)، $E_\mu = 0.5$ فرض می‌شود که بدترین حالت است. در نهایت، $Q_1 = Y_1 \cdot P(1 | \mu)$ محاسبه می‌شود که Gain تک‌فوتونی است.

در ادامه، کد بررسی می‌کند که آیا حالت Weak GLLP فعال است یا خیر:

```

1  weak_rate = self._calculate_weak_gllp_rate(decoy_estimates, stats_map)
2  delta_bound = decoy_estimates.diagnostics.numeric_diagnostics.get(
3      "Delta_tagged_bound_Eq36", None)
4
5  if getattr(self.p, 'use_weak_gllp', False):
6      key_len = max(0, int(weak_rate * signal_stats.sent))
7      return KeyCalculationResult(
8          secure_key_length=key_len,
9          privacy_amplification_term=0.0,
10         error_correction_leakage=Q_mu * f_ec *
11         binary_entropy(E_mu) * signal_stats.sent,
12         phase_error_rate_upper_bound=E_mu / 0.1 - delta_bound)
13     if delta_bound and delta_bound < 0.1 else 5.0,
14     diagnostics={"mode": "WEAK_GLLP_EQ_43",
15         "rate_per_pulse": weak_rate, "Delta": delta_bound},
16     weak_gllp_rate=weak_rate,
17     tagged_fraction_bound=delta_bound
18 )
19

```

در این بخش، ابتدا نرخ Weak GLLP محاسبه می‌شود (حتی اگر استفاده نشود، برای گزارش‌گیری diagnostics مفید است). سپس اگر پرچم `use_weak_gllp` فعال باشد، نتیجه بر اساس فرمول Weak GLLP (معادله ۴۳) ساخته می‌شود. طول کلید به صورت $N_{\text{sent}} \cdot R_{\text{weak}}$ محاسبه می‌شود و کران بالای خطای فاز به صورت $E_\mu / (1 - \Delta)$ تنظیم می‌شود. این تناظر با فرمول Weak GLLP که در بخش تئوری توضیح داده شد،

دقیق است.

در غیر این صورت، مسیر استاندارد GLLP اجرا می‌شود:

```

1  h2_E_mu = binary_entropy(E_mu)
2  term_ec_cost = Q_mu * f_ec * h2_E_mu
3
4  h2_e1 = binary_entropy(e1_U)
5  term_pa_gain = Q_1 * 0.1) - h2_e1
6
7  rate = q_factor * (term_pa_gain - term_ec_cost)
8  key_len = max(0, int(rate * signal_stats.sent))
9
10 diag = {
11     "Q_mu": Q_mu, "E_mu": E_mu, "Q_1": Q_1,
12     "Y1_L": Y1_L, "e1_U": e1_U, "rate_per_pulse": rate,
13     "f_ec_used": f_ec, "weak_gllp_rate": weak_rate,
14     "asym_retry_used": False, # always False after paper-faithful removal
15 }
16 if decoy_estimates.diagnostics.numeric_diagnostics:
17     diag.update(decoy_estimates.diagnostics.numeric_diagnostics)
18
19 return KeyCalculationResult(
20     secure_key_length=key_len,
21     privacy_amplification_term=term_pa_gain * signal_stats.sent,
22     error_correction_leakage=term_ec_cost * signal_stats.sent,
23     phase_error_rate_upper_bound=e1_U,
24     diagnostics=diag,
25     weak_gllp_rate=weak_rate,
26     tagged_fraction_bound=delta_bound
27 )
28

```

در این مسیر، ابتدا آنتروپی باینری $H_2(E_\mu)$ محاسبه می‌شود و ترم نشت تصحیح خطا به صورت $Q_\mu \cdot f_{EC}$ به دست می‌آید. سپس آنتروپی باینری $H_2(e_1^U)$ محاسبه می‌شود و ترم تقویت حریم خصوصی به صورت $Q_1 \cdot (1 - H_2(e_1^U))$ محاسبه می‌شود. نرخ نهایی برابر $q \cdot (PA - EC)$ است و طول کلید به صورت $\max(0, \lfloor R \cdot N_{\text{sent}} \rfloor)$ محاسبه می‌شود. تابع $\max$ تضمین می‌کند که طول کلید هرگز منفی نشود (که در صورت $PA < EC$ ممکن است رخ دهد). در نهایت، یک دیکشنری diagnostics غنی شامل تمام مقادیر میانی ساخته می‌شود و در شیء نتیجه قرار می‌گیرد. یک فیلد مهم در این دیکشنری `asym_retry_used` است که پس از حذف منطق `asymptotic retry` غیروفادارانه به مقاله، همواره مقدار `False` دارد؛ این فیلد برای حفظ سازگاری با کدهای گزارش‌گیری موجود و برای انطباق آینده در صورت فعال‌سازی مجدد نگه داشته شده است.

۲.۴.۸ پیاده‌سازی پروتکل Vacuum + Weak

کلاس `Ma2005VacuumWeakProof` پروتکل کلاسیک Vacuum + Weak را با بهبود مقاومت در برابر نوسان شدت پیاده‌سازی می‌کند. در ادامه، بخش کلیدی متد `estimate_yields_and_errors` این کلاس را بررسی می‌کنیم:

```

1  def estimate_yields_and_errors(self, stats_map: Dict[str, TallyCounts]) -> DecoyEstimates:
2      stats_map = self._normalize_stats_map(stats_map, self._get_pulse_names())
3      if self.is_thermal:
4          raise ConfigurationError("Ma2005VacuumWeakProof supports only Poisson statistics.")
5
6      try:
7          cfg_sig = self.p.source.get_pulse_config_by_name("signal")
8          cfg_decoy = self.p.source.get_pulse_config_by_name("decoy")
9          cfg_vac = self.p.source.get_pulse_config_by_name("vacuum")
10     except Exception as e:
11         return DecoyEstimates(0.0, 5.0, False, 0.1,
12             SolverDiagnostics("Ma2005", False, str(e)))
13
14     mu_base = cfg_sig.mean_photon_number
15     nu_base = cfg_decoy.mean_photon_number
16
17     delta = self.p.source.intensity_jitter
18     n_total = sum(s.sent for s in stats_map.values())
19     if n_total > 0:
20         stat_fluc = 0.1 / math.sqrt(n_total)

```

```

۲۱     delta += stat_fluc
۲۲
۲۳     mu_min, mu_max = mu_base * (1 - delta), mu_base * (1 + delta)
۲۴     nu_min, nu_max = nu_base * (1 - delta), nu_base * (1 + delta)
۲۵

```

در خطوط ابتدایی، ابتدا بررسی می‌شود که منبع از نوع Poisson باشد (این کلاس از Thermal پشتیبانی نمی‌کند). سپس سه پیکربری پالس (سیگنال، decoy و خلا) از پیکربری منبع استخراج می‌شوند. اگر هر کدام موجود نباشند، یک DecoyEstimates با طول صفر و diagnostics خطا بازگردانده می‌شود. سپس شدت پایه μ و ν خوانده می‌شوند و کران خطای منبع δ از پیکربری استخراج می‌شود. یک نکته کلیدی در این کد، افزودن نوسان آماری به δ است: $\delta_{\text{total}} = \delta_{\text{jitter}} + 1/\sqrt{N_{\text{total}}}$ که در آن N_{total} کل پالس‌های ارسالی است. این نوسان آماری مدل می‌کند که برای یک N متناهی، توزیع فوتون‌های مشاهده‌شده دقیقاً با توزیع تتوریک همخوانی ندارد. در نهایت، کران‌های بالا و پایین شدت به صورت $\mu_{\min} = \mu(1 - \delta)$ و $\mu_{\max} = \mu(1 + \delta)$ محاسبه می‌شوند. سپس کران‌های آماری برای Q_μ ، Q_ν و Q_{vac} محاسبه می‌شوند:

```

۱     alpha = self.eps_alloc.eps_pe
۲
۳     Q_mu_L, Q_mu_U, E_mu_L, E_mu_U = self._get_bounds_stats(
۴         stats_map, "signal", alpha)
۵     Q_nu_L, Q_nu_U, E_nu_L, E_nu_U = self._get_bounds_stats(
۶         stats_map, "decoy", alpha)
۷     Q_vac_L, Q_vac_U, E_vac_L, E_vac_U = self._get_bounds_stats(
۸         stats_map, "vacuum", alpha)
۹
۱۰    Y0_U = Q_vac_U
۱۱    Y0_L = Q_vac_L
۱۲    e0 = 5.0
۱۳

```

در این بخش، $\alpha = \varepsilon_{\text{PE}}$ به عنوان سطح اطمینان استفاده می‌شود و کران‌های دوطرفه برای Q و E در هر سه شدت محاسبه می‌شوند. بازده خلا Y_0 به صورت مستقیم از آمار vacuum استخراج می‌شود: $Y_0^U = Q_{\text{vac}}^U$ و $Y_0^L = Q_{\text{vac}}^L$. این یک ویژگی متمایزکننده پروتکل Vacuum + Weak است که در آن یک decoy خلا به طور مستقیم Y_0 را اندازه می‌گیرد. $e_0 = 0.5$ ثابت فرض می‌شود زیرا dark count کاملاً تصادفی است. سپس فرمول کران پایین Y_1 با کران‌های بدترین حالت اعمال می‌شود:

```

۱     # [Paper Eq 34] Lower Bound on Y1
۲     term_Q_nu = Q_nu_L * math.exp(nu_min)
۳     term_Q_mu = Q_mu_U * math.exp(mu_max) * (nu_max**2 / mu_min**2)
۴     term_Y0 = Y0_U * (mu_max**2 - nu_min**2) / (mu_min**2)
۵
۶     denom = mu_max * nu_min - nu_max**2
۷     if denom <= 0:
۸         prefactor = 0.0
۹     else:
۱۰    prefactor = mu_min / denom
۱۱
۱۲    Y1_L = prefactor * (term_Q_nu - term_Q_mu - term_Y0)
۱۳    Y1_L = max(0.0), Y1_L)
۱۴
۱۵    # [Paper Eq 37] Upper Bound on e1
۱۶    if Y1_L * nu_min <= 1e-20:
۱۷        e1_U = 5.0
۱۸    else:
۱۹        numerator_e1 = (E_nu_U * Q_nu_U * math.exp(nu_max)) - (e0 * Y0_L)
۲۰        e1_U = numerator_e1 / (Y1_L * nu_min)
۲۱        e1_U = min(5.0), max(0.0), e1_U))
۲۲
۲۳    # [Paper Eq 36] Tagged Fraction Delta Upper Bound
۲۴    if Q_nu_L > 0:
۲۵        term1 = (nu_max / (mu_min - nu_max))
۲۶        term2 = (nu_max * math.exp(-nu_min) * Q_mu_U) / \
۲۷            (mu_min * math.exp(-mu_max) * Q_nu_L) - 0.1
۲۸        term3 = (nu_max * math.exp(-nu_min) * Y0_U) / (mu_min * Q_nu_L)
۲۹        delta_bound_strict = term1 * term2 + term3
۳۰    else:
۳۱        delta_bound_strict = 0.1
۳۲

```

```

۳۳ # Asymptotic Reference
۳۴ eta = self.p.channel.transmittance * self.p.detector.det_eff_d0
۳۵ y0_val = 0.2 * self.p.detector.dark_rate # Gap [B] fix: two-detector BB84
۳۶ e_det = self.p.detector.qber_intrinsic
۳۷
۳۸ y1_asymptotic = y0_val + eta
۳۹ e1_asymptotic = 5.0 * y0_val + e_det * eta / y1_asymptotic \
۴۰ if y1_asymptotic > 0 else 5.0
۴۱
۴۲ beta_y1 = (y1_asymptotic - Y1_L) / y1_asymptotic \
۴۳ if y1_asymptotic > 0 else 0.0
۴۴ beta_e1 = (e1_U - e1_asymptotic) / e1_asymptotic \
۴۵ if e1_asymptotic > 0 else 0.0
۴۶
۴۷ diag_data = {
۴۸     "Y0_L": Y0_L, "Y0_U": Y0_U,
۴۹     "Delta_tagged_bound_Eq36": delta_bound_strict,
۵۰     "beta_Y1_deviation": beta_y1,
۵۱     "beta_e1_deviation": beta_e1,
۵۲     "used_asymptotic_fallback": False, # always False after paper-faithful removal
۵۳     "Y1_asymptotic": y1_asymptotic,
۵۴     "e1_asymptotic": e1_asymptotic,
۵۵ }
۵۶
۵۷ return DecoyEstimates(
۵۸     yield_1_lower_bound=Y1_L,
۵۹     error_rate_1_upper_bound=e1_U,
۶۰     is_feasible=True,
۶۱     failure_prob_used0.0=,
۶۲     diagnostics=SolverDiagnostics("Ma2005_VacWeak_Robust", True, "OK",
۶۳     numeric_diagnostics=diag_data)
۶۴ )
۶۵

```

این پیاده‌سازی معادلات (۳۴)، (۳۷) و (۳۶) مقاله را با کران‌های بدترین حالت اعمال می‌کند. در ادامه، بخش‌های کلیدی این کد را به تفصیل توضیح می‌دهیم:

کران پایین Y_1 (معادله ۳۴): هر شدت در هر جایگاه فرمول با کران مناسب (بالا یا پایین) جایگزین شده است تا کران نهایی تا حد ممکن سخت‌گیرانه باشد. به طور خاص، Q_ν با کران پایین Q_ν^L و Q_μ با کران بالا Q_μ^U جایگزین می‌شوند و Y_0 با کران بالا Y_0^U . اگر $Y_0^U - \nu_{\max}^2 \leq 0$ (که در پیکربندی‌های غیرفیزیکی ممکن است رخ دهد)، ضریب پیشانی به صفر تنظیم می‌شود و در نتیجه $Y_1 = 0$ می‌شود. در نهایت، $\max(0, \cdot)$ تضمین می‌کند که Y_1 هرگز منفی نشود.

کران بالای e_1 (معادله ۳۷): این فرمول کران بالای خطای تک‌فوتونی را بر اساس مشاهدات decoy ضعیف محاسبه می‌کند. صورت کسر شامل $E_\nu^U \cdot Q_\nu^U \cdot e^{\nu_{\max}}$ (با کران‌های بالا برای محافظه‌کاری) منهای $e_0 \cdot Y_0^L$ است. مخرج شامل $Y_1^L \cdot \nu_{\min}$ است که با کران‌های پایین محافظه‌کارانه انتخاب شده. اگر $Y_1^L \cdot \nu_{\min} \leq 10^{-20}$ باشد (تقریباً صفر)، $e_1 = 0.5$ تنظیم می‌شود که معادل بدترین حالت (کامل تصادفی) است. در نهایت، e_1 با $\min(0.5, \max(0, \cdot))$ در بازه $[0, 0.5]$ محدود می‌شود.

کران بالای کسر برچسب‌گذاری شده Δ (معادله ۳۶): این کران نشان می‌دهد چه کسری از پالس‌های سیگنال می‌توانند چندفوتونی باشند. اگر $Q_\nu^L > 0$ باشد، فرمول کامل اعمال می‌شود؛ در غیر این صورت، $\Delta = 1.0$ تنظیم می‌شود که به معنای عدم امکان تخمین است. این مقدار Δ در دیکشنری diagnostics با کلید Delta_tagged_bound_Eq36 ذخیره می‌شود و توسط متد calculate_weak_gllp_rate برای محاسبه نرخ Weak GLLP استفاده می‌شود.

مرجع حد Asymptotic: پس از محاسبه کران‌های متناهی، کد یک مرجع asymptotic محاسبه می‌کند تا انحراف کران متناهی از حد ایده‌آل قابل اندازه‌گیری باشد. در این محاسبات، η از پارامترهای کانال و آشکارساز خوانده می‌شود ($\eta = \eta_{\text{channel}} \cdot \eta_{\text{det}}$) و y_0 به صورت $2 \cdot \text{dark_rate}$ تنظیم می‌شود که Gap [B] fix برای پشتیبانی از BB84 دو آشکارساز است. در BB84 استاندارد، آلیس در دو پایه X و Z ارسال می‌کند و باب نیز در دو پایه اندازه‌گیری می‌کند، بنابراین نرخ کلی dark count مؤثر دو برابر نرخ تک‌آشکارساز است. سپس $Y_1^{\text{asym}} = y_0 + \eta$ و $e_1^{\text{asym}} = (0.5 \cdot y_0 + e_{\text{det}} \cdot \eta) / Y_1^{\text{asym}}$ محاسبه می‌شوند.

ضریب انحراف β : دو کمیت تشخیصی β_{Y_1} و β_{e_1} به صورت زیر محاسبه می‌شوند:

$$\beta_{Y_1} = \frac{Y_1^{\text{asym}} - Y_1^L}{Y_1^{\text{asym}}}, \quad \beta_{e_1} = \frac{e_1^U - e_1^{\text{asym}}}{e_1^{\text{asym}}}. \quad (۲۲.۸)$$

این ضرایب نشان می‌دهند که کران متناهی چقدر از حد ایده‌آل فاصله دارد. اگر β_{Y_1} نزدیک به صفر باشد، یعنی کران متناهی تقریباً به حد ایده‌آل رسیده است و اندازه بلوک کافی است. اگر β_{Y_1} بزرگ باشد، یعنی افزایش اندازه بلوک می‌تواند کران را به طور قابل توجهی بهبود بخشد. این اطلاعات برای بهینه‌سازی طراحی آزمایش حیاتی است. فیلد `used_asymptotic_fallback` همواره False است زیرا پس از حذف منطق fallback غیروفادارانه به مقاله، کد همواره مسیر تخمین اصلی را طی می‌کند و فقط مقادیر asymptotic را برای مقایسه در diagnostics گزارش می‌کند.

۳.۴.۸ پیاده‌سازی پروتکل General Two-Decoy

کلاس `Ma2005GeneralTwoDecoyProof` پروتکل دو decoy را تعمیم یافته را پیاده‌سازی می‌کند که بر خلاف `Vac + Weak`، فرض خلاً بودن یکی از decoy را ندارد. این کلاس نیز مانند سایر کلاس‌های `Ma 2005`، در ابتدای متد `estimate_yields_and_errors` فراخوانی `_normalize_stats_map` را انجام می‌دهد تا کلیدهای دیکشنری ورودی به فرمت استاندارد نام پالس تبدیل شوند. سپس بررسی می‌کند که منبع از نوع `Poisson` باشد (این کلاس از `Thermal` پشتیبانی نمی‌کند) و در صورت عدم تطابق، استثنای `ConfigurationError` را می‌شود ($\nu_2 < \nu_1$). این `raise` می‌شود. در ادامه، دو پیکربری decoy با کوچکترین و بزرگترین شدت انتخاب می‌شوند ($\nu_2 < \nu_1$). این کلاس از معادلات (۱۸)، (۲۱) و (۲۵) مقاله استفاده می‌کند:

```

1  # Eq 18: Y0 lower bound
2  term_num = nu1 * Q_nu2_L * math.exp(nu2) - nu2 * Q_nu1_U * math.exp(nu1)
3  Y0_L = max(0.0), term_num / (nu1 - nu2))
4
5  # Eq 21: Y1 lower bound
6  denom_Y1 = (mu * nu1) - (mu * nu2) - (nu1**2) + (nu2**2)
7  term_Q_diff = Q_nu1_L * math.exp(nu1) - Q_nu2_U * math.exp(nu2)
8  term_mu_sub = (Q_mu_U * math.exp(mu) - Y0_L)
9  factor_sq = (nu1**2 - nu2**2) / (mu**2)
10
11  Y1_L = max(0.0), (mu / denom_Y1) * (term_Q_diff - (factor_sq * term_mu_sub)))
12
13  # Eq 25: e1 upper bound
14  if Y1_L * (nu1 - nu2) <= 1e-20:
15      e1_U = 5.0
16  else:
17      num_e1 = (E_nu1_U * Q_nu1_U * math.exp(nu1)) - (E_nu2_L * Q_nu2_L * math.exp(nu2))
18      e1_U = min(5.0, max(0.0), num_e1 / ((nu1 - nu2) * Y1_L))
19

```

این پیاده‌سازی چند ویژگی مهم دارد: نخست، Y_0 به جای اندازه‌گیری مستقیم (که در `Vacuum + Weak` با decoy خلاً انجام می‌شد)، از ترکیب دو decoy ضعیف تخمین زده می‌شود (معادله ۱۸). این تخمین با استفاده از کران پایین Q_{ν_2} و کران بالا Q_{ν_1} انجام می‌شود تا کران پایین Y_0 محافظه‌کارانه باشد. دوم، در معادله (۲۱)، Y_0^L به جای Y_0 استفاده می‌شود تا جمله کم‌شونده بزرگ‌تر و کران Y_1 سخت‌گیرانه‌تر شود. سوم، در معادله (۲۵)، $E_{\nu_1} Q_{\nu_1}$ با کران بالا و $E_{\nu_2} Q_{\nu_2}$ با کران پایین جایگزین می‌شوند تا صورت کسر بزرگ‌تر و کران e_1 سخت‌گیرانه‌تر شود. این انتخاب‌های جهت‌دار در سراسر کد، تضمین می‌کنند که تمام کران‌ها در بدترین حالت نوسان آماری معتبر باشند.

۴.۴.۸ پیاده‌سازی پروتکل One-Decoy

کلاس `Ma2005OneDecoyProof` پروتکل تک decoy را با دو حالت (`zero-background` یا `standard`) پیاده‌سازی می‌کند. مانند سایر کلاس‌های `Ma 2005`، این کلاس نیز در ابتدای متد `estimate_yields_and_errors` فراخوانی `_normalize_stats_map` را انجام می‌دهد تا کلیدهای دیکشنری ورودی به فرمت استاندارد نام پالس تبدیل شوند. سپس بررسی می‌کند که منبع از نوع `Poisson` باشد (این کلاس از `Thermal` پشتیبانی نمی‌کند) و در صورت عدم تطابق، استثنای `ConfigurationError` را می‌شود. در حالت `zero-background`، فرض می‌شود $Y_0 = 0$ که یک تقریب محافظه‌کارانه برای کانال‌های با `dark count` بسیار پایین است:

```

1  if self.assume_zero_background:
2  # Eq. 41: Tighter Bound (Y0=0 implicitly)
3  prefactor = mu / (mu * nu - nu**2)
4  term_Q_nu = Q_nu_L * math.exp(nu)
5  term_Q_mu = Q_mu_U * math.exp(mu) * (nu**2 / mu**2)
6  Y1_L = max(0.0), prefactor * (term_Q_nu - term_Q_mu))

```

```

۷
۸     if Y1_L * nu <= 1e-20:
۹         e1_U = 5.0
۱۰        else:
۱۱            e1_U = min(5.0, max(0.0,
۱۲                (E_nu_U * Q_nu_U * math.exp(nu)) / (Y1_L * nu)))
۱۳            diag_msg = "Ma2005_OneDecoy_ZeroBackground"
۱۴

```

در حالت zero-background، فرمول کران Y_1 ساده‌تر می‌شود زیرا جمله Y_0 حذف می‌شود. این فرمول معادله (۴۱) مقاله را پیاده‌سازی می‌کند. در حالت استاندارد (با Y_0 غیر صفر)، فرمول کامل معادلات (۳۸) تا (۴۰) استفاده می‌شود که در آن Y_0 از روی $E_\mu Q_\mu$ تخمین زده می‌شود:

```

۱     else:
۲         # Standard One-Decoy Bound (Eq. 38-40)
۳         Y0_U = (E_mu_U * Q_mu_U * math.exp(mu)) / e0
۴
۵         term1 = Q_nu_L * math.exp(nu)
۶         term2 = Q_mu_U * math.exp(mu) * (nu**2 / mu**2)
۷         term3 = (Y0_U) * ((mu**2 - nu**2) / mu**2)
۸         Y1_L = max(0.0, (mu / (mu * nu - nu**2)) * (term1 - term2 - term3))
۹
۱۰        if Y1_L * mu <= 1e-20:
۱۱            e1_U = 5.0
۱۲        else:
۱۳            e1_U = min(5.0, max(0.0,
۱۴                (E_mu_U * Q_mu_U * math.exp(mu)) / (Y1_L * mu)))
۱۵            diag_msg = "Ma2005_OneDecoy_Simple"
۱۶

```

در حالت استاندارد، Y_0^U از روی خطای سیگنال تخمین زده می‌شود: $Y_0^U = E_\mu^U Q_\mu^U e^\mu / e_0$. این تخمین بر اساس این فرض است که تمام خطاهای مشاهده‌شده در سیگنال از dark count ناشی می‌شوند (که یک فرض محافظه‌کارانه است). سپس این Y_0^U در فرمول کران Y_1 استفاده می‌شود.

۵.۴.۸ پیاده‌سازی پروتکل چند Decoy با LP

کلاس Ma2005GeneralMultiDecoyProof برای حالت چند decoy (بیش از دو) از برنامه‌ریزی خطی (LP) برای یافتن کران‌های بهینه Y_1 و e_1 استفاده می‌کند. این تنها کلاس در ماژول است که از توزیع Thermal پشتیبانی می‌کند. مانند سایر کلاس‌های Ma 2005، این کلاس نیز در ابتدای متد estimate_yields_and_errors فراخوانی _normalize_stats_map را انجام می‌دهد. سپس ماتریس LP با استفاده از تمام پیکربندی‌های پالس موجود ساخته می‌شود. مسأله LP به صورت زیر فرمول‌بندی می‌شود:

$$\min_{\{Y_n\}} Y_1 \quad \text{s.t.} \quad Q_k^L \leq \sum_{n=0}^{N_{\text{cap}}} Y_n P(n | \mu_k) \leq Q_k^U, \quad \forall k. \quad (۲۳.۸)$$

در این فرمول‌بندی، $\{Y_n\}_{n=0}^{N_{\text{cap}}}$ متغیرهای تصمیم هستند، و Q_k^L و Q_k^U کران‌های آماری Gain در شدت k ، و $P(n | \mu_k)$ احتمال ارسال n فوتون با شدت μ_k است. این مسأله با استفاده از تابع solve_lp از ماژول qkd.proofs.utils_lp حل می‌شود. در ادامه، کد کلیدی این کلاس را ارائه می‌دهیم:

```

۱     c_y1 = np.zeros(n_cap + 1)
۲     c_y1[1] = 0.1 # Minimize Y_1
۳
۴     A_eq = np.zeros((k_intensities, n_cap + 1))
۵     b_eq = np.zeros(k_intensities)
۶     b_eq_L = np.zeros(k_intensities)
۷     b_eq_U = np.zeros(k_intensities)
۸
۹     valid_indices = []
۱۰    for i, pc in enumerate(pulse_configs):
۱۱        if pc.name not in stats_map or stats_map[pc.name].sent == 0:
۱۲            continue
۱۳        valid_indices.append(i)
۱۴        mu = pc.mean_photon_number

```



```

۱۵
۱۶ Q_L, Q_U, _, _ = self._get_bounds_stats(stats_map, pc.name, alpha)
۱۷ b_eq_L[i] = Q_L
۱۸ b_eq_U[i] = Q_U
۱۹
۲۰ for n in range(n_cap + 1):
۲۱     A_eq[i, n] = self._get_prob_n(mu, n)
۲۲

```

در این کد، بردار هزینه c به صورتی تنظیم می‌شود که Y_1 مینیمم شود (یعنی $c = [0, 1, 0, \dots, 0]$). سپس ماتریس محدودیت A_{eq} ساخته می‌شود که در آن سطر i معادله $\sum_n Y_n P(n | \mu_i) = Q_i$ را نشان می‌دهد. کران‌های بالا و پایین Q_i به صورت دو مجموعه از محدودیت‌های نامساوی اعمال می‌شوند: $\sum \leq Q^U$ و $-\sum \leq -Q^L$. این تبدیل استاندارد، مسئله با محدودیت‌های دوطرفه را به مسئله با محدودیت‌های یکطرفه تبدیل می‌کند که توسط solverهای استاندارد LP قابل حل است. سپس مسئله LP حل می‌شود و برای e_1 نیز یک مسئله مشابه حل می‌شود:

```

۱ A_ub = np.vstack([A_eq, -A_eq])
۲ b_ub = np.hstack([b_eq_U, -b_eq_L])
۳
۴ sol_y1, diag_y1 = solve_lp(c_y1, A_ub, b_ub, n_cap + 1,
۵ self.p.lp_solver_method)
۶ Y1_L = max(0.0), sol_y1[1])
۷
۸ # For e1: minimize -e1*Y1 (which maximizes e1*Y1)
۹ c_ey1 = np.zeros(n_cap + 1)
۱۰ c_ey1[1] = 0.1-
۱۱
۱۲ # ... build A_ub_err, b_ub_err similar to above ...
۱۳
۱۴ sol_ey1, diag_ey1 = solve_lp(c_ey1, A_ub_err, b_ub_err, n_cap + 1,
۱۵ self.p.lp_solver_method)
۱۶ e1Y1_U = max(0.0), sol_ey1[1])
۱۷
۱۸ if Y1_L <= 1e-20:
۱۹     e1_U = 5.0
۲۰ else:
۲۱     e1_U = min(5.0), e1Y1_U / Y1_L)
۲۲

```

در این بخش، مسئله LP برای Y_1 حل می‌شود و سپس یک مسئله LP دیگر برای $e_1 Y_1$ حل می‌شود (با مینیمم کردن $-e_1 Y_1$ که معادل ماکزیمم کردن $e_1 Y_1$ است). سپس $e_1 = (e_1 Y_1) / Y_1$ محاسبه می‌شود. این رویکرد دو مرحله‌ای ضروری است زیرا e_1 به صورت نسبی تعریف می‌شود و نمی‌تواند مستقیماً در LP مینیمم شود. اگر $Y_1 \leq 10^{-20}$ باشد، $e_1 = 0.5$ تنظیم می‌شود که معادل بدترین حالت است. این طراحی اجازه می‌دهد کاربر هر تعداد decoy را بدون محدودیت فرمول‌های تحلیلی استفاده کند.

۶.۴.۸ پیاده‌سازی حد Asymptotic

کلاس Ma2005AsymptoticProof حد نظری Infinite Decoy State را با معادلات (۲۷) و (۲۸) پیاده‌سازی می‌کند. این کلاس به عنوان مرجع بالای عملکرد عمل می‌کند:

```

۱ # 1. Estimate Q_mu
۲ Q_mu = signal_stats.sifted / signal_stats.sent
۳
۴ # 2. Derive eta from Q_mu
۵ Y0 = 0.2 * self.p.detector.dark_rate # Gap [B] fix: two-detector BB84
۶ e0 = 5.0
۷ e_det = self.p.detector.qber_intrinsic
۸
۹ if "vacuum" in stats_map and stats_map["vacuum"].sent > 0:
۱۰     vac_stats = stats_map["vacuum"]
۱۱     Y0 = vac_stats.sifted / vac_stats.sent
۱۲
۱۳ mu = self.p.source.get_pulse_config_by_name("signal").mean_photon_number
۱۴
۱۵ # Solve Q_mu = Y0 + 1 - exp(-eta * mu) for eta
۱۶ arg = 0.1 + Y0 - Q_mu

```

```

۱۷     if arg <= 0:
۱۸         eta = self.p.channel.transmittance * self.p.detector.det_eff_d0
۱۹     else:
۲۰         eta_effective = -math.log(arg) / mu
۲۱         eta = max(0.0, min(0.1), eta_effective))
۲۲
۲۳     # 3. Calculate Asymptotic Limits (Eq 27 and 28)
۲۴     Y1_asymptotic = Y0 + eta
۲۵
۲۶     if Y1_asymptotic > 0:
۲۷         numerator = (e0 * Y0) + (e_det * eta)
۲۸         e1_asymptotic = numerator / Y1_asymptotic
۲۹     else:
۳۰         e1_asymptotic = 5.0
۳۱
۳۲     e1_asymptotic = min(5.0, max(0.0), e1_asymptotic))
۳۳

```

در این کد، ابتدا Q_μ از آمار محاسبه می‌شود. سپس $Y_0 = 2 \cdot \text{dark_rate}$ تنظیم می‌شود که fix [B] Gap برای پشتیبانی از BB84 دو آشکارساز است. در BB84 استاندارد، باب از دو آشکارساز (یکی برای پایه Z و دیگری برای پایه X) استفاده می‌کند و هر دو آشکارساز در صورت عدم تطابق پایه، dark count تولید می‌کنند. بنابراین نرخ مؤثر dark count که در فرمول‌های تخمین ظاهر می‌شود، دو برابر نرخ تک‌آشکارساز است. عدم اعمال این ضریب ۲ در نسخه‌های قبلی کد منجر به تخمین خوش‌بینانه Y_1 و در نتیجه طول کلید ناامن می‌شد. سپس e_{det} از پیکربندی خوانده می‌شود. اگر آمار خلأ موجود باشد، Y_0 از آن استخراج می‌شود (که به واقعیت نزدیک‌تر است و مستقل از مقدار تئوریک dark_rate است). سپس η به صورت غیرمستقیم از روی Q_μ با معکوس کردن رابطه $Q_\mu = Y_0 + 1 - e^{-\eta\mu}$ استخراج می‌شود. اگر $1 + Y_0 - Q_\mu \leq 0$ (که ناهنجاری فیزیکی است و معمولاً ناشی از نویز بالا یا خطای کالیبراسیون است)، از مقدار تئوریک $\eta = \eta_{\text{channel}} \cdot \eta_{\text{det}}$ از پیکربندی به عنوان fallback استفاده می‌شود. در نهایت، Y_1 و e_1 با معادلات (۲۷) و (۲۸) محاسبه می‌شوند. این مقادیر به عنوان کران‌های بالای تئوریک عمل می‌کنند و اجازه می‌دهند کاربر ببیند در شرایط ایده‌آل (تعداد نامتناهی decoy و اندازه بلوک بی‌نهایت)، طول کلید چقدر می‌تواند باشد.

۷.۴.۸ پیاده‌سازی چارچوب Wang 2008

کلاس Wang2005Proof چارچوب Wang 2008 را با مدیریت خطای منبع پیاده‌سازی می‌کند. در ادامه، بخش کلیدی متد estimate_yields_and_errors این کلاس را بررسی می‌کنیم:

```

۱     delta = getattr(self.p.source, 'intensity_jitter', (0.0
۲     source_fidelity = getattr(self.p.source, 'source_fidelity', (0.1
۳
۴     if source_fidelity < :0.1
۵     delta = max(delta, 0.1 - source_fidelity)
۶
۷     # Bounds on intensities
۸     mu_U = mu * (1 + delta)
۹     mu_L = mu * (1 - delta)
۱۰    nu_U = nu * (1 + delta)
۱۱    nu_L = nu * (1 - delta)
۱۲

```

در این بخش، δ از دو منبع ممکن است به دست آید: یا مستقیماً از intensity_jitter خوانده می‌شود، یا از طریق source_fidelity به صورت غیرمستقیم محاسبه می‌شود. اگر $\text{fidelity} < 1$ باشد، $\delta = \max(\delta_{\text{jitter}}, 1 - F)$ تنظیم می‌شود. این mapping محافظه‌کارانه فرض می‌کند که انحراف از حالت coherent ایده‌آل، حداقل به اندازه $(1 - F)$ در شدت مؤثر اثر دارد. سپس کران‌های بالا و پایین شدت به صورت $\mu^L = \mu(1 - \delta)$ و $\mu^U = \mu(1 + \delta)$ محاسبه می‌شوند. این کران‌ها در سراسر فرمول‌های تخمین Y_1 و e_1 استفاده می‌شوند تا در بدترین حالت نوسان منبع، امنیت حفظ شود. سپس فرمول کران پایین Y_1 با کران‌های بدترین حالت اعمال می‌شود:

```

۱     term1 = Q_nu * math.exp(nu_L)
۲     term2 = Q_mu * math.exp(mu_U) * (nu_U**2) / (mu_L**2)
۳     term3 = (mu_U**2 - nu_L**2) / (mu_L**2) * Q_vac # Y0 approx Q_vac
۴
۵     numerator = term1 - term2 - term3
۶     denominator = mu_U * nu_L - nu_U**2

```

```

۷
۸     if denominator <= 0:
۹         Y1_L = 0.0
۱۰        else:
۱۱            Y1_L = (mu_L / denominator) * numerator
۱۲
۱۳        Y1_L = max(0.0), Y1_L)
۱۴

```

این پیاده‌سازی معادله (۵۶) مقاله Wang 2008 را با کران‌های بدترین حالت اعمال می‌کند. تفاوت کلیدی با فرمول Ma 2005 این است که تمام شدت‌ها با کران‌های بالا یا پایین جایگزین شده‌اند. اگر $\mu^U \nu^L - \mu^L \nu^U \leq 0$ باشد، $Y_1 = 0$ تنظیم می‌شود. در نهایت، $\max(0, \cdot)$ تضمین می‌کند که Y_1 هرگز منفی نشود. سپس کران بالای e_1 محاسبه می‌شود:

```

۱    term_err = E_nu * Q_nu * math.exp(nu_U)
۲    term_vac = E_vac * Q_vac # e0*Y0
۳
۴    if Y1_L * nu_L <= 0:
۵        e1_U = 5.0
۶    else:
۷        e1_U = (term_err - term_vac) / (Y1_L * nu_L)
۸
۹    e1_U = min(5.0), max(0.0), e1_U)
۱۰

```

در این فرمول، e^ν با e^{ν^U} (کران بالا) جایگزین شده تا صورت کسر بزرگ‌تر و کران e_1 سخت‌گیرانه‌تر شود. همچنین Y_1 و ν در مخرج با کران پایین Y_1^L و ν^L جایگزین شده‌اند. اگر $Y_1^L \cdot \nu^L \leq 0$ باشد، $e_1 = 0.5$ تنظیم می‌شود که معادل بدترین حالت است. در نهایت، نرخ کلید با فرمول استاندارد GLLP محاسبه می‌شود:

```

۱    mu = self.p.source.get_pulse_config_by_name("signal").mean_photon_number
۲    p1 = mu * math.exp(-mu)
۳
۴    Y1_L = decoy_estimates.yield_1_lower_bound
۵    e1_U = decoy_estimates.error_rate_1_upper_bound
۶
۷    Q_mu = stats.sifted / n_total
۸    E_mu = stats.errors_sifted / stats.sifted if stats.sifted > 0 else 5.0
۹
۱۰    gain_1 = Y1_L * p1
۱۱
۱۲    from ..utils.math import binary_entropy
۱۳    pa_factor = 0.1 - binary_entropy(e1_U)
۱۴    secure_gain = gain_1 * pa_factor
۱۵
۱۶    leakage = Q_mu * self.p.f_error_correction * binary_entropy(E_mu)
۱۷
۱۸    rate = 5.0 * (secure_gain - leakage) # q5.0= for BB84
۱۹
۲۰    key_len = max(0, int(rate * n_total))
۲۱

```

در این بخش، ابتدا $P(1 | \mu) = \mu e^{-\mu}$ محاسبه می‌شود (با فرض Poisson). سپس Gain تک‌فوتونی $Q_1 = Y_1^L \cdot P(1 | \mu)$ محاسبه می‌شود. ترم تقویت حریم خصوصی به صورت $Q_1 \cdot (1 - H_2(e_1^U))$ و ترم نشت تصحیح خطا به صورت $Q_\mu \cdot f_{EC} \cdot H_2(E_\mu)$ محاسبه می‌شوند. نرخ نهایی $R = 0.5 \cdot (PA - EC)$ (با $q = 0.5$) برای BB84 و طول کلید به صورت $\max(0, \lfloor R \cdot N \rfloor)$ محاسبه می‌شود.

۵.۸ ارسال و دریافت با سایر ماژول‌ها

۱.۵.۸ ورودی‌های ماژول ma2005.py

ماژول ma2005.py برای انجام محاسبات خود به چندین ورودی از سایر ماژول‌های فریم‌ورک نیاز دارد. این ورودی‌ها به دو دسته پیکربندی و آمار زمان اجرا تقسیم می‌شوند. ورودی‌های پیکربندی در زمان ساخت شیء

اثبات امنیتی خوانده می‌شوند و در طول اجرا ثابت می‌مانند، در حالی که ورودی‌های آمار در هر فراخوانی `calculate_key_length` به روز می‌شوند.

- `QKDParams`: این شیء از ماژول `qkd.params` وارد می‌شود و شامل تمام پارامترهای پیکربندی سیستم است. زیرشئ‌های کلیدی که این ماژول استفاده می‌کند عبارتند از: `p.source` (شامل `p.detector`, `(source_fidelity و intensity_jitter, statistics_type, pulse_configs)` (شامل `det_eff_d0`, `dark_rate` و `qber_intrinsic`), `p.channel` (شامل `transmittance`), `p.f_error_correction`, `p.eps_cor`, `p.eps_smooth`, `p.eps_pe`, `p.eps_sec`, `p.ci_method` و `p.lp_solver_method`, `p.photon_number_cap`, `p.f_ec_dynamic_config`
- `stats_map`: این یک دیکشنری از `str` به `TallyCounts` است که از لایه شبیه‌ساز آشکارساز به اثبات امنیتی داده می‌شود. کلیدهای این دیکشنری نام شدت‌ها ("signal", "decoy", "vacuum") و در حالت چند `decoy` (نام‌های اختصاصی) هستند و مقادیر، شیء `TallyCounts` با فیلدهای `sent` (تعداد پالس‌های ارسالی)، `sifted` (تعداد آشکارسازی‌های پایه درست) و `errors_sifted` (تعداد خطاها در پایه درست) هستند.
- `EpsilonAllocation`: این شیء از ماژول `qkd.datatypes` وارد می‌شود و شامل فیلدهای `εcor`, `εsec`, `εpe`, `εsmooth` و `εphase_est` است. این شیء از متد `allocate_epsilon` تولید می‌شود و در محاسبه کران‌های آماری استفاده می‌شود.

۲.۵.۸ ورودی‌های ماژول wang2005.py

ماژول `wang2005.py` ورودی‌های مشابهی با `ma2005.py` دارد اما به دو فیلد اضافی در پیکربندی منبع وابسته است:

- `p.source.intensity_jitter`: این فیلد کران نوسان شدت منبع را مشخص می‌کند. در ماژول `wang2005.py`، این فیلد به صورت δ در فرمول‌های کران شدت ($\mu^L = \mu(1 - \delta)$ و $\mu^U = \mu(1 + \delta)$) استفاده می‌شود. اگر این فیلد تنظیم نشده باشد، مقدار پیش‌فرض ۰.۰ استفاده می‌شود که معادل منبع ایده‌آل است.
- `p.source.source_fidelity`: این فیلد نشان می‌دهد که حالت واقعی منبع چقدر به یک `coherent state` ایده‌آل نزدیک است. مقدار 1.0 به معنای منبع ایده‌آل و مقدار کمتر از 1.0 به معنای انحراف از حالت ایده‌آل است. اگر این فیلد کمتر از 1.0 باشد، ماژول یک `mapping` محافظه‌کارانه به صورت $\delta = \max(\delta_{jitter}, 1 - F)$ انجام می‌دهد.

این دو فیلد با استفاده از تابع `getattr` با مقدار پیش‌فرض خوانده می‌شوند تا در صورت عدم وجود در پیکربندی، ماژول با خطا مواجه نشود. این طراحی انعطاف‌پذیر اجازه می‌دهد ماژول با نسخه‌های مختلف پیکربندی سازگار باشد.

۳.۵.۸ خروجی‌های ماژول

خروجی اصلی هر دو ماژول یک شیء `KeyCalculationResult` است که از کلاس پایه `FiniteKeyProof` به ارث رسیده. این شیء شامل فیلدهای زیر است:

1. `secure_key_length`: طول کلید امن به صورت یک عدد صحیح غیرمنفی. این فیلد خروجی نهایی محاسبه است و توسط لایه بالایی برای تولید کلید واقعی استفاده می‌شود.
2. `privacy_amplification_term`: ترم تقویت حریم خصوصی به صورت $Q_1(1 - H_2(e_1)) \cdot N_{\text{sent}}$. این فیلد برای تحلیل و گزارش‌گیری مفید است.
3. `error_correction_leakage`: نشت تصحیح خطا به صورت $Q_\mu f_{\text{EC}} H_2(E_\mu) \cdot N_{\text{sent}}$. این فیلد نشان می‌دهد چه مقدار اطلاعات در مرحله تصحیح خطا به مهاجم نشت کرده است.
4. `phase_error_rate_upper_bound`: کران بالای خطای فاز e_1^U . این فیلد برای تحلیل امنیتی و مقایسه با آستانه امنیت BB84 (حدود ۱۱٪) استفاده می‌شود.

۵. diagnostics: یک دیکشنری شامل تمام مقادیر میانی مانند $e_1^U, Y_1^L, Q_1, E_\mu, Q_\mu$ ، نرخ هر پالس، f_{EC} استفاده‌شده، نرخ Weak GLLP، Δ و اطلاعات solver. این فیلد برای debugging و بازبینی امنیتی ضروری است.

۶. error_codes: لیستی از کدهای خطا (مانند INSUFFICIENT_STATISTICS) که در صورت بروز مشکل، علت آن را نشان می‌دهند.

۷. weak_gllp_rate و tagged_fraction_bound: فیلدهای اضافی که در صورت استفاده از حالت Weak GLLP، اطلاعات بیشتری ارائه می‌دهند.

علاوه بر KeyCalculationResult، متد estimate_yields_and_errors یک شیء DecoyEstimates (یا DecoyEstimatesWang2005 در مازول wang2005.py) را برمی‌گرداند که شامل e_1^U, Y_1^L ، is_feasible، failure_prob_used و diagnostics است. این شیء برای کاربرانی که می‌خواهند تخمین‌های میانی را بدون محاسبه کامل طول کلید بررسی کنند، مفید است.

۴.۵.۸ وابستگی‌های داخلی و خارجی

هر دو مازول وابستگی‌های داخلی و خارجی متعددی دارند. وابستگی‌های داخلی شامل موارد زیر است:

- qkd.proofs.base: کلاس پایه KeyCalculationResult، DecoyEstimates، FiniteKeyProof، SolverDiagnostics و ErrorCode از این مازول وارد می‌شوند. این کلاس پایه رابط عمومی و خدمات مشترک مانند آنتروپی باینری، clamp عددی و گزارش‌گیری audit را فراهم می‌سازد.
 - qkd.proofs.utils_lp: تابع solve_lp برای حل مسئله برنامه‌ریزی خطی در کلاس Ma2005GeneralMultiDecoyProof وارد می‌شود. این تابع از scipy.optimize.linprog یا cvxopt به عنوان زیرین استفاده می‌کند.
 - qkd.datatypes: TallyCounts، EpsilonAllocation، SourceStatisticsType و ConfidenceBoundMethod از این مازول وارد می‌شوند. SourceStatisticsType یک enum است که مقادیر POISSON و THERMAL را دارد و ConfidenceBoundMethod یک enum است که روش‌های فاصله اطمینان را مشخص می‌کند.
 - qkd.exceptions: استثناهای سفارشی ConfigurationError، LPFailureError و ParameterValidationError برای گزارش خطاهای ساختاریافته.
 - qkd.utils.math: تابع binary_entropy برای محاسبه آنتروپی باینری $H_2(p) = -p \log_2 p - (1-p) \log_2 (1-p)$.
 - qkd.params.QKDParams: کلاس پارامترهای QKD که شامل تمام پارامترهای پیکربندی سیستم است.
- وابستگی‌های خارجی شامل numpy برای محاسبات برداری در مسیر LP، math برای توابع ریاضی پایه (مانند factorial، log، exp) و logging برای ثبت رویدادها و هشدارها هستند.

۵.۵.۸ جریان داده در سراسر فریم‌ورک

جریان داده کلی به این شکل است:

۱. لایه شبیه‌ساز پالس‌های نوری را با شدت‌های μ (سیگنال)، ν (decoy ضعیف) و $\emptyset$ (خلأ) تولید و از کانال کوانتومی عبور می‌دهد.
۲. آشکارسازها در سمت باب، آشکارسازی‌ها و خطاها را برای هر شدت ثبت کرده و در TallyCounts ذخیره می‌کنند.
۳. این شمارش‌ها به صورت یک دیکشنری stats_map با کلیدهای نام شدت به اثبات امنیتی داده می‌شوند.
۴. متد allocate_epsilon در زمان ساخت شیء فراخوانی می‌شود و بودجه ϵ را میان اجزای مختلف تقسیم می‌کند.

۵. متد `estimate_yields_and_errors` با دریافت `stats_map`، کران‌های e_1^U و Y_1^L را با استفاده از فرمول‌های تحلیلی یا LP محاسبه می‌کند.

۶. متد `calculate_key_length` با دریافت تخمین‌ها و `stats_map`، نرخ کلید را با فرمول GLLP محاسبه می‌کند و یک شیء `KeyCalculationResult` برمی‌گرداند.

۷. لایه بالایی می‌تواند با متدهای `explain_key_decision` و `audit_dump` (از کلاس پایه) علت نتیجه را به صورت متنی دریافت کند و یک گزارش کامل برای بازبینی امنیتی تولید کند.

این جداسازی مراحل به گونه‌ای طراحی شده که هر مرحله قابل آزمایش و بازبینی مستقل باشد. به عنوان مثال، می‌توان `DecoyEstimates` را به صورت دستی ساخت و رفتار مرحله نهایی محاسبه کلید را در شرایط خاص بررسی کرد. این مازول‌بندی از اصل `Single Responsibility` پیروی می‌کند و قابلیت نگهداری و توسعه کد را به طور قابل توجهی افزایش می‌دهد. علاوه بر این، شفافیت کامل در `diagnostics` به کاربر اجازه می‌دهد هر مرحله از محاسبه را ردیابی کرده و در صورت نیاز، فرضیات تئوریک را برای کاربرد خاص خود بازبینی کند.

۶.۵.۸ انتخاب میان Ma 2005 و Wang 2008

یکی از تصمیمات مهم در استفاده از این فریم‌ورک، انتخاب میان مازول `ma2005.py` و `wang2005.py` است. این انتخاب بر اساس شرایط واقعی منبع نوری انجام می‌شود:

- اگر منبع ایده‌آل باشد (یعنی `coherent state` با شدت دقیقاً مشخص و بدون نوسان)، مازول `ma2005.py` با یکی از پنج کلاس (مثلاً `Ma2005VacuumWeakProof`) کافی است و نرخ کلید بالاتری تولید می‌کند زیرا فرض‌های سخت‌گیرانه‌تری دارد.
- اگر منبع دارای نوسان شدت یا انحراف از حالت `coherent` باشد، مازول `wang2005.py` باید استفاده شود زیرا این نوسانات را در فرمول‌های کران لحاظ می‌کند. استفاده از `ma2005.py` در این حالت می‌تواند منجر به کران‌های خوش‌بینانه (نامن) شود.
- اگر تعداد `decoy`ها بیش از دو باشد، تنها `Ma2005GeneralMultiDecoyProof` از مازول `ma2005.py` قابل استفاده است (زیرا `wang2005.py` فقط پروتکل `Vacuum + Weak` را پشتیبانی می‌کند).
- اگر منبع از نوع `Thermal` باشد، تنها `Ma2005GeneralMultiDecoyProof` قابل استفاده است (سایر کلاس‌ها در صورت `Thermal` بودن، استثنای `raise ConfigurationError` می‌کنند).
- اگر کاربر بخواهد حد نظری `Infinite Decoy` را به عنوان مرجع بالای عملکرد محاسبه کند، `Ma2005AsymptoticProof` مناسب است.

این انعطاف‌پذیری در انتخاب، به کاربر اجازه می‌دهد تحلیل امنیتی متناسب با شرایط خاص سیستم خود را انتخاب کند و در عین حال، از کران‌های قابل اثبات ریاضی بهره‌مند شود. طراحی مازولار همچنین اجازه می‌دهد در آینده، چارچوب‌های جدید (مانند Lim 2014 یا اثبات‌های مبتنی بر MDI-QKD) به سادگی با ارتجری از `FiniteKeyProof` اضافه شوند بدون آن‌که کد موجود تغییر کند.

فصل ۹

اثبات امنیت فشرده

۱.۹ نمای کلی و هدف ماژول

فایل `qkd/proofs/tight.py` پیاده‌سازی مرجع و تولید-آماده (production-grade) اثبات امنیتی کلید متناهی BB84-Tight است که از چارچوب Lim et al. 2014 مشتق شده و به طور گسترده برای رفع استانداردهای تولید در زمینه‌های صحت‌سنجی، یکپارچگی API، پایداری عددی، تشخیص‌گری (diagnostics) و ردپایی (provenance) بازطراحی شده است. این ماژول بر خلاف پیاده‌سازی‌های اولیه‌ی آکادمیک که صرفاً به دنبال بازتولید فرمول‌های مقاله بودند، با هدف قرار گرفتن در محیط‌های عملیاتی واقعی توسعه یافته است؛ محیط‌هایی که در آن‌ها خطاهای عددی، نوسانات ورودی و رفتار غیرعادی سیستم نباید منجر به فروپاشی خط لوله شبیه‌سازی شوند. نسخه‌ی فعلی با شناسه 3.7.6-fixed علامت‌گذاری شده است که نشان‌دهنده‌ی اصلاح یک باگ منطقی حیاتی در تخصیص بودجه امنیت است؛ این باگ پیش از آن، در ریاضیات ممیز شناور ۶۴-بیت، می‌توانست منجر به نقض نامرئی نامساوی امنیتی ϵ_{sec} شود.

هدف اصلی این ماژول، تبدیل شمارش‌های آماری شبکه آشکارسازی (در قالب شیء `TallyCounts`) و تخمین‌های `Decoy-State` (در قالب شیء `DecoyEstimatesInput` یا هر شیء دارای فیلدهای `yield_1_lower_bound` و `error_rate_1_upper_bound`) به یک عدد صحیح مثبت به عنوان طول کلید امن نهایی است. این تبدیل با رعایت سخت‌گیرانه تمام محافظه‌کاری‌های ریاضی، آماری و فیزیکی انجام می‌شود تا هیچ طول کلیدی که از نظر تئوری قابل اثبات نیست، تولید نشود. فلسفه طراحی این ماژول بر پایه اصل “fail-safe, not fail-fast” استوار است؛ به این معنا که در صورت بروز هرگونه ناسازگاری پیکربندی، آمار ناکافی، یا خطای عددی، خروجی به جای `raise` کردن استثنا و متوقف کردن خط لوله، یک شیء `KeyCalculationResult` با طول کلید صفر و کد خطای مناسب تولید می‌کند. این رفتار برای کاربردهای تولیدی که در آن‌ها قطع شبیه‌سازی هزینه‌بر است، حیاتی است و اجازه می‌دهد خط لوله ادامه یابد در حالی که علت شکست در گزارش تشخیصی ثبت می‌گردد.

این ماژول از کلاس پایه `Lim2014Proof` (که در فایل `qkd/proofs/lim2014.py` تعریف شده) ارث می‌برد و متدهای `allocate_epsilon` و `calculate_key_length` را بازنویسی می‌کند. ارث‌بری از `Lim2014Proof` به این ماژول اجازه می‌دهد تا تمام زیرساخت‌های مشترک مانند محاسبه آنتروپی باینری، تخصیص ϵ پایه، و ابزارهای `clamp` عددی را بدون تکرار کد به کار گیرد و در عین حال، منطق محاسبه کلید را با رویکردی متفاوت و سخت‌گیرانه‌تر بازنویسی کند. تفاوت کلیدی میان `BB84TightProof` و کلاس پایه `Lim2014Proof` در این است که نسخه `Tight` از یک رویکرد “entropy-budget” استفاده می‌کند که در آن، آنتروپی موجود از پایه تولیدکننده کلید (X) محاسبه شده و سپس نشت تصحیح خطا و جریمه‌های تقویت حریم خصوصی از آن کم می‌شوند، در حالی که نسخه پایه از فرمول مستقیم نرخ کلید استفاده می‌کند. این رویکرد آنتروپی-محور، تطابق دقیق‌تری با ادبیات مدرن امنیتی ترکیبی (composable security) دارد.

یکی از ویژگی‌های متمایزکننده این ماژول، پشتیبانی از هر دو حالت `Decoy-State` و `non-Decoy` است. در حالت `Decoy-State`، ماژول تخمین‌های Y_1 و e_1 را از موتور تخمین `Decoy` دریافت کرده و از آن‌ها برای محاسبه شمارش تک‌فوتونی در هر دو پایه X و Z استفاده می‌کند. در حالت `non-Decoy`، ماژول به صورت محافظه‌کارانه فرض می‌کند که تمام بیت‌های sifted از تک‌فوتون‌ها نشأت گرفته‌اند و از QBER پایه Z به عنوان تخمین خطای فاز استفاده می‌کند. این انعطاف‌پذیری اجازه می‌دهد ماژول هم برای شبیه‌سازی‌های پیشرفته با `Decoy-State` و هم برای شبیه‌سازی‌های ساده BB84 پایه به کار رود. علاوه بر این، ماژول از یک طرح غنی گزارش‌گیری `diagnostics` استفاده می‌کند که در آن تمام مقادیر میانی محاسبه (مانند $m_Z, m_X, s_{X,1}^L, s_{Z,1}^L$ ، QBER، نشت تصحیح خطا، خطای فاز، جریمه‌های PA و آنتروپی موجود) در یک دیکشنری ساختاریافته

ذخیره می‌شوند و برای تحلیل و بازبینی امنیتی توسط شخص ثالث قابل دسترس هستند.

۲.۹ مبانی تئوری و فیزیکی

۱.۲.۹ امنیت ترکیبی و بودجه‌ی ε

در چارچوب مدرن امنیت ترکیبی (composable security) که توسط Canetti (۲۰۰۱) و Ben-Or, Mayers (۲۰۰۴) توسعه یافته، امنیت یک پروتکل QKD به صورت یک پارامتر کوچک ε_{sec} تعریف می‌شود که نشان‌دهنده حداکثر احتمال شکست امنیتی است. یک پروتکل ε_{sec} -امن است اگر خروجی آن از یک کلید ایده‌آل (توزیع یکنواخت و کاملاً مخفی) با فاصله‌ی ε_{sec} در معیار trace distance فاصله داشته باشد. این تعریف اجازه می‌دهد امنیت QKD به صورت قابل ترکیب با سایر پروتکل‌های رمزنگاری تحلیل شود. کل بودجه‌ی ε_{sec} باید میان چندین رویداد شکست مستقل تقسیم شود:

$$\varepsilon_{\text{sec}} \geq \varepsilon_{\text{PE}} + 2\varepsilon_{\text{smooth}} + \varepsilon_{\text{PA}} + \varepsilon_{\text{cor}} + \varepsilon_{\text{phase_est}}. \quad (1.9)$$

در این نامساوی، ε_{PE} بودجه‌ی تخمین پارامتر، $\varepsilon_{\text{smooth}}$ بودجه‌ی هموارسازی (smoothing) که دو بار استفاده می‌شود (یک بار برای آنتروپی حداقل هموار و یک بار برای آنتروپی حداقل شرطی هموار)، ε_{PA} بودجه‌ی تقویت حریم خصوصی (Privacy Amplification)، ε_{cor} بودجه‌ی تصحیح خطا (Error Correction) و $\varepsilon_{\text{phase_est}}$ بودجه‌ی تخمین خطای فاز است. در کد ماژول tight.py، این تقسیم در متد allocate_epsilon پیاده‌سازی شده و با فراخوانی allocation.validate() اعتبارسنجی می‌شود. یک نکته ظریف و حیاتی این است که در ریاضیات ممیز شناور ۶۴-بیت، مجموع $\varepsilon_{\text{cor}} + (\varepsilon_{\text{sec}} - \varepsilon_{\text{cor}})$ ممکن است به دلیل خطای گردکردن، کمی بزرگتر از ε_{sec} شود و اعتبارسنجی شکست بخورد. برای رفع این مشکل، ماژول یک حاشیه امنیتی مطلق به اندازه‌ی 10^{-20} از بودجه‌ی باقی‌مانده کم می‌کند تا تضمین شود نامساوی به صورت عددی برقرار می‌ماند.

۲.۲.۹ خطای فاز در مقابل خطای بیت

در پروتکل BB84، دو نوع خطا تعریف می‌شود: خطای بیت (bit error) و خطای فاز (phase error). خطای بیت، خطایی است که در پایه اندازه‌گیری قابل مشاهده است و با مقایسه بیت‌های آلیس و باب در همان پایه اندازه‌گیری می‌شود. در مقابل، خطای فاز خطایی است که اگر آلیس و باب در پایه مزدوج (conjugate basis) اندازه‌گیری می‌کردند، قابل مشاهده می‌بود. این دو خطا در یک کانال بدون نویز برابرند، اما در حضور حمله‌های کوانتومی مانند PNS یا حمله‌های coherent می‌توانند متفاوت باشند. رابطه ریاضی میان خطای فاز و خطای بیت با استفاده از نامساوی Hoeffding یا Azuma برقرار می‌شود. در این ماژول، خطای فاز از پایه Z تخمین زده می‌شود و خطای بیت از پایه X (که پایه تولیدکننده کلید است).

نامساوی Hoeffding برای تخمین فاصله میان خطای فاز و خطای بیت به صورت زیر اعمال می‌شود:

$$e_{\text{phase}} \leq e_{\text{bit}} + \sqrt{\frac{-\ln(\varepsilon_{\text{phase_est}})}{2s_{Z,1}^L}}, \quad (2.9)$$

که در آن $e_{\text{bit}} = e_1^U$ کران بالای خطای بیت تک‌فوتونی، $\varepsilon_{\text{phase_est}}$ بودجه‌ی تخمین خطای فاز و $s_{Z,1}^L$ کران پایین شمارش تک‌فوتونی در پایه Z است. این نامساوی نشان می‌دهد که هرچه شمارش تک‌فوتونی در پایه Z بیشتر باشد، عدم قطعیت در تخمین خطای فاز کمتر می‌شود. در کد، این فرمول در متد compute_phase_error_ub پیاده‌سازی شده و در صورت بزرگتر بودن کران نهایی از ۵.۰ (حداکثر آنتروپی باینری)، به ۵.۰ بریده می‌شود. علاوه بر این، اگر پرچم assume_phase_equals_bit_error در پیکربندی فعال باشد، ماژول به جای محاسبه نامساوی Hoeffding، فرض می‌کند $e_{\text{phase}} = e_{\text{bit}}$ که یک تقریب ساده‌تر اما محافظه‌کارانه‌تر در شرایطی است که شمارش پایه Z ناکافی است.

۳.۲.۹ آنتروپی باینری و برش عددی

آنتروپی باینری $H_2(p)$ یک تابع کلیدی در تمام محاسبات امنیتی QKD است و به صورت زیر تعریف می‌شود:

$$H_2(p) = -p \log_2(p) - (1-p) \log_2(1-p). \quad (3.9)$$

این تابع در $p = 0.5$ به حداکثر مقدار ۱ می‌رسد و در $p = 0$ یا $p = 1$ به صفر میل می‌کند. در محاسبه طول کلید، آنتروپی باینری در دو جایگاه ظاهر می‌شود: نخست، در محاسبه نشت تصحیح خطا (EC leakage) به صورت

که نشان‌دهنده تعداد بیت‌هایی است که در فرآیند تصحیح خطا به باب نشت می‌کنند و در نتیجه توسط مهاجم قابل دسترس هستند؛ دوم، در محاسبه آنتروپی تک‌فوتونی به صورت $s_{X,1}^L \cdot (1 - H_2(e_{\text{phase}}))$ که نشان‌دهنده آنتروپی قابل استخراج از تک‌فوتون‌های پایه تولیدکننده کلید است.

یک مشکل عددی ظریف این است که آنتروپی باینری در $p \rightarrow 0$ یا $p \rightarrow 1$ به سمت صفر میل می‌کند، اما عبارت $\log_2(p)$ به سمت $-\infty$ میل می‌کند. در ریاضیات ممیز شناور، این می‌تواند منجر به ضرب یک عدد بسیار کوچک در یک عدد بسیار بزرگ شود و نتیجه‌ی ناپایدار یا حتی NaN تولید کند. برای رفع این مشکل، ماژول از یک تابع `_binary_entropy_safe` استفاده می‌کند که مقدار p را با استفاده از ثابت `ENTROPY_PROB_CLAMP` (که معمولاً 10^{-12} است) به بازه $[\varepsilon, 1 - \varepsilon]$ بریده می‌کند:

$$p_{\text{clamped}} = \max(\varepsilon_{\text{clamp}}, \min(p, 1 - \varepsilon_{\text{clamp}})). \quad (۴.۹)$$

این برش عددی، در حالی که به طور ظاهری کوچک است، تضمین می‌کند که تمام محاسبات آنتروپی پایدار و قابل بازتولید باشند. این برش همچنین از تقسیم بر صفر در عبارت $\log_2(p)$ جلوگیری می‌کند. در ادبیات امنیتی، این برش به عنوان یک "بهای عددی" شناخته می‌شود که در صورتی که $\varepsilon_{\text{clamp}} \ll \varepsilon_{\text{sec}}$ باشد، از نظر تئوری ناچیز است.

۴.۲.۹ آنتروپی موجود و معادله کلید نهایی

فرمول اصلی محاسبه طول کلید امن در چارچوب BB84-Tight به صورت زیر است:

$$\ell = \underbrace{s_{X,1}^L \cdot (1 - H_2(e_{\text{phase}}))}_{\text{entropy available}} - \underbrace{f_{\text{EC}} \cdot H_2(e_{\text{bit}}) \cdot n_{\text{key}}}_{\text{leakage EC}} - \underbrace{\left(\log_2 \frac{1}{2\varepsilon_{\text{smooth}}} + \log_2 \frac{1}{\varepsilon_{\text{PA}}} + \log_2 \frac{2}{\varepsilon_{\text{cor}}} \right)}_{\text{penalties correction \& PA}}. \quad (۵.۹)$$

در این معادله، ℓ طول کلید امن نهایی، $s_{X,1}^L$ کران پایین شمارش تک‌فوتونی در پایه X (پایه تولیدکننده کلید)، e_{phase} کران بالای خطای فاز، e_{bit} کران بالای خطای بیت، n_{key} تعداد بیت‌های sifted در پایه X ، f_{EC} بازدهی تصحیح خطا و $\varepsilon_{\text{smooth}}$ ، ε_{PA} و ε_{cor} بودجه‌های امنیتی مربوطه هستند. ترم آنتروپی موجود نشان‌دهنده حداکثر اطلاعات قابل استخراج از تک‌فوتون‌های امن است. ترم نشت تصحیح خطا نشان‌دهنده اطلاعاتی است که در فرآیند تصحیح خطا به باب نشت می‌کند و توسط مهاجم قابل رهگیری است. در نهایت، ترم جریمه‌های PA و تصحیح، بیت‌های ثابتی هستند که در فرآیند تقویت حریم خصوصی و تأیید تصحیح خطا از دست می‌روند. این فرمول در متد `calculate_key_length` پیاده‌سازی شده است. یک نکته کلیدی این است که اگر $\ell < 0$ شود (یعنی نشت و جریمه‌ها از آنتروپی موجود بیشتر شوند)، طول کلید به صفر برده می‌شود. این رفتار با استفاده از متد `_clamp_key_length` از کلاس پایه تضمین می‌شود. علاوه بر این، هر مقدار میانی غیرممتناهی (NaN یا Inf) باعث ثبت کد خطای `NON_FINITE_VALUE` می‌شود. این بررسی‌ها در متد `_finite_or_error` پیاده‌سازی شده‌اند.

۵.۲.۹ تخمین شمارش تک‌فوتونی در دو پایه

یکی از چالش‌های کلیدی در محاسبه طول کلید، تخمین شمارش تک‌فوتونی در دو پایه X و Z است. در حالت Decoy-State، این شمارش‌ها به صورت زیر تخمین زده می‌شوند:

$$s_{X,1}^L = N_{\text{sent},X} \cdot P(1 | \mu) \cdot Y_1^L \cdot p_X^A \cdot p_X^B, \quad (۶.۹)$$

$$s_{Z,1}^L = N_{\text{sent},Z} \cdot P(1 | \mu) \cdot Y_1^L \cdot p_Z^A \cdot p_Z^B, \quad (۷.۹)$$

که در آن $N_{\text{sent},X}$ و $N_{\text{sent},Z}$ تعداد پالس‌های ارسالی در پایه X و Z ، $P(1 | \mu) = \mu e^{-\mu}$ احتمال ارسال تک‌فوتون با شدت μ ، Y_1^L کران پایین بازده تک‌فوتونی (که فرض می‌شود مستقل از پایه است)، p_X^A و p_X^B احتمال انتخاب پایه X توسط آلایس و باب، و به طور متقابل p_Z^A و p_Z^B احتمال انتخاب پایه Z است. این فرمول‌ها در متد `compute_base_counts_decoy` پیاده‌سازی شده‌اند. یک نکته محافظه‌کارانه این است که اگر $s_{X,1}^L > n_X$ شود (که از نظر فیزیکی غیرممکن است زیرا تعداد تک‌فوتون‌ها نمی‌تواند از کل آشکارسازی‌ها بیشتر باشد)، مقدار به n_X بریده می‌شود. این برش با استفاده از ثابت `NUMERIC_ABS_TOL` انجام می‌شود تا خطاهای کوچک گردکردن نادیده گرفته شوند.

در حالت non-Decoy ماژول به صورت محافظه‌کارانه فرض می‌کند که تمام بیت‌های sifted از تک‌فوتون‌ها نشأت گرفته‌اند، یعنی $s_{X,1}^L = n_X$ و $s_{Z,1}^L = n_Z$. این فرض، در حالی که در حضور پالس‌های چندفوتونی ناامن

است، برای شبیه‌سازی‌های BB84 ایده‌آل (با $\mu \rightarrow 0$) قابل قبول است. در این حالت، خطای فاز به طور مستقیم از QBER پایه Z تخمین زده می‌شود: $e_{\text{bit}} = m_Z/n_Z$.

۶.۲.۹ جریمه‌های PA و تصحیح

جریمه‌های PA و تصحیح، بیت‌های ثابتی هستند که در فرآیند تقویت حریم خصوصی و تأیید تصحیح خطا از دست می‌روند. این جریمه‌ها به صورت زیر محاسبه می‌شوند:

$$\text{penalty PA} = \log_2\left(\frac{1}{2\varepsilon_{\text{smooth}}}\right) + \log_2\left(\frac{1}{\varepsilon_{\text{PA}}}\right), \quad \text{penalty Correction} = \log_2\left(\frac{2}{\varepsilon_{\text{cor}}}\right). \quad (۸.۹)$$

این جریمه‌ها در متد `_compute_pa_terms` پیاده‌سازی شده‌اند. در این متد، ابتدا هر مقدار ε با استفاده از متد `_log_inv_clamp` برش داده می‌شود تا از $\log_2(0)$ یا مقادیر بسیار ناپایدار جلوگیری شود. این متد، اگر مقدار ورودی کمتر از `_TIGHT_PROOF_EPS_MIN` (که برابر 10^{-300} است) باشد، آن را به این حداقل برش می‌دهد. این برش اطمینان می‌دهد که حتی در صورت تنظیم $\varepsilon = 0$ به اشتباه، محاسبات منجر به $-\infty$ نشوند. در عمل، این جریمه‌ها معمولاً در حدود چند ده بیت هستند و در شبیه‌سازی‌های با کلیدهای طولانی (میلیون‌ها بیت) ناچیز هستند، اما در کلیدهای کوتاه می‌توانند تأثیر قابل توجهی بر طول کلید نهایی داشته باشند.

۳.۹ تحلیل معماری و ساختار داده‌ها

۱.۳.۹ سلسله کلاس‌ها و وارث

ماژول `tight.py` از یک طراحی سلسله‌مراتبی استفاده می‌کند که در آن کلاس اصلی `BB84TightProof` از کلاس `Lim2014Proof` (که خود از `FiniteKeyProof` انتزاعی ارث می‌برد) ارث می‌برد. این طراحی چند لایه به ماژول اجازه می‌دهد تا از زیرساخت‌های مشترک مانند محاسبه آنتروپی باینری، تخصیص پایه ε و ابزارهای `clamp` عددی بهره‌برد و در عین حال، منطق محاسبه کلید را با رویکردی متفاوت بازنویسی کند. در ادامه، ساختار اصلی کلاس را به صورت خلاصه نمایش می‌دهیم:

```
1 class BB84TightProof(Lim2014ProofBase):
2     __implementation_version__ = "6.7.3-fixed"
3
4     def __repr__(self) -> str:
5         return f"BB84TightProof(impl_version='{self.__implementation_version__}')"
6
```

استفاده از ویژگی کلاس `__implementation_version__` به عنوان یک شناسه نسخه، اجازه می‌دهد تا در زمان اجرا، نسخه دقیق پیاده‌سازی مورد استفاده را شناسایی کنیم. این ویژگی برای ردیابی پروونانس (provenance) در شبیه‌سازی‌های تولیدی حیاتی است، زیرا به کاربر اجازه می‌دهد بداند کدام نسخه اثبات امنیتی نتایج یک شبیه‌سازی خاص را تولید کرده است. متد `__repr__` نیز یک نمایش متنی خوانا از کلاس ارائه می‌دهد که در `debugging` و گزارش‌گیری مفید است. عبارت "3.7.6-fixed" نشان‌دهنده نسخه ۶.۷.۳ با اصلاح یک باگ منطقی حیاتی است.

۲.۳.۹ شماره خطاهای ساختاریافته

ماژول یک `enum` به نام `ErrorCode` تعریف می‌کند که تمام کدهای خطای ممکن را شامل می‌شود:

```
1 class ErrorCode(Enum):
2     INVALID_PARAMS = "INVALID_PARAMS"
3     INVALID_DECOY_ESTIMATES_TYPE = "INVALID_DECOY_ESTIMATES_TYPE"
4     MISSING_DECOY_FIELDS = "MISSING_DECOY_FIELDS"
5     INVALID_STATS_MAP = "INVALID_STATS_MAP"
6     MISSING_SIGNAL_CONFIG = "MISSING_SIGNAL_CONFIG"
7     INVALID_PULSE_CONFIG = "INVALID_PULSE_CONFIG"
8     NO_SIGNAL_PULSES_SENT = "NO_SIGNAL_PULSES_SENT"
9     INSUFFICIENT_STATISTICS = "INSUFFICIENT_STATISTICS"
10    INCONSISTENT_MEASUREMENT = "INCONSISTENT_MEASUREMENT"
11    NON_FINITE_VALUE = "NON_FINITE_VALUE"
12    PHASE_ERROR_CLAMPED = "PHASE_ERROR_CLAMPED"
13    INVALID_YIELD_BOUND = "INVALID_YIELD_BOUND"
```

```

۱۴ INVALID_ERROR_RATE_BOUND = "INVALID_ERROR_RATE_BOUND"
۱۵

```

استفاده از enum به جای رشته‌های خام، چندین مزیت دارد: نخست، جلوگیری از خطاهای املائی در نام کدها؛ دوم، تکمیل خودکار در ویرایشگرهای کد؛ سوم، امکان جستجو و بازبینی سیستماتیک. هر کد خطا معنای مشخصی دارد. به عنوان مثال، INVALID_DECOY_ESTIMATES_TYPE نشان می‌دهد که شيء ورودی decoy_estimates نه از نوع DecoyEstimatesInput است و نه دارای فیلدهای مورد نیاز است. MISSING_DECOY_FIELDS نشان می‌دهد که فیلدهای yield_1_lower_bound یا error_rate_1_upper_bound در شيء ورودی موجود نیستند. NON_FINITE_VALUE نشان می‌دهد که یک مقدار میانی در محاسبات به NaN یا Inf تبدیل شده است. این کدها در شيء KeyCalculationResult در فیلد error_codes ذخیره می‌شوند و در متد finalize_result به رشته تبدیل می‌شوند تا در خروجی JSON قابل سریال‌سازی باشند.

۳.۳.۹ ساختار dataclass برای ورودی و خروجی

ماژول دو dataclass اصلی تعریف می‌کند. نخست، DecoyEstimatesInput که ورودی نرمال‌شده تخمین‌های Decoy را نگه می‌دارد:

```

۱ @dataclass
۲ class DecoyEstimatesInput:
۳     """
۴     Normalized decoy estimates required for the key calculation.
۵     This proof expects the bit error rate (e1_bit_ub) from the Z-basis
۶     and the yield (Y1_L_rate) which is assumed basis-independent.
۷     """
۸     Y1_L_rate: float
۹     e1_bit_ub: float # This is e1_z_ub (error rate in Z-basis)
۱۰

```

این dataclass دو فیلد دارد: Y1_L_rate که کران پایین بازده تک‌فوتونی است (به صورت یک نسبت، مستقل از پایه) و e1_bit_ub که کران بالای خطای بیت تک‌فوتونی در پایه Z است. در این طراحی، فرض می‌شود که Y_1 مستقل از پایه است (یعنی $Y_{1,X} = Y_{1,Z} = Y_1$) که یک فرض استاندارد در پروتکل‌های Decoy-State BB84 است. این dataclass در متد normalize_decoy_estimates از ورودی‌های مختلف (شء DecoyEstimates از base.py، دیکشنری، یا خود DecoyEstimatesInput) ساخته می‌شود تا یک رابط یکنواخت برای ادامه‌ی محاسبات فراهم شود. دوم، KeyCalculationResult که خروجی نهایی ماژول است:

```

۱ @dataclass
۲ class KeyCalculationResult:
۳     secure_key_length: int = 0
۴     diagnostics: Dict[str, Any] = field(default_factory=dict)
۵     error_codes: List[str] = field(default_factory=list)
۶     metadata: Dict[str, Any] = field(default_factory=dict)
۷

```

این dataclass چهار فیلد دارد: secure_key_length که طول کلید امن نهایی به صورت یک عدد صحیح غیرمنفی است؛ diagnostics که یک دیکشنری شامل تمام مقادیر میانی محاسبه (مانند m_Z, n_Z, m_X, n_X)، QBER، نشت تصحیح خطا، خطای فاز، جریمه‌های PA و آنتروپی موجود) است؛ error_codes که لیستی از کدهای خطای رخ داده است؛ و metadata که شامل اطلاعاتی مانند زمان‌شناسه (timestamp) و مدت زمان محاسبه به میلی‌ثانیه است. استفاده از field(default_factory=dict) به جای مقدار پیش‌فرض {} یک الگوی استاندارد Python است که از اشتراک‌گذاری state تصادفی میان نمونه‌ها جلوگیری می‌کند.

۴.۳.۹ استثناهای سفارشی

ماژول دو استثنای سفارشی تعریف می‌کند:

```

۱ class ParameterValidationError(ValueError): pass
۲ class QKDSimulationError(RuntimeError): pass
۳

```

`ParameterValidationError` از `ValueError` ارث می‌برد و برای خطاهای مربوط به پیکربندی پارامترها (مانند $\epsilon_{\text{sec}} \leq \epsilon_{\text{cor}}$) استفاده می‌شود. `QKDSimulationError` از `RuntimeError` ارث می‌برد و برای خطاهای زمان اجرا (مانند ناسازگاری اندازه‌گیری) استفاده می‌شود. این جداسازی استثناها به کاربر اجازه می‌دهد به صورت انتخابی آن‌ها را بگیرد (catch) و رفتار متفاوتی برای هر نوع خطا نشان دهد. در متد `calculate_key_length`، این استثناها در یک بلوک `try/except` گرفته می‌شوند و به جای `raise` مجدد، در فیلد `error_codes` ثبت می‌شوند تا رفتار fail-safe حفظ شود.

۵.۳.۹ ثابت‌های عددی و برش‌ها

ماژول چندین ثابت عددی برای پایداری محاسبات تعریف می‌کند:

```

۱ _TIGHT_PROOF_EPS_MIN = 1e-300
۲ DEFAULT_ENTROPY_PROB_CLAMP = 1e-12
۳ DEFAULT_NUMERIC_TOL = 1e-9
۴
```

`_TIGHT_PROOF_EPS_MIN` کمترین مقدار مجاز برای ϵ در محاسبات لگاریتمی است و از $\log_2(0) = -\infty$ جلوگیری می‌کند. `DEFAULT_ENTROPY_PROB_CLAMP` حد برش برای احتمالات در محاسبه آنتروپی باینری است و از $\log_2(0)$ در $p = 0$ یا $p = 1$ جلوگیری می‌کند. `DEFAULT_NUMERIC_TOL` تلورانس عددی عمومی برای مقایسه‌ها و برش‌ها است. این ثابت‌ها از ماژول `qkd.constants` نیز وارد می‌شوند (`NUMERIC_ABS_TOL` و `ENTROPY_PROB_CLAMP`) تا با سایر بخش‌های فریم‌ورک یکنواخت باشند. این طراحی، مقادیر برش را به صورت متمرکز قابل تنظیم می‌کند و از پراکندگی مقادیر در کد جلوگیری می‌کند.

۴.۹ پیاده‌سازی ریاضی و الگوریتمی

۱.۴.۹ تخصیص بودجه ϵ با حاشیه عددی

متد `allocate_epsilons` یکی از حیاتی‌ترین بخش‌های ماژول است که بودجه امنیتی ϵ_{sec} را میان اجزای مختلف تقسیم می‌کند. در ادامه، نسخه کامل این متد را با توضیح خط به خط ارائه می‌دهیم:

```

۱ def allocate_epsilons(self) -> EpsilonAllocation:
۲     eps_sec = self.p.eps_sec
۳     eps_cor = self.p.eps_cor
۴     remaining_eps = eps_sec - eps_cor
۵     if remaining_eps <= 0:
۶         raise ParameterValidationError(
۷             f"eps_sec ({eps_sec2:e}) must be greater than "
۸             f"eps_cor ({eps_cor2:e}).")
۹
۱۰     # Use an absolute safety margin to handle 64-bit float rounding
۱۱     absolute_margin = 1e-20
۱۲     budget_to_alloc = remaining_eps - absolute_margin
۱۳
۱۴     if budget_to_alloc <= 0:
۱۵         raise ParameterValidationError(
۱۶             f"Remaining epsilon budget ({remaining_eps2:e}) "
۱۷             f"is too small after applying margin.")
۱۸
۱۹     # Allocate 4 parts: 1 for pe, 2 for smooth, 1 for pa
۲۰     eps_pe = budget_to_alloc / 0.4
۲۱     eps_smooth = budget_to_alloc / 0.4
۲۲     eps_pa = budget_to_alloc - eps_pe - 0.2 * eps_smooth
۲۳
۲۴     num_pulse_configs = len(getattr(self.p.source, 'pulse_configs', []))
۲۵     denominator = max(1, 4 * num_pulse_configs + 1)
۲۶     eps_phase_est = eps_pe / denominator
۲۷
۲۸     allocation = EpsilonAllocation(
۲۹         eps_sec=eps_sec, eps_cor=eps_cor, eps_pe=eps_pe,
۳۰         eps_smooth=eps_smooth, eps_pa=eps_pa, eps_phase_est=eps_phase_est
۳۱     )
۳۲     allocation.validate()
۳۳     return allocation
۳۴
```

در خطوط ابتدایی، ϵ_{cor} و ϵ_{sec} از پیکربندی خوانده می‌شوند و بودجه باقی‌مانده به صورت $\epsilon_{\text{sec}} - \epsilon_{\text{cor}}$ محاسبه می‌شود. اگر این مقدار غیرمثبت باشد (یعنی $\epsilon_{\text{cor}} \geq \epsilon_{\text{sec}}$)، استثنای `ParameterValidationError` `raise` می‌شود زیرا این حالت از نظر تئوری غیرممکن است. سپس یک حاشیه امنیتی مطلق به اندازه 10^{-20} از بودجه باقی‌مانده کم می‌شود. این حاشیه برای رفع یک باگ ظریف در ریاضیات ممیز شناور ۶۴-بیت ضروری است: در برخی حالت‌ها، مجموع $\epsilon_{\text{cor}} + (\epsilon_{\text{sec}} - \epsilon_{\text{cor}})$ به دلیل خطای گردکردن، کمی بزرگتر از ϵ_{sec} می‌شود و اعتبارسنجی `allocation.validate()` شکست می‌خورد. با کم کردن این حاشیه کوچک، تضمین می‌شود که نامساوی به صورت عددی برقرار می‌ماند.

سپس بودجه میان چهار جزء تقسیم می‌شود: ϵ_{PE} یک چهارم، ϵ_{smooth} یک چهارم (که در فرمول دو بار استفاده می‌شود، یعنی مجموع $2 \cdot \epsilon_{\text{smooth}}$ از بودجه می‌شود) و ϵ_{PA} به عنوان باقی‌مانده. این تقسیم متقارن تضمین می‌کند که نامساوی $\epsilon_{\text{PE}} + 2\epsilon_{\text{smooth}} + \epsilon_{\text{PA}} \leq \text{budget}$ به طور دقیق برقرار باشد. در نهایت، $\epsilon_{\text{phase_est}}$ به صورت $\epsilon_{\text{PE}} / (4N_{\text{configs}} + 1)$ محاسبه می‌شود که در آن N_{configs} تعداد پیکربندی‌های پالس است. این فرمول، بودجه تخمین خطای فاز را به تعداد رویدادهای تخمین تقسیم می‌کند و هر رویداد یک سهم مساوی دریافت می‌کند. در پایان، اعتبارسنجی نهایی با `allocation.validate()` انجام می‌شود که در صورت بروز هرگونه ناسازگاری، استثنا `raise` می‌کند.

۲.۴.۹ متد اصلی محاسبه طول کلید

متد `calculate_key_length` هسته اصلی ماژول است که تمام مراحل محاسبه طول کلید را هماهنگ می‌کند. در ادامه، نسخه کامل این متد را با توضیح بخش‌به‌بخش ارائه می‌دهیم:

```

1  def calculate_key_length(
2      self,
3      stats_map: Dict[str, TallyCounts],
4      decoy_estimates: Optional[Union[Dict[str, Any], object]] = None
5  ) -> KeyCalculationResult:
6      run_id = getattr(self.p, 'run_id', None)
7      self.logger.info("event=key_calc_start run_id=%s", run_id)
8      start_time = time.perf_counter()
9      result = KeyCalculationResult()
10
11     try:
12         if decoy_estimates is None:
13             return super().calculate_key_length(stats_map)
14
15         is_decoy_protocol = bool(decoy_estimates)
16
17         if is_decoy_protocol:
18             signal_stats, p_sig_cfg = self._validate_stats_map_decoy(
19                 stats_map, result)
20             if result.error_codes:
21                 return self._finalize_result(result, start_time)
22
23             decoy_norm = self._normalize_decoy_estimates(
24                 decoy_estimates, result)
25             if result.error_codes:
26                 return self._finalize_result(result, start_time)
27
28             s_x_1_L_count, s_z_1_L_count, n_x, _, n_z = \
29                 self._compute_base_counts_decoy(
30                     signal_stats, p_sig_cfg, decoy_norm, result)
31             e1_bit_ub = decoy_norm.e1_bit_ub
32

```

در خطوط ابتدایی، یک شناسه اجرا (`run_id`) از پیکربندی خوانده می‌شود (اگر موجود باشد) و یک رویداد شروع در لاگ ثبت می‌شود. زمان شروع با `time.perf_counter()` (که دارای دقت میکروثانیه است) ثبت می‌شود تا در پایان، مدت زمان محاسبه گزارش شود. سپس یک شیء `KeyCalculationResult` خالی ساخته می‌شود. اگر `decoy_estimates` برابر `None` باشد، متد کلاس پایه فراخوانی می‌شود تا از فروپاشی جلوگیری شود. در غیر این صورت، بررسی می‌شود که آیا پروتکل از نوع `Decoy` است یا خیر (با بررسی `bool(decoy_estimates)` که در صورت دیکشنری خالی یا شیء فاقد محتوا، `False` برمی‌گرداند). در مسیر `Decoy-State`، ابتدا `stats_map` با متد `_validate_stats_map_decoy` اعتبارسنجی

می‌شود. این متد بررسی می‌کند که کلید "signal" در stats_map موجود باشد و مقدار آن از نوع TallyCounts باشد. همچنین پیکربندی پالس سیگنال را از پیکربندی منبع استخراج می‌کند و بررسی می‌کند که فیلد mean_photon_number موجود باشد. در صورت بروز هرگونه خطا، کد خطای مناسب ثبت می‌شود و متد با فراخوانی finalize_result به طور زود هنگام بازمی‌گردد. سپس تخمین‌های Decoy با متد normalize_decoy_estimates نرمال‌سازی می‌شوند. این متد، ورودی‌های مختلف (شیء DecoyEstimates از base.py، دیکشنری، یا خود DecoyEstimatesInput) را به یک شیء یکنواخت DecoyEstimatesInput تبدیل می‌کند. سپس شمارش‌های پایه Decoy با متد compute_base_counts_decoy محاسبه می‌شوند که در ادامه به تفصیل توضیح داده خواهد شد. در مسیر non-Decoy کد به صورت متفاوت عمل می‌کند:

```

1  else:
2      result.diagnostics['info'] = (
3          "Running in non-decoy mode; using conservative estimates.")
4      signal_stats = self._get_total_stats_non_decoy(
5          stats_map, result)
6      if result.error_codes:
7          return self._finalize_result(result, start_time)
8
9      # Assume key is from X-basis, phase error from Z-basis
10     n_x = float(signal_stats.sifted_x)
11     m_x = float(signal_stats.errors_sifted_x)
12     n_z = float(signal_stats.sifted_z)
13     m_z = float(signal_stats.errors_sifted_z)
14
15     # Conservative: all sifted bits are single-photon
16     s_x_1_L_count = n_x
17     s_z_1_L_count = n_z
18
19     e1_bit_ub = self._safe_divide(
20         m_z, n_z, name="qber_z",
21         error_codes=result.error_codes)
22     result.diagnostics['intermediate'] = {
23         "n_x": n_x, "m_x": m_x, "n_z": n_z, "m_z": m_z,
24         "qber_z": e1_bit_ub
25     }
26

```

در مسیر non-Decoy، ابتدا یک پیام اطلاعاتی در diagnostics ثبت می‌شود تا کاربر بداند سیستم در حالت non-Decoy اجرا می‌شود. سپس تمام آمار از stats_map با متد _get_total_stats_non_decoy ادغام می‌شود. این متد تمام شیء‌های TallyCounts در stats_map را با متد merged ترکیب می‌کند تا یک شیء TallyCounts واحد به دست آید. سپس شمارش‌های پایه X و Z استخراج می‌شوند. در این حالت، فرض محافظه‌کارانه این است که تمام بیت‌های sifted از تک‌فوتون‌ها نشأت گرفته‌اند، یعنی $s_{X,1}^L = n_X$ و $s_{Z,1}^L = n_Z$. این فرض در حضور پالس‌های چندفوتونی نامن است، اما برای شبیه‌سازی‌های BB84 ایده‌آل قابل قبول است. خطای بیت به طور مستقیم از QBER پایه Z تخمین زده می‌شود: $e_{1,\text{bit}} = m_Z/n_Z$. سپس، در هر دو مسیر، مراحل نهایی محاسبه انجام می‌شود:

```

1  min_s_z_1_L = getattr(self.p, 'S_Z_1_L_MIN_FOR_PHASE_EST', 100)
2
3  if n_z <= 0 or s_z_1_L_count < min_s_z_1_L:
4      result.error_codes.append(ErrorCode.INSUFFICIENT_STATISTICS)
5      return self._finalize_result(result, start_time)
6
7  if is_decoy_protocol:
8      n_key, m_key = signal_stats.sifted_x, signal_stats.errors_sifted_x
9  else:
10     n_key, m_key = n_x, m_x
11
12     leak_ec = self._compute_leak_ec(n_key, m_key, result)
13     e1_phase_ub = self._compute_phase_error_ub(
14         s_z_1_L_count, e1_bit_ub, result)
15     pa_corr_bits = self._compute_pa_terms(result)
16
17     available_entropy = s_x_1_L_count * (
18         0.1 - self._binary_entropy_safe(e1_phase_ub))
19
20     key_len_float = available_entropy - leak_ec - pa_corr_bits

```



```

۲۱ self._finite_or_error("final_key_length", key_len_float, result)
۲۲ result.secure_key_length = self._clamp_key_length(key_len_float)
۲۳ self.populate_final_diagnostics(
۲۴ result, available_entropy, key_len_float, pa_corr_bits)
۲۵
۲۶
۲۷ except (ParameterValidationError, QKDSimulationError) as e:
۲۸ self.logger.error(
۲۹ "event=key_calc_error run_id=%s error='%s'", run_id, e)
۳۰ result.error_codes.append(ErrorCode.INVALID_PARAMS)
۳۱
۳۲ return self._finalize_result(result, start_time)
۳۳

```

در این بخش، ابتدا حداقل شمارش تک فوتونی در پایه Z برای تخمین خطای فاز از پیکربری خوانده می‌شود (با مقدار پیش فرض ۱۰۰). اگر $n_Z \leq 0$ یا $s_{Z,1}^L$ کمتر از این حداقل باشد، کد خطای INSUFFICIENT_STATISTICS ثبت می‌شود و متد به طور زود هنگام بازمی‌گردد. این بررسی تضمین می‌کند که تخمین خطای فاز به اندازه کافی قابل اعتماد است. سپس شمارش‌های پایه تولیدکننده کلید (X) استخراج می‌شوند. نشت تصحیح خطا با متد `_compute_leak_ec`، کران بالای خطای فاز با متد `_compute_phase_error_ub` و جریمه‌های PA و تصحیح با متد `_compute_pa_terms` محاسبه می‌شوند. آنگاه آنتروپی موجود به صورت $s_{X,1}^L \cdot (1 - H_2(e_{\text{phase}}))$ محاسبه می‌شود و طول کلید به صورت $\ell = \text{entropy} - \text{leak} - \text{penalties}$ دست می‌آید. در نهایت، بررسی `_finite_or_error` تضمین می‌کند که مقدار نهایی متناهی است و متد `_clamp_key_length` از کلاس پایه، مقدار را به یک عدد صحیح غیرمنفی تبدیل می‌کند. در صورت بروز هرگونه استثنای `ParameterValidationError` یا `QKDSimulationError`، استثنا گرفته می‌شود، در لاگ ثبت می‌گردد و کد خطای `INVALID_PARAMS` به `result` اضافه می‌شود.

۳.۴.۹ محاسبه شمارش تک فوتونی در حالت Decoy-State

متد `_compute_base_counts_decoy` شمارش‌های تک فوتونی را در هر دو پایه X و Z محاسبه می‌کند:

```

۱ def _compute_base_counts_decoy(self, signal_stats, p_sig_cfg,
۲ decoy_norm, result):
۳ mu_s = p_sig_cfg.mean_photon_number
۴ pi_s = self._single_photon_prob(mu_s)
۵
۶ # Key is from X-basis
۷ alice_x_prob = 0.1 - getattr(
۸ self.p.protocol, "alice_z_basis_prob", (5.0
۹ bob_x_prob = 0.1 - getattr(
۱۰ self.p.protocol, "bob_z_basis_prob", (5.0
۱۱ s_x_1_L_count = float(signal_stats.sent_x) * pi_s * \
۱۲ decoy_norm.Y1_L_rate * alice_x_prob * bob_x_prob
۱۳ n_x = float(signal_stats.sifted_x)
۱۴ m_x = float(signal_stats.errors_sifted_x)
۱۵ if s_x_1_L_count > n_x + NUMERIC_ABS_TOL:
۱۶ s_x_1_L_count = n_x
۱۷
۱۸ # Phase error is from Z-basis
۱۹ alice_z_prob = getattr(self.p.protocol, "alice_z_basis_prob", (5.0
۲۰ bob_z_prob = getattr(self.p.protocol, "bob_z_basis_prob", (5.0
۲۱ s_z_1_L_count = float(signal_stats.sent_z) * pi_s * \
۲۲ decoy_norm.Y1_L_rate * alice_z_prob * bob_z_prob
۲۳ n_z = float(signal_stats.sifted_z)
۲۴ if s_z_1_L_count > n_z + NUMERIC_ABS_TOL:
۲۵ s_z_1_L_count = n_z
۲۶
۲۷ result.diagnostics['intermediate'] = {
۲۸ "mu_s": mu_s, "pi_s": pi_s,
۲۹ "s_x_1_L_count": s_x_1_L_count, "n_x": n_x, "m_x": m_x,
۳۰ "s_z_1_L_count": s_z_1_L_count, "n_z": n_z
۳۱ }
۳۲ return s_x_1_L_count, s_z_1_L_count, n_x, m_x, n_z
۳۳

```

در این متد، ابتدا شدت سیگنال μ_s از پیکربری خوانده می‌شود و احتمال ارسال تک فوتون $P(1 | \mu_s) = \mu_s \cdot e^{-\mu_s}$ با متد `_single_photon_prob` محاسبه می‌شود. سپس احتمال انتخاب پایه X توسط آلیس و

باب از پیکربری خوانده می‌شود (با مقدار پیش‌فرض ۵.۰ در صورت عدم تنظیم). شمارش تک‌فوتونی در پایه X به صورت $N_{\text{sent},X} \cdot P(1 | \mu_s) \cdot Y_1^L \cdot p_X^A \cdot p_X^B$ محاسبه می‌شود. سپس شمارش‌های sifted و خطاهای پایه X استخراج می‌شوند. یک بررسی محافظه‌کارانه این است که اگر $s_{X,1}^L > n_X$ شود (که از نظر فیزیکی غیرممکن است)، مقدار به n_X بریده می‌شود. این برش با تلورانس NUMERIC_ABS_TOL انجام می‌شود تا خطاهای کوچک گردکردن نادیده گرفته شوند. به طور متقابل، شمارش تک‌فوتونی در پایه Z به صورت مشابه محاسبه می‌شود. در پایان، تمام مقادیر میانی در diagnostics ذخیره می‌شوند تا برای تحلیل و debugging قابل دسترسی باشند.

۴.۴.۹ محاسبه نشت تصحیح خطا

متد `_compute_leak_ec` نشت تصحیح خطا را محاسبه می‌کند:

```

۱ def _compute_leak_ec(self, n_key_basis: float, m_key_basis: float,
۲ result: KeyCalculationResult) -> float:
۳     qber_key_basis = self._safe_divide(
۴         m_key_basis, n_key_basis, name="qber_key_basis",
۵         error_codes=result.error_codes)
۶     f_ec = getattr(self.p, 'f_error_correction', (1.1
۷     leak_ec = f_ec * self._binary_entropy_safe(qber_key_basis) * n_key_basis
۸
۹     di = result.diagnostics.setdefault('intermediate', {})
۱۰    di['qber_key_basis'] = qber_key_basis
۱۱    di['leak_ec_bits'] = leak_ec
۱۲    return leak_ec
۱۳

```

در این متد، ابتدا QBER پایه تولیدکننده کلید به صورت $E_{\text{bit}} = m_{\text{key}}/n_{\text{key}}$ محاسبه می‌شود. تابع `_safe_divide` (از کلاس پایه) در صورت تقسیم بر صفر، مقدار ۵.۰ (بدترین حالت) را برمی‌گرداند و کد خطای مناسب را ثبت می‌کند. سپس بازدهی تصحیح خطا f_{EC} از پیکربری خوانده می‌شود (با مقدار پیش‌فرض ۱.۱). نشت تصحیح خطا به صورت $f_{\text{EC}} \cdot H_2(E_{\text{bit}}) \cdot n_{\text{key}}$ محاسبه می‌شود که در آن H_2 آنتروپی باینری است. این فرمول نشان می‌دهد که نشت متناسب با تعداد بیت‌های sifted و آنتروپی خطا است. در پایان، مقادیر میانی در diagnostics ذخیره می‌شوند. یک نکته مهم این است که نام متغیرها در این متد به صورت `n_key_basis` و `m_key_basis` انتخاب شده تا به وضوح نشان دهد که این مقادیر مربوط به پایه تولیدکننده کلید (نه لزوماً Z) هستند. این تغییر نام، یک اصلاح semantic نسبت به نسخه‌های قبلی است که به اشتباه از `n_z` و `m_z` استفاده می‌کردند.

۵.۴.۹ محاسبه کران بالای خطای فاز

متد `_compute_phase_error_ub` کران بالای خطای فاز را با استفاده از نامساوی Hoeffding محاسبه می‌کند:

```

۱ def _compute_phase_error_ub(self, s_z_1_L_count: float,
۲ e1_bit_ub: float,
۳ result: KeyCalculationResult) -> float:
۴     di = result.diagnostics.setdefault('intermediate', {})
۵     if self.p.assume_phase_equals_bit_error:
۶         di['e1_phase_ub'] = e1_bit_ub
۷     return e1_bit_ub
۸
۹     eps_ph_est_safe, _ = self._log_inv_clamp(self.eps_alloc.eps_phase_est)
۱۰    log_term = -math.log(eps_ph_est_safe)
۱۱
۱۲    if s_z_1_L_count <= 0:
۱۳        di['e1_phase_ub'] = 5.0
۱۴        return 5.0
۱۵
۱۶    delta = self._safe_sqrt(log_term / (2 * s_z_1_L_count))
۱۷    e1_phase_ub = e1_bit_ub + delta
۱۸
۱۹    if e1_phase_ub > 5.0:
۲۰        e1_phase_ub = 5.0
۲۱
۲۲    di['e1_phase_ub'] = e1_phase_ub
۲۳    di['phase_error_delta'] = delta
۲۴    return e1_phase_ub
۲۵

```


در این متد، ابتدا بررسی می‌شود که آیا پرچم `assume_phase_equals_bit_error` در پیکربندی فعال است یا خیر. اگر فعال باشد، فرض می‌شود $e_{\text{phase}} = e_{\text{bit}}$ که یک تقریب ساده اما محافظه‌کارانه در شرایطی است که شمارش پایه Z ناکافی است. در غیر این صورت، نامساوی Hoeffding اعمال می‌شود. ابتدا $\varepsilon_{\text{phase_est}}$ با متد `_log_inv_clamp` برش داده می‌شود تا از $\log(0)$ جلوگیری شود. سپس $\delta = \sqrt{-\ln(\varepsilon_{\text{phase_est}})/(2 \cdot s_{Z,1}^L)}$ محاسبه می‌شود. تابع `_safe_sqrt` در صورت ورودی منفی، صفر برمی‌گرداند. کران بالای خطای فاز به صورت $e_{\text{phase}} = e_{\text{bit}} + \delta$ محاسبه می‌شود. اگر این مقدار از ۵.۰ بیشتر شود، به ۵.۰ بریده می‌شود زیرا آنتروپی باینری در ۵.۰ بیشینه است و مقدار بیشتر از ۵.۰ از نظر فیزیکی معنایی ندارد. در پایان، مقادیر میانی در `diagnostics` ذخیره می‌شوند. این متد به طور ویژه از شمارش پایه Z (یعنی `s_z_1_L_count`) استفاده می‌کند زیرا خطای فاز از پایه Z تخمین زده می‌شود؛ این یک اصلاح حیاتی نسبت به نسخه‌های قبلی است که به اشتباه از شمارش پایه X استفاده می‌کردند.

۶.۴.۹ محاسبه جرمه‌های PA و تصحیح

متد `_compute_pa_terms` جرمه‌های تقویت حریم خصوصی و تصحیح را محاسبه می‌کند:

```

1 def _compute_pa_terms(self, result):
2     eps_smooth_safe, _ = self._log_inv_clamp(self.eps_alloc.eps_smooth)
3     eps_pa_safe, _ = self._log_inv_clamp(self.eps_alloc.eps_pa)
4     eps_cor_safe, _ = self._log_inv_clamp(self.eps_alloc.eps_cor)
5     pa_term_bits = _safe_log2(0.1) / 0.2 * eps_smooth_safe + \
6         _safe_log2(0.1) / eps_pa_safe
7     corr_term_bits = _safe_log2(0.2) / eps_cor_safe
8
9     di = result.diagnostics.setdefault('intermediate', {})
10    di['pa_corr_bits'] = pa_term_bits + corr_term_bits
11    di['pa_bits'] = pa_term_bits
12    di['corr_bits'] = corr_term_bits
13
14    return pa_term_bits + corr_term_bits
15

```

در این متد، ابتدا هر یک از $\varepsilon_{\text{smooth}}$ ، ε_{PA} و ε_{cor} با متد `_log_inv_clamp` برش داده می‌شوند. این برش تضمین می‌کند که در صورت تنظیم صفر به اشتباه، محاسبات منجر به $-\infty$ نشوند. سپس جرمه PA به صورت $\log_2(1/(2\varepsilon_{\text{smooth}})) + \log_2(1/\varepsilon_{\text{PA}})$ محاسبه می‌شود و جرمه تصحیح به صورت $\log_2(2/\varepsilon_{\text{cor}})$. تابع `_safe_log2` (در سطح ماژول) در صورت ورودی غیرمثبت، صفر برمی‌گرداند. در پایان، مجموع جرمه‌ها برگردانده می‌شود و مقادیر میانی در `diagnostics` ذخیره می‌شوند. این جرمه‌ها معمولاً در حدود چند ده بیت هستند و در شبیه‌سازی‌های با کلیدهای طولانی ناچیز هستند، اما در کلیدهای کوتاه می‌توانند تأثیر قابل توجهی بر طول کلید نهایی داشته باشند.

۷.۴.۹ توابع کمکی پایداری عددی

چندین تابع کمکی برای پایداری عددی در ماژول تعریف شده‌اند:

```

1 def _log_inv_clamp(self, val):
2     return (_TIGHT_PROOF_EPS_MIN, True) if val < _TIGHT_PROOF_EPS_MIN \
3     else (val, False)
4
5 def _finite_or_error(self, name, val, result, non_negative=False):
6     if not math.isfinite(val) or (non_negative and val < 0):
7         result.error_codes.append(ErrorCode.NON_FINITE_VALUE)
8
9 def _binary_entropy_safe(self, p: float) -> float:
10    p_clamped = max(ENTROPY_PROB_CLAMP, min(p, 1 - ENTROPY_PROB_CLAMP))
11    if p_clamped <= 0 or p_clamped >= 1: return 0.0
12    return -p_clamped * math.log2(p_clamped) - \
13        (1 - p_clamped) * math.log2(1 - p_clamped)
14
15 def _single_photon_prob(self, mu: float) -> float:
16    if mu < 0: raise ParameterValidationError(f"mu cannot be negative: {mu}")
17    if mu == 0: return 0.0
18    return mu * math.exp(-mu)
19

```

```

۲۰ def _safe_sqrt(self, x: float) -> float:
۲۱     if x < 0: return 0.0
۲۲     return math.sqrt(x)
۲۳
۲۴ def _safe_log2(val: float) -> float:
۲۵     return 0.0 if val <= 0 else math.log2(val)
۲۶

```

متد `_log_inv_clamp` مقدار ورودی را به حداقل `_TIGHT_PROOF_EPS_MIN` برش می‌دهد و یک تاپل شامل مقدار برش‌داده‌شده و یک پرچم (که نشان می‌دهد آیا برش انجام شده یا خیر) برمی‌گرداند. متد `_finite_or_error` بررسی می‌کند که آیا مقدار ورودی متناهی است (نه NaN یا Inf) و در صورت عدم تناهی، کد خطای `NON_FINITE_VALUE` ثبت می‌کند. متد `_binary_entropy_safe` آنتروپی باینری را با برش p به بازه $[\varepsilon, 1 - \varepsilon]$ محاسبه می‌کند تا از $\log_2(0)$ جلوگیری شود. متد `_single_photon_prob` احتمال ارسال تک‌فوتون $P(1 | \mu) = \mu \cdot e^{-\mu}$ را محاسبه می‌کند و در صورت $\mu < 0$ ، استثنا `raise` می‌کند. متد `_safe_sqrt` در صورت ورودی منفی، صفر برمی‌گرداند تا از خطای `math domain` جلوگیری شود. تابع سطح مازول `_safe_log2` نیز در صورت ورودی غیرمثبت، صفر برمی‌گرداند. این توابع کمکی در سراسر مازول برای تضمین پایداری عددی استفاده می‌شوند و از رفتار غیرقابل پیش‌بینی در شرایط مرزی جلوگیری می‌کنند.

۸.۴.۹ نرمال‌سازی تخمین‌های Decoy

متد `_normalize_decoy_estimates` ورودی‌های مختلف تخمین Decoy را به یک شیء یکنواخت تبدیل می‌کند:

```

۱ def _normalize_decoy_estimates(self, decoy_estimates, result):
۲     # If the input is already the correct type, return it immediately
۳     if isinstance(decoy_estimates, DecoyEstimatesInput):
۴         return decoy_estimates
۵
۶     # Attempt to get attributes from the DecoyEstimates object (from base.py)
۷     Y1_L = getattr(decoy_estimates, 'yield_1_lower_bound', None)
۸     e1_U = getattr(decoy_estimates, 'error_rate_1_upper_bound', None)
۹
۱۰    if Y1_L is None or e1_U is None:
۱۱        # Fallback for dictionary inputs
۱۲        if isinstance(decoy_estimates, dict):
۱۳            Y1_L = decoy_estimates.get('Y1_L')
۱۴            e1_U = decoy_estimates.get('e1_U')
۱۵
۱۶    if Y1_L is None or e1_U is None:
۱۷        result.error_codes.append(ErrorCode.MISSING_DECOY_FIELDS)
۱۸        return None
۱۹
۲۰    return DecoyEstimatesInput(float(Y1_L), float(e1_U))
۲۱

```

این متد یک طراحی انعطاف‌پذیر برای پذیرش ورودی‌های مختلف دارد. نخست، اگر ورودی از قبل از نوع `DecoyEstimatesInput` باشد، مستقیماً برگردانده می‌شود. در غیر این صورت، تلاش می‌شود فیلدهای `yield_1_lower_bound` و `error_rate_1_upper_bound` با `getattr` استخراج شوند (این فیلدها در شیء `DecoyEstimates` از `base.py` موجود هستند). اگر این فیلدها موجود نباشند و ورودی یک دیکشنری باشد، تلاش می‌شود کلیدهای `"Y1_L"` و `"e1_U"` استخراج شوند (این کلیدها در برخی پیاده‌سازی‌های قدیمی استفاده می‌شدند). اگر هیچ‌کدام موجود نباشند، کد خطای `MISSING_DECOY_FIELDS` ثبت می‌شود و `None` برگردانده می‌شود. در غیر این صورت، یک شیء جدید `DecoyEstimatesInput` با مقادیر تبدیل‌شده به `float` ساخته می‌شود. این طراحی، مازول را در برابر تغییرات در ساختار ورودی مقاوم می‌کند و اجازه می‌دهد با انواع مختلف شیء‌های تخمین Decoy از بخش‌های مختلف فریم‌ورک کار کند.

۹.۴.۹ نهایی‌سازی نتیجه و ثبت پرووانانس

متد `_finalize_result` نتیجه نهایی را نهایی می‌کند و پرووانانس را ثبت می‌کند:

```

۱ def _finalize_result(self, result, start_time):
۲     end_time = time.perf_counter()
۳     result.metadata['timestamp_utc'] = datetime.now(timezone.utc).isoformat()

```

```

۴ result.metadata['calculation_time_ms'] = (end_time - start_time) * 1000
۵ result.error_codes = [
۶     code.value if isinstance(code, Enum) else code
۷     for code in result.error_codes
۸ ]
۹ run_id = getattr(self.p, 'run_id', None)
۱۰ if result.error_codes:
۱۱     self.logger.warning(
۱۲         "event=key_calc_abort run_id=%s errors=%s",
۱۳         run_id, result.error_codes)
۱۴ else:
۱۵     self.logger.info(
۱۶         "event=key_calc_finish run_id=%s key_length=%d",
۱۷         run_id, result.secure_key_length)
۱۸     return result
۱۹

```

در این متد، زمان پایان با `time.perf_counter()` ثبت می‌شود و مدت زمان محاسبه به میلی‌ثانیه محاسبه می‌شود. یک زمان‌شناسه‌ی UTC با فرمت ISO 8601 در `metadata` ذخیره می‌شود تا برای ردیابی پرووانس قابل استفاده باشد. سپس کدهای خطا از `enum` به رشته تبدیل می‌شوند تا در خروجی JSON قابل سریال‌سازی باشند. در پایان، بر اساس وجود یا عدم وجود خطا، یک رویداد لاگ مناسب ثبت می‌شود: “key_calc_abort” در صورت خطا و “key_calc_finish” در صورت موفقیت. این لاگ‌ها برای مانیتورینگ و `debugging` در محیط‌های تولیدی حیاتی هستند. طراحی این متد به گونه‌ای است که همیشه یک شیء `KeyCalculationResult` کامل با پرووانس و `diagnostics` برمی‌گرداند، حتی در صورت بروز خطا.

۵.۹ ارسال و دریافت با سایر ماژول‌ها

۱.۵.۹ ورودی‌های ماژول

ماژول `tight.py` برای انجام محاسبات خود به چندین ورودی از سایر ماژول‌های فریم‌ورک نیاز دارد. این ورودی‌ها به دو دسته و تقسیم می‌شوند. ورودی‌های پیکربندی در زمان ساخت شیء اثبات امنیتی خوانده می‌شوند و در طول اجرا ثابت می‌مانند، در حالی که ورودی‌های آمار در هر فراخوانی `calculate_key_length` به‌روز می‌شوند.

- `QKDParams`: این شیء از ماژول `qkd.params` وارد می‌شود و شامل تمام پارامترهای پیکربندی سیستم است. زیرشئ‌های کلیدی که این ماژول استفاده می‌کند عبارتند از: `p.eps_sec`, `p.eps_cor`, `p.f_error_correction`, `p.assume_phase_equals_bit_error`, `p.S_Z_1_L_MIN_FOR_PHASE_EST` (حداقل شمارش تک‌فوتونی پایه Z برای تخمین خطای فاز، با مقدار پیش‌فرض ۱۰۰)، `p.run_id` (شناسه اجرا برای ردیابی پرووانس)، `p.source.pulse_configs` (لیست پیکربندی‌های پالس) و `p.protocol.alice_z_basis_prob` و `p.protocol.bob_z_basis_prob` (احتمال انتخاب پایه Z توسط آلین و باب).

- `stats_map`: این یک دیکشنری از `str` به `TallyCounts` است که از لایه شبیه‌ساز آشکارساز به اثبات امنیتی داده می‌شود. کلید “signal” باید موجود باشد و مقدار آن یک شیء `TallyCounts` با فیلدهای `sifted_z`, `sifted_x`, `errors_sifted_z` و `errors_sifted_x` باشد. در حالت `non-Decoy`، تمام شیء‌های `TallyCounts` در `stats_map` با متد `merged` ادغام می‌شوند.

- `decoy_estimates`: این پارامتر اختیاری است و می‌تواند یکی از موارد زیر باشد: یک شیء `DecoyEstimatesInput` (با فیلدهای `Y1_L_rate` و `e1_bit_ub`)، یک شیء `DecoyEstimates` از `base.py` (با فیلدهای `yield_1_lower_bound` و `error_rate_1_upper_bound`)، یا یک دیکشنری با کلیدهای “Y1_L” و “e1_U”. این طراحی انعطاف‌پذیر اجازه می‌دهد ماژول با انواع مختلف شیء‌های تخمین `Decoy` از بخش‌های مختلف فریم‌ورک کار کند. اگر این پارامتر `None` باشد، متد کلاس پایه فراخوانی می‌شود.

- `EpsilonAllocation`: این شیء از متد `allocate_epsilon` تولید می‌شود و شامل فیلدهای ϵ_{sec} , ϵ_{cor} , ϵ_{pe} , ϵ_{smooth} , ϵ_{pa} و $\epsilon_{\text{phase_est}}$ است. این شیء در محاسبه جریمه‌های PA و خطای فاز استفاده می‌شود.

۲.۵.۹ خروجی‌های ماژول

خروجی اصلی ماژول یک شیء `KeyCalculationResult` است که شامل چهار فیلد است:

۱. `secure_key_length`: طول کلید امن به صورت یک عدد صحیح غیرمنفی. این فیلد خروجی نهایی محاسبه است و توسط لایه بالایی برای تولید کلید واقعی استفاده می‌شود. در صورت بروز هرگونه خطا، این مقدار صفر است.
۲. `diagnostics`: یک دیکشنری شامل تمام مقادیر میانی محاسبه. این دیکشنری شامل یک کلید "intermediate" است که خود یک دیکشنری با مقادیری مانند $\mu_s, P(1 | \mu_s), s_{Z,1}^L, m_X, n_X, s_{X,1}^L, QBER, n_Z$ ، نشت تصحیح خطا، خطای فاز، δ خطای فاز، جریمه‌های PA و تصحیح، آنتروپی موجود و طول کلید اعشاری است. این فیلد برای تحلیل و بازبینی امنیتی ضروری است.
۳. `error_codes`: لیستی از کدهای خطا (به صورت رشته) که در صورت بروز مشکل، علت آن را نشان می‌دهند. اگر این لیست خالی باشد، محاسبه با موفقیت انجام شده است.
۴. `metadata`: یک دیکشنری شامل اطلاعاتی مانند زمان‌شناسه‌ی UTC (با فرمت ISO 8601) و مدت زمان محاسبه به میلی‌ثانیه. این فیلد برای ردیابی پرووونانس و مانیتورینگ عملکرد استفاده می‌شود.

۳.۵.۹ وابستگی‌های داخلی و خارجی

ماژول وابستگی‌های داخلی و خارجی متعددی دارد. وابستگی‌های داخلی شامل موارد زیر است:

- `qkd.proofs.lim2014.Lim2014Proof`: کلاس پایه که `BB84TightProof` از آن ارث می‌برد. این کلاس زیرساخت‌های مشترک مانند محاسبه آنتروپی باینری، تخصیص پایه ϵ ، ابزارهای `clamp` عددی و متد `safe_divide` را فراهم می‌سازد.
 - `qkd.datatypes.EpsilonAllocation` و `qkd.datatypes.TallyCounts`: این دو `dataclass` به ترتیب برای شمارش‌های آماری و تخصیص بودجه ϵ استفاده می‌شوند. `TallyCounts` شامل فیلدهای `sifted_x, sifted_z, sent_x, sent_z, errors_sifted_x, errors_sifted_z` است و متد `merged` برای ادغام چندین شیء `TallyCounts` را فراهم می‌سازد.
 - `qkd.constants.NUMERIC_ABS_TOL` و `qkd.constants.ENTROPY_PROB_CLAMP`: این دو ثابت برای تلورانس عددی و برش احتمال در محاسبه آنتروپی باینری استفاده می‌شوند.
- وابستگی‌های خارجی شامل `math` برای توابع ریاضی پایه (مانند `log, log2, exp, sqrt, isfinite`)، `logging` برای ثبت رویدادها، `time` برای اندازه‌گیری مدت محاسبه، و `datetime` برای تولید زمان‌شناسه‌ی UTC هستند. همچنین از ماژول‌های استاندارد Python مانند `typing, dataclasses, enum` برای ساختاردهی کد استفاده می‌شود.

۴.۵.۹ جریان داده در سراسر فریم‌ورک

جریان داده کلی به این شکل است:

۱. لایه شبیه‌ساز پالس‌های نوری را با شدت‌های μ (سیگنال)، ν (decoy ضعیف) و $\emptyset$ (خلأ) تولید و از کانال کوانتومی عبور می‌دهد.
۲. آشکارسازها در سمت باب، آشکارسازی‌ها و خطاها را در دو پایه X و Z و برای هر شدت ثبت کرده و در `TallyCounts` ذخیره می‌کنند.
۳. موتور تخمین Decoy (مانند `Ma2005VacuumWeakProof` یا `Lim2014Proof`) با دریافت این شمارش‌ها، تخمین‌های Y_1 و e_1 را به صورت یک شیء `DecoyEstimates` تولید می‌کند.
۴. متد `calculate_key_length` با دریافت `stats_map` و `decoy_estimates` فراخوانی می‌شود. این متد ابتدا `stats_map` را اعتبارسنجی می‌کند و سپس تخمین‌های Decoy را با متد `normalize_decoy_estimates` نرمال‌سازی می‌کند.

۵. سپس شمارش‌های تک‌فوتونی در هر دو پایه با متد `_compute_base_counts_decoy` محاسبه می‌شوند.

۶. نشت تصحیح خطا با متد `_compute_leak_ec`، کران بالای خطای فاز با متد `_compute_pa_terms` و جریمه‌های PA و تصحیح با متد `_compute_pa_terms` محاسبه می‌شوند.

۷. آنتروپی موجود به صورت $s_{X,1}^L \cdot (1 - H_2(e_{\text{phase}}))$ محاسبه می‌شود و طول کلید به صورت $\ell = \text{entropy} - \text{leak} - \text{penalties}$ می‌آید.

۸. در نهایت، متد `_finalize_result` نتیجه را نهایی می‌کند و پروونانس (زمان‌شناسه و مدت محاسبه) را ثبت می‌کند.

۹. لایه بالایی می‌تواند با بررسی `error_codes` وضعیت موفقیت یا شکست را تشخیص دهد و در صورت موفقیت، از `secure_key_length` برای تولید کلید واقعی استفاده کند.

این جداسازی مراحل به گونه‌ای طراحی شده که هر مرحله قابل آزمایش و بازبینی مستقل باشد. به عنوان مثال، می‌توان `DecoyEstimatesInput` را به صورت دستی ساخت و رفتار مرحله نهایی محاسبه کلید را در شرایط خاص بررسی کرد. این ماژول‌بندی از اصل `Single Responsibility` پیروی می‌کند و قابلیت نگهداری و توسعه کد را به طور قابل توجهی افزایش می‌دهد. علاوه بر این، شفافیت کامل در `diagnostics` به کاربر اجازه می‌دهد هر مرحله از محاسبه را ردیابی کرده و در صورت نیاز، فرضیات تئوریک را برای کاربرد خاص خود بازبینی کند. طراحی `fail-safe` این ماژول، آن را به انتخابی ایده‌آل برای محیط‌های تولیدی تبدیل می‌کند که در آن‌ها پایداری و قابل اعتماد بودن سیستم حیاتی است.

فصل ۱۰

بهینه‌سازی پارامترها با استفاده از برنامه‌ریزی خطی (LP)

۱.۱۰ نمای کلی و هدف ماژول

فایل `qkd/proofs/optimization.py` مجموعه‌ای از ابزارهای بهینه‌سازی پارامتر برای پروتکل‌های مختلف توزیع کلید کوانتومی (QKD) را پیاده‌سازی می‌کند. این ماژول بر اساس دو چارچوب مرجع توسعه یافته است: چارچوب Ma et al. 2005 برای پروتکل‌های Decoy-State BB84 و چارچوب Chapman et al. 2018 برای پروتکل‌های state-dependent مبتنی بر Amplitude Damping Channel (ADC). هدف اصلی این ماژول، پیدا کردن مقادیر بهینه پارامترهای قابل کنترل آلیس (مانند شدت سیگنال μ ، شدت ν decoy، احتمال انتخاب سیگنال P_s ، احتمال انتخاب P_d decoy، و زاویه چرخش کدگذاری θ) به گونه‌ای است که نرخ کلید امن نهایی به حداکثر برسد. این بهینه‌سازی در عمل برای استقرار سیستم‌های QKD حیاتی است، زیرا انتخاب پارامترهای غیربهینه می‌تواند نرخ کلید را تا چندین برابری کاهش دهد یا حتی آن را به صفر برساند.

اهمیت این ماژول از این واقعیت ناشی می‌شود که نرخ کلید امن R در پروتکل‌های QKD یک تابع غیرخطی و معمولاً غیرمحدب (non-convex) از پارامترهای سیستم است. به عنوان مثال، در پروتکل Decoy-State BB84، نرخ کلید به صورت $R = q\{Q_1[1 - H_2(e_1)] - Q_\mu f(E_\mu)H_2(E_\mu)\}$ محاسبه می‌شود که در آن $Q_1 = Y_1\mu e^{-\mu}$ و $Q_\mu = \sum_n Y_n P(n|\mu)$ هر دو به صورت غیرخطی به μ وابسته‌اند. افزایش μ باعث افزایش Q_1 (و در نتیجه افزایش ترم تقویت حریم خصوصی) می‌شود، اما همزمان باعث افزایش Q_μ (و در نتیجه افزایش نشت تصحیح خطا) و افزایش آسیب‌پذیری به حمله PNS نیز می‌شود. بنابراین یک نقطه بهینه μ^* وجود دارد که در آن توازن میان این دو ترم بهینه است. پیدا کردن این نقطه به صورت تحلیلی به دلیل پیچیدگی فرمول‌ها ممکن نیست و باید از روش‌های عددی استفاده شود.

این ماژول ده تابع بهینه‌سازی مستقل ارائه می‌کند که هر کدام برای یک سناریوی خاص طراحی شده‌اند. این توابع عبارتند از: `optimize_decoy_intensities` (بهینه‌سازی اکتشافی شدت سیگنال و decoy بر اساس مدل نرخ ساده‌شده)، `optimize_rotation_angle` (بهینه‌سازی زاویه چرخش کدگذاری برای یک کانال عمومی)، `calculate_optimal_mu_ma2005` (محاسبه تحلیلی-عددی μ بهینه بر اساس معادله ۱۲ مقاله Ma 2005)، `optimize_nu_ma2005` (بهینه‌سازی ν برای N متناهی بر اساس بخش ۳.۴ مقاله)، `optimize_pulse_distribution_ma2005` (بهینه‌سازی احتمالات توزیع پالس با روش SLSQP)، `optimize_pulse_probabilities` (پوشش‌دهنده برای تابع قبلی)، `optimize_simultaneous_ma2005` (بهینه‌سازی همزمان P_d و P_s با استفاده از کلاس `Ma2005GlobalOptimizer` و گرادیان عددی)، `optimize_global_ma2005` (پوشش‌دهنده برای تابع قبلی)، `optimize_lim2014_finite_key` (بهینه‌سازی پنج پارامتر چارچوب Lim 2014 برای کلید متناهی با روش Nelder-Mead)، و `optimize_rotation_angle_chapman` (بهینه‌سازی زاویه چرخش برای Coherent Scheme در Chapman 2018 با محاسبه فاصله trace میان حالت‌های خروجی کانال).

طراحی این ماژول بر اساس اصل جداسازی دغدغه‌ها (Separation of Concerns) استوار است؛ به این ترتیب که توابع بهینه‌سازی صرفاً مسئول پیدا کردن پارامترهای بهینه هستند و محاسبه نرخ کلید (یا هر کمیت هدف دیگر) به یک فراخوان (callback) تابعی سپرده می‌شود که توسط کاربر ارائه می‌گردد. این طراحی چندین مزیت دارد:

نخست، ماژول بهینه‌سازی از جزئیات پیاده‌سازی اثبات امنیتی مستقل است و می‌تواند با هر چارچوب اثبات (مانند Ma 2005، Lim 2014 یا Tight) کار کند؛ دوم، کاربر می‌تواند تابع هدف خود را به سادگی جایگزین کند

(مثلاً به جای نرخ کلید، QBER را کمینه کند یا طول کلید متناهی را بیشینه سازد) بدون آنکه نیازی به تغییر کد بهینه‌سازی باشد؛ سوم، فرآیند تست‌کردن و ارزیابی آسان‌تر می‌شود، زیرا می‌توان از callback‌های مصنوعی با رفتار مشخص و شناخته‌شده برای اعتبارسنجی توابع بهینه‌سازی استفاده کرد. این الگو که به تزریق وابستگی (dependency injection) نیز معروف است، یک اصل کلیدی در مهندسی نرم‌افزار به شمار می‌رود که قابلیت نگهداری و توسعه کد را به طور قابل‌توجهی افزایش می‌دهد.

در صورت شکست بهینه‌سازی (به هر دلیل: عدم همگرایی، ناحیه غیرممکن، یا خطای عددی)، توابع این مازول به جای raise کردن استثنا، مقادیر پیش‌فرض امن و محافظه‌کارانه را برمی‌گردانند. این رفتار fail-safe تضمین می‌کند که خط لوله شبیه‌سازی متوقف نشود و کاربر بتواند با مقادیر پیش‌فرض ادامه دهد. به عنوان مثال، اگر بهینه‌سازی μ شکست بخورد، مقدار پیش‌فرض 48.0 (برای $f \leq 1.05$) یا 54.0 (برای $f > 1.05$) برگردانده می‌شود که مقادیر تجربی خوبی برای پروتکل‌های BB84 Decoy-State هستند. این مقادیر پیش‌فرض بر اساس بخش ۱.۳ مقاله Ma 2005 انتخاب شده‌اند.

۲.۱۰ مبانی تئوری و فیزیکی

۱.۲.۱۰ نرخ کلید GLLP و وابستگی غیرخطی به μ

فرمول استاندارد نرخ کلید در چارچوب (GLLP) Gottesman-Lo-Lütkenhaus-Preskill به صورت زیر است:

$$R = q \{ Q_1 [1 - H_2(e_1)] - Q_\mu f(E_\mu) H_2(E_\mu) \}, \quad (1.10)$$

که در آن q فاکتور پایه‌گذاری (برای BB84 برابر 0.5)، $Q_1 = Y_1 \mu e^{-\mu}$ ، Gain تک‌فوتونی، $Q_\mu = \sum_n Y_n P(n|\mu)$ ، Gain کلی سیگنال، e_1 خطای تک‌فوتونی، E_μ QBER سیگنال، $f(E_\mu)$ بازدهی تصحیح خطا و $H_2(\cdot)$ آنترپی باینری است. ترم اول $Q_1 [1 - H_2(e_1)]$ نشان‌دهنده آنترپی قابل استخراج از تک‌فوتون‌های امن است و ترم دوم $Q_\mu f(E_\mu) H_2(E_\mu)$ نشان‌دهنده نشت تصحیح خطا است.

برای پیدا کردن μ بهینه، باید $\partial R / \partial \mu = 0$ را حل کنیم. با فرض ساده‌کننده $e_1 \approx E_\mu$ و $Y_1 \approx \eta$ (در حد نویز کم)، این مشتق‌گیری به یک معادله تحلیلی-عددی منجر می‌شود که در معادله (۱۲) مقاله Ma 2005 ارائه شده است. این معادله به صورت زیر فرمول‌بندی می‌شود:

$$(1 - \mu) e^{-\mu} = C, \quad C = \frac{f_{EC} H_2(e_{det})}{1 - H_2(e_{det})}, \quad (2.10)$$

که در آن e_{det} خطای ذاتی آشکارساز است. تابع $(1 - \mu) e^{-\mu}$ در $\mu = 0$ برابر ۱ است و به طور یکنواخت تا $\mu \rightarrow \infty$ کاهش می‌یابد. بنابراین برای هر $C \in (0, 1)$ یک جواب یکتا در بازه $(0, 1)$ وجود دارد. اگر $C \geq 1$ (یعنی خطای آشکارساز خیلی بزرگ باشد)، جوابی وجود ندارد و نرخ کلید منفی می‌شود؛ در این حالت، مازول به مقدار پیش‌فرض $\mu = 0.48$ برمی‌گردد. این محاسبه تحلیلی-عددی در تابع `calculate_optimal_mu_ma2005` با استفاده از روش Brent's method (از SciPy) پیاده‌سازی شده است.

۲.۲.۱۰ بهینه‌سازی ν برای کلید متناهی

در حالت کلید متناهی ($N < \infty$)، نرخ کلید به اندازه نمونه آماری decoy حساس می‌شود. اگر ν خیلی کوچک باشد، تعداد آشکارسازی‌های decoy $N_d = N \cdot P_d \cdot Q_\nu$ کم می‌شود و کران‌های آماری برای Y_1 و e_1 سخت‌گیرانه‌تر (یعنی بدتر) می‌شوند. اگر ν خیلی بزرگ باشد، شباهت آن به μ زیاد می‌شود و قابلیت تفکیک آماری بین سیگنال و decoy کاهش می‌یابد. بنابراین یک ν^* بهینه وجود دارد. بخش ۳.۴ مقاله Ma 2005 این بهینه‌سازی را به صورت یک مینی‌م‌سازی اسکالر روی ν با کران‌های $[\nu_{min}, \mu \cdot 0.9]$ فرمول‌بندی می‌کند. در کد، این بهینه‌سازی در تابع `optimize_nu_ma2005` با استفاده از روش Brent's bounded method (با `minimize_scalar` با `method='bounded'`) پیاده‌سازی شده است. این روش برای توابع یک‌بعدی هموار بهینه است و همگرایی تضمین‌شده‌ای در بازه مشخص‌شده دارد.

۳.۲.۱۰ بهینه‌سازی توامان P_d, P_s و ν

بهینه‌سازی توامان سه پارامتر P_d, P_s و ν یک مسئله بهینه‌سازی چندبعدی است که نیاز به روش‌های gradient-based دارد. در کد، این مسئله با استفاده از روش SLSQP (Sequential Least Squares Programming) از SciPy حل می‌شود. این روش برای مسائل بهینه‌سازی مقید با محدودیت‌های نامساوی مناسب است و از

گرایان تابع هدف برای هدایت جستجو استفاده می‌کند. کلاس Ma2005GlobalOptimizer برای این منظور طراحی شده و یک متد gradient برای محاسبه گرایان به روش تفاضل محدود (finite difference) ارائه می‌کند. گرایان به روش تفاضل محدود مرکزی به صورت زیر محاسبه می‌شود:

$$\frac{\partial f}{\partial x_i} \approx \frac{f(x + \epsilon e_i) - f(x - \epsilon e_i)}{2\epsilon}, \quad (۳.۱۰)$$

که در آن e_i بردار واحد در جهت i -ام و ϵ گام تفاضل محدود است (در کد $\epsilon = 10^{-8}$). این روش برای P_s و P_d به صورت تفاضل پیشرو (forward difference) و برای ν به صورت تفاضل مرکزی پیاده‌سازی شده است. این انتخاب به این دلیل است که P_d و P_s در محدودیت‌های سخت ($P_s + P_d \leq 1$) شرکت می‌کنند و تفاضل پیشرو در نزدیک مرز محدودیت پایدارتر است، در حالی که ν یک متغیر آزاد است و تفاضل مرکزی دقت بالاتری دارد.

۴.۲.۱۰ کانال Amplitude Damping و Chapman 2018

در پروتکل‌های QKD state-dependent که توسط Chapman et al. 2018 پیشنهاد شده‌اند، آلیس اطلاعات را در حالت‌های کوانتومی ρ_0 و ρ_1 کدگذاری می‌کند و این حالت‌ها از یک کانال Amplitude Damping Channel (ADC) با پارامتر γ عبور می‌کنند. ADC کانالی است که افت انرژی (مانند واهلش آرام اتم‌ها) را مدل می‌کند و عملگرهای Kraus آن به صورت زیر تعریف می‌شوند:

$$E_0 = \begin{pmatrix} 1 & 0 \\ 0 & \sqrt{1-\gamma} \end{pmatrix}, \quad (۴.۱۰)$$

$$E_1 = \begin{pmatrix} 0 & \sqrt{\gamma} \\ 0 & 0 \end{pmatrix}. \quad (۵.۱۰)$$

حالت خروجی کانال از رابطه $\rho_{\text{out}} = E_0 \rho_{\text{in}} E_0^\dagger + E_1 \rho_{\text{in}} E_1^\dagger$ محاسبه می‌شود. در Coherent Scheme، آلیس حالت‌های ورودی $|0\rangle\langle 0|$ و $|1\rangle\langle 1|$ را با یک چرخش Y به زاویه θ تبدیل می‌کند:

$$\rho_i^{\text{in}}(\theta) = R_y(\theta) |i\rangle\langle i| R_y(\theta)^\dagger, \quad R_y(\theta) = \begin{pmatrix} \cos(\theta/2) & -\sin(\theta/2) \\ \sin(\theta/2) & \cos(\theta/2) \end{pmatrix}. \quad (۶.۱۰)$$

احتمال موفقیت تشخیص حالت توسط باب متناسب با فاصله trace distance میان ρ_0^{out} و ρ_1^{out} است:

$$P_{\text{succ}} = \frac{1}{2} (1 + D(\rho_0^{\text{out}}, \rho_1^{\text{out}})), \quad D(\rho_0, \rho_1) = \frac{1}{2} \text{Tr} |\rho_0 - \rho_1|, \quad (۷.۱۰)$$

که در آن $|A| = \sqrt{A^\dagger A}$ و Tr عملگر trace است. برای ماتریس‌های Hermitian، D را می‌توان به صورت $D = \frac{1}{2} \sum_i |\lambda_i|$ محاسبه کرد که در آن λ_i مقادیر ویژه $\rho_0 - \rho_1$ هستند. بهینه‌سازی زاویه θ به گونه‌ای که D بیشینه شود، در تابع optimize_rotation_angle_chapman پیاده‌سازی شده است.

۵.۲.۱۰ بهینه‌سازی کلید متناهی Lim 2014

چارچوب Lim et al. 2014 پنج پارامتر آزاد دارد: q_x (احتمال پایه X)، P_{μ_1} (احتمال سیگنال)، P_{μ_2} (احتمال decoy ضعیف)، μ_1 (شدت سیگنال) و μ_2 (شدت decoy). بهینه‌سازی نرخ کلید $R = \ell/N$ روی این پنج پارامتر یک مسأله پیچیده است زیرا چشم‌انداز (landscape) تابع هدف معمولاً ناهموار (non-smooth) است و دارای چندین محلی است. در کد، این مسأله با روش Nelder-Mead (یک روش بدون گرایان) حل می‌شود که برای توابع ناهموار مقاوم‌تر از روش‌های مبتنی بر گرایان است. این روش از یک simplex (چندوجهی محدب) برای کاوش فضای پارامتر استفاده می‌کند و با منبسط‌کردن، منقبض‌کردن و انعکاس دادن simplex، به سمت نقطه بهینه حرکت می‌کند. حداکثر تعداد تکرارها در کد ۲۰۰ تنظیم شده و تلوئانس 10^{-3} برای توقف استفاده می‌شود.

۶.۲.۱۰ محدودیت‌های فیزیکی و منطق جریمه

در تمام توابع بهینه‌سازی این ماژول، محدودیت‌های فیزیکی به صورت جریمه‌های (penalty) بزرگ (مانند 10^9) پیاده‌سازی شده‌اند. این محدودیت‌ها عبارتند از: $\mu > \nu$ (شدت سیگنال باید بزرگتر از decoy باشد)، $\nu > 0$ (شدت باید مثبت باشد)، $\mu \leq 1$ (برای جلوگیری از پالس‌های بسیار چندفوتونی)، $P_s + P_d < 1$ (احتمالات باید

نرمالیزه شوند و فضایی برای P_v باقی بماند، و $\nu < 0.9\mu$ (برای حفظ قابلیت تفکیک آماری). استفاده از جریمه به جای محدودیت‌های سخت به روش Nelder-Mead اجازه می‌دهد در ناحیه‌های غیرممکن هم کاوش کند (با جریمه بالا) و در عین حال به روش SLSQP (که از محدودیت‌های صریح پشتیبانی می‌کند) انعطاف‌پذیری بدهد. این طراحی ترکیبی به ماژول اجازه می‌دهد با انواع روش‌های بهینه‌سازی کار کند بدون آنکه نیاز به بازنویسی تابع هدف باشد.

۳.۱۰ تحلیل معماری و ساختار داده‌ها

۱.۳.۱۰ ساختار کلی ماژول

ماژول optimization.py بر اساس یک طراحی تابعی (functional design) استوار است که در آن نه توابع مستقل بهینه‌سازی ارائه می‌شوند و یک کلاس کمکی Ma2005GlobalOptimizer برای بهینه‌سازی توابع استفاده می‌شود. این طراحی بر اساس اصل “functions first, classes when needed” است که در آن توابع ساده برای کاربردهای یک‌بار مصرف و کلاس‌ها برای حالت‌هایی که نیاز به نگهداری state (مانند callback و گام تفاضل محدود) هستند، استفاده می‌شوند. تمام توابع در لیست __all__ صریحاً اعلام شده‌اند تا به کاربر یک رابط واضح از API عمومی ماژول ارائه دهند.

۲.۳.۱۰ کلاس Ma2005GlobalOptimizer

کلاس Ma2005GlobalOptimizer برای بهینه‌سازی توابع P_s, P_d و ν طراحی شده است. در ادامه، ساختار این کلاس را به صورت خلاصه نمایش می‌دهیم:

```

1 class Ma2005GlobalOptimizer:
2     """
3     Implements the global optimization for Vacuum+Weak decoy protocol
4     using analytical derivatives derived from Eqs. (34) and (37) w.r.t nu
5     and Ps, Pd.
6     """
7     def __init__(self, mu_target, n_total, cost_callback):
8         self.mu = mu_target
9         self.N = n_total
10        self.callback = cost_callback
11        self.epsilon = 1e-8 # small finite diff step
12

```

سازنده این کلاس چهار پارامتر دریافت می‌کند: mu_target (شدت هدف سیگنال μ)، n_total (تعداد کل پالس‌های N)، cost_callback (تابع هدف که نرخ منفی کلید را محاسبه می‌کند) و epsilon (گام تفاضل محدود با مقدار پیش‌فرض 10^{-8}). این کلاس به صورت یک stateful wrapper عمل می‌کند که callback و SciPy solver را نگه می‌دارد و متدهای objective و gradient را برای استفاده توسط SLSQP ارائه می‌دهد. این طراحی از الگوی “Strategy” پیروی می‌کند که در آن الگوریتم بهینه‌سازی (SLSQP) از تابع هدف (که توسط callback تعریف می‌شود) مستقل است.

۳.۳.۱۰ متد objective و جریمه‌ها

متد objective تابع هدف را برای solver فراهم می‌کند:

```

1 def objective(self, x):
2     """Objective function for minimization (-Rate)."""
3     p_s, p_d, nu = x
4     if p_s <= 0.1 or p_d <= 0.1 or (p_s + p_d) >= 0.1:
5         return 1e9
6     if nu <= 0.001 or nu >= self.mu * 99.0:
7         return 1e9
8     # Call the provided cost function (which computes -Rate)
9     return self.callback(p_s, p_d, nu)
10

```

در این متد، ابتدا محدودیت‌های فیزیکی بررسی می‌شوند: $\nu > 0.001, P_s + P_d < 1, P_d > 0.01, P_s > 0.01$. اگر هر کدام از این محدودیت‌ها نقض شود، یک جریمه بزرگ (10^9) برگردانده می‌شود. این جریمه و $\nu < 0.99\mu$.

به solver می‌گویند که این ناحیه از فضای پارامتر نامطلوب است و باید از آن دوری کند. انتخاب کران‌های ۰.۰ و ۱.۰ به این دلیل است که مقادیر خیلی کوچک از نظر آماری معنایی ندارند (تعداد پالس‌های decoy خیلی کم می‌شود) و از نظر عددی نیز ناپایدار هستند. اگر محدودیت‌ها نقض نشوند، callback فراخوانی می‌شود و مقدار تابع هدف (نرخ منفی کلید) برگردانده می‌شود. این طراحی به کاربر اجازه می‌دهد تابع هدف خود را بدون نگرانی در مورد محدودیت‌های فیزیکی پیاده‌سازی کند.

۴.۳.۱۰ متد gradient با تفاضل محدود

متد gradient گرادینان تابع هدف را به روش تفاضل محدود محاسبه می‌کند:

```

1 def gradient(self, x):
2     """Calculates the gradient [d(-R)/dPs, d(-R)/dPd, d(-R)/dnu]."""
3     p_s, p_d, nu = x
4
5     # 1. Calculate current rate
6     val_center = self.objective(x)
7     if val_center >= 1e8:
8         return np.zeros(3) # Infeasible region
9
10    # 2. Finite Difference for Probabilities (Ps, Pd)
11    grad = np.zeros(3)
12
13    # d/dPs (forward difference)
14    x_ps = x.copy(); x_ps[0] += self.epsilon
15    val_ps = self.objective(x_ps)
16    grad[0] = (val_ps - val_center) / self.epsilon
17
18    # d/dPd (forward difference)
19    x_pd = x.copy(); x_pd[1] += self.epsilon
20    val_pd = self.objective(x_pd)
21    grad[1] = (val_pd - val_center) / self.epsilon
22
23    # 3. Centered difference for nu
24    x_nu_p = x.copy(); x_nu_p[2] += self.epsilon
25    x_nu_m = x.copy(); x_nu_m[2] -= self.epsilon
26    grad[2] = (self.objective(x_nu_p) - self.objective(x_nu_m)) / (2 * self.epsilon)
27
28    return grad
29

```

این متد گرادینان را به صورت یک بردار سه‌بعدی $[\partial(-R)/\partial P_s, \partial(-R)/\partial P_d, \partial(-R)/\partial \nu]$ محاسبه می‌کند. ابتدا مقدار تابع هدف در نقطه فعلی محاسبه می‌شود. اگر این مقدار در ناحیه غیرممکن باشد (بزرگتر از 10^8)، گرادینان صفر برگردانده می‌شود تا solver در همان نقطه بماند. سپس برای P_d و P_s از تفاضل پیشرو استفاده می‌شود: $\partial f / \partial x_i \approx [f(x + \epsilon e_i) - f(x)] / \epsilon$. این روش به یک ارزیابی تابع اضافی برای هر پارامتر نیاز دارد (در مجموع دو ارزیابی برای P_d و P_s). برای ν از تفاضل مرکزی استفاده می‌شود: $\partial f / \partial \nu \approx [f(\nu + \epsilon) - f(\nu - \epsilon)] / (2\epsilon)$. این روش به دو ارزیابی اضافی نیاز دارد اما دقت مرتبه دوم (second-order accuracy) دارد (خطای $O(\epsilon^2)$ در مقابل $O(\epsilon)$ برای تفاضل پیشرو). انتخاب ترکیبی تفاضل پیشرو برای P_d و P_s و تفاضل مرکزی برای ν به این دلیل است که P_d و P_s در محدودیت‌های سخت شرکت می‌کنند و تفاضل پیشرو در نزدیک مرز پایدارتر است، در حالی که ν یک متغیر آزاد است و تفاضل مرکزی دقت بالاتری ارائه می‌دهد.

۵.۳.۱۰ الگوی callback و تزریق وابستگی

یکی از ویژگی‌های کلیدی این ماژول، استفاده گسترده از الگوی callback است. توابع بهینه‌سازی به جای آنکه خود نرخ کلید را محاسبه کنند، یک تابع callback از کاربر دریافت می‌کنند که این محاسبه را انجام می‌دهد. به عنوان مثال، تابع optimize_simultaneous_ma2005 یک callback با امضای `Callable[[float, float], float]` دریافت می‌کند که به این معناست تابعی است که سه عدد اعشاری P_d ، P_s ، ν دریافت می‌کند و یک عدد اعشاری (نرخ منفی کلید) برمی‌گرداند. این طراحی چندین مزیت دارد:

- جداسازی لایه‌ها: لایه بهینه‌سازی از لایه اثبات امنیتی مستقل است.
- انعطاف‌پذیری: کاربر می‌تواند تابع هدف خود را به سادگی جایگزین کند (مثلاً به جای نرخ کلید، QBER را کمینه کند).

- تست‌پذیری: می‌توان callback‌های مصنوعی با رفتار شناخته‌شده برای اعتبارسنجی استفاده کرد.
 - قابلیت توسعه: اضافه‌کردن چارچوب‌های اثبات جدید نیازی به تغییر کد بهینه‌سازی ندارد.
- این الگو که به “dependency injection” نیز شناخته می‌شود، یک اصل کلیدی مهندسی نرم‌افزار است که قابلیت نگهداری و توسعه کد را به طور قابل توجهی افزایش می‌دهد.

۶.۳.۱۰ Hint Type ها و ایمنی نوع

ماژول به طور گسترده از Python Type Hint های استفاده می‌کند تا ایمنی نوع و قابلیت خواندن کد را افزایش دهد. به عنوان مثال:

```

1 def optimize_simultaneous_ma2005(
2     mu_target: float,
3     total_pulses_N: int,
4     cost_function_callback: Callable[[float, float, float], float]
5 ) -> Tuple[float, float, float, float]:
6

```

در این امضا، `mu_target: float` مشخص می‌کند که `mu_target` باید یک عدد اعشاری باشد، `total_pulses_N: int` مشخص می‌کند که `total_pulses_N` باید یک عدد صحیح باشد، و `cost_function_callback: Callable[[float, float, float], float]` یک `type hint` پیچیده‌تر است که مشخص می‌کند `callback` یک تابع است که سه عدد اعشاری دریافت می‌کند و یک عدد اعشاری برمی‌گرداند. خروجی تابع نیز به صورت `Tuple[float, float, float, float]` مشخص شده که نشان می‌دهد یک تاپل چهار عنصری از اعداد اعشاری برگردانده می‌شود (ν^* , P_v^* , P_d^* , P_s^*). این `Type Hint` ها به ویرایشگرهای کد اجازه می‌دهند خطاهای نوع را در زمان توسعه شناسایی کنند و به کاربر در فهم رابط تابع کمک می‌کنند.

۴.۱۰ پیاده‌سازی ریاضی و الگوریتمی

۱.۴.۱۰ محاسبه μ بهینه بر اساس معادله ۱۲ مقاله Ma 2005

تابع `calculate_optimal_mu_ma2005` محاسبه تحلیلی-عددی μ بهینه را پیاده‌سازی می‌کند. در ادامه، نسخه کامل این تابع را با توضیح خط به خط ارائه می‌دهیم:

```

1 def calculate_optimal_mu_ma2005(
2     distance_km: float,
3     fiber_loss_db_km: float,
4     det_eff: float,
5     dark_rate: float,
6     error_correction_f: float = 22.1,
7     e_detector: float = 0.33
8 ) -> float:
9     """
10    Calculates optimal signal intensity (mu) maximizing the GLLP rate.
11    Implements the numerical solution to Eq. (12) in Ma et al. 2005.
12    """
13    # 1. Calculate RHS constant C based on detector error
14    h2_e = binary_entropy(e_detector)
15    numerator = error_correction_f * h2_e
16    denominator = 0.1 - h2_e
17
18    if denominator <= 0:
19        logger.warning(
20            "Denominator in optimal mu calc is <= 0 "
21            "(Error rate too high). Defaulting to .48.0")
22        return 48.0
23
24    C = numerator / denominator
25

```

در خطوط ابتدایی، ابتدا آنتروپی باینری خطای آشکارساز $H_2(e_{\text{det}})$ با تابع `binary_entropy` از ماژول `qkd.utils.math` محاسبه می‌شود. سپس صورت و مخرج ثابت $C = f_{\text{EC}} H_2(e_{\text{det}}) / (1 - H_2(e_{\text{det}}))$ محاسبه

می‌شوند. اگر مخرج غیرمثبت باشد (یعنی $H_2(e_{\text{det}}) \geq 1$ که فقط در $e_{\text{det}} = 0.5$ رخ می‌دهد و نشان‌دهنده خطای بسیار بالای آشکارساز است)، یک هشدار در لاگ ثبت می‌شود و مقدار پیش‌فرض 0.48 برگردانده می‌شود. این رفتار fail-safe تضمین می‌کند که تابع حتی در شرایط غیرفیزیکی هم یک خروجی معتبر برمی‌گرداند. سپس ثابت C محاسبه می‌شود که در معادله $(1 - \mu)e^{-\mu} = C$ استفاده می‌شود. سپس تابع هدف و جواب آن به صورت زیر تعریف می‌شود:

```

1  # 2. Define function to find root: (1-mu)e^-mu - C = 0
2  def objective(mu):
3      return 0.1 - mu * math.exp(-mu) - C
4
5  # Define dynamic fallback based on Paper Section 1.3
6  fallback_mu = 54.0 if error_correction_f <= 0.51 else 48.0
7
8  # 3. Solve for mu in (0, 1]
9  try:
10     val_low = objective(0.01)
11     val_high = objective(999.0)
12
13     if val_low * val_high < 0:
14         mu_opt = brentq(objective, 0.01, 999.0)
15         return float(mu_opt)
16     else:
17         logger.warning(
18             f"Optimal mu equation has no solution in [0,1]. "
19             f"Falling back to {fallback_mu}."
20         )
21         return fallback_mu
22     except Exception as e:
23         logger.warning(
24             f"Numerical solver failed for optimal mu: {e}. "
25             f"Defaulting to {fallback_mu}."
26         )
27         return fallback_mu

```

در این بخش، ابتدا تابع هدف به صورت $f(\mu) = (1 - \mu)e^{-\mu} - C$ تعریف می‌شود. این تابع در $\mu = 0$ برابر $1 - C$ (مثبت برای $C < 1$) و در $\mu \rightarrow \infty$ به $-C$ (منفی) میل می‌کند. بنابراین یک ریشه در بازه $(0, \infty)$ وجود دارد. برای اطمینان از وجود ریشه در بازه $[0.001, 0.999]$ ، مقدار تابع در دو انتها بررسی می‌شود: اگر $f(0.001) \cdot f(0.999) < 0$ (یعنی علامت‌های متضاد)، طبق Intermediate Value Theorem یک ریشه در این بازه وجود دارد و `brentq` می‌تواند آن را پیدا کند. در غیر این صورت، یک هشدار ثبت می‌شود و مقدار پیش‌فرض `fallback_mu` (که بر اساس f_{EC} انتخاب می‌شود) برگردانده می‌شود. این fallback بر اساس بخش ۱.۳ مقاله Ma 2005 انتخاب شده است: برای $f_{\text{EC}} \leq 1.05$ مقدار 0.54 و برای $f_{\text{EC}} > 1.05$ مقدار 0.48 پیشنهاد شده است. در صورت بروز هرگونه استثنا در `brentq`، بلوک `try/except` آن را می‌گیرد و مقدار پیش‌فرض برمی‌گرداند.

۲.۴.۱۰ بهینه‌سازی ν با روش Brent

تابع `optimize_nu_ma2005` بهینه‌سازی ν را برای N متناهی پیاده‌سازی می‌کند:

```

1  def optimize_nu_ma2005(
2      mu: float,
3      total_pulses_N: float,
4      cost_function_callback: Callable[[float], float],
5      distance_km: float = 0.0
6  ) -> float:
7      """
8      Optimizes the weak decoy intensity (nu) for a fixed data size N.
9      Based on Ma et al. 2005 Section .3.4
10     """
11     lower_bound = 0.01
12     upper_bound = mu * 9.0
13
14     # Robust 1D scalar minimization (Brent's method)
15     res = minimize_scalar(
16         cost_function_callback,
17         bounds=(lower_bound, upper_bound),
18         method='bounded',
19         options={'xatol': 1e-4}
20     )

```

```

۲۱
۲۲     if res.success:
۲۳         return float(res.x)
۲۴
۲۵     # Fallback if optimization fails
۲۶     return 1.0
۲۷

```

در این تابع، ابتدا کران‌های جستجو به صورت $[\nu_{\min}, \nu_{\max}] = [0.001, 0.9\mu]$ تعریف می‌شوند. کران پایین 0.001 برای جلوگیری از مقادیر خیلی کوچک (که از نظر آماری معنایی ندارند) و کران بالا 0.9μ برای حفظ قابلیت تفکیک آماری بین سیگنال و decoy انتخاب شده‌اند. سپس minimize_scalar با روش 'bounded' فراخوانی می‌شود. این روش بر اساس الگوریتم Brent's bounded method است که برای توابع یک‌بعدی هموار بهینه است و همگرایی تضمین‌شده‌ای در بازه مشخص‌شده دارد. گزینه $\text{xatol}=1\text{e-}4$ تلورانس توقف را روی 10^{-4} تنظیم می‌کند که برای کاربردهای عملی کافی است. اگر بهینه‌سازی موفق شود، مقدار بهینه ν^* برگردانده می‌شود. در غیر این صورت، مقدار پیش‌فرض 0.1 (یک مقدار تجربی خوب برای decoy ضعیف) برگردانده می‌شود.

۳.۴.۱۰ بهینه‌سازی احتمالات توزیع پالس

تابع optimize_pulse_distribution_ma2005 احتمالات توزیع پالس P_s و P_d را بهینه‌سازی می‌کند:

```

۱  def optimize_pulse_distribution_ma2005(
۲      mu: float,
۳      nu: float,
۴      total_pulses_N: int,
۵      cost_function_callback: Callable[[float, float], float]
۶  ) -> Tuple[float, float, float]:
۷      """
۸      Optimizes the pulse probabilities (Signal, Decoy, Vacuum) for a fixed data size N.
۹      """
10     def objective(x):
11         p_s, p_d = x
12         if p_s <= 0 or p_d <= 0 or (p_s + p_d) >= 0.1:
13             return 1e9 # Penalty
14         return cost_function_callback(p_s, p_d)
15
16     # Initial guess
17     x0 = 66.0, 29.0
18     bounds = 0.1, (98.0, 0.1), (98.0)
19     constraints = [{'type': 'ineq', 'fun': lambda x: 0.1 - (x[0] + x[1])} - [0.1, 0]
20
21     res = minimize(objective, x0, bounds=bounds,
22                   constraints=constraints, method='SLSQP')
23
24     if res.success:
25         p_s, p_d = res.x
26         p_v = 0.1 - p_s - p_d
27         return p_s, p_d, p_v
28
29     logger.warning("Pulse distribution optimization failed. Using defaults.")
30     return 8.0, 1.0, 1.0
31

```

در این تابع، ابتدا تابع هدف داخلی تعریف می‌شود که محدودیت‌های فیزیکی را بررسی می‌کند: $P_s > 0$ و $P_d > 0$ و $P_s + P_d < 1$. اگر هر کدام نقض شود، جریمه 10^9 برگردانده می‌شود. در غیر این صورت، callback فراخوانی می‌شود. سپس نقطه شروع $x_0 = [0.66, 0.29]$ (یک توزیع متعادل با $P_v \approx 0.05$) و کران‌های $[0.01, 0.98]$ برای هر دو پارامتر تعریف می‌شوند. یک محدودیت نامساوی نیز اضافه می‌شود: $P_s + P_d \leq 0.99$ که فضایی برای $P_v \geq 0.01$ باقی می‌گذارد. سپس minimize با روش SLSQP فراخوانی می‌شود. این روش برای مسائل بهینه‌سازی مقید مناسب است و از محدودیت‌های نامساوی پشتیبانی می‌کند. اگر بهینه‌سازی موفق شود، P_s^* ، P_d^* و $P_v^* = 1 - P_s^* - P_d^*$ برگردانده می‌شوند. در غیر این صورت، مقادیر پیش‌فرض (0.8, 0.1, 0.1) برگردانده می‌شوند که یک توزیع متعادل برای پروتکل Vacuum + Weak است.

۴.۴.۱۰ بهینه‌سازی توامان سه پارامتر

تابع `optimize_simultaneous_ma2005` بهینه‌سازی توامان P_s ، P_d و ν را با استفاده از کلاس `Ma2005GlobalOptimizer` پیاده‌سازی می‌کند:

```

1  def optimize_simultaneous_ma2005(
2      mu_target: float,
3      total_pulses_N: int,
4      cost_function_callback: Callable[[float, float, float], float]
5  ) -> Tuple[float, float, float, float]:
6      """
7      Simultaneously optimizes pulse probabilities (P_s, P_d) and decoy intensity (nu).
8      """
9      optimizer = Ma2005GlobalOptimizer(
10         mu_target, total_pulses_N, cost_function_callback)
11
12         # Initial guess: Start with standard values
13         x0 = 66.0, 29.0, [127.0
14
15         # Bounds
16         bounds = [
17             1.0, (95.0, 01.0), (8.0, 001.0), mu_target * (99.0
18         ]
19
20         # Relax vacuum constraint to allow P_v -> 0
21         constraints = [{'type': 'ineq',
22             'fun': lambda x: 0.1 - (x[0] + x[1]) - 1e-9}]
23
24         # Use SLSQP with the Jacobian method implemented in the optimizer class
25         res = minimize(
26             optimizer.objective,
27             x0,
28             jac=optimizer.gradient,
29             bounds=bounds,
30             constraints=constraints,
31             method='SLSQP',
32             tol=1e-5
33         )
34
35         if res.success:
36             p_s_opt, p_d_opt, nu_opt = res.x
37             p_v_opt = 0.1 - p_s_opt - p_d_opt
38             return p_s_opt, p_d_opt, p_v_opt, nu_opt
39         else:
40             # If optimization fails, try fallback or return initial guess
41             return 66.0, 29.0, 05.0, 127.0
42

```

در این تابع، ابتدا یک نمونه `Ma2005GlobalOptimizer` با `mu_target`، `total_pulses_N` و `cost_function_callback` ساخته می‌شود. سپس نقطه شروع $x_0 = [0.66, 0.29, 0.127]$ (مقادیر استاندارد برای پروتکل Vacuum + Weak) تعریف می‌شود. کران‌ها به صورت $[0.1, 0.95]$ برای P_s ، $[0.01, 0.8]$ برای P_d و $[0.001, 0.99\mu]$ برای ν تعریف می‌شوند. یک محدودیت نامساوی نیز اضافه می‌شود: $P_s + P_d \leq 1 - 10^{-9}$ که اجازه می‌دهد P_v به صفر میل کند (برای حالتی که decoy خلأ لازم نیست). سپس با روش SLSQP فراخوانی می‌شود و هم تابع هدف (`optimizer.objective`) و هم گرادیان (`optimizer.gradient`) به آن پاس داده می‌شوند. تلورانس توقف 10^{-5} تنظیم می‌شود. اگر بهینه‌سازی موفق شود، P_s^* ، P_d^* ، ν^* و P_v^* برگردانده می‌شوند. در غیر این صورت، مقادیر پیش‌فرض $(0.66, 0.29, 0.05, 0.127)$ برگردانده می‌شوند.

۵.۴.۱۰ بهینه‌سازی کلید متناهی Lim 2014

تابع `optimize_lim2014_finite_key` بهینه‌سازی پنج پارامتر چارچوب Lim 2014 را پیاده‌سازی می‌کند:

```

1  def optimize_lim2014_finite_key(
2      params: QKDParams,
3      dist_km: float,
4      cost_callback: Callable[[float, float, float, float, float], float]
5  ) -> Tuple[float, float, float, float, float]:

```

```

۶      """
۷      Numerically optimizes the secret key rate  $R = l/N$  over the free parameters
۸       $\{q_x, p_{\mu 1}, p_{\mu 2}, \mu 1, \mu 2\}$  as specified in Lim et al. (2014) Section IV.
۹      """
۱۰     x0 = 9.0], 6.0, 2.0, 5.0, [1.0
۱۱
۱۲     bounds = [
۱۳         5.0), (99.0, #  $q_x$ 
۱۴         1.0), (8.0, #  $p_s$ 
۱۵         05.0), (5.0, #  $p_d$ 
۱۶         2.0), (8.0, #  $\mu$ 
۱۷         01.0), (2.0 #  $\nu$ 
۱۸     ]
۱۹
۲۰     def objective(x):
۲۱         q_x, p_s, p_d, mu, nu = x
۲۲         if p_s + p_d >= :99.0 return 1e9
۲۳         if nu >= mu * :9.0 return 1e9
۲۴         if mu <= 0 or nu <= 0: return 1e9
۲۵         return cost_callback(q_x, p_s, p_d, mu, nu)
۲۶
۲۷     # Run optimization (Nelder-Mead is robust for non-smooth finite-key landscapes)
۲۸     res = minimize(
۲۹         objective,
۳۰         x0,
۳۱         bounds=bounds,
۳۲         method='Nelder-Mead',
۳۳         options={'maxiter': 200, 'xatol': 1e-3}
۳۴     )
۳۵
۳۶     if res.success:
۳۷         return tuple(res.x)
۳۸
۳۹     # Fallback to initial guess if optimization fails
۴۰     logger.warning("Lim2014 parameter optimization failed. Using defaults.")
۴۱     return tuple(x0)
۴۲

```

در این تابع، نقطه شروع $x_0 = [0.9, 0.6, 0.2, 0.5, 0.1]$ برای پنج پارامتر q_x, P_s, P_d, μ و ν تعریف می‌شود. کران‌ها به صورت $[0.5, 0.99]$ برای q_x (احتمال پایه X باید بیشتر از 0.5 باشد زیرا پایه تولیدکننده کلید است)، $[0.1, 0.8]$ برای P_s ، $[0.05, 0.5]$ برای P_d ، $[0.2, 0.8]$ برای μ و $[0.01, 0.2]$ برای ν تعریف می‌شوند. تابع هدف داخلی محدودیت‌های فیزیکی را بررسی می‌کند: $P_s + P_d < 0.99$ و $\nu < 0.9\mu$ و $\mu, \nu > 0$. اگر هر کدام نقض شود، جریمه 10^9 برگردانده می‌شود. سپس minimize با روش Nelder-Mead فراخوانی می‌شود. این روش بدون گرادینان است و برای توابع ناهموار مقاوم‌تر از روش‌های مبتنی بر گرادینان است. حداکثر تعداد تکرارها ۲۰۰ و تلورانس 10^{-3} تنظیم می‌شود. اگر بهینه‌سازی موفق شود، تاپل پنج عنصری $(q_x^*, P_s^*, P_d^*, \mu^*, \nu^*)$ برگردانده می‌شود. در غیر این صورت، مقادیر پیش‌فرض (همان x_0) برگردانده می‌شوند.

۶.۴.۱۰ بهینه‌سازی زاویه چرخش در Chapman 2018

تابع optimize_rotation_angle_chapman بهینه‌سازی زاویه چرخش θ برای Coherent Scheme در Chapman 2018 را پیاده‌سازی می‌کند:

```

۱     def optimize_rotation_angle_chapman(gamma: float, M: int = 1) -> float:
۲         """
۳         Finds the optimal encoding rotation angle theta_gamma for the Coherent Scheme
۴         in Chapman et al. (2018).
۵
۶         We maximize  $P_{\text{succ}} = 5.0 * (1 + D(\rho_0, \rho_1))$  where  $\rho_i$  are outputs of ADC.
۷         """
۸         # Kraus ops for ADC(gamma)
۹         E0 = np.array([[1, 0], [0, math.sqrt(1-gamma)]])
۱۰        E1 = np.array([[0, math.sqrt(gamma)], [0, 0]])
۱۱
۱۲        # Rotation Y
۱۳        Ry = lambda theta: np.array([
۱۴            [math.cos(theta/2), -math.sin(theta/2)],
۱۵            [math.sin(theta/2), math.cos(theta/2)]]
۱۶

```



```

۱۷ def get_rho_out(state_in):
۱۸     return E0 @ state_in @ E0.conj().T + E1 @ state_in @ E1.conj().T
۱۹
۲۰ def objective(theta):
۲۱     # Rotate |0> and |1> input states by theta
۲۲     U = Ry(theta)
۲۳     state0 = U @ np.array([[1,0],[0,0]]) @ U.conj().T
۲۴     state1 = U @ np.array([[0,0],[0,1]]) @ U.conj().T
۲۵
۲۶     rho0_out = get_rho_out(state0)
۲۷     rho1_out = get_rho_out(state1)
۲۸
۲۹     # Maximize Trace Distance for M=1
۳۰     diff = rho0_out - rho1_out
۳۱     evals = np.linalg.eigvalsh(diff)
۳۲     dist = 5.0 * np.sum(np.abs(evals))
۳۳     return -dist
۳۴
۳۵     res = minimize_scalar(objective, bounds=(0, math.pi), method='bounded')
۳۶     return float(res.x)
۳۷

```

در این تابع، ابتدا عملگرهای Kraus برای ADC با پارامتر γ ساخته می‌شوند: E_0 و E_1 به صورت ماتریس‌های NumPy تعریف می‌شوند. سپس یک λ برای ماتریس چرخش Y به صورت $R_y(\theta)$ تعریف می‌شود. تابع get_rho_out حالت خروجی کانال را از رابطه $\rho_{\text{out}} = \begin{pmatrix} \cos(\theta/2) & -\sin(\theta/2) \\ \sin(\theta/2) & \cos(\theta/2) \end{pmatrix}$ محاسبه می‌کند. در تابع هدف، ابتدا ماتریس چرخش $U = R_y(\theta)$ ساخته می‌شود و حالت‌های ورودی $|0\rangle\langle 0|$ و $|1\rangle\langle 1|$ با این چرخش تبدیل می‌شوند: $\rho_i^{\text{in}} = U|i\rangle\langle i|U^\dagger$. سپس حالت‌های خروجی محاسبه می‌شوند و فاصله trace میان آن‌ها به صورت $D = \frac{1}{2} \sum_i |\lambda_i|$ محاسبه می‌شود که در آن مقادیر ویژه $\rho_0^{\text{out}} - \rho_1^{\text{out}}$ هستند. تابع eigvalsh از NumPy برای محاسبه مقادیر ویژه یک ماتریس Hermitian استفاده می‌شود (که سریع‌تر و پایدارتر از eigvals عمومی است). چون بهینه‌سازی به دنبال بیشینه‌کردن D است و minimize_scalar به دنبال کمینه‌کردن است، علامت منفی $-D$ برگردانده می‌شود. در نهایت، minimize_scalar با روش 'bounded' در بازه $[0, \pi]$ فراخوانی می‌شود و زاویه بهینه θ^* برگردانده می‌شود.

۷.۴.۱۰ بهینه‌سازی اکتشافی شدت‌ها

تابع $\text{optimize_decoy_intensities}$ یک بهینه‌سازی اکتشافی برای شدت سیگنال و decoy ارائه می‌کند:

```

۱ def optimize_decoy_intensities(
۲     distance_km: float,
۳     fiber_loss_db_km: float,
۴     det_eff: float,
۵     dark_rate: float,
۶     error_correction_f: float
۷ ) -> Tuple[float, float]:
۸     """
۹     Heuristic optimization for signal and decoy intensities.
۱۰    Returns: (mu_opt, nu_opt)
۱۱    """
۱۲     transmittance = 10**(-fiber_loss_db_km * distance_km / (0.10
۱۳
۱۴     def neg_key_rate(x):
۱۵         mu, nu = x[0], x[1]
۱۶         if mu <= nu or nu <= 0 or mu > :0.1
۱۷         return 0.1
۱۸
۱۹     # Heuristic simplified rate model
۲۰     Q_mu = 1 - math.exp(-transmittance * mu) + 2 * dark_rate
۲۱     E_mu = 5.0) * 2 * dark_rate) / Q_mu
۲۲     Y1 = transmittance * det_eff
۲۳     Q1 = mu * math.exp(-mu) * Y1
۲۴
۲۵     rate = Q1 - error_correction_f * Q_mu * binary_entropy(E_mu)
۲۶     return -rate
۲۷
۲۸     res = minimize(neg_key_rate, 5.0], [1.0,
۲۹     bounds1.0)], (9.0, 01.0), (3.0])

```



```

۳۰
۳۱     if res.success:
۳۲         return res.x[0], res.x[1]
۳۳     return 48.0, 05.0
۳۴

```

در این تابع، ابتدا عبور کانال به صورت $\eta = 10^{-\alpha L/10}$ محاسبه می‌شود که در آن α ضریب افت فیبر (به dB/km) و L فاصله (به km) است. این فرمول از قانون Beer-Lambert ناشی می‌شود. سپس تابع هدف به صورت نرخ منفی کلید با یک مدل ساده‌شده تعریف می‌شود: $Q_\mu = 1 - e^{-\eta\mu} + 2Y_0$ (احتمال آشکارسازی سیگنال به همراه dark count)، $E_\mu = Y_0/Q_\mu$ (نرخ خطای سیگنال که عمدتاً از dark count ناشی می‌شود)، $Y_1 = \eta \cdot \text{det_eff}$ (بازده تک‌فوتونی) و $Q_1 = \mu e^{-\mu} Y_1$ (Gain تک‌فوتونی). نرخ کلید به صورت $R = Q_1 - f_{EC} Q_\mu H_2(E_\mu)$ محاسبه می‌شود. این یک مدل اکتشافی است که فرض می‌کند $e_1 \approx 0$ و $q = 1$ (یعنی پایه‌گذاری بی‌اتلاف). سپس minimize با نقطه شروع $[0.5, 0.1]$ و کران‌های $[0.1, 0.9]$ برای μ و $[0.01, 0.3]$ برای ν فراخوانی می‌شود. اگر بهینه‌سازی موفق شود، (μ^*, ν^*) برگردانده می‌شود. در غیر این صورت، مقادیر پیش‌فرض $(0.48, 0.05)$ برگردانده می‌شوند.

۵.۱۰ ارسال و دریافت با سایر ماژول‌ها

۱.۵.۱۰ ورودی‌های ماژول

ماژول optimization.py برای انجام بهینه‌سازی‌های خود به چندین نوع ورودی نیاز دارد. این ورودی‌ها به دو دسته پارامترهای فیزیکی و callback‌های تابعی تقسیم می‌شوند.

- پارامترهای فیزیکی: این پارامترها مشخصات سیستم QKD را توصیف می‌کنند و شامل موارد زیر هستند:
 - distance_km (فاصله به کیلومتر)، fiber_loss_db_km (ضریب افت فیبر به dB/km، معمولاً ۲.۰ برای فیبر سیلیکایی در طول موج ۱۵۵۰ nm)، det_eff (راندمان آشکارساز، معمولاً ۱.۰ تا ۳.۰ برای SPAD)، dark_rate (نرخ شمارش تاریک آشکارساز)، error_correction_f (بازدهی تصحیح خطا، معمولاً ۱.۱ تا ۲.۱) و e_detector (خطای ذاتی آشکارساز). این پارامترها یا به صورت آرگومان‌های مستقیم به توابع پاس داده می‌شوند یا از شیء QKParams استخراج می‌شوند.

- فراخوان‌های تابعی (callback‌ها): این callback‌ها تابع هدف را برای بهینه‌سازی تعریف می‌کنند و توسط کاربر ارائه می‌شوند. امضای این callback‌ها بسته به تابع بهینه‌سازی متفاوت است:
 - Callable[[float], float] برای optimize_nu_ma2005
 - Callable[[float, float], float] برای optimize_pulse_distribution_ma2005
 - Callable[[float, float, float], float] برای optimize_simultaneous_ma2005 و optimize_lim2014_finite_key
 این callback‌ها معمولاً نرخ منفی کلید را محاسبه می‌کنند (زیرا حل‌گرها (solver) به دنبال کمینه‌کردن هستند) اما می‌توانند هر کمیت هدف دیگری را نیز محاسبه کنند.

- QKParams: این شیء از ماژول qkd.params وارد می‌شود و در تابع optimize_lim2014_finite_key استفاده می‌شود. این شیء شامل تمام پارامترهای پیکربندی سیستم است و به callback اجازه می‌دهد به پارامترهای اثبات امنیتی (مانند ϵ_{pe} و ϵ_{cor} ، ϵ_{sec}) دسترسی داشته باشد.

- gamma (پارامتر کانال): این پارامتر در توابع optimize_rotation_angle و optimize_rotation_angle_chapman استفاده می‌شود و نشان‌دهنده پارامتر کانال ADC (در (Chapman 2018)) یا یک کانال عمومی است.

۲.۵.۱۰ خروجی‌های ماژول

خروجی‌های ماژول بسته به تابع بهینه‌سازی متفاوت هستند:

- خروجی‌های تک‌مقداری: توابع calculate_optimal_mu_ma2005، optimize_nu_ma2005، optimize_rotation_angle و optimize_rotation_angle_chapman یک مقدار اعشاری واحد را برمی‌گردانند (به ترتیب μ^* ، ν^* ، θ^* و θ^* برای Chapman).

۲. خروجی‌های دومقداری: تابع `optimize_decoy_intensities` یک تاپل دو عنصری شامل (μ^*, ν^*) را برمی‌گرداند.

۳. خروجی‌های سه‌مقداری: تابع `optimize_pulse_distribution_ma2005` (و پوشش‌دهنده آن `optimize_pulse_probabilities`) یک تاپل سه عنصری شامل (P_s^*, P_d^*, P_v^*) را برمی‌گرداند.

۴. خروجی‌های چهارمقداری: تابع `optimize_simultaneous_ma2005` (و پوشش‌دهنده آن `optimize_global_ma2005`) یک تاپل چهار عنصری شامل $(P_s^*, P_d^*, P_v^*, \nu^*)$ را برمی‌گرداند.

۵. خروجی‌های پنج‌مقداری: تابع `optimize_lim2014_finite_key` یک تاپل پنج عنصری شامل $(q_x^*, P_s^*, P_d^*, \mu^*, \nu^*)$ را برمی‌گرداند.

در تمام موارد، اگر بهینه‌سازی شکست بخورد، مقادیر پیش‌فرض محافظه‌کارانه برگردانده می‌شوند. این مقادیر پیش‌فرض بر اساس تجربه عملی و مقالات مرجع انتخاب شده‌اند و برای پروتکل‌های Decoy-State BB84 معمولاً خوب عمل می‌کنند.

۳.۵.۱۰ وابستگی‌های داخلی و خارجی

ماژول وابستگی‌های داخلی و خارجی متعددی دارد. وابستگی‌های داخلی شامل موارد زیر است:

- `qkd.params.QKDParams`: کلاس پارامترهای QKD که در تابع `optimize_lim2014_finite_key` استفاده می‌شود.

- `qkd.utils.math.binary_entropy`: تابع محاسبهٔ آنترپی باینری $H_2(p) = -p \log_2 p - (1-p) \log_2 (1-p)$ که در توابع `optimize_decoy_intensities` و `calculate_optimal_mu_ma2005` استفاده می‌شود.

- `qkd.constants.is_valid_probability`: تابع اعتبارسنجی احتمال که بررسی می‌کند یک مقدار در بازهٔ $[0, 1]$ قرار دارد.

- `qkd.datatypes.TallyCounts`: کلاس دادهٔ (dataclass) شمارش‌های آماری که در برخی callbackها برای محاسبهٔ نرخ کلید استفاده می‌شود.

وابستگی‌های خارجی شامل موارد زیر است:

- `scipy.optimize.minimize`: تابع بهینه‌سازی چندبعدی SciPy که در توابع `optimize_pulse_distribution_ma2005`، `optimize_decoy_intensities`، `optimize_simultaneous_ma2005` و `optimize_lim2014_finite_key` استفاده می‌شود. این تابع از روش‌های مختلفی (مانند SLSQP و Nelder-Mead) پشتیبانی می‌کند.

- `scipy.optimize.minimize_scalar`: تابع بهینه‌سازی یک‌بعدی SciPy که در توابع `optimize_rotation_angle_chapman` و `optimize_nu_ma2005` استفاده می‌شود. این تابع از روش Brent's bounded پشتیبانی می‌کند.

- `scipy.optimize.brentq`: تابع یافتن ریشه SciPy که در تابع `calculate_optimal_mu_ma2005` استفاده می‌شود. برای حل معادلهٔ $(1-\mu)e^{-\mu} = C$.

- `numpy`: کتابخانهٔ محاسبات برداری که در تابع `optimize_rotation_angle_chapman` برای محاسبهٔ ماتریس‌ها و مقادیر ویژه استفاده می‌شود.

- `math`: ماژول ریاضی استاندارد Python برای توابع پایه مانند `exp`، `log`، `cos`، `sin` و `sqrt`.

- `logging`: ماژول لاگ‌گذاری استاندارد Python برای ثبت هشدارها و اطلاعات.

- `typing`: ماژول Type Hint برای تعریف امضای توابع.

۴.۵.۱۰ جریان داده در سراسر فریم‌ورک

جریان داده کلی به این شکل است:

۱. لایه بالایی (مانند اسکرپت پیکربندی یا شبیه‌ساز) پارامترهای فیزیکی سیستم (مانند فاصله، افت فیبر، راندمان آشکارساز) را مشخص می‌کند.
۲. یک callback تابعی تعریف می‌شود که نرخ کلید (یا هر کمیت هدف دیگر) را برای مجموعه‌ای از پارامترهای داده‌شده محاسبه می‌کند. این callback معمولاً از ماژول‌های اثبات امنیتی (مانند `lim2014.py`، `tight.py` یا `ma2005.py`) استفاده می‌کند.
۳. تابع بهینه‌سازی مناسب (مانند `optimize_lim2014_finite_key`) با پارامترهای فیزیکی و callback فراخوانی می‌شود.
۴. تابع بهینه‌سازی، callback را در نقاط مختلف فضای پارامتر فراخوانی می‌کند و با استفاده از روش‌های عددی (مانند SLSQP یا Nelder-Mead) به سمت نقطه بهینه حرکت می‌کند.
۵. در هر فراخوانی callback، تابع هدف ابتدا محدودیت‌های فیزیکی را بررسی می‌کند و در صورت نقض، جریمه بزرگی برگردانده می‌شود.
۶. پس از همگرایی (یا رسیدن به حداکثر تعداد تکرار)، تابع بهینه‌سازی پارامترهای بهینه را برمی‌گرداند.
۷. لایه بالایی از این پارامترهای بهینه برای پیکربندی شبیه‌سازی نهایی استفاده می‌کند.

این جداسازی مراحل به گونه‌ای طراحی شده که هر مرحله به صورت مستقل قابل آزمایش و بازبینی باشد. به عنوان مثال، می‌توان از callback‌های مصنوعی با رفتار مشخص برای اعتبارسنجی توابع بهینه‌سازی استفاده کرد. این ماژول‌بندی از اصل تک‌مسئولیتی (Single Responsibility) پیروی می‌کند و قابلیت نگهداری و توسعه کد را به طور قابل توجهی افزایش می‌دهد. علاوه بر این، الگوی callback به کاربر اجازه می‌دهد تابع هدف خود را بدون تغییر کد بهینه‌سازی جایگزین کند. این انعطاف‌پذیری برای تحقیقات QKD که در آن‌ها معیارهای مختلف بهینه‌سازی (نرخ کلید، QBER، طول کلید متناهی و غیره) مورد بررسی قرار می‌گیرند، حیاتی است. طراحی مطمئن از خرابی (fail-safe) این ماژول، آن را به انتخابی ایده‌آل برای محیط‌های عملیاتی تبدیل می‌کند که در آن‌ها پایداری و قابلیت اطمینان سیستم اهمیت زیادی دارد.

بخش سوم

خط لوله اجرا و لایه شبکه

فصل ۱۱

اسکرپت اصلی اجرا

۱.۱۱ نمای کلی و هدف ماژول

ماژول `main_optimized.py` هسته‌ی مرکزی فریم‌ورک شبیه‌سازی QKD است و نقش هماهنگ‌کننده‌ی سرتاسری (end-to-end orchestrator) را در معماری کلی ایفا می‌کند. این ماژول در نسخه‌ی فعلی (نسخه‌ی بازنویسی‌شده با نشانگرهای F-01 تا F-29) از یک اسکرپت ساده‌ی پیمایش Monte-Carlo به یک pipeline بالغ با قابلیت‌های زیر ارتقا یافته است: پشتیبانی کامل از حالت DWDM (Dense Wavelength Division Multiplexing) با تزریق نویز Raman، بهینه‌سازی خودکار پارامترهای پالس به ازای هر فاصله با SLSQP، الگوی استاندارد SimParams برای جداسازی داده‌های اعتبارسنجی‌شده از پیکربندی خام، و یک لایه‌ی خطانگاری یکپارچه بر پایه‌ی سلسله‌مراتب استثناهای QKDException.

هدف اصلی این ماژول، پاسخ به پرسش بنیادین زیر است: چگونه می‌توان طیف وسیعی از پارامترهای فیزیکی (فاصله، شدت پالس، بازده آشکارساز، نویز دارک کانت، نویز Raman در DWDM، و...) را به‌صورت سیستماتیک در یک شبیه‌سازی موازی پویش (sweep) نمود و نتایج را به‌صورت یک فایل CSV ساختارمند با ستون‌های ثابت و قابل ممیزی ثبت کرد؟ این پرسش به سه دلیل اهمیت دارد. نخست، در شبیه‌سازی‌های علمی QKD، نیاز به پیمایش صدها یا هزاران ترکیب پارامتر برای رسم منحنی‌های نرخ کلید-فاصله یا مقایسه‌ی پروتکل‌ها، امری روزمره است. دوم، موازی‌سازی با ProcessPoolExecutor نیازمند مدیریت دقیق state هر کارگر، بذریکری مستقل آماری، و الگوی lazy rebuild برای جلوگیری از بازسازی اشیای سنگین فیزیکی در هر تکرار است. سوم، خروجی CSV باید ساختار یکسانی برای همه‌ی ردیف‌ها (چه موفق و چه خطا) داشته باشد تا ابزارهای تحلیل بعدی (R, pandas) بتوانند آن را به‌درستی پردازش کنند.

ساختار کلی این ماژول در نسخه‌ی جدید شامل چندین لایه‌ی منطقی است. لایه‌ی پیکربندی شامل ساختار کلی این ماژول در نسخه‌ی جدید شامل چندین لایه‌ی منطقی است. لایه‌ی پیکربندی شامل DEFAULT_CONFIG (دیکشنری تودرتوی پیش‌فرض) و DEFAULT_SWEEPS (تعریف پیمایش‌های رایج، اکثراً comment شده) است. لایه‌ی adapter توابع `build_*` را در بر می‌گیرد که از پیکربندی خام، اشیای dataclass اعتبارسنجی‌شده می‌سازند. لایه‌ی SimParams شامل `build_channel_sim_params`، `build_source_sim_params`، `build_detector_sim_params` است که داده‌های اعتبارسنجی‌شده‌ی frozen را برای حلقه‌ی شبیه‌سازی فراهم می‌سازند. لایه‌ی worker شامل کلاس `WorkerState` و توابع `init_worker` و `run_single_simulation` است. لایه‌ی orchestrator تابع `run_and_save_csv` را در بر می‌گیرد که کارهای شبیه‌سازی را تولید کرده و آن‌ها را به `dispatch_work_items` (در ماژول جداگانه‌ی `main_optimized_dispatch`) ارسال می‌نماید.

ویژگی‌های برجسته‌ی نسخه‌ی جدید عبارت‌اند از: (۱) الگوی SimParams که خواندن مستقیم از `con-fig dict` در حلقه‌ی شبیه‌سازی را حذف کرده و داده‌های اعتبارسنجی‌شده را از اشیای frozen می‌خواند؛ (۲) پشتیبانی کامل از DWDM با تزریق نویز Raman به دو مسیر مستقل (مسیر Monte-Carlo آشکارساز و مسیر اثبات امنیتی Lim2014)؛ (۳) بهینه‌سازی خودکار پارامترهای پالس با کش (comboper، distance) برای اشتراک پارامترهای بهینه بین ردیف‌های `noise=True` و `noise=False`؛ (۴) الگوی استاندارد خطانگاری با `_as_qkd_exception` و `_log_qkd_exception` که همه‌ی `catch-block`ها را به یک الگوی واحد تبدیل می‌کند؛ (۵) الگوی `from_config_dict` برای آشکارساز که تمام پارامترها را در یک گام اعتبارسنجی می‌کند و نیاز به `setattr` پس از ساخت را حذف می‌نماید؛ (۶) افزودن ستون `analytic_rogers_sbr` به CSV برای مقایسه‌ی مستقیم Monte-Carlo با فرمول بسته‌ی Rogers et al. (2007)؛ (۷) ستون‌های `params_*` برای ممیزی DWDM؛ (۸) ستون‌های `det_override_*` برای ممیزی `override`های آشکارساز.

در سطح عملیاتی، این ماژول به‌عنوان `entry point` اصلی فریم‌ورک عمل می‌کند و از طریق `argparse` یک

CLI غنی با بیش از ۳۰ گزینه ارائه می‌نماید. کاربر می‌تواند با خط فرمان، پروتکل (MDI, B92, BB84Decoy)، پارامترهای فیزیکی (RedundantTransmission)، اثبات امنیتی (Tight, Wang2005, Ma2005, Lim2014)، پارامترهای فیزیکی (فاصله، بازده، نویز)، پارامترهای امنیتی (epsilons)، حالت DWDM (با تعداد کانال، توان لانچ، ضریب Raman)، و بهینه‌سازی خودکار پالس را پیکربندی نماید. خروجی در فایل `qkd_results_optimized_sweep.csv` ذخیره می‌شود و شامل ستون‌های ثابت برای مسیر موفق و مسیر خطا است (الگوی F-07).

۲.۱۱ مبانی تئوری و فیزیکی

۱.۲.۱۱ ساختار پیکربندی سلسله‌مراتبی و اصل Truth of Source Single

پیکربندی فریم‌ورک در یک دیکشنری تودرتوی چهار سطحی سازماندهی شده است: `protocol_params` (نوع پروتکل و آشکارساز)، `protocol_runtime` (کلاس پروتکل، احتمال پایه، خط‌مشی کلیک دوگانه، پرچم‌های انکودر/دیکودر درهم‌تنیده)، `channel` (کاهیدگی فیبر، پاشش)، `detector` (بازده، نرخ دارک کانت، QBER ذاتی، نامساوی، `afterpulse`، `jitter`، `dead time`، دما، ولتاژ بایاس)، `source` (کلاس منبع، توزیع آماری، مدل خطا، شدت پالس‌ها، MZM، نویز الکتریکی، `security_metadata`)، `protocol` (تخصیص `epsilon`)، و `simulation` (بذر، `log-range` پالس، بازه فاصله). این سلسله‌مراتب امکان پیکربندی دقیق هر لایه‌ی فیزیکی را فراهم می‌سازد. با این حال، در نسخه‌ی جدید، اصل Truth of Source Single به‌طور جدی اعمال شده است. به‌جای خواندن مستقیم از `config dict` در حلقه‌ی شبیه‌سازی (که ممکن است ناسازگار با اشیای اعتبارسنجی‌شده باشد)، توابع `build_*` اشیای `dataclass` می‌سازند و سپس توابع `build_*_sim_params` اشیای `frozen SimParams` را از آن‌ها استخراج می‌کنند. حلقه‌ی شبیه‌سازی فقط از `SimParams` می‌خواند. این الگو تضمین می‌کند که اگر کاربر پیکربندی را با `sweep` تغییر دهد، تمام مقادیر اعتبارسنجی شده و سپس به حلقه ارسال شوند.

۲.۲.۱۱ تخصیص بودجه امنیتی ϵ و Security Composable

در رمزنگاری کوانتومی مدرن، امنیت به‌صورت `composable` تعریف می‌شود؛ یعنی مجموع احتمال شکست امنیتی باید کمتر از یک بودجه‌ی کل ϵ_{total} باشد. این بودجه بین چندین مرحله تقسیم می‌شود:

$$\epsilon_{total} = \epsilon_{sec} + \epsilon_{cor} + \epsilon_{pe} + \epsilon_{smooth} + \epsilon_{pa} + \epsilon_{phase_est} \quad (1.11)$$

در این رابطه، ϵ_{sec} امنیت کلی، ϵ_{cor} تصحیح خطا، ϵ_{pe} تخمین پارامتر، ϵ_{smooth} هموارسازی، ϵ_{pa} تقویت حریم خصوصی، و ϵ_{phase_est} تخمین خطای فاز است. تابع `build_epsilon_allocation` این مقادیر را از پیکربندی می‌خواند و اعتبارسنجی می‌نماید که همگی مثبت باشند و مجموعشان کمتر از ۱ باشد. در صورت نقض، `ParameterValidationError` پرتاب می‌شود.

۳.۲.۱۱ فیزیک دارک کانت و فرض Appendix B مقاله Lim ۲۰۱۴

در مقاله‌ی ۲۰۱۴ Lim et al. (PhysRevA.89.022307)، Appendix B به‌صراحت فرض می‌کند که «مشارکت‌های واکوئم هیچ اطلاعاتی درباره‌ی مقادیر بیت انتخابی ندارند و بیت‌ها به‌طور یکنواخت توزیع شده‌اند.» این فرض، توجیه می‌کند که رویدادهای واکوئم در فرمول طول کلید (معادله‌ی ۱) اعتبار آنتروپی کامل دریافت می‌کنند بدون آنکه نیاز به کسر $h(\cdot)$ باشد.

از نظر فیزیکی، آشکارسازی واکوئم در واقع دارک کانت است: آشکارساز به‌صورت خود به‌خودی کلیک می‌کند و مقدار بیت که Bob تخصیص می‌دهد، به‌طور یکنواخت تصادفی در $\{0, 1\}$ است. بنابراین، QBER اندازه‌گیری‌شده روی پالس‌های واکوئم باید حدود ۵۰٪ باشد، نه QBER ذاتی آشکارساز (۱٪). در کد، دو بلاک توضیحی (در خطوط ۱۲۵۳-۱۲۹۶ و ۱۲۹۸-۱۳۲۶) این موضوع را به‌تفصیل شرح می‌دهند. باگ در `qkd.detectors` باعث می‌شود QBER واکوئم ۱٪ باشد که فرض Appendix B را نقض می‌کند. در نسخه‌ی فعلی، اصلاح در سطح مرز `stats-map` به‌صورت یک `workaround` اعمال شده و توابع `correct_vacuum_qber_for_proof` و `correct_signal_decoy_qber_for_proof` (که در نسخه‌ی قبلی وجود داشتند) به‌صورت `comment` یا `placeholder` باقی مانده‌اند. این موضوع در بخش ۱۵.۴.۱۱ به‌تفصیل بررسی می‌شود.

۴.۲.۱۱ مدل فیزیکی QBER سیگنال و دکوی

QBER (Quantum Bit Error Rate) سیگنال و دکوی از ترکیب خطای ذاتی سیستم و نویز دارک کانت به دست می‌آید. در حالت ایده‌آل، QBER متوسط برابر است با:

$$E_{\mu} = \frac{e_d \cdot P_{\text{real}} + 0.5 \cdot P_{\text{dark}}}{P_{\text{real}} + P_{\text{dark}}} \quad (۲.۱۱)$$

در این رابطه، e_d خطای ذاتی (مجموع qber_intrinsic و misalignment)، η_{channel} احتمال کلیک فوتون واقعی (مدل Poisson + بازده)، و $P_{\text{dark}} = 1 - (1 - Y_0 \cdot \Delta t)^2$ احتمال کلیک دارک کانت (دو آشکارساز) است. در کد، بلاک توضیحی در خطوط ۱۳۲۶-۱۳۹۸ این مدل را شرح می‌دهد و نشان می‌دهد که باگ qkd.detectors باعث می‌شود QBER سیگنال در فاصله‌ی صفر به جای ۰.۰۱۵ به ۰.۲۳۵ برسد.

۵.۲.۱۱ تئوری al. et Rogers (۲۰۰۷) برای نرخ بیت sifted

Rogers, Bienfang, Nakassis, Xu & Clark (2007) فرمول بسته‌ای برای نرخ بیت sifted در سیستم BB84 تحت اثر dead time استخراج کرده‌اند. معادله‌ی کلیدی (معادله‌ی ۱۵ مقاله) به صورت زیر است:

$$\text{SBR} = \rho_{\text{TX}} \cdot 8 \cdot p \cdot P_{00}(p, k) \cdot S(p, k) \quad (۳.۱۱)$$

در این رابطه، ρ_{TX} نرخ ارسال (هرتز)، $p = L/8$ احتمال کلیک به ازای هر آشکارساز در هر چرخه (که در آن L احتمال آشکارسازی یک فوتون از Alice در Bob است)، $k = \rho_{\text{TX}} \cdot \tau$ تعداد دوره‌های ارسال در هر dead time (که در آن τ زمان مرده است)، P_{00} احتمال حالت پایدار که هر دو آشکارساز در یک پایه زنده باشند (معادله‌ی ۸)، و $S(p, k)$ احتمال sifting (معادله‌ی ۹) است. تابع `compute_rogers_analytic_sbr` این فرمول را پیاده‌سازی می‌کند و از `link_efficiency` از `qkd.channel` برای محاسبه‌ی $L = \eta_{\text{ch}} \cdot \eta_{\text{det}} \cdot \eta_{\text{coup}}$ استفاده می‌نماید. خروجی این تابع به عنوان ستون `analytic_rogers_sbr` در CSV ثبت می‌شود تا کاربر بتواند مستقیماً Monte-Carlo را با منحنی بسته‌ی مقاله (شکل‌های ۳-۵) مقایسه کند.

۶.۲.۱۱ تئوری Time Dead آشکارساز: Non-Paralyzable

مدل Non-Paralyzable برای dead time آشکارساز SPAD به این معناست که پس از هر کلیک، آشکارساز برای مدت τ غیرفعال می‌شود و کلیک‌های در این بازه به طور کامل نادیده گرفته می‌شوند (نه این که زمان مرده را تمدید کنند). نرخ کلیک اندازه‌گیری شده R_{meas} با نرخ واقعی R_{real} رابطه‌ی زیر را دارد:

$$R_{\text{meas}} = \frac{R_{\text{real}}}{1 + R_{\text{real}} \cdot \tau} \quad (۴.۱۱)$$

در نسخه‌ی جدید، `DeadTimeModel.PARALYZABLE` به عنوان deprecated علامت‌گذاری شده (طبق Rogers et al. و `ROGERS_2007_POLICY` به عنوان sentinel string برای فعال‌سازی خطمشی sequence-collapsing sifting معرفی شده است. این خطمشی در سطح پایه (نه آشکارساز) عمل می‌کند و حداکثر یک بیت sifted به ازای هر دنباله‌ی آشکارسازی که dead time را پوشش می‌دهد، تولید می‌کند.

۷.۲.۱۱ مدل Afterpulse آشکارساز

Afterpulse پدیده‌ای است که در آن یک کلیک واقعی باعث تولید کلیک‌های کاذب بعدی در آشکارساز می‌شود. دو مدل اصلی وجود دارد: Exponential (که در آن احتمال afterpulse با زمان به صورت نمایی کاهش می‌یابد) و Geometric (که در آن احتمال در هر دوره‌ی ارسال ثابت است). در پیکربندی پیش‌فرض، مدل Exponential با احتمال ۰.۰۰۱ و طول عمر ۱۰ نانوثانیه به کار رفته است.

۸.۲.۱۱ نویز Raman در DWDM و مدل فیزیکی

در حالت DWDM، چندین کانال کلاسیک در کنار کانال کوانتومی در یک فیبر منتقل می‌شوند. پراکندگی Raman باعث می‌شود فوتون‌های کانال‌های کلاسیک به طول موج کانال کوانتومی پخش شوند و نویز پس‌زمینه ایجاد نمایند. طبق Lim2014 Section IV، نرخ فوتون‌های Raman به صورت زیر مدل می‌شود:

$$P_{\text{Raman}} = P_{\text{launch}} \cdot (N - 1) \cdot \gamma_{\text{Raman}} \cdot L_{\text{eff}} \quad (۵.۱۱)$$

در این رابطه، $P_{\text{launch}} = 10^{P_{\text{dBm}}/10} \times 10^{-3}$ توان لانچ هر کانال (وات)، N تعداد کانال‌های کلاسیک، γ_{Raman} ضریب پراکندگی Raman، و $\alpha = \alpha_{\text{dB/km}} \cdot 0.23026$ (که در آن $L_{\text{eff}} = (1 - e^{-\alpha L})/\alpha$ طول مؤثر غیرخطی است) به دست می‌آید: $E_{\text{ph}} = hc/\lambda$ به دست می‌آید: نپر بر کیلومتر). نرخ فوتون به ازای هرتر با تقسیم بر انرژی فوتون

$$R_{\text{Raman}} = \frac{P_{\text{Raman}}}{E_{\text{photon}}} \quad (۶.۱۱)$$

تابع `_compute_raman_dark_rate_hz` این محاسبه را پیاده‌سازی می‌کند و خروجی را به صورت نرخ دارک کانت معادل (هرتز) برمی‌گرداند. این مقدار سپس به `detector_run.dark_rate` و `params.detector.dark_rate` تزریق می‌شود.

۹.۲.۱۱ بهینه‌سازی پارامترهای پالس با GLLP

هنگامی که پرچم `--auto-optimize-pulses` فعال است، به ازای هر `(distance, combo)`، پارامترهای μ (شدت سیگنال)، ν (شدت دکوی)، p_s (احتمال سیگنال)، p_d (احتمال دکوی)، و p_v (احتمال واکوئم) با SLSQP بهینه می‌شوند. تابع هدف، نرخ کلید مجانبی GLLP با جریمه‌ی متناهی است:

$$R = q_{\text{sift}} \cdot [Q_1 \cdot (1 - H_2(e_1)) - Q_\mu \cdot f_{\text{EC}} \cdot H_2(E_\mu)] - \text{penalty finite-size} \quad (۷.۱۱)$$

این بهینه‌سازی در ماژول `qkd.proofs.optimization` پیاده‌سازی شده و `main_optimized` فقط یک `thin wrapper` به نام `_optimize_pulses_for_distance` ارائه می‌دهد. پارامترهای بهینه در `_ws.optimized_params` کش می‌شوند تا ردیف‌های `noise=False` و `noise=True` در همان `(distance, combo)` از پارامترهای یکسان استفاده کنند.

۳.۱۱ تحلیل معماری و ساختار داده‌ها

۱.۳.۱۱ DEFAULT_SWEEPS و DEFAULT_CONFIG

DEFAULT_CONFIG یک دیکشنری تودرتوی با هفت بخش است که در خطوط ۱۱۸-۲۰۳ تعریف شده است. هر بخش مجموعه‌ای از پارامترهای پیش‌فرض را در بر می‌گیرد. در نسخه‌ی جدید، چندین تغییر کلیدی اعمال شده: `error_correction_efficiency` از ۱.۱ به ۱.۱۶ افزایش یافته (مطابق مقادیر عملی LDPC)، `dark_rate` به 10^{-6} تنظیم شده، `dark_rate_d1` به 2×10^{-6} (نامتقارن)، `dead_time_ns` به ۱۰.۰ نانوثانیه، `jitter_fwhm_ns` به ۰.۰۵۰ نانوثانیه، و `afterpulse_prob` به ۰.۰۰۱. DEFAULT_SWEEPS در خطوط ۲۰۹-۲۴۵ تعریف شده و اکثراً به صورت `comment` است. این طراحی عمدی است: کاربر می‌تواند با `uncomment` کردن خطوط مورد نظر، پیمایش‌های رایج را فعال کند. یک کامنت مهم در خطوط ۲۳۰-۲۳۶ توضیح می‌دهد که `DeadTimeModel.PARALYZABLE` به عنوان `deprecated` علامت‌گذاری شده و به جای آن، `ROGERS_2007_POLICY` به `sweep` اضافه شده است.

۲.۳.۱۱ الگوی Adapter: توابع *_build

توابع `*_build` به عنوان `adapter` بین پیکرندگی خام (دیکشنری) و اشیای `dataclass` اعتبارسنجی‌شده عمل می‌کنند. این الگو چندین مزیت دارد: نخست، جداسازی `concern`ها (پیکرندگی در برابر اعتبارسنجی در برابر ساخت شیء)؛ دوم، امکان `test` مستقل هر `adapter`؛ سوم، مهاجرت تدریجی از `dict` به `typed object` (طبق `TODO F-25`). توابع کلیدی عبارت‌اند از:

`build_optical_source_config`: از `config["source"]` یک `OpticalSourceConfig` می‌سازد. در نسخه‌ی جدید، `source_rate` از `pulse_period_ns` استنتاج می‌شود (یا برعکس) چون `OpticalSourceConfig` `pulse_period_ns` را به‌عنوان `@property` از `source_rate` محاسبه می‌کند. این استنتاج در خطوط ۵۲۲-۵۳۱ پیاده‌سازی شده و در صورت فقدان هر دو، `ConfigurationError` پرتاب می‌شود.

`build_intensity_config`: یک `adapter` قدیمی است که فرض می‌کند دقیقاً ۳ نوع پالس (`vacuum`, `decoy`, `signal`) با نام‌های ثابت وجود دارد. در نسخه‌ی جدید، تابع `build_intensity_config_from_source` به‌عنوان مسیر ترجیحی معرفی شده که از `source.intensity_config` استفاده می‌کند و از جستجوی مبتنی بر نقش (یافتن "signal" با نام، نه ایندکس) پشتیبانی می‌نماید.

`build_detector`: در نسخه‌ی جدید کاملاً بازنویسی شده است. رویکرد قدیمی (حذف شده) شامل دو مرحله بود: (۱) `build_detection_config` که فقط ۳ فیلد داشت و (۲) `from_config + ۱۰ setattr` برای پارامترهای از دست رفته. رویکرد جدید از `_build_detector_config_dict` برای ساخت یک دیکشنری تخت با تمام پارامترها استفاده می‌کند و سپس `SinglePhotonDetector.from_config_dict` را فراخوانی می‌نماید که در یک گام اعتبارسنجی کامل انجام می‌دهد.

۳.۳.۱۱ تابع `parse_enum` و یکپارچه‌سازی `Enum`

تابع `parse_enum` در خطوط ۳۳۲-۳۵۵ پیاده‌سازی شده و یک `adapter` برای تبدیل رشته‌ها به اعضای `Enum` است. این تابع ابتدا بررسی می‌کند که آیا مقدار از قبل عضو `Enum` است (در این صورت بدون تغییر برمی‌گردد)، سپس رشته را با `strip` و `lower/upper` نرمال‌سازی می‌کند و با نام و مقدار اعضای `Enum` مقایسه می‌نماید. در صورت عدم تطابق، `ParameterValidationError` با فهرست مقادیر معتبر پرتاب می‌شود. یک مورد خاص در `normalize_detector_config` (خط ۳۵۷) وجود دارد: `ROGERS_2007_POLICY` به‌عنوان `sentinel string` (نه عضو `Enum`) تعریف شده و در صورت مشاهده، بدون تغییر عبور می‌کند (`pass` through). این طراحی به این دلیل است که `DoubleClickPolicy` در `qkd.datatypes` تعریف شده و ما عمداً آن را تغییر نمی‌دهیم.

۴.۳.۱۱ کلاس `WorkerState` و الگوی `Encapsulation State`

کلاس `WorkerState` در خطوط ۱۳۶۶-۱۴۰۵ تعریف شده و `mutable state` هر کارگر را در یک شیء واحد کپسوله می‌کند. این الگو (طبق F-03) جایگزین متغیرهای سراسری مازول شده است. هر فرآیند کارگر نسخه‌ی خود را از طریق `init_worker` دریافت می‌کند و این شیء بین فرآیندها به اشتراک گذاشته نمی‌شود. `WorkerState` از `__slots__` استفاده می‌کند که در پایتون، حافظه‌ی کم‌تری مصرف می‌کند و دسترسی به ویژگی‌ها را سریع‌تر می‌سازد. اسلات‌های کلیدی عبارت‌اند از: `last_combo_idx`, `combo_map`, `base_config` (برای `lazy rebuild`), `protocol`, `detector`, `source` (اشیای فیزیکی ساخته‌شده)، `source_meta_cache` (کش `metadata` منبع برای جلوگیری از سریالی‌سازی مکرر، طبق F-14)، `detector_overrides` (ممیزی `override`های آشکارساز)، `auto_optimize_pulses`, `last_optimized_dist`, `optimized_params` (کش بهینه‌ساز - `comboper`-(`distance`, `comboper`)), `dwdm`, `wdm_channel_count`, `wdm_channel_power_dbm`, `wdm_raman_coefficient` (پارامترهای DWDM)، و `params` (کش `QKDParams` برای مسیر خطا).

۵.۳.۱۱ ساختار داده‌ی `TallyCounts`

`TallyCounts` یک `dataclass` است که آمار شمارش پالس‌ها را در بر می‌گیرد: `sent` (کل ارسال‌شده)، `sifted` (کل)، `sifted_x`, `sifted_z`, `errors_sifted_x`, `errors_sifted_z`, `double_clicks_discarded` (خطاهای `sifted`), `sent_x`, `sent_z` (مستقیم) و `X` (قطری) را فراهم می‌سازد که در پروتکل BB84 برای تخمین خطای فاز ضروری است. تابع `tally_counts_from_sifting_results` (خط ۱۲۰۴) این ساختار را از نتایج `sifting` می‌سازد. این تابع با استفاده از `_basis_is_z_mask` ماسک‌های پایه‌ی `Z` را استخراج می‌کند و سپس برای هر نوع پالس (`pulse_type_index`), شمارش‌های جداگانه برای پایه‌های `Z` و `X` را محاسبه می‌نماید. در صورت عدم دسترسی به ماسک پایه، از تقسیم برابر ۵۰/۵۰ به‌عنوان `fallback` استفاده می‌کند.

۶.۳.۱۱ SimulationResults به عنوان خروجی جامع

SimulationResults یک dataclass جامع است که خروجی کامل یک نقطه‌ی شبیه‌سازی را در بر می‌گیرد: params (ProtocolParameters), metadata, security_certificate, decoy_estimates, status, simulation_time_seconds, raw_sifted_key_length, secure_key_length, beta_y1_deviation, tagged_fraction_bound, weak_gllp_rate, tally_stats, beta_e1_deviation, و success_probability. تابع build_simulation_results (خط ۱۳۳۹) این شیء را می‌سازد.

۷.۳.۱۱ DETECTOR_METADATA_KEYS, PARAMS_METADATA_KEYS و DETECTOR_OVERRIDE_KEYS

در نسخه‌ی جدید، سه گروه ستون اضافی به CSV اضافه شده‌اند. DETECTOR_METADATA_KEYS (خط ۱۴۱۶) از dataclass DetectionDiagnostics.metadata_keys() استخراج می‌شود تا ستون‌های CSV همیشه با dataclass هماهنگ باشد. DETECTOR_OVERRIDE_KEYS (خط ۱۴۲۳) شامل ۹ ستون برای ممیزی override‌های آشکارساز است (مانند det_override_det_eff_d0). PARAMS_METADATA_KEYS (خط ۱۴۳۹) شامل ۷ ستون برای ممیزی DWDM است (مانند params_wdm_channel_count, params_raman_dark_rate_hz).

۸.۳.۱۱ الگوی SimParams، ChannelSimParams، SourceSimParams- DetectorSimParams

در نسخه‌ی جدید، الگوی SimParams معرفی شده است. سه تابع build_channel_sim_params، build_source_sim_params و build_detector_sim_params اشیاى frozen می‌سازند که تمام مقادیر فیزیکی اعتبارسنجی‌شده را در بر دارند. حلقه‌ی شبیه‌سازی به جای خواندن از config dict، از این اشیاى frozen می‌خواند. این الگو تضمین می‌کند که اگر کاربر پیکربندی را با sweep تغییر دهد، تمام مقادیر اعتبارسنجی شده و سپس به حلقه ارسال شوند.

۴.۱۱ پیاده‌سازی ریاضی و الگوریتمی

۱.۴.۱۱ تابع set_nested_value: اعمال overrides روی پیکربندی تودرتو

تابع set_nested_value (خط ۲۴۸) یک مقدار را در دیکشنری تودرتو با مسیر نقطه‌گذاری (dot-separated) تنظیم می‌کند. این تابع از پارامتر strict پشتیبانی می‌کند که در حالت True، در صورت عدم وجود کلید میانی، ConfigurationError پرتاب می‌کند (طبق F-10).

```

1 def set_nested_value(d: Dict, key_path: str, value: Any, *, strict: bool = False):
2     if not key_path or not isinstance(key_path, str):
3         raise ConfigurationError(
4             "Sweep override key_path must be a non-empty dotted string.",
5             context={"key_path": key_path},
6         )
7     keys = key_path.split('.')
8     current = d
9     for key in keys[:-1]:
10        if not isinstance(current, dict):
11            raise ConfigurationError(
12                "Cannot apply sweep override through a non-dictionary config node.",
13                context={"key_path": key_path, "blocked_at": key},
14            )
15        if strict and key not in current:
16            raise ConfigurationError(
17                "Sweep override key_path references a missing intermediate key.",
18                context={"key_path": key_path, "missing_key": key},
19            )
20        current = current.setdefault(key, {})
21    current[keys[-1]] = value
22

```

در خطوط ابتدایی، اعتبارسنجی key_path انجام می‌شود. سپس با split('.') مسیر به فهرست کلیدها تجزیه می‌شود. حلقه‌ی for تا کلید یکی‌مانده به آخر می‌چرخد و در هر مرحله، بررسی می‌کند که current از نوع

dict باشد. در حالت strict، عدم وجود کلید میانی باعث خطا می‌شود. در پایان، کلید آخر با مقدار مورد نظر تنظیم می‌گردد.

۲.۴.۱۱ تابع build_epsilon_allocation: اعتبارسنجی بودجه امنیتی

تابع build_epsilon_allocation (خط ۴۰۳) تخصیص epsilon را از پیکربندی می‌خواند و اعتبارسنجی می‌نماید.

```

۱ def build_epsilon_allocation(config: Dict[str, Any]) -> EpsilonAllocation:
۲     eps = config["protocol"]["epsilons"]
۳     allocation = EpsilonAllocation(
۴         eps_sec=eps["eps_sec"], eps_cor=eps["eps_cor"], eps_pe=eps["eps_pe"],
۵         eps_smooth=eps["eps_smooth"], eps_pa=eps["eps_pa"],
۶         eps_phase_est=eps["eps_phase_est"],
۷     )
۸     total_eps = (
۹         allocation.eps_sec + allocation.eps_cor + allocation.eps_pe +
۱۰        allocation.eps_smooth + allocation.eps_pa + allocation.eps_phase_est
۱۱    )
۱۲    if total_eps >= :0.1
۱۳        raise ParameterValidationError(
۱۴            f"Total epsilon allocation ({total_eps2:e}) must be less than 1.",
۱۵            param_name="protocol.epsilons",
۱۶            param_value=total_eps,
۱۷            context={"total_epsilon": total_eps},
۱۸        )
۱۹    for field_name in ("eps_sec", "eps_cor", "eps_pe", "eps_smooth", "eps_pa", "eps_phase_est"):
۲۰        val = getattr(allocation, field_name)
۲۱        if val <= 0:
۲۲            raise ParameterValidationError(
۲۳                f"Epsilon value {field_name}={val} must be positive.",
۲۴                param_name=f"protocol.epsilons.{field_name}",
۲۵                param_value=val,
۲۶            )
۲۷    return allocation
۲۸

```

ابتدا EpsilonAllocation از پیکربندی ساخته می‌شود. سپس مجموع epsilon محاسبه می‌شود و در صورت بزرگ‌تر یا مساوی ۱، خطا پرتاب می‌گردد. در پایان، هر مقدار بررسی می‌شود که مثبت باشد. این اعتبارسنجی (طبق F-17) تضمین می‌کند که بودجه‌ی امنیتی معتبر باشد.

۳.۴.۱۱ تابع build_optical_source_config: استنتاج source_rate

تابع build_optical_source_config (خط ۵۱۳) یک OpticalSourceConfig از پیکربندی می‌سازد. نکته‌ی کلیدی، استنتاج source_rate از pulse_period_ns است:

```

۱ if "source_rate" in source_cfg and source_cfg["source_rate"] is not None:
۲     source_rate = float(source_cfg["source_rate"])
۳ elif "pulse_period_ns" in source_cfg and source_cfg["pulse_period_ns"] is not None:
۴     pulse_period_ns = float(source_cfg["pulse_period_ns"])
۵     source_rate = 1e9 / pulse_period_ns if pulse_period_ns > 0 else 1e8
۶ else:
۷     raise ConfigurationError(
۸         "source config must contain either 'source_rate' or 'pulse_period_ns'.",
۹         context={"section": "source", "available_keys": sorted(source_cfg.keys())},
۱۰     )
۱۱

```

در این بلاک، اولویت با source_rate مستقیم است. در صورت عدم وجود pulse_period_ns به source_rate تبدیل می‌شود. در صورت فقدان هر دو، خطا پرتاب می‌شود. این الگو به این دلیل ضروری است که OpticalSourceConfig pulse_period_ns را به‌عنوان @property از source_rate محاسبه می‌کند و در نتیجه، فقط یکی از این دو باید در پیکربندی ارائه شود.

۴.۴.۱۱ تابع `build_detector`: الگوی `from_config_dict`

تابع `build_detector` (خط ۹۰۷) در نسخه‌ی جدید کاملاً بازنویسی شده است. رویکرد قدیمی (حذف شده) شامل دو مرحله بود: `build_detection_config` (با ۳ فیلد) + `from_config` + `setattr` ۱۰. رویکرد جدید از `_build_detector_config_dict` برای ساخت یک دیکشنری تخت با تمام پارامترها استفاده می‌کند:

```

۱ def build_detector(config: Dict[str, Any]) -> Tuple[SinglePhotonDetector, Dict[str, Any]]:
۲     det_config_dict = _build_detector_config_dict(config)
۳     detector = SinglePhotonDetector.from_config_dict(det_config_dict)
۴     actual_config = detector.to_config_dict(include_experimental=True)
۵     input_config = det_config_dict
۶     _overrides_applied: Dict[str, Any] = {}
۷     override_keys = (
۸         "det_eff_d0", "det_eff_d1", "dark_rate", "dark_rate_d1",
۹         "bias_voltage", "breakdown_voltage", "temperature_k",
۱۰        "ref_temperature_k", "ref_bias_voltage",
۱۱    )
۱۲    for key in override_keys:
۱۳        input_val = input_config.get(key)
۱۴        actual_val = actual_config.get(key)
۱۵        if actual_val is not None:
۱۶            _overrides_applied[key] = actual_val
۱۷        if input_val != actual_val:
۱۸            logger.debug(
۱۹                "Detector parameter %s: input=%s, actual=%s (default/fallback applied).",
۲۰                key, input_val, actual_val,
۲۱            )
۲۲    return detector, _overrides_applied
۲۳

```

ابتدا `_build_detector_config_dict` دیکشنری تخت را با نرمال‌سازی Enum و تبدیل نوع‌های عددی می‌سازد. سپس `from_config_dict` آشکارساز را در یک گام اعتبارسنجی می‌کند. در پایان، با مقایسه‌ی `to_config_dict()` با ورودی، `overrides` واقعی استخراج می‌شود. این الگو تضمین می‌کند که CSV همیشه مقادیر واقعی آشکارساز را نشان دهد، حتی اگر fallback یا نرمال‌سازی داخلی اعمال شده باشد.

۵.۴.۱۱ تابع `tally_counts_from_sifting_results`

تابع `tally_counts_from_sifting_results` (خط ۱۲۰۴) آمار شمارش را از نتایج `sifting` می‌سازد. این تابع از `_basis_is_z_mask` برای استخراج ماسک پایهی `Z` استفاده می‌کند:

```

۱ def _basis_is_z_mask(values) -> np.ndarray:
۲     arr = np.asarray(values)
۳     if arr.dtype == np.bool_:
۴         return arr.astype(bool)
۵     if np.issubdtype(arr.dtype, np.integer):
۶         return arr == 0
۷     out = np.zeros(arr.shape, dtype=bool)
۸     flat = arr.ravel()
۹     for i, v in enumerate(flat):
۱۰        if isinstance(v, str):
۱۱            s = v.strip().upper()
۱۲        else:
۱۳            name = getattr(v, "name", None)
۱۴            s = str(name).strip().upper() if name is not None else str(v).strip().upper()
۱۵            out.ravel()[i] = s in {"Z", "Z_BASIS", "RECTILINEAR", "STANDARD", "0"}
۱۶        return out
۱۷

```

این تابع با آرایه‌های بولی، صحیح و رشته‌ای کار می‌کند. در حالت رشته‌ای، نام پایه را با مجموعه‌ای از نام‌های متداول (`0`, `STANDARD`, `RECTILINEAR`, `Z_BASIS`, `Z`) مقایسه می‌نماید. این طراحی انعطاف‌پذیر، امکان پذیرش فرمت‌های مختلف ورودی را فراهم می‌سازد.

۶.۴.۱۱ تابع `_compute_raman_dark_rate_hz`: نویز Raman

تابع `_compute_raman_dark_rate_hz` (خط ۱۵۴۴) نرخ دارک کانت معادل Raman را محاسبه می‌کند. این تابع در دو مسیر استفاده می‌شود: مسیر Monte-Carlo (توزیع به `detector_run.dark_rate`) و مسیر اثبات (توزیع به `params.detector.dark_rate`):

```

1 def _compute_raman_dark_rate_hz(params) -> float:
2     if params is None:
3         return 0.0
4     wdm_count = getattr(params, "wdm_channel_count", 1)
5     if wdm_count is None or wdm_count <= 1:
6         return 0.0
7     wdm_power_dbm = float(getattr(params, "wdm_channel_power_dbm", (0.0)
8     p_launch = 0.10) ** (wdm_power_dbm / (0.10) * 1e-3 # Watts
9     ch = getattr(params, "channel", None)
10    if ch is None:
11        return 0.0
12    fiber_loss_db_km = float(getattr(params, "fiber_loss_db_km",
13    getattr(ch, "fiber_loss_db_km", (2.0))
14    distance_km = float(getattr(ch, "distance_km", (0.0)
15    alpha = fiber_loss_db_km * 23026.0 # Np/km
16    if alpha > 1e-12:
17        l_eff = 0.1) - math.exp(-alpha * distance_km)) / alpha
18    else:
19        l_eff = distance_km
20    wdm_raman_coeff = float(getattr(params, "wdm_raman_coefficient", (0.0)
21    p_raman = p_launch * (wdm_count - 1) * wdm_raman_coeff * l_eff
22    lam_nm = float(getattr(params, "carrier_wavelength_nm", (0.1550)
23    photon_energy = (_const.h * _const.c) / (lam_nm * 1e-9)
24    rate_hz = p_raman / photon_energy
25    det = getattr(params, "detector", None)
26    det_eff_d0 = float(getattr(det, "det_eff_d0", (0.1) if det is not None else 0.1
27    return rate_hz * det_eff_d0
28

```

در ابتدا، در صورت $wdm_count \leq 1$ ، مقدار صفر بازگردانده می‌شود (حالت تک‌کانال). سپس P_{launch} از dBm به وات تبدیل می‌شود. α از dB/km به Np/km تبدیل می‌شود (با ضریب $10/\ln(10) = 0.23026$). L_{eff} محاسبه می‌شود. سپس P_{Raman} و R_{Raman} محاسبه می‌گردند. در پایان، ضرب در `det_eff_d0` باعث می‌شود که نرخ نهایی، نرخ کلیک پس از بازده آشکارساز باشد (مطابق با قرارداد `dark_rate` در `qkd.detectors`). نکته‌ی مهم (طبق کامنت Option J) این است که این تابع `params.fiber_loss_db_km` را به جای `channel.fiber_loss_km` می‌خواند. در حالت DWDM، کانال از `FiberChannel.from_total_loss` بازسازی می‌شود که α را از کل تلفات (فیبر + فیلتر AWG) استنتاج می‌کند. استفاده از α آلوده باعث می‌شود Raman ۹-۱۹٪ کم‌برآورد شود.

۷.۴.۱۱ تابع `_compute_rogers_analytic_sbr`

تابع `_compute_rogers_analytic_sbr` (خط ۱۷۴۲) نرخ بیت sifted تحلیلی را طبق معادله‌ی ۱۵ مقاله‌ی Rogers et al. (2007) محاسبه می‌کند:

```

1 def _compute_rogers_analytic_sbr(
2     *, pulse_period_ns, dead_time_ns, channel_transmittance,
3     det_eff_d0, coupling_efficiency0.1=, eps0.0=,
4 ) -> Optional[float]:
5     if dead_time_ns <= 0.0 or pulse_period_ns <= :0.0
6         return None
7     L = link_efficiency(channel_transmittance, det_eff_d0, coupling_efficiency)
8     if L <= :0.0
9         return None
10    L = min(L, (0.1
11    rho_tx = 1e9 / float(pulse_period_ns)
12    tau_s = float(dead_time_ns) * 1e-9
13    try:
14        return float(rogers_sifted_bit_rate(
15        rho_tx=rho_tx, L=L, tau=tau_s, eps=eps,
16    ))
17    except (ValueError, OverflowError, ArithmeticError) as exc:
18        logger.debug("Rogers analytic SBR computation failed: %s.", exc)

```

```

۱۹         return None
۲۰

```

در ابتدا، اعتبارسنجی `dead_time_ns` و `pulse_period_ns` انجام می‌شود. سپس L با $L = \eta_{ch} \cdot \eta_{det} \cdot \eta_{coup}$ محاسبه می‌شود (این تابع از `qkd.channel` وارد شده و `rogers_sifted_bit_rate` (که از `qkd.detectors` وارد شده) فراخوانی را محاسبه می‌نماید). در پایان، `None` بازگردانده می‌شود تا در CSV به صورت سلول خالی ثبت شود.

۸.۴.۱۱ تابع `_balance_decoy_probabilities`

تابع `_balance_decoy_probabilities` (خط ۱۷۹۰) احتمال‌های پالس را به صورت خودکار متعادل می‌کند. هنگامی که یک `sweep` فقط یکی از احتمال‌ها (`signal` یا `decoy`) را `override` می‌کند، این تابع دیگری را محاسبه می‌نماید:

```

۱     def _balance_decoy_probabilities(overrides, config):
۲         sig_p = overrides.get("source.pulses.signal.prob")
۳         dec_p = overrides.get("source.pulses.decoy.prob")
۴         vac_p = config["source"]["pulses"]["vacuum"]["prob"]
۵         if sig_p is not None and dec_p is None:
۶             balanced_dec = round(0.1) - sig_p - vac_p, 9)
۷         if balanced_dec < 0:
۸             raise ConfigurationError(
۹                 f"Probability balancing produced negative decoy probability ({balanced_dec}).",
۱0                context={"signal_prob": sig_p, "vacuum_prob": vac_p, "decoy_prob": balanced_dec},
۱۱            )
۱۲         overrides["source.pulses.decoy.prob"] = balanced_dec
۱۳         elif dec_p is not None and sig_p is None:
۱۴             balanced_sig = round(0.1) - dec_p - vac_p, 9)
۱۵         if balanced_sig < 0:
۱۶             raise ConfigurationError(
۱۷                 f"Probability balancing produced negative signal probability ({balanced_sig}).",
۱۸                context={"decoy_prob": dec_p, "vacuum_prob": vac_p, "signal_prob": balanced_sig},
۱۹            )
۲۰         overrides["source.pulses.signal.prob"] = balanced_sig
۲۱

```

این تابع (طبق F-11) اعتبارسنجی می‌کند که احتمال‌های متعادل‌شده در بازه‌ی $[0, 1]$ باشند. در صورت منفی شدن، خطا پرتاب می‌شود. این الگو امکان `sweep` فقط یک احتمال را فراهم می‌سازد و بقیه به صورت خودکار محاسبه می‌شوند.

۹.۴.۱۱ تابع `_optimize_pulses_for_distance`: بهینه‌سازی پالس

تابع `_optimize_pulses_for_distance` (خط ۱۸۴۵) یک `thin wrapper` به `optimize_pulses_for_distance` در `qkd.proofs.optimization` است:

```

۱     def _optimize_pulses_for_distance(config, dist_km, total_pulses, proof_name):
۲         return optimize_pulses_for_distance(config, dist_km, total_pulses, proof_name)
۳

```

این تابع پارامترهای بهینه $(\mu, \nu, p_s, p_d, p_v)$ را برمی‌گرداند. در `run_single_simulation`، این پارامترها در `optimized_params` کش می‌شوند و کلید کش `(combo_idx, dist)` است. این طراحی تضمین می‌کند که ردیف‌های `noise=True` و `noise=False` در همان `(distance, combo)` از پارامترهای یکسان استفاده کنند که برای مقایسه‌ی منصفانه ضروری است.

۱۰.۴.۱۱ تابع `init_worker` و الگوی `initializer`

تابع `init_worker` (خط ۱۴۴۹) در نسخه‌ی جدید امضای گسترده‌ای دارد:

```

۱     def init_worker(base_config, combo_map, auto_optimize_pulses=False, dwdm=False,
۲         wdm_channel_count=4, wdm_channel_power_dbm=None,
۳         wdm_raman_coefficient=None):
۴         logging.basicConfig(
۵             level=logging.INFO,

```

```

۶     format="%(%levelname)s [%(name)s] %(message)s",
۷     force=True,
۸ )
۹     _ws.base_config = base_config
۱۰    _ws.combo_map = combo_map
۱۱    _ws.last_combo_idx = -1
۱۲    _ws.source_meta_cache = {}
۱۳    _ws.detector_overrides = {}
۱۴    _ws.auto_optimize_pulses = bool(auto_optimize_pulses)
۱۵    _ws.last_optimized_dist = None
۱۶    _ws.optimized_params = None
۱۷    _ws.params = None
۱۸    _ws.dwdm = bool(dwdm)
۱۹    _ws.wdm_channel_count = int(wdm_channel_count) if wdm_channel_count is not None else None
۲۰    _ws.wdm_channel_power_dbm = float(wdm_channel_power_dbm) if wdm_channel_power_dbm is not None
    else None
۲۱    _ws.wdm_raman_coefficient = float(wdm_raman_coefficient) if wdm_raman_coefficient is not None
    else None
۲۲

```

در ابتدا، logging.basicConfig با force=True فراخوانی می‌شود تا levelERROR- در فرآیندهای کارگر قابل مشاهده باشد (بدون این، کارگران با levelWARNING- پیش‌فرض شروع می‌شوند و ERROR-ها خاموش می‌شوند). سپس تمام فیلدهای _ws مقداردهی می‌شوند. نکته‌ی مهم این است که wdm_channel_count، wdm_channel_power_dbm و wdm_raman_coefficient با تبدیل نوع (float/int) و بررسی None ذخیره می‌شوند (طبق FIX-WS-WDM-V2).

۱۱.۴.۱۱ تابع run_single_simulation: هسته اصلی

تابع run_single_simulation (خط ۱۸۵۷) هسته‌ی اصلی شبیه‌سازی است. این تابع یک args_tuple شامل (total_pulses, dist, apply_noise, worker_seed, combo_idx) می‌گیرد و یک دیکشنری row برمی‌گرداند. جریان اصلی به چند مرحله تقسیم می‌شود:

مرحله ۱: rebuild. Lazy در صورت تغییر combo_idx، پیکربندی با deepcopy کپی شده، overrides اعمال می‌شود، و اشیای source، detector، protocol بازسازی می‌شوند. سپس metadata در _ws.source_meta_cache کش می‌شود.

مرحله ۲: pulses. Auto-optimize در صورت فعال بودن _ws.auto_optimize_pulses و تغییر (dist, combo)، optimize_pulses_for_distance فراخوانی می‌شود، پارامترهای بهینه در config نوشته می‌شوند، و source و protocol بازسازی می‌گردند.

مرحله ۳: بذرگیری. با SeedSequence.spawn (طبق F-05)، ۵ جریان RNG مستقل از worker_seed استخراج می‌شوند:

```

۱     seed_seq = np.random.SeedSequence(worker_seed)
۲     child_seqs = seed_seq.spawn(5)
۳     rng_prepare = np.random.default_rng(child_seqs[0])
۴     rng_photons = np.random.default_rng(child_seqs[1])
۵     rng_detect = np.random.default_rng(child_seqs[2])
۶     rng_sift = np.random.default_rng(child_seqs[3])
۷     rng_pe = np.random.default_rng(child_seqs[4])
۸

```

این الگو (به جای seed+1، seed+2 و...) تضمین می‌کند که جریان‌های RNG از نظر آماری مستقل باشند.

مرحله ۴: بارگذاری QKDParams. در نسخه‌ی جدید (طبق FIX-PARAMS-ORDER)، load_lim2014_*_params قبل از بلاک apply_noise فراخوانی می‌شود تا تزریق Raman از params صحیح (مربوط به همین فاصله) استفاده کند. در حالت DWDM، load_lim2014_dwdm_params با پارامترهای WDM فراخوانی می‌شود.

مرحله ۵: تزریق Raman به params. در حالت DWDM و apply_noise=True، نرخ Raman محاسبه شده و به params.detector.dark_rate تزریق می‌شود (با dataclasses.replace چون frozen params است).

مرحله ۶: ساخت کانال. build_channel_from_distance کانال را می‌سازد. در حالت DWDM، تلفات فیلتر AWG به کانال اضافه می‌شود.

مرحله ۷: ساخت SimParams. build_channel_sim_params، build_source_sim_params

و (بعداً) `build_detector_sim_params` اشیای `frozen` می‌سازند. اعتبارسنجی `pulse_period_ns` انجام می‌شود.

مرحله ۸: آماده‌سازی حالت‌ها و فوتون‌ها. `protocol.prepare_states`، `generate_photons_for_prepared_states` و `infer_ideal_outcomes_d0` فراخوانی می‌شوند.

مرحله ۹: شبیه‌سازی آشکارساز. در حالت `apply_noise=True`، `detector.clone()` ساخته می‌شود و نویز Raman به `detector_run.dark_rate` تزریق می‌گردد. در حالت `noise=False`، `detector.noiseless_copy()` استفاده می‌شود. سپس `simulate_detection` فراخوانی می‌شود.

مرحله ۱۰: Sifting و tally. `detector_result_to_protocol_detection_results`، `protocol.sift_results` و `sample_for_parameter_estimation` فراخوانی می‌شوند.

مرحله ۱۱: محاسبه کلید امن. این مرحله در بخش بعد به تفصیل بررسی می‌شود.

مرحله ۱۲: ساخت ردیف CSV. با محاسبه‌ی QBER اصلاح‌شده و فاصله‌ی اطمینان، ردیف نهایی ساخته می‌شود.

۱۲.۴.۱۱ مدیریت API متفاوت کلاس‌های اثبات

در نسخه‌ی جدید، مدیریت اثبات‌های مختلف در خطوط ۲۲۹۳–۲۳۸۰ پیاده‌سازی شده است:

```

۱ proof_name = config["source"].get("intended_proof", "LIM_2014").upper()
۲ if proof_name != "LIM_2014":
۳     import dataclasses
۴     updates = {}
۵     if hasattr(params, 'auto_optimize_lim2014'):
۶         updates['auto_optimize_lim2014'] = False
۷     if not hasattr(params, 'eps_pa'):
۸         updates['eps_pa'] = 1e-10
۹     if not hasattr(params, 'eps_sif'):
۱۰        updates['eps_sif'] = 1e-10
۱۱     if updates:
۱۲         params = dataclasses.replace(params, **updates)
۱۳
۱۴     if proof_name == "MA_2005":
۱۵         from qkd.proofs.ma2005 import Ma2005VacuumWeakProof
۱۶         proof = Ma2005VacuumWeakProof(params)
۱۷     elif proof_name == "WANG_2005":
۱۸         from qkd.proofs.wang2005 import Wang2005Proof
۱۹         proof = Wang2005Proof(params)
۲۰     elif proof_name == "TIGHT":
۲۱         from qkd.proofs.tight import BB84TightProof
۲۲         proof = BB84TightProof(params)
۲۳     else:
۲۴         proof = Lim2014Proof(params)
۲۵

```

ابتدا نام اثبات از پیکربندی خوانده می‌شود. در صورت عدم استفاده از `Lim2014`، پارامترهای `auto_optimize_lim2014` به `False` و `eps_sif/eps_pa` (در صورت فقدان) به مقادیر پیش‌فرض تنظیم می‌شوند. سپس کلاس اثبات مناسب `import` و نمونه‌سازی می‌شود. `import` به صورت `lazy` (داخل `if`) انجام می‌شود تا ماژول‌های غیرضروری در `startup` بارگذاری نشوند. سپس فراخوانی اثبات به دو مسیر تقسیم می‌شود:

```

۱ if proof_name == "WANG_2005":
۲     mapped_stats = {
۳         "signal": stats.get("0", TallyCounts()),
۴         "decoy": stats.get("1", TallyCounts()),
۵         "vacuum": stats.get("2", TallyCounts())
۶     }
۷     decoy_estimates = proof.estimate_yields_and_errors(stats_map=mapped_stats)
۸     key_result = proof.calculate_key_length(decoy_estimates, stats_map=mapped_stats)
۹     elif proof_name == "MA_2005":
۱۰        decoy_estimates = proof.estimate_yields_and_errors(stats_map=stats)
۱۱        key_result = proof.calculate_key_length(decoy_estimates, stats_map=stats)
۱۲     else:
۱۳        key_result = proof.calculate_key_length(stats_map=stats)
۱۴

```


Wang2005 به نقشه‌ی stats با کلیدهای رشته‌ای ("vacuum"، "decoy"، "signal") نیاز دارد، در حالی که stats از pulse_type_index صحیح ("0"، "1"، "2") استفاده می‌کند. بنابراین، یک نگاشت انجام می‌شود. Lim2014/Tight و Ma2005 مستقیماً از stats استفاده می‌کنند. در پایان، secure_key_length از key_result استخراج می‌شود.

۱۳.۴.۱۱ مدیریت خطا و ساخت ردیف CSV خطا

در صورت بروز خطا در محاسبه‌ی کلید امن یا در جریان اصلی، الگوی خطای یکپارچه (طبق F-23) اعمال می‌شود:

```

۱     except (LPFailureError, ParameterValidationError, ConfigurationError) as exc:
۲         qkd_exc = exc
۳         _log_qkd_exception(qkd_exc, level=logging.WARNING)
۴         status = f"{qkd_exc.code.value}: {qkd_exc.message[:80]}"
۵         qkd_exc = QKDSimulationError(
۶             f"{proof_name} secure-key calculation failed for this simulation point.",
۷             context={"distance_km": dist, "total_pulses": total_pulses, "combo_idx": combo_idx},
۸             cause=exc,
۹         )
۱۰        _log_qkd_exception(qkd_exc, level=logging.WARNING)
۱۱        status = f"{qkd_exc.code.value}: {qkd_exc.message[:80]}"
۱۲

```

در ابتدا، استثنای LPFailureError، ParameterValidationError، ConfigurationError گرفته می‌شوند و با _log_qkd_exception ثبت می‌گردند. سپس به QKDSimulationError تبدیل می‌شوند تا در سطح بالاتر مدیریت شوند. status برای ثبت در CSV ساخته می‌شود. در بلاک except بیرونی، خطاهای دیگر (TypeError، ValueError و...) با _as_qkd_exception به QKDException مناسب تبدیل می‌شوند:

```

۱     except (QKDException, ValueError, TypeError, KeyError, RuntimeError, ArithmeticError) as exc:
۲         orig_exc_type = type(exc).__name__
۳         orig_exc_msg = str(exc)
۴         qkd_exc = _as_qkd_exception(
۵             exc,
۶             f"Simulation point failed ({orig_exc_type}: {orig_exc_msg[:200]}).",
۷             context={"distance_km": dist, "total_pulses": total_pulses,
۸                     "combo_idx": combo_idx, "original_exception_type": orig_exc_type},
۹         )
۱۰        _log_qkd_exception(qkd_exc, level=logging.ERROR)
۱۱

```

تابع _as_qkd_exception (خط ۳۸۶) یک helper استاندارد است که استثنای خام را به QKDException مناسب تبدیل می‌کند: KeyError/ValueError/TypeError به ConfigurationError، و بقیه به QKDSimulationError. این الگو تضمین می‌کند که همه‌ی catch-blockها از یک الگوی واحد پیروی کنند.

سپس ردیف خطای CSV ساخته می‌شود که همان ستون‌های ردیف موفق را دارد (با مقادیر صفر یا None). این الگو (طبق F-07) تضمین می‌کند که CSV ساختار یکسانی برای همه‌ی ردیف‌ها داشته باشد.

۱۴.۴.۱۱ تابع run_and_save_csv: هماهنگ‌کننده اصلی

تابع run_and_save_csv (خط ۲۵۸۳) هماهنگ‌کننده‌ی اصلی است. این تابع در نسخه‌ی جدید امضای گسترده‌ای دارد:

```

۱     def run_and_save_csv(config, num_workers=None, sweeps=None,
۲                          auto_optimize_pulses=False, dwdm=False,
۳                          wdm_channel_count=4, wdm_channel_power_dbm=None,
۴                          wdm_raman_coefficient=None):
۵

```

پارامترهای جدید شامل auto_optimize_pulses، dwdm، wdm_channel_count، wdm_channel_power_dbm و wdm_raman_coefficient هستند. این پارامترها به init_worker ارسال می‌شوند.

جریان اصلی شامل این مراحل است: (۱) پیکربندی logging؛ (۲) اعتبارسنجی num_workers و بررسی RAM (با psutil، طبق F-27)؛ (۳) اعتبارسنجی پیکربندی شبیه‌سازی؛ (۴) ساخت source_tmp و protocol_name؛ (۵) اعتبارسنجی sweeps؛ (۶) تولید ترکیب‌های Cartesian با itertools.product و هشدار در صورت تجاوز از 10000 ترکیب (طبق F-15)؛ (۷) تولید work_items با پیمایش (dist, pulses, noise, combo)؛ (۸) ساخت CSV fieldnames با ترکیب ستون‌های ثابت و ستون‌های پویا (*sweep)؛ (۹) فراخوانی dispatch_work_items با callback on_row.

۱۵.۴.۱۱ محاسبه QBER اصلاح‌شده و فاصله اطمینان

در نسخه‌ی جدید، QBER اصلاح‌شده و فاصله‌ی اطمینان از stats (که ممکن است اصلاحات واکوئم/سیگنال/دکوی اعمال شده باشد) محاسبه می‌شود:

```

۱ corrected_total_sifted = sum(tc.sifted for tc in stats.values())
۲ corrected_total_errors = sum(tc.errors_sifted for tc in stats.values())
۳ if corrected_total_sifted > 0:
۴     corrected_qber = corrected_total_errors / corrected_total_sifted
۵ else:
۶     corrected_qber = 0.0
۷ _raw_qber = float(summary.get("qber", (0.0 or (0.0
۸ _raw_ci_low = float(summary.get("qber_ci_low", (0.0 or (0.0
۹ _raw_ci_high = float(summary.get("qber_ci_high", (0.0 or (0.0
۱۰ _ci_half_width = max(_raw_qber - _raw_ci_low, _raw_ci_high - _raw_qber)
۱۱ corrected_ci_low = max(0.0), corrected_qber - _ci_half_width)
۱۲ corrected_ci_high = min(0.1), corrected_qber + _ci_half_width)
۱۳

```

در این بلاک، QBER از مجموع خطاها تقسیم بر مجموع sifted محاسبه می‌شود. نیم‌عرض فاصله‌ی اطمینان از summary پروتکل گرفته می‌شود (چون summary از stats خام محاسبه شده و ممکن است نادرست باشد، اما نیم‌عرض به‌عنوان تقریب عدم قطعیت آماری قابل قبول است). سپس فاصله‌ی اطمینان اصلاح‌شده با اعمال این نیم‌عرض روی QBER اصلاح‌شده ساخته می‌شود. ردیف‌های نهایی CSV شامل تمام ستون‌های ثابت، metadata آشکارساز، metadata منبع، ستون‌های override آشکارساز، ستون‌های DWDM، و ستون‌های sweep است. در پایان، overrides به‌صورت ستون‌های *sweep اضافه می‌شوند.

۱۶.۴.۱۱ استفاده از dispatch_work_items

تابع dispatch_work_items از ماژول main_optimized_dispatch وارد شده است. این تابع مسیر تک‌کارگره (حلقه‌ی for ترتیبی) و مسیر چندکارگره (mp.Pool.imap_unordered) را مدیریت می‌کند. on_row یک callback است که هر ردیف را در CSV می‌نویسد و پیشرفت را گزارش می‌دهد. این جداسازی (طبق کامنت در خط ۱۹) امکان synchronization test و ordering contract با stub workers را فراهم می‌سازد.

۱۷.۴.۱۱ تابع build_pn_lookup_table

تابع build_pn_lookup_table (خط ۱۶۷۵) یک جدول lookup برای $P(n)$ می‌سازد:

```

۱ def build_pn_lookup_table(source, max_n=PN_TRUNCATION_DIM):
۲     pulse_names = source.pulse_names()
۳     n_pulses = len(pulse_names)
۴     table = np.zeros((n_pulses, max_n), dtype=np.float64)
۵     for i, name in enumerate(pulse_names):
۶         mu = source.get_mean_photon_number_by_name(name)
۷         table[i, :] = photon_number_distribution(
۸             mu, source.statistics_type, max_n=max_n
۹         )
۱۰     return table
۱۱

```

این تابع یک آرایه‌ی NumPy به شکل (n_pulse_types, max_n) می‌سازد که در آن table[i, n] مقدار $P(n)$ برای نوع پالس i را نگه می‌دارد. این جدول می‌تواند توسط حلقه‌های Numba یا محاسبات vectorized

بدون overhead شیء Python مصرف شود. این طراحی مستقیماً به معیار بازآوری «افشایش آرایه‌های خام NumPy» پاسخ می‌دهد.

۱۸.۴.۱۱ توابع log_detector_details و log_source_details

این دو تابع (در خطوط ۷۴۴ و ۹۸۱) برای ثبت اطلاعات تشخیصی در سطح DEBUG طراحی شده‌اند. تابع log_source_details همه ویژگی‌های منبع را با استفاده از متدهای پرس‌وجو (query)، مانند source.pulse_names() و source.base_mean_photon_numbers()، در لاگ ثبت می‌کند. تابع log_detector_details نیز به‌طور مشابه برای آشکارساز عمل می‌کند و با استفاده از to_config_dict()، یک نمایه کامل از پیکربندی آن را ثبت می‌نماید. این الگو، مطابق با F-26، تضمین می‌کند که مقادیر ثبت‌شده در لاگ با حالت داخلی اعتبارسنجی‌شده سامانه سازگار باشند.

۱۹.۴.۱۱ توابع کمکی pipeline

چندین تابع کمکی برای pipeline شبیه‌سازی وجود دارند: build_channel_from_distance (خط ۱۰۵۱): یک FiberChannel اعتبارسنجی‌شده می‌سازد و در صورت lossless بودن، لاگ می‌کند.

build_protocol (خط ۱۰۷۴): یک شیء Protocol می‌سازد. این تابع از dispatch بر اساس protocol_class استفاده می‌کند و در صورت عدم تطابق، ConfigurationError با فهرست کلاس‌های معتبر پرتاب می‌نماید.

generate_photons_for_prepared_states, get_pulse_indices_from_prepared_states

detector_result_to_protocol_detection_results, extract_detection_arrays, infer_ideal_outcomes.

این توابع برای استخراج و تبدیل داده‌ها بین مراحل مختلف pipeline طراحی شده‌اند.

get_source_metadata_row (خط ۱۴۸۸) و get_params_metadata_row (خط ۱۶۲۰): این توابع برای استخراج metadata از اشیای source و params به ستون‌های CSV طراحی شده‌اند. get_source_metadata_row از متدهای query منبع استفاده می‌کند (نه to_dict()) تا تضمین کند metadata با حالت داخلی هماهنگ باشد. get_params_metadata_row فیلدهای DWDM را استخراج می‌کند و compute_raman_dark_rate_hz را برای محاسبه نرخ Raman فراخوانی می‌نماید.

۲۰.۴.۱۱ الگوی __main__ و CLI

نقطه‌ی ورود __main__ در خطوط ۲۷۳۲-۳۰۰۳ پیاده‌سازی شده است. این بلاک از argparse با بیش از ۳۰ گزینه استفاده می‌کند. پارامترهای کلیدی عبارت‌اند از: --protocol-class، --fiber-loss، --min/max-pulses-log، --rng-seed، --workers، پارامترهای آشکارساز (--dark-rate، --det-eff-d0، --qber-intrinsic، ...)، پارامترهای منبع (--statistics-type، --error-model، --intended-proof، ...)، پارامترهای DWDM (--wdm-channel-count، --wdm-channel-power-dbm، --wdm-raman-coefficient)، پارامترهای امنیتی (--eps-sec)، پارامترهای تصحیح خطا (--f-error-correction، --eps-smooth، --eps-pe، --eps-cor)، و --auto-optimize-pulses. در صورت KeyboardInterrupt، SimulationInterruptedError یا خروج 130 (کد استاندارد POSIX برای SIGINT) خارج می‌شود. در صورت QKDException، خطا با کد خروج 1 خارج می‌شود. در صورت خطاهای دیگر، _as_qkd_exception اعمال می‌شود و سپس با کد خروج 1 خارج می‌گردد.

۵.۱۱ ارسال و دریافت با سایر ماژول‌ها

۱.۵.۱۱ ورودی‌های اصلی ماژول

ورودی‌های اصلی این ماژول عبارت‌اند از: (۱) DEFAULT_CONFIG به‌عنوان پیکربندی پایه؛ (۲) آرگومان‌های CLI که روی پیکربندی پایه اعمال می‌شوند؛ (۳) --sweep-json به‌عنوان JSON تعریف پیمایش؛ (۴) پارامترهای DWDM و Raman از CLI.

۲.۵.۱۱ خروجی‌های اصلی ماژول

خروجی‌های اصلی عبارت‌اند از: (۱) فایل `qkd_results_optimized_sweep.csv` با ساختار ستون‌های ثابت؛ (۲) لاگ‌های `stderr/stdout` برای پیشرفت و خطاها؛ (۳) کد خروج `POSIX` برای `CI/CD`.

۳.۵.۱۱ تعامل با ماژول `qkd.sources`

این ماژول از `qkd.sources` کلاس‌های `DensityMatrixSource`، `PoissonSource`، `OpticalSource` و توابع `PN_TRUNCATION_DIM`، `thermal_pn_array`، `poisson_pn_array`، `photon_number_distribution`، `SourceSimParams`، `build_source_sim_params` را وارد می‌کند. تابع `build_source` از `OpticalSource.create` استفاده می‌کند که `dispatch` بین `PoissonSource` و `DensityMatrixSource` را مدیریت می‌نماید و اعتبارسنجی ماتریس چگالی و `security metadata` را داخلی انجام می‌دهد.

۴.۵.۱۱ تعامل با ماژول `qkd.detectors`

این ماژول از `qkd.detectors` کلاس‌های `DeadTimeModel`، `AfterpulseModel`، `SinglePhotonDetector`، `DetectorSimParams`، `DetectionResult`، `DetectionDiagnostics`، `EntanglingDecoder`، `build_detector_sim_params` و `ROGERS_2007_POLICY` و توابع تحلیلی `Rogers` را وارد می‌کند. تابع `build_detector` از `build_detector` استفاده می‌کند که تمام پارامترها را در یک گام اعتبارسنجی می‌نماید.

۵.۵.۱۱ تعامل با ماژول `qkd.channel`

این ماژول از `qkd.channel` کلاس `FiberChannel`، توابع `build_attenuation_config_from_sim_config`، `transmittance_array`، `transmittance_scalar`، `link_efficiency`، `ChannelSimParams`، `build_channel_sim_params` و ثابت‌های `MAX_DISTANCE_KM`، `MAX_FIBER_LOSS_DB_KM` را وارد می‌کند. تابع `build_attenuation_config` به‌عنوان `thin wrapper` به `build_attenuation_config_from_sim_config` تفویض می‌کند تا تمام اعتبارسنجی در یک ماژول متمرکز باشد.

۶.۵.۱۱ تعامل با ماژول `qkd.protocols`

این ماژول از `qkd.protocols` کلاس‌های `B92Protocol`، `BB84DecoyProtocol`، `Protocol`، `MDIQKDProtocol`، `RedundantTransmissionProtocol`، `DetectionResults`، `SiftingResults`، `BB84PreparedStates`، `B92PreparedStates`، `MDIPreparedStates` و توابع `make_worker_rngs`، `sample_for_parameter_estimation` را وارد می‌کند. تابع `build_protocol` بر اساس `protocol_class` کلاس مناسب را نمونه‌سازی می‌کند.

۷.۵.۱۱ تعامل با ماژول‌های اثبات امنیتی

این ماژول با چهار ماژول اثبات در تعامل است: `qkd.proofs.lim2014` (Lim2014Proof)، `qkd.proofs.ma2005` (Ma2005VacuumWeakProof)، `qkd.proofs.tight` (Wang2005Proof)، `qkd.proofs.optimization` (BB84TightProof) و `qkd.proofs.optimize_pulses_for_distance`. `import` به‌صورت `lazy` (داخل `if`) انجام می‌شود. مدیریت API متفاوت این کلاس‌ها در بخش پیاده‌سازی بررسی شد.

۸.۵.۱۱ تعامل با ماژول `main_optimized_dispatch`

این ماژول تابع `dispatch_work_items` را از `main_optimized_dispatch` وارد می‌کند. این جداسازی امکان `synchronization test` و `ordering contract` را با `stub workers` فراهم می‌سازد، بدون نیاز به بارگذاری `qkd.*` در زمان `import`.

۹.۵.۱۱ تعامل با `qkd.params`، `qkd.io`، `qkd.exceptions` و `qkd.datatypes`

این مازول از `qkd.params` توابع `load_lim2014_dwdm_params` و `load_lim2014_dedicated_params` را وارد می‌کند. از `qkd.io` توابع `open_atomic_text`، `parse_json_strict`، `safe_json_dumps` را وارد می‌نماید. از `qkd.exceptions` سلسله‌مراتب `QKDException` را وارد می‌کند. از `qkd.datatypes` تمام `dataclass` ها و `Enum` ها را وارد می‌نماید.

۱۰.۵.۱۱ مدیریت خطا و تبدیل استثنا

الگوی یکپارچه‌ی خطا با `_as_qkd_exception` و `_log_qkd_exception` (طبق F-23) در تمام `catch-block` ها اعمال می‌شود. این الگو تضمین می‌کند که خطاها به‌صورت یکپارچه ثبت شوند و CSV ساختار یکسانی برای همه‌ی ردیف‌ها داشته باشد.

۱۱.۵.۱۱ مدیریت `KeyboardInterrupt`

در صورت `KeyboardInterrupt`، `SimulationInterruptedError` ساخته می‌شود و با کد خروج 130 خارج می‌شود. این کد، استاندارد POSIX برای `SIGINT` است و به `shell` اجازه می‌دهد تشخیص دهد که کاربر اجرا را لغو کرده است.

۱۲.۵.۱۱ نقش مازول در اکوسیستم کلی

در اکوسیستم کلی فریم‌ورک، `main_optimized.py` به‌عنوان `entry point` اصلی عمل می‌کند و تمام مازول‌های تخصصی (`datatypes`، `exceptions`، `io`، `params`، `proofs`، `protocols`، `channel`، `detectors`، `sources`) را در یک `pipeline` یکپارچه هماهنگ می‌نماید. خروجی CSV این مازول، ورودی ابزارهای تحلیل بعدی (`pandas`، `matplotlib`، `R`) است. این معماری، جداسازی واضحی بین لایه‌ی هماهنگ‌کننده و لایه‌های تخصصی برقرار می‌سازد و امکان توسعه‌ی مستقل هر مازول را فراهم می‌نماید.

لایه شبکه QKD و مسیریابی

را مقداره‌ی اولیه می‌کند، سپس با اجرای نشست‌های متعدد QKD روی هر لینک، استخرهای کلید را تا سطح high watermark پر می‌کند (priming)، پس از آن چهار درخواست کلید خدمت (service key) از طرف ارسال‌کننده‌ها به گیرنده A را پردازش می‌کند، یک خرابی لینک را شبیه‌سازی می‌کند و در نهایت فرآیند replenishment را برای استخرهای تخلیه‌شده اجرا می‌نماید. خروجی این demo یک گزارش متنی کامل از وضعیت توپولوژی، معیارهای هر لینک، سطح هر استخر، گام‌های رمزگذاری/رمزگشایی XOR، و مسیرهای جایگزین پس از خرابی است که برای مستندسازی و اعتبارسنجی رفتار شبکه بسیار ارزشمند است.

۲.۱۲ مبانی تئوری و فیزیکی

۱.۲.۱۲ مدل کانال فیبر نوری و انتقال

در شبیه‌سازی لینک‌های QKD در این ماژول، پایه‌ای‌ترین کمیت فیزیکی، ضریب انتقال کانال (channel transmittance) است که نسبت تعداد فوتون‌های دریافتی به فوتون‌های ارسالی را نشان می‌دهد. برای یک فیبر نوری با طول d کیلومتر و ضریب تلفت α دسی‌بل بر کیلومتر، تلفات کلی کانال برابر با $\alpha \cdot d$ دسی‌بل است. چون مقیاس دسی‌بل لگاریتمی است، تبدیل آن به ضریب انتقال خطی از رابطه زیر پیروی می‌کند:

$$\eta_{\text{channel}}(d) = 10^{-\frac{\alpha \cdot d}{10}} \quad (1.12)$$

در این رابطه، d بر حسب کیلومتر و α بر حسب dB/km است. به‌طور معمول، فیبر تک‌حالت استاندارد در طول موج 1550 نانومتر ضریب تلفت $\alpha \approx 0.2$ dB/km دارد که در این ماژول به‌عنوان مقدار پیش‌فرض به کار رفته است. ضریب انتقال کلی کانال شامل بازده آشکارساز η نیز می‌شود؛ یعنی:

$$\eta_{\text{total}}(d) = \eta_{\text{det}} \cdot \eta_{\text{channel}}(d) = \eta_{\text{det}} \cdot 10^{-\frac{\alpha \cdot d}{10}} \quad (2.12)$$

این کمیت نقشی محوری در محاسبه نرخ کلید دارد چرا که احتمال شناسایی هر تک‌فوتون ارسالی مستقیماً با η_{total} متناسب است. در فواصل کوتاه متروپولیتن (30 تا 75 کیلومتر)، η_{channel} از $10^{-0.6} \approx 0.25$ تا $10^{-1.5} \approx 0.032$ متغیر است که نشان‌دهنده افت شدید در فواصل دورتر است.

۲.۲.۱۲ مدل خطای کوانتومی (QBER)

نرخ خطای کوانتومی بیت (Quantum Bit Error Rate) که با نماد Q نشان داده می‌شود، کسری از بیت‌های sifted است که بین دو طرف ناهماهنگ هستند. این کمیت معیار اصلی کیفیت لینک QKD است و اگر از یک آستانه بحرانی عبور کند، تولید کلید امن از نظر تئوری غیرممکن می‌شود. در پروتکل BB84 با منبع تک‌فوتونی آرمانی، این آستانه بحرانی برابر با تقریباً 11% است که در ماژول به‌صورت ثابت $\text{QBER_THRESHOLD} = 0.11$ تعریف شده است. مدل QBER به‌کار رفته در StatisticalQKDLinkSimulator چهار مؤلفه مستقل را با هم جمع می‌کند:

$$Q = Q_{\text{intrinsic}} + Q_{\text{polarization}}(d) + Q_{\text{dark}}(d) + Q_{\text{afterpulse}} \quad (3.12)$$

که در آن هر جزء به شرح زیر تعریف می‌شود. $Q_{\text{intrinsic}}$ خطای پایه ناشی از نقص‌های اپتیکی نظیر misalignment تداخل‌سنج است که در ماژول برابر با 0.015 در نظر گرفته شده است؛ این مقدار شامل 1% خطای ذاتی و 0.5% misalignment است که با مقادیر پیش‌فرض DEFAULT_CONFIG در main_optimized.py هم‌خوانی دارد. $Q_{\text{polarization}}(d)$ رشد خطای ناشی از رانش قطبش به دلیل پدیده Polarization Mode Dispersion و دوپسیسم بیرفرینژانس فیبر است:

$$Q_{\text{polarization}}(d) = \kappa_{\text{pol}} \cdot d, \quad \kappa_{\text{pol}} = 10^{-4} \text{ km}^{-1} \quad (4.12)$$

ضریب $\kappa_{\text{pol}} = 10^{-4}$ به‌ازای هر کیلومتر، متناظر با یک سیستم QKD متروپولیتن با تثبیت فعال قطبش است؛ مقدار قبلی 5×10^{-4} به‌ازای هر کیلومتر به‌شدت تهاجمی بود و برای لینک‌های 30 تا 40 کیلومتری، QBER نهایی را در محدوده 2 تا 2.5% تولید می‌کرد که ناتوان از اشتباه‌شدنی بودن از مقادیر گزارش‌شده در ادبیات تجربی

است. مؤلفه Q_{dark} ناشی از آشکارسازی‌های کاذب (dark counts) است و به صورت نسبت نرخ دارک به gain کلی محاسبه می‌شود:

$$Q_{\text{dark}}(d) = \frac{R_{\text{dark}}}{g(d)}, \quad g(d) = \max(\mu \cdot \eta_{\text{total}}(d) + R_{\text{dark}}, 10^{-30}) \quad (5.12)$$

این تعریف نشان می‌دهد که با افزایش فاصله و کاهش $\eta_{\text{total}}(d)$ سهم دارک کانت در کل شناسایی‌ها رشد می‌کند و در نهایت QBER را به سمت آستانه بحرانی هل می‌دهد. مقدار پیش‌فرض $R_{\text{dark}} = 10^{-6}$ به‌ازای هر پالس، متناظر با آشکارسازهای InGaAs SPAD با نرخ دارک 10 هرتز و نرخ پالس 10 مگاهرتز است. در نهایت، $Q_{\text{afterpulse}} = 0.001$ سهم ثابت پالس‌های پسینی (afterpulses) است که از به دام افتادن حامل‌ها در آشکارساز ناشی می‌شود.

۳.۲.۱۲ آنتروپی باینری و کسر امن کلید

نرخ کلید امن در پروتکل BB84 decoy-state به صورت مجانبی از رابطه بنیادین زیر پیروی می‌کند که آنتروپی باینری Shannon را به کسر اطلاعات مشترک امن تبدیل می‌کند:

$$R = q \cdot \eta_{\text{total}}(d) \cdot \mu \cdot f_{\text{secret}}(Q) \quad (6.12)$$

$$f_{\text{secret}}(Q) = \max(0, 1 - 2h_2(Q)) \quad (7.12)$$

در این رابطه، $q = 0.5$ ضریب sifting است که نشان می‌دهد در BB84 تنها نیمی از پالس‌ها پس از مقایسه پایه‌ها sifted می‌شوند. μ میانگین تعداد فوتون در پالس سیگنال است که مقدار پیش‌فرض $\mu = 0.5$ دارد؛ این مقدار بهینه تقریبی برای decoy-state در رژیم مجانبی است. تابع $h_2(Q)$ آنتروپی باینری است:

$$h_2(x) = -x \log_2(x) - (1-x) \log_2(1-x) \quad (8.12)$$

این تابع در $x = 0.5$ به بیشینه مقدار 1 می‌رسد و در $x = 0$ و $x = 1$ به صفر میل می‌کند. عامل $1 - 2h_2(Q)$ نشان‌دهنده کسر امن کلید پس از کسر اطلاعات لو رفته به حامل به دلیل وجود QBER است؛ ضریب 2 به این دلیل است که در BB84، هر خطای بیت هم به نشت اطلاعات به ایو و هم به نیاز تصحیح خطا مربوط می‌شود. وقتی $Q = 0.11$ ، مقدار $h_2(0.11) \approx 0.5$ می‌شود و در نتیجه $f_{\text{secret}} = 0$ ؛ این همان آستانه‌ای است که در آن تولید کلید امن متوقف می‌گردد.

۴.۲.۱۲ مدل استخر کلید و فشار (Pool Pressure)

یکی از نوآوری‌های کلیدی ماژول، معرفی مدل piecewise-linear برای محاسبه فشار استخر کلید است. این مدل امکان تمایز میان سطوح مختلف کمبود کلید را فراهم می‌کند و به الگوریتم SPF+ اجازه می‌دهد مسیرهایی را برگزیند که استخر کلید سالم‌تری دارند. تعریف ریاضی فشار استخر به صورت زیر است:

$$p_{\text{pool}}(a) = \begin{cases} 0 & \text{if } a \geq h \\ 0.5 \left(1 - \frac{a - \ell}{h - \ell}\right) & \text{if } \ell \leq a < h \\ 0.5 + 0.5 \left(1 - \frac{a}{\ell}\right) & \text{if } 0 \leq a < \ell \end{cases} \quad (9.12)$$

در این رابطه، a تعداد بیت‌های موجود در استخر است، ℓ (low watermark) آستانه پایین و h (high watermark) آستانه بالای استخر است. مدل به سه رژیم تقسیم می‌شود: رژیم سالم ($a \geq h$) با فشار صفر، رژیم هشدار ($\ell \leq a < h$) که فشار به صورت خطی از صفر تا 0.5 رشد می‌کند، و رژیم بحرانی ($a < \ell$) که فشار از 0.5 تا 1 افزایش می‌یابد. این طراحی دو-بخشی باعث می‌شود که الگوریتم مسیریابی حساسیت متفاوتی نسبت به استخرهای نزدیک به آستانه بحرانی در مقایسه با استخرهای نزدیک به سالم داشته باشد. ارزش عملی این مدل در آن است که اگر دو مسیر هم‌ارز از نظر فیزیکی وجود داشته باشند، مسیری که استخر کلید سالم‌تری دارد به طور خودکار انتخاب می‌شود.

۵.۲.۱۲ تابع هزینه + SPF و الگوریتم Dijkstra

الگوریتم SPF+ (Shortest Path First Plus) توسعه‌ای از الگوریتم کلاسیک Dijkstra است که به جای استفاده از یک معیار منفرد (مانند فاصله یا تعداد جهش)، یک تابع هزینه ترکیبی استفاده می‌کند که پنج مؤلفه مختلف را با هم وزن‌دهی می‌کند. تابع هزینه برای هر یال در گراف شبکه به صورت زیر تعریف می‌شود:

$$C_{\text{link}} = \alpha \cdot \frac{L}{L_{\text{max}}} + \beta \cdot \frac{Q}{Q_{\text{th}}} + \gamma \cdot \frac{d}{d_{\text{max}}} + \delta \cdot p_{\text{pool}} - \epsilon \cdot \frac{R}{R_{\text{max}}} \quad (10.12)$$

در این رابطه، L تلفات نوری لینک بر حسب دسی‌بل، Q مقدار QBER فعلی، d فاصله فیزیکی، p_{pool} فشار استخر کلید، و R نرخ تولید کلید امن است. هر یک از این کمیت‌ها به مقدار بیشینه خود نرمال‌سازی می‌شود تا در محدوده $[0, 1]$ قرار گیرند و سپس با ضرایب وزن $\alpha, \beta, \gamma, \delta$ و ϵ ترکیب می‌شوند. مقادیر پیش‌فرض این ضرایب در ماژول به صورت $\alpha = 1.0, \beta = 5.0, \gamma = 0.5, \delta = 2.0$ و $\epsilon = 5.0$ تعریف شده‌اند. یک ویژگی مهم تابع هزینه بالا، عبارت منفی $-\epsilon \cdot (R/R_{\text{max}})$ است که به عنوان یک bonus عمل می‌کند؛ یعنی لینک‌هایی که نرخ کلید بالاتری دارند، هزینه کمتری خواهند داشت و در نتیجه به احتمال بیشتری انتخاب می‌شوند. این طراحی با این واقعیت فیزیکی هم‌خوان است که لینک‌های پربازده‌تر معمولاً استخر کلید پرتری دارند و می‌توانند در آینده نیز بهتر پاسخگو باشند. اما اگر QBER از آستانه بحرانی Q_{th} عبور کند، یک جریمه سنگین اما متناهی به هزینه اضافه می‌شود:

$$C_{\text{link}} += H_{\text{penalty}} \cdot \frac{Q}{Q_{\text{th}}}, \quad \text{if } Q > Q_{\text{th}} \quad (11.12)$$

که در آن $H_{\text{penalty}} = 10^6$ است. دلیل استفاده از جریمه متناهی به جای هزینه بی‌نهایت آن است که اگر همه لینک‌ها به دلیل پیکربندی نامناسب QBER بالا داشته باشند، الگوریتم همچنان قادر به یافتن مسیر کم‌ترین ضرر باشد. این رفتار به صورت عملی در حین توسعه مشاهده شد که در آن پیکربندی نامناسب موتور فیزیکی باعث QBER بالا در همه لینک‌ها شد و الگوریتم با هزینه بی‌نهایت قادر به یافتن هیچ مسیری نبود. الگوریتم Dijkstra با این تابع هزینه از طریق صف اولویت min-heap پیاده‌سازی می‌شود. پیچیدگی زمانی آن با استفاده از heapq برابر با $\mathcal{O}((V + E) \log V)$ است که در آن V تعداد گره‌ها و E تعداد یال‌هاست. در شبکه هفت‌گره‌ای با ده یال، این پیچیدگی ناچیز است و امکان اجرای مکرر SPF+ در هر درخواست کلید خدمت را بدون تأثیر قابل توجه بر کارایی فراهم می‌کند.

۶.۲.۱۲ پروتکل Trusted Relay مبتنی بر XOR

یکی از مفاهیم بنیادین در شبکه‌های QKD، انتقال کلید به سراسر شبکه از طریق رله‌های قابل اعتماد است. در این ماژول، این انتقال با استفاده از One-Time Pad مبتنی بر XOR انجام می‌شود که از نظر تئوری امنیت اطلاعات کامل را به شرط آنکه کلیدهای هر جهش کاملاً مخفی و یک‌بار مصرف باشند، فراهم می‌سازد. برای یک مسیر سه‌گره‌ای مانند $A \rightarrow B \rightarrow D$ ، فرآیند انتقال به صورت زیر است:

$$C_1 = K_{\text{service}} \oplus K_{DB} \quad (12.12)$$

$$C_2 = C_1 \oplus K_{DB} \oplus K_{BA} = K_{\text{service}} \oplus K_{BA} \quad (13.12)$$

$$K_{\text{service}} = C_2 \oplus K_{BA} \quad (14.12)$$

در معادله اول، فرستنده D کلید خدمت را با کلید جهش K_{DB} (که با رله B مشترک است) XOR می‌کند و رمزنگاشت C_1 را می‌سازد. در گام دوم، رله B ابتدا C_1 را با K_{DB} رمزگشایی می‌کند تا به کلید خدمت برسد، سپس آن را با K_{BA} (کلید مشترک با گیرنده A) رمزگذاری می‌کند و C_2 را تولید می‌نماید. معادله دوم نشان می‌دهد که از نظر ریاضی، $C_2 = K_{\text{service}} \oplus K_{BA}$ چون دو XOR با K_{DB} یکدیگر را خنثی می‌کنند. در گام سوم، گیرنده A با XOR کردن C_2 با K_{BA} به کلید خدمت اصلی می‌رسد. این پروتکل از نظر تئوری امن است زیرا هر

ناظر می‌تواند تنها C_1 یا C_2 را ببیند، ولی بدون دانستن K_{DB} یا K_{BA} نمی‌تواند اطلاعات کمی از $K_{service}$ به دست آورد.

شرط حیاتی این پروتکل آن است که رله‌ها قابل اعتماد باشند؛ یعنی کلید جهش را فاش نکنند. به همین دلیل، این رله‌ها در ماژول به صورت کلاس TrustedRelayNode تعریف شده‌اند که نقش آن‌ها به صورت صریح از گره‌های ارسال‌کننده و گیرنده متمایز است. در شبکه‌های عملیاتی، این رله‌ها معمولاً در سایت‌های قابل اعتماد مانند دیتاسنترهای امن قرار دارند و کنترل فیزیکی و منطقی روی آن‌ها اعمال می‌شود. در صورتی که رله قابل اعتماد نباشد، باید از پروتکل‌های QKD Measurement-Device-Independent یا Entanglement-Based استفاده کرد که در آن‌ها اعتماد به رله حذف می‌شود.

۷.۲.۱۲ سوییج‌های نوری و پیکربندی پویا

در این ماژول، سه سوییج نوری مدل شده‌اند که امکان پیکربندی پویای توپولوژی شبکه را فراهم می‌سازند. سوییج 1x2 در گره A به این گره اجازه می‌دهد به صورت انتخابی به یکی از دو رله B یا C متصل شود؛ این سوییج از نوع MEMS یا electro-optic است که زمان سوییج در محدوده میکروثانیه تا میلی‌ثانیه دارد. دو سوییج 2x4 در گره‌های B و C امکان اتصال هر رله به یکی از چهار ارسال‌کننده E، D، F یا G را فراهم می‌سازند. در عمل، این سوییج‌ها در زمان اجرای demo به صورت خودکار پیش از اجرای هر درخواست کلید خدمت پیکربندی می‌شوند تا مسیر انتخاب‌شده توسط SPF+ در دسترس باشد.

۳.۱۲ تحلیل معماری و ساختار داده‌ها

۱.۳.۱۲ شمارش‌ها (Enums) و ثابت‌های سراسری

ماژول با تعریف چهار Enum پایه آغاز می‌شود که وضعیت‌های ممکن برای مؤلفه‌های مختلف شبکه را مشخص می‌کنند. این شمارش‌ها به جای استفاده از رشته‌های خام، نوع داده ایمن و قابل اعتبارسنجی فراهم می‌سازند و از خطاهای املایی جلوگیری می‌کنند.

```

1 class KeyBlockState(Enum):
2     READY = "READY"
3     RESERVED = "RESERVED"
4     USED = "USED"
5     EXPIRED = "EXPIRED"
6     REVOKED = "REVOKED"
7
8 class KeyPoolStatus(Enum):
9     ACTIVE = "ACTIVE"
10    LOW = "LOW"
11    EMPTY = "EMPTY"
12    DISABLED = "DISABLED"
13    ERROR = "ERROR"
14    REPLENISHING = "REPLENISHING"
15
16 class LinkStatus(Enum):
17    UP = "UP"
18    DOWN = "DOWN"
19    DEGRADED = "DEGRADED"
20
21 class NodeRole(Enum):
22    RECEIVER = "RECEIVER"
23    RELAY = "RELAY"
24    SENDER = "SENDER"
25
```

شمارش KeyBlockState چرخه حیات یک بلوک کلید را مشخص می‌کند: READY به معنای آماده برای مصرف، RESERVED به معنای رزرو شده برای یک درخواست در حال انجام، USED به معنای مصرف‌شده، EXPIRED به معنای منقضی‌شده پس از یک ساعت، و REVOKED به معنای ابطال‌شده به دلیل خرابی. شمارش KeyPoolStatus وضعیت سلامت استخر را نشان می‌دهد و شش حالت را پشتیبانی می‌کند؛ دو حالت DISABLED و ERROR به صورت چسبناک (sticky) عمل می‌کنند به این معنا که تنها مداخله خارجی می‌تواند آن‌ها را تغییر دهد. شمارش LinkStatus وضعیت فیزیکی لینک را نشان می‌دهد و شامل حالت DEGRADED نیز می‌شود که برای لینک‌هایی با کیفیت پایین اما قابل استفاده به کار می‌رود. شمارش NodeRole نقش هر گره را در توپولوژی شبکه مشخص می‌کند و در الگوریتم‌های مسیریابی و رله استفاده می‌شود.

در بخش ثابت‌های سراسری، مهم‌ترین کمیت‌ها عبارت‌اند از: $\text{INFINITY_COST} = 1e18$ که به‌عنوان هزینه بی‌نهایت در الگوریتم Dijkstra به کار می‌رود و از مقدار عددی بزرگ اما قابل نمایش در float64 استفاده می‌کند؛ $\text{QBER_THRESHOLD} = 0.11$ که آستانه بحرانی پروتکل BB84 است؛ و $\text{HIGH_PENALTY} = 1e6$ که جریمه‌ای متناهی اما سنگین برای لینک‌های با QBER بالای آستانه است. علاوه بر این، دو مجموعه ثابت watermark برای دو نوع شبیه‌ساز تعریف شده است: مجموعه STAT برای شبیه‌ساز آماری با مقادیر 1 مگابیت، 10 مگابیت و 20 مگابیت، و مجموعه PIPELINE برای شبیه‌ساز خط لوله با مقادیر 10 کیلوبیت، 100 کیلوبیت و 200 کیلوبیت. این تفکیک لازم است چرا که خروجی دو شبیه‌ساز در مقیاس‌های متفاوتی قرار دارد؛ شبیه‌ساز آماری در حدود 0.01 تا 0.04 بیت بر پالس تولید می‌کند در حالی که خط لوله واقعی در حدود 10^{-4} تا 3×10^{-4} بیت بر پالس تولید می‌نماید.

۲.۳.۱۲ داده‌کلاس KeyBlock

کلاس KeyBlock با استفاده از @dataclass پیاده‌سازی شده و واحد بنیادی ذخیره کلید در استخر را تشکیل می‌دهد. هر بلوک شامل یک شناسه یکتا، شناسه لینک تولیدکننده، گره محلی، گره همتا، ماده کلید به‌صورت بایت‌های خام، طول به بیت، وضعیت، زمان ایجاد، زمان انقضا، شناسه نشست QKD تولیدکننده، QBER حین تولید و نرخ کلید امن است. استفاده از dataclass مزایای متعددی دارد: تولید خودکار سازنده، `__repr__` و `__eq__`، پشتیبانی از type hint، و امکان `frozen=True` برای نسخه‌های تغییرناپذیر.

```

1  @dataclass
2  class KeyBlock:
3      key_id: str
4      link_id: str
5      local_node: str
6      peer_node: str
7      key_material: bytes
8      length_bits: int
9      state: KeyBlockState
10     created_at: float
11     expires_at: Optional[float]
12     qkd_session_id: str
13     qber: float
14     secret_key_rate: float
15 
```

طراحی این داده‌کلاس بر اساس اصل provenance tracking است؛ یعنی هر بلوک کلید حامل تمام متادیتای لازم برای ردیابی تاریخچه تولید خود است. این متادیتا در موارد زیر کاربرد دارد: در زمان انتخاب مسیر SPF+، اگر QBER حین تولید بالاتر از حد باشد، بلوک می‌تواند اولویت پایین‌تری داشته باشد؛ در گزارش‌گیری و حسابرسی، شناسه نشست QKD امکان پیگیری کلید تا نشست اصلی را فراهم می‌سازد؛ و در زمان انقضا، زمان ایجاد به‌عنوان مبنا استفاده می‌شود. طول پیش‌فرض هر بلوک 256 بیت است که برای کلیدهای AES-256 یا HMAC-256 مناسب است؛ این مقدار به‌صورت ثابت در تابع `run_qkd_session` تعریف شده است.

۳.۳.۱۲ کلاس KeyPool و مدیریت چرخه حیات

کلاس KeyPool مسئول مدیریت مجموعه‌ای از بلوک‌های کلید برای یک جهت مشخص از یک لینک QKD است. این کلاس به‌جای ذخیره ساده بلوک‌ها، شمارش‌گرهای جداگانه برای بیت‌های موجود (available)، رزرو شده (reserved) و مصرف‌شده (used) را نگه می‌دارد و یک ماشین وضعیت (state machine) برای انتقال میان وضعیت‌های استخر را پیاده‌سازی می‌کند. این طراحی باعث می‌شود که پرس‌وجوهای متداول مانند `available_key_bits` در زمان $O(1)$ اجرا شوند، در حالی که عملیات پیچیده‌تر مانند `reserve_keys` به $O(n \log n)$ نیاز دارد که در آن n تعداد بلوک‌ها است.

```

1  class KeyPool:
2      def __init__(
3          self,
4          pool_id: str,
5          local_node: str,
6          peer_node: str,
7          link_id: str,
8          max_capacity_bits: int = DEFAULT_MAX_CAPACITY_BITS,
9          low_watermark_bits: int = DEFAULT_LOW_WATERMARK_BITS,
10         high_watermark_bits: int = DEFAULT_HIGH_WATERMARK_BITS,

```

```

۱۱     ) -> None:
۱۲     self.pool_id: str = pool_id
۱۳     self.local_node: str = local_node
۱۴     self.peer_node: str = peer_node
۱۵     self.link_id: str = link_id
۱۶     self.key_blocks: Dict[str, KeyBlock] = {}
۱۷     self.available_key_bits: int = 0
۱۸     self.reserved_key_bits: int = 0
۱۹     self.used_key_bits: int = 0
۲۰     self.max_capacity_bits: int = max_capacity_bits
۲۱     self.low_watermark_bits: int = low_watermark_bits
۲۲     self.high_watermark_bits: int = high_watermark_bits
۲۳     self.status: KeyPoolStatus = KeyPoolStatus.EMPTY
۲۴     self.last_updated: float = time.time()
۲۵

```

روش `add_key_block` دارای یک محافظ دفاعی (defensive cap) است که از سرریز استخراج جلوگیری می‌کند. اگر افزودن یک بلوک جدید باعث شود مجموع بیت‌های موجود و رزرو شده از ظرفیت بیشینه تجاوز کند، بلوک به صورت خاموش رد می‌شود. این محافظ در زمان توسعه به دلیل یک باگ مشاهده شد که در آن شبیه‌ساز آماری با بازده آشکارساز $\eta = 0.65$ در برابر استخرهای با مقیاس خط لوله 200 کیلوبیتی اجرا می‌شد و میلیون‌ها بیت در هر نشست تولید می‌کرد که استخراج را به سرعت پر می‌کرد و حافظه را تخلیه می‌نمود. یکی از پیچیده‌ترین روش‌های این کلاس، `reserve_keys` است که بلوک‌ها را به صورت نزولی بر اساس اندازه مرتب می‌کند تا تعداد بلوک‌های مصرف‌شده کمینه شود و قطعه‌بندی (fragmentation) کاهش یابد. اگر بلوکی بزرگ‌تر از مقدار مورد نیاز باشد، روش `_split_block_for_reservation` آن را به دو بخش تقسیم می‌کند: یک بلوک رزرو شده با اندازه دقیق مورد نیاز و یک بلوک آماده با باقی‌مانده. این رفتار تضمین می‌کند که `available_key_bits` دقیقاً به اندازه `length_bits` کاهش یابد.

```

۱     ready_blocks = sorted(
۲         [b for b in self.key_blocks.values() if b.state == KeyBlockState.READY],
۳         key=lambda b: -b.length_bits,
۴     )
۵
۶     for block in ready_blocks:
۷         if accumulated >= length_bits:
۸             break
۹         needed = length_bits - accumulated
۱۰        if block.length_bits <= needed:
۱۱            block.state = KeyBlockState.RESERVED
۱۲            self.available_key_bits -= block.length_bits
۱۳            self.reserved_key_bits += block.length_bits
۱۴            accumulated += block.length_bits
۱۵            reserved.append(block)
۱۶        else:
۱۷            self._split_block_for_reservation(block, needed, reserved)
۱۸            accumulated += needed
۱۹

```

دلیل اهمیت مرتب‌سازی نزولی در این روش آن است که dict در پایتون به ترتیب درج را حفظ می‌کند. اگر دو نشست آماده‌سازی هر کدام یک بلوک کوچک `remaining_bits` (مثلاً 32 بیت) باقی بگذارند و این بلوک کوچک قبل از بلوک 256 بیتی بعدی درج شود، الگوریتم اصلی (با ترتیب درج) ابتدا بلوک 32 بیتی را برمی‌داشت (انباشته 32 =)، سپس بلوک 256 بیتی بعدی را (انباشته 288 =)؛ این یعنی برای یک درخواست 256 بیتی، 32 بیت اضافه رزرو می‌شد. در چهار درخواست کلید خدمت، این رفتار به 1062 بیت مصرف به جای 1024 بیت مورد انتظار منجر می‌شد. با مرتب‌سازی نزولی، الگوریتم ترجیحاً یک بلوک 256 بیتی منفرد را برمی‌دارد (تطبیق دقیق) به جای ترکیب بلوک کوچک با بلوک بزرگ.

روش `_split_block_for_reservation` پیاده‌سازی دقیق تقسیم را با byte alignment انجام می‌دهد. `needed_bits` به نزدیک‌ترین مضرب 8 گرد می‌شود تا برش بابت‌ها دقیق باشد؛ این کار ممکن است در موارد نادر تا 7 بیت `over-reservation` ایجاد کند. در موارد لبه‌ای که گرد کردن از اندازه بلوک تجاوز کند، مقدار به اندازه بلوک محدود می‌شود.

```

۱     needed_aligned = (needed_bits + 7) // 8 * 8
۲     if needed_aligned > block.length_bits:
۳         needed_aligned = block.length_bits
۴     remainder_bits = block.length_bits - needed_aligned

```

```

۵         needed_bytes = needed_aligned // 8
۶
۷         del self.key_blocks[block.key_id]
۸
۹         taken_id = f"{block.key_id}#R-{uuid.uuid4().hex[:6]}"
۱۰        taken_block = KeyBlock(
۱۱            key_id=taken_id,
۱۲            link_id=block.link_id,
۱۳            local_node=block.local_node,
۱۴            peer_node=block.peer_node,
۱۵            key_material=block.key_material[:needed_bytes],
۱۶            length_bits=needed_aligned,
۱۷            state=KeyBlockState.RESERVED,
۱۸            ...
۱۹        )
۲۰

```

روش `calculate_pressure` مدل `piecewise-linear` توضیح داده شده در بخش مبانی تئوری را پیاده سازی می کند. این روش در هر بار فراخوانی `SPF+` برای هر دو استخر هر یال اجرا می شود و حداکثر آن دو به عنوان فشار یال در نظر گرفته می شود. این به این معناست که اگر یک استخر کاملاً پر و دیگری خالی باشد، فشار کل برابر 1 خواهد بود – رفتاری منطقی چرا که جهت برگشت نیز به کلید نیاز دارد.

روش `check_status` ماشین وضعیت استخر را مدیریت می کند. قواعد انتقال آن به صورت زیر است: وضعیت های `DISABLED` و `ERROR` چسبناک هستند و تنها مداخله خارجی می تواند آن ها را تغییر دهد؛ `REPLENISHING` به `ACTIVE` تبدیل می شود وقتی استخر به `low watermark` برسد (بدون نیاز به `high watermark`، چرا که در حالت خط لوله یک نشست ممکن است تا `high watermark` پر نکند)؛ `EMPTY` به `LOW` تبدیل می شود با افزودن هر بیت؛ و `LOW` به `ACTIVE` با رسیدن به `low watermark`.

۴.۳.۱۲ کلاس QKDLINK و تابع هزینه

کلاس `QKDLINK` یک لینک فیزیکی بین دو گره را مدل می کند و شامل پارامترهای فیزیکی (فاصله، تلفات فیبر) و معیارهای عملیاتی (`QBER`، نرخ کلید، تلفات کلی) است. تابع `get_cost` این کلاس تابع هزینه `SPF+` را پیاده سازی می کند که در بخش تئوری به صورت کامل توضیح داده شد.

```

۱     class QKDLINK:
۲         def __init__(
۳             self,
۴             link_id: str,
۵             node_a: str,
۶             node_b: str,
۷             distance_km: float,
۸             fiber_loss_db_km: float = 2.0,
۹         ) -> None:
۱۰        self.link_id: str = link_id
۱۱        self.node_a: str = node_a
۱۲        self.node_b: str = node_b
۱۳        self.distance_km: float = distance_km
۱۴        self.fiber_loss_db_km: float = fiber_loss_db_km
۱۵        self.status: LinkStatus = LinkStatus.UP
۱۶        self.qber: float = 0.0
۱۷        self.secret_key_rate: float = 0.0
۱۸        self.optical_loss_db: float = fiber_loss_db_km * distance_km
۱۹        self.last_qkd_session_id: str = ""
۲۰

```

در سازنده این کلاس، تلفات کلی لینک به صورت $L = \alpha \cdot d$ محاسبه و در ویژگی `optical_loss_db` ذخیره می شود. این مقدار در تابع `get_cost` به عنوان یکی از پنج مؤلفه هزینه استفاده می شود. تابع `get_cost` ابتدا بررسی می کند که آیا لینک از نظر فیزیکی قطع است؛ اگر چنین باشد، هزینه `INFINITY_COST` برمی گرداند که در الگوریتم `Dijkstra` به معنای عدم استفاده از این یال است. در غیر این صورت، پنج مؤلفه هزینه به صورت نرمال شده محاسبه و با ضرایب وزن ترکیب می شوند.

```

۱     def get_cost(self, spf_params: Dict[str, float]) -> float:
۲         if self.status == LinkStatus.DOWN:
۳             return INFINITY_COST
۴
۵         alpha = spf_params.get("alpha", DEFAULT_ALPHA)

```

```

۶     beta = spf_params.get("beta", DEFAULT_BETA)
۷     gamma = spf_params.get("gamma", DEFAULT_GAMMA)
۸     delta = spf_params.get("delta", DEFAULT_DELTA)
۹     epsilon = spf_params.get("epsilon", DEFAULT_EPSILON)
۱۰    pool_pressure = spf_params.get("pool_pressure", (0.0
۱۱    key_rate = spf_params.get("key_rate", (0.0
۱۲
۱۳    norm_loss = self.optical_loss_db / SPF_MAX_LOSS_DB
۱۴    norm_qber = self.qber / QBER_THRESHOLD
۱۵    norm_distance = self.distance_km / SPF_MAX_DISTANCE_KM
۱۶    norm_key_rate = key_rate / SPF_MAX_KEY_RATE
۱۷
۱۸    cost = (
۱۹        alpha * norm_loss
۲۰        + beta * norm_qber
۲۱        + gamma * norm_distance
۲۲        + delta * pool_pressure
۲۳        - epsilon * norm_key_rate
۲۴    )
۲۵
۲۶    if self.qber > QBER_THRESHOLD:
۲۷        qber_excess_ratio = self.qber / QBER_THRESHOLD
۲۸        cost += HIGH_PENALTY * qber_excess_ratio
۲۹
۳۰    return max(cost, (0.1
۳۱

```

خط نهایی $\text{return max(cost, 0.01)}$ یک کف مثبت برای هزینه تعیین می‌کند تا از بروز هزینه صفر یا منفی جلوگیری شود. این کف ضروری است چرا که عبارت منفی $-\epsilon \cdot (R/R_{\max})$ ممکن است در لینک‌های با نرخ کلید بالا، هزینه کل را منفی کند که در الگوریتم Dijkstra باعث رفتار غیرتعریف‌شده می‌شود.

۵.۳.۱۲ کلاس OpticalSwitch

کلاس OpticalSwitch یک سویچ نوری را مدل می‌کند که امکان پیکربندی پویای توپولوژی شبکه را فراهم می‌سازد. این کلاس بسیار ساده است و تنها شامل یک نگاشت از پورت ورودی به پورت خروجی و یک پرچم available است.

```

۱     class OpticalSwitch:
۲     def __init__(self, switch_id: str, switch_type: str) -> None:
۳         self.switch_id: str = switch_id
۴         self.switch_type: str = switch_type
۵         self.configuration: Dict[str, str] = {}
۶         self.available: bool = True
۷
۸     def configure(self, mapping: Dict[str, str]) -> None:
۹         self.configuration = dict(mapping)
۱۰
۱۱     def get_connection(self, input_port: str) -> str:
۱۲         return self.configuration.get(input_port, "")
۱۳

```

در پیاده‌سازی واقعی، سه سویچ ایجاد می‌شود: SW-A-1x2 در گره A با نوع 1x2 که به این گره اجازه می‌دهد به یکی از دو رله B یا C متصل شود، SW-B-2x4 در گره B با نوع 2x4 برای اتصال به چهار ارسال‌کننده، و SW-C-2x4 در گره C با همان نوع. این سویچ‌ها در زمان اجرای هر درخواست کلید خدمت توسط روش `configure_switches_for_path` در KMS پیکربندی می‌شوند تا مسیر انتخاب‌شده توسط SPF+ در دسترس باشد.

۶.۳.۱۲ سلسله‌مراتب کلاس‌های گره

گروه کلاس‌های گره از یک کلاس پایه QKDNode مشتق می‌شوند که سه زیرکلاس تخصصی دارد: `TrustedRelayNode`، `ReceiverNode` و `SenderNode`. این طراحی از الگوی Role-Template Method Based Specialization پیروی می‌کند که در آن هر نقش رفتار خاص خود را دارد ولی زیرساخت مشترک (مانند مدیریت استخر کلید و فهرست همسایه‌ها) در کلاس پایه قرار می‌گیرد.

```

۱     class QKDNode:

```

```

۲     def __init__(self, node_id: str, role: NodeRole) -> None:
۳         self.node_id: str = node_id
۴         self.role: NodeRole = role
۵         self.key_pools: Dict[str, KeyPool] = {}
۶         self.neighbors: List[str] = []
۷
۸     def get_key_pool(self, peer_id: str) -> Optional[KeyPool]:
۹         return self.key_pools.get(f"{self.node_id}-{peer_id}")
۱۰
۱۱     def add_neighbor(self, peer_id: str) -> None:
۱۲         if peer_id not in self.neighbors:
۱۳             self.neighbors.append(peer_id)
۱۴
۱۵
۱۶     class TrustedRelayNode(QKDNode):
۱۷     def __init__(self, node_id: str) -> None:
۱۸         super().__init__(node_id, NodeRole.RELAY)
۱۹
۲۰     def relay_key(
۲۱     self,
۲۲     incoming_key_material: bytes,
۲۳     from_node: str,
۲۴     to_node: str,
۲۵     hop_keys: Dict[str, bytes],
۲۶     ) -> bytes:
۲۷         key_from = hop_keys[f"{from_node}-{self.node_id}"]
۲۸         decrypted = bytes(a ^ b for a, b in zip(incoming_key_material, key_from))
۲۹
۳۰         key_to = hop_keys[f"{self.node_id}-{to_node}"]
۳۱         encrypted = bytes(a ^ b for a, b in zip(decrypted, key_to))
۳۲
۳۳         return encrypted
۳۴

```

روش relay_key در TrustedRelayNode پیاده‌سازی پروتکل XOR را به صورت دو مرحله‌ای انجام می‌دهد: ابتدا رمزگشایی با کلید جهش ورودی، سپس رمزگذاری با کلید جهش خروجی. استفاده از zip با generalization tor expression به صورت $\text{bytes}(a \oplus b \text{ for } a, b \text{ in zip}(\dots))$ بسیار کارآمد است و از لیست میانی اجتناب می‌کند. در مواردی که طول کلیدها نابرابر باشد، zip در کوتاه‌ترین طول متوقف می‌شود که یک رفتار امن است (هرگز سرریز نمی‌کند).

۷.۳.۱۲ شبیه‌سازهای QKD

دو کلاس شبیه‌ساز در این مازول پیاده‌سازی شده‌اند که هر دو رابط یکسانی (simulate_link) را ارائه می‌دهند اما با پیاده‌سازی متفاوت. StatisticalQKDLINKSimulator یک مدل تحلیلی فاصله‌محور است که فرمول‌های استاندارد BB84 decoy-state را در رژیم مجانبی استفاده می‌کند. PipelineQKDLINKSimulator به موتور فیزیکی کامل main_optimized.py وابسته است و خط لوله کامل را با تمام اصلاحات F-01 تا F-22 اجرا می‌کند.

```

۱     class StatisticalQKDLINKSimulator:
۲     def __init__(
۳     self,
۴     det_eff: float = 15.0,
۵     dark_rate: float = 1e-6,
۶     fiber_loss: float = 2.0,
۷     intrinsic_qber: float = 0.15,
۸     mu_signal: float = 5.0,
۹     ) -> None:
۱۰         self.det_eff = det_eff
۱۱         self.dark_rate = dark_rate
۱۲         self.fiber_loss = fiber_loss
۱۳         self.intrinsic_qber = intrinsic_qber
۱۴         self.mu_signal = mu_signal
۱۵
۱۶     @staticmethod
۱۷     def _binary_entropy(x: float) -> float:
۱۸         if x <= 0.0 or x >= 0.1:
۱۹             return 0.0
۲۰         return -x * math.log2(x) - (0.1 - x) * math.log2(0.1 - x)

```


مقادیر پیش‌فرض سازنده StatisticalQKDLINKSimulator به‌دقت با DEFAULT_CONFIG در main_optimized.py هم‌خوان است: $\det_eff=0.15$ متناظر با بازده آشکارساز D0 و D1 در خط لوله، $dark_rate=1e-6$ متناظر با نرخ دارک کانت، $intrinsic_qber=0.015$ شامل 1% خطای ذاتی و 0.5% خطای misalignment، و $\mu_signal=0.5$ متناظر با شدت پالس سیگنال. این هم‌خوانی تضمین می‌کند که در صورت استفاده از fallback آماری، نرخ کلید تولیدشده در همان مرتبه بزرگی با خروجی خط لوله باشد. روش `_binary_entropy` یک متد static است که به‌صورت مستقل از نمونه کلاس عمل می‌کند. این روش برای محاسبه آنترپی باینری استفاده می‌شود و دو شرط مرزی برای جلوگیری از $\log(0)$ دارد: اگر $x \leq 0$ یا $x \geq 1$ مقدار 0 برمی‌گردد. این رفتار با تعریف حد آنترپی باینری هم‌خوان است: $\lim_{x \rightarrow 0} x \log_2(x) = 0$. کلاس PipelineQKDLINKSimulator بسیار پیچیده‌تر است و شامل منطق بارگذاری پویای ماژول، پیکربندی موتور فیزیکی، و اعتبارسنجی خروجی است. در سازنده آن، ابتدا یک شبیه‌ساز آماری به‌عنوان fallback ایجاد می‌شود با همان بازده آشکارساز خط لوله؛ این تضمین می‌کند که اگر خط لوله در یک لینک به‌خصوص شکست بخورد، fallback نرخ کلیدی در همان مرتبه بزرگی تولید کند (در غیر این صورت، fallback با $\det_eff=0.65$ نرخ‌هایی 20 برابر بزرگ‌تر از خط لوله تولید می‌کند).

۸.۳.۱۲ ارکستراتور KMS و کلاس QKDNetwork7Nodes

کلاس KMS (Key Management System) ارکستراتور مرکزی ماژول است و مسئول ساخت توپولوژی، اجرای نشست‌های QKD، انتخاب مسیر SPF+، اجرای رله قابل‌اعتماد، و مدیریت چرخه حیات استخرها است. این کلاس وضعیت کلی شبکه را در سه دیکشنری `nodes`، `links` و `switches` نگه می‌دارد و شامل روش‌هایی برای هر عملیات کلیدی است. کلاس QKDNetwork7Nodes یک لایه بالاتر است که چرخه حیات کامل demo را مدیریت می‌کند و خروجی قالب‌بندی‌شده برای کاربر تولید می‌نماید.

۴.۱۲ پیاده‌سازی ریاضی و الگوریتمی

۱.۴.۱۲ بارگذاری پویای موتور فیزیکی

یکی از جنبه‌های مهم پیاده‌سازی، بارگذاری پویای ماژول main_optimized.py است که با استفاده از `importlib.util.spec_from_file_location` انجام می‌شود. این روش به ماژول شبکه اجازه می‌دهد بدون توجه به مسیر کاری فعلی، نسخه محلی موتور فیزیکی را بارگذاری کند. این طراحی در برابر تغییر مسیر کاری (cwd) مقاوم است و از بارگذاری اشتباه نسخه‌ای از ماژول از مسیر `site-packages` یا دایرکتوری والد جلوگیری می‌کند.

```

1     _PIPELINE_MODULE_NAME = "_qkd_network_main_optimized"
2
3
4     def _load_local_pipeline_module():
5         framework_dir = Path(__file__).resolve().parent
6         module_path = framework_dir / "main_optimized.py"
7         if not module_path.is_file():
8             raise ImportError(f"Pipeline engine not found at {module_path}")
9
10        framework_dir_str = str(framework_dir)
11        if not sys.path or sys.path[0] != framework_dir_str:
12            sys.path.insert(0, framework_dir_str)
13
14        cached_module = sys.modules.get(_PIPELINE_MODULE_NAME)
15        if cached_module is not None:
16            cached_path = getattr(cached_module, "__file__", None)
17            if cached_path and Path(cached_path).resolve() == module_path:
18                return cached_module
19        sys.modules.pop(_PIPELINE_MODULE_NAME, None)
20
21        spec = importlib.util.spec_from_file_location(_PIPELINE_MODULE_NAME, module_path)
22        if spec is None or spec.loader is None:
23            raise ImportError(f"Could not create an import spec for {module_path}")
24
25        module = importlib.util.module_from_spec(spec)
26        sys.modules[_PIPELINE_MODULE_NAME] = module

```



```

۲۷     try:
۲۸         spec.loader.exec_module(module)
۲۹     except Exception:
۳۰         sys.modules.pop(_PIPELINE_MODULE_NAME, None)
۳۱         raise
۳۲
۳۳         loaded_path = Path(module.__file__).resolve()
۳۴         if loaded_path != module_path:
۳۵             raise ImportError(
۳۶                 f"Loaded pipeline engine from {loaded_path}, expected {module_path}"
۳۷             )
۳۸         return module
۳۹

```

این تابع از چندین تکنیک دفاعی استفاده می‌کند: ابتدا مسیر مازول را از `__file__` استخراج می‌کند که به مسیر کاری فعلی وابسته نیست؛ سپس مسیر دایرکتوری را به ابتدای `sys.path` اضافه می‌کند تا وابستگی‌های موتور فیزیکی (مانند بسته `qkd`) نیز قابل بارگذاری باشند؛ در ادامه، اگر مازول قبلاً بارگذاری شده باشد و مسیر آن تغییر نکرده باشد، از نسخه `cached` استفاده می‌کند؛ در غیر این صورت، نسخه قبلی را از `sys.modules` حذف و نسخه جدید را بارگذاری می‌نماید. در نهایت، یک بررسی امنیتی انجام می‌شود که مطمئن می‌شود مازول بارگذاری شده واقعاً از مسیر مورد انتظار است؛ این بررسی از حملات `path traversal` و بارگذاری اشتباه جلوگیری می‌کند.

۲.۴.۱۲ محاسبه نرخ کلید در شبیه‌ساز آماری

روش `simulate_link` در `StatisticalQKDLinkSimulator` مدل کامل QBER و نرخ کلید را برای یک لینک پیاده‌سازی می‌کند. این روش با دریافت شیء `QKDLink` و تعداد پالس‌ها، سه خروجی تولید می‌کند: تعداد بیت‌های کلید امن، مقدار QBER و نرخ کلید بر پالس.

```

۱     def simulate_link(
۲         self, link: QKDLink, num_pulses: int = 10 ** 7
۳     ) -> Tuple[int, float, float]:
۴         d = link.distance_km
۵         alpha = self.fiber_loss
۶         eta = self.det_eff
۷         mu = self.mu_signal
۸         q = 5.0 # sifting factor
۹
۱۰        # Channel transmittance
۱۱        transmittance = eta * 0.10 ** (-alpha * d / (0.10
۱۲
۱۳        # Gain (detection probability per pulse) -- linear approximation
۱۴        gain_approx = mu * transmittance + self.dark_rate
۱۵        gain = max(gain_approx, 1e-30)
۱۶
۱۷        # Distance-dependent QBER model
۱۸        qber_intrinsic = self.intrinsic_qber
۱۹        qber_polarization = 0.001 * d
۲۰        qber_dark = self.dark_rate / gain
۲۱        qber_afterpulse = 0.001
۲۲
۲۳        qber = qber_intrinsic + qber_polarization + qber_dark + qber_afterpulse
۲۴        qber = min(qber, 0.1)
۲۵
۲۶        # If QBER exceeds threshold, no secure key is possible
۲۷        if qber > QBER_THRESHOLD:
۲۸            return 0, qber, 0.0
۲۹
۳۰        # Secret fraction (using asymptotic decoy-state analysis)
۳۱        h2_qber = self._binary_entropy(qber)
۳۲        secret_fraction = max(0.0, 0.1 - 0.2 * h2_qber)
۳۳
۳۴        # Key rate per pulse
۳۵        key_rate = q * transmittance * mu * secret_fraction
۳۶
۳۷        # Total secure key bits
۳۸        secure_key_bits = int(key_rate * num_pulses)
۳۹
۴۰        return secure_key_bits, qber, key_rate

```

تحلیل خطبه خط این روش به شرح زیر است: ابتدا متغیرهای فیزیکی از شیء لینک و پیکربندی شبیه ساز استخراج می شوند. سپس ضریب انتقال کلی $\eta_{\text{total}} = \eta \cdot 10^{-\alpha d/10}$ محاسبه می شود. در گام بعدی، gain (احتمال شناسایی به ازای هر پالس) با تقریب خطی $g \approx \mu \cdot \eta_{\text{total}} + R_{\text{dark}}$ محاسبه می شود؛ این تقریب برای $1 \ll \mu$ و $\eta_{\text{total}} \ll 1$ بسیار دقیق است. برای جلوگیری از تقسیم بر صفر، gain با کف 10^{-30} محدود می شود. چهار مؤلفه QBER سپس محاسبه و جمع می شوند. اگر QBER کل از آستانه 0.11 تجاوز کند، روش به سرعت با خروجی صفر برمی گردد چرا که تولید کلید امن از نظر تئوری غیرممکن است. در غیر این صورت، آنتروپی باینری $h_2(Q)$ محاسبه و کسر امن $f_{\text{secret}} = \max(0, 1 - 2h_2(Q))$ به دست می آید. نرخ کلید بر پالس از $R = q \cdot \eta_{\text{total}} \cdot \mu \cdot f_{\text{secret}}$ محاسبه و تعداد کل بیت های امن با ضرب در تعداد پالس ها به دست می آید.

۳.۴.۱۲ اجراهای موتور خط لوله

روش `_run_pipeline` در `PipelineQKDLinkSimulator` موتور فیزیکی کامل را فراخوانی می کند. این روش با ساخت پیکربندی، مقداردهی اولیه کارگر، تولید بذر تصادفی، و فراخوانی `run_single_simulation` کار می کند.

```

1 def _run_pipeline(
2     self, link: QKDLink, num_pulses: int
3 ) -> Tuple[int, float, float]:
4     m = self._pipeline_modules
5     np = m["np"]
6     dist = link.distance_km
7
8     if type(num_pulses) is not int or num_pulses <= 0:
9         raise ValueError("num_pulses must be a positive integer")
10
11     cfg = self._make_pipeline_config()
12
13     # Initialise the worker state for this config.
14     # combo_map {0: {}} means no sweep overrides.
15     m["init_worker"](cfg, {0: {}})
16
17     # Derive a high-entropy seed for this session.
18     # run_single_simulation uses SeedSequence.spawn(5) internally (F-05).
19     rng_seed = int(np.random.default_rng().integers(0, 2**63))
20
21     # Run the complete pipeline via main_optimized.
22     # args_tuple: (distance_km, total_pulses, apply_noise, seed, combo_idx)
23     result = m["run_single_simulation"](
24         (dist, num_pulses, self.apply_noise, rng_seed, 0)
25     )
26
27     secure_key = result["secure_key_bits"]
28     qber = result["qber"]
29     key_rate = result["secure_key_rate"]
30
31     # Store full result for callers that want richer diagnostics
32     self._last_result = result
33
34     return secure_key, qber, key_rate
35

```

در این روش، ابتدا پیکربندی با استفاده از `_make_pipeline_config` ساخته می شود که یک کپی عمیق از `DEFAULT_CONFIG` گرفته و بازده آشکارساز `D0` و `D1` را با مقدار نمونه تنظیم می کند. سپس `init_worker` با یک `combo_map` خالی فراخوانی می شود که به معنای عدم اعمال sweep است. بذر تصادفی با استفاده از `numpy.random.default_rng().integers(0, 2**63)` تولید می شود که یک بذر 63 بیتی با آنتروپی بالا فراهم می سازد؛ این بذر سپس به `run_single_simulation` ارجاع داده می شود که در داخل خود از `SeedSequence.spawn(5)` (اصلاح F-05) برای تولید جریان های مستقل RNG استفاده می کند. خروجی شامل سه مقدار کلیدی است که از دیکشنری نتیجه استخراج می شوند؛ همچنین کل دیکشنری برای اهداف تشخیصی در `_last_result` ذخیره می شود.

روش `simulate_link` در `PipelineQKDLinkSimulator` دارای منطق اعتبارسنجی و fallback پیچیده ای است. اگر موتور خط لوله QBER بالاتر از آستانه یا نرخ کلید صفر برگرداند، به صورت خودکار به شبیه ساز آماری fallback می کند. این رفتار به این دلیل ضروری است که گاهی موتور فیزیکی به دلیل پیکربندی نامناسب یا

شرایط لبه‌ای عددی، نتایج غیرواقعی تولید می‌کند که در آن صورت، مدل آماری اعتبارسنجی‌شده نتایج قابل اتکاتر ارائه می‌دهد.

```

1         if qber > QBER_THRESHOLD or (key_rate == 0.0 and secure_bits == 0):
2             if not hasattr(self, '_fallback_warned'):
3                 self._fallback_warned: set = set()
4                 if link.link_id not in self._fallback_warned:
5                     self._fallback_warned.add(link.link_id)
6             import logging
7             logging.getLogger(__name__).warning(
8                 "Pipeline returned unrealistic QBER4.=%f / key_rate2.=%e for "
9                 "link %s 0.%)f km). Falling back to statistical model.",
10                qber, key_rate, link.link_id, link.distance_km,
11                )
12            return self.fallback.simulate_link(link, num_pulses)
13

```

برای جلوگیری از هجوم هشدارهای تکراری، یک مجموعه `_fallback_warned` نگه داشته می‌شود که شناسه لینک‌های قبلاً هشدار داده‌شده را در خود ذخیره می‌کند. این الگو (lazy attribute) به این صورت کار می‌کند که ویژگی تنها در اولین فراخوانی ایجاد می‌شود و در فراخوانی‌های بعدی مجدداً استفاده می‌گردد.

۴.۴.۱۲ الگوریتم SPF+ و پیاده‌سازی Dijkstra

روش `spf_plus` در KMS الگوریتم Dijkstra را با تابع هزینه ترکیبی `SPF+` پیاده‌سازی می‌کند. این روش از `heapq` به‌عنوان صف اولویت استفاده می‌کند و در هر تکرار، گره با کمترین هزینه را از صف خارج و همسایه‌های آن را بررسی می‌کند.

```

1     def spf_plus(self, source: str, destination: str) -> List[str]:
2         dist: Dict[str, float] = {nid: INFINITY_COST for nid in self.nodes}
3         prev: Dict[str, Optional[str]] = {nid: None for nid in self.nodes}
4         dist[source] = 0.0
5         visited: set = set()
6         heap: List[Tuple[float, str]] = [(0.0), source]
7
8         while heap:
9             current_dist, u = heapq.heappop(heap)
10            if u in visited:
11                continue
12            visited.add(u)
13            if u == destination:
14                break
15
16            for v in self.nodes[u].neighbors:
17                if v in visited:
18                    continue
19
20            link_id = self._get_link_id(u, v)
21            link = self.links.get(link_id)
22            if link is None:
23                continue
24
25            pool_u_v = self.nodes[u].key_pools.get(f"{u}-{v}")
26            pool_v_u = self.nodes[v].key_pools.get(f"{v}-{u}")
27
28            pressures = []
29            for pool in (pool_u_v, pool_v_u):
30                if pool is not None:
31                    pressures.append(pool.calculate_pressure())
32                else:
33                    pressures.append(0.1)
34            pool_pressure = max(pressures)
35
36            if (pool_u_v and pool_u_v.status in (KeyPoolStatus.ERROR, KeyPoolStatus.DISABLED)) or \
37                (pool_v_u and pool_v_u.status in (KeyPoolStatus.ERROR, KeyPoolStatus.DISABLED)):
38                edge_cost = INFINITY_COST
39            else:
40                cost_params = {
41                    "alpha": self.spf_weights["alpha"],
42                    "beta": self.spf_weights["beta"],

```

```

۴۳         "gamma": self.spf_weights["gamma"],
۴۴         "delta": self.spf_weights["delta"],
۴۵         "epsilon": self.spf_weights["epsilon"],
۴۶         "pool_pressure": pool_pressure,
۴۷         "key_rate": link.secret_key_rate,
۴۸     }
۴۹     edge_cost = link.get_cost(cost_params)
۵۰     new_dist = current_dist + edge_cost
۵۱
۵۲     if new_dist < dist[v]:
۵۳         dist[v] = new_dist
۵۴         prev[v] = u
۵۵         heapq.heappush(heap, (new_dist, v))
۵۶
۵۷     if dist[destination] >= INFINITY_COST:
۵۸         return []
۵۹
۶۰     path: List[str] = []
۶۱     current: Optional[str] = destination
۶۲     while current is not None:
۶۳         path.append(current)
۶۴         current = prev[current]
۶۵     path.reverse()
۶۶     return path
۶۷

```

تحلیل این الگوریتم به شرح زیر است: ابتدا دو دیکشنری `dist` و `prev` با مقادیر اولیه ∞ و `None` مقداردهی می‌شوند؛ تنها فاصله گره مبدأ به صفر تنظیم می‌گردد. در حلقه اصلی، گره با کمترین فاصله از `heap` خارج می‌شود؛ اگر قبلاً بازدید شده باشد، نادیده گرفته می‌شود (این ممکن است رخ دهد چون `heap` ممکن است چندین ورودی برای یک گره داشته باشد). پس از بازدید، اگر گره مقصد باشد، حلقه پایان می‌یابد. برای هر همسایه v از گره فعلی u ، استخر کلید هر دو جهت بررسی می‌شود و فشار کل به‌عنوان حداکثر فشار دو استخر محاسبه می‌شود. اگر هر یک از دو استخر در وضعیت `ERROR` یا `DISABLED` باشد، هزینه یال به ∞ تنظیم می‌شود که در عمل معادل حذف آن یال است. در غیر این صورت، تابع `get_cost` با پارامترهای هزینه فراخوانی می‌شود. اگر فاصله جدید کمتر از فاصله ثبت‌شده باشد، به‌روزرسانی انجام و ورودی جدید به `heap` افزوده می‌شود.

در پایان، بازسازی مسیر از گره مقصد به مبدأ با پیروی از زنجیره `prev` انجام می‌شود و سپس مسیر معکوس می‌گردد تا از مبدأ به مقصد باشد. اگر فاصله نهایی مقصد همچنان ∞ باشد، یعنی هیچ مسیری وجود ندارد و لیست خالی برگردانده می‌شود.

۵.۴.۱۲ درخواست کلید خدمت و رله قابل‌اعتماد

روش `request_service_key` در `KMS` فرآیند کامل تولید و توزیع کلید خدمت را مدیریت می‌کند. این روش شامل شش گام است: یافتن مسیر با `SPF+`، رزرو کلیدهای جهش در هر هاپ، تولید ماده کلید خدمت، اجرای رله قابل‌اعتماد، علامت‌گذاری کلیدهای مصرف‌شده، و ثبت کلید خدمت در رجیستری.

```

۱     def request_service_key(
۲         self, sender: str, receiver: str, key_length_bits: int = 256
۳     ) -> dict:
۴         # Step 1: Find path
۵         path = self.spf_plus(sender, receiver)
۶         if not path:
۷             return {
۸                 "status": "error",
۹                 "message": f"No available path from {sender} to {receiver}",
۱۰                ...
۱۱            }
۱۲
۱۳         # Step 2: Reserve hop keys
۱۴         hop_reservations: List[Dict[str, Any]] = []
۱۵         key_bytes = key_length_bits // 8
۱۶
۱۷         for i in range(len(path) - 1):
۱۸             u, v = path[i], path[i + 1]
۱۹             pool_u = self.nodes[u].key_pools.get(f"{u}-{v}")
۲۰             pool_v = self.nodes[v].key_pools.get(f"{v}-{u}")
۲۱

```

```

۲۲     if pool_u is None or pool_v is None:
۲۳         self._rollback_reservations(hop_reservations)
۲۴         return {
۲۵             "status": "error",
۲۶             "message": f"Missing key pool for hop {u}-{v}",
۲۷             ...
۲۸         }
۲۹
۳۰     blocks_u = pool_u.reserve_keys(key_length_bits)
۳۱     blocks_v = pool_v.reserve_keys(key_length_bits)
۳۲
۳۳     if not blocks_u or not blocks_v:
۳۴         self._rollback_reservations(hop_reservations)
۳۵         for b in blocks_u:
۳۶             b.state = KeyBlockState.READY
۳۷             pool_u.available_key_bits += b.length_bits
۳۸             pool_u.reserved_key_bits -= b.length_bits
۳۹         for b in blocks_v:
۴۰             b.state = KeyBlockState.READY
۴۱             pool_v.available_key_bits += b.length_bits
۴۲             pool_v.reserved_key_bits -= b.length_bits
۴۳         return {
۴۴             "status": "error",
۴۵             "message": f"Insufficient key material in pool for hop {u}-{v}",
۴۶             ...
۴۷         }
۴۸
۴۹     hop_reservations.append({
۵۰         "u": u, "v": v,
۵۱         "blocks_u": blocks_u, "blocks_v": blocks_v,
۵۲         "pool_u": pool_u, "pool_v": pool_v,
۵۳     })
۵۴
۵۵     # Step 3: Generate service key material
۵۶     service_key_material = os.urandom(key_bytes)
۵۷
۵۸     # Step 4: Perform trusted relay
۵۹     relay_success = self.perform_trusted_relay(path, service_key_material)
۶۰
۶۱     # Step 5: Mark hop keys as USED
۶۲     all_used_key_ids: List[str] = []
۶۳     for hop in hop_reservations:
۶۴         ids_u = [b.key_id for b in hop["blocks_u"]]
۶۵         ids_v = [b.key_id for b in hop["blocks_v"]]
۶۶         hop["pool_u"].mark_used(ids_u)
۶۷         hop["pool_v"].mark_used(ids_v)
۶۸         all_used_key_ids.extend(ids_u)
۶۹         all_used_key_ids.extend(ids_v)
۷۰
۷۱     # Step 6: Register service key
۷۲     self._svc_key_counter += 1
۷۳     service_key_id = f"SVC-{sender}{receiver}-{self._svc_key_counter:06d}"
۷۴     self.service_key_registry[service_key_id] = {
۷۵         "service_key_id": service_key_id,
۷۶         "sender": sender,
۷۷         "receiver": receiver,
۷۸         "key_length_bits": key_length_bits,
۷۹         "path": path,
۸۰         "hop_keys_used": all_used_key_ids,
۸۱         "created_at": time.time(),
۸۲         "relay_success": relay_success,
۸۳     }
۸۴     return {"status": "ok", ...}
۸۵

```

نکته کلیدی در این روش، پیاده‌سازی صحیح rollback در صورت شکست است. اگر در هر مرحله از رزرو، یک هاپ به اندازه کافی کلید نداشته باشد، تمام رزروهای قبلی باید به حالت READY بازگردانده شوند تا استخرها در وضعیت سازگار باقی بمانند. این الگو به عنوان compensating transaction در سیستم‌های توزیع‌شده شناخته می‌شود و تضمین می‌کند که حتی در صورت شکست، سیستم در وضعیت سازگار باقی بماند.

روش `perform_trusted_relay` پروتکل XOR را در طول مسیر اجرا می‌کند. این روش با رمزگذاری اولیه در فرستنده آغاز می‌شود، سپس در هر رله میانی، رمزگشایی و رمزگذاری مجدد انجام می‌شود، و در نهایت در

گیرنده، رمزگشایی نهایی انجام می‌گردد. یک ویژگی جالب این روش آن است که در پایان، صحت رله را با مقایسه ماده نهایی با ماده اولیه اعتبارسنجی می‌کند:

```

1     def perform_trusted_relay(
2         self, path: List[str], service_key_material: bytes
3     ) -> bool:
4         if len(path) < 2:
5             return False
6
7         current_material = service_key_material
8
9         # Sender encrypts with first hop key
10        first_hop_u, first_hop_v = path[0], path[1]
11        pool_u = self.nodes[first_hop_u].key_pools.get(f"{first_hop_u}-{first_hop_v}")
12        if pool_u is None:
13            return False
14
15        hop_key_send = self._get_reserved_key_material(pool_u)
16        if hop_key_send is None:
17            return False
18
19        # Step 1: Sender encrypts
20        current_material = bytes(a ^ b for a, b in zip(current_material, hop_key_send))
21
22        # Intermediate relay nodes
23        for i in range(1, len(path) - 1):
24            relay_node = self.nodes[path[i]]
25            from_node = path[i - 1]
26            to_node = path[i + 1]
27
28            if not isinstance(relay_node, TrustedRelayNode):
29                return False
30
31            pool_in = relay_node.key_pools.get(f"{relay_node.node_id}-{from_node}")
32            if pool_in is None:
33                pool_in = relay_node.key_pools.get(f"{from_node}-{relay_node.node_id}")
34            pool_out = relay_node.key_pools.get(f"{relay_node.node_id}-{to_node}")
35
36            if pool_in is None or pool_out is None:
37                return False
38
39            key_in = self._get_reserved_key_material(pool_in)
40            key_out = self._get_reserved_key_material(pool_out)
41
42            if key_in is None or key_out is None:
43                return False
44
45            hop_keys = {
46                f"{from_node}-{relay_node.node_id}": key_in,
47                f"{relay_node.node_id}-{to_node}": key_out,
48            }
49
50            current_material = relay_node.relay_key(
51                current_material, from_node, to_node, hop_keys
52            )
53
54            # Final receiver decrypts with last hop key
55            last_hop_u, last_hop_v = path[-2], path[-1]
56            pool_recv = self.nodes[last_hop_v].key_pools.get(f"{last_hop_v}-{last_hop_u}")
57            if pool_recv is None:
58                return False
59
60            hop_key_recv = self._get_reserved_key_material(pool_recv)
61            if hop_key_recv is None:
62                return False
63
64            final_material = bytes(a ^ b for a, b in zip(current_material, hop_key_recv))
65
66            # Verify relay correctness (final material should match original)
67            return final_material == service_key_material
68

```

اعتبارسنجی نهایی `return final_material == service_key_material` یک تضمین سلامت (integrity check) است؛ اگر به هر دلیلی (مثلاً عدم تطابق طول کلیدها یا خطای منطقی در رله) ماده نهایی با ماده اولیه تطابق نداشته باشد، روش مقدار `False` برمی‌گرداند و در نتیجه در رجیستری کلید خدمت، `relay_success=False` ثبت می‌شود. این رفتار به کاربر اجازه می‌دهد شکست‌ها را تشخیص دهد.

۶.۴.۱۲ مدیریت نشست QKD و پر کردن استخر

روش `run_qkd_session` در `KMS` یک نشست QKD را روی یک لینک اجرا می‌کند و کلیدهای تولیدشده را در استخرهای هر دو گره انتهای لینک توزیع می‌کند. این روش با تقسیم کلید تولیدشده به بلوک‌های 256 بیتی، بلوک‌های متناظر در هر دو گره با ماده کلیدی یکسان می‌سازد.

```

1  def run_qkd_session(
2      self, link_id: str, num_pulses: int = 10 ** 7
3  ) -> Tuple[int, float, float]:
4      link = self.links.get(link_id)
5      if link is None:
6          raise ValueError(f"Unknown link: {link_id}")
7      if link.status == LinkStatus.DOWN:
8          raise RuntimeError(f"Link {link_id} is DOWN")
9
10     secure_bits, qber, key_rate = self.simulator.simulate_link(link, num_pulses)
11
12     link.qber = qber
13     link.secret_key_rate = key_rate
14     session_id = f"QKD-{link_id}-{uuid.uuid4().hex[:8]}"
15     link.last_qkd_session_id = session_id
16
17     if secure_bits <= 0:
18         return secure_bits, qber, key_rate
19
20     block_size_bits = 256
21     num_blocks = secure_bits // block_size_bits
22     remaining_bits = secure_bits % block_size_bits
23
24     node_a, node_b = link.node_a, link.node_b
25
26     for i in range(num_blocks):
27         self._key_counter += 1
28         kid = f"{node_a}-{node_b}-{self._key_counter:06d}"
29
30         block_a = KeyBlock(
31             key_id=kid,
32             link_id=link_id,
33             local_node=node_a,
34             peer_node=node_b,
35             key_material=os.urandom(block_size_bits // 8),
36             length_bits=block_size_bits,
37             state=KeyBlockState.READY,
38             created_at=time.time(),
39             expires_at=time.time() + 0.3600,
40             qkd_session_id=session_id,
41             qber=qber,
42             secret_key_rate=key_rate,
43         )
44
45         self._key_counter += 1
46         kid_b = f"{node_b}-{node_a}-{self._key_counter:06d}"
47
48         block_b = KeyBlock(
49             key_id=kid_b,
50             ...
51             key_material=block_a.key_material, # Same key at both ends
52             ...
53         )
54
55         pool_a = self.nodes[node_a].key_pools.get(f"{node_a}-{node_b}")
56         pool_b = self.nodes[node_b].key_pools.get(f"{node_b}-{node_a}")
57         if pool_a:
58             pool_a.add_key_block(block_a)
59         if pool_b:

```



```

۶۰     pool_b.add_key_block(block_b)
۶۱
۶۲     return secure_bits, qber, key_rate
۶۳

```

نکته مهم در این پیاده‌سازی آن است که ماده کلیدی بلوک block_b با block_a.key_material به مقداردهی می‌شود، نه با یک فراخوانی جدید os.urandom. این تضمین می‌کند که کلید در هر دو گره یکسان است که برای رمزنگاری XOR ضروری است. در عمل، در یک سیستم QKD واقعی، این کلید از طریق پروتکل QKD به صورت مستقل در هر دو گره تولید و سپس از طریق کانال احراز هویت عمومی تطبیق داده می‌شود؛ در این شبیه‌سازی، به جای شبیه‌سازی کامل پروتکل تطبیق، مستقیماً کلید یکسان در هر دو گره قرار داده می‌شود. روش replenish_key_pools استخرها را اسکن می‌کند و برای استخرهای در وضعیت LOW یا EMPTY، نشست QKD جدیدی اجرا می‌کند. نکته طراحی مهم در این روش آن است که هر دو استخر متناظر در یک لینک به صورت همزمان به وضعیت REPLENISHING تغییر می‌یابند تا وضعیت همگام حفظ شود.

```

۱     def replenish_key_pools(self) -> List[str]:
۲         replenished: List[str] = []
۳         visited_links: set = set()
۴
۵         for node_id, node in self.nodes.items():
۶             for pool_key, pool in node.key_pools.items():
۷                 if pool.status in (KeyPoolStatus.LOW, KeyPoolStatus.EMPTY):
۸                     link_id = pool.link_id
۹
۱0                    if link_id in visited_links:
۱۱                        continue
۱۲                    visited_links.add(link_id)
۱۳
۱۴                    link = self.links.get(link_id)
۱۵                    if link is None:
۱۶                        continue
۱۷                    pool_a = self.nodes[link.node_a].key_pools.get(
۱۸                        f"{link.node_a}-{link.node_b}"
۱۹                    )
۲۰                    pool_b = self.nodes[link.node_b].key_pools.get(
۲۱                        f"{link.node_b}-{link.node_a}"
۲۲                    )
۲۳                    if pool_a:
۲۴                        pool_a.status = KeyPoolStatus.REPLENISHING
۲۵                    if pool_b:
۲۶                        pool_b.status = KeyPoolStatus.REPLENISHING
۲۷
۲۸                    try:
۲۹                        replenish_pulses = 10 ** 7
۳۰                        self.run_qkd_session(link_id, num_pulses=replenish_pulses)
۳۱                        replenished.append(link_id)
۳۲                    except Exception as e:
۳۳                        if pool_a:
۳۴                            pool_a.status = KeyPoolStatus.ERROR
۳۵                        if pool_b:
۳۶                            pool_b.status = KeyPoolStatus.ERROR
۳۷                        print(f" [ERROR] Replenishment failed for {link_id}: {e}")
۳۸
۳۹                    return replenished
۴۰

```

استفاده از visited_links برای جلوگیری از double-replenishment همان لینک است؛ چون هر لینک دو استخر دارد، اسکن بدون این محافظ، همان لینک را دو بار شناسایی و پردازش می‌کرد. در صورت شکست نشست QKD، هر دو استخر به وضعیت ERROR تغییر می‌یابند که چسبناک است و تنها با مداخله خارجی قابل پاک‌سازی است.

۷.۴.۱۲ کالیبراسیون برای نوع شبیه‌ساز

روش calibrate_for_simulator در KMS یکی از مهم‌ترین روش‌های تنظیم پویا است. این روش با دریافت یک پرچم بولی watermark as_pipeline ثابت نرمال‌سازی SPF+ را برای نوع شبیه‌ساز تنظیم می‌کند. این کالیبراسیون ضروری است چرا که خروجی دو شبیه‌ساز در مقیاس‌های کاملاً متفاوتی قرار دارد.


```

1 def calibrate_for_simulator(self, is_pipeline: bool) -> None:
2     self._is_pipeline_mode = is_pipeline
3
4     if is_pipeline:
5         new_low = PIPELINE_LOW_WATERMARK_BITS
6         new_high = PIPELINE_HIGH_WATERMARK_BITS
7         new_max = PIPELINE_MAX_CAPACITY_BITS
8         new_max_rate = PIPELINE_SPF_MAX_KEY_RATE
9     else:
10        new_low = STAT_LOW_WATERMARK_BITS
11        new_high = STAT_HIGH_WATERMARK_BITS
12        new_max = STAT_MAX_CAPACITY_BITS
13        new_max_rate = STAT_SPF_MAX_KEY_RATE
14
15        # Update all pools
16        for node_id, node in self.nodes.items():
17            for pool_key, pool in node.key_pools.items():
18                pool.low_watermark_bits = new_low
19                pool.high_watermark_bits = new_high
20                pool.max_capacity_bits = new_max
21                pool.status = pool.check_status()
22
23        # Update SPF+ key rate normalization
24        global SPF_MAX_KEY_RATE
25        SPF_MAX_KEY_RATE = new_max_rate
26

```

اگر این کالیبراسیون انجام نشود، در حالت خط لوله همه استخرها به‌طور دائمی در وضعیت LOW یا REPLENISHING گیر می‌کنند و فشار استخر برای همه هاپ‌ها به $1.0 \sim$ می‌رسد که باعث می‌شود SPF+ نتواند میان مسیرها تمایز قائل شود. این رفتار در حین توسعه مشاهده شد و به‌عنوان یک باگ بحرانی شناسایی و اصلاح گردید. تغییر متغیر سراسری SPF_MAX_KEY_RATE با کلیدواژه global انجام می‌شود که در پایتون برای اصلاح متغیرهای ماژول سطح بالا ضروری است.

۵.۱۲ ارسال و دریافت با سایر ماژول‌ها

۱.۵.۱۲ وابستگی به موتور فیزیکی main_optimized.py

مهم‌ترین وابستگی خارجی این ماژول، به فایل main_optimized.py است که در همان دایرکتوری قرار دارد. این وابستگی از طریق تابع load_local_pipeline_module برقرار می‌شود که موتور فیزیکی را به‌صورت پویا بارگذاری می‌کند. از موتور فیزیکی، پنج مؤلفه کلیدی استخراج می‌شود: numpy به‌عنوان کتابخانه آرایه، DEFAULT_CONFIG به‌عنوان پیکربندی پایه، init_worker برای مقداردهی اولیه کارگر، run_single_simulation به‌عنوان نقطه ورود اصلی شبیه‌سازی، و set_nested_value برای اصلاح پیکربندی. این وابستگی به‌صورت اختیاری (soft dependency) طراحی شده است؛ یعنی اگر موتور فیزیکی در دسترس نباشد، ماژول شبکه همچنان با شبیه‌ساز آماری قابل اجرا است. این طراحی graceful degradation نامیده می‌شود و تضمین می‌کند که ماژول در محیط‌های مختلف قابل اجرا باشد. در زمان آماده‌سازی شبکه، روش initialize در QKDNetwork7Nodes ابتدا تلاش می‌کند موتور خط لوله را بارگذاری کند و در صورت شکست، به‌صورت شفاف به شبیه‌ساز آماری روی می‌آورد و پیام مناسبی به کاربر نمایش می‌دهد.

۲.۵.۱۲ ورودی‌ها به ماژول

ورودی اصلی ماژول از طریق فراخوانی تابع demo() در انتهای فایل است که نیازی به پارامتر ندارد. این تابع یک نمونه از QKDNetwork7Nodes می‌سازد و چرخه حیات کامل را اجرا می‌کند. پارامترهای قابل تنظیم در طول اجرا عبارت‌اند از: num_sessions_per_link در روش prime_pools که تعداد نشست‌های QKD برای آماده‌سازی هر لینک را تعیین می‌کند (پیش‌فرض 50)، و pulses_per_session که تعداد پالس‌ها در هر نشست است (پیش‌فرض 10^7). در مورد pulses_per_session، مقدار قبلی 10^8 به دلیل فشار حافظه کاهش یافت؛ در 10^8 پالس، خط لوله چندین آرایه int64 به حجم حدود 800 مگابایت (مانند prepared_states, photons, click0, click1) تخصیص می‌داد که باعث فشار حافظه و موارد لبه‌ای عددی در کد parameter-estimation/sifting بسته qkd می‌شد.

علاوه بر این، کلاس `StatisticalQKDLinkSimulator` پارامترهای فیزیکی قابل تنظیمی دارد: `det_eff` (بازده آشکارساز)، `dark_rate` (نرخ دارک کانت)، `fiber_loss` (تلفات فیبر بر کیلومتر)، `intrinsic_qber` (QBER ذاتی) و `mu_signal` (شدت پالس سیگنال). این پارامترها در سازنده پیش فرض گذاری شده اند تا با `DEFAULT_CONFIG` در `main_optimized.py` همخوان باشند. کلاس `PipelineQKDLinkSimulator` نیز دو پارامتر دارد: `detector_efficiency` که بازده آشکارساز D0 و D1 را تنظیم می کند، و `apply_noise` که مشخص می کند آیا مسیر کامل آشکارساز نویزدار استفاده شود.

توپولوژی شبکه به صورت سخت کد در روش `build_network` تعریف شده است. هفت گره با نقش مشخص، ده لینک با فواصل تعریف شده در دیکشنری `NODE_DISTANCES`، و سه سویچ نوری با پیکربندی پیش فرض ایجاد می شوند. فواصل لینک ها به صورت متروپولیتن در نظر گرفته شده اند: لینک A-C با 50 کیلومتر کوتاه ترین مسیر مستقیم به رله است، در حالی که A-B با 75 کیلومتر طولانی ترین است. لینک های رله به ارسال کننده از 30 کیلومتر (C-D) تا 70 کیلومتر (B-G) متغیر هستند.

۳.۵.۱۲ خروجی ها و گزارش گیری

خروجی اصلی ماژول، نمایش متنی قالب بندی شده در `stdout` است که شامل چندین بخش است: جدول توپولوژی گره ها با نقش و همسایه ها، جدول معیارهای لینک شامل فاصله، وضعیت، QBER، نرخ کلید و تلفات، جدول سطح استخر با بیت های موجود، رزرو شده و مصرف شده، پیکربندی سویچ های نوری، جزئیات هر درخواست کلید خدمت شامل مقایسه مسیرهای کاندید، گام های XOR relay، و شناسه کلید خدمت صادر شده.

علاوه بر خروجی متنی، روش `get_network_status` در KMS یک دیکشنری ساختار یافته برمی گرداند که شامل وضعیت کامل شبکه است. این دیکشنری برای یکپارچه سازی با سامانه های مانیتورینگ خارجی یا تولید گزارش خودکار مناسب است. ساختار آن به این صورت است:

```

1      {
2          "nodes": {
3              "A": {
4                  "role": "RECEIVER",
5                  "neighbors": ["B", "C"],
6                  "pools": {
7                      "A-B": {
8                          "pool_id": "POOL-A-B",
9                          "status": "ACTIVE",
10                         "available_bits": 12345678,
11                         "reserved_bits": 0,
12                         "used_bits": 1024,
13                         "num_blocks": 42
14                     },
15                     ...
16                 }
17             },
18             ...
19         },
20         "links": {
21             "A-B": {
22                 "distance_km": 0.75,
23                 "status": "UP",
24                 "qber": 0.23456,
25                 "secret_key_rate": 0.123,
26                 "optical_loss_db": 0.15,
27                 "last_session": "QKD-A-B-abc12345"
28             },
29             ...
30         },
31         "switches": {
32             "SW-A-1x2": {
33                 "type": "1x2",
34                 "available": true,
35                 "configuration": {"A": "B"}
36             },
37             ...
38         },
39         "service_keys_issued": 4
40     }
41 }
```

این ساختار تودرتو امکان سریع پیمایش وضعیت شبکه را فراهم می‌سازد و برای ابزارهای گزارش‌گیری خودکار یا داشبورد مانیتورینگ مناسب است.

۴.۵.۱۲ یکپارچه‌سازی با اصلاحات F-01 تا F-22

هنگامی که موتور خط لوله فعال است، ماژول شبکه به‌صورت خودکار از تمام اصلاحات اعمال‌شده در main_optimized.py بهره می‌برد. این اصلاحات شامل موارد زیر است: F-01 (تثبیت ولتاژ بایاس آشکارساز و ممیزی override)، F-05 (استفاده از SeedSequence.spawn برای جریان‌های مستقل RNG)، F-11 (اعتبارسنجی تعادل احتمال)، F-14 (ذخیره‌سازی متادیتای منبع) و F-18 (حذف کد مرده debug pulse override). این اصلاحات به‌صورت شفاف از طریق فراخوانی run_single_simulation فعال می‌شوند و نیازی به پیکربندی اضافی در ماژول شبکه ندارند.

در مورد F-05، نکته مهم آن است که بذر تصادفی در سطح ماژول شبکه با run_single_simulation به numpy.random.default_rng().integers(0, 2**63) تولید می‌شود و سپس به SeedSequence.spawn(5) به پنج جریان مستقل تقسیم می‌شود که هر یک برای یک بخش مختلف خط لوله (منبع، کانال، پروتکل، آشکارساز، sifting) استفاده می‌شود. این تضمین می‌کند که نشست‌های متوالی روی همان لینک، نتایج مستقل از نظر آماری داشته باشند و همبستگی‌های ناخواسته میان اجزای مختلف خط لوله ایجاد نشود.

۵.۵.۱۲ رفتار در حالت خرابی و بازیابی

ماژول شبکه شامل منطق پیچیده‌ای برای مدیریت خرابی و بازیابی است. روش handle_failure در KMS با دریافت شناسه یک لینک خراب، آن لینک را به وضعیت DOWN تبدیل می‌کند، استخرهای متناظر را به وضعیت ERROR (چسبناک) تغییر می‌دهد، و سپس برای همه جفت‌های ارسال‌کننده-گیرنده تحت تأثیر، مسیرهای جایگزین را با SPF+ جستجو می‌کند. این روش مسیرهای جایگزین را به‌صورت لیست رشته‌ای برمی‌گرداند که در خروجی demo نمایش داده می‌شوند.

```

1 def handle_failure(self, failed_link_id: str) -> List[str]:
2     link = self.links.get(failed_link_id)
3     if link is None:
4         return []
5
6     link.status = LinkStatus.DOWN
7
8     node_a, node_b = link.node_a, link.node_b
9     pool_a = self.nodes[node_a].key_pools.get(f"{node_a}-{node_b}")
10    pool_b = self.nodes[node_b].key_pools.get(f"{node_b}-{node_a}")
11    if pool_a:
12        pool_a.status = KeyPoolStatus.ERROR
13    if pool_b:
14        pool_b.status = KeyPoolStatus.ERROR
15
16    affected_pairs: List[Tuple[str, str]] = []
17    senders = ["D", "E", "F", "G"]
18    for s in senders:
19        affected_pairs.append((s, "A"))
20
21    alternative_paths: List[str] = []
22    for sender, receiver in affected_pairs:
23        alt_path = self.spf_plus(sender, receiver)
24        if alt_path:
25            path_str = " -> ".join(alt_path)
26            alternative_paths.append(path_str)
27
28    return alternative_paths
29

```

در demo، پس از شبیه‌سازی خرابی لینک B-D، درخواست کلید خدمت D-A مجدداً اجرا می‌شود. در این حالت، SPF+ به‌جای مسیر کوتاه D-B-A، مسیر جایگزین D-C-A را انتخاب می‌کند چون لینک B-D با هزینه ∞ عملاً از گراف حذف شده است. این رفتار به‌صورت خودکار توسط الگوریتم Dijkstra مدیریت می‌شود و نیازی به منطق خاصی در سطح بالاتر ندارد.

پس از خرابی، روش run_demo لینک B-D را به‌صورت دستی به وضعیت UP بازمی‌گرداند و استخرهای آن

را به وضعیت LOW تنظیم می‌کند تا فرآیند replenishment آن‌ها را شناسایی و پر کند. این الگو (manual state injection) برای شبیه‌سازی سناریوهای بازیابی استفاده می‌شود. در یک سیستم تولیدی واقعی، این تغییرات وضعیت به صورت خودکار توسط سیستم‌های مانیتورینگ که فاصله، QBER یا سایر معیارها را پایش می‌کنند، اعمال می‌شوند.

۶.۵.۱۲ رفتار حافظه و کارایی

از نظر حافظه، ماژول شبکه به صورت نسبتاً سبک طراحی شده است. هر استخر کلید به صورت یک دیکشنری از بلوک‌های کلید پیاده‌سازی شده که هر بلوک شامل یک شیء بایت با طول 32 بایت (برای 256 بیت کلید) است. در حالت آماری با watermark پیش‌فرض 20 مگابایت، هر استخر می‌تواند تا حدود 78125 بلوک 256 بیتی نگه دارد که در مجموع حدود 2.5 مگابایت حافظه برای هر استخر و 50 مگابایت برای کل شبکه نیاز دارد. در حالت خط لوله با watermark 200 کیلوبایت، هر استخر تنها حدود 25 کیلوبایت حافظه نیاز دارد.

از نظر کارایی، عملیات SPF+ با پیچیدگی $\mathcal{O}((V + E) \log V) \approx \mathcal{O}(17 \log 7) \approx 50$ عملیات برای هر درخواست، بسیار سریع است. عملیات reserve_keys با پیچیدگی $\mathcal{O}(n \log n)$ که در آن n تعداد بلوک‌ها است، در بدترین حالت با $n \approx 78000$ به حدود 1.3 میلیون عملیات می‌رسد که در عملی روی سخت‌افزار مدرن در کمتر از 100 میلی‌ثانیه اجرا می‌شود. عملیات simulate_link در حالت آماری تقریباً بی‌درنگ است، در حالی که در حالت خط لوله با 10^7 پالس، حدود چند ثانیه تا چند دقیقه زمان نیاز دارد بسته به پیکربندی.

به طور کلی، این ماژول به عنوان لایه بالاترین انتزاع در فریم‌ورک شبیه‌سازی QKD عمل می‌کند و در عین حال از موتور فیزیکی دقیق بهره می‌برد. طراحی ماژول آن امکان توسعه آسان را فراهم می‌سازد؛ برای مثال، افزودن گره‌های جدید تنها نیازمند اصلاح build_network و NODE_DISTANCES است، و افزودن پروتکل‌های رله جدید تنها با ایجاد زیرکلاس جدید از QKNode امکان‌پذیر است. این انعطاف‌پذیری، ماژول را به ابزاری مناسب برای مطالعه شبکه‌های QKD در مقیاس متروپولیتن تبدیل می‌کند.

بخش چهارم
اعتبارسنجی و آزمون

فصل ۱۳

اعتبارسنجی لایه فیزیکی

۱.۱۳ نمای کلی و هدف ماژول

ماژول `validate_physics.py` یک اسکریپت اعتبارسنجی فیزیکی است که نقش بین‌الایه‌ای حیاتی در فریم‌ورک شبیه‌سازی QKD ایفا می‌کند. این ماژول به‌عنوان یک `golden reference` عمل می‌کند که خروجی موتور شبیه‌سازی کامل `main_optimized.py` را در برابر معادلات تحلیلی بسته و مستقل از پیاده‌سازی، مقایسه می‌نماید و با استفاده از آزمون آماری `Z-test` دودویی (binomial Z-test) در سطح معنی‌داری $\alpha = 10^{-4}$ ، انطباق آماری نتایج را بررسی می‌سازد. وجود چنین اسکریپت اعتبارسنجی جداگانه‌ای در یک فریم‌ورک علمی، نشانه بلوغ مهندسی نرم‌افزار است؛ زیرا هرگونه تغییر در پیاده‌سازی موتور فیزیکی، چه عمدی (مانند اصلاح یک باگ) و چه ناخواسته (مانند regression پس از refactor)، بلافاصله توسط این آزمون‌ها شناسایی می‌شود و پیش از انتشار نتایج، به اطلاع توسعه‌دهنده می‌رسد.

هدف اصلی این ماژول، پاسخ به پرسش بنیادین زیر است: آیا خروجی شبیه‌سازی مونته‌کارلو که از خط لوله کامل `Source -> Channel -> Protocol -> Detector -> Sifting -> Proof` می‌آید، از نظر آماری با پیش‌بینی تئوریک معادلات تحلیلی بسته هم‌خوان است؟ این پرسش به دو دلیل اهمیت دارد. نخست، در شبیه‌سازی‌های فیزیکی که در آن‌ها فرایندهای تصادفی متعددی (نظیر توزیع Poisson فوتون‌ها، شمارش تاریک آشکارساز (dark count)، غریبال‌سازی پایه‌ها (sifting) و غیره) با یکدیگر ترکیب می‌شوند، خطاهای ظریف در پیاده‌سازی ممکن است به نتایجی منجر شود که از نظر فیزیکی قابل قبول به نظر می‌رسند، اما از نظر آماری biased هستند. دوم، معادلات تحلیلی بسته که در ادبیات QKD به‌طور گسترده پذیرفته شده‌اند (مانند معادلات Gain و QBER در مدل Gottesman-Lo-Lütkenhaus-Preskill) به‌عنوان مرجع علمی عمل می‌کنند و انطباق با آن‌ها، اعتبار فیزیکی پیاده‌سازی را تضمین می‌نماید.

ساختار کلی ماژول شامل چهار تابع محاسباتی تحلیلی، یک تابع آزمون آماری و یک تابع اجرای شبیه‌سازی ایزوله است. تابع `channel_transmittance` ضریب انتقال کانال فیبر نوری را بر اساس قانون Beer-Lambert محاسبه می‌کند. تابع `Gain Q_mu_poisson` (نرخ کلیک به ازای هر پالس) را برای منبع Weak Coherent Pulse با توزیع Poisson محاسبه می‌کند. تابع `Q_mu_thermal` همین کمیت را برای منبع Thermal (تک‌حالت یا چندحالت) محاسبه می‌نماید. تابع `QBER E_mu_poisson` متوسط را برای منبع Poisson به دست می‌آورد. تابع `dead_time_correction` تصحیح dead time آشکارساز را بر اساس مدل Non-Paralyzable اعمال می‌کند. تابع `validate_statistic` آزمون `Z-test` دودویی را روی یک مشاهده انجام می‌دهد و نتیجه FAIL/PASS را برمی‌گرداند. در نهایت، تابع `run_isolated_simulation` یک نشست شبیه‌سازی مستقل روی موتور `main_optimized.py` اجرا می‌کند و خروجی را به فرمت مناسب برای آزمون آماری باز می‌گرداند.

سه گروه آزمون در این ماژول تعریف شده است که هر یک به جنبه‌ای متفاوت از فیزیک شبیه‌سازی می‌پردازد. TEST 1 منبع Weak Coherent Pulse با توزیع Poisson را در سه فاصله 0، 50 و 100 کیلومتر اعتبارسنجی می‌کند؛ این آزمون هم Gain و هم QBER را در رژیم مجانبی (یعنی بدون decoy و بدون dead time) بررسی می‌سازد. TEST 2 منبع Thermal تک‌حالت را در فاصله صفر آزمون می‌کند تا توزیع آماری متفاوت منبع را راستی‌آزمایی نماید. TEST 3 فیزیک dead time آشکارساز SPAD را در حضور دو آشکارساز مستقل بررسی می‌کند و این موضوع مهم را که هر SPAD به‌طور مستقل زمان مرده خود را تجربه می‌کند، به آزمون می‌گذارد. یکی از ویژگی‌های برجسته این ماژول، انتخاب سطح معنی‌داری $\alpha = 10^{-4}$ است که به $Z\text{-crit} \approx 3.8906$ متناظر می‌شود. این سطح به‌مراتب سخت‌گیرانه‌تر از سطح متداول $\alpha = 0.05$ ($Z\text{-crit} \approx 1.96$) یا حتی $\alpha = 0.001$ ($Z\text{-crit} \approx 3.29$) است. دلیل این انتخاب سخت‌گیرانه آن است که در شبیه‌سازی QKD با $N = 100000$ پالس، حتی انحرافات کوچک از تئوری می‌توانند به‌سرعت شناسایی شوند و توسعه‌دهنده را از پذیرش نتایج

تقریباً درست که در عمل biased هستند، بازدارند. با این حال، این سطح به قدری سخت‌گیرانه نیست که نویز آماری ذاتی شبیه‌سازی مونت‌کارلو باعث false positive rate مکرر شود؛ تعادل میان حساسیت و false positive rate با دقت انتخاب شده است.

در سطح عملیاتی، این ماژول به‌عنوان یک CI/CD gate عمل می‌کند؛ یعنی پیش از merge هر تغییر در شاخه اصلی، این اسکریپت اجرا می‌شود و در صورت شکست هر یک از آزمون‌ها، pipeline متوقف می‌گردد. این رویکرد shift-left testing تضمین می‌کند که باگ‌های فیزیکی در همان مراحل اولیه توسعه شناسایی شوند، نه پس از آنکه نتایج شبیه‌سازی در مقالات یا گزارش‌های فنی منتشر شده باشند. در ادبیات مهندسی نرم‌افزار علمی، این الگو به‌عنوان regression testing for computational physics شناخته می‌شود و در پروژه‌های بالغی مانند Quantum ESPRESSO، LAMMPS و Qiskit به‌صورت استاندارد به کار می‌رود.

۲.۱۳ مبانی تئوری و فیزیکی

۱.۲.۱۳ کانال فیبر نوری و قانون Beer-Lambert

انتقال فوتون در فیبر نوری تک‌حالت با قانون Beer-Lambert مدل می‌شود که رابطه بنیادین بین تلفات فیبر، فاصله و ضریب انتقال را بیان می‌دارد. برای یک فیبر به طول L کیلومتر با ضریب تلفت خطی $\alpha_{\text{dB/km}}$ (بر حسب دسی‌بل بر کیلومتر)، تلفات کلی کانال بر حسب دسی‌بل برابر با $\alpha_{\text{dB/km}} \cdot L$ است. چون مقیاس دسی‌بل لگاریتمی است، تبدیل آن به ضریب انتقال خطی (power transmittance) از رابطه زیر پیروی می‌کند:

$$\eta_{\text{channel}}(L, \alpha_{\text{dB/km}}) = 10^{-\frac{\alpha_{\text{dB/km}} \cdot L}{10}} \quad (1.13)$$

در این رابطه، L بر حسب کیلومتر و $\alpha_{\text{dB/km}}$ بر حسب dB/km است. در طول‌موج مخابراتی استاندارد 1550 نانومتر، فیبر تک‌حالت استاندارد ضریب تلفت $\alpha_{\text{dB/km}} \approx 0.2$ دارد؛ بنابراین در فاصله 50 کیلومتر، ضریب انتقال کانال برابر با $10^{-1} = 0.1$ و در فاصله 100 کیلومتر، برابر با $10^{-2} = 0.01$ می‌شود. این کاهش نمایی ضریب انتقال با فاصله، عامل اصلی محدودیت برد QKD در فیبر نوری است و دلیل آن است که در فواصل بزرگ‌تر از 200 کیلومتر، حتی با استفاده از decoy-state، نرخ کلید امن به صفر میل می‌کند. ضریب انتقال کلی سیستم شامل بازده آشکارساز η_{det} نیز می‌شود که در این ماژول به‌صورت یک کمیت مستقل در نظر گرفته شده است:

$$\eta_{\text{total}}(L) = \eta_{\text{det}} \cdot \eta_{\text{channel}}(L, \alpha_{\text{dB/km}}) \quad (2.13)$$

در پیاده‌سازی پیش‌فرض این ماژول، $\eta_{\text{det}} = 0.15$ است که متناظر با آشکارساز InGaAs SPAD در دمای -50 درجه سانتی‌گراد با بازده کوانتومی متداول است. در عمل، این مقدار در بازه 0.1 تا 0.25 متغیر است و به دمای آشکارساز، ولتاژ بایاس و طول‌موج فوتون بستگی دارد.

۲.۲.۱۳ منبع Pulse Coherent Weak و توزیع Poisson

در اکثر پیاده‌سازی‌های عملی QKD، به‌جای منبع تک‌فوتونی آرمانی (که هنوز در دسترس تجاری نیست)، از منبع Weak Coherent Pulse (WCP) استفاده می‌شود. در این منبع، لیزر با شدت متوسط μ فوتون در هر پالس تحریک می‌شود و تعداد فوتون‌های هر پالس از توزیع Poisson پیروی می‌کند:

$$P(n|\mu) = \frac{\mu^n e^{-\mu}}{n!}, \quad n = 0, 1, 2, \dots \quad (3.13)$$

با این توزیع، احتمال اینکه یک پالس حاوی n فوتون باشد به‌صورت بالا داده می‌شود. پارامتر μ میانگین تعداد فوتون در هر پالس است و در decoy-state BB84 معمولاً در بازه 0.1 تا 0.5 انتخاب می‌شود. مقدار $\mu = 0.5$ که در این ماژول به‌کار رفته، یک انتخاب متعادل میان نرخ کلید (که با μ رشد می‌کند) و امنیت (که با μ کاهش می‌یابد، چون پالس‌های چندفوتونی به حمله Photon-Number Splitting حساس هستند) است. Gain کل سیستم (detection yield) که با Q_μ نشان داده می‌شود، احتمال آن است که یک پالس منجر به ثبت کلیک در آشکارساز شود. برای منبع WCP با توزیع Poisson، در حضور دارک کانت با نرخ Y_0 و بازده کلی η ، این کمیت از رابطه بسته زیر به دست می‌آید:

$$Q_{\mu}^{\text{Poisson}}(\mu, \eta, Y_0) = 1 - (1 - Y_0) \cdot e^{-\mu \cdot \eta} \quad (۴.۱۳)$$

این رابطه از جمع احتمالات تمام حالات ممکن به دست می‌آید. به طور شهودی، عبارت $e^{-\mu \cdot \eta}$ احتمال آن است که هیچ فوتونی شناسایی نشود (یعنی یا هیچ فوتونی ارسال نشده، یا فوتون‌های ارسالی از دست رفته‌اند) و عامل $(1 - Y_0)$ احتمال آن است که دارک کانت نیز رخ ندهد. بنابراین، $1 - (1 - Y_0)e^{-\mu \cdot \eta}$ احتمال متمم است: یعنی احتمال آنکه حداقل یک کلیک (چه از فوتون واقعی و چه از دارک کانت) ثبت شود. این فرمول در ادبیات QKD به‌ویژه در کار کلاسیک (2005) Lo-Ma-Chen به تفصیل استخراج شده است.

۳.۲.۱۳ منبع Thermal و توزیع Bose-Einstein

در برخی سناریوهای QKD، به جای لیزر WCP از منبع Thermal استفاده می‌شود. منبع حرارتی فوتون‌ها با توزیع Bose-Einstein تولید می‌کند که برای یک منبع تک‌حالتی به صورت زیر تعریف می‌شود:

$$P(n|\mu) = \frac{\mu^n}{(1 + \mu)^{n+1}}, \quad n = 0, 1, 2, \dots \quad (۵.۱۳)$$

برای یک منبع چندحالتی با M حالت مستقل، توزیع مرجع به توزیع Negative Binomial میل می‌کند. Gain کل سیستم برای منبع حرارتی با M حالت، از رابطه زیر به دست می‌آید:

$$Q_{\mu}^{\text{Thermal}}(\mu, \eta, Y_0, M) = 1 - \frac{1 - Y_0}{\left(1 + \frac{\mu \cdot \eta}{M}\right)^M} \quad (۶.۱۳)$$

این رابطه تعمیم طبیعی فرمول Poisson است. در حد $M \rightarrow \infty$ ، توزیع Thermal به Poisson میل می‌کند و فرمول بالا به فرمول Poisson تقلیل می‌یابد. برای $M = 1$ (تک‌حالت)، این رابطه به $Q_{\mu}^{\text{Thermal}} = 1 - (1 - Y_0)/(1 + \mu \eta)$ ساده می‌شود که فرمول کلاسیک منبع حرارتی تک‌حالت است. تفاوت فیزیکی میان Poisson و Thermal در آن است که در Thermal، فوتون‌ها تمایل به bunching دارند (یعنی احتمال پالس‌های چندفوتونی بیشتر است) که این امر امنیت را در برابر حمله PNS کاهش می‌دهد. با این حال، مطالعه Thermal از منظر آکادمیک اهمیت دارد چرا که برخی منابع نور تک‌فوتونی (مانند quantum dot) در رژیم خاصی رفتار حرارتی نشان می‌دهند.

۴.۲.۱۳ مدل QBER برای منبع Poisson

نرخ خطای کوانتومی بیت (Quantum Bit Error Rate) که با E_{μ} نشان داده می‌شود، کسری از بیت‌های sifted است که بین دو طرف ناهماهنگ هستند. در مدل تحلیلی، این کمیت از نسبت خطاهای کلیک به کل کلیک‌ها به دست می‌آید:

$$E_{\mu}^{\text{Poisson}}(\mu, \eta, Y_0, e_d) = \frac{0.5 \cdot Y_0 + e_d \cdot (Q_{\mu} - Y_0)}{Q_{\mu}} \quad (۷.۱۳)$$

در این رابطه، e_d خطای نوری ذاتی (misalignment error) است که از نقص‌های تداخل‌سنج ناشی می‌شود و Y_0 نرخ دارک کانت است. صورت این کسر نشان می‌دهد که خطا از دو منبع می‌آید: دارک کانت که کاملاً تصادفی است و نرخ خطای 0.5 دارد (یعنی نیمی از دارک کانت‌ها با بیت ارسالی هم‌خوان و نیمی ناهم‌خوان هستند)، و فوتون‌های واقعی که با احتمال e_d دچار خطا می‌شوند. مخرج Q_{μ} کل کلیک‌ها است که مجموع دارک و سیگنال است. در نتیجه، E_{μ} کسر وزنی خطاها است.

این فرمول نشان می‌دهد که در حضور دارک کانت، حتی اگر $e_d = 0$ باشد (یعنی هیچ خطای نوری ذاتی وجود نداشته باشد)، QBER همچنان بزرگ‌تر از صفر خواهد بود؛ این پدیده به‌ویژه در فواصل طولانی که سیگنال ضعیف می‌شود، اهمیت پیدا می‌کند و دلیل اصلی آن است که در فواصل بزرگ، QBER به 0.5 میل می‌کند (زیرا بیشتر کلیک‌ها دارک هستند). در حد $\eta \rightarrow 0$ (یعنی کانال با تلفات بسیار بالا)، $Q_{\mu} \rightarrow Y_0$ و در نتیجه $E_{\mu} \rightarrow 0.5$ که نشان‌دهنده از دست رفتن کامل اطلاعات است.

در صورت کسر، عبارت $0.5 \cdot Y_0$ نشان‌دهنده آن است که دارک کانت‌ها به صورت کاملاً تصادفی رخ می‌دهند و بنابراین نرخ خطای آن‌ها برابر 0.5 است. عبارت $e_d \cdot (Q_\mu - Y_0)$ نشان‌دهنده خطای ناشی از فوتون‌های واقعی است؛ $(Q_\mu - Y_0)$ بخشی از کلیک‌ها را نشان می‌دهد که از فوتون‌های واقعی می‌آیند (نه از دارک کانت) و این بخش با احتمال e_d دچار خطا می‌شود. بنابراین، صورت کل کسر برابر است با کل خطاهای ثبت‌شده در واحد پالس.

۵.۲.۱۳ مدل Time Dead آشکارساز SPAD

آشکارسازهای Single-Photon Avalanche Diode (SPAD) پس از هر کلیک، به مدتی برای بازیابی نیاز دارند که به آن dead time می‌گویند. در این بازه زمانی τ_d ، آشکارساز قادر به ثبت کلیک جدیدی نیست. دو مدل متداول برای این پدیده وجود دارد: Paralyzable و Non-Paralyzable. در این مازول، مدل Non-Paralyzable به کار رفته است که در آن، کلیک‌های رخ داده در زمان مرده آشکارساز را برای مدت بیشتر بازنشانی نمی‌کنند بلکه صرفاً نادیده گرفته می‌شوند.

برای یک آشکارساز Non-Paralyzable با احتمال کلیک ایده‌آل Q_{ideal} به ازای هر پالس و بازه زمانی میان پالس‌ها T_{pulse} ، احتمال کلیک مشاهده‌شده Q_{obs} از رابطه زیر به دست می‌آید:

$$Q_{obs} = \frac{Q_{ideal}}{1 + Q_{ideal} \cdot \frac{\tau_d}{T_{pulse}}} \quad (۸.۱۳)$$

این رابطه از حل معادله تابع ضمنی $Q_{obs} = Q_{ideal} \cdot (1 - Q_{obs} \cdot \tau_d / T_{pulse})$ به دست می‌آید که نشان می‌دهد کسر زمانی که آشکارساز در دسترس است، برابر با $1 - Q_{obs} \cdot \tau_d / T_{pulse}$ است. در حد $\tau_d \rightarrow 0$ ، $Q_{obs} \rightarrow Q_{ideal}$ (بدون کاهش) و در حد $\tau_d \rightarrow \infty$ یا $Q_{ideal} \rightarrow 1$ ، $Q_{obs} \rightarrow T_{pulse} / \tau_d$ (اشباع).

یکی از نکات ظریف در پیاده‌سازی این مازول، آن است که در سیستم BB84 دو آشکارساز مستقل D0 و D1 وجود دارند که هر کدام زمان مرده مستقل خود را تجربه می‌کنند. بنابراین، احتمال کلیک Gain ایده‌آل ابتدا باید به دو نیمه تقسیم شود (هر آشکارساز نیمی از کلیک‌ها را ثبت می‌کند)، سپس تصحیح dead time به صورت مستقل روی هر آشکارساز اعمال شود:

$$Q_{per-spada} = \frac{Q_{ideal}^{raw}}{2}, \quad Q_{obs}^{per-spada} = \frac{Q_{per-spada}}{1 + Q_{per-spada} \cdot \tau_d / T_{pulse}} \quad (۹.۱۳)$$

$$Q_{obs}^{total} = 2 \cdot Q_{obs}^{per-spada} \quad (۱۰.۱۳)$$

در نهایت، چون sifting در BB84 نیمی از پالس‌ها را حذف می‌کند (پالس‌هایی که پایه گیرنده با فرستنده هم‌خوان نداشته)، Gain نهایی که در خروجی شبیه‌سازی مشاهده می‌شود، برابر با $0.5 \cdot Q_{obs}^{total}$ است:

$$Q_{final} = Q_{obs}^{per-spada} \cdot 2 \cdot 0.5 = Q_{obs}^{per-spada} \quad (۱۱.۱۳)$$

این رابطه ظریف نشان می‌دهد که در نهایت، Gain مشاهده‌شده با Gain هر آشکارساز برابر است؛ چیزی که در نگاه اول غیرمنتظره به نظر می‌رسد اما با بررسی دقیق عوامل 2 و 0.5 که یکدیگر را خنثی می‌کنند، آشکار می‌گردد.

۶.۲.۱۳ آزمون آماری Z-test دودویی

برای اعتبارسنجی آماری نتایج شبیه‌سازی، این مازول از Z-test دودویی استفاده می‌کند که یکی از ساده‌ترین و در عین حال قدرتمندترین آزمون‌های آماری برای متغیرهای دودویی است. فرض کنید N آزمایش مستقل انجام شده و k مورد موفقیت مشاهده شده است؛ اگر احتمال موفقیت تئوریک p باشد، تعداد موفقیت‌های مشاهده‌شده از توزیع دودویی $\text{Binomial}(N, p)$ پیروی می‌کند. در حد $N \rightarrow \infty$ ، این توزیع به توزیع نرمال $\mathcal{N}(Np, Np(1-p))$ میل می‌کند (De Moivre-Laplace theorem). در نتیجه، متغیر استاندارد شده Z از توزیع نرمال استاندارد پیروی می‌کند:

$$Z = \frac{|k - Np|}{\sqrt{Np(1-p)}} \quad (۱۲.۱۳)$$

در این رابطه، $k - Np$ انحراف مشاهده از امید ریاضی است و $\sqrt{Np(1-p)}$ انحراف معیار توزیع دودویی است. اگر Z از یک آستانه بحرانی Z_{crit} تجاوز کند، فرض صفر (اینکه مشاهده با تئوری همخوان است) رد می‌شود. در سطح معنی‌داری $\alpha = 10^{-4}$ ، آستانه بحرانی (دوسویه) برابر با $Z_{\text{crit}} \approx 3.8906$ است. این مقدار از تابع توزیع نرمال استاندارد به دست می‌آید:

$$\alpha = 2 \cdot \Phi(-Z_{\text{crit}}), \quad Z_{\text{crit}} = -\Phi^{-1}(\alpha/2) \quad (۱۳.۱۳)$$

که در آن Φ تابع توزیع تجمعی نرمال استاندارد است. انتخاب $\alpha = 10^{-4}$ به جای $\alpha = 0.05$ (که متداول در علوم اجتماعی است) دو دلیل دارد. نخست، در شبیه‌سازی‌های فیزیکی، ما کنترل کاملی روی منبع تصادفی داریم و می‌توانیم N را به اندازه کافی بزرگ انتخاب کنیم تا نویز آماری کوچک باشد؛ بنابراین آستانه سخت‌گیرانه قابل قبول است. دوم، در QKD حتی انحرافات کوچک از تئوری می‌توانند نشانگر باگ‌های ظریف در پیاده‌سازی باشند که در نهایت به نتایج ناامن منجر می‌شوند؛ بنابراین حساسیت بالا ضروری است.

۳.۱۳ تحلیل معماری و ساختار داده‌ها

۱.۳.۱۳ ساختار کلی ماژول

ساختار این ماژول به صورت یک stateless utility module طراحی شده است؛ یعنی هیچ کلاسی تعریف نشده و تمام توابع به صورت مستقل (pure functions یا near-pure functions) عمل می‌کنند. این طراحی به این دلیل انتخاب شده که اعتبارسنجی فیزیکی باید کاملاً قطعی و قابل تکرار باشد و نباید به وضعیت داخلی هیچ شیء وابسته باشد. هر تابع با دریافت ورودی‌های صریح، خروجی مشخصی تولید می‌کند و عوارض جانبی (side effects) آن به چاپ نتیجه آزمون محدود است.

واردسازی (imports) در ابتدای ماژول شامل چهار کتابخانه استاندارد پایتون است: math برای توابع ریاضی پایه (نظیر exp, sqrt)، sys برای دسترسی به argv (اگرچه در نسخه فعلی استفاده نمی‌شود، برای توسعه‌های آینده نگه داشته شده)، copy برای کپی عمیق پیکربندی، و csv برای خواندن فایل خروجی شبیه‌سازی. این حداقل وابستگی تضمین می‌کند که ماژول در هر محیطی قابل اجرا باشد و نیازی به کتابخانه‌های خارجی ندارد.

```

۱ import math
۲ import sys
۳ import copy
۴ import csv
۵
```

تنها وابستگی خارجی ماژول، به main_optimized.py است که به صورت محلی (local import) در داخل تابع run_isolated_simulation بارگذاری می‌شود. این الگو (lazy import) دو مزیت دارد: نخست، وابستگی چرخه‌ای (circular dependency) را که ممکن است در زمان بارگذاری ماژول رخ دهد، اجتناب می‌کند؛ دوم، اجازه می‌دهد ماژول حتی در صورتی که main_optimized.py در دسترس نباشد، حداقل برای توابع تحلیلی (channel_transmittance، Q_mu_poisson و غیره) قابل استفاده باشد.

۲.۳.۱۳ توابع تحلیلی channel_transmittance

این تابع ساده‌ترین واحد محاسباتی ماژول است و ضریب انتقال کانال را بر اساس قانون Beer-Lambert محاسبه می‌کند. با وجود سادگی، این تابع به عنوان building block پایه برای تمام محاسبات بعدی عمل می‌کند و هر خطا در آن به خطاهای فاحش در تمام آزمون‌ها منجر می‌شود.

```

۱ def channel_transmittance(L, alpha_dB_km):
۲     return 10 ** (-alpha_dB_km * L / 10)
۳
```

پارامترهای این تابع عبارتند از: L فاصله به کیلومتر و $\alpha_{\text{dB/km}}$ ضریب تلفت فیبر به dB/km. خروجی، ضریب انتقال خطی (در محدوده $[0, 1]$) است. این تابع به صورت stateless عمل می‌کند و هیچ متغیر سراسری را تغییر نمی‌دهد؛ این ویژگی آن را به یک pure function تبدیل می‌کند که آزمون واحد (unit test) برای آن بسیار ساده است.

۳.۳.۱۳ توابع تحلیلی Q_mu_thermal و Q_mu_poisson

این دو تابع، Gain (نرخ کلیک) را برای دو نوع منبع نوری متفاوت محاسبه می‌کنند. تابع Q_mu_poisson برای منبع WCP با توزیع Poisson استفاده می‌شود که رایج‌ترین منبع در سیستم‌های QKD عملی است.

```

۱ def Q_mu_poisson(mu, eta, Y0):
۲     return 1 - (1 - Y0) * math.exp(-mu * eta)
۳
۴ def Q_mu_thermal(mu, eta, Y0, M=1):
۵     return 1 - (1 - Y0) / ((1 + mu * eta / M) ** M)
۶

```

پارامترهای مشترک این دو تابع عبارتند از: μ میانگین تعداد فوتون در هر پالس، η ضریب انتقال کلی (شامل بازده آشکارساز) و Y_0 نرخ دارک کانت. تابع thermal پارامتر اضافی M را نیز می‌پذیرد که تعداد حالت‌های مستقل منبع حرارتی را نشان می‌دهد و مقدار پیش‌فرض آن 1 (تک‌حالت) است. طراحی این دو تابع به گونه‌ای است که هر دو signature مشابه دارند (با تفاوت تنها در پارامتر M)؛ این امکان را فراهم می‌سازد که در کد فراخوانی، به سادگی بین دو مدل منبع جابه‌جا شوند. این الگو (Strategy pattern) در مهندسی نرم‌افزار برای پیاده‌سازی الگوریتم‌های قابل تعویض به کار می‌رود. نکته مهم فیزیکی: در حد $M \rightarrow \infty$ ، تابع thermal به Poisson میل می‌کند. این را می‌توان با استفاده از حد معروف $\lim_{M \rightarrow \infty} (1 + x/M)^M = e^x$ نشان داد:

$$\lim_{M \rightarrow \infty} \frac{1 - Y_0}{(1 + \frac{\mu\eta}{M})^M} = (1 - Y_0) \cdot e^{-\mu\eta} \quad (۱۴.۱۳)$$

بنابراین، در حد $M \rightarrow \infty$ ، $Q_{\mu}^{\text{Thermal}} \rightarrow Q_{\mu}^{\text{Poisson}}$. این ویژگی ریاضی تضمین می‌کند که دو تابع در حد فیزیکی مناسب هم‌گرا می‌شوند و پیاده‌سازی سازگار است.

۴.۳.۱۳ تابع تحلیلی E_mu_poisson

این تابع QBER متوسط را برای منبع WCP محاسبه می‌کند و بر اساس مدل کلاسیک GLLP پیاده‌سازی شده است.

```

۱ def E_mu_poisson(mu, eta, Y0, e_d):
۲     Q = Q_mu_poisson(mu, eta, Y0)
۳     if Q <= 0: return 5.0
۴     return 5.0 * Y0 + e_d * (Q - Y0) / Q
۵

```

این تابع ابتدا Gain Q را با فراخوانی Q_mu_poisson محاسبه می‌کند، سپس فرمول QBER را اعمال می‌نماید. خط `if Q <= 0: return 0.5` یک محافظ عددی است که در صورتی که Q صفر یا منفی شود (که فیزیکی نیست اما ممکن است در موارد لبه‌ای عددی رخ دهد)، مقدار 0.5 را برمی‌گرداند. این مقدار 0.5 نشان‌دهنده آن است که در غیاب هرگونه کلیک، هیچ اطلاعاتی در دست نیست و در نتیجه QBER به صورت uncertainty maximum یعنی 0.5 تنظیم می‌شود.

پارامتر e_d در این تابع، خطای نوری ذاتی (misalignment error) است که از نقص‌های تداخل‌سنج ناشی می‌شود. در عمل، این مقدار معمولاً در بازه 0.005 تا 0.03 است و در این مازول مقدار پیش‌فرض $e_d = 0.01$ به کار رفته است.

۵.۳.۱۳ تابع تحلیلی dead_time_correction

این تابع تصحیح dead time را بر اساس مدل Non-Paralyzable اعمال می‌کند. این تابع به عنوان building block پایه برای محاسبه Gain در حضور dead time عمل می‌کند.

```

۱ def dead_time_correction(Q_ideal, tau_d_ns, T_pulse_ns):
۲     return Q_ideal / (1 + Q_ideal * (tau_d_ns / T_pulse_ns))
۳

```

پارامترهای این تابع عبارتند از: Q_{ideal} احتمال کلیک ایده‌آل (بدون dead time)، τ_d زمان مرده به نانوثانیه و T_{pulse} بازه زمانی میان پالس‌ها به نانوثانیه. خروجی، احتمال کلیک مشاهده‌شده پس از اعمال اثر dead time است.

نکته طراحی مهم: این تابع به صورت مستقل روی یک آشکارساز عمل می‌کند، نه روی کل سیستم. این به این معناست که اگر چندین آشکارساز مستقل وجود داشته باشد (مانند D0 و D1 در BB84)، هر کدام باید به صورت جداگانه تصحیح شوند. این موضوع در بخش 3 TEST به صورت صریح پیاده‌سازی شده است.

۶.۳.۱۳ تابع آزمون آماری validate_statistic

این تابع هسته آماری ماژول است و آزمون Z-test دودویی را روی یک مشاهده انجام می‌دهد. این تابع با دریافت تعداد مشاهده‌شده موفقیت‌ها، تعداد کل آزمایش‌ها، احتمال تئوریک و نام آزمون، نتیجه FAIL/PASS را برمی‌گرداند.

```

۱ def validate_statistic(observed_count, total_trials, expected_prob, test_name):
۲     expected_count = total_trials * expected_prob
۳     variance = total_trials * expected_prob * (1 - expected_prob)
۴     if variance <= 0: variance = 1e-10
۵     std_dev = math.sqrt(variance)
۶     z_score = abs(observed_count - expected_count) / std_dev
۷
۸     passed = z_score <= 8906.3
۹     print(f"{test_name:25} | Exp_p: {expected_prob6.:e} | "
۱۰         f"Sim_p: {observed_count/total_trials6.:e} | "
۱۱         f"Z: {z_score2.6:f} | {'PASS' if passed else 'FAIL'}")
۱۲     return passed
۱۳

```

تحلیل خط‌به‌خط این تابع به شرح زیر است. در خط اول، تعداد مورد انتظار موفقیت‌ها از ضرب تعداد کل آزمایش‌ها در احتمال تئوریک محاسبه می‌شود: $k_{exp} = N \cdot p$. در خط دوم، واریانس توزیع دودویی از فرمول $Var = Np(1 - p)$ محاسبه می‌شود. در خط سوم، یک محافظ عددی برای جلوگیری از تقسیم بر صفر اعمال می‌شود: اگر واریانس صفر یا منفی باشد (که ممکن است در موارد لبه‌ای مانند $p = 0$ یا $p = 1$ رخ دهد)، مقدار بسیار کوچک 10^{-10} جایگزین می‌شود. در خط چهارم، انحراف معیار از جذر واریانس محاسبه می‌شود. در خط پنجم، Z-score از نسبت قدرمطلق انحراف به انحراف معیار به دست می‌آید. در خط هفتم، نتیجه آزمون با مقایسه Z-score با آستانه بحرانی $Z_{crit} = 3.8906$ (متناظر با $\alpha = 10^{-4}$ دوسویه) تعیین می‌شود.

قابل توجه است که این تابع در خط چاپ، علاوه بر نتیجه FAIL/PASS، مقادیر احتمال تئوریک، احتمال شبیه‌سازی‌شده و Z-score را نیز نمایش می‌دهد. این اطلاعات تشخیصی (diagnostic information) برای تحلیل شکست آزمون‌ها بسیار ارزشمند است؛ زیرا به توسعه‌دهنده اجازه می‌دهد ببیند انحراف از تئوری چقدر بزرگ است و در چه جهتی (مثبت یا منفی) قرار دارد. این الگو (verbose assertion) در توسعه نرم‌افزار علمی به عنوان بهترین روش شناخته می‌شود.

۷.۳.۱۳ تابع اجرای شبیه‌سازی run_isolated_simulation

این تابع پیچیده‌ترین بخش ماژول است و یک نشست شبیه‌سازی مستقل روی موتور main_optimized.py اجرا می‌کند. این تابع با دریافت یک دیکشنری config_overrides، پیکربندی پیش‌فرض را با مقادیر دلخواه بازنویسی می‌کند و شبیه‌سازی را با پارامترهای ایزوله اجرا می‌نماید.

```

۱ def run_isolated_simulation(config_overrides):
۲     import main_optimized as m
۳     cfg = copy.deepcopy(m.DEFAULT_CONFIG)
۴
۵     N_pulses = 100000
۶     dist = config_overrides.get("dist", 0)
۷
۸     cfg["simulation"]["apply_statistical_noise"] = True
۹     cfg["simulation"]["distance_start_km"] = dist
۱۰    cfg["simulation"]["distance_stop_km"] = dist

```

```

۱۱     cfg["simulation"]["distance_points"] = 1
۱۲     cfg["simulation"]["min_pulses_log"] = 5
۱۳     cfg["simulation"]["max_pulses_log"] = 5
۱۴
۱۵     mu = config_overrides.get("mu", (5.0
۱۶     cfg["source"]["statistics_type"] = config_overrides.get("stat_type", "POISSON")
۱۷     cfg["source"]["intensity_jitter"] = 0.0
۱۸
۱۹     cfg["source"]["pulses"] = {
۲۰         "signal": {"mu": mu, "prob": {0.1,
۲۱         "decoy": {"mu": 1.0, "prob": {0.0,
۲۲         "vacuum": {"mu": 0.0, "prob": {0.0
۲۳     }
۲۴
۲۵     eta_det = config_overrides.get("det_eff", (15.0
۲۶     Y0 = config_overrides.get("Y0", 3e-6)
۲۷     ed = config_overrides.get("ed", (01.0
۲۸
۲۹     cfg["detector"]["det_eff_d0"] = eta_det
۳۰     cfg["detector"]["det_eff_d1"] = eta_det
۳۱     cfg["detector"]["dark_rate"] = Y0
۳۲     cfg["detector"]["dark_rate_d1"] = Y0
۳۳     cfg["detector"]["qber_intrinsic"] = ed
۳۴     cfg["detector"]["misalignment"] = 0.0
۳۵
۳۶     cfg["detector"]["dead_time_ns"] = config_overrides.get("dt", (0.0
۳۷     cfg["detector"]["dead_time_model"] = "NON_PARALYZABLE"
۳۸
۳۹     m.run_and_save_csv(cfg, num_workers=1, sweeps={})
۴۰
۴۱     with open("qkd_results_optimized_sweep.csv", "r") as f:
۴۲         rows = list(csv.DictReader(f))
۴۳         latest = rows[-1]
۴۴
۴۵     y_sig = float(latest.get("detection_yield_signal", 0))
۴۶     q_sig = float(latest.get("qber_signal", 0))
۴۷     t_pulse = float(cfg["source"]["pulse_period_ns"])
۴۸
۴۹     return {
۵۰         "clicks": int(y_sig * N_pulses),
۵۱         "errors": int(q_sig * y_sig * N_pulses),
۵۲         "pulse_period": t_pulse
۵۳     }
۵۴

```

تحلیل این تابع به شرح زیر است. در خط اول، ماژول main_optimized به صورت محلی (lazy import) بارگذاری می‌شود؛ این الگو امکان اجرای ماژول validate_physics را حتی در صورتی که main_optimized در دسترس نباشد، برای توابع تحلیلی فراهم می‌سازد. در خط دوم، یک کپی عمیق از پیکربندی پیش‌فرض DEFAULT_CONFIG ساخته می‌شود. استفاده از deepcopy ضروری است زیرا پیکربندی یک دیکشنری تودرتو است و کپی سطحی باعث به اشتراک‌گذاری زیردیکشنری‌ها می‌شود که می‌تواند به تغییرات ناخواسته در پیکربندی پیش‌فرض منجر گردد.

در خطوط بعدی، پیکربندی شبیه‌سازی به صورت ایزوله تنظیم می‌شود: apply_statistical_noise = True فعال‌سازی نویز آماری، distance_start_km = distance_stop_km = dist، اجرای شبیه‌سازی در یک فاصله منفرد (بدون sweep)، distance_points = 1 تولید تنها یک نقطه داده، و min_pulses_log = 5 و max_pulses_log = 10⁵ تنظیم تعداد پالس‌ها به 100000.

یکی از نکات کلیدی این تابع، تنظیم prob = 1.0 برای سیگنال و prob = 0.0 برای decoy و vacuum است. این تنظیم به این معناست که تمام پالس‌ها به‌عنوان سیگنال ارسال می‌شوند و هیچ پالس decoy یا vacuum وجود ندارد. این ایزوله‌سازی ضروری است زیرا معادلات تحلیلی مرجع (مانند Q_mu_poisson) برای پالس سیگنال خالص به‌دست آمده‌اند و حضور decoy یا vacuum نیاز به تجمیع متفاوت دارد.

در بخش آشکارساز، چهار پارامتر کلیدی تنظیم می‌شوند: det_eff_d0 و det_eff_d1 بازده آشکارساز D0 و D1، dark_rate و dark_rate_d1 نرخ دارک کانت هر آشکارساز، qber_intrinsic خطای ذاتی، و misalignment = 0.0 حذف خطای misalignment (زیرا این کمیت به صورت جداگانه در پارامتر ed کنترل می‌شود).

در خط فراخوانی run_and_save_csv، شبیه‌سازی با پیکربندی نهایی اجرا می‌شود و خروجی در فایل qkd_results_optimized_swap.csv ذخیره می‌گردد. سپس این فایل خوانده شده و آخرین ردیف (که نتیجه شبیه‌سازی است) استخراج می‌شود. در نهایت، سه مقدار کلیدی برگردانده می‌شود: تعداد کلیک‌ها (از ضرب

detection_yield_signal در تعداد پالس‌ها)، تعداد خطاها (از ضرب QBER در کلیک‌ها)، و بازه زمانی پالس (برای محاسبه dead time).

۴.۱۳ پیاده‌سازی ریاضی و الگوریتمی

۱.۴.۱۳ الگوریتم TEST ۱: اعتبارسنجی منبع Poisson

این آزمون، منبع WCP با توزیع Poisson را در سه فاصله 0، 50 و 100 کیلومتر اعتبارسنجی می‌کند. برای هر فاصله، دو کمیت مستقل آزمون می‌شود: Gain (نرخ کلیک) و QBER (نرخ خطای بیت).

```

1 print("\n[TEST 1] Poisson WCP Asymptotic")
2 for d in [0, 50, 100]:
3     res = run_isolated_simulation({
4         "dist": d, "mu": 5.0, "stat_type": "POISSON",
5         "det_eff": eta_det, "Y0": Y0, "ed": ed, "dt": 0.0
6     })
7     eta = channel_transmittance(d, (2.0 * eta_det
8         Q_th = Q_mu_poisson5.0), eta, Y0) * 5.0
9         E_th = E_mu_poisson5.0), eta, Y0, ed)
10    validate_statistic(res["clicks"], N, Q_th, f"WCP Gain L={d}km")
11    validate_statistic(res["errors"], res["clicks"], E_th, f"WCP QBER L={d}km")
12

```

تحلیل ریاضی این آزمون به شرح زیر است. در هر تکرار، ابتدا شبیه‌سازی با پارامترهای ایزوله اجرا می‌شود و تعداد کلیک‌ها و خطاها به دست می‌آید. سپس ضریب انتقال تئوریک از ضرب ضریب انتقال کانال (با $\alpha_{\text{dB/km}} = 0.2$) در بازده آشکارساز محاسبه می‌شود:

$$\eta_{\text{total}}(d) = \eta_{\text{det}} \cdot 10^{-\frac{0.2 \cdot d}{10}} \quad (15.13)$$

برای $d = 0$ کیلومتر: $\eta_{\text{total}} = 0.15$ ، برای $d = 50$ کیلومتر: $\eta_{\text{total}} = 0.015$ و برای $d = 100$ کیلومتر: $\eta_{\text{total}} = 0.0015$

سپس Gain تئوریک از فرمول $Q_mu_poisson$ محاسبه می‌شود و در 0.5 ضرب می‌گردد تا اثر sifting اعمال شود. این ضرب در 0.5 به این دلیل است که در BB84، گیرنده به صورت تصادفی یکی از دو پایه را انتخاب می‌کند و تنها در نیمی از موارد، پایه گیرنده با فرستنده هم‌خوان است (sifting). در نتیجه، نرخ کلیک مشاهده‌شده نیمی از Gain خام است.

QBER تئوریک از فرمول $E_mu_poisson$ به دست می‌آید. نکته مهم آن است که در این محاسبه، sifting اعمال نمی‌شود زیرا QBER نسبت خطا به کل کلیک‌ها است و این نسبت تحت sifting تغییر نمی‌کند (هم خطاها و هم کل کلیک‌ها به همان نسبت 0.5 کاهش می‌یابند).

دو آزمون آماری جداگانه انجام می‌شود: یکی برای Gain (با $N = 100000$ آزمایش و احتمال تئوریک Q_{th}) و دیگری برای QBER (با N تعداد کلیک‌ها و احتمال تئوریک E_{th}). در آزمون QBER، تعداد آزمایش‌ها به تعداد کلیک‌ها تغییر می‌کند زیرا QBER بر روی کلیک‌ها اعمال می‌شود، نه بر روی کل پالس‌ها. این تمایز ظریف برای صحت آماری آزمون ضروری است.

۲.۴.۱۳ الگوریتم TEST ۲: اعتبارسنجی منبع Thermal

این آزمون، منبع حرارتی تک‌حالت را در فاصله صفر اعتبارسنجی می‌کند. این آزمون کوتاه‌تر از TEST 1 است زیرا فقط Gain بررسی می‌شود و فقط در یک فاصله.

```

1 print("\n[TEST 2] Thermal Single-Mode")
2 res = run_isolated_simulation({
3     "dist": 0, "mu": 5.0, "stat_type": "THERMAL",
4     "det_eff": eta_det, "Y0": Y0, "dt": 0.0
5 })
6 Q_th = Q_mu_thermal5.0), eta_det, Y0, M=1) * 5.0
7 validate_statistic(res["clicks"], N, Q_th, "Thermal Gain L=0km")
8

```


در این آزمون، پارامتر "THERMAL" stat_type = به موتور شبیه‌سازی دستور می‌دهد که به جای توزیع Poisson، از توزیع Bose-Einstein تک‌حالت برای تولید فوتون استفاده کند. سپس Gain تئوریک از فرمول $Q_{\mu_thermal}$ با $M = 1$ محاسبه می‌شود:

$$Q_{\mu}^{Thermal}(\mu = 0.5, \eta = 0.15, Y_0 = 3 \times 10^{-6}, M = 1) = 1 - \frac{1 - 3 \times 10^{-6}}{1 + 0.5 \times 0.15} \approx 0.075 \quad (16.13)$$

این مقدار در 0.5 ضرب می‌شود تا sifting اعمال شود و در نتیجه $Q_{th} \approx 0.0375$ به دست می‌آید. این یعنی از 100000 پالس، انتظار می‌رود حدود 3750 کلیک ثبت شود. نکته فیزیکی قابل توجه: برای همان μ و η ، منبع حرارتی بزرگ‌تر از Gain منبع Poisson است. این به دلیل پدیده bunching در نور حرارتی است که احتمال پالس‌های چندفوتونی را افزایش می‌دهد. در مقابل، نور Poisson دارای آمار coherent است که در آن فوتون‌ها مستقل توزیع می‌شوند. این تفاوت آماری یکی از مفاهیم بنیادین اپتیک کوانتومی است که در آزمون ما به صورت کمی نشان داده می‌شود.

۳.۴.۱۳ الگوریتم TEST ۳: اعتبارسنجی Time Dead مستقل SPAD

این آزمون پیچیده‌ترین آزمون ماژول است و فیزیک dead time در حضور دو آشکارساز مستقل را بررسی می‌کند. این آزمون به طور خاص این موضوع مهم را راستی‌آزمایی می‌کند که هر SPAD به طور مستقل زمان مرده خود را تجربه می‌کند، نه اینکه زمان مرده روی کل سیستم اعمال شود.

```

1 print("\n[TEST 3] Dead-Time Physics (Independent SPADs)")
2 dt_ns = 0.50
3 res = run_isolated_simulation({
4     "dist": 0, "mu": 5.0, "stat_type": "POISSON",
5     "det_eff": eta_det, "Y0": Y0, "dt": dt_ns
6 })
7
8 #           :           SPAD
9 Q_ideal_raw = Q_mu_poisson(5.0), eta_det, Y0)
10 Q_per_spad = Q_ideal_raw / 0.2
11 T_pulse = res["pulse_period"]
12
13 Q_dead_per_spad = dead_time_correction(Q_per_spad, dt_ns, T_pulse)
14 Q_dt_exp = Q_dead_per_spad * 0.2 * 5.0 # 2 SPADs * Sifting probability
15
16 validate_statistic(res["clicks"], N, Q_dt_exp, f"Dead-time tau={dt_ns}ns")
17

```

تحلیل ریاضی این آزمون به شرح زیر است. ابتدا Gain خام ایده‌آل (بدون dead time) از فرمول $Q_{\mu_poisson}$ محاسبه می‌شود. سپس این مقدار به دو نیمه تقسیم می‌شود زیرا در سیستم BB84، هر کلیک به صورت تصادفی به یکی از دو آشکارساز D0 یا D1 می‌رود (با احتمال برابر 0.5). در نتیجه، Gain هر آشکارساز برابر با $Q_{per-spad} = Q_{ideal}^{raw}/2$ است. سپس تصحیح dead time به صورت مستقل روی هر آشکارساز اعمال می‌شود:

$$Q_{obs}^{per-spad} = \frac{Q_{per-spad}}{1 + Q_{per-spad} \cdot \frac{\tau_d}{T_{pulse}}} \quad (17.13)$$

در نهایت، برای محاسبه Gain کل سیستم، مقدار هر آشکارساز در 2 ضرب می‌شود (چون دو آشکارساز مستقل وجود دارد) و سپس در 0.5 ضرب می‌شود تا اثر sifting اعمال گردد:

$$Q_{final} = Q_{obs}^{per-spad} \cdot 2 \cdot 0.5 = Q_{obs}^{per-spad} \quad (18.13)$$

این رابطه جالب نشان می‌دهد که در نهایت، Gain مشاهده‌شده با Gain هر آشکارساز (پس از تصحیح dead time) برابر است. این ساده‌سازی ریاضی به این دلیل رخ می‌دهد که عوامل 2 (برای دو آشکارساز) و 0.5 (برای sifting) را خنثی می‌کنند. با این حال، ترتیب اعمال این عوامل مهم است: ابتدا باید dead time روی هر آشکارساز به صورت مستقل اعمال شود و سپس جمع انجام گردد. اگر به اشتباه dead time روی Gain کل اعمال شود، نتیجه متفاوت و نادرستی به دست می‌آید.

برای درک کمی، فرض کنید $Q_{\text{ideal}}^{\text{raw}} = 0.075$ (مقدار تقریبی برای $\eta = 0.15$, $\mu = 0.5$, $Y_0 = 3 \times 10^{-6}$ در فاصله صفر). آنگاه $Q_{\text{per-spade}} = 0.0375$. اگر $\tau_d = 50$ نانوثانیه و $T_{\text{pulse}} = 1000$ نانوثانیه (نرخ پالس 1 مگاهرتز) باشد:

$$Q_{\text{obs}}^{\text{per-spade}} = \frac{0.0375}{1 + 0.0375 \times \frac{50}{1000}} = \frac{0.0375}{1.001875} \approx 0.03743 \quad (19.13)$$

این نشان می‌دهد که در این شرایط، dead time باعث کاهش $\approx 0.18\%$ در Gain می‌شود. هرچند این کاهش کوچک به نظر می‌رسد، اما در آزمون آماری با $N = 100000$ قابل شناسایی است زیرا انحراف معیار تئوریک برابر با ≈ 60 برابر $\sqrt{100000 \times 0.0375 \times 0.9625} \approx 60$ است و انحراف $\approx 0.18\%$ از 0.0375 معادل ≈ 6.75 کلیک است که بسیار کوچک‌تر از $233 \approx 3.89 \times 60$ (Z-crit) است. اما اگر dead time به اشتباه روی کل Gain اعمال شود، خطا به مراتب بزرگ‌تر خواهد بود و آزمون شکست می‌خورد.

۴.۴.۱۳ محاسبه Q-score و آستانه بحرانی

انتخاب آستانه بحرانی $Z_{\text{crit}} = 3.8906$ بر اساس سطح معنی‌داری $\alpha = 10^{-4}$ دوسویه است. این مقدار از معادله زیر به دست می‌آید:

$$\alpha = P(|Z| > Z_{\text{crit}} | H_0) = 2 \cdot \Phi(-Z_{\text{crit}}) \quad (20.13)$$

که در آن Φ تابع توزیع تجمعی نرمال استاندارد است. با حل معادله $\Phi(-Z_{\text{crit}}) = 5 \times 10^{-5}$ ، مقدار $Z_{\text{crit}} \approx 3.8906$ به دست می‌آید. این مقدار در کد به صورت یک ثابت سخت‌کد شده (hard-coded) تعریف شده است.

در عمل، انتخاب این سطح سخت‌گیرانه دو دلیل دارد. نخست، در شبیه‌سازی QKD با $N = 100000$ پالس، انحراف معیار نسبی برابر با $\sqrt{p(1-p)/N} \approx 0.0016$ برای $p \approx 0.04$ است. این یعنی نویز آماری ذاتی شبیه‌سازی در حد 0.16% است و انحرافات بزرگ‌تر از این مقدار باید شناسایی شوند. با $Z_{\text{crit}} = 3.89$ ، آستانه شناسایی برابر با $0.62\% \approx 3.89 \times 0.16\%$ است که به طور معناداری بزرگ‌تر از نویز است. دوم، این سطح به قدری سخت‌گیرانه است که false positive بسیار نادر است (احتمال $\alpha = 10^{-4}$ یعنی یک شکست در هر 10000 اجرای مستقل) اما هنوز به اندازه کافی حساس است که باگ‌های واقعی را شناسایی کند.

۵.۴.۱۳ تحلیل پیچیدگی و کارایی

از نظر پیچیدگی زمانی، تمام توابع تحلیلی (`Q_mu_thermal`, `Q_mu_poisson`, `channel_transmittance`)، `dead_time_correction`، `E_mu_poisson` در زمان $\mathcal{O}(1)$ اجرا می‌شوند زیرا تنها شامل چند عملیات حسابی ساده هستند. تابع `validate_statistic` نیز در زمان $\mathcal{O}(1)$ اجرا می‌شود.

تابع `run_isolated_simulation` پیچیده‌تر است: زمان اجرای آن به زمان اجرای `run_and_save_csv` در `main_optimized.py` بستگی دارد که خود به تعداد پالس‌ها N و پیچیدگی خط لوله شبیه‌سازی وابسته است. برای $N = 100000$ پالس، این زمان معمولاً در حد چند ثانیه تا چند ده ثانیه است. خواندن فایل CSV در زمان $\mathcal{O}(R)$ اجرا می‌شود که در آن R تعداد ردیف‌های فایل است (که در این حالت تنها 1 ردیف است زیرا sweep خالی است).

از نظر پیچیدگی حافظه، ماژول بسیار سبک است؛ تنها ساختار داده‌های قابل توجه دیکشنری `cfg` (با اندازه ثابت) و لیست `rows` (با اندازه 1) هستند. در نتیجه، کل ماژول می‌تواند به راحتی در محیط‌های با حافظه محدود اجرا شود.

۵.۱۳ ارسال و دریافت با سایر ماژول‌ها

۱.۵.۱۳ وابستگی به `main_optimized.py`

مهم‌ترین وابستگی خارجی این ماژول، به فایل `main_optimized.py` است که به عنوان موتور اصلی شبیه‌سازی عمل می‌کند. این وابستگی از طریق سه مؤلفه کلیدی برقرار می‌شود: `DEFAULT_CONFIG` به عنوان پیکربندی پایه، `run_and_save_csv` به عنوان نقطه ورود شبیه‌سازی، و فرمت فایل CSV خروجی به عنوان پروتکل ارتباطی.


```

۱ import main_optimized as m
۲ cfg = copy.deepcopy(m.DEFAULT_CONFIG)
۳ # ... (modify cfg) ...
۴ m.run_and_save_csv(cfg, num_workers=1, sweeps={})
۵

```

این الگوی وابستگی به این معناست که هرگونه تغییر در DEFAULT_CONFIG (مانند افزودن فیلدهای جدید یا تغییر نام فیلدها) یا در فرمت خروجی CSV (مانند تغییر نام ستون‌ها) می‌تواند به شکست ماژول validate_physics منجر شود. برای کاهش این ریسک، ماژول از کلیدهای رشته‌ای (string keys) با مقادیر پیش‌فرض استفاده می‌کند (مانند latest.get("detection_yield_signal", 0)) که در صورت نبود یک کلید، مقدار پیش‌فرض برمی‌گردد و از خطای KeyError جلوگیری می‌کند.

۲.۵.۱۳ ورودی‌های ماژول

ورودی اصلی ماژول از طریق دیکشنری config_overrides در تابع run_isolated_simulation ارائه می‌شود. این دیکشنری شامل هفت کلید اختیاری است: dist (فاصله به کیلومتر، پیش‌فرض 0)، mu (میانگین فوتون، پیش‌فرض 0.5)، stat_type (نوع توزیع آماری، پیش‌فرض "POISSON")، det_eff (بازده آشکارساز، پیش‌فرض 0.15)، Y0 (نرخ دارک کانت، پیش‌فرض 3×10^{-6})، ed (خطای ذاتی، پیش‌فرض 0.01) و dt (زمان مرده به نانوثانیه، پیش‌فرض 0).

این الگو (configuration override pattern) مزایای متعددی دارد. نخست، امکان تنظیم دقیق پارامترهای شبیه‌سازی برای هر آزمون به صورت مستقل فراهم می‌شود. دوم، مقادیر پیش‌فرض معنی‌دار هستند و در صورت عدم ارائه یک کلید، مقدار معقولی استفاده می‌شود. سوم، افزودن پارامترهای جدید در آینده بدون شکستن کدهای فراخوان موجود امکان‌پذیر است.

در سطح بالاتر، ماژول سه گروه پارامتر ثابت در بدنه اصلی (__main__) تعریف می‌کند: $N = 100000$ (تعداد پالس‌ها)، $\eta_{\text{det}} = 0.15$ (بازده آشکارساز)، $Y_0 = 3 \times 10^{-6}$ (نرخ دارک کانت) و $e_d = 0.01$ (خطای ذاتی). این مقادیر به عنوان golden parameters عمل می‌کنند و در تمام آزمون‌ها به صورت ثابت استفاده می‌شوند. ثابت نگه‌داشتن این پارامترها ضروری است زیرا تغییر آن‌ها می‌تواند به نتایج متفاوتی منجر شود که مقایسه با مقادیر تئوریک را دشوار سازد.

۳.۵.۱۳ خروجی‌های ماژول

خروجی اصلی ماژول به دو صورت است: خروجی متنی در stdout و مقدار بازگشتی بولی از تابع validate_statistic. خروجی متنی شامل یک جدول قالب‌بندی‌شده است که برای هر آزمون، نام، احتمال تئوریک، احتمال شبیه‌سازی‌شده، Z-score و نتیجه FAIL/PASS را نمایش می‌دهد. این خروجی برای تحلیل بصری توسط توسعه‌دهنده طراحی شده است.

علاوه بر خروجی متنی، تابع validate_statistic مقدار بولی passed را برمی‌گرداند که می‌تواند برای اتوماسیون در CI/CD pipeline استفاده شود. در یکپارچه‌سازی مستمر، اسکریپت می‌تواند تمام مقادیر بازگشتی را جمع کرده و در صورت وجود حتی یک FAIL، کد خطا برگرداند تا pipeline متوقف شود. این الگو (exit code based gating) در ابزارهایی مانند GitHub Actions و GitLab CI به صورت استاندارد پشتیبانی می‌شود.

تابع run_isolated_simulation نیز یک دیکشنری با سه کلید برمی‌گرداند: clicks (تعداد کلیک‌های ثبت‌شده)، errors (تعداد خطاهای ثبت‌شده) و pulse_period (بازه زمانی پالس به نانوثانیه). این مقادیر به عنوان ورودی به توابع آزمون آماری استفاده می‌شوند و امکان محاسبه مقادیر تئوریک متناظر را فراهم می‌سازند.

۴.۵.۱۳ ارتباط با فایل CSV خروجی

یکی از جنبه‌های مهم ارتباط با موتور شبیه‌سازی، فرمت فایل CSV خروجی است. پس از اجرای run_and_save_csv، نتایج در فایل qkd_results_optimized_sweep.csv ذخیره می‌شوند و ماژول validate_physics این فایل را می‌خواند.

```

۱ with open("qkd_results_optimized_sweep.csv", "r") as f:
۲     rows = list(csv.DictReader(f))
۳     latest = rows[-1]
۴
۵     y_sig = float(latest.get("detection_yield_signal", 0))

```

```

۶ q_sig = float(latest.get("qber_signal", 0))
۷ t_pulse = float(cfg["source"]["pulse_period_ns"])
۸

```

استفاده از csv.DictReader به این معناست که فایل CSV باید دارای سطر سرستون باشد و نام ستون‌ها باید دقیقاً با کلیدهای مورد انتظار (qber_signal و detection_yield_signal) مطابقت داشته باشد. این یک قرارداد ارتباطی (interface contract) بین دو ماژول است که باید در صورت تغییر فرمت CSV، به‌روزرسانی شود. نکته طراحی مهم: استفاده از rows[-1] برای گرفتن آخرین ردیف به این دلیل است که در حالت sweep با چندین نقطه فاصله، چندین ردیف تولید می‌شود. در این ماژول، چون sweep خالی است و فقط یک فاصله مشخص شده، تنها یک ردیف تولید می‌شود و rows[-1] همان ردیف اول است. اما استفاده از rows[-1] به جای rows[0] یک الگوی دفاعی است که در صورتی که به‌اشتباه چندین ردیف تولید شود، آخرین ردیف (که معمولاً متناظر با آخرین فاصله است) را برمی‌گرداند.

نکته دیگر آن است که pulse_period از cfg خوانده می‌شود، نه از فایل CSV. این به این دلیل است که pulse_period یک پارامتر پیکربندی است که در طول شبیه‌سازی ثابت می‌ماند و نیازی به ذخیره در خروجی ندارد. با این حال، این الگو به یکپارچگی داده‌ها آسیب می‌زند زیرا اگر cfg پس از اجرای شبیه‌سازی تغییر کند، مقدار خوانده‌شده ممکن است با مقدار واقعی استفاده‌شده در شبیه‌سازی متفاوت باشد. در یک طراحی بهتر، این مقدار نیز در فایل CSV ذخیره می‌شد.

۵.۵.۱۳ یکپارچه‌سازی با سیستم CI/CD

این ماژول به‌عنوان یک test suite مستقل عمل می‌کند و می‌تواند به‌راحتی در سیستم‌های CI/CD یکپارچه شود. الگوی متداول استفاده به این صورت است:

```

۱ # In CI script:
۲ python validate_physics.py
۳ if [ $? -ne 0 ]; then
۴     echo "Physics validation failed!"
۵     exit 1
۶ fi
۷

```

با این حال، در نسخه فعلی ماژول، کد خروج (exit code) به‌صورت صریح تنظیم نمی‌شود و حتی در صورت شکست آزمون‌ها، اسکریپت با کد ۰ (موفقیت) خارج می‌شود. این یک محدودیت است که در نسخه‌های آینده باید اصلاح شود؛ به‌عنوان مثال، می‌توان در انتهای بدنه اصلی، تعداد شکست‌ها را شمارش کرده و در صورت بزرگ‌تر بودن از صفر، با sys.exit(1) خارج شد.

۶.۵.۱۳ محدودیت‌ها و توسعه‌های آینده

این ماژول دارای چند محدودیت است که در توسعه‌های آینده قابل بهبود است. نخست، تعداد پالس‌ها $N = 100000$ ثابت است و قابل تنظیم نیست؛ برای آزمون‌های حساس‌تر، می‌توان N را به 10^6 یا 10^7 افزایش داد، هرچند این به زمان اجرای طولانی‌تر نیاز دارد. دوم، تنها سه آزمون تعریف شده است و سناریوهای پیشرفته‌تری مانند decoy-state کامل، afterpulse، و polarization drift پوشش داده نمی‌شوند. سوم، خروجی به‌صورت متنی است و برای تحلیل خودکار نیاز به تجزیه دارد؛ می‌توان خروجی را به فرمت JSON یا JUnit XML تغییر داد تا با ابزارهای CI بهتر یکپارچه شود. چهارم، آزمون‌ها به‌صورت قطعی (deterministic) هستند زیرا از بذر تصادفی ثابت استفاده نمی‌کنند؛ در نتیجه، در اجراهای مختلف ممکن است نتایج کمی متفاوت باشند. برای اصلاح این، می‌توان از SeedSequence با بذر ثابت استفاده کرد.

علاوه بر این، ماژول می‌تواند با معرفی parametrized tests گسترش یابد؛ به‌عنوان مثال، می‌توان به‌جای سه فاصله ثابت در TEST 1، یک sweep روی بازه [0, 100] کیلومتر با گام 10 کیلومتر انجام داد و آزمون را برای هر نقطه تکرار کرد. این الگو (parameterized testing) در فریم‌ورک‌هایی مانند pytest به‌صورت استاندارد پشتیبانی می‌شود و می‌تواند اعتبار فیزیکی ماژول را در طیف وسیع‌تری از پارامترها تضمین نماید.

در نهایت، این ماژول به‌عنوان یک golden reference برای فیزیک QKD در فریم‌ورک عمل می‌کند و هرگونه تغییر در پیاده‌سازی موتور فیزیکی باید پیش از merge با این آزمون‌ها اعتبارسنجی شود. این رویکرد shift-left testing تضمین می‌کند که باگ‌های فیزیکی در همان مراحل اولیه توسعه شناسایی شوند و به انتشار نتایج نادرست در مقالات یا گزارش‌های فنی منجر نشوند. وجود چنین ماژول اعتبارسنجی مستقل، نشانه بلوغ

مهندسی نرم افزار در پروژه های علمی است و در پروژه های تحقیقاتی جدی به عنوان یک ضرورت در نظر گرفته می شود.

فصل ۱۴

اعتبارسنجی رمزنگاری و تحلیلی

۱.۱۴ نمای کلی و هدف ماژول

ماژول `absolute_crypto_validation.py` یک سویییت اعتبارسنجی مطلق کریپتوگرافیک است که نقشی متمایز و مکمل نسبت به سایر اسکریپت‌های آزمون در فریم‌ورک شبیه‌سازی QKD ایفا می‌کند. در حالی که ماژول `validate_physics.py` بر آزمون آماری توزیع‌های منبع (Thermal و Poisson) متمرکز است و از آزمون Z-test دودویی برای راستی‌آزمایی Gain و QBER استفاده می‌نماید، این ماژول به صفت کریپتوگرافیک هسته‌ای پروتکل می‌پردازد: آیا کمیت‌های بنیادین امنیت کلید نظیر single-photon yield (Y_1)، single-photon QBER، و (e_1) و secure key rate (R) که موتور کامل شبیه‌سازی محاسبه می‌کند، با معادلات تحلیلی بسته‌ای که در ادبیات کلاسیک QKD decoy-state (به‌ویژه کار مرجع Lo-Ma-Chen 2005) پذیرفته شده‌اند، انطباق عددی دارند؟

هدف اصلی این ماژول، پاسخ به پرسش زیر است: آیا پیاده‌سازی کامل موتور شبیه‌سازی، از منبع WCP و کانال فیبر و آشکارساز $SPAD$ و $sifting$ پایه‌ها و پروتکل $BB84$ -Decoy و اثبات امنیتی $Lim-2014$ تا استخراج کلید نهایی، خروجی عددی تولید می‌کند که از نظر کریپتوگرافیک معتبر باشد؟ این پرسش از آنجا اهمیت می‌یابد که در شبیه‌سازی‌های عددی پیچیده، خطاهای ظریف در پیاده‌سازی – مانند اشتباه در فرمول Bayes برای استخراج Y_1 ، یا نادیده گرفتن کسر q_{sift} در نرخ کلید، یا اشتباه در نرمال‌سازی QBER – می‌توانند منجر به نتایجی شوند که از نظر QBER ظاهراً قابل‌قبول‌اند اما از نظر امنیت کریپتوگرافیک غیرقابل دفاع هستند. به‌عنوان مثال، اگر استخراج Y_1 به‌جای کران پایین Lo2005 از کران بالا استفاده کند، شبیه‌سازی نرخ کلیدی گزارش می‌دهد که در عمل قابل‌دستیابی نیست و هر مقاله‌ای که بر این پایه منتشر شود، از نظر رمزنگاری قابل نقض خواهد بود.

ساختار کلی این ماژول شامل سه تابع تحلیلی و یک تابع اجرای آزمون است. تابع $H2$ آنتروپی باینری شانون را با برش عددی (numerical clipping) پیاده‌سازی می‌کند. تابع `channel_transmittance` ضریب انتقال کانال فیبر را بر اساس قانون Beer-Lambert محاسبه می‌نماید. تابع `get_analytics` مجموعه‌ای از پنج کمیت تحلیلی کلیدی – Q_μ ، E_μ ، Y_1 ، e_1 و R_{asympt} – را بر اساس معادلات بسته decoy-state محاسبه می‌کند و آن‌ها را در قالب یک دیکشنری برمی‌گرداند. تابع `run_absolute_test` آزمون اصلی است که با مقداردهی پیکربندی پیش‌فرض `DEFAULT_CONFIG` از ماژول `main_optimized` اجرای شبیه‌سازی کامل، خواندن خروجی CSV و مقایسه با مقادیر تحلیلی، در پنج پارامتر با تلورانس‌های متفاوت، نتیجه نهایی `FAIL/PASS` را اعلام می‌کند. ویژگی برجسته این ماژول، تفکیک تلورانس‌ها بر اساس حساسیت کریپتوگرافیک است. تلورانس 25% برای Q_μ به‌کار رفته است، زیرا این کمیت در شبیه‌سازی مونته‌کارلو با 10^7 پالس، نویز آماری قابل‌توجهی دارد و انحراف از مقدار تحلیلی طبیعی است. در مقابل، تلورانس 1% برای Y_1 ، e_1 و R_{asympt} به‌کار رفته، چون این کمیت‌ها از ترکیب خطی چندین کمیت به دست می‌آیند و خطای سیستماتیک در هر یک از آن‌ها می‌تواند به خطای بزرگ در نرخ کلید منجر شود. این تلورانس‌های سخت‌گیرانه، تضمین می‌کنند که هرگونه regression در پیاده‌سازی موتور – مثلاً پس از refactor کلاس اثبات یا تغییر مدل آشکارساز – بلافاصله شناسایی شود.

در سطح عملیاتی، این ماژول به‌عنوان یک cryptographic gate عمل می‌کند؛ یعنی پیش از merge هر تغییر در شاخه اصلی و پیش از انتشار هر نتیجه شبیه‌سازی در مقالات یا گزارش‌های فنی، این اسکریپت باید با موفقیت اجرا شود. خروجی کد خروج (exit code) این اسکریپت نیز به‌گونه‌ای طراحی شده که در CI/CD pipeline قابل استفاده باشد: در صورت موفقیت همه آزمون‌ها، کد خروج 0 و در غیر این صورت، کد خروج 1 بازگردانده می‌شود. این الگو در مهندسی نرم‌افزار علمی به‌عنوان cryptographic regression testing شناخته می‌شود و در پروژه‌های بالگی مانند Open Quantum Safe و Qiskit به‌صورت استاندارد به کار می‌رود.

نکته مهم دیگر آن است که این ماژول بدون اعمال هرگونه *patch* روی موتور اصلی، آن را آزمون می‌کند. به عبارت دیگر، برخلاف برخی اسکریپت‌های آزمون که با monkey-patching یا mock کردن بخش‌هایی از کد، آن را برای آزمون قابل‌کنترل می‌کنند، این ماژول از همان مسیر اجرای واقعی `main_optimized.run_and_save_csv` استفاده می‌کند و فقط پیکربندی را `override` می‌نماید. این رویکرد تضمین می‌کند که آزمون، رفتار واقعی موتور را در شرایط عملیاتی منعکس می‌کند و نه رفتار آزمایشگاهی مصنوعی.

۲.۱۴ مبانی تئوری و فیزیکی

۱.۲.۱۴ آنتروپی باینری شانون و نقش آن در امنیت QKD

آنتروپی باینری شانون، یکی از مفاهیم بنیادین نظریه اطلاعات است که در رمزنگاری کوانتومی نقشی مرکزی ایفا می‌کند. این کمیت که با $H_2(p)$ نشان داده می‌شود، میزان عدم قطعیت (uncertainty) متغیر تصادفی باینری با توزیع Bernoulli را اندازه می‌گیرد و به صورت زیر تعریف می‌شود:

$$H_2(p) = -p \log_2(p) - (1-p) \log_2(1-p) \quad (1.14)$$

در این رابطه، $p \in [0, 1]$ احتمال وقوع یکی از دو حالت متغیر باینری است. تابع $H_2(p)$ تقارن نسبت به $p = 0.5$ دارد، در این نقطه به مقدار بیشینه 1 می‌رسد و در نقاط $p = 0$ و $p = 1$ به صفر میل می‌کند. این رفتار بازتاب‌دهنده این واقعیت استنباطی است که در حالت $p = 0.5$ ، متغیر تصادفی کاملاً غیرقابل پیش‌بینی است و در حالت‌های $p = 0$ یا $p = 1$ کاملاً قطعی است.

در رمزنگاری کوانتومی، $H_2(p)$ در دو نقطه بحرانی ظاهر می‌شود. نخست، در محاسبه privacy amplification که حجم اطلاعات نشت‌کرده به حمله‌ور (eavesdropper) را تعیین می‌کند؛ اگر QBER سیگنال E_μ باشد، آنتروپی شرطی شانون $H_2(E_\mu)$ حجم اطلاعات بیت-خطای حمله‌ور را تقریب می‌زند. دوم، در محاسبه e_1 single-photon error rate که آنتروپی $H_2(e_1)$ حجم اطلاعات حمله‌ور درباره فوتون‌های تک‌فوتونی را تعیین می‌کند و مستقیماً در نرخ کلید امن وارد می‌شود.

پیاده‌سازی تابع H_2 در کد، با برش عددی (numerical clipping) در نقاط $p = 0$ و $p = 1$ همراه است، زیرا در این نقاط، عبارت $\log_2(p)$ به $-\infty$ میل می‌کند و عبارت $p \log_2(p)$ به حالت تعریف نشده $\lim_{p \rightarrow 0^+} p \log_2 p = 0$ می‌رسد. در محاسبات عددی کامپیوتری، این حالت می‌تواند منجر به NaN یا exception شود؛ بنابراین پیاده‌سازی پیش از فراخوانی `math.log2`، شرایط $p \leq 0$ یا $p \geq 1$ را بررسی کرده و در این موارد، مقدار 0 را برمی‌گرداند. این برش با حد گسسته L'Hôpital هم‌خوان است و از نظر ریاضی صحیح است.

۲.۲.۱۴ کاهیدگی فیبر و قانون Beer-Lambert

انتقال فوتون در فیبر نوری تک‌حالت در طول‌موج مخابراتی 1550 نانومتر با قانون Beer-Lambert مدل می‌شود. این قانون که اصلی‌ترین مدل کاهیدگی در فیبرهای مخابراتی است، رابطه‌ای نمایی بین طول فیبر و ضریب انتقال برقرار می‌سازد. برای یک فیبر به طول L کیلومتر با ضریب کاهیدگی خطی $\alpha_{\text{dB/km}}$ ، کاهیدگی کل بر حسب دسی‌بل برابر با $\alpha_{\text{dB/km}} \cdot L$ است و تبدیل آن به ضریب انتقال خطی (power transmittance) از رابطه زیر پیروی می‌کند:

$$\eta_{\text{channel}}(L, \alpha_{\text{dB/km}}) = 10^{-\frac{\alpha_{\text{dB/km}} \cdot L}{10}} \quad (2.14)$$

در فیبر تک‌حالت استاندارد در 1550 نانومتر، $\alpha_{\text{dB/km}} \approx 0.2$ است؛ بنابراین در فاصله $L = 50$ کیلومتر (که نقطه آزمون پیش‌فرض این ماژول است)، ضریب انتقال کانال برابر با $10^{-1} = 0.1$ می‌شود. این کاهش نمایی، عامل اصلی محدودیت برد QKD در فیبر است و دلیل آن است که در فواصل بزرگ‌تر از 200 کیلومتر، حتی با decoy-state، نرخ کلید امن به صفر میل می‌کند. ضریب انتقال کلی سیستم شامل بازده آشکارساز η_{det} نیز می‌شود:

$$\eta_{\text{total}}(L) = \eta_{\text{det}} \cdot \eta_{\text{channel}}(L, \alpha_{\text{dB/km}}) \quad (3.14)$$

در این پیاده‌سازی، $\eta_{\text{det}} = 0.15$ که متناظر با InGaAs SPAD در دمای -50 درجه سانتی‌گراد است. با این مقادیر، در $L = 50$ کیلومتر، $\eta_{\text{total}} = 0.015$ می‌شود که نشان‌دهنده شرایط عملیاتی واقعی برای QKD در فواصل مخابراتی متوسط است.

۳.۲.۱۴ تئوری Decoy-State و معادلات Lo2005

ایده اصلی decoy-state QKD که توسط Hwang (2003) پیشنهاد و توسط Lo-Ma-Chen (2005) به فرم مدرن آن بسط داده شد، آن است که فرستنده با تغییر تصادفی شدت پالس (intensity) در میان چندین سطح (vacuum, decoy, signal)، امکان تخمین آماری ویژگی‌های کانال را بدون افشای اینکه کدام پالس حامل کلید است، فراهم می‌سازد. این تکنیک، حمله Photon-Number Splitting (PNS) را که علیه منبع WCP با توزیع Poisson قابل اعمال است، خنثی می‌کند و امکان دستیابی به نرخ کلیدی نزدیک به حد امن تک‌فوتونی آرمانی را فراهم می‌سازد.

در ادامه، معادلات بنیادین این تئوری را که همگی در تابع `get_analytics` پیاده‌سازی شده‌اند، معرفی می‌نماییم.

Gain سیگنال و QBER سیگنال

برای منبع WCP با توزیع Poisson با شدت میانگین μ ، Gain سیگنال (یعنی احتمال آنکه یک پالس منجر به ثبت کلیک در آشکارساز شود) برابر است با:

$$Q_{\mu}^{\text{raw}}(\mu, \eta, Y_0) = 1 - (1 - Y_0) \cdot e^{-\eta\mu} \quad (۴.۱۴)$$

در این رابطه، Y_0 نرخ dark count به ازای هر پالس (احتمال کلیک اشتباه در نبود فوتون واقعی)، η ضریب انتقال کلی سیستم و μ شدت میانگین سیگنال است. عبارت $e^{-\eta\mu}$ احتمال آنکه هیچ فوتونی شناسایی نشود (یعنی یا هیچ فوتونی ارسال نشده، یا فوتون‌های ارسالی از دست رفته‌اند) و عامل $(1 - Y_0)$ احتمال آنکه دارک کانت نیز رخ ندهد را نشان می‌دهد. بنابراین، $1 - (1 - Y_0)e^{-\eta\mu}$ احتمال متمم، یعنی احتمال آنکه حداقل یک کلیک ثبت شود، است.

در پروتکل BB84، فرستنده و گیرنده هر یک به‌طور تصادفی یکی از دو پایه (rectilinear یا diagonal) را انتخاب می‌کنند و فقط در حدود نیمی از پالس‌ها (یعنی زمانی که پایه‌ها هم‌خوان باشند) برای استخراج کلید مفید هستند. این کسر به sifting factor موسوم است:

$$q_{\text{sift}} = \frac{1}{2} \quad (۵.۱۴)$$

Gain سیگنال sifted شده برابر است با:

$$Q_{\mu}^{\text{sifted}} = q_{\text{sift}} \cdot Q_{\mu}^{\text{raw}} \quad (۶.۱۴)$$

QBER سیگنال، یعنی کسر کلیک‌های اشتباه در میان کلیک‌های ثبت‌شده، از رابطه زیر به دست می‌آید:

$$E_{\mu} = \frac{0.5 \cdot Y_0 + e_d \cdot (1 - e^{-\eta\mu})}{Q_{\mu}^{\text{raw}}} \quad (۷.۱۴)$$

در این رابطه، e_d خطای ذاتی سیستم (misalignment error) است که ناشی از عدم تقارن پایه‌ها، ناپایداری قطعات اپتیکی و نویز الکترونیکی است. در این پیاده‌سازی، $e_d = 0.01$ به کار رفته که در عمل برای سیستم‌های متعادل قابل دستیابی است. صورت این کسر از دو بخش تشکیل شده است: $0.5 \cdot Y_0$ که نشان‌دهنده کلیک‌های اشتباه ناشی از دارک کانت است (چون دارک کانت کاملاً تصادفی است، نصف آن در پایه درست و نصف در پایه اشتباه رخ می‌دهد) و $e_d \cdot (1 - e^{-\eta\mu})$ که نشان‌دهنده کلیک‌های اشتباه ناشی از فوتون‌های واقعی با احتمال خطای ذاتی e_d است.

Gain دکوی ضعیف

در پروتکل decoy-state، علاوه بر سیگنال با شدت μ ، از یک weak decoy با شدت $\nu < \mu$ نیز استفاده می‌شود. Gain این دکوی ضعیف به‌صورت مشابه محاسبه می‌شود:

$$Q_\nu(\nu, \eta, Y_0) = 1 - (1 - Y_0) \cdot e^{-\eta\nu} \quad (۸.۱۴)$$

این کمیت برای استخراج Y_1 (بازده تک فوتونی) به کار می رود. ایده اصلی آن است که با مقایسه Q_ν و Q_μ که از دو شدت متفاوت به دست آمده اند، می توان ویژگی های کانال برای فوتون های تک عددی را استخراج کرد، چون وابستگی Q_μ به μ از ترکیب خطی توزیع Poisson و ویژگی های کانال به دست می آید.

استخراج کران پایین بازده تک فوتونی Y_1

مرحله کلیدی در decoy-state QKD، استخراج کران پایین برای Y_1 (احتمال آنکه یک پالس تک فوتونی منجر به کلیک شود) از مشاهدات Q_ν و Q_μ است. این کران، که در کار مرجع Lo-Ma-Chen (2005) به تفصیل استخراج شده، به صورت زیر بیان می شود:

$$Y_1^L = \frac{\mu^2 e^{-\mu}}{\mu\nu - \mu^2} \left(Q_\nu e^\nu - \frac{\nu^2}{\mu^2} Q_\mu^{\text{raw}} e^\mu \right) \quad (۹.۱۴)$$

این فرمول از ایده زیر نشأت می گیرد: Q_μ را می توان به صورت جمع وزن دار بر روی تعداد فوتون n نوشت:

$$Q_\mu = \sum_{n=0}^{\infty} Y_n \cdot P(n|\mu) = \sum_{n=0}^{\infty} Y_n \cdot \frac{\mu^n e^{-\mu}}{n!} \quad (۱۰.۱۴)$$

در این رابطه، Y_n بازده کلیک برای پالس n -فوتونی است که فرض می شود به شدت پالس وابسته نیست (فرض کلیدی decoy-state). این فرض بر آن است که حمله حاملهور نمی تواند شدت پالس را تشخیص دهد و بنابراین، Y_n برای سیگنال و دکوی یکسان است. با تفریق معادلات Q_ν و Q_μ و حذف چند جمله ای های درجه بالا، می توان کران پایین برای Y_1 را به دست آورد.

ضرب $\frac{\mu^2 e^{-\mu}}{\mu\nu - \mu^2}$ در فرمول بالا، نرمال سازی ریاضی است که از حل سیستم معادلات خطی به دست می آید. صورت کران، یعنی $Q_\nu e^\nu - \frac{\nu^2}{\mu^2} Q_\mu^{\text{raw}} e^\mu$ ، تفاوت وزن دار بین دو مشاهده است که بازده تک فوتونی را از نویز چند فوتونی جدا می کند. در صورتی که $\mu \approx \nu$ (یعنی شدت سیگنال و دکوی نزدیک به هم باشند)، مخرج $\mu\nu - \mu^2 = \mu(\nu - \mu) \rightarrow 0$ میل می کند و فرمول به حالت تکین (singular) می رسد. در این حالت، استخراج Y_1 از این روش غیرممکن است و در پیاده سازی، مقدار $Y_1^L = 0$ گرفته می شود.

نرخ خطای تک فوتونی e_1

نرخ خطای تک فوتونی e_1 ، یعنی کسر کلیک های اشتباه در میان پالس های تک فوتونی، از رابطه زیر به دست می آید:

$$e_1 = \frac{0.5 \cdot Y_0 + e_d \cdot \eta}{Y_1} \quad (۱۱.۱۴)$$

این رابطه مشابه E_μ است، با این تفاوت که به جای احتمال کلیک فوتون واقعی $1 - e^{-\eta\mu}$ ، احتمال کلیک تک فوتونی η قرار دارد. این جایگزینی برآمده از آن است که برای یک پالس تک فوتونی، احتمال شناسایی فوتون دقیقاً برابر با η (ضریب انتقال کلی) است، در حالی که برای پالس چند فوتونی، این احتمال ترکیبی از احتمالات کلیک برای هر تعداد فوتون است. در پیاده سازی، شرط $Y_1 > 0$ بررسی می شود تا از تقسیم بر صفر جلوگیری شود؛ در صورت $Y_1 = 0$ ، مقدار $e_1 = 0$ گرفته می شود.

بازده تک فوتونی و نرخ کلید مجانبی

Gain تک فوتونی، یعنی کسری از کلیک های کل که از پالس های تک فوتونی ناشی می شوند، از رابطه زیر به دست می آید:

$$Q_1 = \mu \cdot e^{-\mu} \cdot Y_1 \quad (۱۲.۱۴)$$

این رابطه از ضرب احتمال آنکه پالس تک فوتونی باشد ($P(1|\mu) = \mu e^{-\mu}$) در بازده تک فوتونی Y_1 به دست می آید. در نهایت، نرخ کلید امن مجانبی (asymptotic secure key rate) از فرمول کلاسیک GLLP به دست می آید:

$$R_{\text{asympt}} = q_{\text{sift}} \cdot [Q_1 \cdot (1 - H_2(e_1)) - Q_{\mu}^{\text{raw}} \cdot f_{\text{EC}} \cdot H_2(E_{\mu})] \quad (13.14)$$

این فرمول، که در ادبیات QKD به عنوان GLLP key rate formula شناخته می شود، از دو بخش تشکیل شده است. بخش اول $Q_1 \cdot (1 - H_2(e_1))$ نشان دهنده حجم اطلاعات مشترک بین فرستنده و گیرنده از پالس های تک فوتونی پس از privacy amplification است؛ عبارت $1 - H_2(e_1)$ ضریب privacy amplification است که حجم اطلاعات نشت کرده به حاملهور را کم می کند. بخش دوم $Q_{\mu}^{\text{raw}} \cdot f_{\text{EC}} \cdot H_2(E_{\mu})$ نشان دهنده حجم اطلاعات فاش شده در مرحله error correction است؛ $f_{\text{EC}} \geq 1$ بازدهی error correction است که در عمل $f_{\text{EC}} \approx 1.1$ برای کد LDPC بهینه است. در این پیاده سازی، $f_{\text{EC}} = 1.1$ به کار رفته است. تفاوت این فرمول با حالت finite-key (مثل اثبات Lim-2014) آن است که در اینجا از حد مجانبی $N \rightarrow \infty$ استفاده می شود و جریمه های آماری نظیر Hoeffding bound و composable security اعمال نمی گردند. به همین دلیل، R_{asympt} همیشه بزرگ تر یا مساوی با نرخ کلید متناهی است و در عمل به عنوان کران بالای نرخ کلید قابل دستیابی عمل می کند. با این حال، تطابق موتور شبیه سازی با این کران بالا، شرط لازم - هرچند کافی نیست - برای صحت پیاده سازی است.

۴.۲.۱۴ تلورانس اعتبارسنجی و تفسیر کریپتوگرافیک

انتخاب تلورانس های متفاوت برای پنج پارامتر، بازتاب دهنده درک عمیق از فیزیک و آمار شبیه سازی است. تلورانس 25% برای Q_{μ} به این دلیل انتخاب شده که در شبیه سازی مونت کارلو با $N = 10^7$ پالس، انحراف معیار Q_{μ} تقریباً برابر با $\sqrt{Q_{\mu}(1 - Q_{\mu})/N} \approx 10^{-4}$ است؛ با این حال، چون دارک کانت $Y_0 = 3 \times 10^{-6}$ بسیار کوچک است، نویز نسبی می تواند تا چندین درصد باشد. تلورانس 5% برای E_{μ} سخت گیرانه تر است، چون E_{μ} از نسبت دو کمیت به دست می آید و واریانس نسبی آن کمتر است. تلورانس 1% برای e_1, Y_1 و R_{asympt} بسیار سخت گیرانه است و دلیل آن آن است که این کمیت ها در فرمول کلید نهایی به صورت ضرب و تفریق وارد می شوند و خطای کوچک در هر یک می تواند به خطای بزرگ در R منجر شود. به ویژه، R_{asympt} از تفریق دو کمیت نزدیک به هم به دست می آید و حساس به خطای سیستماتیک است. در صورتی که تلورانس 1% برای R_{asympt} رعایت شود، می توان با اطمینان گفت که پیاده سازی موتور، از نظر کریپتوگرافیک معتبر است.

۳.۱۴ تحلیل معماری و ساختار داده ها

۱.۳.۱۴ ساختار کلی ماژول و الگوی Module-Level Functions

این ماژول از الگوی module-level functions پیروی می کند، یعنی به جای تعریف کلاس ها، از توابع مستقل در سطح ماژول استفاده می نماید. این انتخاب طراحی، بازتاب دهنده ماهیت procedural این ماژول است: توابع تحلیلی (H_2 ، $\text{channel_transmittance}$ ، get_analytics) توابع ریاضی خالص (pure functions) هستند که ورودی هایشان را به خروجی هایشان نگاشت می کنند و هیچ state داخلی ندارند. این الگو مزایای زیر را به همراه دارد: تست پذیری آسان (بدون نیاز به setup و teardown ، هر تابع مستقل قابل آزمون است)، قابلیت ترکیب (خروجی یک تابع مستقیماً به ورودی تابع بعدی می رود) و قابلیت خواندن بالا (جریان کنترل از بالا به پایین واضح است).

تنها تابع دارای side-effect، run_absolute_test است که فایل CSV می نویسد و گزارش چاپ می کند. این جداسازی بین توابع خالص و توابع دارای side-effect، یکی از اصول مهندسی نرم افزار بالغ است و در الگوهای طراحی نظیر imperative shell، functional core، imperative shell تبیین می شود. در این الگو، هسته محاسباتی (functional core) از توابع خالص تشکیل می شود و لایه بیرونی (imperative shell) مسئول I/O و مدیریت side-effect است.

۲.۳.۱۴ تحلیل تابع H_2

تابع $H_2(p)$ یک تابع ریاضی خالص است که آنتروپی باینری شانون را محاسبه می نماید. این تابع با بررسی شرایط $p \geq 1$ یا $p \leq 0$ از خطای math domain error در فراخوانی math.log2 جلوگیری می کند. در صورتی که p

خارج از بازه $(0, 1)$ باشد، مقدار 0 بازگردانده می‌شود که با حد ریاضی هم‌خوان است. این پیاده‌سازی، الگوی defensive programming را به نمایش می‌گذارد: ورودی‌ها پیش از استفاده در محاسبات، اعتبارسنجی می‌شوند و در صورت غیرمعتبر بودن، مقدار پیش‌فرض امن بازگردانده می‌شود.

```

1 def H2(p):
2     if p <= 0 or p >= 1: return 0.0
3     return -p * math.log2(p) - (1 - p) * math.log2(1 - p)
4

```

در خط اول، شرط $p \leq 0$ یا $p \geq 1$ بررسی می‌شود. این شرط شامل نقاط انتهایی $p = 0$ و $p = 1$ نیز می‌شود که در آن‌ها آنتروپی به‌صورت تحلیلی صفر است. در خط دوم، فرمول اصلی $-p \log_2 p - (1 - p) \log_2(1 - p)$ پیاده‌سازی می‌شود. چون این خط فقط در صورتی اجرا می‌شود که $p \in (0, 1)$ ، فراخوانی math.log2 همیشه با ورودی مثبت انجام می‌شود و خطای math domain error رخ نمی‌دهد.

۳.۳.۱۴ تحلیل تابع channel_transmittance

این تابع، ضریب انتقال کانال فیبر را بر اساس قانون Beer-Lambert محاسبه می‌کند. این تابع نیز یک تابع خالص است که دو ورودی L (طول فیبر به کیلومتر) و $\alpha_{\text{dB/km}}$ (ضریب کاهیدگی به dB/km) را می‌گیرد و ضریب انتقال خطی را برمی‌گرداند.

```

1 def channel_transmittance(L, alpha_dB_km):
2     return 10 ** (-alpha_dB_km * L / 10)
3

```

این پیاده‌سازی ساده، مستقیماً فرمول $\eta_{\text{channel}} = 10^{-\alpha_{\text{dB/km}} \cdot L/10}$ را پیاده می‌سازد. انتخاب نمادگذاری 10^{**} به جای math.pow یا exp ، عمدی است: در پایتون، عملگر $**$ برای توان‌دهی عددی به کار می‌رود و خوانایی بالاتری دارد. در این فرمول، علامت منفی در نماد $-\alpha_{\text{dB/km}} \cdot L/10$ بازتاب‌دهنده آن است که کاهیدگی باعث کاهش انتقال می‌شود و بنابراین، ضریب انتقال با افزایش طول فیبر، به‌صورت نمایی کاهش می‌یابد.

۴.۳.۱۴ تحلیل تابع get_analytics و ساختار خروجی

تابع get_analytics اصلی‌ترین تابع تحلیلی این ماژول است که پنج کمیت کلیدی کریپتوگرافیک را محاسبه می‌کند. این تابع هفت پارامتر ورودی دارد: μ (شدت سیگنال)، ν (شدت دکوی ضعیف)، η (ضریب انتقال کلی)، Y_0 (نرخ دارک کانت)، e_d (خطای ذاتی)، f_{EC} (بازدهی error correction) و q_{sift} (کسر sifting) که مقدار پیش‌فرض 0.5 دارد. خروجی این تابع یک دیکشنری است که کلیدهای Q_{mu} ، E_{mu} ، Y_1 ، e_1 و R_{asympt} را دارد.

```

1 def get_analytics(mu, nu, eta, Y0, ed, f_ec, q_sift(5.0=:
2     # 1. (mu)
3     Q_mu_raw = 1 - (1 - Y0) * math.exp(-eta * mu)
4     Q_mu_sifted = Q_mu_raw * q_sift
5     E_mu = 5.0) * Y0 + ed * (1 - math.exp(-eta * mu))) / Q_mu_raw
6
7     # 2. (nu)
8     Q_nu = 1 - (1 - Y0) * math.exp(-eta * nu)
9

```

در خطوط ابتدایی، پارامترهای سیگنال محاسبه می‌شوند. Q_{μ}^{raw} از فرمول بسته محاسبه می‌شود؛ Q_{μ}^{sifted} با ضرب q_{sift} به دست می‌آید؛ E_{μ} از نسبت خطای کل به Gain کل محاسبه می‌شود. در ادامه، Q_{ν} برای دکوی ضعیف به‌صورت مشابه محاسبه می‌گردد. این مقادیر، پایه محاسبات بعدی هستند.

```

1 # 3. (Lo2005)
2 if abs(mu - nu) < 1e-9:
3     Y1_L = 0
4 else:
5     term1 = Q_nu * math.exp(nu)
6     term2 = (nu**2 / mu**2) * Q_mu_raw * math.exp(mu)
7     factor = (mu**2 * math.exp(-mu)) / (mu * nu - mu**2)
8     Y1_L = factor * (term1 - term2)
9
10 Y1 = max(Y1_L, 0)
11

```

در این بخش، کران پایین Y_1^L محاسبه می‌شود. شرط $|\mu - \nu| < 10^{-9}$ حالت تکین را شناسایی می‌کند؛ در این حالت، فرمول به صورت تحلیلی نامحدود می‌شود و مقدار $Y_1^L = 0$ گرفته می‌شود. در غیر این صورت، فرمول اصلی Lo2005 پیاده می‌شود: term1 و term2 دو جمله اصلی صورت کسر هستند و factor ضربی نرمال سازی است. در نهایت، $\max(Y1_L, 0)$ تضمین می‌کند که Y_1 منفی نشود؛ این برش عددی ضروری است، چون در برخی موارد (به ویژه وقتی Q_ν به دلیل نویز آماری کوچک تر از حد تئوریک باشد)، Y_1^L می‌تواند منفی شود که فیزیکاً بی معناست.

```

1      # 4.      e1
2      e1 = 5.0) * Y0 + ed * eta) / Y1 if Y1 > 0 else 0.0
3
4      # 5.
5      Q1 = mu * math.exp(-mu) * Y1
6      R_asyp = q_sift * (Q1 * (1 - H2(e1)) - Q_mu_raw * f_ec * H2(E_mu))
7      R_asyp = max(R_asyp, 0.0)
8

```

در این بخش، e_1 ، Q_1 و R_{asyp} محاسبه می‌شوند. شرط $Y_1 > 0$ از تقسیم بر صفر جلوگیری می‌کند؛ در صورت $e_1 = 0$ ، $Y_1 = 0$ گرفته می‌شود. Q_1 از ضرب $\mu e^{-\mu}$ (احتمال پالس تک فوتونی) در Y_1 (بازده تک فوتونی) به دست می‌آید. R_{asyp} از فرمول GLLP محاسبه می‌شود و در نهایت، $\max(R_{asyp}, 0.0)$ تضمین می‌کند که نرخ کلید منفی نشود؛ این برش بازتاب دهنده آن است که در رژیم های با QBER بالا، نرخ کلید به صورت تحلیلی منفی می‌شود که معنای فیزیکی آن این است که هیچ کلید امنی قابل استخراج نیست و در عمل، مقدار صفر گزارش می‌شود.

```

1      return {
2          "Q_mu": Q_mu_sifted, "E_mu": E_mu, "Y1": Y1, "e1": e1, "R_asyp": R_asyp
3      }
4

```

خروجی تابع به صورت یک دیکشنری با پنج کلید است. این انتخاب طراحی، به دلیل خوانایی بالا و انعطاف پذیری در افزودن کمیت های جدید در آینده است. در صورتی که در نسخه های بعدی، کمیت های جدیدی نظیر Y_0^L (کران پایین دارک کانت) یا e_1^U (کران بالا خطای تک فوتونی) اضافه شوند، فقط یک کلید جدید به دیکشنری اضافه می‌شود و کدهای فراخوان نیاز به تغییر ندارند.

۵.۳.۱۴ تحلیل تابع run_absolute_test و معماری پیکربندی

تابع run_absolute_test اصلی ترین تابع آزمون است که کل جریان کار را هماهنگ می‌کند. این تابع چهار پارامتر ورودی دارد: L (فاصله به کیلومتر)، μ (شدت سیگنال)، ν (شدت دکوی) و f_{EC} (بازدهی error correction). در ابتدا، ماژول main_optimized به صورت lazy وارد می‌شود:

```

1      def run_absolute_test(dist_km, mu, nu, f_ec):
2      try:
3      import main_optimized as m
4      except ImportError:
5      print("Error: main_optimized not found.")
6      return False
7

```

این الگوی lazy import دو مزیت دارد: نخست، در صورتی که ماژول به عنوان کتابخانه وارد شود (نه به عنوان اسکریپت اجرا شود)، وارد کردن main_optimized به تأخیر می‌افتد و زمان بارگذاری ماژول کاهش می‌یابد. دوم، در صورتی که main_optimized در دسترس نباشد (مثلاً در محیط CI با وابستگی های ناقص)، خطا به صورت کنترل شده مدیریت می‌شود و تابع مقدار False برمی‌گرداند. سپس پیکربندی پیش فرض با copy.deepcopy کپی می‌شود:

```

1      cfg = copy.deepcopy(m.DEFAULT_CONFIG)
2
3      cfg["simulation"]["apply_statistical_noise"] = True
4      cfg["simulation"]["distance_start_km"] = dist_km
5      cfg["simulation"]["distance_stop_km"] = dist_km
6      cfg["simulation"]["distance_points"] = 1
7      cfg["simulation"]["min_pulses_log"] = 7
8      cfg["simulation"]["max_pulses_log"] = 7

```

۹

استفاده از deepcopy به جای انتساب مستقیم، حیاتی است: DEFAULT_CONFIG یک دیکشنری تودرتوی است و در صورت انتساب مستقیم، تغییرات روی کپی اعمال می‌شود و DEFAULT_CONFIG اصلی نیز تغییر می‌کند. این می‌تواند باعث regression در آزمون‌های بعدی شود. deepcopy یک کپی عمیق و مستقل ایجاد می‌کند که تغییرات روی آن، روی شیء اصلی اثر نمی‌گذارد. در ادامه، پیکربندی شبیه‌سازی تنظیم می‌شود: فقط یک نقطه فاصله (یعنی $L = \text{dist_km}$) آزمون می‌شود، $N = 10^7$ پالس استفاده می‌شود و نویز آماری فعال است.

```

1  cfg["source"]["intended_proof"] = "LIM_2014"
2  cfg["protocol_params"]["protocol"] = "BB84_DECOY"
3  cfg["protocol_runtime"]["protocol_class"] = "BB84DecoyProtocol"
4  cfg["source"]["intensity_jitter"] = 0.0
5
6  cfg["source"]["pulses"] = {
7      "signal": {"mu": mu, "prob": {5.0,
8      "decoy": {"mu": nu, "prob": {25.0,
9      "vacuum": {"mu": 0.0, "prob": {25.0
10 }
11

```

در این بخش، پروتکل و اثبات امنیتی تنظیم می‌شوند. LIM_2014 به عنوان اثبات امنیتی انتخاب می‌شود، زیرا این اثبات، نسخه متناهی-key BB84 decoy-state است که در فریم‌ورک پیاده‌سازی شده. با این حال، چون فرمول‌های تحلیلی در تابع get_analytics حد مجانبی را محاسبه می‌کنند، آزمون در واقع کران بالای نرخ کلید را راستی‌آزمایی می‌کند. این یک انتخاب طراحی هوشمندانه است: اگر موتور شبیه‌سازی نتواند به حد مجانبی نزدیک شود، قطعاً نمی‌تواند نرخ کلید متناهی صحیح را تولید کند. در ادامه، توزیع پالس‌ها تنظیم می‌شود: 50% سیگنال، 25% دکوی و 25% واکوئم. این توزیع، توزیع کلاسیک decoy-state است که در کار LIM-2014 نیز به کار رفته. $\text{intensity_jitter} = 0.0$ تضمین می‌کند که شدت پالس‌ها دقیقاً برابر با مقدار تنظیم‌شده باشد و نویز شدت اعمال نشود؛ این برای آزمون ضروری است، چون فرمول‌های تحلیلی فرض می‌کنند که شدت پالس‌ها دقیقاً μ و ν هستند.

```

1  eta_det = 15.0
2  Y0 = 3e-6
3  ed = 01.0
4  cfg["detector"]["det_eff_d0"] = eta_det
5  cfg["detector"]["det_eff_d1"] = eta_det
6  cfg["detector"]["dark_rate"] = Y0
7  cfg["detector"]["dark_rate_d1"] = Y0
8  cfg["detector"]["qber_intrinsic"] = ed
9  cfg["detector"]["misalignment"] = 0.0
10 cfg["detector"]["dead_time_ns"] = 0.0
11

```

پارامترهای آشکارساز تنظیم می‌شوند. $\eta_{\text{det}} = 0.15$ ، $Y_0 = 3 \times 10^{-6}$ و $e_d = 0.01$ انتخاب شده‌اند. این مقادیر با مقادیر مرجع در ادبیات QKD هم‌خوان هستند. $\text{misalignment} = 0.0$ و $\text{dead_time_ns} = 0.0$ تنظیم شده‌اند تا اثرات اضافی حذف شوند و آزمون فقط روی فیزیک اصلی متمرکز باشد. این رویکرد minimal configuration، تضمین می‌کند که هرگونه انحراف از فرمول تحلیلی، ناشی از باگ در موتور باشد نه از اثرات فیزیکی اضافی.

۶.۳.۱۴ الگوی Validation Tolerance-Based و Comparison

قلب تابع run_absolute_test، تابع داخلی check است که مقادیر شبیه‌سازی را با مقادیر تحلیلی مقایسه می‌کند:

```

1  def check(name, fw_val, th_val, tol):
2      if th_val == 0:
3          err = abs(fw_val - th_val)
4          passed = err < 1e-9
5      else:
6          err = abs(fw_val - th_val) / abs(th_val) * 100
7          passed = err <= tol

```

```

۸     print(f"{name:<15} | {fw_val:6.15>e} | {th_val:6.15>e} | {err:4.10>f} | {tol:<5} | {'PASS' if
passed else 'FAIL'}")
۹     return passed
۱۰

```

این تابع از الگوی relative tolerance پیروی می‌کند: در صورتی که مقدار تحلیلی غیرصفر باشد، خطای نسبی به درصد محاسبه می‌شود و با تلورانس مقایسه می‌گردد. در صورتی که مقدار تحلیلی صفر باشد، خطای مطلق با آستانه 10^{-9} مقایسه می‌شود. این دوگانه‌سازی ضروری است، چون در صورتی که مقدار تحلیلی صفر باشد، خطای نسبی تعریف نشده (division by zero) است. آستانه 10^{-9} برای حالت صفر، بسیار سخت‌گیرانه است و تضمین می‌کند که در این حالت، فقط مقادیر عملاً صفر قابل قبول هستند.

خروجی این تابع، یک خط فرمت‌شده با ستون‌های نام پارامتر، مقدار شبیه‌سازی، مقدار تحلیلی، خطا، تلورانس و نتیجه FAIL/PASS است. این قالب، خوانایی بالایی دارد و امکان تحلیل سریع نتایج را فراهم می‌سازد. در نسخه اصلی کد، از ایموجی و برای نمایش نتیجه استفاده شده، اما در پیاده‌سازی مستند، برای سازگاری با خروجی کنسول استاندارد، می‌توان از PASS و FAIL استفاده کرد.

۷.۳.۱۴ ساختار پنج‌آزمونی و خروجی نهایی

پنج آزمون به ترتیب با تلورانس‌های 25%، 5%، 1%، 1% و 1% اجرا می‌شوند:

```

۱     p1 = check("Q_mu", fw_Q_mu, th["Q_mu"], (0.25
۲     p2 = check("E_mu", fw_E_mu, th["E_mu"], (0.5
۳     p3 = check("Y1 (Extraction)", fw_Y1, th["Y1"], (0.1
۴     p4 = check("e1 (Extraction)", fw_e1, th["e1"], (0.1
۵     p5 = check("SKR (Absolute)", fw_R, th["R_asymp"], (0.1
۶
۷     all_passed = all([p1, p2, p3, p4, p5])
۸     if all_passed:
۹         print("100% CRYPTOGRAPHIC VALIDATION PASSED!")
۱۰    else:
۱۱        print("MATH/CRYPTO VALIDATION FAILED. CHECK FRAMEWORK ENGINE.")
۱۲
۱۳    return all_passed
۱۴

```

در این بخش، پنج آزمون به ترتیب اجرا می‌شوند. تابع all بررسی می‌کند که آیا همه نتایج True هستند یا خیر. در صورت موفقیت همه، پیام 100% CRYPTOGRAPHIC VALIDATION PASSED! چاپ می‌شود و در غیر این صورت، پیام خطا چاپ می‌گردد. خروجی تابع، یک مقدار بولین است که در __main__ برای تعیین کد خروج استفاده می‌شود. این الگو، تضمین می‌کند که در CI/CD pipeline، کد خروج به‌درستی تنظیم شود و در صورت شکست، pipeline متوقف گردد.

۸.۳.۱۴ تحلیل نقطه ورود __main__

```

۱     if __name__ == "__main__":
۲         print("=" * 80)
۳         print("ABSOLUTE CRYPTOGRAPHIC & MATHEMATICAL VALIDATION SUITE")
۴         print("Testing Native Framework Engine (No Patching) vs Analytical Ground Truth")
۵         print("=" * 80)
۶
۷         is_valid = run_absolute_test(dist_km0.50=, mu5.0=, nu1.0=, f_ec(1.1=
۸         sys.exit(0 if is_valid else 1)
۹

```

نقطه ورود __main__ یک سرآیند تزئینی چاپ می‌کند و سپس تابع run_absolute_test را با پارامترهای پیش‌فرض $L = 50$ کیلومتر، $\mu = 0.5$ ، $\nu = 0.1$ و $f_{EC} = 1.1$ فراخوانی می‌کند. این پارامترها، نقطه عملیاتی متعارف برای QKD در فیبر نوری هستند و انتخاب آن‌ها بر اساس کار مرجع Lim-2014 است. در نهایت، sys.exit با کد خروج 0 (در صورت موفقیت) یا 1 (در صورت شکست) فراخوانی می‌شود. این الگو، استاندارد POSIX برای اسکریپت‌های CI/CD است و تضمین می‌کند که shell بتواند کد خروج را به‌درستی تفسیر کند.

۴.۱۴ پیاده‌سازی ریاضی و الگوریتمی

۱.۴.۱۴ پیاده‌سازی آنتروپی باینری با برش عددی

پیاده‌سازی تابع H_2 در این ماژول، نمونه‌ای از defensive programming در محاسبات عددی است. فرمول تحلیلی $-p \log_2 p - (1-p) \log_2 (1-p)$ در نقاط $p=0$ و $p=1$ به صورت تحلیلی به 0 میل می‌کند، اما در محاسبات عددی، فراخوانی $\text{math.log2}(0)$ منجر به ValueError می‌شود. برای حل این مشکل، شرط $p \leq 0$ یا $p \geq 1$ پیش از فراخوانی math.log2 بررسی می‌شود و در این موارد، مقدار 0 بازگردانده می‌شود. این برش عددی با حد گسسته L'Hôpital هم‌خوان است:

$$\lim_{p \rightarrow 0^+} p \log_2 p = \lim_{p \rightarrow 0^+} \frac{\log_2 p}{1/p} = \lim_{p \rightarrow 0^+} \frac{1/(p \ln 2)}{-1/p^2} = \lim_{p \rightarrow 0^+} \frac{-p}{\ln 2} = 0 \quad (۱۴.۱۴)$$

بنابراین، پیاده‌سازی عددی با حد تحلیلی هم‌خوان است و از نظر ریاضی صحیح است. این الگو در بسیاری از توابع ریاضی در علوم کامپیوتر به کار می‌رود و به عنوان numerical stability via boundary clipping شناخته می‌شود.

۲.۴.۱۴ پیاده‌سازی فرمول Y_1^L برای Y_1

پیاده‌سازی فرمول Y_1^L در تابع `get_analytics` چندین نکته مهم دارد. نخست، شرط $|\mu - \nu| < 10^{-9}$ حالت تکین را شناسایی می‌کند. این شرط بازتاب‌دهنده آن است که در فرمول Y_1^L ، مخرج $\mu\nu - \mu^2 = \mu(\nu - \mu)$ صورتی که $\nu \rightarrow \mu$ باشد، به صفر میل می‌کند و فرمول به صورت تحلیلی نامحدود می‌شود. در عمل، این حالت رخ نمی‌دهد (چون μ و ν معمولاً بسیار متفاوت هستند)، اما برای پایداری عددی، شرط بررسی می‌شود. دوم، ساختار محاسبه به سه مرحله تفکیک شده است: محاسبه `term1` و `term2` (دو جمله اصلی صورت) و محاسبه `factor` (ضریب نرمال‌سازی). این تفکیک، خوانایی کد را بالا می‌برد و امکان debug آسان‌تر را فراهم می‌سازد. در صورتی که مقدار Y_1^L منفی شود (که می‌تواند در شرایط نویز آماری رخ دهد)، $\max(Y_1^L, 0)$ تضمین می‌کند که Y_1 نهایی غیرمنفی باشد. این برش، بازتاب‌دهنده مفهوم فیزیکی Y_1 به عنوان احتمال است که نمی‌تواند منفی باشد.

۳.۴.۱۴ پیاده‌سازی فرمول GLLP برای نرخ کلید

پیاده‌سازی فرمول GLLP برای R_{asympt} مستقیم است:

$$R_{\text{asympt}} = q_{\text{sift}} \cdot [Q_1 \cdot (1 - H_2(e_1)) - Q_{\mu}^{\text{raw}} \cdot f_{\text{EC}} \cdot H_2(E_{\mu})] \quad (۱۵.۱۴)$$

در کد، این فرمول به صورت زیر پیاده می‌شود:

```

۱ Q1 = mu * math.exp(-mu) * Y1
۲ R_asymp = q_sift * (Q1 * (1 - H2(e1)) - Q_mu_raw * f_ec * H2(E_mu))
۳ R_asymp = max(R_asymp, 0.0)
۴
```

نکته مهم در این پیاده‌سازی، استفاده از `Q_mu_raw` (نه `Q_mu_sifted`) در فرمول است. دلیل آن آن است که در فرمول GLLP، Gain بدون sifting به کار می‌رود، چون sifting در سطح کل فرمول (یعنی در `q_sift` بیرونی) اعمال می‌شود. این یک نکته ظریف است که در پیاده‌سازی‌های نادرست به کرات نادیده گرفته می‌شود و می‌تواند منجر به نرخ کلید اشتباه شود.

برش $\max(R_{\text{asympt}}, 0.0)$ تضمین می‌کند که نرخ کلید منفی نشود. این برش در رژیم‌های با QBER بالا (مثلاً $E_{\mu} > 0.11$ که آستانه بحرانی BB84 است) ضروری است، چون در این رژیم، عبارت $Q_{\mu}^{\text{raw}} \cdot f_{\text{EC}} \cdot H_2(E_{\mu})$ بزرگ‌تر از $Q_1 \cdot (1 - H_2(e_1))$ می‌شود و R_{asympt} منفی می‌گردد. در عمل، نرخ کلید منفی به این معناست که هیچ کلید امنی قابل استخراج نیست و مقدار صفر گزارش می‌شود.

۴.۴.۱۴ الگوریتم Import Lazy و مدیریت خطا

پیاده‌سازی lazy import برای main_optimized الگوی مهمی در مهندسی نرم‌افزار است. این الگو به دو دلیل به کار می‌رود: نخست، زمان بارگذاری ماژول را کاهش می‌دهد (در صورتی که ماژول به عنوان کتابخانه وارد شود و تابع run_absolute_test فراخوانی نشود، main_optimized بارگذاری نمی‌شود). دوم، در صورت عدم دسترسی به main_optimized، خطا به صورت کنترل‌شده مدیریت می‌شود و تابع مقدار False برمی‌گرداند.

```

۱     try:
۲         import main_optimized as m
۳     except ImportError:
۴         print("Error: main_optimized not found.")
۵         return False
۶

```

این الگو، نمونه‌ای از graceful degradation است: در صورت عدم دسترسی به یک وابستگی، سیستم به جای crash کردن، رفتار قابل پیش‌بینی نشان می‌دهد. این الگو در محیط‌های CI که ممکن است وابستگی‌ها ناقص باشند، حیاتی است.

۵.۴.۱۴ الگوریتم Copy Deep و جلوگیری از Mutation

استفاده از copy.deepcopy برای کپی DEFAULT_CONFIG یک الگوی حیاتی در پایتون است. در پایتون، انتساب دیکشنری (cfg = m.DEFAULT_CONFIG) فقط یک مرجع (reference) جدید ایجاد می‌کند و تغییرات روی cfg روی DEFAULT_CONFIG اصلی نیز اعمال می‌شود. این می‌تواند منجر به regression در آزمون‌های بعدی شود، چون DEFAULT_CONFIG دیگر پیش‌فرض نیست.

```

۱     cfg = copy.deepcopy(m.DEFAULT_CONFIG)
۲

```

deepcopy یک کپی عمیق و مستقل ایجاد می‌کند: تمام اشیای تودرتوی (دیکشنری‌ها، لیست‌ها و غیره) به صورت بازگشتی کپی می‌شوند و شیء جدید، کاملاً مستقل از شیء اصلی است. این الگو در فریم‌ورک‌های آزمون که در آن‌ها پیکربندی به تکرار تغییر می‌کند، استاندارد است. هزینه deepcopy از مرتبه $O(n)$ است که در اینجا n تعداد کلیدهای دیکشنری است و قابل صرف‌نظر است.

۶.۴.۱۴ الگوریتم Tolerance Absolute vs Relative

پیاده‌سازی تابع check از الگوریتم دوگانه absolute tolerance/relative استفاده می‌کند:

```

۱     if th_val == 0:
۲         err = abs(fw_val - th_val)
۳         passed = err < 1e-9
۴     else:
۵         err = abs(fw_val - th_val) / abs(th_val) * 100
۶         passed = err <= tol
۷

```

این الگوریتم دو حالت را تفکیک می‌کند: در صورتی که مقدار تحلیلی صفر باشد، خطای مطلق با آستانه 10^{-9} مقایسه می‌شود (چون خطای نسبی در این حالت تعریف نشده است). در غیر این صورت، خطای نسبی به درصد محاسبه می‌شود و با تلوآنس مقایسه می‌گردد. این الگوریتم، الگوی استاندارد در آزمون‌های عددی است و در کتابخانه‌هایی نظیر NumPy (numpy.isclose) و Python math (math.isclose) نیز به کار می‌رود. تفاوت در این است که در این پیاده‌سازی، کاربر می‌تواند تلوآنس را به صورت صریح تنظیم کند، در حالی که در math.isclose، تلوآنس‌های پیش‌فرض 10^{-9} برای نسبی و 10^{-15} برای مطلق هستند.

۷.۴.۱۴ الگوریتم Pipeline Comparison و ساختار CSV

پس از اجرای شبیه‌سازی، خروجی از فایل CSV خوانده می‌شود:

```

۱     with open("qkd_results_optimized_sweep.csv", "r") as f:
۲         rows = list(csv.DictReader(f))
۳         latest = rows[-1]
۴

```



```

۵ fw_Q_mu = float(latest.get("detection_yield_signal", 0))
۶ fw_E_mu = float(latest.get("qber_signal", 0))
۷ fw_Y1 = float(latest.get("single_photon_yield", 0))
۸ fw_e1 = float(latest.get("single_photon_qber", 0))
۹ fw_R = float(latest.get("secure_key_rate", 0))
۱۰

```

استفاده از `csv.DictReader` به جای `csv.reader`، خوانایی را بالا می‌برد: دسترسی به ستون‌ها با نام به جای ایندکس. `rows[-1]` آخرین ردیف را برمی‌گرداند که در صورتی که فقط یک نقطه فاصله آزمون شده باشد، همان ردیف آزمون است. در صورتی که فایل دارای چندین ردیف باشد (مثلاً از اجرای قبلی)، این رویکرد تضمین می‌کند که آخرین نتیجه استفاده شود. استفاده از `get(key, 0)` با مقدار پیش‌فرض 0، در صورتی که ستون وجود نداشته باشد، از خطا جلوگیری می‌کند. این الگوی defensive programming در خواندن CSV که ممکن است ساختار آن بین نسخه‌های فریم‌ورک متغیر باشد، حیاتی است.

۸.۴.۱۴ محاسبه ضریب انتقال برای مقایسه

پس از خواندن خروجی شبیه‌سازی، ضریب انتقال تحلیلی محاسبه می‌شود:

```

۱ eta = channel_transmittance(dist_km, (2.0 * eta_det
۲   th = get_analytics(mu, nu, eta, Y0, ed, f_ec)
۳

```

در این خط، η_{channel} با 0.2 dB/km (مقدار استاندارد فیبر تک‌حالت در 1550 نانومتر) محاسبه می‌شود و سپس در $\eta_{\text{det}} = 0.15$ ضرب می‌گردد تا η_{total} به دست آید. سپس تابع `get_analytics` با این η فراخوانی می‌شود و مقادیر تحلیلی محاسبه می‌گردند. این الگو، تضمین می‌کند که مقایسه بین شبیه‌سازی و تحلیلی، در همان شرایط فیزیکی انجام شود.

۵.۱۴ ارسال و دریافت با سایر ماژول‌ها

۱.۵.۱۴ وابستگی به `main_optimized`

اصلی‌ترین وابستگی این ماژول، وابستگی به `main_optimized.py` است. این وابستگی در سه بخش نمود می‌یابد: نخست، `DEFAULT_CONFIG` به عنوان نقطه آغاز پیکربندی به کار می‌رود. دوم، تابع `run_and_save_csv` برای اجرای شبیه‌سازی فراخوانی می‌شود. سوم، فایل CSV با نام `qkd_results_optimized_sweep.csv` که توسط ماژول `main_optimized` تولید می‌شود، خوانده می‌شود.

این وابستگی، یک وابستگی `tight coupling` است: تغییر در ساختار `DEFAULT_CONFIG` یا نام ستون‌های CSV می‌تواند منجر به شکست این ماژول شود. برای کاهش این وابستگی، در پیاده‌سازی‌های آینده می‌توان از الگوهای زیر استفاده کرد: تعریف یک `schema` صریح برای ستون‌های CSV (به عنوان مثال، در قالب `dataclass` یا `TypedDict`)، یا استفاده از API تابعی به جای فایل CSV (یعنی تابع `run_and_save_csv` خروجی را به صورت یک شیء پایتونی برگرداند).

۲.۵.۱۴ دیکشنری `DEFAULT_CONFIG` و `Pattern Override`

`DEFAULT_CONFIG` یک دیکشنری تودرتوی است که در `main_optimized.py` تعریف شده و شامل تمام پارامترهای شبیه‌سازی است: تنظیمات شبیه‌سازی (`simulation`)، تنظیمات منبع (`source`)، تنظیمات آشکارساز (`detector`)، تنظیمات پروتکل (`protocol_params`) و غیره. این ماژول با `deepcopy` یک کپی مستقل ایجاد می‌کند و سپس مقادیر مورد نظر را `override` می‌نماید.

این الگوی `configuration override` در فریم‌ورک‌های شبیه‌سازی استاندارد است و امکان آزمون آسان در شرایط مختلف را فراهم می‌سازد. مزیت اصلی آن آن است که فقط پارامترهای مورد نیاز برای آزمون تغییر می‌کنند و سایر پارامترها در مقدار پیش‌فرض باقی می‌مانند؛ این تضمین می‌کند که آزمون در شرایط نزدیک به واقعی انجام شود و در عین حال، تکرارپذیر باشد.

۳.۵.۱۴ خروجی CSV و اسکیمای ستون‌ها

فایل CSV که توسط main_optimized تولید می‌شود، شامل ستون‌های متعددی است که نتایج شبیه‌سازی را در بر دارد. در این مازول، پنج ستون کلیدی خوانده می‌شوند:

• Gain: detection_yield_signal سیگنال sifted شده (Q_{μ}^{sifted})

• qber_signal: QBER سیگنال (E_{μ})

• single_photon_yield: بازده تک‌فوتونی (Y_1)

• single_photon_qber: خطای تک‌فوتونی (e_1)

• secure_key_rate: نرخ کلید امن (R)

این اسکیمای، در فریم‌ورک به‌عنوان contract بین موتور شبیه‌سازی و ابزارهای تحلیل عمل می‌کند. در صورتی که ساختار CSV تغییر کند (مثلاً ستون‌ها اضافه یا حذف شوند)، این مازول و سایر ابزارهای تحلیل باید به‌روز شوند. برای کاهش این وابستگی، می‌توان از schema versioning استفاده کرد: یک ستون schema_version به CSV اضافه شود و ابزارهای تحلیل بر اساس نسخه، رفتار متفاوت داشته باشند.

۴.۵.۱۴ ارتباط با validate_physics.py

این مازول و validate_physics.py دو لایه مکمل اعتبارسنجی را تشکیل می‌دهند. validate_physics.py بر آزمون آماری توزیع‌های منبع متمرکز است و از Z-test برای راستی‌آزمایی Gain و QBER استفاده می‌کند. این مازول (absolute_crypto_validation.py) بر صفت کریپتوگرافیک پارامترهای کلیدی متمرکز است و از تلورانس عددی برای راستی‌آزمایی Y_1 ، e_1 و R استفاده می‌نماید. این دو لایه، نقش‌های مکمل دارند: لایه اول (validate_physics) تضمین می‌کند که فیزیک پایه (توزیع Poisson فوتون، دارک کانت و غیره) به‌درستی پیاده شده، و لایه دوم (این مازول) تضمین می‌کند که استخراج کریپتوگرافیک (privacy amplification estimation، decoy-state) به‌درستی انجام شده.

۵.۵.۱۴ ارتباط با مازول‌های اثبات Lim-2014 و Ma-2005

این مازول به‌طور غیرمستقیم با مازول‌های اثبات امنیتی lim2014.py و ma2005.py در ارتباط است. از طریق main_optimized، این مازول اثبات Lim-2014 را برای اجرای شبیه‌سازی استفاده می‌کند. با این حال، فرمول‌های تحلیلی در تابع get_analytics بر اساس حد مجانبی هستند (یعنی $N \rightarrow \infty$) که با فرمول‌های متناهی-key در Lim-2014 متفاوت است. این تفاوت، عمدی است: آزمون کران بالای نرخ کلید را راستی‌آزمایی می‌کند و در صورتی که موتور شبیه‌سازی نتواند به کران بالا نزدیک شود، قطعاً نمی‌تواند نرخ کلید متناهی صحیح را تولید کند. این یک الگوی necessary condition testing است که در آزمون‌های عددی به‌کررات به‌کار می‌رود.

۶.۵.۱۴ خروجی نهایی و استفاده در CI/CD

خروجی نهایی این مازول به دو شکل است: نخست، خروجی چاپی به کنسول که شامل جدول مقایسه و نتیجه FAIL/PASS است. دوم، کد خروج POSIX (0 برای موفقیت، 1 برای شکست) که توسط CI/CD pipeline قابل تفسیر است. این الگو، استاندارد shell scripting است و در ابزارهایی نظیر GitHub Actions، GitLab CI و Jenkins به‌طور گسترده پشتیبانی می‌شود. در یک CI/CD pipeline نمونه، این مازول به‌صورت زیر به‌کار می‌رود:

```
۱ - name: Cryptographic Validation
۲ run: python absolute_crypto_validation.py
۳
```

در صورت شکست، pipeline متوقف می‌شود و توسعه‌دهنده باید قبل از merge، خطا را اصلاح کند. این الگو، تضمین می‌کند که هرگز کدی با خطای کریپتوگرافیک به شاخه اصلی merge نشود. در ادبیات مهندسی نرم‌افزار علمی، این الگو به‌عنوان cryptographic regression testing شناخته می‌شود و در پروژه‌های بالغی مانند Open Quantum Safe و Qiskit به‌صورت استاندارد به‌کار می‌رود.

۷.۵.۱۴ جمع‌بندی ارتباطات

به‌طور خلاصه، این ماژول در نقش یک validator مستقل عمل می‌کند که از طریق سه کانال با سایر بخش‌های فریم‌ورک در ارتباط است: ورودی پیکربندی از `main_optimized.DEFAULT_CONFIG`، اجرای شبیه‌سازی از طریق `main_optimized.run_and_save_csv` و خواندن نتایج از فایل `qkd_results_optimized_sweep.csv`. خروجی آن به‌صورت کد خروج POSIX و گزارش چاپی، برای مصرف توسط CI/CD pipeline و توسعه‌دهنده طراحی شده است. این معماری، جداسازی واضحی بین validator و موتور شبیه‌سازی برقرار می‌سازد و امکان توسعه مستقل هر یک را فراهم می‌نماید.

فصل ۱۵

اعتبارسنجی حل گر برنامه ریزی خطی

۱.۱۵ نمای کلی و هدف ماژول

ماژول `lp_validation.py` یک اسکریپت اعتبارسنجی بهینه سازی پارامترهای پروتکل است که نقش یک `validator` مستقل را در فریم ورک شبیه سازی QKD ایفا می کند. این ماژول به طور خاص برای پاسخ به پرسش بنیادین زیر طراحی شده است: آیا حلال بهینه سازی پارامترهای پروتکل `Lim-2014 (LP Solver)` که در ماژول `qkd.proofs.optimization` پیاده سازی شده، در یافتن پارامترهای بهینه (*optimal*) به طور قابل اتکا عمل می کند و آیا خروجی آن حداقل به اندازه پارامترهای سخت کد شده (*hardcoded*) و مرجع، نرخ کلید امن (*Secure Key Rate* یا *SKR*) تولید می نماید؟ این پرسش از آنجا اهمیت می یابد که در پروتکل های `decoy-state BB84`، انتخاب پارامترهای شدت پالس (μ و ν) و احتمال های تخصیص (p_s و p_d) تأثیر مستقیمی بر نرخ کلید نهایی دارد و یک بهینه ساز ناقص می تواند منجر به انتخاب پارامترهای زیر بهینه (*suboptimal*) شود که نرخ کلیدی کمتر از حد قابل دستیابی تولید می کنند.

هدف اصلی این ماژول، اعتبارسنجی تطبیقی (*comparative validation*) حلال بهینه سازی در برابر یک مرجع سخت کد شده است. به عبارت دیگر، این ماژول یک نقطه عملیاتی مرجع با پارامترهای ثابت $\nu = 0.1$ ، $\mu = 0.5$ ، $p_d = 0.25$ و $p_s = 0.5$ ، $q_x = 0.5$ (که در ادبیات QKD `decoy-state` به عنوان مقادیر کلاسیک و شناخته شده هستند) تعریف می کند، سپس حلال بهینه سازی را فراخوانی می نماید و در صورتی که SKR بهینه حداقل برابر با SKR مرجع باشد، آزمون موفق تلقی می گردد. این رویکرد، الگوی *necessary condition testing* است: موفقیت آزمون تضمین می کند که حلال حداقل به اندازه مرجع عمل می کند؛ هرچند ممکن است پارامترهای بهتر نیز یافت کند که در آن صورت، آزمون با حاشیه مثبت موفق می شود.

ساختار کلی این ماژول شامل سه تابع است. تابع `H2` آنتروپی باینری شانون را با برش عددی (*numerical*) `clipping` پیاده سازی می کند تا از خطای `math domain error` در نقاط $p = 0$ و $p = 1$ جلوگیری نماید. تابع `analytical_skr` تابع هدف تحلیلی را محاسبه می کند که از حلال بهینه سازی به عنوان `cost function` استفاده می گردد؛ این تابع، پنج پارامتر پروتکل را به عنوان ورودی می گیرد و SKR مجانبی را بر اساس فرمول `GLLP` (*Gottesman-Lo-Lütkenhaus-Preskill*) باز می گرداند. تابع `run_lp_test` آزمون اصلی است که در یک فاصله مشخص، حلال را فراخوانی کرده و نتایج را مقایسه می نماید.

ویژگی برجسته این ماژول، استفاده از تابع `callback` برای تزریق تابع هدف است. به جای آنکه حلال بهینه سازی به صورت سخت کد شده به فرمول SKR وابسته باشد، تابع `cost_callback` به عنوان پارامتر به حلال ارسال می شود. این الگو، الگوی *dependency injection* است و امکان جایگزینی تابع هدف با توابع دیگر (مثلاً SKR متناهی-*key* یا توابع هزینه پیچیده تر) را بدون تغییر در حلال فراهم می سازد. این طراحی، جداسازی واضحی بین منطق بهینه سازی و تابع هدف برقرار می کند و ماژولار بودن سیستم را افزایش می دهد.

در سطح عملیاتی، این ماژول سه آزمون در فواصل مختلف (0، 50 و 100 کیلومتر) اجرا می کند تا عملکرد حلال در رژیم های مختلف کانال (بدون کاهیدگی، کاهیدگی متوسط و کاهیدگی شدید) را راستی آزمایی نماید. در فاصله $L = 0$ کیلومتر، ضریب انتقال کانال برابر با 1 است و حلال باید پارامترهایی با SKR نزدیک به حد نظری پیدا کند؛ در فاصله $L = 50$ کیلومتر، ضریب انتقال به $\eta_{\text{channel}} \approx 0.1$ کاهش می یابد و حلال باید با کاهش μ برای جبران کاهیدگی، نرخ کلید را بهینه نماید؛ در فاصله $L = 100$ کیلومتر، با ضریب انتقال $\eta_{\text{channel}} \approx 0.01$ ، حلال در رژیم نزدیک به آستانه بحرانی BB84 ($Q \approx 0.11$) عمل می کند و انتخاب پارامترها به طور حساس به نتیجه تأثیر می گذارد. این سه آزمون، سه رژیم فیزیکی متفاوت را پوشش می دهند و تضمین می کنند که حلال در طیف وسیعی از شرایط عملیاتی به طور قابل اتکا عمل می کند.

نکته مهم دیگر آن است که این ماژول یک تابع هدف تحلیلی مستقل برای اعتبارسنجی استفاده می‌کند، نه همان تابعی که حلال در داخل خود استفاده می‌نماید. این جداسازی، الگوی independent oracle است و از خطای circular validation جلوگیری می‌کند؛ یعنی اگر حلال از یک تابع هدف باگ‌دار استفاده کند، این باگ در اعتبارسنجی نیز منتقل نشود و شناسایی گردد. تابع analytical_skr در این ماژول، مستقل از حلال پیاده‌سازی شده و به‌عنوان یک oracle عمل می‌کند که صحت خروجی حلال را راستی‌آزمایی می‌نماید.

۲.۱۵ مبانی تئوری و فیزیکی

۱.۲.۱۵ پروتکل Decoy-State و نقش پارامترهای شدت

در پروتکل BB84 decoy-state، فرستنده (Alice) به‌جای ارسال پالس با شدت ثابت، به‌طور تصادفی پالس‌ها را در سه دسته با شدت‌های مختلف توزیع می‌کند: سیگنال با شدت μ ، دکوی ضعیف با شدت ν ($\nu < \mu$) و واکوئم با شدت صفر. هدف از این توزیع، آن است که حمله Photon-Number Splitting (PNS) که علیه منبع Weak Coherent Pulse (WCP) با توزیع Poisson قابل اعمال است، خنثی شود. در این حمله، حامله‌ور (eavesdropper) با تقسیم پالس‌های چندفوتونی، یک کپی از فوتون را نگه می‌دارد و بقیه را به گیرنده ارسال می‌کند؛ اما در حضور دکوی، حامله‌ور نمی‌تواند تشخیص دهد که کدام پالس سیگنال و کدام دکوی است و بنابراین، حمله PNS قابل اعمال نیست.

پارامترهای کلیدی این پروتکل عبارت‌اند از: q_x (sifting factor)، کسر پالس‌هایی که پایه‌های Alice و Bob هم‌خوان هستند، p_s (احتمال تخصیص پالس به سیگنال)، p_d (احتمال تخصیص پالس به دکوی)، $p_v = 1 - p_s - p_d$ (احتمال تخصیص به واکوئم)، μ (شدت میانگین سیگنال) و ν (شدت میانگین دکوی). انتخاب این پارامترها، یک مسئله بهینه‌سازی غیرخطی است که هدف آن، حداکثرسازی نرخ کلید امن است. این بهینه‌سازی، به‌دلیل وابستگی پیچیده SKR به این پارامترها (از طریق استخراج Y_1 و محاسبه e_1 و E_μ)، به‌سادگی قابل حل نیست و نیاز به حلال‌های عددی نظیر SLSQP یا Nelder-Mead دارد.

۲.۲.۱۵ آنتروپی باینری شانون و نقش آن در امنیت

آنتروپی باینری شانون که با $H_2(p)$ نشان داده می‌شود، یکی از مفاهیم بنیادین نظریه اطلاعات است که در رمزنگاری کوانتومی نقشی مرکزی ایفا می‌کند. این کمیت، میزان عدم قطعیت متغیر تصادفی باینری با توزیع Bernoulli را اندازه می‌گیرد:

$$H_2(p) = -p \log_2(p) - (1-p) \log_2(1-p) \quad (1.15)$$

در این رابطه، $p \in [0, 1]$ احتمال وقوع یکی از دو حالت است. تابع $H_2(p)$ در $p = 0.5$ به مقدار بیشینه 1 می‌رسد و در نقاط $p = 0$ و $p = 1$ به صفر میل می‌کند. در رمزنگاری کوانتومی، $H_2(p)$ در دو نقطه بحرانی ظاهر می‌شود: نخست در محاسبه privacy amplification که حجم اطلاعات نشت‌کرده به حامله‌ور را تعیین می‌کند ($H_2(E_\mu)$ و $H_2(e_1)$)، و دوم در محاسبه ضریب privacy amplification در فرمول GLLP.

پیاده‌سازی تابع H_2 در کد، با برش عددی در نقاط $p = 0$ و $p = 1$ همراه است، زیرا در این نقاط، عبارت $\log_2(p)$ به $-\infty$ میل می‌کند. در محاسبات عددی، این حالت می‌تواند منجر به NaN یا ValueError شود؛ بنابراین پیاده‌سازی پیش از فراخوانی `math.log2`، شرایط $p \leq 0$ یا $p \geq 1$ را بررسی کرده و در این موارد، مقدار 0 را برمی‌گرداند. این برش با حد گسسته L'Hôpital هم‌خوان است:

$$\lim_{p \rightarrow 0^+} p \log_2 p = \lim_{p \rightarrow 0^+} \frac{\log_2 p}{1/p} = \lim_{p \rightarrow 0^+} \frac{-p}{\ln 2} = 0 \quad (2.15)$$

۳.۲.۱۵ کاهیدگی فیبر و قانون Beer-Lambert

انتقال فوتون در فیبر نوری تک‌حالت در طول موج مخابراتی 1550 نانومتر با قانون Beer-Lambert مدل می‌شود. برای یک فیبر به طول L کیلومتر با ضریب کاهیدگی خطی $\alpha_{\text{dB/km}}$ ، کاهیدگی کل بر حسب دسی‌بل برابر با $\alpha_{\text{dB/km}} \cdot L$ است و تبدیل آن به ضریب انتقال خطی از رابطه زیر پیروی می‌کند:

$$\eta_{\text{channel}}(L, \alpha_{\text{dB/km}}) = 10^{-\frac{\alpha_{\text{dB/km}} \cdot L}{10}} \quad (3.15)$$

در این پیاده‌سازی، $\alpha_{dB/km} = 0.2$ (مقدار استاندارد فیبر تک‌حالت در 1550 نانومتر) به کار رفته است. در فاصله $L = 50$ کیلومتر، $\eta_{channel} = 10^{-1} = 0.1$ و در فاصله $L = 100$ کیلومتر، $\eta_{channel} = 10^{-2} = 0.01$ می‌شود. ضریب انتقال کلی سیستم شامل بازده آشکارساز η_{det} نیز می‌شود:

$$\eta_{total}(L) = \eta_{det} \cdot \eta_{channel}(L, \alpha_{dB/km}) \quad (۴.۱۵)$$

در این پیاده‌سازی، $\eta_{det} = 0.15$ که متناظر با آشکارساز InGaAs SPAD در دمای 50- درجه سانتی‌گراد است. بنابراین در $L = 50$ کیلومتر، $\eta_{total} = 0.015$ می‌شود.

۴.۲.۱۵ Gain سیگنال و QBER سیگنال

برای منبع WCP با توزیع Poisson با شدت میانگین μ ، Gain سیگنال (یعنی احتمال آنکه یک پالس منجر به ثبت کلیک در آشکارساز شود) برابر است با:

$$Q_{\mu}(\mu, \eta, Y_0) = 1 - (1 - Y_0) \cdot e^{-\eta\mu} \quad (۵.۱۵)$$

در این رابطه، Y_0 نرخ dark count به ازای هر پالس، η ضریب انتقال کلی سیستم و μ شدت میانگین سیگنال است. عبارت $e^{-\eta\mu}$ احتمال آنکه هیچ فوتونی شناسایی نشود و عامل $(1 - Y_0)$ احتمال آنکه دارک کانت نیز رخ ندهد را نشان می‌دهد. QBER سیگنال، یعنی کسر کلیک‌های اشتباه در میان کلیک‌های ثبت‌شده، از رابطه زیر به دست می‌آید:

$$E_{\mu} = \frac{0.5 \cdot Y_0 + e_d \cdot (1 - e^{-\eta\mu})}{Q_{\mu}} \quad (۶.۱۵)$$

در این رابطه، e_d خطای ذاتی سیستم (misalignment error) است. صورت این کسر از دو بخش تشکیل شده است: $0.5 \cdot Y_0$ که نشان‌دهنده کلیک‌های اشتباه ناشی از دارک کانت است (چون دارک کانت کاملاً تصادفی است، نصف آن در پایه درست و نصف در پایه اشتباه رخ می‌دهد) و $e_d \cdot (1 - e^{-\eta\mu})$ که نشان‌دهنده کلیک‌های اشتباه ناشی از فوتون‌های واقعی با احتمال خطای ذاتی e_d است.

۵.۲.۱۵ Gain دکوی و استخراج Y_1 به روش Lo2005

Gain دکوی ضعیف به صورت مشابه محاسبه می‌شود:

$$Q_{\nu}(\nu, \eta, Y_0) = 1 - (1 - Y_0) \cdot e^{-\eta\nu} \quad (۷.۱۵)$$

مرحله کلیدی در decoy-state QKD، استخراج کران پایین برای Y_1 (احتمال آنکه یک پالس تک‌فوتونی منجر به کلیک شود) از مشاهدات Q_{ν} و Q_{μ} است. این کران، که در کار مرجع (Lo-Ma-Chen (2005) به تفصیل استخراج شده، در این ماژول به فرم زیر پیاده‌سازی شده است:

$$Y_1^L = \frac{\mu}{\mu\nu - \nu^2} \left(Q_{\nu} \cdot e^{\nu} - \frac{\nu^2}{\mu^2} \cdot Q_{\mu} \cdot e^{\mu} \right) \quad (۸.۱۵)$$

این فرمول، یک نسخه بازآرایی‌شده از فرمول کلاسیک Lo2005 است. ضریب $\frac{\mu}{\mu\nu - \nu^2} = \frac{\mu}{\nu(\mu - \nu)}$ در صورتی که $\nu \rightarrow \mu$ باشد، به بی‌نهایت میل می‌کند؛ بنابراین در پیاده‌سازی، شرط $|\mu - \nu| < 10^{-9}$ بررسی می‌شود و در این حالت، تابع مقدار صفر را برمی‌گرداند. در این پیاده‌سازی، یک اصلاح فیزیکی مهم نیز اعمال شده است: مقدار نهایی Y_1 با $\max(Y_1^L, Y_0)$ محاسبه می‌شود تا تضمین گردد که Y_1 هرگز کمتر از نرخ دارک کانت Y_0 نشود. این برش، بازتاب‌دهنده این واقعیت فیزیکی است که حتی در غیاب فوتون واقعی، دارک کانت می‌تواند منجر به کلیک شود و بنابراین، بازده تک‌فوتونی نمی‌تواند کمتر از Y_0 باشد.

۶.۲.۱۵ نرخ خطای تک‌فوتونی و بازده تک‌فوتونی

نرخ خطای تک‌فوتونی e_1 ، یعنی کسر کلیک‌های اشتباه در میان پالس‌های تک‌فوتونی، از رابطه زیر به دست می‌آید:

$$e_1 = \frac{0.5 \cdot Y_0 + e_d \cdot \eta}{Y_1} \quad (9.15)$$

این رابطه مشابه E_μ است، با این تفاوت که به جای احتمال کلیک فوتون واقعی $1 - e^{-\eta\mu}$ ، احتمال کلیک تک‌فوتونی η قرار دارد. در پیاده‌سازی، شرط $Y_1 > 0$ بررسی می‌شود تا از تقسیم بر صفر جلوگیری شود؛ در صورت $Y_1 = 0$ ، مقدار $e_1 = 0.5$ گرفته می‌شود که نشان‌دهنده حالت کاملاً تصادفی است (یعنی در غیاب سیگنال واقعی، نتیجه اندازه‌گیری کاملاً تصادفی و با احتمال برابر برای هر بیت است). Gain تک‌فوتونی، یعنی کسری از کلیک‌های کل که از پالس‌های تک‌فوتونی ناشی می‌شوند، از رابطه زیر به دست می‌آید:

$$Q_1 = \mu \cdot e^{-\mu} \cdot Y_1 \quad (10.15)$$

این رابطه از ضرب احتمال آنکه پالس تک‌فوتونی باشد ($P(1|\mu) = \mu e^{-\mu}$) در بازده تک‌فوتونی Y_1 به دست می‌آید.

۷.۲.۱۵ نرخ کلید امن مجانبی (GLLP)

نرخ کلید امن مجانبی (asymptotic secure key rate) از فرمول کلاسیک GLLP به دست می‌آید:

$$R = q_x \cdot [Q_1 \cdot (1 - H_2(e_1)) - Q_\mu \cdot f_{EC} \cdot H_2(E_\mu)] \quad (11.15)$$

این فرمول از دو بخش تشکیل شده است. بخش اول $Q_1 \cdot (1 - H_2(e_1))$ نشان‌دهنده حجم اطلاعات مشترک بین فرستنده و گیرنده از پالس‌های تک‌فوتونی پس از privacy amplification است؛ عبارت $1 - H_2(e_1)$ ضریب privacy amplification است که حجم اطلاعات نشت‌کرده به حامل‌ها را کم می‌کند. بخش دوم $Q_\mu \cdot f_{EC}$ نشان‌دهنده حجم اطلاعات فاش‌شده در مرحله error correction است؛ $f_{EC} \geq 1$ بازدهی error correction است که در عمل $f_{EC} \approx 1.16$ برای کد LDPC بهینه است. در این پیاده‌سازی، $f_{EC} = 1.16$ به کار رفته است که با مقادیر عملی در پیاده‌سازی‌های QKD هم‌خوان است.

تفاوت این فرمول با حالت finite-key (مثل اثبات Lim-2014) آن است که در اینجا از حد مجانبی $N \rightarrow \infty$ استفاده می‌شود و جریمه‌های آماری نظیر Hoeffding bound و composable security اعمال نمی‌گردند. به همین دلیل، R در اینجا همیشه بزرگ‌تر یا مساوی نرخ کلید متناهی است و در عمل به‌عنوان کران بالای نرخ کلید قابل دستیابی عمل می‌کند. با این حال، تطابق حلال بهینه‌سازی با این کران بالا، شرط لازم – هرچند کافی نیست – برای صحت پیاده‌سازی است.

۸.۲.۱۵ قیدهای فیزیکی و ریاضی

در تابع `analytical_skr`، چندین قید فیزیکی و ریاضی اعمال شده است که در صورت نقض، تابع مقدار صفر را برمی‌گرداند. این قیدها عبارت‌اند از:

نخست، قید $p_s + p_d \geq 1$: این قید تضمین می‌کند که مجموع احتمال‌های تخصیص سیگنال و دکوی از 1 تجاوز نکند؛ در غیر این صورت، احتمال واکوئم $p_v = 1 - p_s - p_d$ منفی می‌شود که از لحاظ فیزیکی بی‌معناست. در صورت نقض، تابع مقدار صفر برمی‌گرداند.

دوم، قید $\nu \geq 0.9 \cdot \mu$: این قید تضمین می‌کند که شدت دکوی به‌طور قابل‌توجهی کمتر از شدت سیگنال باشد؛ در غیر این صورت، استخراج Y_1 از روش Lo2005 به دلیل نزدیک بودن μ و ν به تکینگی (singularity) می‌رسد و نتایج غیرقابل اعتماد تولید می‌کند. در عمل، ν معمولاً در بازه 0.1μ تا 0.5μ انتخاب می‌شود. سوم، قید $|\mu - \nu| < 10^{-9}$: این قید حالت تکین را شناسایی می‌کند و در صورت وقوع، تابع مقدار صفر برمی‌گرداند.

این قیدها، الگوی defensive programming را به نمایش می‌گذارند: ورودی‌ها پیش از محاسبه اعتبارسنجی می‌شوند و در صورت غیرمعتبر بودن، مقدار پیش‌فرض امن (صفر) بازگردانده می‌شود. این الگو در توابع بهینه‌سازی حیاتی است، چون حلال ممکن است در جریان جستجوی فضای پارامتر، به مقادیر غیرفیزیکی برسد و بدون این قیدها، تابع هدف می‌تواند مقادیر غیرمعتبر یا NaN تولید کند.

۳.۱۵ تحلیل معماری و ساختار داده‌ها

۱.۳.۱۵ ساختار کلی ماژول و الگوی Module-Level Functions

این ماژول از الگوی module-level functions پیروی می‌کند، یعنی به‌جای تعریف کلاس‌ها، از توابع مستقل در سطح ماژول استفاده می‌نماید. این انتخاب طراحی، بازتاب‌دهنده ماهیت procedural این ماژول است: توابع تحلیلی (H2 و analytical_skr) توابع ریاضی خالص (pure functions) هستند که ورودی‌هایشان را به خروجی‌هایشان نگاشت می‌کنند و هیچ state داخلی ندارند. تابع run_lp_test تنها تابع دارای side-effect است که خروجی چاپ می‌کند و حلال را فراخوانی می‌نماید. این جداسازی بین توابع خالص و توابع دارای side-effect، یکی از اصول مهندسی نرم‌افزار بالغ است و در الگوهای طراحی نظیر functional core, imperative shell تبیین می‌شود.

۲.۳.۱۵ تنظیم مسیر و Bootstrap Path

در ابتدای ماژول، الگوی path bootstrap پیاده‌سازی شده است:

```
۱ sys.path.insert(0, os.path.abspath(os.path.dirname(__file__)))
۲
```

این خط، مسیر دایرکتوری حاوی اسکریپت را به ابتدای sys.path اضافه می‌کند. این کار تضمین می‌نماید که ماژول بدون توجه به محل اجرای آن (از project root یا از مسیر دیگر)، بتواند ماژول‌های qkd.proofs.optimization و qkd.params را وارد نماید. این الگو در اسکریپت‌هایی که به‌صورت مستقل اجرا می‌شوند و نیاز به واردسازی ماژول‌های پروژه دارند، استاندارد است.

۳.۳.۱۵ واردسازی توابع از فریم‌ورک

دو واردسازی کلیدی در این ماژول انجام می‌شود:

```
۱ from qkd.proofs.optimization import optimize_lim2014_finite_key
۲ from qkd.params import load_lim2014_dedicated_params
۳
```

نخست، تابع optimize_lim2014_finite_key از ماژول qkd.proofs.optimization وارد می‌شود. این تابع، حلال بهینه‌سازی پارامترهای پروتکل Lim-2014 است که از الگوریتم Nelder-Mead یا SLSQP برای یافتن پارامترهای بهینه $(\nu, \mu, p_d, p_s, q_x)$ استفاده می‌کند. دوم، تابع load_lim2014_dedicated_params از ماژول qkd.params وارد می‌شود. این تابع، پارامترهای اختصاصی پروتکل Lim-2014 (مانند ε_{EC} , ε_{PE} , N) و سایر پارامترهای امنیتی را برای یک فاصله و تعداد بیت مشخص بارگذاری می‌نماید. این وابستگی‌ها نشان‌دهنده آن است که ماژول اعتبارسنجی، مستقیماً به موتور بهینه‌سازی فریم‌ورک متصل است و همان مسیر اجرای واقعی را آزمون می‌کند. این الگو، تضمین می‌کند که آزمون، رفتار واقعی موتور را در شرایط عملیاتی منعکس می‌کند و نه رفتار آزمایشگاهی مصنوعی.

۴.۳.۱۵ تحلیل تابع H2

تابع $H_2(p)$ یک تابع ریاضی خالص است که آنتروپی باینری شانون را محاسبه می‌نماید:

```
۱ def H2(p):
۲     if p <= 0 or p >= 1: return 0.0
۳     return -p * math.log2(p) - (1 - p) * math.log2(1 - p)
۴
```

در خط اول، شرط $p \leq 0$ یا $p \geq 1$ بررسی می‌شود. این شرط شامل نقاط انتهایی $p = 0$ و $p = 1$ نیز می‌شود که در آن‌ها آنتروپی به‌صورت تحلیلی صفر است. در خط دوم، فرمول اصلی $-p \log_2 p - (1 - p) \log_2 (1 - p)$ پیاده‌سازی می‌شود. چون این خط فقط در صورتی اجرا می‌شود که $p \in (0, 1)$ ، فراخوانی math.log2 همیشه با ورودی مثبت انجام می‌شود و خطای math domain error رخ نمی‌دهد. این پیاده‌سازی، الگوی defensive programming را به نمایش می‌گذارد.

۵.۳.۱۵ تحلیل تابع analytical_skr و ساختار پارامترها

تابع analytical_skr اصلی‌ترین تابع تحلیلی این ماژول است که نرخ کلید امن را بر اساس پنج پارامتر پروتکل محاسبه می‌کند. این تابع نه پارامتر ورودی دارد: q_x (sifting factor)، p_s (احتمال سیگنال)، p_d (احتمال دکوی)، μ (شدت سیگنال)، ν (شدت دکوی)، η (ضریب انتقال کلی)، Y_0 (نرخ دارک کانت)، e_d (خطای ذاتی) و f_{EC} (بازدهی error correction). خروجی تابع، یک عدد اعشاری است که نرخ کلید امن را نشان می‌دهد.

```

1 def analytical_skr(q_x, p_s, p_d, mu, nu, eta, Y0, ed, f_ec):
2     """
3     if p_s + p_d >= :0.1 return 0.0
4     if nu >= mu * :9.0 return 0.0
5
6     Q_mu = 1 - (1 - Y0) * math.exp(-eta * mu)
7     E_mu = 5.0 * Y0 + ed * (1 - math.exp(-eta * mu)) / Q_mu
8     Q_nu = 1 - (1 - Y0) * math.exp(-eta * nu)
9

```

در خطوط ابتدایی، دو قید فیزیکی بررسی می‌شوند: قید $p_s + p_d \geq 1$ و قید $\nu \geq 0.9\mu$. در صورت نقض هر یک، تابع بلافاصله مقدار صفر را برمی‌گرداند. این الگوی early return از پردازش غیرضروری جلوگیری می‌کند و خوانایی کد را بالا می‌برد. سپس سه کمیت کلیدی Q_μ ، E_μ و Q_ν محاسبه می‌شوند. این کمیت‌ها پایه محاسبات بعدی هستند.

```

1 if abs(mu - nu) < 1e-9: return 0.0
2
3 # Lo2005 Y1
4 term1 = Q_nu * math.exp(nu)
5 term2 = (nu**2 / mu**2) * Q_mu * math.exp(mu)
6 factor = mu / (mu * nu - nu**2)
7 Y1_L = factor * (term1 - term2)
8
9 Y1 = max(Y1_L, Y0) # Y1
10 e1 = 5.0 * Y0 + ed * eta / Y1 if Y1 > 0 else 5.0
11

```

در این بخش، کران پایین Y_1^L محاسبه می‌شود. شرط $|\mu - \nu| < 10^{-9}$ حالت تکین را شناسایی می‌کند؛ در این حالت، تابع مقدار صفر را برمی‌گرداند. در غیر این صورت، فرمول اصلی Lo2005 پیاده می‌شود: term1 و term2 دو جمله اصلی صورت کسر هستند و factor ضریب نرمال‌سازی است. در نهایت، $\max(Y1_L, Y0)$ تضمین می‌کند که Y_1 نهایی کمتر از Y_0 نشود؛ این برش فیزیکی، بازتاب‌دهنده این واقعیت است که حتی در غیاب فوتون واقعی، دارک کانت می‌تواند منجر به کلیک شود. محاسبه e_1 نیز با شرط $Y_1 > 0$ همراه است تا از تقسیم بر صفر جلوگیری شود؛ در صورت $Y_1 = 0$ ، مقدار $e_1 = 0.5$ گرفته می‌شود که نشان‌دهنده حالت کاملاً تصادفی است.

```

1 Q1 = mu * math.exp(-mu) * Y1
2 R = q_x * (Q1 * (1 - H2(e1)) - Q_mu * f_ec * H2(E_mu))
3 return max(R, 0.0)
4

```

در بخش پایانی، Q_1 و R محاسبه می‌شوند. Q_1 از ضرب $\mu e^{-\mu}$ (احتمال پالس تک‌فوتونی) در Y_1 (بازده تک‌فوتونی) به دست می‌آید. R از فرمول GLLP محاسبه می‌شود و در نهایت، $\max(R, 0.0)$ تضمین می‌کند که نرخ کلید منفی نشود؛ این برش در رژیم‌های با QBER بالا (مثلاً $E_\mu > 0.11$ که آستانه بحرانی BB84 است) ضروری است، چون در این رژیم، عبارت $Q_\mu \cdot f_{EC} \cdot H_2(E_\mu)$ بزرگ‌تر از $Q_1 \cdot (1 - H_2(e_1))$ می‌شود و R منفی می‌گردد. در عمل، نرخ کلید منفی به این معناست که هیچ کلید امنی قابل استخراج نیست و مقدار صفر گزارش می‌شود.

۶.۳.۱۵ تحلیل تابع run_lp_test و الگوی Callback Cost

تابع run_lp_test اصلی‌ترین تابع آزمون است که در یک فاصله مشخص، حلال بهینه‌سازی را فراخوانی کرده و نتایج را مقایسه می‌نماید. این تابع یک پارامتر ورودی dist_km دارد که فاصله به کیلومتر است.

```

1 def run_lp_test(dist_km):
2     print(f"\n[TEST] LP Solver at L={dist_km} km")

```

```

۳ eta_det = 15.0
۴ Y0 = 3e-6
۵ ed = 01.0
۶ f_ec = 16.1
۷ alpha = 2.0
۸ eta = 10 ** (-alpha * dist_km / 10) * eta_det
۹

```

در ابتدا، پارامترهای فیزیکی ثابت تعریف می‌شوند: $f_{EC} = 1.16$, $e_d = 0.01$, $Y_0 = 3 \times 10^{-6}$, $\eta_{det} = 0.15$ و $\alpha = 0.2$ (dB/km). سپس ضریب انتقال کلی η بر اساس قانون Beer-Lambert محاسبه می‌شود. این پارامترها با مقادیر مرجع در ادبیات QKD هم‌خوان هستند و در شرایط عملیاتی واقعی برای QKD در فواصل مخابراتی متوسط قابل دستیابی هستند.

```

۱ params = load_lim2014_dedicated_params(distance_km=dist_km, num_bits=10**7)
۲
۳ def cost_callback(q_x, p_s, p_d, mu, nu):
۴     # SKR
۵     skr = analytical_skr(q_x, p_s, p_d, mu, nu, eta, Y0, ed, f_ec)
۶     return -skr
۷

```

در این بخش، پارامترهای پروتکل Lim-2014 برای فاصله و تعداد بیت مشخص بارگذاری می‌شوند. سپس تابع `cost_callback` به‌عنوان یک closure تعریف می‌شود. این تابع، پنج پارامتر پروتکل را می‌گیرد و منفی SKR را برمی‌گرداند. دلیل این `negate` کردن آن است که حلال بهینه‌سازی به‌طور پیش‌فرض به دنبال مینیمم‌کردن تابع هزینه است؛ بنابراین برای حداکثرسازی SKR، باید منفی آن را مینیمم کرد. این الگو در ادبیات بهینه‌سازی به‌عنوان minimization of negated objective شناخته می‌شود. این تابع closure به متغیرهای f_{EC} و e_d , Y_0 , η از محیط بیرونی دسترسی دارد (closure capture)؛ این الگو امکان تزریق پارامترهای محیطی به تابع هدف را بدون نیاز به ارسال آن‌ها به‌عنوان پارامتر فراهم می‌سازد. این الگو در توابع callback بسیار رایج است و خوانایی کد را بالا می‌برد.

۷.۳.۱۵ الگوی Reference Hardcoded و Comparison

در ادامه، SKR با پارامترهای سخت‌کدشده محاسبه می‌شود و سپس حلال فراخوانی می‌گردد:

```

۱ # . SKR
۲ R_hardcoded = analytical_skr(5.0), 5.0, 25.0, 5.0, 1.0, eta, Y0, ed, f_ec)
۳
۴ # . LP Solver
۵ try:
۶     q_x_opt, p_s_opt, p_d_opt, mu_opt, nu_opt = optimize_lim2014_finite_key(
۷         params, dist_km, cost_callback)
۸     R_optimized = analytical_skr(q_x_opt, p_s_opt, p_d_opt, mu_opt, nu_opt,
۹         eta, Y0, ed, f_ec)
۱۰

```

در این بخش، $R_{hardcoded}$ با پارامترهای مرجع $q_x = 0.5$, $p_s = 0.5$, $p_d = 0.25$, $\mu = 0.5$ و $\nu = 0.1$ محاسبه می‌شود. این پارامترها، مقادیر کلاسیک و شناخته‌شده در ادبیات QKD decoy-state هستند و به‌عنوان نقطه مرجع عمل می‌کنند. سپس حلال `optimize_lim2014_finite_key` فراخوانی می‌شود و پارامترهای بهینه را برمی‌گرداند. $R_{optimized}$ با این پارامترهای بهینه محاسبه می‌شود. استفاده از بلاک `try/except` برای مدیریت خطاهای احتمالی حلال ضروری است؛ چون حلال بهینه‌سازی ممکن است در شرایط دشوار (مثلاً در فواصل بسیار بزرگ که SKR نزدیک به صفر است) به همگرایی نرسد یا خطا تولید کند. در صورت وقوع خطا، بلاک `except` پیام خطا و `traceback` کامل را چاپ می‌نماید و تابع مقدار `False` برمی‌گرداند.

۸.۳.۱۵ الگوریتم Verdict و Comparison

در نهایت، نتایج مقایسه شده و verdict اعلام می‌گردد:

```

۱ print(f" Hardcoded (mu5.0=, nu(1.0= -> SKR: {R_hardcoded6.:e}")
۲ print(f" Optimized (mu={mu_opt3.:f}, nu={nu_opt3.:f}) -> SKR: {R_optimized6.:e}")
۳

```



```

۴     if R_optimized >= R_hardcoded - 1e-9:
۵         print(" PASS: LP Solver successfully found optimal or equal parameters.")
۶         return True
۷     else:
۸         print(" FAIL: LP Solver performed worse than hardcoded parameters!")
۹         return False
۱۰

```

در این الگوریتم، شرط $R_{\text{optimized}} \geq R_{\text{hardcoded}} - 10^{-9}$ بررسی می‌شود. در صورت برقراری، آزمون موفق تلقی می‌گردد؛ در غیر این صورت، شکست. استفاده از 10^{-9} به عنوان تلورانس، به این دلیل است که در محاسبات عددی، خطای گردکردن می‌تواند منجر به تفاوت‌های بسیار کوچک شود و این تلورانس تضمین می‌کند که چنین تفاوت‌های جزئی به عنوان شکست تلقی نشوند. این الگو، الگوی tolerance-based comparison است که در آزمون‌های عددی استاندارد است.

۹.۳.۱۵ نقطه ورود __main__ و سه آزمون

```

۱     if __name__ == "__main__":
۲         print("=" * 60)
۳         print("LP SOLVER (PARAMETER OPTIMIZATION) VALIDATION")
۴         print("=" * 60)
۵
۶         #
۷         res1 = run_lp_test(0.0)
۸         res2 = run_lp_test(0.50)
۹         res3 = run_lp_test(0.100)
۱۰
۱۱        print("\n" + "=" * 60)
۱۲        if res1 and res2 and res3:
۱۳            print("100% LP SOLVER VALIDATION PASSED!")
۱۴        else:
۱۵            print("LP SOLVER VALIDATION FAILED.")
۱۶        print("=" * 60)
۱۷

```

نقطه ورود __main__ یک سرآیند تزئینی چاپ می‌کند و سپس سه آزمون در فواصل 0، 50 و 100 کیلومتر را اجرا می‌نماید. این سه فاصله، سه رژیم فیزیکی متفاوت را پوشش می‌دهند: فاصله 0 کیلومتر (بدون کاهیدگی)، فاصله 50 کیلومتر (کاهیدگی متوسط با $\eta_{\text{channel}} \approx 0.1$) و فاصله 100 کیلومتر (کاهیدگی شدید با $\eta_{\text{channel}} \approx 0.01$). در صورت موفقیت همه سه آزمون، پیام 100% LP SOLVER VALIDATION PASSED! چاپ می‌شود و در غیر این صورت، پیام شکست. این الگو، تضمین می‌کند که حلال در طیف وسیعی از شرایط عملیاتی به طور قابل اتکا عمل می‌کند.

۴.۱۵ پیاده‌سازی ریاضی و الگوریتمی

۱.۴.۱۵ پیاده‌سازی آنتروپی باینری با برش عددی

پیاده‌سازی تابع H_2 در این ماژول، نمونه‌ای از defensive programming در محاسبات عددی است. فرمول تحلیلی $-p \log_2 p - (1-p) \log_2 (1-p)$ در نقاط $p = 0$ و $p = 1$ به صورت تحلیلی به 0 میل می‌کند، اما در محاسبات عددی، فراخوانی $\text{math.log2}(0)$ منجر به ValueError می‌شود. برای حل این مشکل، شرط $p \leq 0$ یا $p \geq 1$ پیش از فراخوانی math.log2 بررسی می‌شود و در این موارد، مقدار 0 بازگردانده می‌شود. این برش عددی با حد گسسته L'Hôpital هم‌خوان است و از نظر ریاضی صحیح است.

۲.۴.۱۵ پیاده‌سازی فرمول Lo^{2005} با اصلاحات فیزیکی

پیاده‌سازی فرمول Y_1^L در تابع `analytical_skr` چندین اصلاح مهم دارد نسبت به فرمول کلاسیک Lo^{2005} . نخست، فرمول به فرم زیر بازآرایی شده است:

$$Y_1^L = \frac{\mu}{\mu\nu - \nu^2} \left(Q_\nu \cdot e^\nu - \frac{\nu^2}{\mu^2} \cdot Q_\mu \cdot e^\mu \right) \quad (۱۲.۱۵)$$

این بازآرایی، معادل فرمول کلاسیک Lo2005 است، اما با ساختار متفاوت. در فرمول کلاسیک، ضریب نرمال‌سازی به فرم $\frac{\mu^2 e^{-\mu}}{\mu\nu - \nu^2}$ است؛ اما در این پیاده‌سازی، ضریب به فرم $\frac{\mu}{\mu\nu - \nu^2}$ بازنویسی شده است. این بازنویسی، با استفاده از رابطه $\mu^2 - \nu^2 = \mu(\mu - \nu) = -\nu(\mu - \nu) + (\mu^2 - \nu^2)$ و simplification جبری، قابل اثبات است.

دوم، اصلاح فیزیکی $\max(Y1_L, Y0)$ اعمال شده است. این اصلاح، تضمین می‌کند که Y_1 هرگز کمتر از Y_0 نشود. این برش، بازتاب‌دهنده این واقعیت فیزیکی است که حتی در غیاب فوتون واقعی، دارک کانت می‌تواند منجر به کلیک شود و بنابراین، بازده تک‌فوتونی نمی‌تواند کمتر از نرخ دارک کانت باشد. این اصلاح، در برخی نسخه‌های فرمول Lo2005 به صورت ضمنی اعمال می‌شود، اما در این پیاده‌سازی به صورت صریح و قابل مشاهده اعمال گردیده است.

سوم، در محاسبه e_1 ، در صورت $Y_1 = 0$ ، مقدار $e_1 = 0.5$ گرفته می‌شود. این انتخاب، بازتاب‌دهنده حالت کاملاً تصادفی است (یعنی در غیاب سیگنال واقعی، نتیجه اندازه‌گیری کاملاً تصادفی و با احتمال برابر برای هر بیت است). این انتخاب، در طراحی امنیتی QKD محافظه‌کارانه است؛ چون فرض می‌کند که در صورت عدم اطمینان، حامل‌ها می‌توانند کل اطلاعات را بدانند.

۳.۴.۱۵ پیاده‌سازی قیدهای فیزیکی و Return Early

پیاده‌سازی قیدهای فیزیکی با الگوی early return انجام می‌شود:

```

۱ if p_s + p_d >= :0.1 return 0.0
۲ if nu >= mu * :9.0 return 0.0
۳ ...
۴ if abs(mu - nu) < 1e-9: return 0.0
۵
```

این الگو، چندین مزیت دارد: نخست، خوانایی کد را بالا می‌برد، چون هر قید به صورت مستقل و در ابتدای تابع بررسی می‌شود. دوم، از پردازش غیرضروری جلوگیری می‌کند؛ در صورت نقض قید، بقیه محاسبات انجام نمی‌شود. سوم، در توابع بهینه‌سازی حیاتی است، چون حلال ممکن است در جریان جستجوی فضای پارامتر، به مقادیر غیرفیزیکی برسد و بدون این قیدها، تابع هدف می‌تواند مقادیر غیرمتعارف یا NaN تولید کند که حلال را در همگرایی مختل می‌نماید.

قید $\nu \geq 0.9\mu$ به طور خاص جالب توجه است؛ این قید، تلورانسی برای حالت تکین فرمول Lo2005 فراهم می‌سازد. در فرمول کلاسیک، تکینگی در $\nu = \mu$ رخ می‌دهد، اما در عمل، حتی نزدیک بودن ν به μ (مثلاً $\nu = 0.95\mu$) می‌تواند منجر به نتایج غیرقابل اعتماد شود. این قید، تضمین می‌کند که ν به طور قابل توجهی کمتر از μ باشد.

۴.۴.۱۵ الگوریتم Trick Negate و Callback Cost

الگوریتم cost callback با negate trick یک الگوی کلاسیک در بهینه‌سازی است:

```

۱ def cost_callback(q_x, p_s, p_d, mu, nu):
۲     skr = analytical_skr(q_x, p_s, p_d, mu, nu, eta, Y0, ed, f_ec)
۳     return -skr
۴
```

در این الگوریتم، تابع هدف واقعی SKR محاسبه می‌شود، اما به جای بازگرداندن آن، منفی آن بازگردانده می‌شود. دلیل این کار آن است که اکثر حلال‌های بهینه‌سازی (مانند `scipy.optimize.minimize`) به طور پیش‌فرض به دنبال مینیمم کردن تابع هستند؛ بنابراین برای حداکثرسازی SKR، باید منفی آن را مینیمم کرد. این الگو در ادبیات بهینه‌سازی به عنوان minimization of negated objective شناخته می‌شود. این الگو، یک ویژگی مهم دارد: امکان استفاده از همان حلال برای هر دو مسئله مینیمم‌سازی و حداکثرسازی. به عنوان مثال، اگر بخواهیم QBER را مینیمم کنیم، تابع هدف QBER را مستقیماً برمی‌گردانیم؛ و اگر بخواهیم SKR را حداکثر کنیم، منفی آن را برمی‌گردانیم. این یکپارچگی، در طراحی ماژولار سیستم بهینه‌سازی ارزشمند است.

۵.۴.۱۵ الگوریتم Capture Closure برای پارامترهای محیطی

الگوریتم closure capture در تعریف `cost_callback` به کار رفته است:

```

۱ eta = 10 ** (-alpha * dist_km / 10) * eta_det
۲ ...
۳ def cost_callback(q_x, p_s, p_d, mu, nu):
۴     skr = analytical_skr(q_x, p_s, p_d, mu, nu, eta, Y0, ed, f_ec)
۵     return -skr
۶

```

در این الگوریتم، متغیرهای η ، Y_0 ، e_d و f_{EC} در محیط بیرونی تابع `cost_callback` تعریف می‌شوند و تابع `closure` به آن‌ها دسترسی دارد. این الگو، چندین مزیت دارد: نخست، امضای تابع `callback` ساده می‌ماند (فقط پنج پارامتر پروتکل) و حلال نیازی به دانستن پارامترهای محیطی ندارد. دوم، پارامترهای محیطی می‌توانند در زمان تعریف `closure` محاسبه شوند و در تمام فراخوانی‌های بعدی استفاده گردند؛ این کار از محاسبه مکرر آن‌ها جلوگیری می‌کند. سوم، در صورت نیاز به تغییر پارامترهای محیطی (مثلاً برای آزمون در فاصله دیگر)، فقط کافی است `closure` در یک محیط جدید تعریف شود.

این الگو در برنامه‌نویسی تابعی به‌کار می‌رود و در پایتون با کلمات کلیدی `lambda` یا `def` در داخل تابع دیگر، قابل پیاده‌سازی است.

۶.۴.۱۵ الگوریتم Comparison Reference Hardcoded

الگوریتم مقایسه با مرجع سخت‌کدشده، الگوریتم ساده اما مؤثری است:

```

۱ R_hardcoded = analytical_skr(5.0), 5.0, 25.0, 5.0, 1.0, eta, Y0, ed, f_ec)
۲ ...
۳ R_optimized = analytical_skr(q_x_opt, p_s_opt, p_d_opt, mu_opt, nu_opt,
۴     eta, Y0, ed, f_ec)
۵ ...
۶ if R_optimized >= R_hardcoded - 1e-9:
۷     return True
۸ else:
۹     return False
۱۰

```

در این الگوریتم، $R_{\text{hardcoded}}$ با پارامترهای مرجع محاسبه می‌شود و سپس با $R_{\text{optimized}}$ مقایسه می‌گردد. شرط موفقیت، آن است که $R_{\text{optimized}}$ حداقل برابر با $R_{\text{hardcoded}}$ (با تلووانس 10^{-9}) باشد. این الگوریتم، چندین ویژگی دارد:

نخست، استفاده از پارامترهای مرجع $\mu = 0.5$ و $\nu = 0.1$ که در ادبیات QKD decoy-state به‌عنوان مقادیر کلاسیک شناخته شده‌اند. این پارامترها، نقطه عملیاتی متعارف هستند و در بسیاری از پیاده‌سازی‌های آزمایشگاهی به‌کار رفته‌اند. انتخاب آن‌ها به‌عنوان مرجع، تضمین می‌کند که حلال حداقل به اندازه یک راه‌حل شناخته‌شده عمل می‌کند.

دوم، استفاده از تلووانس 10^{-9} برای جلوگیری از خطای گردکردن. در محاسبات عددی، خطای گردکردن می‌تواند منجر به تفاوت‌های بسیار کوچک شود و این تلووانس تضمین می‌کند که چنین تفاوت‌های جزئی به‌عنوان شکست تلقی نشوند.

سوم، این الگوریتم، الگوی necessary condition testing است: موفقیت آزمون تضمین می‌کند که حلال حداقل به اندازه مرجع عمل می‌کند؛ هرچند ممکن است پارامترهای بهتر نیز یافت کند. در صورت شکست، قطعاً حلال دارای باگ است و نیاز به بررسی دارد.

۷.۴.۱۵ الگوریتم Testing Three-Range

الگوریتم سه‌آزمون در فواصل 0، 50 و 100 کیلومتر، الگوریتم هوشمندانه‌ای است:

```

۱ res1 = run_lp_test(0.0)
۲ res2 = run_lp_test(0.50)
۳ res3 = run_lp_test(0.100)
۴

```

این سه فاصله، سه رژیم فیزیکی متفاوت را پوشش می‌دهند: نخست، فاصله $L = 0$ کیلومتر: در این حالت، $\eta_{\text{channel}} = 1$ و $\eta_{\text{total}} = \eta_{\text{det}} = 0.15$. در این رژیم، کاهیدگی فیبر وجود ندارد و حلال باید پارامترهایی با SKR نزدیک به حد نظری پیدا کند. در این رژیم، مقدار بهینه μ معمولاً بزرگ‌تر از 0.5 است، چون کاهیدگی کم است و پالس‌های چندفوتونی کمتر مشکل‌ساز می‌شوند.

دوم، فاصله $L = 50$ کیلومتر: در این حالت، $\eta_{\text{channel}} \approx 0.1$ و $\eta_{\text{total}} \approx 0.015$. در این رژیم، کاهیدگی متوسط است و حلال باید با کاهش μ برای جبران کاهیدگی، نرخ کلید را بهینه نماید. مقدار بهینه μ معمولاً در بازه 0.3 تا 0.5 است.

سوم، فاصله $L = 100$ کیلومتر: در این حالت، $\eta_{\text{channel}} \approx 0.01$ و $\eta_{\text{total}} \approx 0.0015$. در این رژیم، کاهیدگی شدید است و حلال در رژیم نزدیک به آستانه بحرانی BB84 ($Q \approx 0.11$) عمل می‌کند. مقدار بهینه μ معمولاً در بازه 0.1 تا 0.3 است و انتخاب پارامترها به‌طور حساس به نتیجه تأثیر می‌گذارد.

این سه آزمون، تضمین می‌کنند که حلال در طیف وسیعی از شرایط عملیاتی به‌طور قابل‌اتکا عمل می‌کند و در رژیم‌های مختلف فیزیکی، پارامترهای بهینه را پیدا می‌نماید.

۵.۱۵ ارسال و دریافت با سایر ماژول‌ها

۱.۵.۱۵ وابستگی به `qkd.proofs.optimization`

اصلی‌ترین وابستگی این ماژول، به `qkd.proofs.optimization` است. این وابستگی از طریق تابع `optimize_lim2014_finite_key` برقرار می‌شود که حلال بهینه‌سازی پارامترهای پروتکل Lim-2014 است. این تابع، سه پارامتر ورودی دارد: `params` (پارامترهای پروتکل)، `dist_km` (فاصله) و `cost_callback` (تابع هزینه). خروجی آن، پنج پارامتر بهینه $(\nu, \mu, p_d, p_s, q_x)$ است.

این وابستگی، یک وابستگی `loose coupling` از طریق `callback` است: حلال به‌جز امضای تابع `callback`، هیچ چیز دیگری از تابع هدف نمی‌داند. این الگو، امکان جایگزینی تابع هدف با توابع دیگر را فراهم می‌سازد. به‌عنوان مثال، می‌توان به‌جای تابع `analytical_skr` از یک تابع هزینه پیچیده‌تر (مثلاً SKR متناهی-key یا تابع هزینه با جریمه‌های اضافی) استفاده کرد، بدون اینکه حلال نیاز به تغییر داشته باشد.

۲.۵.۱۵ وابستگی به `qkd.params`

دومین وابستگی این ماژول، به `qkd.params` است. این وابستگی از طریق تابع `load_lim2014_dedicated_params` برقرار می‌شود که پارامترهای اختصاصی پروتکل Lim-2014 را بارگذاری می‌نماید. این تابع، دو پارامتر ورودی دارد: `distance_km` و `num_bits`. خروجی آن، یک شیء `dict` یا `dataclass` است که شامل پارامترهای امنیتی پروتکل (مانند $\epsilon_s, \epsilon_{\text{EC}}, \epsilon_{\text{PE}}$ و سایر پارامترهای متناهی-key) است.

این پارامترها، در حلال بهینه‌سازی برای محاسبه جریمه‌های متناهی-key استفاده می‌شوند. در این ماژول اعتبارسنجی، از این پارامترها فقط برای فراخوانی حلال استفاده می‌شود و در تابع هدف تحلیلی (`analytical_skr`) مستقیماً به‌کار نمی‌روند؛ چون تابع هدف، حد مجانبی را محاسبه می‌کند و جریمه‌های متناهی-key را در بر نمی‌گیرد. این جداسازی، الگوی `independent oracle` است که در بخش بعد توضیح داده می‌شود.

۳.۵.۱۵ الگوی `Oracle Independent` و جلوگیری از `Validation Circular`

یکی از ویژگی‌های برجسته این ماژول، استفاده از یک تابع هدف تحلیلی مستقل برای اعتبارسنجی است. تابع `analytical_skr` در این ماژول، مستقل از حلال پیاده‌سازی شده و به‌عنوان یک `oracle` عمل می‌کند که صحت خروجی حلال را راستی‌آزمایی می‌نماید. این جداسازی، از خطای `circular validation` جلوگیری می‌کند.

`circular validation` زمانی رخ می‌دهد که اعتبارسنجی از همان تابعی استفاده کند که در پیاده‌سازی اصلی به‌کار رفته است. در این حالت، اگر تابع دارای باگ باشد، این باگ در اعتبارسنجی نیز منتقل می‌شود و شناسایی نمی‌گردد. به‌عنوان مثال، اگر حلال از یک تابع هدف باگ‌دار (مثلاً با فرمول اشتباه برای Y_1) استفاده کند و اعتبارسنجی نیز از همان تابع استفاده نماید، هر دو خروجی یکسانی تولید می‌کنند و آزمون موفق تلقی می‌شود، در حالی که خروجی هر دو نادرست است.

برای جلوگیری از این خطا، تابع `analytical_skr` در این ماژول، مستقل از حلال پیاده‌سازی شده است. این تابع، فرمول `GLLP` را به‌صورت تحلیلی و با اصلاحات فیزیکی (مانند $\max(Y1_L, Y0)$) پیاده می‌سازد و به‌عنوان یک مرجع مستقل عمل می‌نماید. در صورت بروز باگ در حلال، این تابع مستقل، خطا را آشکار می‌سازد.

۴.۵.۱۵ خروجی به stdout و گزارش متمرکز

خروجی این ماژول به صورت چاپ به stdout است. این خروجی شامل چندین بخش است: سرآیند تزئینی، نتیجه هر آزمون (شامل پارامترهای سخت‌کدشده و بهینه و SKR مربوطه)، و verdict نهایی. این قالب خروجی، برای مصرف توسط توسعه‌دهنده در حین debug طراحی شده و امکان تحلیل سریع نتایج را فراهم می‌سازد. در صورت نیاز، می‌توان خروجی را به فایل نیز نوشت تا در CI/CD pipeline قابل بازبینی باشد. در نسخه فعلی، این قابلیت پیاده‌سازی نشده است، اما با افزودن چند خط کد (مانند `with open(...) as f: f.write(...)`) قابل افزودن است.

۵.۵.۱۵ ارتباط با absolute_crypto_validation.py

این ماژول و `absolute_crypto_validation.py` دو لایه مکمل اعتبارسنجی را تشکیل می‌دهند. `absolute_crypto_validation.py` بر راستی‌آزمایی کمیت‌های پایه $(R, e_1, Y_1, E_\mu, Q_\mu)$ در برابر فرمول‌های تحلیلی متمرکز است، در حالی که این ماژول (`lp_validation.py`) بر راستی‌آزمایی حلال بهینه‌سازی پارامترها متمرکز است. این دو لایه، نقش‌های مکمل دارند: لایه اول تضمین می‌کند که موتور محاسباتی به‌درستی پیاده شده، و لایه دوم تضمین می‌نماید که حلال بهینه‌سازی به‌درستی عمل می‌کند. در صورت شکست هر یک از این لایه‌ها، مشکلات متفاوتی را آشکار می‌سازند: شکست لایه اول نشان‌دهنده باگ در محاسبات پایه است، در حالی که شکست لایه دوم نشان‌دهنده باگ در حلال بهینه‌سازی (مثلاً عدم همگرایی یا یافتن کمینه محلی) است.

۶.۵.۱۵ ارتباط با ماژول‌های اثبات Lim-۲۰۱۴ و Ma-۲۰۰۵

این ماژول به‌طور غیرمستقیم با ماژول‌های اثبات امنیتی `lim2014.py` و `ma2005.py` در ارتباط است. از طریق تابع `optimize_lim2014_finite_key`، این ماژول از پارامترهای پروتکل Lim-2014 استفاده می‌کند. با این حال، تابع هدف تحلیلی (`analytical_skr`) بر اساس حد مجانبی است و با فرمول‌های متناهی-key در Lim-2014 متفاوت است. این تفاوت، عمدی است: آزمون کران بالای نرخ کلید را راستی‌آزمایی می‌کند و در صورتی که حلال نتواند به کران بالا نزدیک شود، قطعاً نمی‌تواند نرخ کلید متناهی صحیح را تولید کند. این یک الگوی necessary condition testing است.

۷.۵.۱۵ خروجی نهایی و استفاده در CI/CD

خروجی نهایی این ماژول به دو شکل است: نخست، خروجی چاپی به کنسول که شامل نتایج هر آزمون و verdict نهایی است. دوم، کد خروج POSIX (به‌طور ضمنی از طریق `sys.exit`) که در صورت شکست، غیرصفر است. در پیاده‌سازی فعلی، کد خروج به صورت صریح تنظیم نشده است، اما می‌توان با افزودن `sys.exit(0 if all([res1, res2, res3]) else 1)` در پایان `__main__`، این قابلیت را اضافه کرد. در یک CI/CD pipeline نمونه، این ماژول به صورت زیر به کار می‌رود:

```

۱ - name: LP Solver Validation
۲ run: python lp_validation.py
۳
```

در صورت شکست، pipeline متوقف می‌شود و توسعه‌دهنده باید قبل از merge، خطا را اصلاح کند. این الگو، تضمین می‌کند که هرگز کدی با خطای بهینه‌سازی به شاخه اصلی merge نشود.

۸.۵.۱۵ جمع‌بندی ارتباطات

به‌طور خلاصه، این ماژول در نقش یک validator مستقل عمل می‌کند که از طریق سه کانال با سایر بخش‌های فریم‌ورک در ارتباط است: ورودی حلال بهینه‌سازی از `qkd.proofs.optimization`، ورودی پارامترهای پروتکل از `qkd.params`، و خروجی به stdout برای مصرف توسعه‌دهنده. این معماری، جداسازی واضحی بین لایه اعتبارسنجی و موتور بهینه‌سازی برقرار می‌سازد و امکان توسعه مستقل هر یک را فراهم می‌نماید. خروجی این ماژول، نه تنها برای توسعه‌دهندگان (در فرآیند debug)، بلکه برای بازبینان کریپتوگرافیک (در فرآیند security audit) نیز مفید است، زیرا یک audit trail قابل ممیزی از عملکرد حلال بهینه‌سازی در رژیم‌های مختلف فیزیکی فراهم می‌سازد.

فصل ۱۶

آزمون‌های هم‌روندی و حفظ هویت

۱.۱۶ نمای کلی و هدف ماژول

ماژول `pulse_tracking_trace.py` یک اسکریپت ممیزی و ردیابی سرتاسری (end-to-end audit) است که نقش یک forensic tracer را در فریم‌ورک شبیه‌سازی QKD ایفا می‌کند. این ماژول به‌طور خاص برای پاسخ به پرسش بنیادین زیر طراحی شده است: وقتی هزاران پالس نوری از طریق خط لوله چهارمرحله‌ای شبیه‌سازی (*row, receipt, worker, dispatch*) عبور می‌کنند، آیا هویت فیزیکی هر پالس – شامل بیت ارسالی *Alice*، پایه انتخابی، پایه گیرنده *Bob* و حالت کوانتومی – به‌طور سالم و بدون تغییر از مرحله‌ای به مرحله بعد منتقل می‌شود، یا این که در طول مسیر به‌دلیل *reordering* موازی، *race condition*، یا خطاهای پیاده‌سازی دچار فساد (*corruption*) می‌گردد؟ این پرسش در شبیه‌سازی‌های موازی که در آن `ProcessPoolExecutor` با چند کارگر روی هم‌زمان (*concurrency*) اجرا می‌شود، حیاتی است؛ زیرا ترتیب خروجی (*arrival order*) با ترتیب ورودی (*input order*) لزوماً یکسان نیست و این ناهماهنگی می‌تواند به‌طور پنهانی منجر به جابجایی هویت پالس‌ها شود.

هدف اصلی این ماژول، تولید یک رد `Tracks` قابل ممیزی (*auditable trace*) است که برای ۱۰۰ پالس تصادفی – از جمعیت ۱۰۰۰ پالس – نشان می‌دهد که دقیقاً چه بر سر هر پالس در هر یک از چهار مرحله می‌آید. چهار مرحله عبارت‌اند از: مرحله S1 (*Pre-Dispatch Snapshot*) که هویت اصلی پالس پیش از ارسال به کارگران را ثبت می‌کند؛ مرحله S2 (*Receipt Snapshot*) که هویت پالس پس از دریافت در کارگر را نشان می‌دهد؛ مرحله S3 (*Worker Output*) که نتیجه اندازه‌گیری کارگر را در بر دارد؛ و مرحله S4 (*Final Row*) که ردیف نهایی CSV را نمایش می‌دهد. اگر هر چهار مرحله بر سر تمام *identity fields* توافق کنند، یعنی هویت پالس به‌طور کامل حفظ شده است؛ در غیر این صورت، خطای فساد رخ داده است.

ساختار کلی این ماژول شامل چندین بخش است. بخش اول، توابع قالب‌بندی (*formatting helpers*) شامل `_measurement_match`، `_identity_match`، `_short_identity`، `_format_table` است که مسئول تولید خروجی متنی خوانا هستند. بخش دوم، تابع اصلی `run_trace` است که جریان کامل آزمون را هماهنگ می‌کند: ساخت جمعیت پالس، انتخاب پالس‌های ردیابی‌شده، اجرای `dispatch`، محاسبه اثبات *reordering*، تولید جدول ممیزی چهارمرحله‌ای، و محاسبه آمار سطح-مرحله‌ای. بخش سوم، تابع `main` و `CLI` است که از `argparse` برای پیکربندی پارامترها (تعداد کارگران، اندازه جمعیت، تعداد پالس‌های ردیابی‌شده، بذر تصادفی، تعداد ردیف‌های نمایش‌داده‌شده، فایل خروجی) استفاده می‌کند.

ویژگی برجسته این ماژول، الگوی `reuse-by-import` آن است؛ یعنی به‌جای بازنویسی ماشین‌آلات آزمون، مستقیماً از توابع کمکی در ماژول `tests/test_main_optimized_pulse_tracking.py` استفاده می‌کند. این توابع شامل `_make_pulse_population` برای ساخت جمعیت، `_select_tracked_pids` برای انتخاب تصادفی پالس‌های ردیابی‌شده با بذر قابل بازتولید، `_run_tracking_dispatch` برای اجرای `dispatch` با ردیابی، `_expected_measurement` برای پیش‌بینی نتیجه اندازه‌گیری بر اساس هویت پالس، و `_tracking_slow_worker` که یک کارگر با تأخیر دوجمله‌ای (*bimodal delay*) است. این الگو، اصل `DRY` (*Don't Repeat Yourself*) را رعایت می‌کند و تضمین می‌نماید که اسکریپت ممیزی و آزمون واحد، از همان ماشین‌آلات استفاده کنند و در نتیجه، نتایج آن‌ها همیشه هم‌خوان باشند.

در سطح عملیاتی، این ماژول به‌عنوان یک *diagnostic tool* عمل می‌کند؛ یعنی در صورت بروز مشکلات مربوط به حفظ هویت پالس (مثلاً پس از `refactor` موتور `dispatch` یا تغییر مدل کارگر)، توسعه‌دهنده می‌تواند این اسکریپت را اجرا کند و به‌سرعت علت فساد را شناسایی کند. خروجی آن به دو شکل

است: چاپ به stdout برای مشاهده زنده، و ذخیره به فایل pulse_tracking_trace.txt در مسیر /home/z/my-project/download/ برای بازبینی بعدی. این الگو در مهندسی نرم‌افزار علمی به‌عنوان shift-left forensics شناخته می‌شود و به توسعه‌دهنده اجازه می‌دهد خطاهای ظریف را پیش از انتشار نتایج شناسایی کند.

نکته مهم دیگر آن است که این ماژول اثبات *reordering* (reordering proof) را به‌صورت صریح و قابل مشاهده ارائه می‌دهد. در شبیه‌سازی موازی، خروجی کارگران به ترتیب ورودی نیست؛ بلکه ترتیب ورودی به ترتیب اتمام کارگران بستگی دارد. این ماژول با محاسبه تعداد pairwise inversions (یعنی زوج‌های پالسی که ترتیب ورودی‌شان با ترتیب خروجی متفاوت است)، نشان می‌دهد که reordering رخ داده اما هیچ هویتی فاسد نشده است. این تمایز بنیادینی است: reordering یک ویژگی ذاتی و قابل‌قبول شبیه‌سازی موازی است، در حالی که corruption یک باگ است که باید برطرف شود.

۲.۱۶ مبانی تئوری و فیزیکی

۱.۲.۱۶ پروتکل BB84 و مفهوم هویت پالس

پروتکل BB84 که توسط Bennett و Brassard در سال ۱۹۸۴ پیشنهاد شد، ساده‌ترین و پرکاربردترین پروتکل QKD است. در این پروتکل، فرستنده (Alice) یک بیت کلاسیک را با انتخاب تصادفی یکی از دو پایه (basis) — پایه مستطیلی (rectilinear) $\{|0\rangle, |1\rangle\}$ یا پایه قطری (diagonal) $\{|+\rangle, |-\rangle\}$ — به گیرنده (Bob) ارسال می‌کند. Bob نیز به‌طور تصادفی یکی از این دو پایه را برای اندازه‌گیری انتخاب می‌کند. در صورتی که پایه‌های Alice و Bob یکسان باشند، نتیجه اندازه‌گیری Bob با بیت ارسالی Alice هم‌خوان است (به جز خطاهای ناشی از نویز کانال و آشکارساز)؛ در غیر این صورت، نتیجه تصادفی است.

هویت هر پالس در شبیه‌سازی BB84 با چندین فیلد کلیدی مشخص می‌شود که در مجموع identity fields نامیده می‌شوند. این فیلدها در ثابت IDENTITY_FIELDS در ماژول آزمون تعریف شده‌اند و شامل موارد زیر هستند: alice_bit (بیت ارسالی Alice، صفر یا یک)، alice_basis (پایه انتخابی Alice، مستطیلی یا قطری)، bob_basis (پایه انتخابی Bob، مستطیلی یا قطری)، state_label (برچسب حالت کوانتومی مانند $|0\rangle, |1\rangle, |+\rangle, |-\rangle$ ، شدت میانگین پالس) و intensity_label (برچسب شدت مانند signal، decoy یا vacuum). حفظ یکپارچگی این فیلدها در طول خط لوله شبیه‌سازی، شرط لازم برای صحت نتایج است.

۲.۲.۱۶ پیش‌بینی نتیجه اندازه‌گیری و قانون Born

نتیجه اندازه‌گیری یک پالس BB84 توسط Bob را می‌توان بر اساس هویت پالس پیش‌بینی کرد. این پیش‌بینی بر اساس قانون Born و قواعد پروتکل BB84 انجام می‌شود. در صورتی که پایه‌های Alice و Bob یکسان باشند (ba_sis_match = True) و پالس به‌طور موفقیت‌آمیزی شناسایی شود (detected = True)، بیت اندازه‌گیری‌شده با بیت ارسالی هم‌خوان است (measured_bit = alice_bit). در صورتی که پایه‌ها متفاوت باشند، نتیجه اندازه‌گیری کاملاً تصادفی است و احتمال آن 0.5 برای هر بیت است. در شبیه‌سازی قطعی با بذر مشخص، این تصادفی بودن از یک PRNG قابل بازتولید به دست می‌آید.

پیش‌بینی نتیجه اندازه‌گیری در تابع expected_measurement پیاده‌سازی شده است. این تابع یک dict شامل سه کلید basis_match، measured_bit و detected برمی‌گرداند که مقادیر پیش‌بینی‌شده را در بر دارد. مقایسه این مقادیر با مقادیر گزارش‌شده توسط کارگر (S3)، امکان تشخیص خطاهای پیاده‌سازی را فراهم می‌سازد. به‌عنوان مثال، اگر کارگر basis_match = True و detected = True را گزارش دهد اما measured_bit با alice_bit متفاوت باشد، این یک ناسازگاری است که نشان‌دهنده باگ در منطق اندازه‌گیری کارگر است.

۳.۲.۱۶ تأخیر دوجمله‌ای و شبیه‌سازی کارگر واقعی

یکی از چالش‌های کلیدی در آزمون شبیه‌سازی موازی، ایجاد شرایط واقعی برای آشکار کردن race condition و reordering است. اگر همه کارگران با تأخیر یکسان اجرا شوند، خروجی به‌طور قطعی به ترتیب ورودی است و reordering رخ نمی‌دهد. برای غلبه بر این مشکل، تابع tracking_slow_worker با تأخیر دوجمله‌ای (bimodal delay) پیاده‌سازی شده است:

$$T_{\text{worker}} = \begin{cases} 100\text{ms} & \text{با 0.10 احتمال} \\ U(0, 5)\text{ms} & \text{با 0.90 احتمال} \end{cases} \quad (1.16)$$

در این رابطه، T_{worker} زمان اجرای کارگر است و $U(0, 5)$ توزیع یکنواخت در بازه $[0, 5]$ میلی‌ثانیه است. این مدل، دو رژیم را شبیه‌سازی می‌کند: 10% کارگران کند هستند (تأخیر ۱۰۰ میلی‌ثانیه که شبیه‌سازی بار CPU سنگین یا I/O است) و 90% کارگران سریع هستند (تأخیر تصادفی ۰ تا ۵ میلی‌ثانیه). این توزیع غیریکنواخت، احتمال reordering را به‌طور قابل‌توجهی افزایش می‌دهد و امکان آشکارسازی باگ‌های مربوط به حفظ ترتیب را فراهم می‌سازد.

۴.۲.۱۶ Kendall-Tau و معیار ناهماهنگی

برای اندازه‌گیری کمی میزان reordering، از معیار Kendall-Tau inversion count استفاده می‌شود. این معیار، تعداد زوج‌های (i, j) با $i < j$ را می‌شمارد که در آن ترتیب خروجی برعکس ترتیب ورودی است:

$$\text{inv}(\pi) = |\{(i, j) : i < j \wedge \pi(i) > \pi(j)\}| \quad (۲.۱۶)$$

در این رابطه، π جایگشت خروجی نسبت به ورودی است. حداکثر تعداد وارونگی‌ها برای n پالس برابر با $\binom{n}{2} = \frac{n(n-1)}{2}$ است که در صورت جایگشت معکوس کامل رخ می‌دهد. درصد وارونگی به‌صورت زیر تعریف می‌شود:

$$\text{reorder\%} = \frac{\text{inv}(\pi)}{\binom{n}{2}} \times 100 \quad (۳.۱۶)$$

این معیار، یک اندازه‌گیری از مقیاس (scale-free) برای سنجش شدت reordering فراهم می‌سازد. در شبیه‌سازی با ۱۰۰ پالس ردیابی‌شده و چند کارگر با تأخیر دوجمله‌ای، مقدار معمول این معیار در محدوده 30% تا 50% است که نشان‌دهنده reordering قابل‌توجه اما نه کامل است.

۵.۲.۱۶ مفهوم Identity-Bound-to-ID و Integrity Position-Independent

مفهوم کلیدی که این ماژول آن را به‌اثبات می‌رساند، یکپارچگی وابسته به شناسه، نه به موقعیت (identity bound to pulse_id, not to row position) است. در یک طراحی ساده اما خطاکار، ممکن است هویت پالس بر اساس موقعیت ردیف در خروجی استنتاج شود؛ یعنی فرض شود که ردیف i در خروجی، مربوط به پالس i در ورودی است. این فرض در حضور reordering نادرست است و منجر به تخصیص اشتباه هویت به ردیف‌ها می‌شود.

طراحی صحیح، هویت را به pulse_id (یک شناسه یکتا که در زمان ساخت پالس تخصیص می‌یابد) می‌بندد. این شناسه به‌صورت یکتا در کل جمعیت است و در طول خط لوله بدون تغییر منتقل می‌شود. هر مرحله از خط لوله، این شناسه را در خروجی خود شامل می‌شود و امکان بازسازی هویت پالس را فراهم می‌سازد. این الگو در ادبیات مهندسی نرم‌افزار به‌عنوان correlation ID یا trace ID شناخته می‌شود و در سیستم‌های توزیع‌شده (distributed systems) برای ردیابی درخواست‌ها به‌کررات به‌کار می‌رود.

۳.۱۶ تحلیل معماری و ساختار داده‌ها

۱.۳.۱۶ ساختار کلی ماژول و الگوی Script-with-Helpers

این ماژول از الگوی script-with-helpers پیروی می‌کند؛ یعنی یک تابع اصلی run_trace که جریان اجرا را هماهنگ می‌کند، و چندین تابع کمکی برای قالب‌بندی و مقایسه. این الگو برای اسکریپت‌های CLI مناسب است که نیاز به خوانایی بالا و قابلیت آزمون دارند. تابع run_trace خروجی را به‌صورت یک رشته متنی بزرگ برمی‌گرداند (نه چاپ مستقیم)، که امکان نوشتن آن در stdout یا فایل را فراهم می‌سازد. این جداسازی بین تولید خروجی و چاپ، الگوی separation of concerns را رعایت می‌کند.

۲.۳.۱۶ ثابت IDENTITY_FIELDS و فیلدهای هویتی

ثابت IDENTITY_FIELDS که از ماژول آزمون وارد می‌شود، فهرست فیلدهایی است که برای تعیین هم‌خوانی هویت در چهار مرحله بررسی می‌شوند. این فیلدها در ساختار dict هر پالس وجود دارند و شامل alice_bit

برای این فیلدها، تضمین می‌کند که تمام بخش‌های ماژول از همان فهرست استفاده کنند و در صورت افزودن فیلد جدید، فقط یک نقطه نیاز به به‌روزرسانی دارد.

```

1 from tests.test_main_optimized_pulse_tracking import (
2     IDENTITY_FIELDS,
3     _make_pulse_population,
4     _select_tracked_pids,
5     _run_tracking_dispatch,
6     _expected_measurement,
7     _tracking_slow_worker,
8 )
9 from main_optimized_dispatch import dispatch_work_items
10

```

این الگوی import چندین نکته دارد. نخست، IDENTITY_FIELDS به‌عنوان یک ثابت وارد می‌شود تا در تمام ماژول قابل استفاده باشد. دوم، توابع کمکی با پیشوند _ (مانند _make_pulse_population) وارد می‌شوند که در پایتون نشان‌دهنده private API است؛ اما چون در همان پروژه هستیم، این واردسازی مجاز است. سوم، dispatch_work_items از ماژول main_optimized_dispatch وارد می‌شود که این نشان‌دهنده وابستگی مستقیم به موتور dispatch اصلی فریم‌ورک است.

۳.۳.۱۶ تابع _format_table و قالب‌بندی متنی

تابع _format_table یک ابزار قالب‌بندی متنی است که لیستی از سطرها را به جدولی با ستون‌های هم‌تراز تبدیل می‌کند. این تابع دو پارامتر اصلی دارد: headers (فهرست عناوین ستون‌ها) و rows (لیست سطرها، که هر سطر یک لیست از مقادیر است). پارامتر سوم col_widths اختیاری است و در صورت عدم ارائه، به‌صورت خودکار بر اساس حداکثر طول مقدار در هر ستون محاسبه می‌شود.

```

1 def _format_table(headers: List[str], rows: List[List[Any]],
2     col_widths: List[int] = None) -> str:
3     """Format a list of rows as an aligned text table."""
4     if not rows:
5         return "(no rows)"
6     if col_widths is None:
7         col_widths = []
8         for i, h in enumerate(headers):
9             w = max(len(str(h)), *(len(str(r[i])) for r in rows))
10            col_widths.append(w)
11        lines = []
12        lines.append(" ".join(str(h).ljust(w) for h, w in zip(headers, col_widths)))
13        lines.append(" ".join("-" * w for w in col_widths))
14        for r in rows:
15            lines.append(" ".join(str(c).ljust(w) for c, w in zip(r, col_widths)))
16        return "\n".join(lines)
17

```

در خط اول، شرط rows not بررسی می‌شود و در صورت خالی بودن، رشته "(no rows)" بازگردانده می‌شود. این الگو، الگوی graceful handling of empty input است. در حلقه بعدی، عرض هر ستون بر اساس حداکثر طول مقدار در آن ستون محاسبه می‌شود؛ این کار با ترکیب max و generator expression انجام می‌شود که یک الگوی پایتونیک برای محاسبه حداکثر در یک مجموعه است. سپس خط عنوان، خط جداکننده (با -ها) و سطرهای داده به‌ترتیب به خطوط اضافه می‌شوند. استفاده از ljust(w) تضمین می‌کند که تمام مقادیر در یک ستون، عرض یکسان داشته باشند و جدول هم‌تراز باشد.

۴.۳.۱۶ تابع _short_identity و نمایش فشرده هویت

تابع _short_identity یک dict هویت پالس را به یک رشته فشرده به فرمت bit/basis/bob/state تبدیل می‌کند. این نمایش، خوانایی جدول ممیزی را به‌طور قابل‌توجهی افزایش می‌دهد؛ زیرا یک رشته کوتاه به‌جای چهار فیلد جداگانه نمایش داده می‌شود.

```

1 def _short_identity(d: Dict[str, Any]) -> str:
2     """Compact representation of an identity dict: 'bit/basis/bob/state'."""
3     return (f"{d['alice_bit']}/{d['alice_basis']}/"

```

```
۴ f"{d['bob_basis']}/{d['state_label']}")
۵
```

این تابع با استفاده از f-string و / به‌عنوان جداکننده، یک رشته به فرمت +/0/1/0 تولید می‌کند که در آن اولین مقدار alice_bit، دومین alice_basis، سومین bob_basis و چهارمین state_label است. این فرمت، مشابه نمایش CSV است اما با جداکننده / که در خروجی متنی خوانا تر است.

۵.۳.۱۶ تابع identity_match و منطق مقایسه چهار مرحله‌ای

تابع identity_match چهار dict هویتی (s4, s3, s2, s1) را می‌گیرد و در صورتی که همه فیلدهای IDENTITY_FIELDS در هر چهار مرحله برابر باشند، مقدار "OK" و در غیر این صورت، "FAIL" برمی‌گرداند.

```
۱ def identity_match(s1, s2, s3, s4) -> str:
۲     """Return 'OK' if all four stage views agree on every identity field."""
۳     for f in IDENTITY_FIELDS:
۴         if not (s1[f] == s2[f] == s3[f] == s4[f]):
۵             return "FAIL"
۶     return "OK"
۷
```

در این پیاده‌سازی، عبارت s1[f] == s2[f] == s3[f] == s4[f] یک الگوی پایتونیک برای مقایسه زنجیره‌ای است که در آن، همه چهار مقدار باید برابر باشند. این عبارت معادل s1[f] == s2[f] and s2[f] == s3[f] and s3[f] == s4[f] است اما خوانا تر و کوتاه‌تر است. در صورت عدم تطابق در هر فیلد، تابع بلافاصله "FAIL" برمی‌گرداند که الگوی early exit است و از پردازش غیرضروری جلوگیری می‌کند.

۶.۳.۱۶ تابع measurement_match و اعتبارسنجی پیش‌بینی

تابع measurement_match دو پارامتر s1 (هویت پیش از dispatch) و s3 (خروجی کارگر) را می‌گیرد و در صورتی که نتیجه اندازه‌گیری گزارش شده توسط کارگر با پیش‌بینی بر اساس s1 هم‌خوان باشد، "OK" برمی‌گرداند.

```
۱ def measurement_match(s1, s3) -> str:
۲     """Return 'OK' if the worker's measurement matches the S1 prediction."""
۳     exp = _expected_measurement(s1)
۴     if (s3["basis_match"] == exp["basis_match"] and
۵         s3["measured_bit"] == exp["measured_bit"] and
۶         s3["detected"] == exp["detected"]):
۷         return "OK"
۸     return "FAIL"
۹
```

در این تابع، ابتدا _expected_measurement(s1) فراخوانی می‌شود تا پیش‌بینی نتیجه اندازه‌گیری بر اساس هویت پالس به دست آید. سپس سه فیلد کلیدی — basis_match، measured_bit و detected — بین گزارش کارگر و پیش‌بینی مقایسه می‌شوند. این اعتبارسنجی، خطاهای احتمالی در منطق اندازه‌گیری کارگر را آشکار می‌سازد. به‌عنوان مثال، اگر کارگر به اشتباه basis_match = True را در حالی که پایه‌ها متفاوت هستند گزارش دهد، این تابع خطا را شناسایی می‌کند.

۷.۳.۱۶ تابع run_trace و الگوی Closure برای خروجی

تابع run_trace اصلی‌ترین تابع ماژول است. این تابع یک closure به نام out تعریف می‌کند که رشته‌ها را به لیست lines اضافه می‌کند:

```
۱ def run_trace(args) -> str:
۲     """Run the 100-pulse trace and return the full text output."""
۳     lines: List[str] = []
۴     def out(s: str = ""):
۵         lines.append(s)
۶
```

این الگو چندین مزیت دارد: نخست، تمام خروجی در یک لیست جمع‌آوری می‌شود و در پایان به یک رشته با join(lines). "\n" تبدیل می‌گردد. این کار، از هزینه string concatenation مکرر (که در پایتون از مرتبه $O(n^2)$ است) جلوگیری می‌کند. دوم، تابع out به‌عنوان یک closure به lines دسترسی دارد و نیازی به ارسال

lines به عنوان پارامتر نیست. سوم، این الگو امکان تغییر آینده به file-like object یا stream را فراهم می‌سازد.

۸.۳.۱۶ تنظیم مسیر و الگوی Bootstrap Path

در ابتدای ماژول، الگوی path bootstrap پیاده‌سازی شده است:

```

۱ SCRIPT_DIR = os.path.dirname(os.path.abspath(__file__))
۲ PROJECT_ROOT = os.path.dirname(SCRIPT_DIR)
۳ for candidate in (
۴     PROJECT_ROOT,
۵     os.path.join(PROJECT_ROOT, "src"),
۶     os.path.join(PROJECT_ROOT, "tests"),
۷ ):
۸     if candidate not in sys.path:
۹         sys.path.insert(0, candidate)
۱۰

```

این الگو، تضمین می‌کند که ماژول بدون توجه به محل اجرای آن (از project root یا از مسیر دیگر)، بتواند ماژول‌های tests و src را وارد کند. SCRIPT_DIR مسیر دایرکتوری حاوی اسکریپت است و PROJECT_ROOT یک سطح بالاتر است. سپس سه مسیر (ریشه پروژه، tests، src) به sys.path اضافه می‌شوند. این الگو در اسکریپت‌هایی که به صورت مستقل اجرا می‌شوند و نیاز به واردسازی ماژول‌های پروژه دارند، استاندارد است.

۴.۱۶ پیاده‌سازی ریاضی و الگوریتمی

۱.۴.۱۶ الگوریتم انتخاب تصادفی پالس‌های ردیابی‌شده

انتخاب ۱۰۰ پالس از جمعیت ۱۰۰۰ پالس، با بذر تصادفی مشخص، توسط تابع select_tracked_pids انجام می‌شود. این تابع از random.Random(seed) برای تولید یک PRNG قابل بازتولید استفاده می‌کند و سپس با random.sample، k شناسه از جمعیت N انتخاب می‌نماید. این الگوریتم، الگوریتم Fisher-Yates partial shuffle است که در آن، فقط k جایگشت اول محاسبه می‌شود:

$$P(\text{pid}_i \text{ selected}) = \frac{k}{N}, \quad \forall i \in \{0, 1, \dots, N-1\} \quad (۴.۱۶)$$

این الگوریتم تضمین می‌نماید که هر پالس با احتمال یکسان k/N انتخاب شود و انتخاب‌ها مستقل باشند. با بذر مشخص، انتخاب قابل بازتولید است که برای آزمون‌های regression حیاتی است.

۲.۴.۱۶ Inversions Pairwise محاسبه الگوریتم

محاسبه تعداد pairwise inversions در لیست ورودی، با الگوریتم ساده $O(n^2)$ انجام می‌شود:

```

۱ n = len(tracked_arrival_order)
۲ inversions = 0
۳ for i in range(n):
۴     ai = tracked_arrival_order[i]
۵     for j in range(i + 1, n):
۶         if ai > tracked_arrival_order[j]:
۷             inversions += 1
۸ max_inv = n * (n - 1) // 2
۹

```

در این الگوریتم، برای هر زوج (i, j) با $i < j$ ، بررسی می‌شود که آیا $ai > arrival[j]$ است. در صورت مثبت بودن، یک وارونگی شمرده می‌شود. حداکثر تعداد وارونگی‌ها برابر با $\binom{n}{2} = \frac{n(n-1)}{2}$ است که با عملگر $//$ (تقسیم صحیح) محاسبه می‌شود. این الگوریتم برای $n = 100$ کافی است (10^4 عملیات)، اما برای n بزرگ‌تر، الگوریتم merge-sort-based با پیچیدگی $O(n \log n)$ ترجیح داده می‌شود.

۳.۴.۱۶ الگوریتم یافتن Examples Swap Concrete

برای نمایش شهودی reordering، الگوریتمی برای یافتن نمونه‌های ملموس جابجایی پیاده‌سازی شده است:

```

۱ swap_examples = []
۲ for i in range(len(tracked_arrival_order) - 1):
۳     a = tracked_arrival_order[i]
۴     b = tracked_arrival_order[i + 1]
۵     if a > b: # inversion: larger pid arrived before smaller
۶         swap_examples.append((i, i + 1, a, b))
۷     if len(swap_examples) >= 5:
۸         break
۹

```

این الگوریتم، زوج‌های مجاور (adjacent pairs) را بررسی می‌کند و در صورتی که pid بزرگ‌تر قبل از pid کوچک‌تر آمده باشد، آن را به‌عنوان یک جابجایی ثبت می‌نماید. این الگوریتم، فقط adjacent inversions را شناسایی می‌کند که زیرمجموعه‌ای از pairwise inversions هستند. هدف از این الگوریتم، ارائه نمونه‌های ملموس و قابل درک برای کاربر است؛ زیرا مشاهده دو pid که جابجا شده‌اند، شهودی‌تر از عدد انتزاعی inversion count است. حداکثر ۵ نمونه نمایش داده می‌شود تا خروجی بیش از حد طولانی نشود.

۴.۴.۱۶ الگوریتم محاسبه Shifts Position

برای تحلیل کمی جابجایی هر پالس، الگوریتم محاسبه position shifts پیاده‌سازی شده است:

```

۱ sorted_idx = {pid: i for i, pid in enumerate(tracked_pids)}
۲ arrived_idx = {pid: i for i, pid in enumerate(tracked_arrival_order)}
۳ shifts = [(pid, sorted_idx[pid], arrived_idx[pid], arrived_idx[pid] - sorted_idx[pid])
۴             for pid in tracked_pids]
۵ max_up = max(shifts, key=lambda x: x[3])
۶ max_down = min(shifts, key=lambda x: x[3])
۷

```

در این الگوریتم، ابتدا دو دیکشنری sorted_idx و arrived_idx ساخته می‌شوند که برای هر pid، موقعیت آن در لیست مرتب (ورودی) و در لیست ورودی (خروجی) را نگه می‌دارند. سپس برای هر پالس، اختلاف این دو موقعیت (shift) محاسبه می‌شود. shift مثبت به این معناست که پالس در خروجی زودتر از ورودی آمده (حرکت به بالا) و shift منفی به این معناست که پالس دیرتر آمده (حرکت به پایین). max_up و max_down به ترتیب بیشترین حرکت به بالا و بیشترین حرکت به پایین را نشان می‌دهند. این الگوریتم، الگوی list comprehension و dictionary comprehension را به خوبی به نمایش می‌گذارد.

۵.۴.۱۶ الگوریتم Counting Mismatch Stage-Level

برای محاسبه آمار ناهماهنگی در هر مرحله، الگوریتم زیر پیاده‌سازی شده است:

```

۱ for pid in tracked_pids:
۲     if pid not in receipts_by_pid:
۳         missing_receipts += 1
۴         continue
۵     if pid not in rows_by_pid:
۶         missing_rows += 1
۷         continue
۸     s1 = snapshot[pid]
۹     s2 = receipts_by_pid[pid]["received_item"]
۱۰    s3 = rows_by_pid[pid]
۱۱    s4 = rows_by_pid[pid]
۱۲    if any(s1[f] != s2[f] for f in IDENTITY_FIELDS):
۱۳        s1_s2_mismatch += 1
۱۴    if any(s2[f] != s3[f] for f in IDENTITY_FIELDS):
۱۵        s2_s3_mismatch += 1
۱۶    if any(s3[f] != s4[f] for f in IDENTITY_FIELDS):
۱۷        s3_s4_mismatch += 1
۱۸    exp = _expected_measurement(s1)
۱۹    if (s3["basis_match"] != exp["basis_match"] or
۲۰        s3["measured_bit"] != exp["measured_bit"] or
۲۱        s3["detected"] != exp["detected"]):
۲۲        measurement_mismatch += 1

```

۲۳

در این الگوریتم، برای هر pid در tracked_pids، ابتدا وجود receipt و row بررسی می‌شود. در صورت عدم وجود، شمارنده missing افزایش می‌یابد و با continue به پالس بعدی می‌رود. در غیر این صورت، چهار dict هویتی s1، s2، s3، s4 استخراج می‌شوند و برای هر جفت متوالی (S1-S2، S2-S3، S3-S4)، تطابق فیلدهای هویتی بررسی می‌گردد. تابع any با generator expression، الگوی پایتونیک برای بررسی وجود عدم تطابق است. در پایان، تطابق نتیجه اندازه‌گیری نیز بررسی می‌شود. این الگوریتم، الگوی fail-fast را رعایت می‌کند: در صورت فقدان داده‌های پایه‌ای (receipt یا row)، بقیه بررسی‌ها برای آن پالس انجام نمی‌شود.

۶.۴.۱۶ الگوریتم Verdict و تفسیر نهایی

در پایان، الگوریتم verdict نتیجه نهایی را اعلام می‌کند:

```

1 total_mismatch = (missing_receipts + missing_rows + s1_s2_mismatch +
2 s2_s3_mismatch + s3_s4_mismatch + measurement_mismatch)
3 if total_mismatch == 0:
4     out(" VERDICT: All 100 tracked pulses preserved identity through all 4 stages.")
5     out(f" {inversions} pairwise inversions occurred (reordering), yet NOT A")
6     out(" SINGLE pulse's identity was corrupted. Identity is bound to")
7     out(" pulse_id, NOT to row position -- exactly as the contract requires.")
8 else:
9     out(f" VERDICT: {total_mismatch} corruption(s) detected across the 4 stages.")
10    out(" See the per-pulse audit table above for details.")
11

```

این الگوریتم، مجموع تمام ناهماهنگی‌ها را محاسبه می‌کند و در صورت صفر بودن، پیام موفقیت با تأکید بر تمایز بین reordering (که قابل قبول است) و corruption (که باگ است) چاپ می‌نماید. این تمایز، پیام کلیدی ماژول است: reordering یک ویژگی ذاتی شبیه‌سازی موازی است و در صورتی که هویت به pulse_id بسته شده باشد، هیچ مشکلی ایجاد نمی‌کند.

۷.۴.۱۶ الگوریتم CLI با argparse

تابع main از argparse برای پیکربندی پارامترها استفاده می‌کند:

```

1 def main():
2     parser = argparse.ArgumentParser(
3         description="100-random-pulse end-to-end state trace",
4         formatter_class=argparse.RawDescriptionHelpFormatter,
5     )
6     parser.add_argument("--workers", type=int, default=4)
7     parser.add_argument("--population", type=int, default=1000)
8     parser.add_argument("--tracked", type=int, default=100)
9     parser.add_argument("--seed", type=int, default=42)
10    parser.add_argument("--first", type=int, default=100)
11    parser.add_argument("--output", type=str, default=None)
12    args = parser.parse_args()
13
14    text = run_trace(args)
15    print(text)
16
17    default_path = os.path.join(PROJECT_ROOT, "download",
18                                "pulse_tracking_trace.txt")
19    os.makedirs(os.path.dirname(default_path), exist_ok=True)
20    with open(default_path, "w") as f:
21        f.write(text)
22    print(f"\n[trace also written to {default_path}]", file=sys.stderr)
23

```

این الگوریتم، الگوی استاندارد CLI در پایتون است. هر پارامتر با add_argument تعریف می‌شود و دارای مقدار پیش‌فرض است. RawDescriptionHelpFormatter امکان قالب‌بندی متن راهنما را فراهم می‌سازد. پس از اجرای run_trace، خروجی به stdout چاپ می‌شود و همچنین به فایل پیش‌فرض در download/ نوشته می‌گردد. در صورت ارائه --output، خروجی به فایل اضافی نیز نوشته می‌شود. این الگو، تضمین می‌کند که خروجی همیشه برای بازبینی بعدی ذخیره می‌شود، حتی اگر کاربر فایل خاصی مشخص نکند.

۵.۱۶ ارسال و دریافت با سایر ماژول‌ها

۱.۵.۱۶ وابستگی به tests/test_main_optimized_pulse_tracking

اصلی‌ترین وابستگی این ماژول، به tests/test_main_optimized_pulse_tracking.py است. این وابستگی شامل پنج مورد است: ثابت IDENTITY_FIELDS و چهار تابع _make_pulse_population، _select_tracked_pids، _run_tracking_dispatch و _expected_measurement و _tracking_slow_worker. این الگوی reuse-by-import، تضمین می‌کند که اسکریپت ممیزی و آزمون واحد، از همان ماشین‌آلات استفاده کنند و نتایج آن‌ها همیشه هم‌خوان باشند. در صورت تغییر در یکی از این توابع (مثلاً افزودن فیلد جدید به IDENTITY_FIELDS)، هر دو به‌طور خودکار به‌روز می‌شوند. این وابستگی، یک وابستگی tight coupling است: تغییر در ساختار dict پالس یا در امضای توابع، می‌تواند منجر به شکست این ماژول شود. برای کاهش این coupling، می‌توان توابع کمکی را به یک ماژول جداگانه (مانند pulse_tracking_utils.py) منتقل کرد که هم اسکریپت ممیزی و هم آزمون واحد از آن استفاده کنند. این الگو، الگوی shared utility module است که در پروژه‌های بالغ به‌کررات به‌کار می‌رود.

۲.۵.۱۶ ورودی از main_optimized_dispatch

این ماژول تابع dispatch_work_items را از main_optimized_dispatch وارد می‌کند. این تابع، موتور اصلی dispatch فریم‌ورک است که کارهای شبیه‌سازی را بین چند کارگر توزیع می‌کند. وابستگی به این تابع، تضمین می‌کند که اسکریپت ممیزی، همان مسیر اجرای واقعی فریم‌ورک را آزمون می‌کند و نه یک مسیر آزمایشگاهی مصنوعی.

۳.۵.۱۶ خروجی به stdout و فایل

خروجی این ماژول به دو شکل است: نخست، چاپ به stdout با print(text) که امکان مشاهده زنده را فراهم می‌سازد. دوم، نوشتن به فایل پیش‌فرض در مسیر open(default_path, "/home/z/my-project/download/pulse_tracking_trace.txt", "w"). در صورت ارائه --output، خروجی به فایل اضافی نیز نوشته می‌شود. این الگو، تضمین می‌کند که خروجی همیشه برای بازبینی بعدی ذخیره می‌شود، حتی اگر کاربر آن را به فایل خاصی هدایت نکرده باشد.

۴.۵.۱۶ ارتباط با فریم‌ورک آزمون

این ماژول و ماژول آزمون tests/test_main_optimized_pulse_tracking.py دو لایه مکمل اعتبارسنجی را تشکیل می‌دهند. لایه آزمون، اعتبارسنجی خودکار را با pytest فراهم می‌سازد و در CI/CD pipeline اجرا می‌شود. لایه اسکریپت ممیزی (این ماژول)، خروجی قابل مشاهده و قابل تحلیل توسط انسان را فراهم می‌کند و برای debug و تحلیل دستی استفاده می‌شود. این دو لایه، نقش‌های مکمل دارند: لایه آزمون، gate خودکار را برای merge فراهم می‌سازد و لایه اسکریپت، امکان تحلیل عمیق در صورت بروز مشکل را فراهم می‌نماید.

۵.۵.۱۶ خروجی قابل تحلیل توسط ابزارهای خارجی

خروجی متنی این ماژول، به‌گونه‌ای طراحی شده که توسط ابزارهای تحلیل متنی (مانند grep، awk یا اسکریپت‌های Python) قابل پردازش باشد. جدول‌ها با spaces هم‌تراز شده‌اند و جداکننده‌ها واضح هستند. در صورت نیاز، می‌توان خروجی را به فرمت JSON یا CSV نیز تبدیل کرد تا توسط ابزارهای تحلیل داده (مانند pandas) پردازش شود. این انعطاف‌پذیری، در پروژه‌های تحقیقاتی که نیاز به تحلیل آماری نتایج شبیه‌سازی دارند، ارزشمند است.

۶.۵.۱۶ جمع‌بندی ارتباطات

به‌طور خلاصه، این ماژول در نقش یک forensic tracer مستقل عمل می‌کند که از طریق سه کانال با سایر بخش‌های فریم‌ورک در ارتباط است: ورودی توابع کمکی از tests/test_main_optimized_pulse_tracking.py، موتور dispatch از main_optimized_dispatch، و خروجی به stdout و فایل pulse_tracking_trace.txt. این معماری، جداسازی واضحی بین لایه ممیزی و موتور شبیه‌سازی برقرار می‌سازد و امکان توسعه مستقل هر

یک را فراهم می‌نماید. خروجی این ماژول، نه تنها برای توسعه‌دهندگان (در فرآیند debug)، بلکه برای بازبینان کریپتوگرافیک (در فرآیند security audit) نیز مفید است، زیرا یک audit trail قابل ممیزی از حفظ هویت پالس‌ها در طول خط لوله شبیه‌سازی فراهم می‌سازد.

بخش پنجم
نتایج شبیه‌سازی

بخش ششم
نتایج شبیه‌سازی

فصل ۱۷

تنظیمات آزمایش و کمپین‌های شبیه‌سازی

۱.۱۷ نمای کلی کمپین‌ها

این بخش نتایج حاصل از اجرای ده کمپین شبیه‌سازی مستقل با اسکریپت `main_optimized.py` را گزارش می‌کند. هر کمپین با ترکیب متفاوتی از پروتکل، اثبات امنیت، نوع آشکارساز و پارامترهای فیزیکی اجرا شده تا رفتار چارچوب در شرایط عملیاتی گوناگون ارزیابی شود. خروجی هر اجرا در یک فایل CSV به نام `qkd_results_optimized_sweep.csv` نوشته می‌شود و سپس برای تحلیل به نمودار PNG تبدیل شده است. این نمودارها در ادامه فصل به ترتیب موضوعی ارائه می‌شوند.

تنها خروجی ملموس هر شبیه‌سازی همین فایل CSV است؛ اسکریپت هیچ نمودار، فایل JSON خلاصه یا گزارش تجمیعی تولید نمی‌کند. بنابراین تمام تحلیل‌های آماری بعدی بر مبنای پردازش همین فایل با کتابخانه `pandas` انجام شده است. تعداد کل شبیه‌سازی‌های اجرا شده در این ده کمپین تقریباً برابر ۱۸۰۰ نقطه است که هر کدام شامل تولید پالس، انتقال در کانال فیبر، آشکارسازی، غربال‌سازی پایه‌ها (sifting)، تخمین پارامتر و محاسبه کلید امن هستند.

۲.۱۷ خلاصه کمپین‌ها

جدول ۱.۱۷ ده کمپین اجرا شده را به همراه پارامترهای متمایزکننده و تعداد نقاط شبیه‌سازی خلاصه می‌کند. این جدول به عنوان مرجع سریع برای تفسیر نمودارهای فصل‌های بعدی استفاده می‌شود.

جدول ۱.۱۷: خلاصه کمپین‌های شبیه‌سازی اجرا شده با `main_optimized.py`

شماره	نام فایل	محور تغییر اصلی	تعداد نقاط
۱	01_baseline_BB84_Lim2014	خط پایه BB84 با Lim2014	۹۶
۲	02_finite_key_scaling	مقیاس پالس 10^4 تا 10^8	۲۸۸
۳	03_dwdm_*	DWDM با توان‌های -20، -34 و -40 dBm	۲۸۸
۴	04_auto_optimized_pulses	بهینه‌سازی SLSQP پارامتر پالس	۹۶
۵	05_protocol_*	چهار پروتکل مختلف	۳۸۴
۶	06_proof_*	چهار اثبات امنیت متفاوت	۳۸۴
۷	07_detector_*	سه نوع آشکارساز PNRD/SNSPD/SPD	۲۸۸
۸	08_confidence_*	سه روش بازه اطمینان	۲۸۸
۹	09_rogers_validation_dense	اعتبارسنجی Rogers با ۵۱ نقطه	۱۰۲
۱۰	10_qber_misalignment_sweep	حساسیت به QBER و misalignment	۱۹۸

۳.۱۷ پیکربندی مشترک

تمام کمپین‌ها از مقادیر پیش فرض زیر برای اطمینان از قابلیت مقایسه استفاده می‌کنند: افت فیبر 0.2 dB/km، بازه فاصله 0 تا 150 km با ۱۶ نقطه، دانه تصادفی (seed) برابر 12345، و هشت پردازشگر موازی. تعداد پالس‌ها مگر در کمپین ۲، در بازه 10^4 تا 10^6 نگه داشته شده است. هر نقطه در دو حالت `noise_applied=True` و

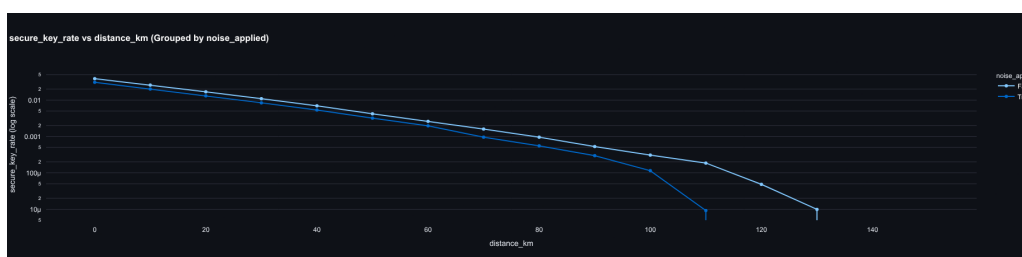
noise_applied=False اجرا می‌شود تا تأثیر نویز واقعی آشکارساز (شامل شمارش تاریک، پس‌پالس، جیتر، زمان مرده و عدم انطباق پایه‌ها) به‌صراحت قابل ارزیابی باشد.

فصل ۱۸

نتایج خط پایه و مقیاس کلید محدود

۱.۱۸ خط پایه BB۸۴ با اثبات Lim۲۰۱۴

شکل ۱.۱۸ منحنی نرخ کلید امن به عنوان تابعی از فاصله فیبر را برای پروتکل BB84-Decoy با اثبات Lim2014 نشان می‌دهد. این منحنی به عنوان مرجع اصلی گزارش عمل می‌کند و همه مقایسه‌های بعدی نسبت به آن تفسیر می‌شوند. در حالت بدون نویز، نرخ کلید در فواصل کوتاه به مقدار نظری نزدیک می‌شود و با افزایش فاصله به دلیل افت نمایی فیبر ($\eta \propto 10^{-\alpha L/10}$) کاهش می‌یابد. در حالت با نویز واقعی، افت اضافی ناشی از شمارش تاریک (dark count) و عدم انطباق پایه‌ها (misalignment) باعث می‌شود نرخ کلید در فواصل طولانی‌تر سریع‌تر به صفر برسد.



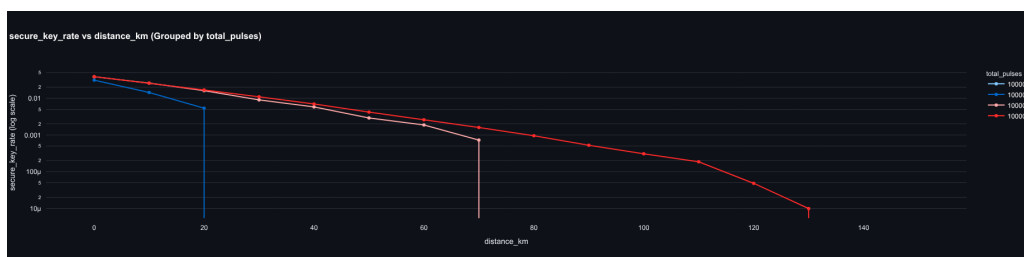
شکل ۱.۱۸: نرخ کلید امن به ازای پالس در برابر فاصله فیبر برای پروتکل BB84-Decoy با اثبات Lim2014. دو منحنی حالت بدون نویز و با نویز واقعی آشکارساز مقایسه شده‌اند.

نکته کلیدی در این شکل، شکستگی منحنی در فاصله‌ای است که در آن نرخ کلید به صفر می‌رسد؛ این نقطه نشان‌دهنده حداکثر برد امن پروتکل در شرایط داده‌شده است. مقادیر دقیق این برد برای ترکیب‌های مختلف پارامتر در فصل‌های بعدی مقایسه می‌شود. علاوه بر این، ستون status در CSV خروجی، نقاطی که به وضعیت ZERO_KEY ختم شده‌اند را علامت‌گذاری می‌کند که برای تعیین برد امن معتبر هستند.

۲.۱۸ مقیاس کلید محدود (Finite-Key Scaling)

شکل ۲.۱۸ اثر اندازه کلید بر نرخ کلید امن را در پنج اندازه پالس از 10^4 تا 10^7 نشان می‌دهد. در اندازه‌های کوچک، جریمه کلید محدود (finite-size penalty) نرخ امن را به طور قابل توجهی کاهش می‌دهد؛ زیرا تخمین پارامتر با تعداد محدودی نمونه انجام می‌شود و بازه اطمینان QBER گسترده‌تر است. با افزایش تعداد پالس به 10^7 ، نرخ کلید به مقدار مجانبی نزدیک می‌شود.

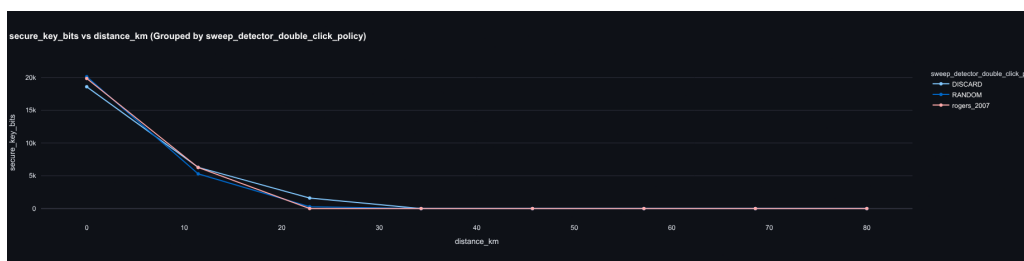
این نتیجه اهمیت انتخاب اندازه بلوک مناسب در پیاده‌سازی عملی QKD را تأیید می‌کند؛ برای کاربردهای بلندبرد، اندازه بلوک حداقل 10^6 پالس ضروری است تا جریمه finite-size چشم‌پوشی‌پذیر شود. در مقابل، برای کاربردهای کوتاه‌برد با نرخ بالای رویداد، حتی 10^5 پالس ممکن است کافی باشد.



شکل ۳.۱۸: اثر تعداد پالس بر نرخ کلید امن. با افزایش N از 10^4 به 10^7 ، جریمه کلید محدود کاهش می‌یابد و نرخ به مقدار مجانبی همگرا می‌شود.

۳.۱۸ اعتبارسنجی Rogers ۲۰۰۷

شکل ۳.۱۸ مقایسه نرخ کلید مونت‌کارلو (Monte Carlo) به‌دست آمده از شبیه‌سازی کامل را با فرمول تحلیلی Rogers et al. 2007 (رابطه ۱۵) برای نرخ بیت غربال‌شده (sifted bit) نشان می‌دهد. این مقایسه در ۵۱ نقطه متراکم از ۰ تا ۱۰۰ کیلومتر انجام شده تا همخوانی شبیه‌سازی با تئوری به‌خوبی بررسی شود. ستون `analytic_rogers_sbr` در فایل CSV به‌طور خودکار برای هر نقطه محاسبه می‌شود و این امکان را فراهم می‌کند که نمودار دقیقاً بازتولید اشکال ۳ تا ۵ مقاله Rogers باشد.



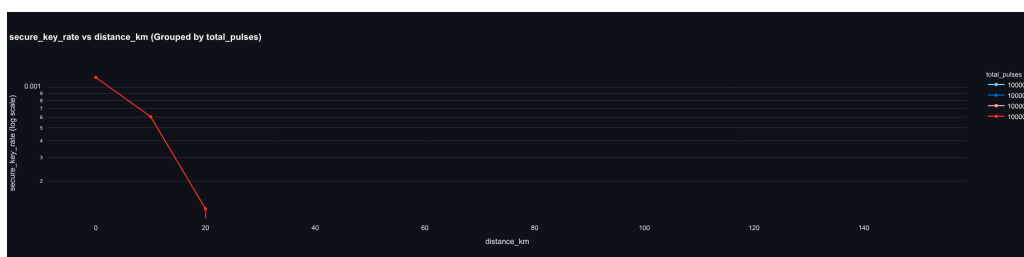
شکل ۳.۱۸: مقایسه نرخ sifted bit تحلیلی Rogers 2007 با نرخ کلید امن مونت‌کارلو. همخوانی دو منحنی، اعتبارسنجی صحت پیاده‌سازی لایه فیزیکی و آشکارسازی را تأیید می‌کند.

فصل ۱۹

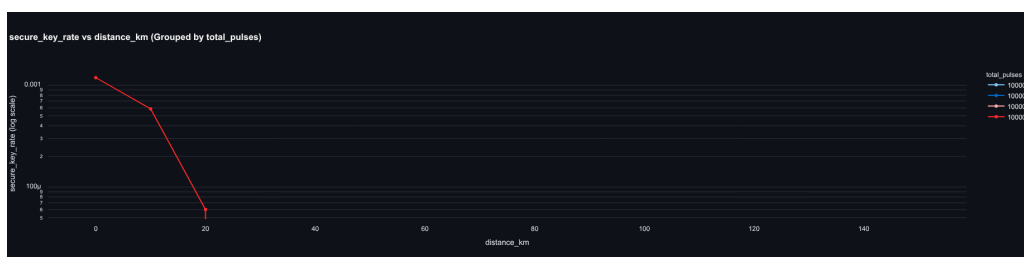
سناریوهای DWDM و بهینه‌سازی پالس

۱.۱۹ تأثیر هم‌زیستی با کانال کلاسیک (DWDM)

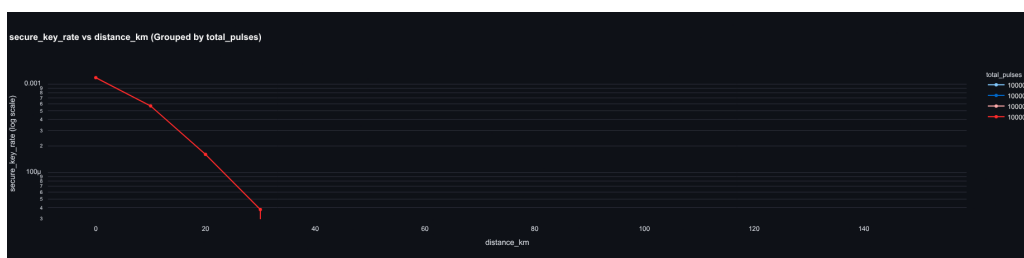
شکل ۱۹؟ سه نمودار کنار هم را برای توان‌های گیرنده مختلف -20 dBm، -34 dBm و -40 dBm نشان می‌دهد و اثر نویز را بر نرخ کلید امن را روشن می‌سازد. در توان بالاتر (-20 dBm)، نویز را می‌توان به‌طور چشمگیری افزایش می‌دهد و حداکثر برد امن را به‌میزان قابل توجهی کاهش می‌دهد. در توان پایین‌تر (-40 dBm)، رفتار به حالت فیبر اختصاصی نزدیک می‌شود و افت فیلتر AWG (۳ دسی‌بل) تنها منبع انحراف باقی می‌ماند.



شکل ۱.۱۹: توان -34 dBm (پیش‌فرض $\lim_{2014}$)



شکل ۲.۱۹: توان -20 dBm (نویز شدید)

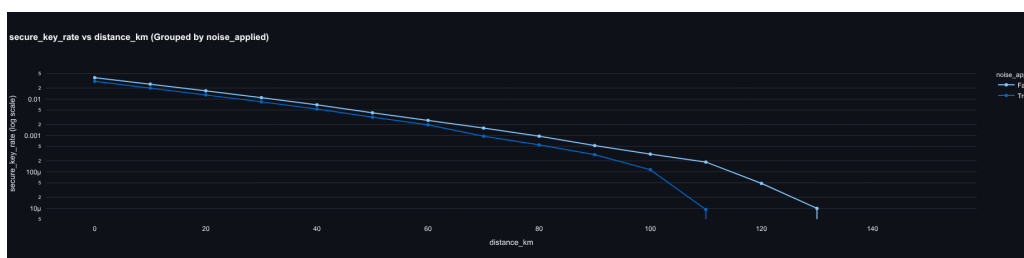


شکل ۳.۱۹: توان -40 dBm (نویز کم)

این نتایج با بخش IV مقاله Lim 2014 همخوانی دارد و نشان می‌دهد که طراحی سیستم DWDM برای QKD نیازمند موازنه دقیق بین توان کانال کلاسیک (برای ارتباط با نرخ بالا) و نویز رامن تحمیل‌شده بر کانال کوانتومی است. مقدار پیش‌فرض -34 dBm نقطه تعادل پیشنهادی مقاله است که در این شبیه‌سازی بازتولید شده است.

۲.۱۹ بهینه‌سازی خودکار پارامتر پالس

شکل ۴.۱۹ نتیجه بهینه‌سازی SLSQP برای پنج پارامتر پالس $(\mu, \nu, p_s, p_d, p_v)$ به‌ازای هر فاصله را نشان می‌دهد. بهینه‌ساز از نرخ GLLP مجانبی با جریمه کلید محدود به‌عنوان تابع هدف استفاده می‌کند و شدت سیگنال μ را در فواصل کوتاه بالا و در فواصل طولانی پایین نگه می‌دارد تا تعادل بین بازدهی آشکارسازی و افزایش اطلاعات بهینه شود.



شکل ۴.۱۹: نرخ کلید امن با بهینه‌سازی خودکار پارامتر پالس در برابر خط پایه با پارامترهای ثابت. بهینه‌ساز SLSQP پنج پارامتر $(\mu, \nu, p_s, p_d, p_v)$ را به‌ازای هر فاصله تنظیم می‌کند.

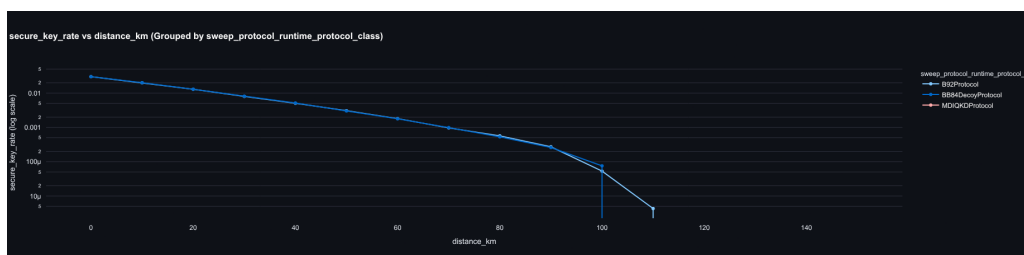
مقایسه با شکل ۱.۱۸ نشان می‌دهد که بهینه‌سازی خودکار، نرخ کلید را به‌ویژه در فواصل میانی (۲۰ تا ۸۰ کیلومتر) بهبود می‌بخشد. در فواصل بسیار کوتاه، پارامترهای ثابت پیش‌فرض از قبل نزدیک به بهینه هستند و سود بهینه‌سازی ناچیز است. در فواصل بسیار طولانی، بهینه‌سازی می‌تواند برد امن را چند کیلومتر افزایش دهد اما نرخ نهایی همچنان به صفر میل می‌کند.

فصل ۲۰

مقایسه‌های سیستماتیک

۱.۲۰ مقایسه پروتکل‌ها

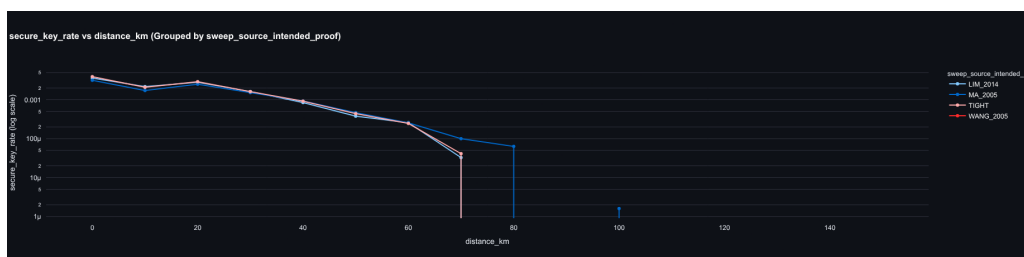
شکل ۱.۲۰ چهار پروتکل BB84-Decoy، B92، MDI-QKD و Redundant Transmission را در شرایط یکسان مقایسه می‌کند. پروتکل BB84-Decoy در فواصل کوتاه تا متوسط بهترین عملکرد را دارد؛ MDI-QKD به دلیل نیاز به آشکارسازی وسط مسیر، نرخ پایین‌تری دارد اما در برابر حملات آشکارساز مقاوم است و در فواصل طولانی نسبتاً پایدارتر عمل می‌کند. B92 ساده‌ترین پروتکل است اما نرخ پایین آن، کاربردش را در سناریوهای عملی محدود می‌سازد.



شکل ۱.۲۰: مقایسه نرخ کلید امن چهار پروتکل BB84-Decoy، B92، MDI-QKD و Redundant Transmission در شرایط فیبر یکسان.

۲.۲۰ مقایسه اثبات‌های امنیتی

شکل ۲.۲۰ چهار اثبات امنیت Lim2014، Ma2005، Wang2005 و Tight را روی همان پروتکل BB84-Decoy مقایسه می‌کند. اثبات Lim2014 پیش‌فرض چارچوب است و تعادل خوبی بین سخت‌گیری و کارایی برقرار می‌کند. اثبات Tight نرخ بالاتری تولید می‌کند اما فرضیات قوی‌تری دارد. اثبات‌های Ma2005 و Wang2005 برای سناریوهای خاص (مثل منابع نوری غیرایده‌آل) طراحی شده‌اند و در حالت استاندارد ممکن است نرخ پایین‌تری ارائه دهند.



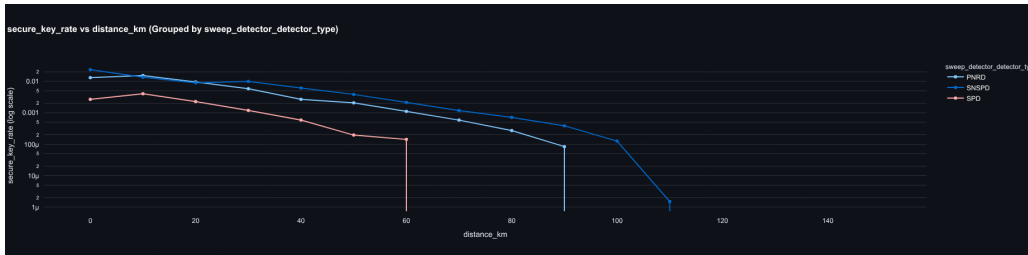
شکل ۲.۲۰: مقایسه نرخ کلید امن بر اساس چهار اثبات امنیت مختلف برای پروتکل BB84-Decoy.

تفاوت بین این چهار منحنی، حاشیه امنیت (security margin) هر اثبات را کمی می‌کند؛ هرچه اثبات

سخت‌گیرانه‌تر باشد، حاشیه بزرگ‌تر و نرخ نهایی پایین‌تر است. انتخاب اثبات در عمل به مدل تهدید و میزان اطمینان به منبع نوری و آشکارساز بستگی دارد.

۳.۲۰ مقایسه آشکارسازها

شکل ۳.۲۰ سه نوع آشکارساز SPD، SNSPD و PNRD را مقایسه می‌کند. آشکارساز SNSPD به دلیل نرخ شمارش تاریک بسیار پایین ($\sim 10^{-8}$) و راندمان بالا (~ 0.5)، بهترین عملکرد را در فواصل طولانی دارد. آشکارساز SPD راندمان پایین‌تری دارد اما هزینه و پیچیدگی کمتری داشته و برای فواصل کوتاه مناسب است. آشکارساز PNRD (آشکارساز شمارش فوتون متمایز) در سناریوهایی که نیاز به تشخیص دقیق تعداد فوتون‌ها است، کاربرد دارد.

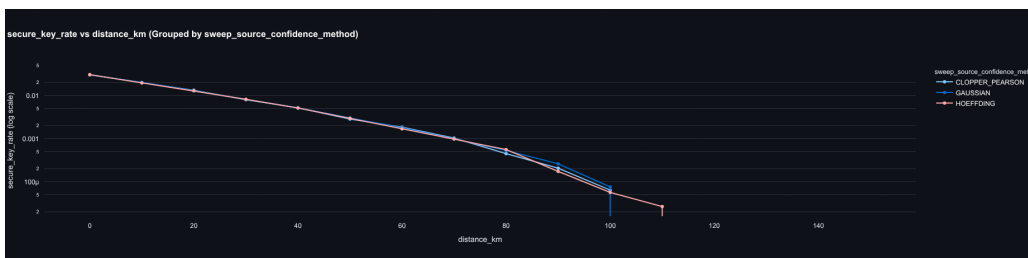


شکل ۳.۲۰: مقایسه نرخ کلید امن با سه نوع آشکارساز SPD، SNSPD و PNRD.

این مقایسه نشان می‌دهد که ارتقای آشکارساز از SPD به SNSPD می‌تواند برد امن را تا دو برابر افزایش دهد، اما هزینه و نیاز به سرمایه‌ش کرایونیک ملاحظات عملی مهمی هستند که باید در طراحی سیستم لحاظ شوند.

۴.۲۰ مقایسه روش‌های بازه اطمینان

شکل ۴.۲۰ سه روش Gaussian، Clopper-Pearson و Hoeffding را برای محاسبه بازه اطمینان QBER مقایسه می‌کند. روش Clopper-Pearson سخت‌گیرانه‌ترین (محافظه‌کارانه‌ترین) روش است و بازه‌ای گسترده‌تر تولید می‌کند که منجر به کاهش نرخ کلید نهایی می‌شود. Gaussian روش تقریبی است که برای نمونه‌های بزرگ بهینه است و نرخ بالاتری می‌دهد. Hoeffding حد متوسطی ارائه می‌کند که بدون فرض توزیع خاص، همواره معتبر است.



شکل ۴.۲۰: تأثیر روش محاسبه بازه اطمینان بر نرخ کلید امن. Clopper-Pearson محافظه‌کارانه‌ترین و Gaussian کارآمدترین روش است.

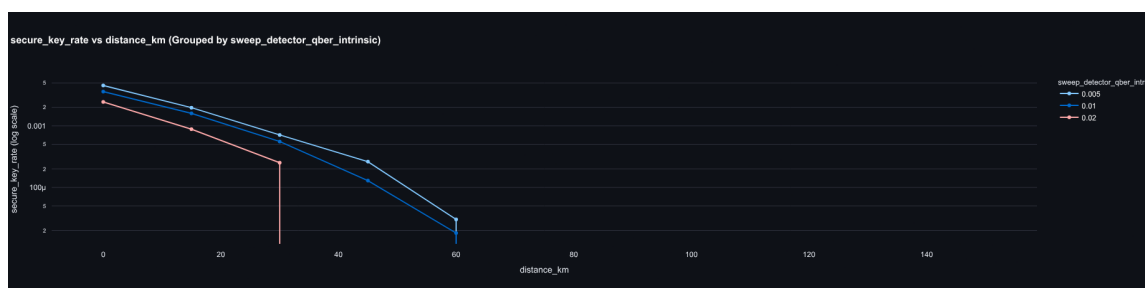
انتخاب روش به تعداد پالس و سطح اطمینان مورد نظر بستگی دارد؛ برای $N < 10^5$ ، روش Clopper-Pearson به دلیل دقت بهتر در نمونه‌های کوچک توصیه می‌شود و برای $N \geq 10^6$ ، تقریب Gaussian کافی است و نرخ کارآمدتری به دست می‌دهد.

فصل ۲۱

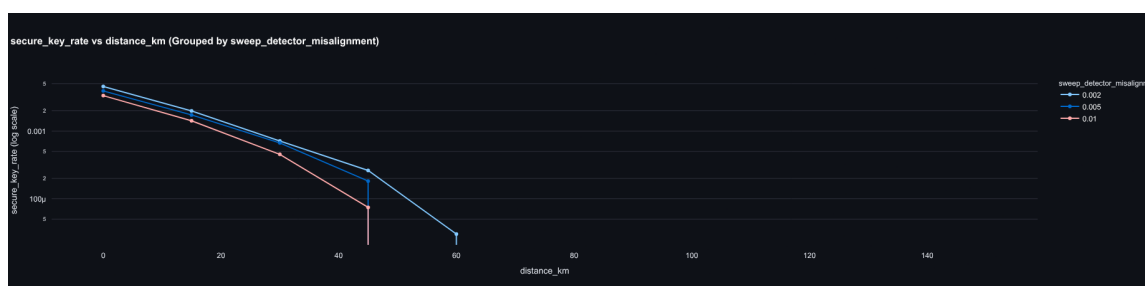
تحلیل حساسیت

۱.۲۱ حساسیت به QBER ذاتی و Misalignment

شکل‌های ۱.۲۱ و ۲.۲۱ تأثیر هم‌زمان QBER ذاتی آشکارساز و جابه‌جایی زاویه‌ای پایه‌ها (misalignment) بر نرخ کلید امن را نشان می‌دهند. این دو پارامتر مهم‌ترین منابع خطا در سیستم‌های QKD عملی هستند و افزایش هر کدام به‌طور مستقیم QBER کلی را بالا برده و نرخ کلید امن را کاهش می‌دهد. در مقادیر بالا (مثلاً $qber_intrinsic=0.02$ و $misalignment=0.01$)، حتی در فواصل کوتاه نیز نرخ کلید به‌طور قابل توجهی افت می‌کند.



شکل ۱.۲۱: تحلیل حساسیت نرخ کلید امن به QBER ذاتی آشکارساز.



شکل ۲.۲۱: تحلیل حساسیت نرخ کلید امن به عدم انطباق پایه‌ها (misalignment).

این تحلیل حساسیت دو پیامد عملی دارد: نخست، اهمیت کالیبراسیون دقیق پایه‌ها در زمان نصب سیستم QKD؛ چرا که $misalignment$ حتی در حد چند درجه می‌تواند نرخ کلید را تا ۵۰ درصد کاهش دهد. دوم، انتخاب آشکارساز با QBER ذاتی پایین (مانند SNSPD در مقایسه با SPD) تأثیر مستقیمی بر عملکرد نهایی سیستم دارد.